

SPEKTRUM LEHRBUCH

Carsten Vogt

Betriebssysteme



Spektrum
LEHRBUCH UND CO. KG

Carsten Vogt

Betriebssysteme

Spektrum Akademischer Verlag Heidelberg · Berlin

Autor:
Prof. Dr. Carsten Vogt
Fachhochschule Köln
e-mail: carsten.vogt@fh-koeln.de

Die Deutsche Bibliothek – CIP-Einheitsaufnahme

Vogt, Carsten:
Betriebssysteme / Carsten Vogt. – Heidelberg : Spektrum, Akad. Verl., 2001
ISBN 3-8274-1117-3

1. Auflage 2001
© 2001 Spektrum Akademischer Verlag GmbH Heidelberg · Berlin

Der Verlag und der Autor haben alle Sorgfalt walten lassen, um vollständige und akkurate Informationen in diesem Buch zu publizieren. Der Verlag übernimmt weder Garantie noch die juristische Verantwortung oder irgendeine Haftung für die Nutzung dieser Informationen, für deren Wirtschaftlichkeit oder fehlerfreie Funktion für einen bestimmten Zweck. Der Verlag übernimmt keine Gewähr dafür, dass die beschriebenen Verfahren, Programme usw. frei von Schutzrechten Dritter sind. Die Wiedergabe von Gebrauchsnamen, Handelsnamen, Warenbezeichnungen usw. in diesem Buch berechtigt auch ohne besondere Kennzeichnung nicht zu der Annahme, dass solche Namen im Sinne der Warenzeichen- und Markenschutz-Gesetzgebung als frei zu betrachten wären und daher von jedermann benutzt werden dürften.

Alle Rechte, insbesondere die der Übersetzung in fremde Sprachen, sind vorbehalten. Kein Teil des Buches darf ohne schriftliche Genehmigung des Verlages fotokopiert oder in irgendeiner anderen Form reproduziert oder in eine von Maschinen verwendbare Form übertragen oder übersetzt werden.

Lektorat: Dr. Andreas Rüdinger/Bianca Alton
Umschlaggestaltung: Kurt Bitsch, Birkenau
Druck und Verarbeitung: Franz Spiegel Buch GmbH, Ulm

Vorwort

In den Informatik-Abteilungen großer Buchhandlungen findet man unter der Rubrik „Betriebssysteme“ zwei Arten von Büchern. Die erste Sorte besetzt meist ein ganzes Regal oder sogar mehrere: Das sind die Ratgeber, die die Benutzeroberflächen sowie die Systemverwaltung bestimmter Betriebssysteme behandeln. Sie sind primär auf Anwender ausgerichtet, die Hilfe bei der Installation und Benutzung ihrer Computer benötigen. Die zweite Sorte findet meist auf einem oder zwei Regalbrettern Platz: Das sind die Bücher, die sich schwerpunktmäßig mit Aspekten unterhalb der Benutzeroberfläche befassen – nämlich mit der internen Realisierung eines Betriebssystems und seiner Programmierschnittstelle. Sie zielen auf eine Leserschaft, die ein tiefergehendes Verständnis der Thematik bekommen möchte. Hierzu gehören beispielsweise Studierende der Informatik und benachbarter Fächer, Programmierer oder ganz einfach Computerbesitzer, die am Innenleben ihres Rechners interessiert sind.

Dieses Lehrbuch gehört zur zweiten Kategorie: Es soll eine (hoffentlich) leicht lesbare, aber dennoch präzise und hinreichend detaillierte Einführung in das Gebiet der Betriebssysteme geben. Das Buch ist aus Lehrveranstaltungen (Vorlesungen, Übungen und Laborpraktika) an der Fachhochschule Köln hervorgegangen. Wie diese Veranstaltungen versucht es, eine Balance zu finden zwischen den Ansprüchen einer wissenschaftlichen Ausbildung und den handfest-praktischen Interessen der Studierenden. Es stellt dazu jeweils zunächst die wichtigsten Betriebssystemkonzepte allgemein – d.h. losgelöst von einem konkreten Betriebssystem – dar. Anschließend werden die Ansätze an realen Systemen veranschaulicht. Dabei werden sowohl die Implementation der Mechanismen als auch ihre Benutzung an der Programmierschnittstelle behandelt, und es wird ebenfalls die Benutzerschnittstelle kurz angesprochen.

Die Praxisbeispiele basieren vornehmlich auf UNIX und Linux: Studierende und andere Interessierte haben häufig Linux auf ihren privaten PCs installiert, möchten daher gern Näheres über ihr Betriebssystem erfahren und in seine Systemprogrammierung einsteigen. Von Fall zu Fall werden aber auch andere Systeme berücksichtigt, insbesondere die Betriebssysteme Windows NT und Windows 2000 sowie Dienste für verteilte Systeme wie z.B. CORBA.

Am Anfang des Buchs steht eine Einführung in die Aufgaben, grundlegenden Konzepte und Strukturen von Betriebssystemen. Es wird eine Übersicht über eine Reihe von Beispielbetriebssystemen sowie ein kurzer Überblick über den Aufbau von UNIX gegeben. Anschließend wird, ausgehend vom Prozessbegriff, die Problematik der nebenläufigen Programmierung behandelt, wobei insbesondere auf die Synchronisation von nebenläufigen Programmen und die Kommunikation zwischen ihnen eingegangen wird. Es folgt eine Diskussion der Verwaltung des Speichers – erstens zur Aufnahme der Daten der laufenden Aktivitäten, zweitens zur Realisierung des Dateisystems. Der

Aspekt der Systemsicherheit wird im darauf folgenden Kapitel besprochen. Die letzten beiden Kapitel befassen sich mit verteilten Systemen, wozu zunächst eine kompakte Einführung in die Grundlagen der Datenkommunikation gegeben und anschließend auf die wichtigsten Techniken verteilter Systemsoftware eingegangen wird.

Übungsaufgaben zu den einzelnen Kapiteln sollen den Stoff weiter vertiefen und dazu anregen, einige Dinge auch einmal praktisch auszuprobieren. Nähere Informationen hierzu bietet ein Anhang, der eine thematisch geordnete Übersicht über die wichtigsten Funktionen der UNIX/Linux-C-Schnittstelle und eine Sammlung entsprechender Programmieraufgaben enthält. Ein kommentiertes Literaturverzeichnis mit Hinweisen auf weiterführende Literatur schließt das Buch ab.

Zum Verständnis des Texts werden Grundkenntnisse eines Betriebssystems aus Benutzersicht, Grundlagenwissen über die Programmiersprache C sowie Basiskenntnisse der Hardwarearchitektur von Computern vorausgesetzt.

Abschließend noch eine stilistische Anmerkung: Trotz vielfacher Versuche bietet die deutsche Sprache leider immer noch keine elegante Möglichkeit, die weibliche und männliche Form eines Worts durch einen gemeinsamen Oberbegriff abzudecken. Aus Gründen der Lesbarkeit wird daher in diesem Buch jeweils nur die männliche Form verwendet. Selbstverständlich sind aber auch immer die Benutzerinnen, Programmiererinnen usw. gemeint.

Über Bemerkungen zum Buch, Fehlermeldungen und Verbesserungsvorschläge würde ich mich freuen – bitte per Mail an carsten.vogt@fh-koeln.de!

Inhalt

1	Einführung	1
1.1	Grundlegende Aufgaben und Konzepte	1
1.1.1	Betriebssysteme in Rechensystemen	1
1.1.2	Betriebssystem-Konzepte	3
1.1.3	Betriebsmittel	5
1.1.4	Schnittstellen.....	6
1.2	Betriebsarten.....	6
1.2.1	Einbenutzerbetrieb	8
1.2.2	Stapelverarbeitung	8
1.2.3	Mehrprogrammbetrieb	10
1.2.4	Dialogbetrieb.....	11
1.3	Strukturen	12
1.3.1	Aufbau der Software.....	12
1.3.2	Der Kern eines Betriebssystems	18
1.4	Betriebssystembeispiele	20
1.4.1	Betriebssysteme für Großrechner	20
1.4.2	Betriebssysteme für Workstations und Minirechner.....	21
1.4.3	Betriebssysteme für PCs	22
1.4.4	Betriebssysteme für mobile und eingebettete Systeme	23
1.4.5	Das Beispiel-Betriebssystem dieses Buchs.....	24
1.5	Übungsaufgaben	25
2	Das UNIX-Betriebssystem	29
2.1	Geschichte	29
2.2	Eigenschaften und Struktur	33
2.2.1	Allgemeine Eigenschaften	33
2.2.2	Die Schalenstruktur.....	34
2.2.3	Systemschnittstelle und Bibliotheksfunktionen.....	35
2.3	Benutzersicht	37
2.3.1	Dateisystem und E/A-Geräte	38
2.3.2	Shell und Benutzerkommandos	40
2.3.3	Dienstprogramme.....	43
2.4	Übungsaufgaben	45
3	Grundlagen des Prozesskonzepts	47
3.1	Grundidee und Implementierungsaspekte	47
3.1.1	Der Prozessbegriff	47
3.1.2	Threading	50

3.1.3	Implementierungsaspekte	52
3.1.4	Prozesse und Threads in UNIX / Linux.....	55
3.2	Prozesswechsel	63
3.2.1	Dispatching	63
3.2.2	Scheduling.....	64
3.2.3	Prozesswechsel in UNIX / Linux.....	69
3.3	Traps und Interrupts	73
3.3.1	Grundkonzepte	73
3.3.2	Implementierungsaspekte	74
3.3.3	Schachtelung von Interruptbehandlungen.....	75
3.4	Anforderungen der Praxis	77
3.4.1	Realzeit	77
3.4.2	Fehlertoleranz	81
3.5	Übungsaufgaben	81
4	Prozesssynchronisation und -kommunikation	89
4.1	Prozesssynchronisation	89
4.1.1	Problemstellung	89
4.1.2	Einfache Synchronisationsmechanismen	91
4.1.3	Semaphore.....	94
4.1.4	Weitere Techniken zur Synchronisation	103
4.1.5	Deadlocks.....	105
4.2	Prozesskommunikation	109
4.2.1	Shared Memory.....	110
4.2.2	Nachrichtenbasierte Systeme: allgemeine Prinzipien.....	113
4.2.3	Pipes.....	117
4.2.4	Message Queues.....	120
4.2.5	Sockets	122
4.3	Übungsaufgaben	127
5	Speicherhierarchie.....	131
5.1	Komponenten der Speicherhierarchie	131
5.1.1	Hauptspeicher	132
5.1.2	Cache.....	136
5.1.3	Plattenspeicher	139
5.2	Swapping	144
5.2.1	Allgemeines Schema.....	144
5.2.2	Swapping in UNIX System V	144
5.3	Virtueller Speicher	148
5.3.1	Grundlegende Überlegungen	148
5.3.2	Segmentorientierter Speicher	153
5.3.3	Seitenorientierter Speicher (Paging)	156
5.3.4	Pagingstrategien.....	161
5.3.5	Speicherorganisation im praktischen Einsatz	164
5.4	Übungsaufgaben	166

6	Dateisystem und Ein-/Ausgabe	171
6.1	Dateisysteme	171
6.1.1	Grundlegende Begriffe.....	171
6.1.2	Logische Strukturen	172
6.1.3	Realisierung von Verzeichnissen	176
6.1.4	Plattenorganisation.....	183
6.1.5	Fehlertoleranz	188
6.2	Ein-/Ausgabe	190
6.2.1	E/A-Hardware	190
6.2.2	E/A-Software	193
6.2.3	E/A-Konzept in UNIX / Linux	194
6.3	Übungsaufgaben	196
7	Sicherheit	199
7.1	Problematik	199
7.2	Externe Sicherheit	202
7.2.1	Grundlegende Techniken	202
7.2.2	Wissensbasierte Authentifizierung	204
7.2.3	Biometrische Authentifizierung.....	206
7.3	Interne Sicherheit	209
7.3.1	Schutzringe	210
7.3.2	Zugriffskontrollen	211
7.4	Weitere Gebiete.....	216
7.4.1	Kryptologie	216
7.4.2	Viren und Würmer	220
7.5	Übungsaufgaben	222
8	Verteilte Systeme: Grundlagen	225
8.1	Eng und lose gekoppelte Systeme	225
8.1.1	Arten der Nebenläufigkeit.....	226
8.1.2	Betriebssystemklassen	229
8.1.3	Ziele, Vorteile und Probleme verteilter Systeme	231
8.2	Grundlagen der Datenkommunikation	233
8.2.1	Nachrichten und Protokolle	234
8.2.2	ISO/OSI-Referenzmodell für geschichtete Protokolle.....	238
8.2.3	Bitübertragungsschicht	241
8.2.4	Klassifizierung von Netzen.....	243
8.2.5	Medienzugangsschicht.....	243
8.2.6	Vermittlungs- und Transportschicht	245
8.3	Übungsaufgaben	249
9	Verteilte Systeme: Techniken	251
9.1	Dateisystem	251
9.1.1	Grundlegende Ansätze	251
9.1.2	Strukturen und Zugriffe	252

9.1.3	Caching und Replikation.....	257
9.1.4	Beispiele verteilter Datei- und Namensdienste	258
9.2	Kommunikation und Kooperation.....	262
9.2.1	Client-Server mit Request-Reply	262
9.2.2	Remote Procedure Call	264
9.2.3	Sun RPC.....	266
9.3	Prozessverwaltung.....	268
9.3.1	Lastverteilung und Fehlertoleranz	268
9.3.2	Synchronisation.....	269
9.3.3	Threading	271
9.4	Systembeispiele.....	272
9.4.1	Mach und OSF/1	272
9.4.2	OSF / DCE	273
9.4.3	OMG CORBA	274
9.4.4	X-Window.....	277
9.5	Übungsaufgaben.....	278
Anhang: UNIX-C-Schnittstelle		283
A.1	Einführende Bemerkungen.....	283
A.2	Dateiverwaltung	284
A.3	Prozessverwaltung.....	289
A.4	Semaphore.....	295
A.5	Shared Memory	300
A.6	Pipes	303
A.7	Message Queues.....	306
A.8	Sockets	311
A.9	Programmieraufgaben	317
Lösungen der Aufgaben		325
	zu Kapitel 1.....	325
	zu Kapitel 2.....	326
	zu Kapitel 3.....	328
	zu Kapitel 4.....	331
	zu Kapitel 5.....	337
	zu Kapitel 6.....	339
	zu Kapitel 7.....	341
	zu Kapitel 8.....	343
	zu Kapitel 9.....	345
	zum Anhang „UNIX-C-Schnittstelle“	346
Literatur.....		347
	Kommentierte Literaturempfehlungen	347
	Quellenverzeichnis	352
Index		355

1 Einführung

Thema dieses Buchs sind *Betriebssysteme* – also die grundlegende Software, ohne die die Benutzung eines Computers kaum möglich wäre. In diesem einführenden Kapitel soll zuerst der Begriff „Betriebssystem“ genauer spezifiziert werden. Es folgt eine Übersicht über die wichtigsten Konzepte und Strukturen von Betriebssystemen, und schließlich werden einige konkrete Betriebssysteme als Beispiele für verschiedene Einsatzgebiete dargestellt.

1.1 Grundlegende Aufgaben und Konzepte

Um zunächst einmal zu klären, was ein Betriebssystem überhaupt ist, führen wir hier versuchsweise zwei „Definitionen“ aus der Literatur an:

- „Ein Betriebssystem ist wie der elektrische Strom: Man sieht es nicht, aber man kommt nicht ohne es aus.“
(sinngemäßes Zitat aus einem alten MS-DOS-Handbuch)
- „Betriebssystem: Die Programme eines digitalen Rechensystems, die zusammen mit den Eigenschaften der Rechanlage die Basis der möglichen Betriebsarten des digitalen Rechensystems bilden und insbesondere die Abwicklung von Programmen steuern und überwachen.“
(DIN 44300)

Für eine Einführung in das Thema sind beide Zitate nicht sonderlich gut geeignet: Das erste ist, um es vorsichtig auszudrücken, nicht sehr präzise, das zweite ist nicht sehr eingängig. Es ist daher sinnvoller, sich schrittweise an den Begriff heranzuarbeiten.

1.1.1 Betriebssysteme in Rechensystemen

Offensichtlich kann man bei einem Rechensystem zwei „Sichten“ einnehmen, nämlich die *Hardwaresicht* und die *Anwendungssicht*. Aus Hardwaresicht besteht ein Rechensystem aus einer Menge kooperierender Hardwarekomponenten, insbesondere

- einem *Prozessor* (einer *CPU*), der Maschinenprogramme ausführt,

- einem *Hauptspeicher*, der eine begrenzte Menge von Daten kurzfristig speichert und dem Prozessor einen direkten, relativ raschen Zugriff erlaubt,
- einer *Festplatte*, einem *Diskettenlaufwerk* und einem *CD-ROM-Laufwerk*, die die längerfristige Speicherung größerer Datenmengen ermöglichen, aber nur relativ langsam zugreifbar sind,
- einer *Tastatur* und einer *Maus*, über die der Benutzer Daten eingeben kann,
- einem *Bildschirm* und einem *Drucker*, über die das System Resultate an den Benutzer ausgeben kann, und
- einer *Netzkarte*, über die das System an ein Kommunikationsnetz angeschlossen ist.

Aus Anwendungssicht stellt ein Rechensystem benutzerfreundliche Konzepte bereit, mit denen Daten und Informationen verarbeitet werden können. Dazu gehören z.B.

- ein klar strukturiertes *Dateisystem* mit entsprechenden Dienstprogrammen,
- eine *Programmierungsumgebung* zum Schreiben und zur Übersetzung von Programmen,
- einfach ansprechbare Dienste zur *Ein- und Ausgabe* von Daten sowie zum Zugriff auf Internet-Angebote,
- Funktionen zur *Systemverwaltung* und
- eventuell die *Multi-User-Fähigkeit*, d.h. die Unterstützung mehrerer Benutzer, die gleichzeitig am System arbeiten.

Zwischen den beiden Sichten besteht eine große Kluft, da ihre „Abstraktionsniveaus“ sehr unterschiedlich sind. Um diese Kluft zu überbrücken, benötigt das Rechensystem eine Softwarekomponente, die die Hardware für den Anwender benutzbar macht oder, etwas präziser ausgedrückt, die die abstrakte Anwendungssicht auf der Grundlage der realen Hardwarebestandteile realisiert.

Die Dienste, die diese Komponente anbieten und implementieren muss, sind sehr vielfältig. Ein Beispiel für einen solchen Dienst ist das Kopieren einer Datei: Gibt der Benutzer einen Kopierbefehl ein, so müssen leere Bereiche auf der Festplatte belegt, die Bytes der Datei dorthin kopiert und anschließend das Dateienverzeichnis entsprechend aktualisiert werden.

Die Softwarekomponente, die eine solch zentrale Rolle spielt, ist das *Betriebssystem* (engl. *operating system*). Es setzt also unmittelbar auf der Hardware auf, steuert sie und bietet nach oben die erwähnten benutzer- und programmiererfreundlichen Dienste an (siehe Abb. 1.1). Zur Realisierung der Dienste enthält das Betriebssystem interne Programme sowie Datenstrukturen, die die benötigten Informationen speichern. Das Betriebssystem deckt damit zwei große Aufgabenbereiche ab:

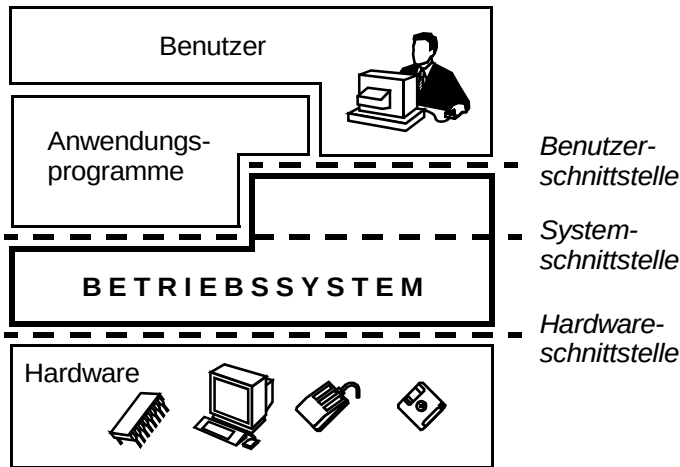


Abb. 1.1 Position des Betriebssystems

- Es verwaltet die *Rechnerbetriebsmittel*, also die Komponenten des Rechensystems (siehe Abschnitt 1.1.3).
- Es stellt komfortable *Schnittstellen* für Benutzer und Programmierer bereit (siehe Abschnitt 1.1.4). Die *Benutzerschnittstelle* ermöglicht dem Anwender, den Rechner zu steuern und seine Dienste aufzurufen. Die *Programmierschnittstelle* enthält Funktionsprototypen (= Funktions„köpfe“) zur Einbindung in Programme, so dass diese ebenfalls die Dienste des Betriebssystems nutzen können.

Die Benutzeroberfläche, wie sie dem normalen Anwender eines Rechensystems vertraut ist, offenbart also nur einen recht kleinen Teil des Betriebssystems. Sein Hauptteil liegt unter dieser Oberfläche verborgen; um ihn geht es in diesem Buch.

1.1.2 Betriebssystem-Konzepte

Obwohl eine Vielfalt verschiedener Betriebssysteme existiert, gibt es doch eine Reihe von allgemein verwendeten Konzepten. Die wichtigsten davon sind die folgenden:

- *Prozesse*: Das Prozesskonzept ist die Grundlage der Steuerung der Aktivitäten in einem System. Unter einem Prozess versteht man, grob gesagt, ein Programm, das gerade ausgeführt wird und dem dabei Daten und Hardwarekomponenten zur Verfügung stehen. So wird beispielsweise ein (übersetztes) C-Programm bei seiner Ausführung intern durch einen Prozess repräsentiert. Ein Multiprogramm-Betrieb wird realisiert, indem der Prozessor zwischen Prozessen hin- und hergeschaltet wird.
- *Speicherhierarchie*: Die Speicherhardware eines Computers umfasst zumindest einen Haupt- und einen Plattenspeicher, also zwei Komponenten mit stark unterschiedli-

chen Eigenschaften. Daten und Programmcode der Prozesse müssen während ihrer Ausführung im Hauptspeicher liegen, der aber im Allgemeinen zu klein ist, um alle Prozesse aufnehmen zu können. Das Betriebssystem muss also die Speicherinhalte so auf Haupt- und Plattenspeicher verteilen und zwischen diesen beiden Speichern hin- und hertransportieren, dass eine effiziente Prozessaufführung möglich ist.

- *Dateisystem und Ein-/Ausgabe:* „Langlebige“ Daten werden in Dateien auf Peripheriegeräten (z.B. Plattenspeichern) abgelegt. Zudem werden über Peripheriegeräte Daten ein- und ausgegeben. Eine Aufgabe des Betriebssystems ist, das Dateisystem mit den entsprechenden Verwaltungsoperationen zu realisieren und die Peripheriegeräte geeignet anzusteuern.
- *Sicherheit:* Die Durchsetzung von Sicherheitskonzepten ist ebenfalls ein Aufgabenbereich des Betriebssystems. Dazu gehören der Schutz sowohl des Gesamtsystems als auch einzelner Daten und Programme vor unberechtigter Benutzung oder Zerstörung. Techniken, die hier eingesetzt werden, sind u.a. Zugangskontrollen zum System (z.B. durch Passwörter), Zugriffskontrollen auf Dateien und Verschlüsselung von Daten.
- *Multiprozessorsysteme / Verteilte Systeme:* Die moderne Hardwaretechnik erlaubt die echt nebenläufige Ausführung mehrerer Aktivitäten. Hier gibt es erstens *Multiprozessorsysteme* mit mehreren Prozessoren in einem Computer, die über einen Bus und einen gemeinsamen Speicher kommunizieren. Zweitens gibt es *verteilte Systeme*, in denen mehrere auseinanderliegende Rechensysteme über ein Kommunikationsnetz gekoppelt sind. Die Verwaltungsaufgaben des Betriebssystems und auch die Programmierung sind hier wesentlich komplexer als in einem „lokalen“ Einprozessorsystem.

Die genannten Themenbereiche sind die Kernpunkte dieses Buchs. Wir werden jeweils in eigenen Kapiteln Probleme und Lösungsansätze allgemein diskutieren und dann konkrete Beispiele geben.

Daneben gibt es im Zusammenhang mit Betriebssystemen noch eine Reihe weiterer wichtiger Punkte. Dazu gehören unter anderem die folgenden:

- *Fehlertoleranz:* Ein Rechensystem sollte möglichst auch dann noch korrekt arbeiten, wenn ein Teil seiner Komponenten ausgefallen ist oder falsche Ergebnisse liefert. Im Idealfall ist dann zwar die Leistung reduziert, das System läuft aber ansonsten mit seinem vollen Funktionsumfang weiter – ein Verhalten, das mit *Graceful Degradation* bezeichnet wird. Fehlertoleranz wird durch *Redundanz* erreicht, d.h. durch Mehrfachführung von Hard- und Softwarekomponenten sowie Daten mit Fehlererkennungs- und -korrekturcodes.
- *Realzeitbetrieb:* Zahlreiche Anwendungsgebiete, wie z.B. Anlagensteuerungen, verlangen, dass die Ergebnisse eines Programms innerhalb einer vorgegebenen Zeitspanne vorliegen. Im Realzeitbetrieb muss das Betriebssystem den Prozessen die Betriebsmittel so zuteilen, dass diese Zeitanforderungen eingehalten werden. Wichtig ist hier insbesondere die Verwaltung des Prozessors, aber auch des Speichers.

- *Benutzeroberflächen*: Die Benutzeroberfläche dient zur Kommunikation zwischen Anwender und System. Das Betriebssystem muss erstens eine ansprechende Benutzerschnittstelle zu seiner eigenen Bedienung enthalten. Zweitens muss es Funktionen bereitstellen, mit denen aus Anwendungsprogrammen heraus auf diese Benutzerschnittstelle zugegriffen werden kann. So findet die Kommunikation sowohl mit dem Betriebssystem selbst als auch mit Anwendungsprogrammen über eine einheitliche Schnittstelle statt.

Die drei letztgenannten Punkte werden nicht in eigenen Kapiteln behandelt, aber es werden an geeigneten Stellen entsprechende Techniken erläutert. Auf das ebenfalls wichtige Gebiet der Systemverwaltung wird nicht eingegangen, da dies den Rahmen dieses Buchs sprengen würde. Stattdessen wird auf Spezialliteratur zu den einzelnen konkreten Betriebssystemen verwiesen.

In allen aufgeführten Teilgebieten muss das Betriebssystem seine beiden Aufgaben erfüllen – die Verwaltung der Betriebsmittel und die Bereitstellung von Schnittstellen. Diese beiden Punkte sollen nun etwas genauer betrachtet werden.

1.1.3 Betriebsmittel

Die *Betriebsmittel* eines Rechensystems sind die Komponenten, die das System für seine Aktivitäten benötigt. Unter diese weitgefaste Definition fallen die meisten Teile eines Computers, so dass das Betriebssystem eine größere Anzahl verschiedenartiger Betriebsmittel verwalten muss. Man kann sie nach verschiedenen Kriterien klassifizieren:

Erstens können *Hardware-* und *Softwarebetriebsmittel* unterschieden werden. Zu den Hardwarebetriebsmitteln gehören beispielsweise der Prozessor, der Speicher, die Ein-/Ausgabegeräte und die Netzkarte; Softwarebetriebsmittel sind Daten und Programme.

Zweitens differenziert man zwischen *einmal benutzbaren* und *mehrmals benutzbaren* Betriebsmitteln. Einmal benutzbare Betriebsmittel sind nach ihrer Verwendung verbraucht (z.B. Datenpakete, die aus dem Kommunikationsnetz geholt werden, oder Druckerpapier); mehrmals benutzbare Betriebsmittel können mehrfach hintereinander verwendet werden (z.B. Prozessor, Speicherzellen, schreibgeschützte Dateien).

Eine drittes Kriterium unterscheidet danach, ob ein Betriebsmittel *exklusiv benutzbar* oder *nicht-exklusiv benutzbar* ist, d.h. ob höchstens eine Aktivität oder ob mehrere Aktivitäten gleichzeitig darauf zugreifen können. Beispielsweise ist ein Drucker exklusiv benutzbar und eine Datei, auf die nur lesend zugegriffen wird, nicht-exklusiv benutzbar.

Schließlich kann viertens ein Betriebsmittel *entziehbar* oder *nicht-entziehbar* sein. Ein entziehbares Betriebsmittel kann während seiner Benutzung einer Aktivität weggenommen, zwischenzeitlich einer anderen Aktivität zugeteilt und später der ersten Aktivität zur Weiterarbeit zurückgegeben werden, ohne dass das Auswirkungen auf das Resultat hat. Bei einem nicht-entziehbaren Betriebsmittel ist das nicht so. Ein Drucker ist beispielsweise nicht-entziehbar, da es bei Entzug und Wiedertzuteilung vorkommen kann, dass Texte durcheinander ausgedruckt werden – wodurch die Freude der Benutzer an ihren Ausdrucken geschmälert wird. Der Prozessor ist, wie später gezeigt wird, entziehbar.

1.1.4 Schnittstellen

An seinen Schnittstellen stellt das Betriebssystem Funktionen und Dienste bereit, über die auf das Rechensystem zugegriffen werden kann. Man unterscheidet hier zwischen Schnittstellen für Benutzer (*Benutzerschnittstelle*, *Benutzeroberfläche*, *user interface*) und Schnittstellen für Programmierer oder Anwendungsprogramme (*Programmierschnittstelle*, *Systemschnittstelle*, *application programming interface API*).

Wie bereits gesagt, verbirgt das Betriebssystem damit die realen Hardwareverhältnisse (es „abstrahiert“ von ihnen) und spiegelt die Existenz einer *virtuellen Maschine* vor. Diese virtuelle Maschine ist (mehr oder weniger) komfortabel zu bedienen und erspart dem Benutzer oder Programmierer, sich unmittelbar mit den realen Bits und Bytes der Hardware herumschlagen zu müssen. Sie bietet darüber hinaus zahlreiche Konzepte und Dienste an, die durch die Hardware allein nicht realisiert werden. Beispiele hierfür sind der *Multi-User-Betrieb* (d.h. mehrere Benutzer können gleichzeitig auf einer Maschine mit real nur einem oder einigen wenigen Prozessoren arbeiten), ein *homogener Speicher* (d.h. für Benutzerprogramme spielt es keine Rolle, ob ihre Daten gerade im Hauptspeicher oder auf dem Plattenspeicher liegen), *Datenschutz* sowie eine gewisse *Robustheit gegen Fehler*.

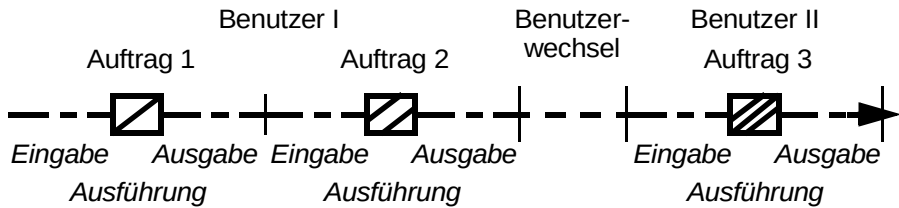
Zur Benutzeroberfläche gehört insbesondere eine *Kommandosprache*, über die der Benutzer mit dem System kommunizieren kann. Allgemein bekannt ist die zeichen- und zeilenorientierte Kommandosprache des klassischen MS-DOS, die auch von aktuellen Windows-Versionen unterstützt wird. Im folgenden Kapitel wird die entsprechende Kommandosprache von UNIX behandelt. Neben die klassischen zeichenorientierten Benutzerschnittstellen traten im Laufe der Zeit graphische, fensterorientierte Oberflächen. Aktuell im Kommen sind multimediale Schnittstellen, die neben Text und Graphik weitere Medien wie Ton und Bewegtbild unterstützen. Zudem wird mit Oberflächen experimentiert, die neben dem Sehen und Hören weitere Sinne ansprechen und so die Bandbreite der Kommunikation zwischen Benutzer und Maschine weiter vergrößern sollen. Das Stichwort heißt hier *Virtuelle Realität*.

Die Programmierschnittstelle, die näher an der Hardware liegt, bietet eine Sammlung von Funktionsprototypen (= Funktions„köpfen“, z.B. in der Sprache C), die in Programmen benutzt werden können. Da Anwendungsprogramme auf dieser Schnittstelle aufsetzen und verschiedene Betriebssysteme unterschiedliche Schnittstellen haben, kann eine Anwendung im Allgemeinen nur auf einem bestimmten System laufen. Auf typische Funktionen einer Programmierschnittstelle wird später genauer eingegangen.

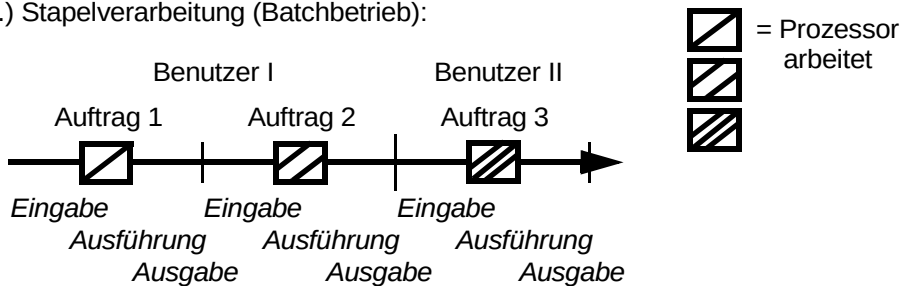
1.2 Betriebsarten

Betriebssysteme sind unterschiedlich leistungsfähig: Ihre Betriebsmittelverwaltung ist mehr oder weniger effizient, sie bieten viele oder nur wenige Dienste und Konzepte an, und ihre Oberfläche ist benutzerfreundlich oder auch weniger gut zu handhaben. Das

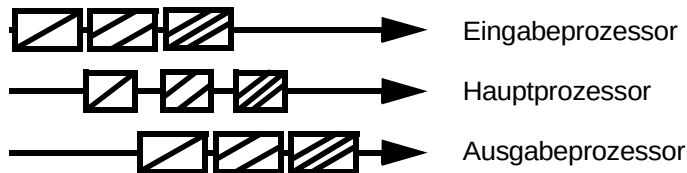
a.) Einbenutzerbetrieb mit Einzelauftragsbearbeitung (Single User Mode):



b.) Stapelverarbeitung (Batchbetrieb):



c.) Stapelverarbeitung mit Spooling:



d.) Mehrprogrammbetrieb (Multiprogramming):



Abb. 1.2 Zeitverhalten unter den verschiedenen Betriebsarten

liegt erstens an der fortschreitenden technischen Entwicklung, die tendenziell zu immer umfassenderen Systemen geführt hat. Zweitens sind die verschiedenen Hardwareplattformen unterschiedlich leistungsfähig, und drittens stellen unterschiedliche Anwendungsbereiche unterschiedliche Anforderungen an das Betriebssystem.

Ein wichtiges Kennzeichen für die Leistungsfähigkeit und auch den Anwendungsbereich eines Betriebssystems ist die *Betriebsart*. In einem Rechensystem, das im Allgemeinen mehrere Aufträge gleichzeitig zu bearbeiten hat, bestimmt die Betriebsart wesentlich den zeitlichen Ablauf der Ausführung dieser Aufträge. Die Spanne möglicher Abläufe reicht von „streng sequentiell“ (= hintereinander) bis „voll nebenläufig“.

1.2.1 Einbenutzerbetrieb

Die einfachste Betriebsart ist der *Einbenutzerbetrieb* (engl. *single user mode*) mit *Einzelauftragsbearbeitung*. Hier belegt ein einzelner Benutzer das gesamte Rechensystem für eine bestimmte Zeit und erteilt dabei hintereinander mehrere Aufträge, die streng sequentiell (d.h. einer nach dem anderen) abgearbeitet werden (siehe Abb. 1.2, Teil a). Ein neuer Auftrag wird erst eingegeben, wenn sein Vorgänger vollständig bearbeitet worden ist. Ist der Benutzer mit seiner Arbeit fertig, gibt er das System an den nächsten Benutzer ab. Es sind nur für einen Benutzer Ein- und Ausgabegeräte vorhanden.

Wie Abbildung 1.2, Teil a, zeigt, ist der Prozessor bei dieser Betriebsart sehr schlecht ausgelastet. Er bearbeitet nur während eines Bruchteils der Zeit Aufträge; in der restlichen Zeit, in der Ein- und Ausgaben sowie Benutzerwechsel stattfinden, liegt er brach.

Den Einbenutzerbetrieb gab es früher und gibt es noch heute. In den frühen fünfziger Jahren war dies die einzige Betriebsart, die technisch möglich war. Man hatte damals also einen doppelten Nachteil zu tragen: Die Hardware war teuer und langsam und wurde zudem noch schlecht ausnutzt. Heute findet man den Einbenutzerbetrieb in Personal Computern und Workstations, die einzelnen Benutzern zur Verfügung stehen. Da die entsprechende Hardware relativ billig ist, ist zumindest in diesem Bereich eine gute Auslastung nicht mehr so entscheidend wie früher. Zudem ist heute die Beschränkung auf die Einzelauftragsbearbeitung meist weggefallen: Es ist zwar nur ein Benutzer am System tätig, der aber mehrere Aufträge nebenläufig bearbeiten lassen kann.

1.2.2 Stapelverarbeitung

Um die Auslastung des Prozessors zu verbessern, ging man in den frühen sechziger Jahren zu einer Automatisierung der Auftragsein- und -ausgabe über. Die entsprechende Betriebsart wird als *Stapelverarbeitung* oder *Batchbetrieb* (engl. *batch processing*, *batch mode*) bezeichnet.

Einfache Stapelverarbeitung

In der Frühzeit der Stapelverarbeitung wurden Aufträge über Lochkarten eingegeben (siehe Abb. 1.3): Ein Auftrag (engl. *job*) war auf einer Folge von Lochkarten „abge-
locht“, d.h. durch ein Lochmuster codiert. Er wurde durch eine Reihe von Steuerkommandos an das Betriebssystem eingeleitet (formuliert in einer *job control language*), denen das eigentliche Programm und die zugehörigen Daten folgten. Mehrere Aufträge wurden zu einem Lochkartenstapel (engl. *batch*) zusammengefasst, der über einen Lochkartenleser eingelesen und dabei, mit Hilfe eines einfachen Eingaberechners, auf ein Magnetband übertragen wurde. Das Magnetband wurde dann in den Hauptrechner eingelesen, wo die Aufträge bearbeitet wurden. Die Ausgabe, meist auf einen Drucker, verlief auf dieselbe Weise über ein Ausgabeband und einen Ausgaberechner.

Auch hier ist die Auftragsbearbeitung streng sequentiell (siehe Abb. 1.2, Teil b): Ein Auftrag wird vom Eingabeband eingelesen und ausgeführt. Nach der Ausgabe der Er-

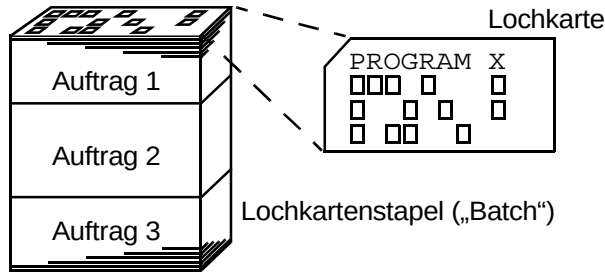


Abb. 1.3 Lochkartenstapel („Batch“)

gebnisse wird der nächste Auftrag eingelesen usw. Somit hat auch bei der (reinen) Stapelverarbeitung jeweils ein einzelner Auftrag die vollständige Kontrolle über die Maschine.

Die Vorteile gegenüber dem Einbenutzerbetrieb liegen in der Verkürzung der Ein-/Ausgabezeiten und im Wegfall der Benutzerwechselzeiten. Nachteilig ist, dass die Stapelverarbeitung nur für Anwendungen geeignet ist, die keine Interaktion mit dem Benutzer erfordern (interaktive Textverarbeitung ist damit z.B. nicht möglich), und dass Benutzer lange auf die Ergebnisse warten müssen. Zudem ist die Auslastung des Prozessors hier zwar besser, aber immer noch nicht gut: Müssen beispielsweise während einer Programmausführung Daten vom „Hintergrundspeicher“ (z.B. seinerzeit einer Magnettrommel) eingelesen werden, so liegt der Prozessor währenddessen brach. Es ist nämlich kein anderer Auftrag im System vorhanden, den er in der Zwischenzeit bearbeiten könnte.

Spooling

In den späten sechziger Jahren wurde die Ausnutzung des Prozessors weiter verbessert: Die Stapelverarbeitung wurde durch das *Spooling*konzept ergänzt, bei dem die Ein- und Ausgabe von Aufträgen nebenläufig zur Bearbeitung anderer Aufträge geschieht. Spoolingsysteme erfordern eine erweiterte Hardware (siehe Abb. 1.4). Neben dem Hauptprozessor, der Aufträge ausführt, gibt es besondere Ein-/Ausgabe-Prozessoren, die für das Einlesen der Aufträge in den Hauptspeicher und für die Ausgabe der Ergebnisse aus dem Hauptspeicher dienen. Diese Prozessoren sind *DMA-fähig* (*DMA* = *direct memory access*): Sie können Daten über den Bus direkt in den bzw. aus dem Hauptspeicher transportieren – ohne Beteiligung des Hauptprozessors. Somit kann bereits während der Bearbeitung eines Auftrags sein Nachfolger eingelesen werden; ebenso kann die Bearbeitung des Folgeauftrags schon beginnen, während die Ergebnisse seines Vorgängers noch ausgegeben werden. Diese zeitliche Überlappung steigert die Auslastung der Betriebsmittel (siehe Abb. 1.2, Teil c).

Die Prozessoren müssen ihre Buszugriffe koordinieren um sicherzustellen, dass jeweils nur einer von ihnen Daten über den Bus schickt. Dies geschieht durch hardware-gestützte *Bussperren*, mit denen Prozessoren den Bus exklusiv für sich belegen können. Dies führt natürlich zu einer (moderaten) Verzögerung der anderen Prozessoren, da ih-

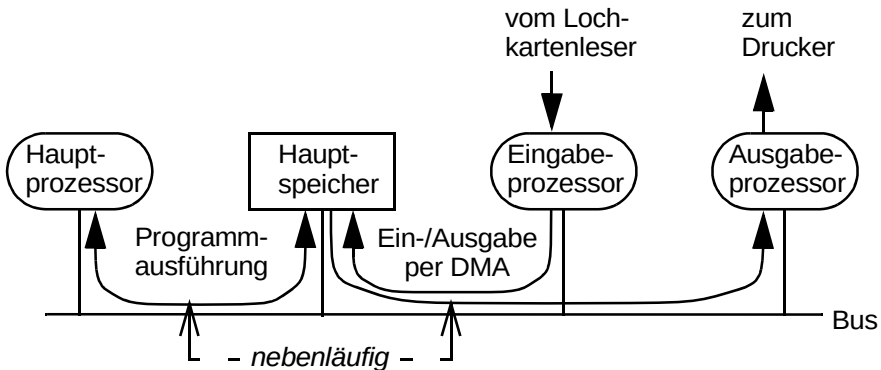


Abb. 1.4 Struktur eines klassischen Spooling-Systems

nen Buszyklen „gestohlen“ werden. In Großrechnern, die gesonderte Busse für Hauptprozessor und Ein-/Ausgabe-Prozessoren haben, tritt dieses Problem nicht auf.

Batchbetrieb und Spooling in modernen Rechensystemen

Stapelverarbeitung und Spooling finden sich auch in modernen Computern. Zeitaufwendige Aufträge, wie beispielsweise Simulationen oder die Auswertung großer Datenmengen, können als *Batchprozesse* „im Hintergrund“ ablaufen: Immer, wenn der Prozessor nicht für den Dialogbetrieb mit einem Benutzer am Bildschirm benötigt wird, bearbeitet er einen Batchprozess weiter. Hierdurch ergibt sich eine gute Prozessorauslastung.

Druckaufträge werden zumeist durch spezielle *Spoolingprozesse* bearbeitet: Benutzerprozesse erteilen die Aufträge nicht dem Drucker selbst, sondern einem Spoolingprozess, der sie dann nebenläufig zu den Benutzerprozessen bearbeitet.

1.2.3 Mehrprogrammbetrieb

Nachteilig am Stapelbetrieb ist, wie bereits diskutiert, die immer noch recht schlechte Ausnutzung der Betriebsmittel, da Aufträge streng sequentiell bearbeitet werden. Der Prozessor liegt zu oft brach – insbesondere bei Aufträgen, die häufig auf Ereignisse, wie z.B. Antworten von Ein-/Ausgabegeräten, warten müssen.

Eine wesentliche Verbesserung brachte hier der *Mehrprogrammbetrieb* (engl. *multi-programming*), der in den siebziger Jahren aufkam. Diese Betriebsart erlaubt eine flexiblere Prozessorzuteilung: Der Prozessor kann nicht mehr nur am Ende einer Auftragsbearbeitung zu einem anderen Auftrag wechseln, sondern zusätzlich auch „zwischen-durch“. Das bedeutet, dass die Ausführung eines Auftrags zugunsten einer anderen Tätigkeit unterbrochen und später wieder aufgenommen werden kann. Der Prozessor wird somit dynamisch zwischen Aktivitäten „hin- und hergeschaltet“. Eine Umschaltung findet typischerweise statt,

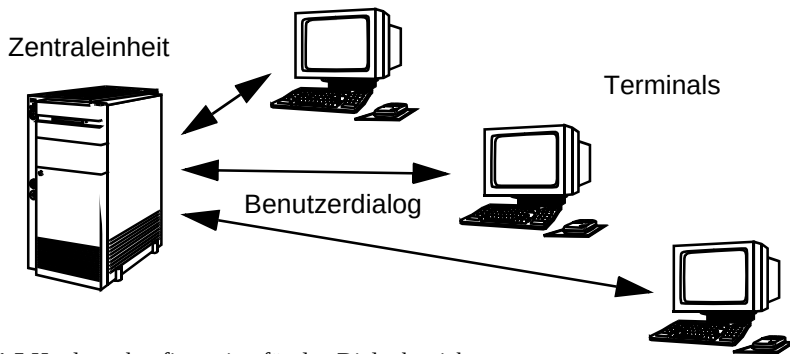


Abb. 1.5 Hardwarekonfiguration für den Dialogbetrieb

- wenn der gerade ausgeführte Auftrag auf ein Ereignis (z.B. die Antwort eines Ein-/Ausgabegeräts) warten muss, den Prozessor also momentan nicht benötigt,
- wenn ein dringenderer Auftrag rasch bearbeitet werden muss oder
- um mehrere gleichberechtigte Aufträge gleichmäßig („fair“) zu bearbeiten, um also lange Wartezeiten zu vermeiden.

Im Mehrprogrammbetrieb werden also mehrere Aufträge (*pseudo-nebenläufig* (engl. *concurrent*)) bearbeitet, wobei der Prozessor im schnellen Wechsel umgeschaltet wird. Abbildung 1.2, Teil d, zeigt das entsprechende Zeitverhalten.

Neben der besseren Ausnutzung der Betriebsmittel ermöglicht der Mehrprogrammbetrieb ein *Timesharing*: Mehrere Benutzer können gleichzeitig am System arbeiten, wobei jeder von ihnen die Illusion eines Computers hat, der nur für ihn arbeitet. Das setzt natürlich eine ausreichend schnelle Hardware voraus.

Der Mehrprogrammbetrieb wird schon seit langem standardmäßig auf Workstations und Großrechnern unterstützt, seit geraumer Zeit in verstärktem Maße auch auf PCs (z.B. durch die heutigen Windows-Betriebssysteme und Linux). Zusätzlich findet man aber auch, wie schon diskutiert, Stapelverarbeitungs- und Einzelbenutzer-Konzepte.

1.2.4 Dialogbetrieb

Der Mehrprogrammbetrieb allein räumt noch nicht den zweiten Nachteil der Stapelverarbeitung aus, dass nämlich während der Auftragsausführung keine Interaktion mit dem Benutzer stattfinden kann. Dies ist erst im *Dialogbetrieb* möglich, einer Betriebsart, die mit dem Mehrprogrammbetrieb kombiniert werden kann.

Abbildung 1.5 zeigt die benötigte Hardwarekonfiguration: Jeder Benutzer hat sein eigenes *Terminal*, über das er die Ausführung seiner Aufträge interaktiv steuern kann. Im Gegensatz zur Stapelverarbeitung ist hier also die Ein- und Ausgabe voll unter der Kontrolle des Benutzers; es findet während der Auftragsausführung ein *Dialog* zwischen ihm und dem Rechensystem statt.

In vielen der heutigen Rechensysteme treten Mehrprogramm- und Dialogbetrieb gemeinsam auf, was aber nicht zwingend notwendig ist: Das klassische MS-DOS beispielsweise unterstützte den Dialogbetrieb, aber keinen vollen Mehrprogrammbetrieb.

1.3 Strukturen

Charakteristisch ist für ein Betriebssystem auch die Struktur seiner Software. Abbildung 1.1 zeigte, dass die Betriebssystemsoftware den Bereich zwischen Hardware-schnittstelle und System- sowie Benutzerschnittstelle ausfüllt: Sie setzt „unten“ auf der realen Hardware auf und bietet „oben“ Funktionen und Dienste für Anwendungsprogramme und Benutzer.

Der Teil der Software zwischen Hardware und Systemschnittstelle wird *Kern* des Betriebssystems (engl. *kernel*) genannt. Bevor wir auf den Begriff des Kerns näher eingehen, diskutieren wir zunächst verschiedene Möglichkeiten, die Software eines Betriebssystems zu strukturieren.

1.3.1 Aufbau der Software

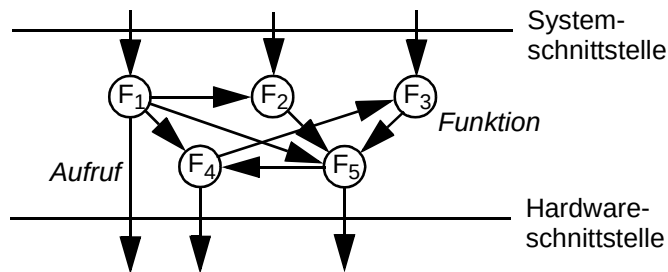
Monolithisches System

Im einfachsten Fall ist ein Betriebssystem *monolithisch* („aus einem Block“) aufgebaut. Im Extremfall bedeutet das, dass es überhaupt keine geordnete Struktur hat (siehe Abb. 1.6, Teil a): Das Betriebssystem besteht dann aus Programmstücken (*Funktionen*), die sich beliebig gegenseitig aufrufen und beliebig auf interne Daten zugreifen dürfen.

Ein monolithisches System mit einem Mindestmaß an interner Struktur könnte nach [Tan92] aussehen wie in Abbildung 1.6, Teil b: Die Systemschnittstelle definiert ein Funktionsangebot, d.h. eine Menge von Schnittstellenfunktionen mit festgelegten Namen und Parametern, die von außen aufgerufen werden können. Beispielsweise könnte hier eine Funktion „Erzeuge Datei“ angeboten werden. Unterhalb der Systemschnittstelle nimmt eine Hauptfunktion Systemaufrufe entgegen und ruft ihrerseits entsprechende System(dienst)funktionen auf. Jeder Schnittstellenfunktion entspricht dabei genau eine Systemfunktion. Zur Unterstützung der Systemfunktionen gibt es Hilfsfunktionen – beispielsweise eine Funktion, die freie Speicherbereiche auf dem Plattenpeicher sucht und belegt und u.a. von der Systemfunktion „Erzeuge Datei“ aufgerufen wird.

Der Kern des Betriebssystems besteht hier also aus Hauptfunktion, Systemfunktionen und Hilfsfunktionen sowie aus zugehörigen internen Datenstrukturen. Charakteristisch ist dabei, dass alle Systemfunktionen auf derselben Stufe stehen, also nicht hierarchisch geordnet sind. Sie können dieselben Hilfsfunktionen benutzen und auf dieselben Daten zugreifen. Beispiele für Betriebssysteme mit einem ähnlich strukturierten Kern sind MS-DOS und UNIX.

a.) monolithisches System (völlig unstrukturiert):



b.) monolithisches System (mit minimaler Struktur nach [Tan92]):

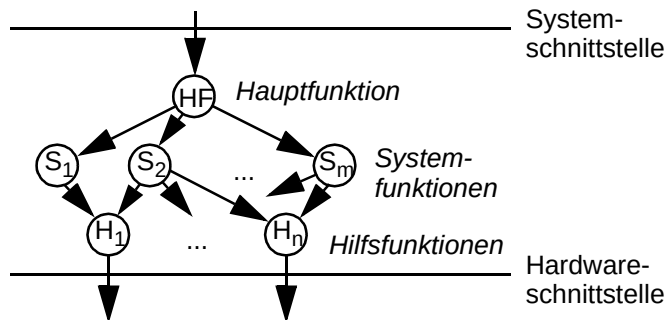


Abb. 1.6 monolithisches Betriebssystem

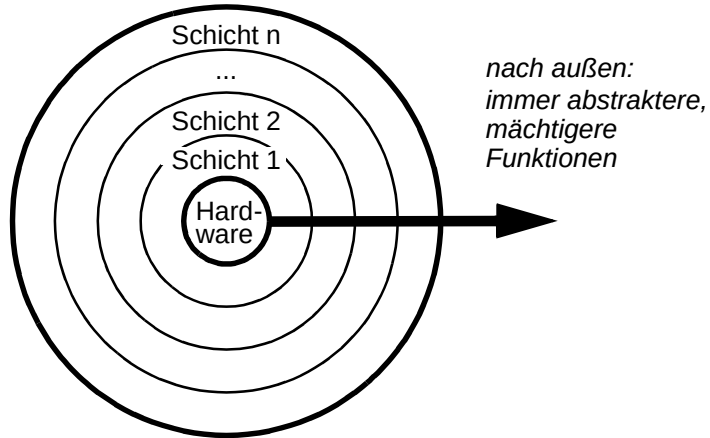
Hierarchisches System

Hierarchische Betriebssysteme folgen, im Gegensatz zu monolithischen Systemen, einem strengen Ordnungsprinzip. Der Ausgangspunkt ist hier die Hardware mit ihren einfachen Diensten. Auf dieser Grundlage werden schrittweise immer leistungsfähigere und abstraktere Funktionen definiert, bis schließlich die Funktionen der äußeren Schnittstelle erreicht sind. In einer solchen Hierarchie gibt es also keine Gleichberechtigung der Funktionen mehr.

Bildlich kann man sich die Struktur als eine Folge ineinanderliegender „Schalen“ oder „Schichten“ vorstellen, die um die Hardware gelegt werden und jeweils eine Schnittstelle für die nächstäußere Schale / Schicht bieten (siehe Abb. 1.7, Teil a). Entscheidend ist dabei, dass die Funktionen einer Schicht ausschließlich auf den Funktionen der nächstinneren Schicht beruhen, indem sie diese (und nur diese) aufrufen.

Um den Ansatz zu verdeutlichen, betrachten wir kurz ein praktisches Beispiel (siehe Abb. 1.7, Teil b): Die abstrakte Funktion „Erzeuge und initialisiere Datei“ ist in der äußersten Schicht angesiedelt, unmittelbar unterhalb der Systemschnittstelle. Sie ruft drei Funktionen aus der mittleren Schicht auf, nämlich „Belege freie Plattenblöcke“, „Initialisiere Plattenblöcke“ und „Initialisiere Verzeichniseintrag“. Diese Funktionen benut-

- a.) allgemeines Schichtenkonzept: Benutzer, Anwendungsprogramme



- b.) Beispiel für Schichten:

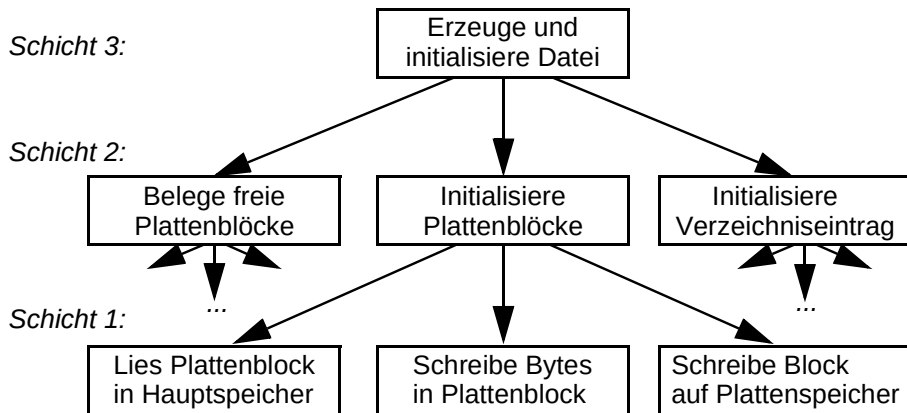


Abb. 1.7 Schichten in einem hierarchischen Betriebssystem

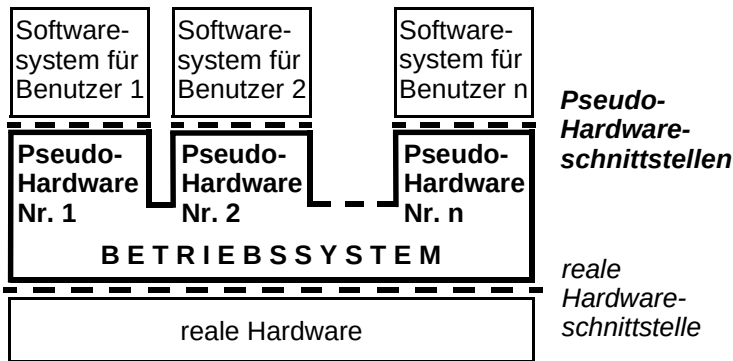
zen wiederum Funktionen der innersten Schicht, so z.B. „Lies Plattenblock in Hauptspeicher“, „Schreibe Byte in Plattenblock“ und „Schreibe Block auf Plattenspeicher“.

Reale hierarchisch aufgebaute Systeme besitzen im Allgemeinen wesentlich mehr als nur drei Schichten.

Virtuelle Maschinen

Die beiden bislang besprochenen Ansätze abstrahieren schon in ihrer untersten Schicht von der Hardware. Sie bieten also eine interne, hardwareunabhängige Schnittstelle, auf der dann höhere Konzepte wie z.B. ein Mehrprogrammbetrieb realisiert werden können. Bei *virtuellen Maschinen* (engl. *virtual machines*) ist das nicht so: Hier setzt die

a.) Betriebssystem als virtuelle Maschine:



b.) Virtuelle Maschine zur Ausführung von Java-Applets:

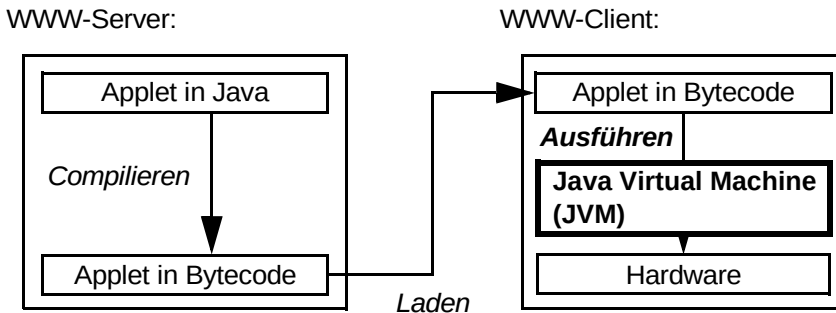


Abb. 1.8 Virtuelle Maschinen

unterste Schicht, wie bisher, auf der realen Hardware auf, bietet aber an ihrer oberen Schnittstelle wiederum die Funktionen dieser Hardware (oder auch einer anderen), und zwar vervielfacht (siehe Abb. 1.8, Teil a): Jedem Benutzer steht damit seine eigene *virtuelle Hardware* zur Verfügung; das Betriebssystem spiegelt ihm also vor, dass er unmittelbar auf dieser Hardware arbeitet und dass ihm diese allein zur Verfügung steht. Auf dieser Schnittstelle kann er beispielsweise sein eigenes Betriebssystem installieren, z.B. Windows NT oder UNIX.

Die Aufgabe einer „virtuellen Maschine“ liegt also im Wesentlichen darin, die Zugriffe über die verschiedenen virtuellen Hardwareschnittstellen so zu koordinieren, dass sich auf der realen Hardware keine Konflikte ergeben. Ein klassisches Beispiel für ein solches Betriebssystem ist VM (siehe Unterkapitel 1.4).

In jüngerer Zeit hat sich das Konzept der virtuellen Maschinen durch die *Java Virtual Machine (JVM)* recht stark verbreitet. *Java* ist eine moderne objektorientierte Programmiersprache, die C++ relativ ähnlich ist. Java-Programme können in eine Zwischensprache (den so genannten *Bytecode*) übersetzt werden, die zwar maschinennah ist, jedoch plattformunabhängig, d.h. nicht an eine bestimmte Prozessorhardware gebunden.

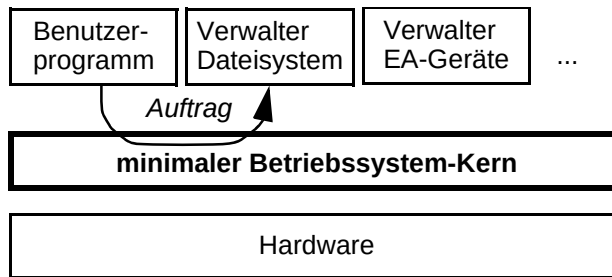


Abb. 1.9 minimaler Betriebssystemkern („Microkernel“)

Die Interpretation des Java-Bytecodes, also seine Ausführung, geschieht durch die JVM-Software. Da JVM-Implementationen für zahlreiche Hardware- und Betriebssystemplattformen existieren, kann derselbe Bytecode ohne Neuübersetzung auf verschiedenen Rechnern ausgeführt werden. Dies wird insbesondere im *World Wide Web* (WWW) ausgenutzt, wo Java-(Bytecode-)Programme (so genannte *Applets*) von WWW-Servern heruntergeladen und mit Hilfe der JVM des Browsers lokal ausgeführt werden können (siehe Abb. 1.8, Teil b).

Minimale Kerne

Bei den Ansätzen, die bisher besprochen wurden, werden alle (oder zumindest sehr viele) Funktionen des Betriebssystems innerhalb des Kerns realisiert. Der Nachteil dabei ist, dass der Kern recht groß wird und nur relativ schwer verändert oder erweitert werden kann. Alternativ kann man einen *minimalen Kern* (*Mikrokern*, engl. *microkernel*) implementieren, der nur die allernötigsten Grundfunktionen eines Betriebssystems enthält – z.B. Basisdienste für den Datenaustausch (siehe Abb. 1.9). Alle anderen Dienste (z.B. das Dateisystem) werden in eigenen *Modulen* (Bausteinen) oberhalb der eigentlichen Systemschnittstelle bereitgestellt, die leicht ausgetauscht oder geändert werden können.

Betriebssysteme mit minimalem Kern arbeiten also nach dem so genannten *Client-Server-Prinzip*: Hier bieten einige Komponenten (die *Server*) bestimmte Dienste an (im Beispiel oben das Dateisystem), die von anderen Komponenten (den *Clients*) aufgerufen werden können. Solche Betriebssysteme eignen sich insbesondere für den Einsatz in verteilten Systemen (siehe Kapitel 8 und 9), da Client und Server nicht unbedingt auf demselben Rechner laufen müssen. Nachteilig bei Systemen mit minimalem Kern ist, dass die Kommunikation zwischen den Modulen recht aufwendig ist. Sie führt in der Praxis meist zu solchen Leistungseinbußen, dass der Kern doch wieder vergrößert werden muss.

Ein klassisches Beispiel für ein Betriebssystem mit minimalem Kern ist *Mach* (siehe Abschnitt 9.4.1). Auch bei der Entwicklung von *Windows NT* wurde versucht, den Kern möglichst klein zu halten und höhere Dienste in eigene Softwaremodule auszulagern (siehe Abb. 1.10). *Windows NT* besteht daher in seiner ursprünglichen Architektur aus mehreren Schichten:

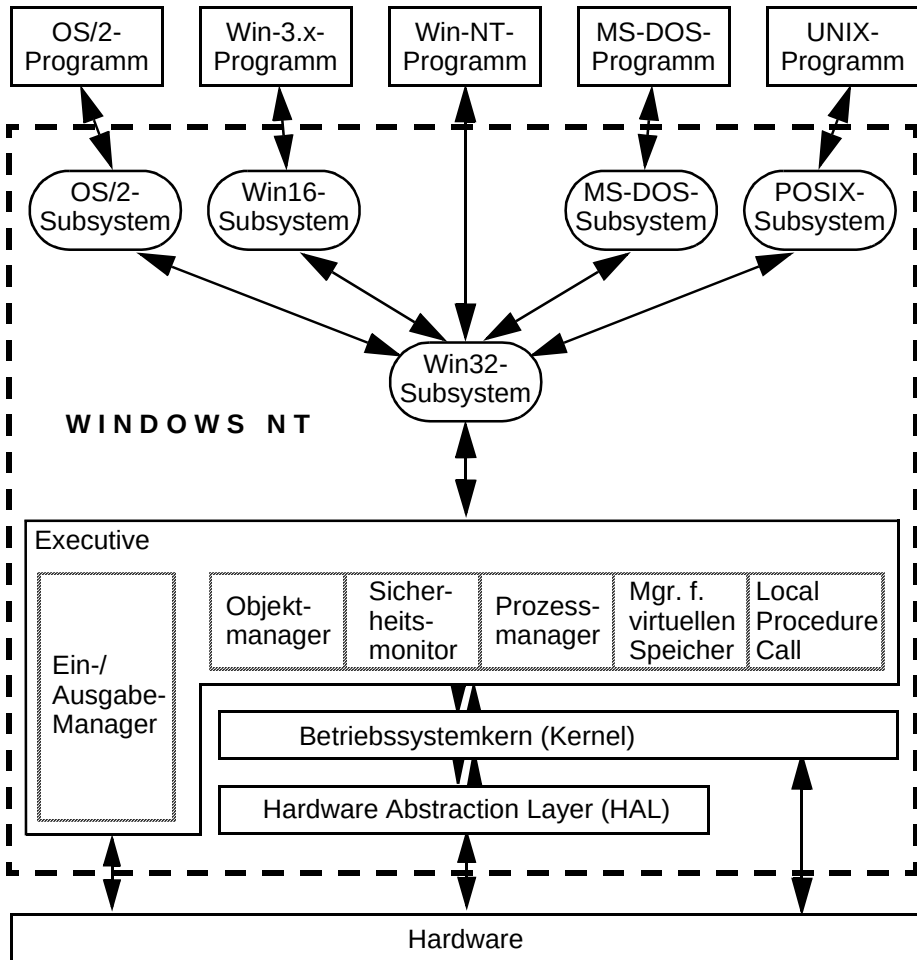


Abb. 1.10 Architektur von Windows NT (nach [SiGa98])

- Der *Hardware Abstraction Layer (HAL)* setzt unmittelbar auf der Hardware auf und bietet nach oben die Schnittstelle einer virtuellen Maschine, die die reale Hardware versteckt und auf der ein Großteil des übrigen Systems basiert. Windows NT ist damit relativ leicht portabel, da beim Übergang zu einem anderen Prozessor im Wesentlichen nur der HAL angepasst werden muss. Beim Wechsel zu einer anderen Prozessorfamilie muss allerdings neben dem HAL auch der Betriebssystemkern (siehe nächster Absatz) geändert werden, da er sich auf Eigenschaften der Rechnerarchitektur stützt.
- Der darüber liegende *Betriebssystemkern* realisiert einige wenige Basisdienste von Windows NT, die die Verwaltung der Aktivitäten des Systems (das so genannte „Thread Scheduling“ und „Synchronisationsmechanismen“ für Multiprozessorsysteme-

me), die Reaktion auf Signale von Hard- und Software („Interrupt- und Exceptionhandling“) und das Wiederhochfahren des Systems nach Spannungsausfällen betreffen.

- Der *Executive* ist eine Sammlung weiterer Dienste: Der Objektmanager stellt ein allgemeines objektorientiertes Konzept zur Verwaltung aller Systemkomponenten bereit, der Sicherheitsmonitor dient zur Prüfung und Durchsetzung von Zugriffsrechten, der Prozessmanager verwaltet die Aktivitäten im System und der Manager für den virtuellen Speicher ihren Speicherplatz, der Local-Procedure-Call-Mechanismus ermöglicht den Datenaustausch zwischen Aktivitäten, und der Ein-/Ausgabe-Manager ist für den Zugriff auf Peripheriegeräte zuständig. Letzterer darf, aus Effizienzgründen, unmittelbar auf die Hardware zugreifen.
- Auf den *Executive* setzen mehrere *Subsysteme* auf. Sie bieten nach oben Schnittstellen (= Sammlungen von Funktionen), die von Anwendungsprogrammen genutzt werden können. Neben dem Win32-Subsystem mit der „eigentlichen“ Windows-NT-Systemschnittstelle gibt es das OS/2-, das Win16-, das MS-DOS- und das POSIX-Subsystem, die Schnittstellen für OS/2- Windows-3.x-, MS-DOS- bzw. UNIX-Programme bereitstellen. Zudem gibt es Sicherheitssubsysteme (in der Abbildung nicht dargestellt), die den Systemzugang (das „Login“) kontrollieren.

Die Architektur von *Windows 2000*, dem Nachfolger von Windows NT, sieht ähnlich aus. Hier greift allerdings der Ein-/Ausgabe-Manager nicht direkt auf die Hardware zu, sondern über eine darunter liegende Schicht von Gerätetreibern, die ihrerseits auf dem HAL aufsetzen. Neue Bestandteile des Executives sind der Plug-and-Play-Manager und der Power-Manager, die sich um das Einbinden neuer Hardwarekomponenten bzw. die Verringerung des Energieverbrauchs kümmern.

1.3.2 Der Kern eines Betriebssystems

Im vorigen Abschnitt wurde mehrfach der Begriff des *Betriebssystemkerns* benutzt. Der Kern ist, um die Definition zu wiederholen, der Teil der Betriebssystemsoftware, der auf der Hardware aufsetzt und nach oben die Systemschnittstelle definiert (siehe Abb. 1.11). Die Systemschnittstelle bietet Funktionen, die aus Programmen heraus aufgerufen werden können – beispielsweise zum Schreiben von Daten in eine Datei, zum Senden eines Signals an ein Peripheriegerät oder zur Abfrage der Systemuhr. Bei den Programmen, die die Systemschnittstelle benutzen, handelt es sich um Benutzerprogramme oder weitere Software des Betriebssystems (z.B. in Windows NT die Subsysteme). Der Kern eines Betriebssystems erfüllt die folgenden beiden Aufgaben:

- Er „abstrahiert“ von spezifischen Eigenschaften der Hardware (er „verbirgt“ sie also) und bietet somit nach oben eine einheitliche, hardwareunabhängige Schnittstelle zum System.

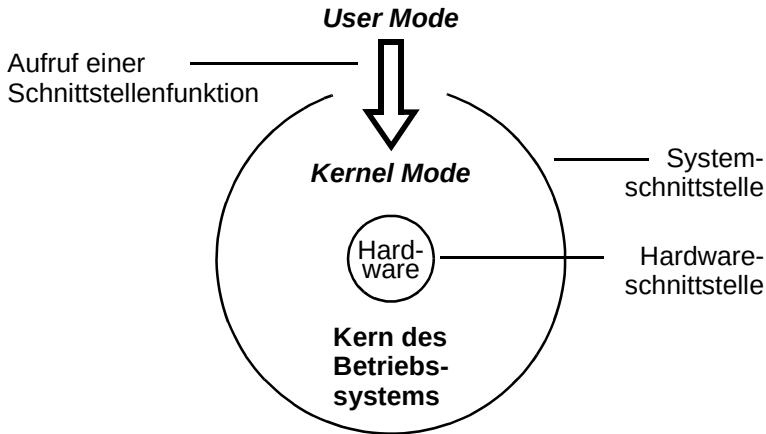


Abb. 1.11 Kern und Ausführungsmodi

- Er realisiert grundlegende Verwaltungsoperationen, wie z.B. die Zuteilung des Prozessors und des Speichers oder die Durchsetzung von Schutzmechanismen für Dateien, also Operationen, die fundamental für das Funktionieren des Systems sind.

Im Zusammenhang mit dem Kern wird ein Sicherheitskonzept realisiert. Es stützt sich auf verschiedene *Ausführungsmodi*, die unterschiedlich große Rechte (*Privilegien*) bieten. Sie garantieren, dass sicherheitskritische Operationen (beispielsweise, im Extremfall, ein Maschinenbefehl zum Anhalten des Prozessors) nur durch berechtigte Anwendungen (z.B. Programme des Systemverwalters) ausgeführt werden können.

Es wird zwischen mindestens zwei Ausführungsmodi unterschieden, nämlich dem *User Mode* und dem *Kernel Mode* (siehe Abb. 1.11): Programmcode des Benutzers wird außerhalb des Kerns im User Mode ausgeführt und hat dabei nur geringe Privilegien. Er hat keine oder nur eingeschränkte Zugriffsrechte auf Daten des Betriebssystems und darf bestimmte sicherheitskritische Maschinenbefehle nicht benutzen. Betriebssystemfunktionen innerhalb des Kerns werden im Kernel Mode ausgeführt. Sie genießen hohe Privilegien, haben beispielsweise uneingeschränkte Rechte zur Änderung von Betriebssystemdaten und dürfen sämtliche Maschinenbefehle ausführen.

Die unterschiedlichen Privilegien werden durch die Hardware kontrolliert, was im Prinzip folgendermaßen geschieht: Der Prozessor besitzt ein bestimmtes Flag (ein *Mode Bit* in einem Register), das den aktuellen Ausführungsmodus angibt – z.B. Mode Bit = 0 für den User Mode, Mode Bit = 1 für den Kernel Mode. Moderne Prozessoren unterstützen sogar mehr als diese zwei Modi, heutige Intelprozessoren beispielsweise vier. Frühere Prozessoren, wie z.B. der Intel 8088, hatten kein solches Bit, weswegen entsprechende Betriebssysteme, wie z.B. MS-DOS, keine unterschiedlichen Modi implementieren konnten.

Die Hardware führt bestimmte privilegierte Operationen nur dann aus, wenn das Flag den Kernel Mode anzeigt. Der Übergang in den privilegierten Modus, d.h. das „Hochsetzen“ des Flags, ist nur unter bestimmten Randbedingungen möglich – so zum Bei-

spiel, wenn gleichzeitig eine Funktion der Systemschnittstelle aufgerufen wird (siehe Abb. 1.11). Damit wird sichergestellt, dass ein normaler Benutzer nur dann höhere Rechte bekommt, wenn er in eine Funktion eintritt, deren Schritte das Betriebssystem festgelegt hat. Ein Benutzer darf also sicherheitskritische Operationen nur unter Betriebssystemkontrolle durchführen.

Windows NT / 2000 ist ein Beispiel für ein Betriebssystem, das verschiedene Modi unterscheidet (siehe Abb. 1.10): HAL, Kernel und Executive arbeiten im privilegierten Modus, während die Subsysteme und Anwendungsprogramme im nichtprivilegierten Modus laufen. Auch UNIX hat zwei Ausführungsmodi; wir werden hierauf in Abschnitt 2.2.3 zurückkommen.

1.4 Betriebssystembeispiele

Zum Abschluss der Einführung sollen nun kurz einige Betriebssystembeispiele betrachtet werden. Zunächst ist zu bemerken, dass *das* Standardbetriebssystem nicht existiert: Erstens sind dazu die Hardwareplattformen und Anwendungsbereiche zu unterschiedlich. Zweitens haben Betriebssysteme eine sehr lange Lebensdauer, so dass mehrere Generationen unterschiedlich weit entwickelter Betriebssysteme nebeneinander existieren. Drittens gibt es auf dem Markt mehrere Betriebssystemhersteller, die in teilweise harter Konkurrenz zueinander stehen: Herausragende Beispiele hierfür sind der Kampf von IBM mit OS/2 gegen Microsoft mit MS-DOS und Windows 95 in der zweiten Hälfte der neunziger Jahre sowie der aktuelle Wettbewerb zwischen UNIX / Linux und den Windows-Systemen.

Bei aller Konkurrenz gab und gibt es jedoch auch Bemühungen zur Vereinheitlichung von Betriebssystemen, so beispielsweise das Firmenkonsortium *Open Group* und die UNIX-Standards (siehe Kapitel 2). Einen etwas anderen Charakter hat die „Standardisierung“, die die Firma Microsoft anstrebt – nämlich eine Dominierung des Markts durch ihre Windows-Betriebssysteme.

Betriebssysteme können nach unterschiedlichen Kriterien klassifiziert werden. Eine Möglichkeit ist die Differenzierung anhand der Rechnerplattform, auf der sie laufen – also Großrechner, Workstations, Personal Computer, mobile oder eingebettete Systeme.

1.4.1 Betriebssysteme für Großrechner

Prominente Beispiele für Großrechner-Betriebssysteme sind *MVS/ESA (Multiple Virtual Storage System / Enterprise System Architecture)* und sein Nachfolger *OS/390*, das heute eines der beiden Hauptbetriebssysteme für IBM-Großrechner („Mainframes“) ist. *MVS* und *OS/390*, deren Ursprünge in das Jahr 1974 zurückgehen, werden auf der Hardwareserie *IBM 370* bzw. *IBM 390* eingesetzt – einer Reihe von Maschinen mit un-

terschiedlicher Leistungsfähigkeit, aber ähnlicher Architektur. Die Betriebssysteme dienen hauptsächlich für Batch- und Transaktionsbetrieb, unterstützen jedoch auch interaktive Anwendungen. Seit Frühjahr 2001 bietet IBM unter der Bezeichnung *z/OS* eine Nachfolgeneration von *OS/390* an, die im Bereich der Hochleistungsserver für kommerzielle netzbasierte Applikationen eingesetzt werden soll.

VM (Virtual Machine) ist ein weiteres IBM-Betriebssystem für Großrechner, das es seit Anfang der siebziger Jahre gibt. *VM* ist nach dem Strukturierungsprinzip „Virtuelle Maschinen“ aufgebaut: Die *VM*-Komponente *CP (Control Program)* basiert auf der realen Hardware und bietet nach oben eine virtuelle Hardwareschnittstelle (siehe Abschnitt 1.3.1). Die Komponente *CMS (Conversational Monitor System)*, die darauf aufsetzt, steuert den Dialogbetrieb mit dem Benutzer. Anstelle von *CMS* kann man oberhalb von *CP* auch andere Betriebssysteme benutzen, wie z.B. *MVS*, *DOS*, oder das „IBM-UNIX“ *AIX*.

Ein weiteres Beispiel für ein Großrechner-Betriebssystem ist *BS 2000* der Firma Siemens, das (ähnlich wie *OS/390*) auf einer ganzen Serie von Hardwareplattformen mit unterschiedlicher Leistungsfähigkeit eingesetzt werden kann.

1.4.2 Betriebssysteme für Workstations und Minirechner

Eines der wichtigsten Betriebssysteme überhaupt ist *UNIX*, dessen Anfänge in die frühen siebziger Jahre zurückgehen. *UNIX* ist ein Mehrbenutzer-Betriebssystem, das voll multiprogrammfähig ist sowie Dialog- und Batchbetrieb unterstützt. *UNIX* ist prinzipiell unabhängig von einer bestimmten Hardwareplattform. Es wurde zunächst hauptsächlich für Workstations eingesetzt, ist aber seit einiger Zeit auch auf dem PC-Sektor im Kommen, und zudem gibt es Versionen für Großrechner.

In jüngerer Zeit gerät *UNIX* verstärkt unter Druck von Seiten *Windows NT* und *Windows 2000*. Zwar sind die *Windows*-Systeme von ihrem Ursprung her PC-Betriebssysteme; dies fällt aber immer schwächer ins Gewicht, da sich die Leistungsfähigkeit von PCs und Workstations zunehmend angleicht. Das *UNIX*-Derivat *Linux* macht dafür *Windows* auf seiner angestammten PC-Plattform immer größere Konkurrenz. *UNIX* und *Linux* werden in Kapitel 2 näher charakterisiert.

Das Betriebssystem *Mach* hat seinen Ursprung im Jahr 1984, ist die Basis für *OSF/1*, das Standard-Betriebssystem der Open Group (früher Open Software Foundation = *OSF*), und beeinflusste die Entwicklung der Architektur von *Windows NT*. *Mach* verfügt über einen „Microkernel“ (siehe Abschnitt 1.3.1), auf dem eine „Emulation“ anderer Betriebssysteme, z.B. *UNIX*, möglich ist. Das bedeutet, dass oberhalb von *Mach* die Systemschnittstellen anderer Betriebssysteme realisiert werden können. *Mach* galt lange Zeit als ein Betriebssystem der Zukunft, da es insbesondere für den Einsatz auf nebenläufiger Hardware (Multiprozessoren, verteilte Systeme) geeignet ist, eine gute Portabilität hat und, last but not least, über die politische Unterstützung der *OSF* verfügte. In letzter Zeit ist es allerdings stiller um *Mach* geworden, und *OSF/1* wird seit 1995 nicht mehr weiterentwickelt.

Beispiele für Minirechner-Betriebssysteme sind *Open VMS* von DEC für *VAX*-Rechner und *OS/400* von IBM für *AS/400*-Computer, die hauptsächlich in mittelständischen Betrieben oder als Abteilungsrechner eingesetzt werden.

1.4.3 Betriebssysteme für PCs

MS-DOS

Das ursprüngliche Betriebssystem für Personal Computer ist *MS-DOS* (*Microsoft Disk Operating System*) von Microsoft. Sein anfänglicher Erfolg war eng mit dem Aufkommen der IBM-PCs Anfang der achtziger Jahre verbunden, und es diente lange Zeit als Standard-Betriebssystem für PCs mit Intel-Prozessoren. Später wurde es dort überwiegend zusammen mit der fensterorientierten Benutzeroberfläche *Windows 3.x* benutzt (und wird es manchmal heute noch). MS-DOS ist ein Einbenutzer-Betriebssystem, das praktisch kein Multiprogramming erlaubt, wohl aber Dialog- und Batchbetrieb unterstützt.

Windows-Betriebssysteme

MS-DOS mit *Windows 3.x* wurde durch *Windows 95* (Nachfolger: *Windows 98* und *Windows ME* – ME = „Millennium Edition“) sowie *Windows NT* (Nachfolger: *Windows 2000*) abgelöst, die einen wesentlichen technischen Fortschritt gegenüber den veralteten MS-DOS-Konzepten bieten. Neben zahlreichen weiteren modernen Ansätzen verfügen sie über Multiprogramming-Konzepte, verbesserte Verfahren zur Speicherverwaltung, Netzfähigkeit sowie eine integrierte graphische Oberfläche. Während sich *Windows 95 / 98 / ME* primär an den Privatanutzer richten, sind *Windows NT / 2000* eher für den kommerziellen Einsatz gedacht. Daraus ergeben sich Unterschiede in Ansprüchen und Leistungsumfang: *Windows NT / 2000* stellen tendenziell höhere Hardwareanforderungen als *Windows 95 / 98 / ME*, unterstützen dafür aber die volle *Win32-Programmierschnittstelle* und bieten mehr Möglichkeiten für den Einsatz in Rechnernetzen. Ziel von Microsoft ist, die NT/2000-Linie und die 95/98/ME-Linie miteinander zu verschmelzen.

Windows 2000 wurde ursprünglich als *Windows NT 5.0* bezeichnet, stellt also eine unmittelbare Weiterentwicklung der letzten NT-Version 4.0 dar. Es hat viele Eigenschaften von NT übernommen und bietet darüber hinaus eine Reihe weiterer Dienste. Hierzu gehört z.B. das von *Windows 95* her bekannte „Plug and Play“ zur (hoffentlich) leichteren Installation von neuen Hardwarekomponenten. Am interessantesten ist aber sicherlich die Fortentwicklung und Neueinführung von Konzepten zur Unterstützung von vernetzten Computern, wie beispielsweise Lastverteilung, Sicherheitsmechanismen, Fehlertoleranz und insbesondere Verzeichnisdienste zur Buchführung über Ressourcen. Wir werden hierauf in Kapitel 9 noch zurückkommen.

Windows NT und *Windows 2000* wurden bzw. werden in verschiedenen Versionen angeboten, die aber jeweils auf derselben Softwarearchitektur basieren: *Windows NT Workstation* ist ein „Desktop-Betriebssystem“, das für den Einsatz auf einem Arbeitsplatz-Computer gedacht ist, während *Windows NT Server* zusätzlich Funktionen eines Netzwerkbetriebssystems zum Einsatz in einem Rechnernetz enthält. Die Workstationversion von *Windows 2000* heißt *Windows 2000 Professional*, und als Serverversionen werden *Windows 2000 Server*, *Advanced Server* und *Datacenter Server* an-

geboten. Letztere unterscheiden sich in der Anzahl der unterstützten Rechnerknoten im Verbund, den Größen des direkt zugreifbaren Hauptspeichers und damit auch den möglichen Anwendungen.

Weitere PC-Betriebssysteme

Das Betriebssystem *OS/2* wurde ab 1985 zunächst gemeinsam durch IBM und Microsoft entwickelt, wurde jedoch später zu IBM's Konkurrenzprodukt zu MS-DOS und dessen Nachfolgern. Wesentlich früher als die Windows-Systeme realisierte *OS/2* moderne Betriebssystem-Eigenschaften, so unter anderem Multiprogramming, einen „virtuellen Speicher“ (siehe Kapitel 5), verschiedene Schutzmechanismen, Netzfähigkeit sowie die graphische Bedienoberfläche *Presentation Manager*. Trotz dieser technischen Vorteile konnte sich *OS/2* schon gegenüber MS-DOS / Windows 3.x nicht durchsetzen, da für Microsoft-Betriebssysteme sehr viel Anwendungssoftware existiert. IBM hat seine Weiterentwicklung mittlerweile aufgegeben.

MacOS ist das Betriebssystem für die Personal Computer der Firma Apple, die zwar längst nicht dieselbe Verbreitung gefunden haben wie Windows/Intel-basierte PCs, aber bevorzugt in Bereichen eingesetzt werden, in denen viel graphisches Design und Bildverarbeitung betrieben werden. Der Lisa-PC von Apple war der erste PC mit einer graphischen, fensterorientierten Oberfläche – lange vor den Windows-Systemen.

Weitere Beispiele für PC-Betriebssysteme sind *Linux* (das „PC-UNIX“), auf das wir noch im Detail eingehen werden, sowie *Netware* von Novell.

1.4.4 Betriebssysteme für mobile und eingebettete Systeme

Während die Trennlinie zwischen Workstations und PCs immer stärker verschwimmt, hat sich weiter unterhalb eine neue Gerätelinie etabliert – nämlich mobile Systeme wie *Handheld Computers* (*Palmtops*, *Personal Digital Assistants* – PDAs) und *Mobiltelefone* (*Smartphones*). Wegen der eingeschränkten Hardwaremöglichkeiten können hier die klassischen Betriebssysteme nicht eingesetzt werden, sondern es sind Spezialentwicklungen nötig. Beispiele für entsprechende Betriebssysteme sind

- *PalmOS*, das von der Firma 3COM auf ihrer Palm-Handheld-Serie eingesetzt wird,
- *Epoc*, ein Betriebssystem des Symbian-Konsortiums für Psion-Handhelds und Mobiltelefone, sowie
- *Windows CE* von Microsoft, das auch auf eingebettete Systeme zielt – Systeme, bei denen der Computer in ein größeres Gerät integriert ist, um es zu steuern oder eine verbesserte Kommunikation mit der Umwelt zu ermöglichen.

Gemeinsam sind diesen Systemen eine effiziente Ausnutzung der beschränkten Ressourcen (Prozessor- und Speicherkapazität, Eingabemöglichkeiten, Bildschirmanzeige und insbesondere Energie), die Unterstützung verschiedener Möglichkeiten des Infor-

mationsaustauschs (per Kabel, Infrarot- oder Funkwellen mit einem PC oder dem Internet) und die Integration bestimmter Anwendungsdienste (z.B. Adressverzeichnisse, Terminkalender, To-Do-Listen sowie Internet-Applikationen wie Mailing und WWW-Zugriff). Zur Programmierung bieten die Betriebssysteme Schnittstellen, wobei auch Anwendungen in höheren Programmiersprachen (z.B. C++, Java) erstellt werden können.

Neben den genannten Systemen gibt es eine Reihe weiterer Betriebssysteme für Spezialbereiche – so z.B. *Betanova* für die Set-Top-Box *dbox*, die für das digitale Fernsehen benutzt wird, und *OSEK*, ein Standard für eingebettete Systeme in Kraftfahrzeugen.

1.4.5 Das Beispiel-Betriebssystem dieses Buchs

In diesem Buch sollen die wichtigsten Betriebssystemkonzepte anhand eines konkreten Beispiels illustriert werden. Da hierzu ein modernes, weit verbreitetes Betriebssystem dienen soll, kommen vornehmlich UNIX / Linux und Windows NT in Frage. Aus Platzgründen können nicht beide besprochen werden, so dass eins von ihnen ausgewählt werden muss. Die Wahl fällt auf UNIX / Linux, und zwar aus folgenden Gründen:

- Die Programmierschnittstelle von UNIX / Linux ist einfacher gestaltet als die von Windows NT. Beispielsweise haben die Schnittstellenfunktionen von Windows NT im Allgemeinen mehr Parameter als die von UNIX, was einerseits eine größere Flexibilität ermöglicht, andererseits aber zumindest für den Anfänger verwirrend sein kann.

Zur Illustration soll kurz die Systemfunktion zur Erzeugung eines neuen Prozesses betrachtet werden: Wie Abbildung 1.12 zeigt, hat die UNIX-Funktion `fork()` keinen einzigen Parameter, das entsprechende Gegenstück `CreateProcess()` in Windows NT zehn. Der Fairness halber ist zwar zu sagen, dass `fork()` in vielen Fällen durch den Aufruf einer `exec()`-Funktion ergänzt wird, die für den neuen Prozess ein Programm in einer anderen Datei aufruft. `CreateProcess()` vereinigt beide Funktionalitäten, ist aber immer noch wesentlich komplexer als `fork()` und `exec()` zusammen.

- Der Quellcode von Windows NT wird streng geheim gehalten. Dagegen gibt es im UNIX-Sektor „Open-Source“-Versionen, deren Quellcode offenliegt und so einen tieferen Einblick in Betriebssystemmechanismen ermöglicht – insbesondere das Paradebeispiel Linux, aber neuerdings z.B. auch Solaris von Sun Microsystems. Studierende können Linux auf ihren privaten PCs installieren, den Quellcode inspizieren und mit ihm experimentieren.

Für Anwender ist bei einem Open-Source-System auch interessant, dass sie nachprüfen können, welche Operationen das Betriebssystem auf ihren Computern ausführt. Dieser Gesichtspunkt ist ein Grund für immer wieder laut werdende Überlegungen, im öffentlichen und kommerziellen Bereich eingesetzte Windows-Betriebssysteme durch Linux zu ersetzen.

UNIX / Linux:

```
int fork();
```

Windows NT:

```
BOOL CreateProcess (
    LPCTSTR          lpAppName,
    LPCTSTR          lpCommandLine,
    LPSECURITY_ATTRIBUTES lpProcessAttributes,
    LPSECURITY_ATTRIBUTES lpThreadAttributes,
    BOOL             bInheritHandles,
    DWORD            dwCreateFlags,
    LPVOID           lpEnv,
    LPCTSTR          lpCurrentDirectory,
    LPSTARTUPINFO     lpStartupInfo,
    LPPROCESS_INFORMATION lpProcessInformation
);
```

Abb. 1.12 Funktionen zur Prozesserzeugung in UNIX / Linux und Windows NT

- UNIX ist das auf dem Workstation-Markt führende Betriebssystem und hat eine starke Stellung im Server-Bereich (wenn auch hier eine wachsende Konkurrenz von Seiten Windows NTs und seines Nachfolgers Windows 2000 besteht). Im PC-Bereich wird Linux auch im kommerziellen Sektor immer beliebter.

In diesem Buch wird also UNIX / Linux als durchgehendes Systembeispiel dienen. Wir betrachten dabei sowohl die „klassische“ UNIX-System-V-Implementation nach [Bach86] als auch den aktuellen Linux-Kern 2.x, der allgemein frei verfügbar ist (siehe auch Unterkapitel 2.1). Zudem werden Aspekte anderer Betriebssysteme besprochen – insbesondere Windows NT und auch Windows 2000.

1.5 Übungsaufgaben

1. Wissensfragen

- a.) Was sind die beiden Hauptaufgaben eines Betriebssystems?
- b.) Nennen Sie drei Betriebsarten.
- c.) Beschreiben Sie stichwortartig, was man unter „Spooling“ versteht.

- d.) Wie ist ein hierarchisches Betriebssystem aufgebaut?
- e.) Was ist der Kern eines Betriebssystems?
- f.) Was sind „Kernel Mode“ und „User Mode“?

2. Klassifikation von Betriebsmitteln

Ergänzen Sie die folgende Tabelle: Tragen Sie zunächst in die Titelzeile die Merkmale ein, anhand derer Betriebsmittel klassifiziert werden können. Ordnen Sie dann die angegebenen Betriebsmittel entsprechend ein.

Klassen →				
CPU				
CD-Brenner				
CD-Rohling (Write-Only)				
Datei <code>prog.c</code> mit C-Programmcode während der Programm-erstellung				
Datei <code>prog.exe</code> mit ausführbarem Programm				

3. Sequentielle Bearbeitung und Multiprogramming im Vergleich

Gegeben sind drei Aufträge A, B, C, die zum Zeitpunkt 0 zur Bearbeitung anstehen. Für jeden Auftrag müssen die folgenden Schritte ausgeführt werden: Dateneingabe 1 → warten, bis CPU (= Prozessor) frei → CPU-Benutzung 1 → Dateneingabe 2 → warten, bis CPU frei → CPU-Benutzung 2. Die folgende Tabelle gibt die Zeitdauern der einzelnen Schritte an:

	Eingabe 1	CPU 1	Eingabe 2	CPU 2
Auftrag A	15	2	10	3
Auftrag B	10	6	5	2
Auftrag C	5	5	10	5

Die Wartezeit auf die CPU ist nicht vorherbestimmt; sie ergibt sich aus dem Wettbewerb der Aufträge um die CPU.

Es gelten die folgenden Annahmen:

- Die Prioritätsstufung ist $A > B > C$, d.h.: Bei sequentieller Bearbeitung wird erst A vollständig bearbeitet, dann B und dann C. Bei Multiprogramming wird die CPU jeweils demjenigen der ausführungswilligen Aufträge zugeteilt, der die höchste Priorität hat; dabei können niederpriorie Aufträge unterbrochen werden.
- Unter Multiprogramming gilt: Wird ein Auftrag unterbrochen, so kann er später fortgesetzt werden, wobei seine bisherige CPU-Benutzung angerechnet wird. Ein Auftrag braucht zur Eingabe die CPU nicht; Eingaben können unbeschränkt nebenläufig zu Aktionen anderer Aufträge erfolgen.
- Das Umschalten der CPU zwischen Aufträgen kostet keine Zeit.

Lösen Sie nun für sequentielle Bearbeitung und Multiprogramming jeweils die folgenden Aufgaben:

a.) Geben Sie an, wann die CPU welchen Auftrag bedient und wann sie „idle“ (= unbeschäftigt) ist.

b.) Berechnen Sie die folgenden Werte:

- Aufenthaltsdauern der Aufträge A, B und C (= Zeitdauer vom Zeitpunkt 0 bis zur Fertigstellung des Auftrags)
- mittlere Aufenthaltsdauer (= arithmetisches Mittel der drei Aufenthaltsdauern)
- Auslastung der CPU (= prozentualer Anteil der Zeit der Auftragsbearbeitungen an der Gesamtzeit)

2 Das UNIX-Betriebssystem

Das grundlegende Beispiel-Betriebssystem dieses Buchs ist UNIX: In den folgenden Kapiteln werden wir jeweils Betriebssystemaspekte zuerst allgemein darstellen und sie dann anhand von UNIX und / oder seiner Freewareversion Linux konkretisieren. Bevor wir aber mit der Besprechung dieser allgemeinen Themenbereiche beginnen, geben wir zunächst eine kurze Übersicht über die Geschichte von UNIX, seine Struktur und seine wichtigsten Konzepte.

2.1 Geschichte

Ursprünge

Der Ursprung von UNIX liegt im *MULTICS*-Betriebssystem, das in den späten sechziger Jahren an den amerikanischen Bell Laboratories, einer gemeinsamen Tochter der Telefongesellschaft AT&T und der Elektrogerätefirma Western Electric, entwickelt wurde. MULTICS („MULTiplexed Information and Communication Service“) sollte in der Lage sein, hunderte Benutzer gleichzeitig zu unterstützen. Das war für die damaligen technischen Verhältnisse ein sehr ehrgeiziges Ziel und wurde so auch einer der Gründe für den (zumindest wirtschaftlichen) Misserfolg des Projekts. Zwei Mitarbeiter der Bell Labs, *Ken Thompson* und *Dennis Ritchie*, begannen daraufhin, eine abgespeckte MULTICS-Version für die *PDP-Minicomputer*-Serie zu schreiben, die damals recht weit verbreitet war. Dieses neue Betriebssystem erhielt – in Anspielung auf die Bezeichnung MULTICS – den Namen *UNIX*. Seine ersten Versionen erschienen in den frühen siebziger Jahren.

UNIX war zunächst in Assembler geschrieben, wurde aber bald in die höhere Programmiersprache *B* portiert, um seine Übertragbarkeit zwischen verschiedenen Hardwareplattformen zu verbessern. Da *B* einige Schwächen aufwies (es hatte z.B. keine Struktur-Datentypen), entwickelte Ritchie eine Nachfolgesprache, die er folgerichtig *C* nannte und in die UNIX dann portiert wurde. Zudem entstand eine Benutzeroberfläche von UNIX, die *Shell*.

UNIX verbreitete sich rasch an Hochschulen. Das lag erstens daran, dass die PDP-11 damals im Hochschulbereich sehr populär war, und zweitens, dass aus wettbewerbsrechtlichen Gründen die Bell Labs bei der Weitergabe des UNIX-Quellcodes nur eine geringe Gebühr verlangen durften.

UNIX-Versionen

Ausgehend von AT&T's Version „System I, Version 7“ aus dem Jahre 1979, wurde UNIX in verschiedenen Linien weiterentwickelt (siehe Abb. 2.1). Die AT&T-Linie mündete 1983 in das *UNIX System V* (lies „System Five“), das zu AT&T's „offizieller“ UNIX-Version wurde und in mehreren *Releases* mit jeweils verbesserten Leistungen und erweitertem Funktionsumfang fortgeschrieben wurde. 1984 gab AT&T die *System V Interface Definition (SVID)* heraus, die die Funktionen der Systemschnittstelle, die C-Bibliotheken und die Dienstprogramme definierte.

Eine zweite UNIX-Linie, die *Berkeley Software Distribution (BSD)*, wurde seit Beginn der achtziger Jahre an der *University of California at Berkeley* entwickelt. Die erste Version, *BSD 4.1*, basierte auf UNIX System I, Version 7, und unterschied sich von diesem durch eine bessere Verwaltung des Arbeitsspeichers, ein schnelleres Dateisystem, Möglichkeiten zum Datenaustausch zwischen Prozessen und verschiedene Dienstprogramme, wie *C-Shell* und *vi-Editor*.

Aus diesen beiden Hauptlinien entwickelten sich die verschiedenen UNIX-Versionen der einzelnen Computer-Hersteller sowie die bekannten Freeware-UNIXe. So brachte beispielsweise die Firma *Sun Microsystems* Mitte der achtziger Jahre die erste Version ihres *SunOS*-Betriebssystems heraus, das auf BSD 4.2 basierte und zusätzlich eine graphische Bedienoberfläche mit Fenstern sowie Konzepte zur Unterstützung eines verteilten Dateisystems in einem Computernetz bot. Die *SunOS*-Linie wurde später in *Solaris* umbenannt. *Hewlett-Packard (HP)* und *IBM* vertreiben ebenfalls UNIX-Versionen unter den Namen *HP-UX* bzw. *AIX*, und auch weitere Hersteller bieten für ihre Hardwareplattformen UNIX-Betriebssysteme an. Frei erhältliche UNIX-Versionen für PCs sind *FreeBSD*, das von einer freien Programmierergruppe entwickelt wurde, und insbesondere *Linux*.

Linux wurde 1991 von *Linus Torvalds*, einem finnischen Studenten, initiiert und mit Kern und Dienstprogrammen in den folgenden Jahren stetig weiterentwickelt, woran eine größere Zahl freiwilliger Programmierer beteiligt war. Linux ist nach wie vor frei erhältlich; zudem werden von kommerziellen Anbietern so genannte *Linux-Distributionen* vertrieben, die u.a. eine komfortable Installation ermöglichen sollen. Die verschiedenen Distributionen basieren aber alle auf demselben Linux-Kern in seiner aktuellen Version, dessen Versionsnummer (siehe Abb. 2.1) übrigens nicht verwechselt werden darf mit den rasch steigenden Versionsnummern der Distributionen.

Standards

Um dem Auseinanderstreben der verschiedenen UNIX-Versionen (insbesondere System V und BSD) entgegenzuwirken, gibt es seit Ende der achtziger Jahre Standardisierungsbemühungen:

- Das *IEEE (Institute of Electrical and Electronics Engineers)* initiierte *POSIX (Portable Operating System Interface)* zur Standardisierung von Betriebssystemschnittstellen. Hierdurch soll die Portabilität von Quellcode zwischen verschiedenen Systemen ermöglicht werden.

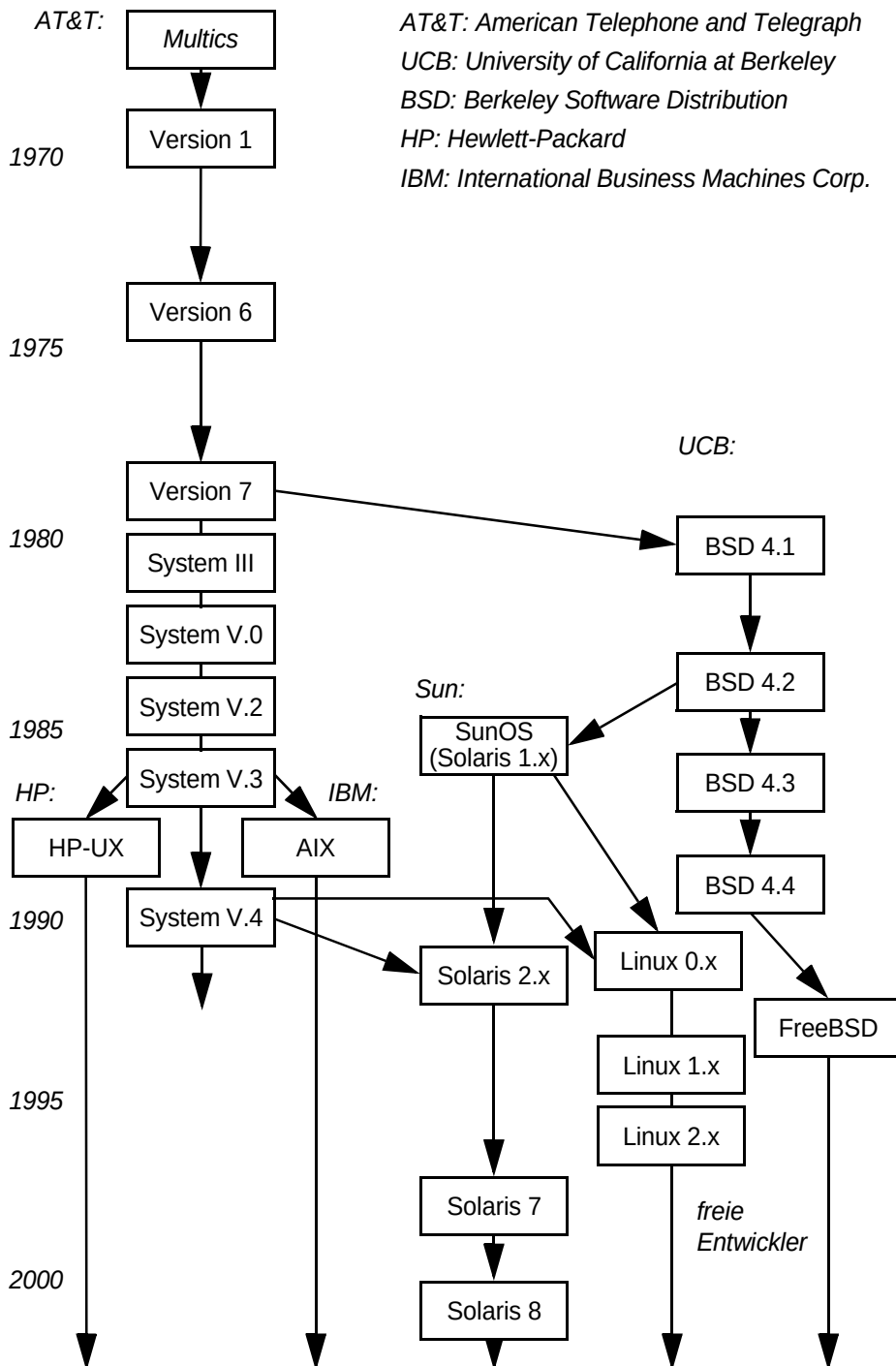


Abb. 2.1 Stammbaum einiger UNIX-Versionen (auf der Basis von [GuOb95] und [SiGa98])

Der *POSIX-Standard 1003.1* aus dem Jahre 1990 legt z.B. einen Satz von ca. 60 Bibliotheksfunktionen für die C-Schnittstelle des Betriebssystems (siehe Unterkapitel 2.2) fest [Lew91]. Moderne UNIX-Versionen unterstützen im Allgemeinen diesen Funktionssatz und bieten darüber hinaus eine Vielzahl weiterer Funktionen.

Andere Standards 1003.x betreffen beispielsweise die Shell, Sicherheitsaspekte, Dateizugriffe und Realzeiterweiterungen. Hier ist insbesondere *POSIX 1003.1b* (früher POSIX.4) zu nennen, wo u.a. Funktionsschnittstellen zur Prozesssynchronisation und -kommunikation festgelegt werden [Gall95].

- *X/Open*, ein 1984 gegründetes Industriekonsortium (später *Open Group*), gab 1992 den *X/Open Portability Guide (XPG)* heraus. Er standardisiert UNIX-Systemaufrufe mit entsprechenden Funktionsbibliotheken, Benutzerkommandos und Dienstprogramme, Datenbankkonzepte sowie Konzepte für den Datenaustausch, wobei er sich teilweise auf POSIX bezieht.

Später erhielt X/Open das Recht, Betriebssysteme hinsichtlich ihrer Kompatibilität zu einem „Standard-UNIX“ zu zertifizieren. Als Grundlage hierfür wurde 1997 die *Single UNIX Specification*, Vs. 2 (auch bekannt als *UNIX-98-Spezifikation*) festgelegt [Open97].

- Als dritte Gruppe ist in diesem Zusammenhang noch die *Open Software Foundation (OSF)* zu erwähnen. Sie entwickelte ein eigenes Betriebssystem namens *OSF/1*, das unter anderem eine UNIX-Schnittstelle bietet. Wir werden darauf und auf andere OSF-Standards bei der Diskussion verteilter Systeme zurückkommen. Die OSF schloss sich 1996 mit X/Open zur Open Group zusammen.

Allein die Existenz mehrerer Standardisierungsgremien zeigt, dass es einen einzigen UNIX-Standard noch nicht gibt. Vielmehr gilt (in gewissen Grenzen) eine Aussage aus der „Informatik-Folklore“, die man u.a. in [Tan96] findet: „The nice thing about standards is that you have so many to choose from. Furthermore, if you do not like any of them, you can just wait for next year's model.“ In der Praxis ist dies jedoch längst nicht so kritisch, wie es scheinen mag, denn es gibt viele Gemeinsamkeiten und zudem unterstützen viele UNIX-Versionen mehrere Schnittstellenstandards – insbesondere SVID und POSIX.

UNIX/Linux-Versionen in diesem Buch

Die Darstellung der UNIX-Kern-Strukturen in diesem Buch berücksichtigt zwei Implementationen, nämlich den „klassischen“ System-V-Kern, wie er in [Bach86] ausführlich diskutiert wird, und den Linux-Kern 2.x, dessen Quellcode frei eingesehen werden kann und der u.a. auch in [SiGa98], [Her99] und [Rus99] beschrieben wird.

Die Beschreibung der Programmierschnittstelle orientiert sich an der System V Interface Definition SVID. Sie wird (zumindest in ihren wesentlichen Bestandteilen) von modernen UNIX-Versionen unterstützt – insbesondere auch von Linux. Für den POSIX-Standard wird auf die weiterführende Literatur verwiesen – z.B. [Lew91] und [Gall95].

2.2 Eigenschaften und Struktur

2.2.1 Allgemeine Eigenschaften

UNIX ist ein Betriebssystem, das primär für Minicomputer und später Workstations entwickelt wurde. Mittlerweile wird UNIX aber auch, in Gestalt von Linux und FreeBSD, zunehmend auf PCs eingesetzt, und man findet es auch auf Großrechnern.

Die vorteilhaften Eigenschaften von UNIX / Linux lassen sich schlagwortartig folgendermaßen aufzählen:

- Zumindest aus Sicht des Benutzers und des Systemprogrammierers weist UNIX / Linux eine klare Strukturierung auf. So besitzt es insbesondere einen abgeschlossenen Kern mit einer wohldefinierten Schnittstelle.
- Die Kommandosprache ist umfangreich und flexibel. Sie ermöglicht unter anderem die Bildung von komplexeren Kommandos durch Verknüpfung einfacher Bausteine.
- Mehrbenutzer- und Mehrprogrammbetrieb werden unterstützt.
- Dialogbetrieb und die Ausführung von Batchprozessen „im Hintergrund“ sind gleichzeitig möglich.
- Das wohlstrukturierte „hierarchische“ Dateisystem erlaubt eine einheitliche Sicht auf Dateien und Ein-/Ausgabe-Geräte.
- Es existieren zahlreiche Dienstprogramme.
- Die Portabilität zwischen verschiedenen Hardwareplattformen ist gut, da Kern und Dienstprogramme größtenteils in C geschrieben sind.

UNIX hatte und hat aber auch einige Nachteile, so zum Beispiel:

- UNIX bot ursprünglich keine Unterstützung für Realzeitbetrieb. Es gibt allerdings mittlerweile den POSIX-1003.1b-Standard für UNIX-Realzeiterweiterungen (früher POSIX.4 genannt) [Gall95], und die UNIX-Implementationen verschiedener Hersteller enthalten ebenfalls entsprechende Konzepte.
- Spezielle Dateizugriffsverfahren (z.B. indexsequentieller Zugriff, Zugriff mit Schlüsselwort) fehlen.
- UNIX ist in einigen Bereichen dem Benutzer gegenüber recht „schweigsam“, insbesondere was Fehlermeldungen betrifft.

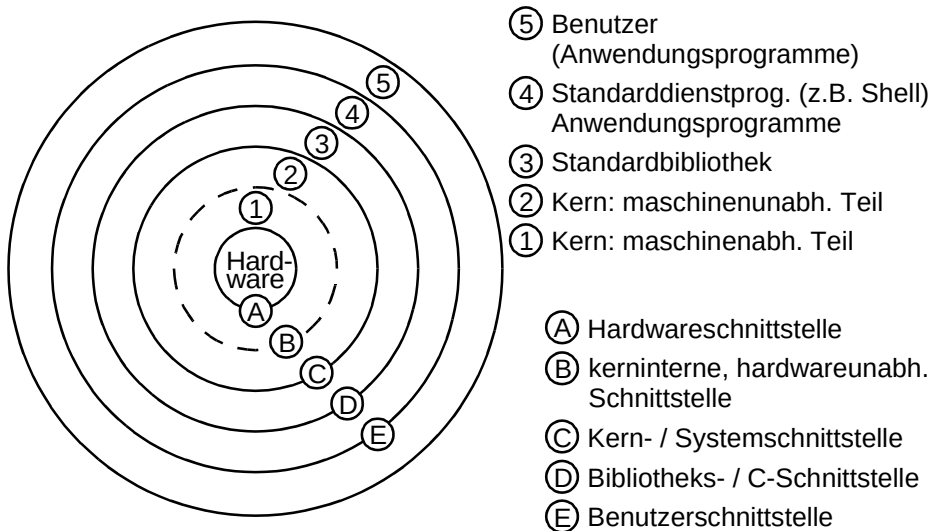


Abb. 2.2 allgemeine Schalenstruktur von UNIX und Linux

2.2.2 Die Schalenstruktur

Da UNIX einen monolithischen Kern besitzt, kann man es nicht als hierarchisches Betriebssystem im Sinne von Abschnitt 1.3.1 bezeichnen. Dies gilt auch für Linux. Trotzdem lassen sich mehrere „Schalen“ unterscheiden (siehe Abb. 2.2):

Unmittelbar oberhalb der Hardware befindet sich der *maschinenabhängige Teil des Kerns*. Er ist in C und Assembler geschrieben und kann daher nicht unmittelbar zwischen verschiedenen Hardwareplattformen portiert werden. Zu seinen Aufgaben gehören beispielsweise die Verwaltung von Signalen der Hardware und der Aufruf gerätespezifischer Ein-/Ausgabe-Funktionen. Der maschinenabhängige Teil bietet, innerhalb des Kerns, eine hardwareunabhängige Schnittstelle, auf die der maschinenunabhängige Teil des Kerns aufsetzt.

Der *maschinenunabhängige Teil des Kerns* ist ausschließlich in C geschrieben und daher portabel, d.h. zwischen unterschiedlichen Rechnern übertragbar. Er realisiert die Verwaltungsroutinen für das Prozess- und das Dateisystem sowie Ein-/Ausgabe-Mechanismen höherer Stufe. Seine obere Schnittstelle, die *Systemschnittstelle*, definiert Funktionen, die von darüber liegender Software aufgerufen werden können – u.a. von Funktionen der „Standardbibliothek“.

Die *Standardbibliothek* ermöglicht es, Funktionen der Systemschnittstelle in C-Programme einzubinden. Eine solche gesonderte Schicht ist nötig, da Funktionen der Systemschnittstelle über einen so genannten Trap-Mechanismus aufgerufen werden, der nicht unmittelbar in C-Programmen benutzt werden kann. Wir werden auf diesen Mechanismus im nächsten Abschnitt noch genauer eingehen. Die obere Schnittstelle dieser Schicht, die *Bibliotheksschnittstelle* oder auch *C-Schnittstelle*, stellt eine Anzahl von C-Funktionsprototypen zur Verfügung, die in C-Programmen verwendet werden können.

Auf der C-Schnittstelle setzen u.a. *Standardprogramme* auf, wie z.B. Editoren, Textverarbeitungsprogramme, Compiler usw. Zu diesen Standardprogrammen gehört auch die *Shell*. Sie ist der „Kommandointerpretierer“ für Dialogsitzungen und Batchdateien, nimmt also Kommandos z.B. des Benutzers entgegen und steuert ihre Ausführung. Die obere Schnittstelle der Shell ist die *Benutzerschnittstelle*, über die der Anwender das System bedienen kann.

Am Rande sei kurz angemerkt, dass bereits MS-DOS aus mehreren Schichten bestand, die ähnlich wie die hier dargestellten aussehen: Das *BIOS* (Basic Input Output System) implementiert hardwareabhängige Treiberfunktionen (= Steuerungsfunktionen) für die einzelnen Hardwarekomponenten, der Kern enthält Funktionen zur Verwaltung des Speichers, der Programmausführung und des Dateisystems, und die *Shell* realisiert die Benutzerschnittstelle. Wie Abbildung 1.10 zeigte, sind Windows NT und Windows 2000 ebenfalls in mehrere Schichten gegliedert.

In UNIX und Linux existieren also, wie in vielen anderen Betriebssystemen auch, mehrere Schichten und Schnittstellen. Damit kann man hier mindestens drei verschiedene Sichten annehmen, nämlich

- die *Benutzersicht* an der Benutzerschnittstelle,
- die *Sicht des Systemprogrammierers* an der C-Schnittstelle oder Systemschnittstelle und
- die *interne Sicht des Kerns*, d.h. die Sicht auf die betriebssysteminterne Implementierung der Funktionen und Datenstrukturen des Kerns.

Wir werden die Konzepte der UNIX/Linux-Systemschnittstelle und des UNIX- bzw. Linux-Kerns zunächst in ihren Grundlagen besprechen und dann in den nachfolgenden Kapiteln vertiefen. Zudem werden wir im Anhang die wichtigsten Schnittstellenfunktionen auflisten. Auf die Benutzerschnittstelle werden wir, relativ knapp, in Unterkapitel 2.3 eingehen.

2.2.3 Systemschnittstelle und Bibliotheksfunktionen

UNIX/ Linux bietet an seiner System- und der darauf aufsetzenden Bibliotheksschnittstelle zahlreiche Funktionen, die zur Steuerung und Verwaltung von Prozessen, Speicherbereichen, Dateien und Ein-/Ausgabegeräten sowie zur Erledigung weiterer Aufgaben benutzt werden können. Alle diese Funktionen werden auf einheitliche Weise aufgerufen und ausgeführt.

Ausführung von Systemfunktionen

Der Aufruf und die Ausführung einer Systemfunktion umfassen die folgende Schritte (siehe Abb. 2.3): Beim Aufruf werden Funktionsparameter in Prozessorregistern oder im Funktionsaufrufstack abgelegt. Anschließend wird ein *Trap*-Assembler-Befehl aus-

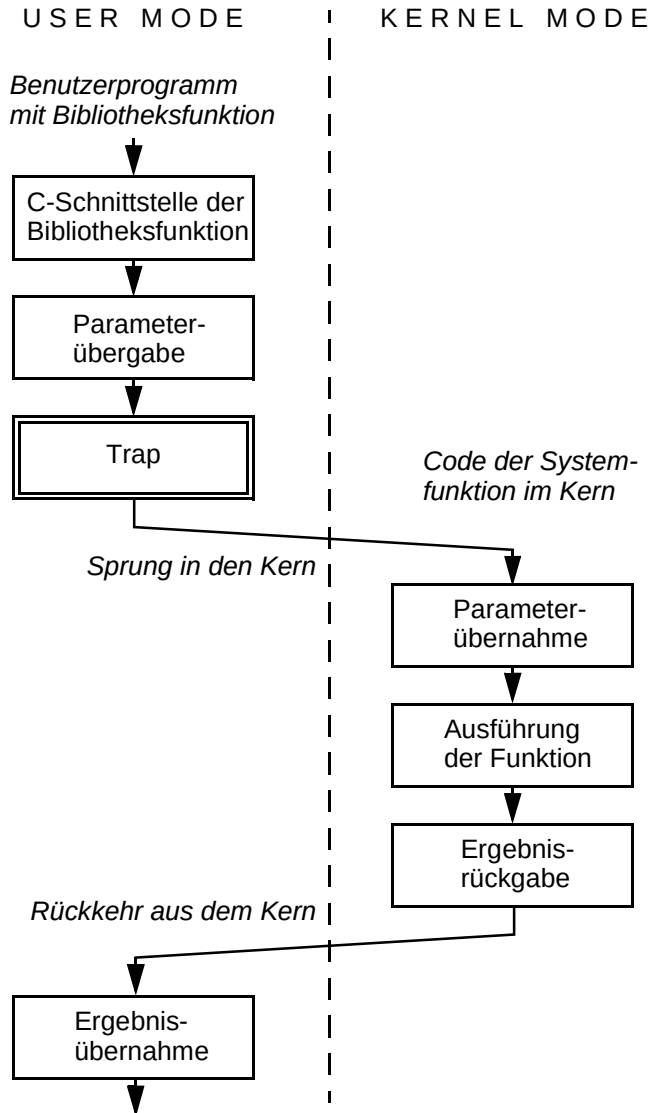


Abb. 2.3 Systemaufruf in UNIX

geführt. Dieser Befehl erzeugt ein Unterbrechungsereignis (*Interrupt*, siehe Unterkapitel 3.3) und zwingt damit die Programmausführung, an eine Stelle zu springen, die durch das Betriebssystem festgelegt wurde. Bei dieser Stelle handelt es sich um den Anfang des Codestücks im Betriebssystemkern, das die Systemfunktion realisiert. Gleichzeitig mit dem Sprung in die Funktion wird der Ausführungsmodus vom User Mode in den Kernel Mode umgeschaltet. Da diese beiden Schritte (Umschalten des Modus und Sprung in den vorgegebenen Code des Kerns) nur gemeinsam ausgeführt werden kön-

nen, ist sichergestellt, dass kein Prozess sich durch Umschalten in den Kernel Mode Privilegien verschaffen und anschließend möglicherweise „böartigen“ benutzerdefinierten Code ausführen kann.

Die Kern-Funktion holt die Parameter aus den Registern oder vom Stack, führt ihren Code aus und schreibt eventuelle Rückgabewerte in die Register oder auf den Stack. Anschließend wird in das aufrufende Programm zurückgesprungen, wobei der Ausführungsmodus in den User Mode zurückwechselt. Das aufrufende Programm kann dann die Resultate abholen.

Einbindung in C-Programme

Systemfunktionen, die nur über den Trap-Mechanismus erreichbar sind, können nicht unmittelbar in C-Programme eingebunden werden, da es in C keine Trap-Befehle gibt. Zur Anpassung dieser Funktionen an die „C-Welt“ stellt UNIX / Linux eine Sammlung von *Bibliotheksfunktionen* zur Verfügung. Diese Funktionen bieten eine C-Funktionschnittstelle, so dass sie aus C-Programmen heraus aufgerufen werden können; ihre Körper sind aber (zumindest teilweise) in Assembler-Sprache geschrieben, so dass sie ihrerseits durch einen Trap-Befehl Systemfunktionen aufrufen können. Jede Bibliotheksfunktion ist genau einer Systemfunktion zugeordnet, aber es kann für jede Systemfunktion mehrere Bibliotheksfunktionen geben.

Die UNIX/Linux-Verzeichnisse `/lib` und `/usr/lib` enthalten Dateien mit dem Objektcode der Bibliotheksfunktionen, der bei der Übersetzung eines Benutzerprogramms automatisch hinzugebunden wird. Entsprechende Header-Dateien findet man in den Verzeichnissen `/usr/include` und `/usr/include/sys`. Sie speichern die erforderlichen Typdefinitionen, Konstanten und zum Teil auch Funktionsprototypen und können durch den `#include`-Befehl in C-Programme eingefügt werden. Viele Funktionsprototypen sind dem C-Compiler sogar standardmäßig bekannt, so dass für sie kein `#include` erforderlich ist.

Aus Sicht des C-Programmierers können also Systemfunktionen über die Bibliothek wie normale C-Funktionen benutzt werden: Sie werden mit Hilfe von `#include` eingebunden, durch einen C-Funktionsaufruf ausgeführt und liefern anschließend im Allgemeinen einen Wert größer oder gleich 0 bei fehlerfreier Ausführung und eine -1 im Fehlerfall zurück. Ein spezifischer Fehlercode ist in der Variable `errno` gespeichert (Details siehe Einleitung des Anhangs).

2.3 Benutzersicht

Da der Schwerpunkt dieses Buchs nicht auf der Benutzeroberfläche eines Betriebssystems liegt, geben wir hier nur eine kurze Übersicht über die UNIX/Linux-Benutzerschnittstelle. Für nähere Informationen verweisen wir auf die einschlägige Literatur – z.B. [GuOb95] oder [Her98].

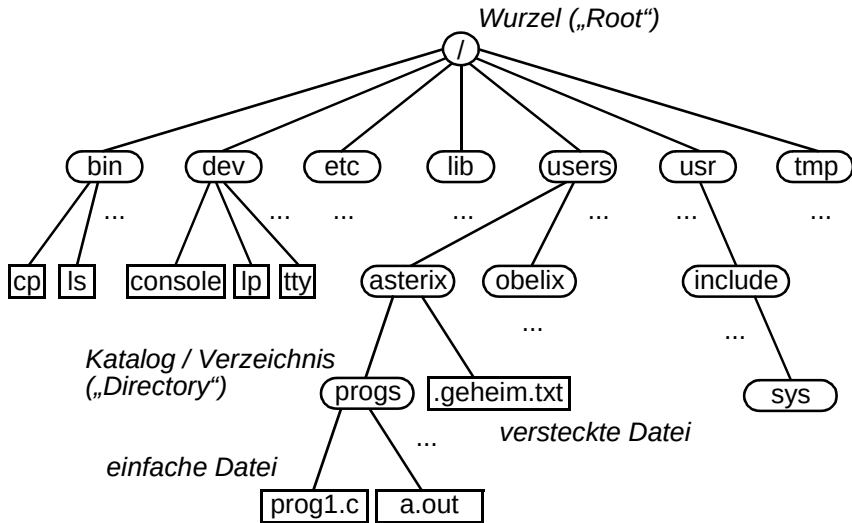


Abb. 2.4 Dateisystem in UNIX / Linux (typische Form)

2.3.1 Dateisystem und E/A-Geräte

Dateien dienen, in UNIX / Linux wie in allen anderen Betriebssystemen, zur längerfristigen Speicherung von Informationen. Generell werden zwei Arten von Dateien unterschieden: Erstens gibt es *einfache Dateien*, die Daten oder Programme enthalten. Zweitens gibt es *Verzeichnisse (Kataloge, Directories)*, in denen Dateien und weitere Verzeichnisse stehen.

Dateibaum

Da in Verzeichnissen weitere Verzeichnisse eingetragen sein können, ergibt sich ein *hierarchisches Dateisystem*, das durch die bekannte Baumstruktur dargestellt wird (siehe Abb. 2.4). Die Knoten in einem solchen *Dateibaum* entsprechen den einfachen Dateien (in der Abbildung: Rechtecke) und den Verzeichnissen (in der Abbildung: Kreise oder Ovale) des Dateisystems. Hat ein Knoten Söhne (= Nachfolger im Baum), so handelt es sich bei der zugehörigen Datei um ein Verzeichnis, in dem genau die Dateien verzeichnet sind, die diesen Söhnen entsprechen. Eine einfache Datei kann nur ein Blattknoten (= Knoten ohne Nachfolger) sein. Verzeichnisse, die keine Einträge haben, sind ebenfalls Blattknoten. Wenn wir im Folgenden von Dateien sprechen, so sind damit einfache Dateien oder Verzeichnisse gemeint.

Der Dateibaum hat eine eindeutige *Wurzel* (engl. *root*), die dem *Wurzelverzeichnis* (engl. *root directory*) entspricht. Typischerweise enthält das Wurzelverzeichnis die folgenden Unterverzeichnisse:

- `bin`: „Binärdateien“ für Benutzerkommandos, wie z.B. `cp` oder `ls` (siehe unten).

- `dev`: Gerädateien. *Ein-/Ausgabegeräte (E/A-Geräte)* können mit denselben Operationen wie „normale“ Dateien angesprochen werden. Ihnen entsprechen Einträge im Unterverzeichnis `/dev` (`dev` = devices). `tty` ist beispielsweise das Terminal, von dem aus das aktuelle Programm ausgeführt wird, `lp` der Drucker und `console` das Terminal, von dem aus der Systemverwalter das System steuert.
- `etc`: Dateien zur Systemadministration.
- `lib`: Programmbibliotheken.
- `users`: private Verzeichnisse der Benutzer – manchmal auch `home` o.ä. genannt.
- `usr/include`, `/usr/include/sys`: Headerdateien zur C-Programmierung.
- `tmp`: temporäre Dateien.

Je nach System können weitere Unterverzeichnisse vorhanden sein. Auf Linux-Systemen findet man z.B. zusätzlich das Verzeichnis `proc`, in dem Informationen über die gerade laufenden Aktivitäten abgefragt werden können.

Dateinamen und -attribute

Dateien haben *Namen*. Ursprünglich war die Länge eines UNIX-Dateinamens auf 14 Zeichen beschränkt; seit BSD 4.2 unterstützen viele Systeme aber Dateinamen bis zu 255 Zeichen. Eine Datei kann „versteckt“ werden („*Hidden File*“), indem man ihr einen Namen gibt, der mit einem Punkt beginnt; sie wird dann beim Auflisten des Verzeichnisinhalts nur unter bestimmten Bedingungen angezeigt. Eine Dateinamenserweiterung (z.B. `.txt`, `.c`) ist optional, es können mehrere Erweiterungen hintereinander gehängt werden, und ihre Länge ist nur durch den oben genannten Zahlenwert begrenzt.

Eine Datei hat neben ihrem Namen eine Reihe von *Attributen*, die ihre Eigenschaften angeben. Mit dem Kommando `ls -l` können die wichtigsten Attributwerte der Dateien ausgegeben werden. Eine typische Ausgabe dieses Kommandos lautet beispielsweise

```
-rwxr-xr-- 1 asterix comic 1024 Nov 11 11:11 prog1.c
```

Die erste Position ganz links gibt die Art der Datei an (u.a. `-` für eine einfache Datei, `d` für ein Verzeichnis, `b` oder `c` für ein Gerät und `l` für ein Link – siehe unten). Es folgen neun Angaben, die (jeweils in Dreiergruppen) die Zugriffsrechte des Dateibesitzers, der Benutzergruppe, der die Datei zugeordnet ist, und aller anderen Benutzer beschreiben; hier gibt es u.a. `r` für Lesen („Read“), `w` für Schreiben („Write“) und `x` für Ausführen („eXecute“). Die Zahl rechts davon gibt die Anzahl der Links auf die Datei (siehe unten) an. Anschließend werden der Name des Dateibesitzers und der Benutzergruppe genannt, der die Datei zugeordnet ist. Rechts daneben stehen die Größe der Datei in Bytes, Datum und Uhrzeit der letzten Änderung sowie der Dateiname.

Eine Datei wird durch Angabe des *Pfads* identifiziert, der im Dateibaum von der Wurzel zum Dateiknoten verläuft. Man nennt diese Art der Identifikation *absolute Identifikation*. Beispielsweise würde im Dateibaum aus Abbildung 2.4 die Datei mit dem Namen `prog1.c` durch den Pfad `/users/asterix/progs/prog1.c` bezeichnet. Das Symbol `/` dient also als Identifikator für die Wurzel und als Trennzeichen zwischen den Verzeichnis- und Dateinamen.

Da es mühsam ist, stets den ganzen Pfadnamen anzugeben, gibt es die folgende Alternative: Jeder Prozess, und damit jeder Benutzer, ist zu jedem Zeitpunkt in einem so genannten *Arbeitsverzeichnis* (engl. *working directory*) aktiv. Das Arbeitsverzeichnis kann durch das `cd`-Kommando dynamisch geändert werden. Dateien können durch eine Pfadangabe *relativ* zum Arbeitsverzeichnis identifiziert werden. Ist beispielsweise (siehe Abb. 2.4) `/users/asterix` momentan das Arbeitsverzeichnis, so bezeichnet `progs/prog1.c` dieselbe Datei wie im vorherigen Beispiel. Mit `..` kann man dabei auch zum Vaterverzeichnis (und iterativ zu noch höheren Verzeichnissen im Baum) übergehen, also z.B. mit `..` zum Verzeichnis `users` und mit `../lib` zum Verzeichnis `lib`. Das Arbeitsverzeichnis selbst kann mit einem `.` identifiziert werden.

Jeder Benutzer hat ein Heimatverzeichnis (engl. *home directory*), das durch die Tilde `~` bezeichnet wird. Für den Benutzer `asterix` bezeichnet z.B. `~/progs/prog1.c` die Datei `/users/asterix/progs/prog1.c`, sofern `/users/asterix` als sein Home Directory vereinbart ist.

Dateinamen müssen nicht vollständig angegeben werden, sondern es können *Platzhalter* (engl. *wildcards*) verwendet werden. `?` ist dabei der Platzhalter für ein beliebiges Zeichen (z.B. `asterix/prog?.c`), `*` für eine beliebige Zeichenfolge (z.B. `asterix/*`).

Zu den wichtigsten Befehlen zur Benutzung des Dateisystems gehören `ls` zum Auflisten eines Verzeichnisses, `cp` zum Kopieren, `rm` zum Löschen einer Datei und `mv` zum Umbenennen einer Datei und `/` oder zum Transport in ein anderes Verzeichnis sowie `mkdir` und `rmdir` zum Erzeugen bzw. Löschen von Verzeichnissen.

Schließlich ist noch zu erwähnen, dass es eine Möglichkeit gibt, die strenge Baumstruktur „aufzuweichen“: Für ein Verzeichnis, das in einem Oberverzeichnis steht, dürfen nämlich weitere Verweise (engl. *links*), die ebenfalls auf dieses Verzeichnis zeigen, in andere (Ober-)Verzeichnisse aufgenommen werden. Hierzu dient das Kommando `ln`. So wird beispielsweise durch `ln /users/asterix/progs /users/obelix/programme` das Verzeichnis `progs` zusätzlich im Verzeichnis `/users/obelix` unter dem Namen `programme` zugreifbar gemacht.

2.3.2 Shell und Benutzerkommandos

Um auf einem UNIX/Linux-System arbeiten zu können, muss sich ein Benutzer zunächst „*einloggen*“. Er gibt dazu seinen *Benutzernamen* und sein *Passwort* an, die vom Betriebssystem überprüft werden. Diese Prüfung ist entscheidend für die Systemsicherheit, da mit dem Namen Ausführungs- und Zugriffsrechte verbunden sind. Im System gibt es zumindest den Benutzer „*root*“ – den Systemverwalter, der maximale Rechte hat und seinerseits weitere *Benutzerkonten* (engl. *accounts*) einrichten kann.

Shell

Für den neu eingeloggtten Benutzer wird ein *Shell-Prozess* gestartet, der sich mit einer *Eingabeaufforderung (Prompt)* auf dem Bildschirm meldet. Der Benutzer kann nun ein Kommando eingeben, das er mit der Return-Taste abschließt. Die Shell analysiert das Kommando und startet seine Ausführung.

Für UNIX-Systeme existieren mehrere verschiedene Shell-Programme: Neben der ursprünglichen *Bourne-Shell* gibt es die *Korn-Shell*, die zur Bourne-Shell aufwärtskompatibel ist. In ihr kann man Kommandozeilen editieren, zuvor eingegebene Kommandos durch den so genannten History-Mechanismus abfragen, neue Kommandonamen mit Hilfe des Alias-Mechanismus vereinbaren und Prozesse zwischen Vorder- und Hintergrundausführung (siehe unten) verschieben. Auch die *C-Shell*, die nicht aufwärtskompatibel zur Bourne-Shell ist, kennt u.a. den History- und den Alias-Mechanismus sowie das Verschieben von Prozessen zwischen Vorder- und Hintergrund. Die *bash-Shell* („Bourne-Again-Shell“) für Linux und FreeBSD ist kompatibel zur Bourne-Shell und teilweise auch zur Korn-Shell.

Jede Shell ist in einer *Umgebung* aktiv, die Informationen für die Kommandoausführung enthält. Hierzu gehören beispielsweise das Arbeitsverzeichnis, vorgegebene Suchpfade für Kommandodateien und Aliasdefinitionen.

Benutzerkommandos

Allgemein sieht die Syntax eines UNIX/Linux-Kommandos, das von einer Shell akzeptiert wird, folgendermaßen aus:

```
kommando_name {-option} {argument} [&]
```

Im Anschluss an den Kommandonamen können Optionen angegeben werden, die jede einzeln mit einem - eingeleitet werden. Es folgen die erforderlichen Parameter (auch „Argumente“ genannt). Beispielsweise werden durch `ls -l dat* prog*` alle Dateien, deren Name mit `dat` oder `prog` beginnt, mit ihren Attributwerten aufgelistet. Das Zeichen & dient zur Erzeugung von Hintergrundprozessen (siehe unten). In UNIX/Linux existieren Kommandos

- zur *Dateiverwaltung* (siehe oben),
- zur *Prozessverwaltung* (z.B. `ps` zur Anzeige der gerade laufenden Prozesse und `kill` zum Löschen eines Prozesses),
- zur *Sitzungsverwaltung* (z.B. `who` zur Anzeige der gerade aktiven Benutzer und `exit` zum Ausloggen),
- zur *Ausgabe* von Dateiinhalten auf Geräte (z.B. `cat` und `more` zur Anzeige des Inhalts von Dateien auf dem Bildschirm und `lp` zum Drucken einer Datei) sowie
- zum Aufruf von *Dienstprogrammen* (siehe Abschnitt 2.3.3).

Eine Erörterung aller Kommandos würde den Rahmen dieses Buchs sprengen. Wir verweisen hierfür auf die weiterführende Literatur (z.B. [GuOb95] oder [Her98]) oder das UNIX-Kommando `man`: `man String` zeigt die Dokumentation zum UNIX-Benutzerkommando *String* an (z.B. `man ps`).

Die Ausführung eines Kommandos geschieht, wie die Ausführung sämtlicher Benutzer- und Systemaktivitäten, durch einen *Prozess*. Man unterscheidet dabei zwischen Vordergrund- und Hintergrundprozessen: *Vordergrundprozesse* starten üblicherweise nach Eingabe eines Kommandos und blockieren weitere Shell-Eingaben bis zum Ende der Ausführung, d.h. es kann bis dahin keine Ausführung eines weiteren Kommandos gestartet werden. *Hintergrundprozesse* (*Batchprozesse*, in manchen Fällen auch *Daemons* genannt) dienen vornehmlich für Aktivitäten des Systems, die nebenläufig zu den interaktiven Benutzerprogrammen ablaufen, wie z.B. das Warten auf eintreffende Mails, die Verwaltung des Druckers usw. Daneben ist es aber auch möglich, Benutzerkommandos „im Hintergrund“ auszuführen. Das ist beispielsweise sinnvoll, um während einer länger andauernden Ausführung (z.B. eines längeren Übersetzungslaufs) weitere Kommandos eingeben zu können. Ein Hintergrundprozess wird gestartet, indem man an das Kommando das Zeichen `&` anfügt.

Im einfachsten Fall werden also bei einer Kommando-Ausführung die folgenden Schritte abgearbeitet:

- Der Benutzer gibt ein Kommando über die Tastatur ein.
- Die Shell analysiert das Kommando.
- Die Shell erzeugt einen Prozess.
- Der Prozess führt das Kommando aus.
- Der Prozess liefert das Ergebnis an die Shell zurück und terminiert (= „stirbt“).
- Die Shell gibt das Ergebnis an den Benutzer weiter.
- Die Shell gibt ein neues Bereitzeichen aus.

Verknüpfungen von Kommandos

UNIX/Linux-Kommandos können auf verschiedene Arten miteinander kombiniert werden. So lassen sich mit dem Trennzeichen `;` mehrere Kommandos *sequentialisieren*, also hintereinander ausführen. Beispielsweise listet `ls neu*`; `ls alt*` zuerst alle Dateien auf, die mit `neu` beginnen, und anschließend alle Dateien, die mit `alt` beginnen.

Ganze Kommandofolgen können in *Kommandodateien* (*Shellscripts*) abgelegt werden. Nach Eingabe des Namens der Datei wird die Kommandofolge ausgeführt. Dabei ist auch die Übergabe von Parametern möglich. Sie werden innerhalb der Kommandodatei mit `$i` angesprochen; `i` ist dabei eine Zahl, die die Position des Parameters in der

Liste der übergebenen Parameter angibt. Enthält beispielsweise die Datei mit dem Namen `loesche` die Kommandofolge `rm $1.c; rm $1.o; rm $1.exe`, so löscht die Eingabe von `loesche prog` die Dateien `prog.c`, `prog.o` und `prog.exe`. Es wird also `$i` jeweils durch den `i`-ten Parameter textuell ersetzt.

Durch *Fließbandverarbeitung (Piping)* ist es möglich, die Standardausgabe eines Kommandos direkt in die Standardeingabe eines zweiten Kommandos zu leiten. Der entsprechende Operator ist das Zeichen `|`. Beispielsweise wird durch `ls -l | more` die Ausgabe des `ls`-Kommandos durch `more` seitenweise auf dem Bildschirm angezeigt.

Umlenkungen

Die Resultate eines Kommando werden im Allgemeinen auf den Bildschirm ausgegeben, seine Eingabewerte kommen im Allgemeinen von der Tastatur. Mit anderen Worten: Die *Standardeingabe* für Kommandos ist die Tastatur, die *Standardausgabe* der Bildschirm. Man kann jedoch Standardein- und -ausgabe auch *umlenken*: Z.B. listet das Kommando `ls -l > verzeichnis` die Dateien des Arbeitsverzeichnisses nicht auf dem Bildschirm auf, sondern schreibt die Liste in die Datei `verzeichnis`. Die Eingabe wird mit dem Zeichen `<` umgelenkt.

2.3.3 Dienstprogramme

UNIX/Linux bietet zudem eine Reihe von Dienstprogrammen. Standardmäßig ist der *ASCII-Editor* `vi` vorhanden, der zwar (zumindest für den Anfänger) etwas mühsam zu bedienen ist, aber mehrere Vorteile hat: Erstens ist er auf jedem UNIX/Linux-System verfügbar. Zweitens kann `vi` als tastaturorientierter Editor auch dann benutzt werden, wenn man über das lokale Terminal auf einem entfernt liegenden System arbeitet – bei mausgestützten Werkzeugen ist dies oft nicht oder nur mit größerem Aufwand möglich. Drittens hat `vi` eine recht kompakte Kommandostruktur und erlaubt daher häufig ein rascheres Arbeiten als mit mausorientierten Editoren. `vi`-Kenntnisse gehören daher zur Grundausstattung eines UNIX/Linux-Benutzers. Optional sind weitere Editoren vorhanden, wie z.B. *emacs*.

Programmierwerkzeuge

Zur Programmierung können Compiler, Linker und Debugger benutzt werden. Das wichtigste Dienstprogramm ist hier der *C-Compiler* und *-Linker*, der standardmäßig mit `cc` aufgerufen wird (oder im Fall des GNU-C-Compilers mit `gcc`). Er übersetzt ein C-Programm über mehrere Zwischenstufen in ein ausführbares Programm.

Abbildung 2.5 illustriert die Vorgehensweise von `cc`: `cc` akzeptiert als Eingabe eine oder mehrere Programmdateien, die entweder C-Quellcode enthalten können oder Code, der bereits in eine Zwischenstufe übersetzt wurde. Die Art des zu übersetzenden Programms wird durch das Suffix (= die Dateinamenserweiterung) der Eingabedatei

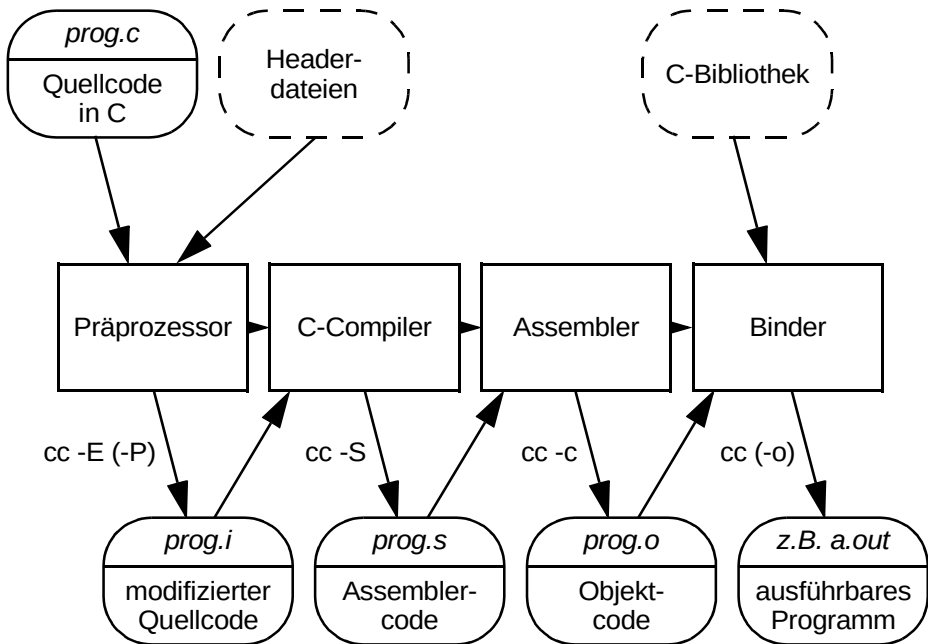


Abb. 2.5 Schritte des UNIX-C-Compilers

identifiziert: `.c` bezeichnet ein Programm in Quellcode, `.i` ein Programm in modifiziertem Quellcode (erzeugt durch den Präprozessor – siehe unten), `.s` ein Programm in Assemblercode und `.o` oder `.a` ein Programm in Objektcode.

`cc` besteht aus mehreren Komponenten, die je einen Übersetzungsschritt durchführen und Code in Zwischensprache oder schließlich ein lauffähiges Programm liefern. Diese Komponenten sind zwar meist in eigenen ausführbaren Dateien realisiert, sollten aber nur über das `cc`-Kommando mit entsprechenden Optionen aufgerufen werden:

- Der *Präprozessor* bearbeitet `#include`-Anweisungen, Anweisungen zur bedingten Übersetzung sowie Makros und liefert ein modifiziertes Programm in Quellcode. Er wird mit den Optionen `cc -E -P` (Resultat: modifizierter reiner C-Quellcode) oder `cc -E` (Resultat: modifizierter C-Quellcode plus Zusatzinformationen) aufgerufen. Das standardmäßige Suffix einer durch den Präprozessor erzeugten Datei ist `.i`; in manchen Fällen wird jedoch keine solche Datei erzeugt, sondern das Resultat wird auf die Standardausgabe geschrieben.
- Der eigentliche *C-Compiler* wird mit der Option `cc -S` aufgerufen und erzeugt Assemblercode (Dateisuffix `.s`). Neben Dateien, die der Präprozessor erzeugt hat, können auch Quellcodedateien mit Präprozessoranweisungen als Eingabe dienen; `cc` ruft dann zuvor automatisch den Präprozessor auf. Überhaupt gilt allgemein, dass beim Aufruf einer Übersetzungsstufe automatisch alle erforderlichen vorherigen Stufen ausgeführt werden, falls nötig.

- Der *Assembler* wird über die Option `cc -c` angesprochen und erzeugt Programme in Objektcode (Dateisuffix `.o`).
- Der *Link Editor* akzeptiert Dateien mit Objektcode und Bibliotheksdateien („Archive“ = Dateien mit vorübersetzten Funktionen) als Eingabe und bindet sie zu einem ausführbaren Programm zusammen. Sofern nicht mit der Option `-o` ein Name für die ausführbare Programmdatei vereinbart wurde, heißt sie standardmäßig `a.out`.

In den meisten Fällen werden die Stufen nicht einzeln aufgerufen, sondern es wird aus einem Quellprogramm unmittelbar ausführbarer Code generiert. Hierzu wird `cc` ohne Optionen aufgerufen. Beispielsweise erzeugt der Aufruf `cc test.c` aus `test.c` direkt ein ausführbares Programm `a.out`.

`cc` kann, wie gesagt, auch mehrere Teilprogramme zu einem lauffähigen Programm zusammenbinden: So generiert beispielsweise `cc -o ausf part1.c part2.c` aus den beiden Teilprogrammen `part1.c` und `part2.c` ausführbaren Code in der Datei `ausf`. Alle oder einige der Teilprogramme können dabei bereits in vorübersetzter Form, also in Objektcode vorliegen. Zum Beispiel könnte zunächst mit `cc -c part1.c` die Objektcodedatei `part1.o` erzeugt werden und die endgültige Übersetzung anschließend mit `cc -o ausf part1.o part2.c` stattfinden.

Ein mächtiges Kommando zur Übersetzung von Programmsystemen ist *make*. Es stützt sich auf so genannte *makefiles*, die vom Benutzer definiert werden können und die durchzuführenden Übersetzungsschritte sowie Abhängigkeiten zwischen den Programmdateien angeben.

Neben dem C-Übersetzungssystem gibt es zahlreiche weitere Dienstprogramme. So können beispielsweise die Programme `grep`, `sed` und `awk` Textstücke in Dateien suchen und Dateiinhalte verändern. `lex` und `yacc` analysieren die Struktur von Zeichenketten. Außerdem existieren noch Programme zur Buchführung über Aktivitäten und zur Abrechnung.

2.4 Übungsaufgaben

1. Wissensfragen

- a.) In welchem Zusammenhang stehen UNIX und die Programmiersprache C?
- b.) Wofür steht die Abkürzung SVID?
- c.) Was ist der Unterschied zwischen der UNIX-Benutzerschnittstelle und der UNIX-C-Schnittstelle (Bibliotheksschnittstelle)? Wie heißt der Teil von UNIX, der zwischen diesen beiden Schnittstellen liegt?
- d.) Was geschieht bei einem UNIX-Systemaufruf?
- e.) An welcher Stelle sind die Ein-/Ausgabegeräte verzeichnet, die an ein UNIX-System angeschlossen sind?
- f.) Welche Bedeutung für die Shell haben die Zeichen `*`, `|`, `;`, `<`, `>`?

2. UNIX-Aufbau

Zeichnen Sie eine Skizze, die den geschichteten Aufbau von UNIX zeigt. Geben Sie dabei die Namen der Schichten und ihrer Schnittstellen an.

3. UNIX-Dateisystem

- a.) Welche Rechte können für Zugriffe auf eine UNIX-Datei vergeben werden?
- b.) Was bedeutet die folgende Bildschirmausgabe?

```
-rw-r--r-- 4 stud1 studis 511722 Dec 17 10:24 dipl_arb
```

- c.) Warum sollte man mit der Verwendung des Kommandos `rm *` besonders vorsichtig sein? (*bitte nicht praktisch ausprobieren!*)

4. UNIX-C-Compiler

- a.) Warum macht es Sinn, manche C-Programme nicht sofort in ausführbare Maschinenprogramme zu übersetzen, sondern zunächst Objektcodedateien zu generieren?
- b.) Wie kann man zu den Dateien `p1.c` und `p2.c` entsprechende Objektcodedateien erzeugen?
- c.) Wie kann man die Dateien `pa.c`, `pb.c` und `pc.o` zu einem ausführbaren Programm `p.exe` zusammenbinden?

5. UNIX-Shellscripts

- a.) Wozu dient das Shellscript mit dem folgenden Inhalt?

```
ls -l $1* | more; ls -l $1* > $2
```

- b.) Schreiben Sie ein Shellscript, das alle Dateien mit einem bestimmten Suffix (= Dateinamenserweiterung) aus dem aktuellen Arbeitsverzeichnis in ein Archivverzeichnis kopiert. Dazu soll zunächst das Archivverzeichnis als Unterverzeichnis des „Home Directory“ des aufrufenden Benutzers erzeugt werden. Das Suffix der Dateien und der Name des Archivverzeichnisses sollen beim Aufruf des Shellscripts als erster bzw. zweiter Parameter übergeben werden.

Hinweis: Der `cp`-Befehl lässt sich unter anderem in der Form `cp Dateiname Verzeichnisname` benutzen und kopiert so die genannte Datei in das genannte Verzeichnis. Es können hier auch Wildcards eingesetzt werden.

3 Grundlagen des Prozesskonzepts

In einem Multiprogramming-Computer fordern mehrere Benutzer gleichzeitig, dass Aktivitäten für sie durchgeführt werden. Außerdem sind weitere Aktivitäten zur Verwaltung des Systems erforderlich. Das Betriebssystem muss den Rechner so steuern, dass diese verschiedenen Anforderungen durch den real vorhandenen Einzelprozessor bedient werden. Die Grundlage hierfür ist das *Prozesskonzept*, das in diesem Kapitel besprochen wird.

3.1 Grundidee und Implementierungsaspekte

3.1.1 Der Prozessbegriff

Kapitel 1 gab schon eine grobe Vorstellung davon, was mit dem Begriff „*Prozess*“ gemeint ist: Ein Prozess entspricht einer Aktivität innerhalb des Systems. So wird in UNIX und Linux beispielsweise ein Prozess erzeugt, um ein Kommando oder eine Kommandofolge auszuführen. Im Folgenden soll der Prozessbegriff präzisiert werden.

Ein Analogiebeispiel

Wir betrachten zunächst ein Analogbeispiel: Eine Firma hat einen Schichtbetrieb eingeführt, um ihre Geräte besser auszulasten. Diese Neuerung betrifft auch die Buchhaltung, so dass jetzt je zwei Buchhalter abwechselnd denselben Schreibtisch mit denselben Arbeitsmitteln benutzen müssen. Ein Buchhalter verlässt nun pünktlich zu Schichtwechsel seinen Schreibtisch und unterbricht dabei seine Tätigkeit an einer beliebigen Stelle. Sein Kollege übernimmt dann Schreibtisch und Arbeitsmittel und nimmt seine eigene Tätigkeit auf, die nicht unbedingt etwas mit der Arbeit seines Vorgängers zu tun hat.

Es wird sehr bald deutlich, dass diese Vorgehensweise zu Problemen führt. Der erste Buchhalter bemerkt nämlich bei seiner Rückkehr zum folgenden Schichtwechsel, dass die Zwischenergebnisse aus seiner vorherigen Schicht verloren gegangen sind: Die Akten sind an völlig anderen Stellen aufgeschlagen, als er sie zurückgelassen hatte, und auch die Ergebnisse, die der Tischrechner für ihn ausgedruckt hatte, sind nicht mehr vorhanden. Stattdessen findet er die Resultate seines Kollegen vor. Buchhalter 1 kann also nicht unmittelbar dort fortsetzen, wo er aufgehört hatte, sondern muss sich seine alten Zwischenergebnisse wieder mühsam erarbeiten.

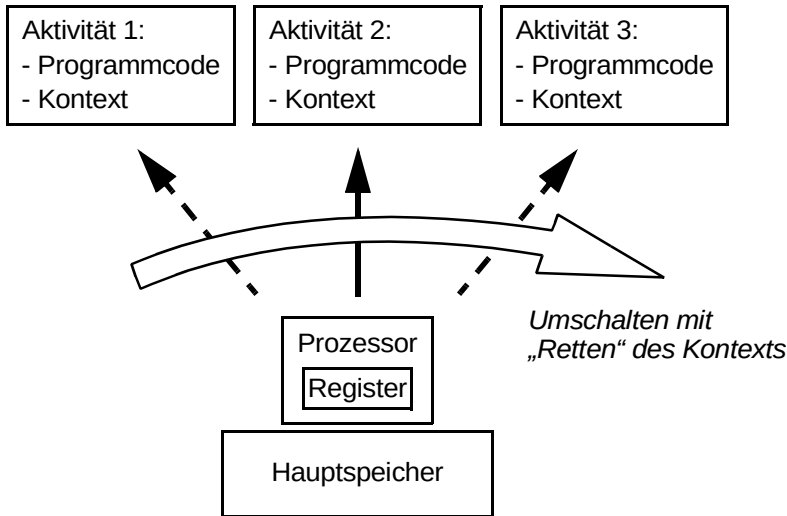


Abb. 3.1 Prozessor und Aktivitäten

Umgehend wird eine Unternehmensberatung beauftragt, das Problem zu lösen. Nach einiger Zeit liefert diese, nebst einer höheren Rechnung, ein umfangreiches Gutachten ab, das im Kern die folgende Vorgehensweise empfiehlt: Jeder Buchhalter soll vor Übergabe des Schreibtischs den aktuellen Zustand notieren – also insbesondere die zuletzt berechneten Resultate des Tischrechners festhalten, und aufschreiben, an welcher Stelle die einzelnen Akten aufgeschlagen waren. Mit diesen Informationen, die er sorgsam verwahren muss, soll er dann beim nächsten Schichtwechsel den Schreibtisch wieder in den Zustand zurückversetzen, in dem er ihn verlassen hatte.

Das Beispiel lässt sich unmittelbar auf die Verwaltung eines Ein-Prozessor-Computers übertragen: Der Schreibtisch mit den Arbeitsmitteln entspricht dem Prozessor mit den weiteren Rechnerbetriebsmitteln, und die Buchhalter sind die Aktivitäten, die diese Betriebsmittel benutzen wollen. Damit wird also klar unterschieden zwischen der real vorhandenen Hardware und den Aktivitäten, die darauf ablaufen.

Prozessor und Prozesse

Im Detail sieht das Szenario folgendermaßen aus (siehe Abb. 3.1): In Hardware existiert ein einzelner Prozessor mit Registern (= kleinen, schnell zugreifbaren Speicherstellen) und einem Hauptspeicher (Details hierzu in Kapitel 5). Auf diesem Prozessor sind mehrere *Aktivitäten* auszuführen, also Programme, die Daten verarbeiten. Die Aktivitäten befinden sich bei ihrer Ausführung in einem *Kontext*, der wesentlich durch den Inhalt der Prozessorregister und Daten im Hauptspeicher bestimmt wird. Jede Aktivität hat dabei ihren eigenen Kontext, der meist unabhängig von den Kontexten der anderen Aktivitäten ist.

Wie bei der Einführung des Begriffs „Multiprogramming“ in Abschnitt 1.2.3 besprochen, wird der Prozessor im raschen Wechsel (mehrmals pro Sekunde) zwischen den Aktivitäten hin- und hergeschaltet. Gibt eine Aktivität den Prozessor ab, so wird ihr Kontext „gerettet“, d.h. insbesondere der Inhalt der Register an einer sicheren Stelle zwischengespeichert und der Hauptspeicherbereich der Aktivität geeignet vor dem Überschreiben geschützt. Erhält die Aktivität den Prozessor erneut zugeteilt, so wird ihr alter Kontext „wiederhergestellt“, also insbesondere die Register mit den seinerzeit gespeicherten Werten geladen. Sie kann damit dort fortgesetzt werden, wo sie zuvor unterbrochen wurde.

Eine Aktivität wird im Betriebssystem durch einen *Prozess* (engl. *process* oder *task*) dargestellt. Der Prozessbegriff wird dabei wie folgt definiert:

- Ein Prozess ist eine (Software-)Einheit, die durch das Betriebssystem auf dem realen Prozessor zur Ausführung gebracht werden kann. Ein Prozess umfasst dabei seinen Programmcode und einen *Kontext*, in dem der Code ausgeführt wird.
- Zum Kontext eines Prozesses gehören die Registerinhalte des Prozessors, dem Prozess zugeordnete Bereiche des direkt zugreifbaren Speichers, durch den Prozess geöffnete Dateien, dem Prozess zugeordnete Peripheriegeräte und Verwaltungsinformationen über den Prozess.
- Ein Prozess befindet sich zu jedem Zeitpunkt in einem bestimmten *Zustand*. Der Zustand gibt an, ob der Prozess gerade auf dem Prozessor ausgeführt wird und ob er momentan überhaupt ausgeführt werden kann. Es finden dynamisch Zustandsübergänge, also Veränderungen des Prozesszustands statt.

Wichtig ist also auch der Zustand des Prozesses. Es existieren mindestens die folgenden Prozesszustände (siehe Abb. 3.2):

- „rechnend“ („running“): Der Prozess wird momentan auf dem Prozessor ausgeführt.
- „bereit“ („ready“): Der Prozess könnte unmittelbar weiter ausgeführt werden, wartet aber auf die Zuteilung des Prozessors.
- „blockiert“ („blocked“): Der Prozess kann momentan nicht weiter ausgeführt werden, da er auf das Eintreten eines Ereignisses wartet (z.B. auf die Fertigmeldung eines E/A-Geräts oder auf die Nachricht eines anderen Prozesses).

Abbildung 3.2 gibt zudem alle möglichen *Zustandsübergänge* an: Beim Umschalten des Prozessors zwischen Prozessen wechseln diese entsprechend zwischen den Zuständen „rechnend“ und „bereit“. Ein Prozess, der auf ein Ereignis warten möchte (z.B. auf das Eintreffen von Daten von der Festplatte), geht von „rechnend“ nach „blockiert“ über. Tritt das Ereignis dann ein, so wird der wartende Prozess von „blockiert“ nach „bereit“ überführt, da er nun weiterlaufen kann. Neu gestartete Prozesse beginnen ihre Existenz im Zustand „bereit“. Ein Prozess kann in jedem Zustand beendet werden (*terminieren*), wobei er das Diagramm verlässt.

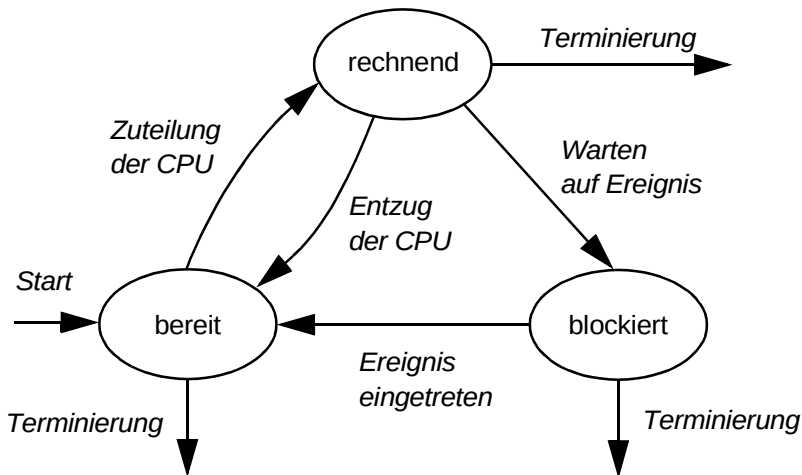


Abb. 3.2 allgemeines Zustandsdiagramm für Prozesse

Man kann Prozesse grob klassifizieren in *Benutzerprozesse* und *Systemprozesse*. Beispiele für Benutzerprozesse sind die Shell, ein Betriebssystemkommando oder ein Benutzerprogramm in Ausführung; Beispiele für Systemprozesse sind Ein-Ausgabe-Prozesse oder Mail-„Daemons“, d.h. Prozesse, die u.a. auf das Eintreffen von elektronischer Post warten.

Zusammenfassend lässt sich also Folgendes festhalten: Ein Prozess ist eine Aktivität, die auf dem Rechner durchgeföhrt wird – entweder unabhängig von anderen Aktivitäten oder in Kooperation mit ihnen. Er ist definiert durch seinen Programmtext und seinen Kontext. Das Betriebssystem stellt zusammen mit der Hardware Mechanismen bereit, die Aktivitäten auf das reale System „abzubilden“, d.h. die Prozesse auf der realen Hardware auszuföhren. Aus abstrakter Sicht – insbesondere aus Sicht von Programmen oberhalb der Systemschnittstelle – stellt sich das System als eine Menge von unabhängigen oder kooperierenden Prozessen dar.

3.1.2 Threading

Die Kontexte zweier Prozesse sind im Allgemeinen voneinander abgeschottet. Jeder Prozess besitzt also seine eigenen Registerinhalte, Speicherbereiche, geöffnete Dateien usw. und kann auf den Kontext eines anderen Prozesses nicht zugreifen. Diese strenge Trennung der Kontexte ist zwar für die Systemsicherheit und -fehlerrobustheit vorteilhaft (ein Prozess kann einem anderen Prozess nicht „ins Handwerk pfuschen“), hat aber auch Nachteile:

Erstens muss beim Umschalten des Prozessors ein vollständiger Kontextwechsel stattfinden. Das kann eine größere Zahl von Umspeicherungen erfordern und daher recht zeitaufwendig sein.

Prozess (Task)

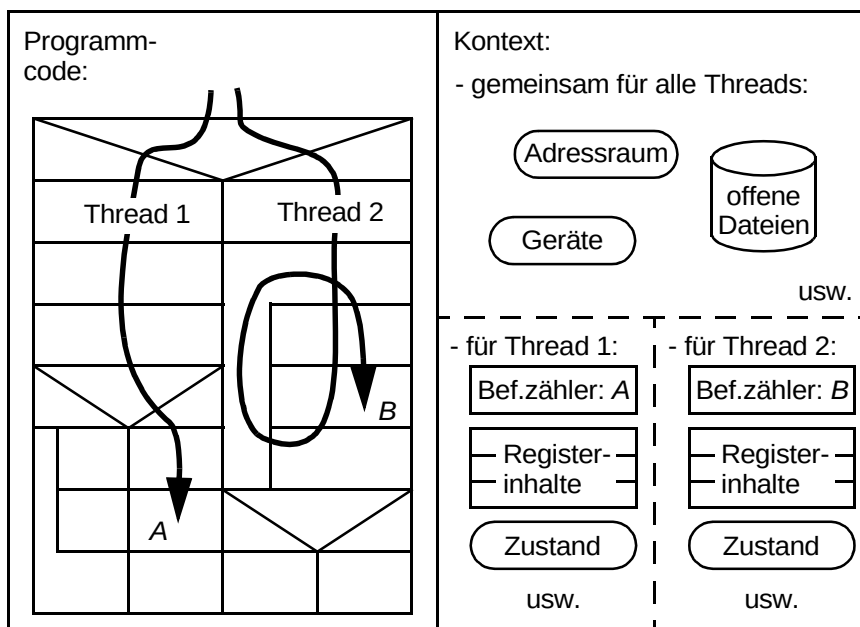


Abb. 3.3 Tasks und Threads

Zweitens ist es für Aktivitäten, die stark miteinander kooperieren, oft gar nicht wünschenswert, dass ihre Speicher voneinander getrennt sind. Sie möchten vielmehr auf ihre Daten unmittelbar gegenseitig zugreifen können.

Beide Probleme lassen sich durch das Konzept des *Threading* (wörtliche Übersetzung: „Fädellung“) lösen. Hier gibt es keinen Eins-zu-Eins-Zusammenhang zwischen Programmausführung und Kontext mehr, sondern es können mehrere nebenläufige Programmausführungen im selben Kontext aktiv sein, d.h. ihre Betriebsmittel gemeinsam benutzen (siehe Abb. 3.3). Es existieren also mehrere Aktivitätsträger, die mit demselben Datenbestand dasselbe Programm oder Programmsystem gleichzeitig ausführen – allerdings möglicherweise an unterschiedlichen Stellen. Dies entspricht mehreren „Kontrollfäden“, die beim Durchlaufen des „Labyrinths des Programmcodes“ abgewickelt werden (wie im Struktogramm von Abb. 3.3 links dargestellt) – daher der Name. Warum es überhaupt Sinn macht, dasselbe Programm mehrfach und an unterschiedlichen Stellen auszuführen, wird in nachfolgenden Abschnitten noch näher besprochen.

Die Aktivitätsträger unterscheiden sich nur in ihren Befehlszählern (sie geben an, an welchen Stellen des Codes sich die Ausführungen gerade befinden), den Registerinhalten, den Zuständen, eventuell ihren Stacks und noch einigen wenigen kleineren Betriebsmitteln (z.B. Zeitgebern / Timern). Der Prozessor kann also zwischen ihnen umgeschaltet werden, ohne dass dazu ein aufwendiger Kontextwechsel nötig wäre. Da alle Aktivitäten auf denselben Daten arbeiten, können sie unmittelbar kommunizieren und kooperieren, sind dafür aber auch nicht gegeneinander geschützt.

In diesem Zusammenhang werden die folgenden Fachbegriffe verwendet:

- Ein *Prozess* (engl. *process* oder *task*) ist ein Kontext zusammen mit dem Programmcode.
- Ein *Thread* oder *Lightweight Process* (= *leichtgewichtiger Prozess*) ist eine Aktivität (= Programmausführung) innerhalb eines Prozesses.

Threads gibt es u.a. in verschiedenen UNIX-Standards und -Implementationen (POSIX 1003.1c mit seinen „pthreads“, Solaris, AIX, Linux), in Windows NT / 2000 sowie in der Programmiersprache Java.

3.1.3 Implementierungsaspekte

Nach der Definition des Prozess- und des Threadbegriffs interessiert nun, wie Prozesse und Threads realisiert werden können. In diesem Abschnitt sollen einige allgemeine Bemerkungen zur Implementierung gemacht werden. Details bezüglich UNIX / Linux folgen im nächsten Abschnitt.

Aufgaben bei der Realisierung

Bei der Implementierung eines Prozesskonzepts stehen die folgenden Teilaufgaben an:

- Das Betriebssystem muss Buch über die Prozesse führen, damit es überhaupt „weiß“, welche Aktivitäten momentan zur Bearbeitung anstehen und was für Eigenschaften sie haben. Benötigt wird also für jeden Prozess eine interne Darstellung, eine so genannte *Prozessrepräsentation*.
- Prozesse müssen erzeugt, gestartet, terminiert (= beendet) und gelöscht werden können.
- Es müssen Zustandsübergänge von Prozessen stattfinden können. Hierzu gehört sowohl die technische Durchführung eines Übergangs als auch die strategische Entscheidung, wann welcher Zustandswechsel eintreten soll. Hier ist insbesondere die Entscheidung wichtig, wann der Prozessor welchem ausführungsbereiten Prozess zugeteilt werden soll.

Benötigt werden also zwei Dinge: Erstens muss das Betriebssystem Datenstrukturen führen, die die existierenden Prozesse beschreiben und darüber hinaus die Prozesse nach ihren jeweiligen Zuständen sortieren. Zweitens müssen Funktionen vorhanden sein, die Prozessrepräsentationen erzeugen und löschen, die den Prozessor einem Prozess zuteilen und ihn wieder entziehen und die die übrigen Zustandsübergänge durchführen.

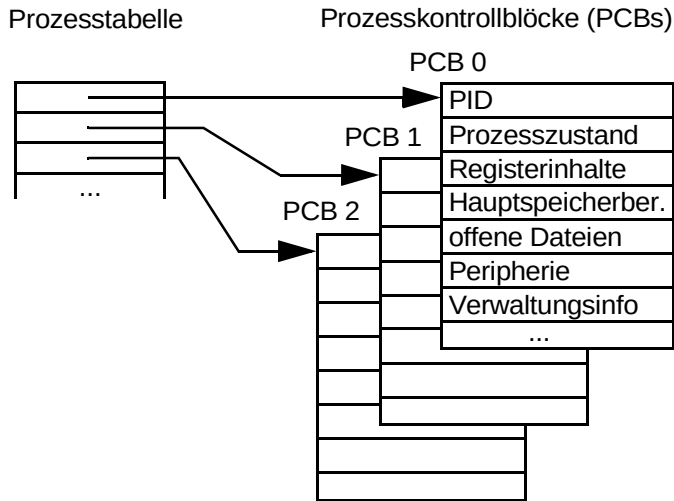


Abb. 3.4 Prozessstabelle und Prozesskontrollblöcke

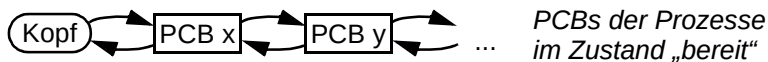
Prozesskontrollblöcke und Prozessstabelle

Die generelle Vorgehensweise bei der Realisierung wird in Abbildung 3.4 dargestellt: Jeder Prozess hat eine *Prozessnummer* (engl. *process identifier*, *PID*), die ihn eindeutig identifiziert. Zudem besitzt er einen eigenen *Kontrollblock* (engl. *process control block*, *PCB*). Das ist eine Datenstruktur, die sämtliche Informationen über den Prozess enthält, die das Betriebssystem zu seiner Ausführung braucht. Dazu gehören

- die Prozessnummer *PID*,
- der aktuelle Prozesszustand,
- die Inhalte der Prozessorregister – insbesondere der Befehlszähler, das Programmstatuswort (für Bits / Flags wie z.B. zur Signalisierung von arithmetischen Überläufen) und der Stack Pointer (für die Verwaltung geschachtelter Funktionsaufrufe),
- eine Liste der dem Prozess zugeordneten Hauptspeicherbereiche,
- für den Prozess geöffnete Dateien,
- dem Prozess zugeordnete Peripheriegeräte sowie
- Verwaltungsinformationen des Betriebssystems – z.B. die Priorität = Wichtigkeit des Prozesses sowie Abrechnungsdaten.

Die PCBs aller existierenden Prozesse sind in einer *Prozessstabelle* (engl. *process table*) zusammengefasst, die als Array oder verkettete Liste realisiert werden kann. Die Pro-

„bereit“-Liste:



„blockiert“-Listen:

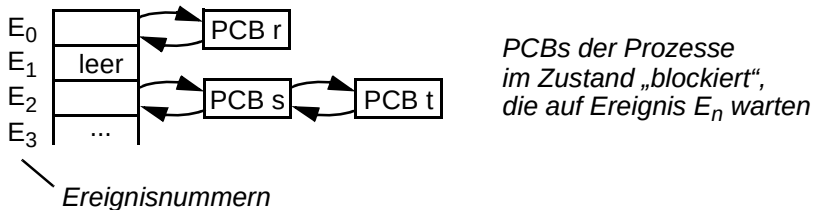


Abb. 3.5 Zustandslisten

zesstabelle ist damit die grundlegende Datenstruktur zur Prozessverwaltung im Betriebssystem. Informationen über einen bestimmten Prozess werden gefunden, indem die Prozesstabelle nach dem PCB mit der passenden PID durchsucht wird. Um die Suchzeiten kurz zu halten, können spezielle Techniken wie z.B. Hashing oder Suchbäume benutzt werden.

Neben der Prozesstabelle führt das Betriebssystem verkettete Listen für die Prozesse in den Zuständen „bereit“ und „blockiert“ (siehe Abb. 3.5). So enthält die „bereit“-Liste die PCBs aller Prozesse, die momentan auf dem Prozessor ausgeführt werden können; sie kann beispielsweise nach den Prioritäten (d.h. der Wichtigkeit) der Prozesse geordnet sein (siehe Abschnitt 3.2.2). Für den Zustand „blockiert“ existieren mehrere Listen: Jedem möglichen Ereignis ist eine eigene Liste zugeordnet, in der die PCBs aller Prozesse stehen, die auf dieses Ereignis warten.

Zur Erzeugung eines Prozesses werden zuerst eine freie PID belegt und ein PCB neu generiert oder ein unbenutzter PCB aus einer Liste mit freien PCBs ausgelesen. Der PCB wird mit geeigneten Anfangswerten vorbesetzt, wobei u.a. die benötigten Speicherbereiche bereitgestellt und initialisiert werden (siehe unten), und anschließend in die Prozesstabelle und die „bereit“-Liste eingefügt.

Zur Löschung eines Prozesses wird sein PCB aus seiner aktuellen Zustandsliste und der Prozesstabelle entfernt, die vom Prozess belegten Betriebsmittel sowie seine PID werden freigegeben, und der PCB wird gelöscht oder in die Liste der freien PCBs eingefügt.

Ein Zustandsübergang wird durch das Überwechseln des PCBs in die entsprechende Liste realisiert. Für den Zustand „rechnend“ wird keine Liste geführt; stattdessen gibt es eine spezielle Variable des Betriebssystems, die auf den PCB des momentan ausgeführten Prozesses verweist. Bei Übergang in diesen Zustand werden die Registerinhalte des Prozessors aus dem PCB geladen, beim Übergang aus diesem Zustand werden sie in den PCB kopiert („gerettet“).

Thread Packages

Das Thread-Konzept wird im Allgemeinen durch ein *Thread Package* implementiert. Es umfasst alle Mechanismen, die zur Benutzung von Threads gebraucht werden: Erzeugung, Löschung, Koordination usw. Ein Thread Package kann entweder im Betriebssystemkern liegen, wie beispielsweise in Linux oder Windows NT / 2000, oder oberhalb der Systemschnittstelle im User Mode ausgeführt werden, wie zum Beispiel in der verteilten Umgebung OSF/DCE (siehe Abschnitt 9.4.2).

3.1.4 Prozesse und Threads in UNIX / Linux

Auch UNIX und Linux benutzen Prozesse und Threads, um die Aktivitäten im System darzustellen. Im Folgenden wird die Vorgehensweise bei der Realisierung von UNIX/Linux-Prozessen und Threads skizziert. Zuvor sollen jedoch noch einige allgemeine Informationen über UNIX/Linux-Prozesse gegeben werden.

Ein Prozess in UNIX oder Thread in Linux wird durch eine eindeutige PID identifiziert, die ihm bei seiner Erzeugung zugeteilt wird. Die Prozessnummern 0 und 1 sind unter UNIX speziellen Systemprozessen zugeordnet, nämlich dem *Swapper-Prozess* bzw. dem *Init-Prozess*, die grundlegende Verwaltungsaufgaben übernehmen. Unter Linux trägt der Init-Prozess die Nummer 0; einen Swapper-Prozess gibt es nicht.

Neben seiner PID hat jeder Prozess eine Benutzer- und eine Benutzergruppenidentifikation (*UID* bzw. *GID*), die seinem Besitzer und dessen Gruppe entsprechen und beispielsweise seine Rechte für Dateizugriffe festlegen (siehe Abschnitt 2.3.1: Rechtebits der Dateien). Die UID 0 steht dabei für den Systemverwalter (*Super User*, *root*).

UNIX/Linux-Prozesse sind prinzipiell voneinander abgeschottet, arbeiten also im Allgemeinen in jeweils eigenen Speicherbereichen, und kommunizieren über Systemaufrufe. Threads unter Linux können allerdings Speicherbereiche gemeinsam zugeteilt werden. Zudem können bestimmte Speicherbereiche als *Shared Memory* vereinbart werden, auf die dann auch mehrere Prozesse aus unterschiedlichen Kontexten heraus zugreifen können (siehe Abschnitt 4.2.1).

Erzeugung und Terminierung

Zur dynamischen Erzeugung von UNIX/Linux-Prozessen wird die Schnittstellenfunktion `fork()` benutzt. Sämtliche Prozesse entstehen durch eine `fork()`-Ausführung – einzige Ausnahme ist der UNIX-Swapper- bzw. Linux-Init-Prozess, der beim Hochfahren des Systems als erster Prozess gewissermaßen „aus dem Nichts“ erzeugt wird.

Ruft ein Prozess (*Vaterprozess* genannt, engl. geschlechtsneutral *parent*) `fork()` auf, so entsteht dadurch ein zusätzlicher neuer Prozess (ein *Sohnprozess*, engl. *child*), der eine exakte Kopie des Vaterprozesses ist. Insbesondere haben Vater und Sohn denselben Programmcode, und ihre Befehlszähler haben unmittelbar nach der Ausführung von `fork()` denselben Wert, verweisen also auf dieselbe Stelle im Programmtext. Allerdings haben die beiden Prozesse streng voneinander getrennte Speicherbereiche: Der Sohn übernimmt zwar die aktuellen Speicherinhalte des Vaters in seinen eigenen Spei-

cher, kann sie aber dort anschließend unabhängig vom Vater ändern. Allgemeiner gesprochen: Vater und Sohn besitzen jeweils ihren eigenen Kontext.

`fork()` dupliziert also einen Prozess: *Ein* Prozess ruft `fork()` auf, und *zwei* Prozesse kehren aus diesem Aufruf zurück, wobei beide denselben Programmcode und denselben Wert des Befehlszählers haben. Allerdings sind die Rückgabewerte von `fork()` in den beiden Prozessen verschieden: Im Vaterprozess liefert `fork()` die PID des Sohnprozesses zurück (die stets von 0 verschieden ist), im Sohnprozess eine 0. Durch Vergleich des Rückgabewerts mit 0 und anschließende Verzweigung kann man dann den Sohn ein anderes Programmstück ausführen lassen als den Vater (siehe Beispiel weiter unten).

Ein Prozess beendet sich selbst, wenn er das Ende des Programmtexts erreicht oder die Schnittstellenfunktion `exit()` aufruft. An `exit()` wird ein ganzzahliger Parameter als Terminierungscode (z.B. zur Beschreibung eines Fehlers) übergeben, der an den Vaterprozess weitergegeben wird.

Darüber hinaus können Prozesse andere Prozesse mit Hilfe der Schnittstellenfunktion `kill()` terminieren: `kill(pid, SIGKILL)` sendet das Signal `SIGKILL` an den Prozess mit der Nummer `pid` und beendet ihn damit. Allerdings kontrolliert UNIX / Linux, ob der aufrufende Prozess den anderen Prozess überhaupt terminieren darf; beispielsweise hat kein einfacher Benutzer das Recht, die Prozesse eines anderen Benutzers „abzuschießen“. Außer `SIGKILL` gibt es noch andere Signale, die der empfangende Prozess durch *Signalhandler* (= Funktionen, die bei Empfang des Signals ausgeführt werden) abfangen oder auch ignorieren kann (siehe auch Abschnitt 4.1.4).

Das folgende Programmstück ist ein typisches Beispiel für die Benutzung von `fork()` und `exit()`:

```
...
int i;
int status, err;

if (fork() == 0) {
    /* Sohnprozess: */
    err = execv("programm", ...);
    exit(err);
}

/* Vaterprozess: */
wait(&status);
...
```

Durch den Aufruf von `fork()` wird eine Kopie des ausführenden Prozesses erzeugt. Insbesondere besitzen anschließend Vater und Sohn jeweils eine eigene Variable namens `i` und können im Folgenden nur auf ihr eigenes `i` zugreifen.

Der Sohnprozess verzweigt nach seiner Erzeugung in den `if`-Block des Programms, da ihm der `fork()`-Aufruf eine 0 zurückgeliefert hat. Er ruft mit der Schnittstellenfunktion `execv()` das Programm in der Datei `programm` auf und terminiert nach

dessen Beendigung mit `exit()`, wobei der Rückgabecode der programm-Ausführung an den Vaterprozess zurückgegeben wird.

Der Vaterprozess hat währenddessen seine Ausführung hinter dem `if`-Block fortgesetzt, da sein `fork()`-Aufruf die von 0 verschiedene PID des Sohns zurücklieferte. Er wartet dort auf das `exit()` seines Sohns. Dies geschieht durch die Funktion `wait()`, mit der sich ein Prozess bis zur Terminierung eines seiner Söhne blockiert. Über deren Referenzparameter erhält der Vater u.a. den Rückgabecode des Sohns, den er mit Hilfe von Makros in `sys/wait.h` oder `waitstatus.h` analysieren kann.

Es ist noch anzumerken, dass der Sohn nicht unbedingt ein Programm in einer anderen Datei aufrufen muss, sondern dass sein Code auch unmittelbar im `if`-Block stehen kann. In jedem Fall muss aber das abschließende `exit()` vorhanden sein, da sonst der Sohn im Vatercode weiterlaufen würde.

Weitere Programmbeispiele finden sich bei den Übungsaufgaben am Ende dieses Kapitels und im Anhang.

Threads in Linux

In Linux können nicht nur neue Prozesse, sondern auch neue Threads innerhalb eines bestehenden Kontexts erzeugt werden. Hierfür gibt es die Funktion `clone()`. Sie unterscheidet sich von `fork()` darin, dass `fork()` für den neuen Prozess einen vollständig neuen Kontext generiert, während `clone()` die Möglichkeit bietet, den gesamten Kontext oder auch Teile davon dem aufrufenden Prozess und dem neu erzeugten Thread gemeinsam zur Verfügung zu stellen. Welche Teile des Kontexts neu erzeugt werden, kann durch Parameter festgelegt werden.

Sowohl `fork()` als auch `clone()` erzeugen eine neue Aktivität mit einer eigenen PID, für die intern dieselbe Menge von Verwaltungsinformationen geführt wird (Details siehe weiter unten). Linux-intern wird also gar keine Unterscheidung zwischen Prozessen und Threads gemacht: Ein Thread ist für Linux ein Prozess, der seinen Kontext ganz oder teilweise mit anderen Prozessen teilt. Im weiteren Verlauf dieses Kapitels werden wir also nicht mehr zwischen Linux-Prozessen und -Threads unterscheiden.

Prozesszustände

Wie Abbildung 3.6 zeigt, hat UNIX / Linux ein erweitertes Zustandsdiagramm, in dem der Zustand „rechnend“ aufgespalten ist in die Zustände „rechnend (User Mode)“ und „rechnend (Kernel Mode)“. Das entspricht der Tatsache, dass ein UNIX/Linux-Prozess entweder im privilegierten oder im nichtprivilegierten Modus aktiv ist (siehe Kapitel 2). Außerdem gibt es, wie bisher, die Zustände „bereit“ und „blockiert“ sowie zusätzlich einen Zustand „Zombie“, der weiter unten erläutert wird.

Zu den Zustandsübergängen gehören, neben den bereits besprochenen, die Übergänge zwischen den beiden neuen „rechnend“-Zuständen. Sie entsprechen dem Aufruf einer Systemfunktion bzw. der Rückkehr daraus. Man beachte, dass Übergänge aus oder nach „blockiert“ / „bereit“ nur nach bzw. aus „rechnend (Kernel Mode)“, nicht jedoch „rechnend (User Mode)“ möglich sind. Das liegt daran, dass diese Übergänge nur im

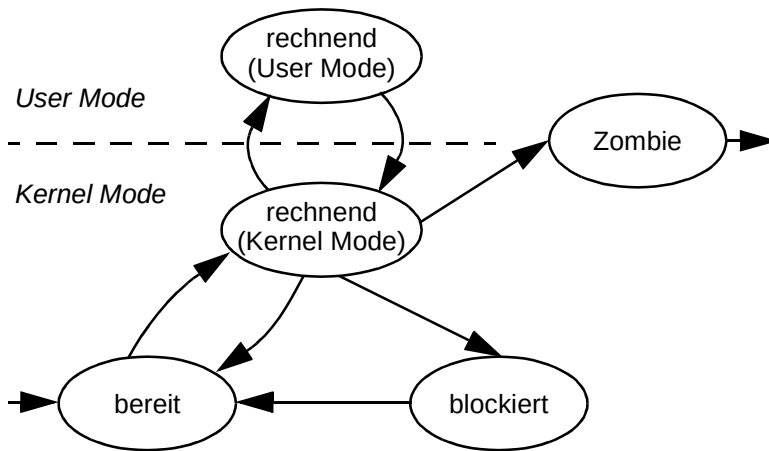


Abb. 3.6 Zustandsdiagramm für UNIX/Linux-Prozesse (vereinfacht)

Zusammenhang mit einem Systemaufruf oder einem so genannten Interrupthandler stattfinden können. Wir werden darauf in Unterkapitel 3.3 noch genauer eingehen.

Ein terminierter Prozess wird nicht sofort vollständig gelöscht, sondern geht zunächst in den *Zombie-Zustand* über, wobei seine Einträge in der Prozesstabelle erhalten bleiben. Damit kann der Vaterprozess auch später noch per `wait()` Informationen über seinen verstorbenen Sohn entgegennehmen und den Sohn dabei endgültig löschen. Terminiert der Vater ohne `wait()`-Aufrufe für alle seine Söhne, so werden die verbleibenden „Waisen“ (laufende Prozesse und Zombies) vom Init-Prozess „geerbt“, der seinerseits die nötigen `wait()`-Aufrufe tätigt.

Speicherkonzept

UNIX/Linux-Prozesse haben ein bestimmtes *Speicherlayout*, d.h. eine festgelegte Anordnung ihres Programmcodes und ihrer Daten in ihrem Adressraum. Der Begriff „Adressraum“ charakterisiert die Menge der Speicherzellen, auf die der Prozess direkt zugreifen kann. Es handelt sich hier um den so genannten „virtuellen Adressraum“, der mit Hilfe von Betriebssystem und Hardware auf den realen Haupt- und Plattenspeicher abgebildet wird. Kapitel 5 wird auf diese Technik noch ausführlich eingehen.

Abbildung 3.7 stellt das logische Speicherlayout eines UNIX/Linux-Prozesses in seiner einfachsten Form dar. Der Speicher besteht aus mindestens aus drei Teilen:

- *Text*: Der Text-Teil enthält den ausführbaren Programmcodes des Prozesses. Auf ihn darf normalerweise nur lesend zugegriffen werden („Read-Only“).
- *Data*: Der Data-Teil umfasst zwei Bereiche. Der *initialisierte Bereich* nimmt die Konstanten und initialisierten Variablen auf, die zu Beginn der Programmausführung vorhanden sind. In ihn werden bei Prozessstart die entsprechenden Anfangswerte ein-

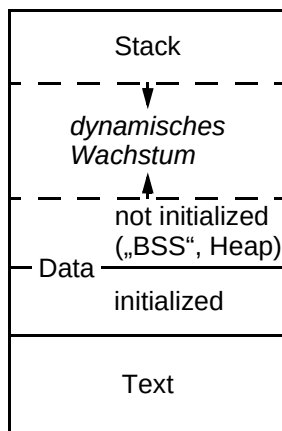


Abb. 3.7 logisches Speicherlayout eines UNIX/Linux-Prozesses (einfachste Form)

getragen. Im *nichtinitialisierten Bereich* (auch *BSS* genannt) stehen erstens die nicht-initialisierten Variablen, die bei Beginn des Programms existieren (also z.B. die globalen Variablen eines C-Programms, denen kein Anfangswert zugewiesen wurde). Zweitens können hier Datenstrukturen während der Programmausführung dynamisch erzeugt werden (z.B. in C durch die Funktion `malloc()`) – es handelt sich hierbei also um den „Heap“ des Programms. Der BSS-Bereich wird bei Start des Prozesses reserviert, aber nicht mit Werten vorbelegt. Auf den Data-Teil darf lesend und schreibend zugegriffen werden („Read-Write“).

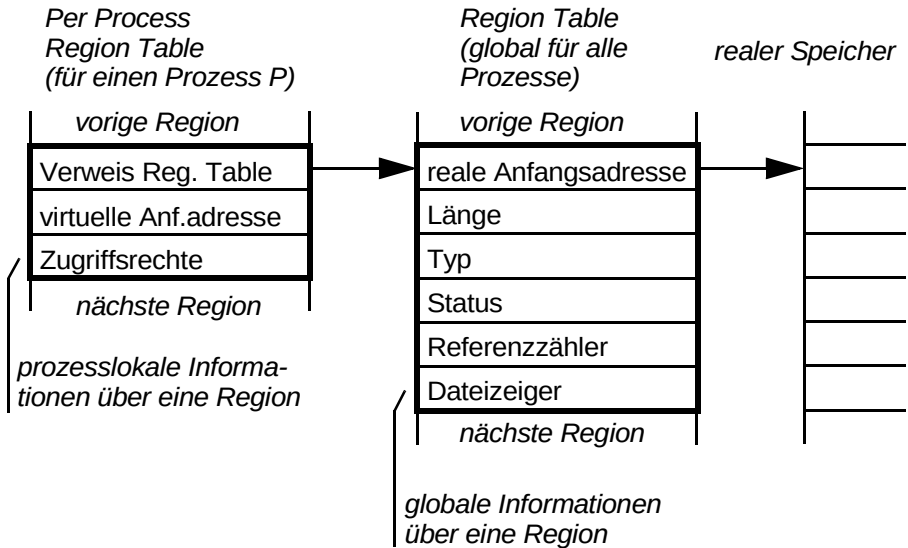
- *Stack*: Der Stack wird zur Realisierung von beliebig geschachtelten Funktionsaufrufen benutzt und dabei nach dem bekannten LIFO-Prinzip (Last-In-First-Out) mit den Operationen „Push“ und „Pop“ verwaltet. Beim Aufruf einer Funktion wird ein neuer *Frame* auf dem Stack abgelegt (Push), d.h. eine Datenstruktur mit den Funktionsparametern, den lokalen Variablen der Funktion, der Rücksprungadresse sowie einem Zeiger auf den Frame der aufrufenden Funktion. Beim Rücksprung aus dem Funktionsaufruf wird der entsprechende Frame wieder vom Stack entfernt (Pop). Auf den Stack darf lesend und schreibend zugegriffen werden. Für User und Kernel Mode gibt es zwei getrennte Stacks.

Während der Text-Teil und der initialisierte Data-Teil ihre Größe nicht ändern, können der BSS-Teil und der Stack dynamisch wachsen. Der BSS-Teil, der im unteren Adressbereich des Prozesses liegt, wächst in den meisten UNIX-Implementationen „nach oben“ (also in Richtung höherer Speicheradressen); der Stack im oberen Adressbereich wächst ihm entgegen.

In neueren UNIX-Versionen und Linux kann der Prozessspeicher noch weitere Komponenten enthalten, wie z.B. „memory-mapped regions“, in die Teile von Dateien abgebildet werden können.

Die Speicherbereiche der Prozesse werden in UNIX / Linux durch *Regionen* repräsentiert. Eine Region entspricht einem zusammenhängenden virtuellen Adressbereich

a.) Regionen in UNIX System V:



b.) Regionen in Linux:

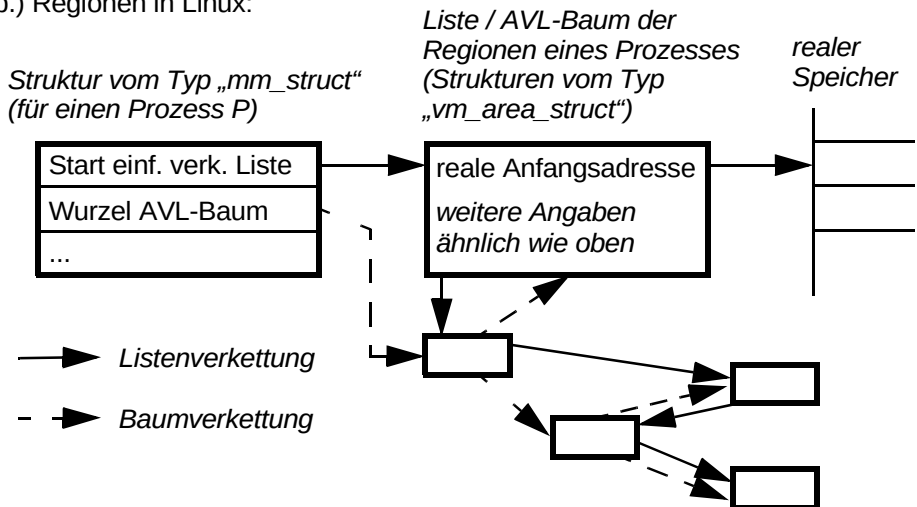


Abb. 3.8 Verwaltung der Regionen in UNIX System V und in Linux

eines Prozesses, der individuell geschützt und, falls gewünscht, mit anderen Prozessen gemeinsam benutzt („geshert“) werden kann. Text-Teil, Daten-Teil und Stack eines Prozesses bildet jeweils mindestens eine eigene Region.

Zur Verwaltung der Regionen führt das klassische UNIX System V mehrere interne Datenstrukturen (siehe Abb. 3.8, Teil a): Eine globale *Region Table* enthält für jede Re-

gion, die momentan benutzt werden kann, einen eigenen Eintrag, der die folgenden Informationen speichert:

- Lage im realen Speicher (eine so genannte Seitentabelle – siehe Kapitel 5) und Länge.
- Typ: „text“, „shared memory“, „private data or stack“, wobei „shared memory“ insbesondere für Programme sinnvoll ist, die von mehreren Prozessen zur selben Zeit ausgeführt werden.
- Status: z.B. „locked“ = z.Zt. für Zugriffe gesperrt, „being loaded“ = z.Zt. auf dem Transport in den Hauptspeicher.
- Referenzzähler: Anzahl der Prozesse, die diese Region gerade benutzen.
- Verweis auf die Datei, aus der der Inhalt der Region ursprünglich geladen wurde.

Neben der globalen Region Table gibt es für jeden Prozess eine eigene *Per Process Region Table* (kurz *Pregion*). Hier sind die Regionen verzeichnet, auf denen der Prozess gerade arbeitet. Für jede dieser Regionen enthält die Tabelle einen Verweis auf den zugehörigen Eintrag in der Region Table, die Anfangsadresse im virtuellen Adressraum des Prozesses sowie die Zugriffsrechte des Prozesses (Read, Read/Write oder Read/Execute).

In Linux gibt es keine solche Zweistufigkeit aus prozessspezifischen und globalen Regionentabellen (siehe Abb. 3.8, Teil b). Hier ist jedem Prozess unmittelbar eine Menge von Datenstrukturen (Typ „*vm_area_struct*“) zugeordnet, die seine Regionen (auch „Segmente“ genannt) beschreiben. In einer Struktur dieses Typs stehen u.a. die Anfangs- und Endadresse der Region im Adressraum des Prozesses, Typ- und Schutzinformationen sowie Informationen über die Datei, aus der die Region geladen wurde.

Eine weitere Datenstruktur (Typ „*mm_struct*“) nimmt sämtliche Regioneninformationen eines Prozesses auf. In ihr sind eine einfach verkettete Liste und zusätzlich ein rascher zugreifbarer, nach Speicheradressen sortierter AVL-Baum verankert, in denen die entsprechenden *vm_area_struct*-Strukturen stehen. Zudem sind hier Informationen zur Abbildung der virtuellen Adressen auf reale Haupt- und Plattenspeicherbereiche enthalten – eine so genannte Seitentabelle (siehe Kapitel 5).

Prozessrepräsentation

Ein Multiprogramming-Betriebssystem führt, wie bereits gesagt, eine Prozesstabelle, in der es Informationen über alle seine Prozesse ablegt. Auch der UNIX- und der Linux-Kern enthalten eine solche Tabelle.

In UNIX System V ist die Tabelle aus Effizienzgründen real in zwei Teilen abgespeichert (siehe Abb. 3.9): Die eigentliche *Prozesstabelle* enthält die Prozessinformationen, die ständig benötigt werden. Sie befindet sich immer im Hauptspeicher. In der *User Structure* (*U area*) stehen die Informationen, die nur gebraucht werden, wenn sich der Prozess im Hauptspeicher befindet. Wie später noch genauer diskutiert wird, befinden

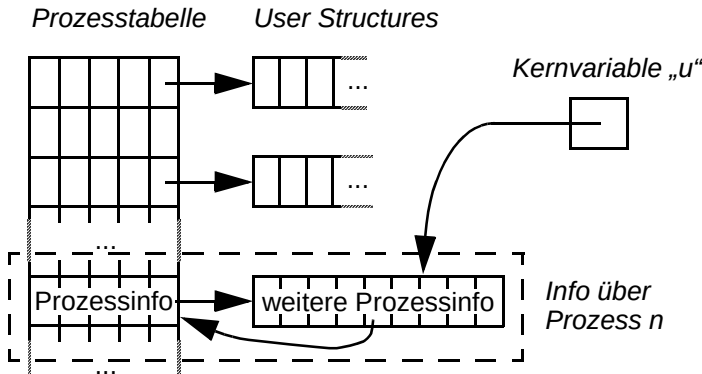


Abb. 3.9 Prozessrepräsentation in UNIX System V

sich nämlich aus Platzgründen nicht alle aktiven Prozesse mit ihren Daten und ihrem Code im Hauptspeicher, sondern es sind einige von ihnen auf den Plattenpeicher „ausgelagert“. Im Gegensatz zur Prozesstabelle wird die User Structure stets mit dem zugehörigen Prozess auf den Plattenpeicher verdrängt, was Hauptspeicherplatz spart.

Die Prozesstabelle in UNIX System V enthält im Wesentlichen Informationen zur Speicherverwaltung (Größe des Prozesses, Zeiger auf die Text-, Data- und Stack-Regionen sowie auf die User Structure), zur Entscheidung über die Prozessorzuteilung („Schedulingparameter“: Priorität des Prozesses, kürzlich verbrauchte CPU-Zeit, kürzlich blockiert verbrauchte Zeit), zur Behandlung von Signalen (u.a. die Angabe, welche Signale abgefangen, ignoriert oder temporär blockiert werden sollen), Abrechnungsdaten (z.B. die bislang verbrauchte Prozessorzeit sowie „Quoten“ = Obergrenzen für die Anzahl zugeteilter Betriebsmittel) und weitere Informationen, wie z.B. den Prozesszustand, das Ereignis, auf das gerade gewartet wird, die eigene PID, die PID des Vaterprozesses sowie die zugeordnete UID und die GID.

In der User Structure sind weitere Informationen über den Prozess gespeichert. Dazu gehören seine aktuellen Inhalte der Prozessorregister, Informationen über den momentan ausgeführten Systemaufruf (u.a. Parameter und Rückgabewert), eine Tabelle mit Angaben über die aktuell geöffneten Dateien, das Arbeitsverzeichnis, weitere Abrechnungsinformationen, ein Zeiger auf den Eintrag in der Prozesstabelle sowie weitere Daten, die hier nicht näher besprochen werden sollen. Offensichtlich werden die Informationen in der User Structure nicht benötigt, wenn der Prozess auf den Plattenpeicher ausgelagert wurde und somit momentan nicht ausgeführt werden kann.

Die Per Process Region Table befindet sich, je nach Implementation, in der Prozesstabelle oder der User Structure – entweder unmittelbar oder in Gestalt eines Verweises.

In Linux gibt es keine gesonderte User Structure, sondern es stehen sämtliche Prozessinformationen in einem großen Tabelleneintrag. Er ist zwar etwas anders strukturiert als seine Gegenstücke in System V, umfasst aber im Wesentlichen dieselben Informationen. Um einen schnelleren Durchlauf durch alle Prozesse zu ermöglichen, sind die Prozessinformationen nicht nur im Array gespeichert, sondern zudem in eine doppelt verkettete Liste eingehängt.

Der Kern von UNIX und Linux enthält eine globale Variable, die stets auf die Prozessinformationen des gerade ausgeführten Prozesses zeigt (User Structure unter System V, Prozesstabelleneintrag unter Linux). Ihr Wert ändert sich also immer, wenn der Prozessor einem anderen Prozess zugeteilt wird. Über diese Variable kann der Kern also stets direkt auf Informationen über den Prozess zugreifen, der gerade auf dem Prozessor läuft.

Zur Erzeugung eines neuen Prozesses, d.h. bei der Ausführung von `fork()`, werden eine freie PID belegt, ein freies Feld in der Prozesstabelle gesucht und (im Fall von System V) eine User Structure erzeugt. Anschließend wird (fast) der gesamte Inhalt der User Structure und des Prozesstabelleneintrags des Vaterprozesses in die User Structure bzw. den Tabelleneintrag des Sohnprozesses kopiert. Das einzige, was nicht kopiert wird, sind die Verweise auf die Data- und Stack-Regionen des Vaterprozesses. Stattdessen werden neue Speicherbereiche für den Sohn belegt (und in der Prozesstabelle registriert), in die dann die Inhalte der entsprechenden Regionen des Vaterprozesses kopiert werden. Hier ist also deutlich zu sehen, dass bei `fork()` der Sohnprozess als exakte Kopie des Vaterprozesses entsteht, dabei aber seine eigenen Speicherbereiche erhält.

Etwas anders sieht es bei der Linux-Funktion `clone()` aus: Hier kann durch einen Parameter festgelegt werden, dass Vater und Sohn ihren Speicher gemeinsam benutzen sollen. In diesem Fall verweisen also Vater- und Sohn-Kontext auf dieselben Regionen.

Unter Linux benutzt man zudem die „copy-on-write“-Technik, um den Kopieraufwand niedrig zu halten. Das bedeutet, dass während des `fork()` oder `clone()` generell keine Daten kopiert werden, sondern Vater und Sohn (noch) auf denselben Speicherbereich zeigen. Erst wenn einer der Prozesse auf die Region schreibend zugreift, wird ihr Inhalt dupliziert und damit physisch eine neue Region angelegt – es sei denn, es wurde eine gemeinsame Benutzung vereinbart.

3.2 Prozesswechsel

In einem Multiprogramming-System wird der Prozessor im raschen Wechsel zwischen Prozessen umgeschaltet. Das Betriebssystem hat hierbei zwei Aufgaben: Es muss erstens, mit Hilfe der Hardware, diese Umschaltung technisch bewerkstelligen, also das so genannte *Dispatching* durchführen. Zweitens muss es strategische Entscheidungen über die Ausführungsreihenfolge der Prozesse treffen, was mit dem Begriff *Scheduling* bezeichnet wird. Durch das Scheduling wird festgelegt, wann Prozesswechsel stattfinden und zu welchem der ausführungsbereiten Prozesse umgeschaltet wird.

3.2.1 Dispatching

Ein Dispatcher (Prozessumschalter) ist für die Durchführung der Zustandsübergänge von Prozessen zuständig, wie sie in Abschnitt 3.1.1 besprochen wurden. Bei Übergängen, die den Zustand „rechnend“ nicht betreffen, werden Prozesskontrollblöcke aus

verketteten Listen entfernt und in sie eingefügt, also einfache Listenoperationen durchgeführt (siehe Abschnitt 3.1.3).

Übergänge aus dem oder in den Zustand „rechnend“ bedeuten dagegen einen Wechsel des aktuell ausgeführten Prozesses: Der Prozessor wird einem Prozess entzogen, wobei dessen Kontext „gerettet“ werden muss, und einem anderen Prozess zugeteilt, wobei dessen alter Kontext wiederhergestellt werden muss. Ein solcher Übergang (ein so genannter *Context Switch*) geht folgendermaßen vor sich: Der Dispatcher wird i.a. durch ein bestimmtes Ereignis („Interruptsignal“, wie z.B. Blockierung des laufenden Prozesses oder eine Meldung der Uhr) aktiviert. Er speichert die Registerinhalte des Prozessors im Kontrollblock des gerade ausgeführten Prozesses und kopiert dann die Registerinhalte des neu auszuführenden Prozesses aus dessen Kontrollblock in die Prozessorregister.

Um Prozesswechsel rasch durchführen zu können und den Verwaltungsaufwand dabei niedrig zu halten, ist Hardwareunterstützung erforderlich. So gibt es in modernen Prozessoren spezielle Maschinenbefehle, die die Inhalte sämtlicher betroffener Prozessorregister in einem Schritt laden bzw. abspeichern. Darüber hinaus gibt es Prozessoren mit mehreren Registersätzen, von denen jeder einem Prozess zugeordnet ist und zwischen denen sehr schnell umgeschaltet werden kann. Umschaltzeiten in heutigen Systemen liegen typischerweise zwischen 1 und 1000 Mikrosekunden.

Da zu jedem Zeitpunkt ein Prozess auf dem Prozessor ausgeführt werden soll, gibt es eine spezielle „Idle Task“. Sie erhält den Prozessor, wenn kein anderer Prozess ausführungsbereit ist.

3.2.2 Scheduling

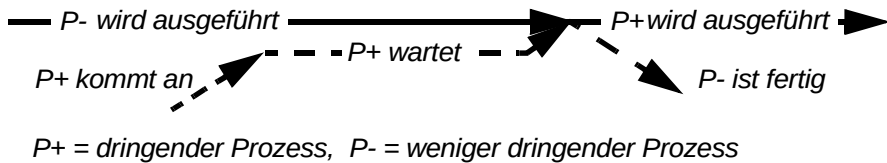
Durch das Scheduling legt das Betriebssystem fest, in welcher Reihenfolge die ausführungsbereiten Prozesse bearbeitet werden sollen. In diesem Abschnitt werden *Schedulingstrategien* besprochen, also Verfahren, mit denen aus der Menge der bereiten Prozesse einer zur Ausführung ausgewählt wird. Später wird das zusätzliche Problem diskutiert, welche Prozesse überhaupt ausführungsbereit sein sollen und welche (zeitweise) auf den Plattenspeicher ausgelagert werden (siehe Kapitel 5). Darüber hinaus ist interessant, zu welchen Zeitpunkten eine Umschaltung überhaupt stattfinden kann.

Ziele

Ziel beim Scheduling ist, die Prozesse möglichst effizient auszuführen, wobei der Effizienzbegriff natürlich von der Wahl eines Leistungskriteriums abhängt. Mögliche Kriterien sind hier:

- Wichtige oder dringende Prozesse sollen möglichst rasch bedient werden.
- Die mittlere Antwortzeit soll minimiert werden, also die Zeitdauer, während derer ein Benutzer am Terminal auf das Ergebnis warten muss.

a.) nicht-unterbrechendes Scheduling:



b.) unterbrechendes Scheduling:

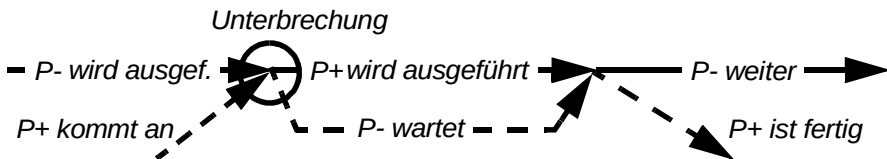


Abb. 3.10 nicht-unterbrechendes vs. unterbrechendes Scheduling

- Die Ausnutzung des Prozessors für Benutzerprozesse soll gegenüber den Verwaltungsaktivitäten maximiert werden.

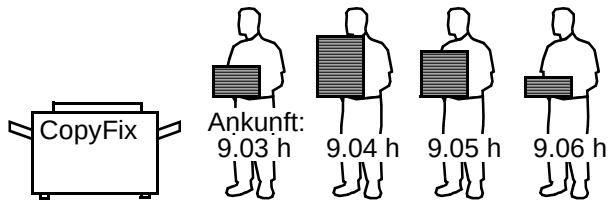
Die Leistungskriterien sind teilweise gegensätzlich, so dass – je nach Kriterium – unterschiedliche Schedulingstrategien sinnvoll sind.

Nicht-unterbrechendes vs. unterbrechendes Scheduling

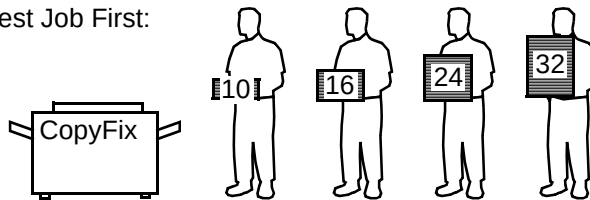
Schedulingstrategien können in zwei Klassen eingeteilt werden: Beim *nicht-unterbrechenden Scheduling* (engl. *non-preemptive scheduling*, auch *nicht-verdrängendes Scheduling* genannt) wird ein gestarteter Prozess ohne Unterbrechung bis zu seinem Ende ausgeführt. Zwischen seinem Beginn und seiner Terminierung wird der Prozessor also nicht zu einem anderen Prozess umgeschaltet. Problematisch ist hierbei, dass ein Prozess den Prozessor so lange belegen kann, wie er möchte. Damit kommen andere Prozesse für längere Zeit nicht zum Zuge, obwohl sie möglicherweise dringender sind (siehe Abb. 3.10, Teil a). Man kann hier zwar, wie früher in Windows-3.x-Anwendungen, ein *kooperatives Multitasking* implementieren, bei der Prozesse den Prozessor in gewissen Abständen freiwillig abgeben. Das ist aber keine durchgreifende Lösung, da das Betriebssystem auf den „guten Willen“ der Prozesse angewiesen ist.

Daher wird in modernen Betriebssystemen, wie z.B. in UNIX, Linux und den aktuellen Windows-Systemen, ein *unterbrechendes Scheduling* (engl. *preemptive scheduling*, auch *verdrängendes Scheduling* genannt) benutzt. Hier kann das Betriebssystem einen Prozess während seiner Ausführung zugunsten anderer Prozesse unterbrechen, d.h. der Prozessor kann zwischenzeitlich einem anderen Prozess zugeteilt werden. Solche Unterbrechungen finden insbesondere dann statt, wenn während der Prozessaufführung ein dringenderer Prozess ausführungsbereit wird (siehe Abb. 3.10, Teil b). Der unterbrochene Prozess wird später fortgesetzt.

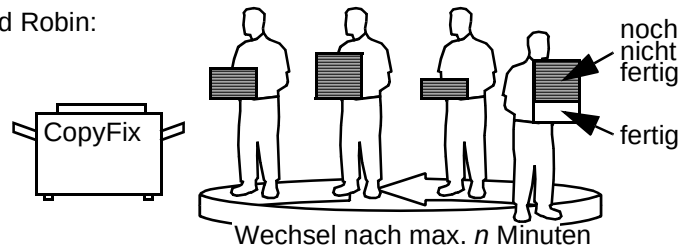
First Come First Served:



Shortest Job First:



Round Robin:



Prioritäten:

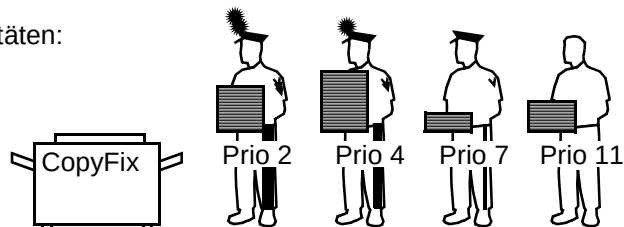


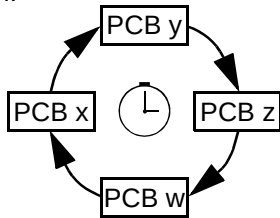
Abb. 3.11 Schedulingstrategien am Beispiel einer Warteschlange vor einem Kopiergerät

Schedulingstrategien

Im Laufe der Zeit wurde eine Vielzahl von Schedulingstrategien entwickelt, von denen hier die folgenden kurz angesprochen werden sollen (siehe dazu auch Abb. 3.11):

- *First Come First Served (FCFS)*: Die Prozesse werden in der Reihenfolge ihres Eintreffens oder Entstehens bedient – wie bei einer Warteschlange von Kunden in einem

logisch:



Weitergabe der CPU, wenn

- Zeitquantum abgelaufen
- Prozess blockiert
- Prozess terminiert

Implementation durch „bereit“-Liste:

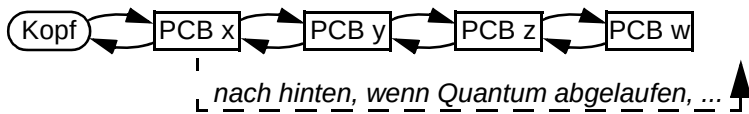


Abb. 3.12 Round-Robin-Strategie

Geschäft. Die Strategie ist von Natur aus nicht-unterbrechend. Problematisch ist, dass Prozesse mit kurzen Bearbeitungsdauern unverhältnismäßig lange warten müssen, wenn längerdauernde Prozesse vor ihnen eingetroffen sind. „Kurzläufer“ werden also benachteiligt.

- *Shortest Job First (SJF)*: Hier wird jeweils der Prozess mit der kürzesten Bearbeitungsdauer zuerst ausgeführt. Die Strategie kann sowohl als nicht-unterbrechend als auch als unterbrechend implementiert werden; im letzteren Fall wird der aktuelle Prozess sofort unterbrochen, wenn ein neuer Prozess mit einer kürzeren Bearbeitungsdauer eintrifft. Im Gegensatz zu FCFS bevorzugt SJF Kurzläufer.

Man kann zeigen, dass dieses Verfahren (unter der Voraussetzung, dass alle Prozesse gleichzeitig eintreffen) unter allen Strategien die kleinstmögliche mittlere Wartezeit liefert. Problematisch ist allerdings, dass die Bearbeitungsdauern der Prozesse a priori bekannt sein müssen, was in der Praxis selten der Fall ist.

- *Round Robin mit Zeitquantum („Zeitscheibenverfahren“)*: Bei dieser Strategie sind die Prozesse in einer zyklischen Liste angeordnet (siehe Abb. 3.12). Ihnen ist ein *Zeitquantum* (auch *Zeitscheibe*, engl. *time slice*, genannt) zugeordnet, das die maximale Zeit festlegt, für die ein Prozess den Prozessor ohne Unterbrechung besitzen darf. Ist das Zeitquantum abgelaufen oder blockiert / terminiert der aktuelle Prozess, so geht der Prozessor zum nächsten Prozess in der Liste über. Die Prozesse werden also reihum bedient, wobei jeder Prozess in jeder Runde den Prozessor für die Dauer von maximal einem Zeitquantum erhält.

Je länger die Bearbeitungsdauer eines Prozesses ist, desto mehr Runden werden zu seiner Ausführung benötigt. Round Robin bevorzugt also, wie SJF, Kurzläufer, ohne jedoch die Bearbeitungsdauern der Prozesse kennen zu müssen. Die Bevorzugung ist umso ausgeprägter, je kleiner das Zeitquantum ist. Dessen Wert sollte aber nicht zu

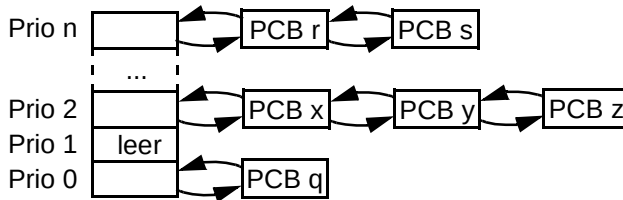


Abb. 3.13 Multilevel Queueing

klein gewählt werden, da es dann zu einem relativ hohen Verwaltungsaufwand für die zahlreichen Umschaltungen kommt. Typische Zeitquanten liegen im zweistelligen Millisekundenbereich.

Round Robin ist von Natur aus eine unterbrechende Strategie.

- **Prioritätengesteuertes Scheduling:** Prozesse werden hier in der Reihenfolge ihrer Wichtigkeit oder Dringlichkeit bearbeitet. Jedem Prozess ist dabei eine ganze Zahl als Prioritätswert zugeordnet, wobei i.a. niedrige Werte eine hohe Dringlichkeit (d.h. eine „hohe Priorität“) anzeigen, und es wird jeweils der Prozess mit der höchsten Priorität ausgeführt. Die Strategie ist im Allgemeinen unterbrechend.

Die Priorität eines Prozesses kann von internen Kriterien (z.B. Verhältnis seiner Ein-Ausgabezeit zur Rechenzeit, Anzahl und Art der benötigten Betriebsmittel, Hintergrundprozess oder interaktiver Prozess) oder externen Kriterien (Rang des Benutzers, geforderte Echtzeiteigenschaften) abhängen. Sie kann *statisch* sein, d.h. sich während der gesamten Lebensdauer des Prozesses nicht ändern, oder *dynamisch*, d.h. von Zeit zu Zeit angepasst werden.

Strategien für die Anpassung von Prioritäten sind beispielsweise das *Aging* oder das *Multilevel Feedback Queueing*. Beim Aging wächst die Priorität mit zunehmendem Alter des Prozesses. Beim Multilevel Feedback Queueing hängt die Priorität vom bisherigen Prozessverhalten ab; Prozesse mit langen Wartezeiten und / oder geringer bisheriger Prozessorbelastung erhalten eine höhere Priorität. Dies wird z.B. in UNIX und Windows NT praktiziert (UNIX-Details siehe nächster Abschnitt).

Der Vorteil der dynamischen gegenüber den statischen Prioritäten liegt darin, dass damit niederpriorie Prozesse vor dem „Verhungern“ bewahrt werden können: Prozesse mit einer anfangs niedrigen Priorität werden im Laufe ihrer Wartezeit schrittweise hochgestuft und erhalten somit schließlich den Prozessor, selbst wenn zwischenzeitlich ständig neue Prozesse mit hoher Priorität eintreffen.

Prioritätengesteuertes Scheduling kann wie in Abbildung 3.13 skizziert implementiert werden („*Multilevel Queueing*“): Für jede Priorität wird eine eigene Warteschlange eingerichtet, die die ausführungsbereiten Prozesse dieser Priorität enthält. Die „bereit“-Liste aus Abbildung 3.5 wird also in mehrere Listen aufgeteilt. Es wird jeweils ausschließlich die (nichtleere) Warteschlange mit der höchsten Priorität bedient. Innerhalb einer Warteschlange kann beispielsweise die Round-Robin-Strategie benutzt werden, aber auch andere Strategien sind denkbar. Prozesse wechseln zwischen den Warteschlangen, wenn sich ihre Prioritäten dynamisch ändern.

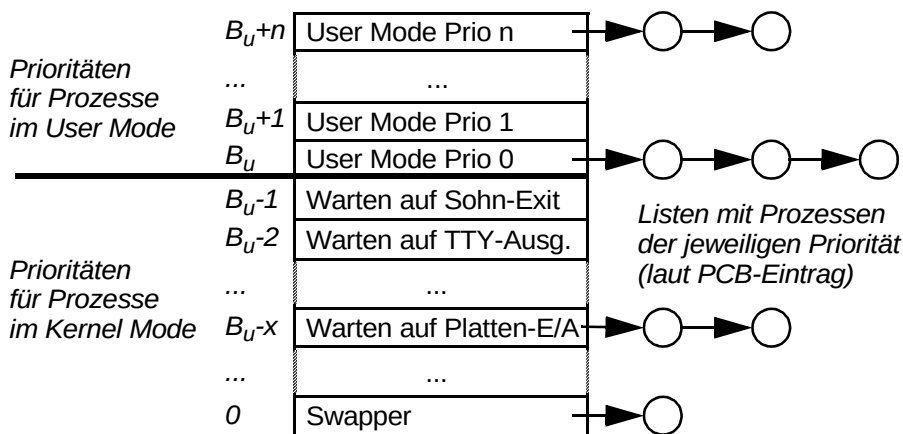


Abb. 3.14 Multilevel Queueing in UNIX System V

3.2.3 Prozesswechsel in UNIX / Linux

Die Schedulingstrategie von UNIX System V ist darauf ausgerichtet, interaktiven Prozessen möglichst kurze Antwortzeiten zu bieten. Das System soll also gegenüber Benutzern, die am Terminal interaktiv arbeiten, rasch reagieren können. Linux ermöglicht darüber hinaus, dringende „Realzeitprozesse“ zu benennen, die gegenüber „Time-Sharing-Prozessen“ ohne Realzeitanforderungen bevorzugt werden.

Scheduling in UNIX

UNIX System V implementiert ein prioritätengesteuertes Scheduling mit dynamischen Prioritäten: Je weniger ein Prozess in der jüngsten Vergangenheit den Prozessor belastet hat, desto höher ist seine Priorität. Damit sind interaktive Prozesse gegenüber Batchprozessen im Vorteil, da sie häufig relativ lange „Pausen“ machen, in denen sie auf Benutzereingaben warten.

Die Steuerung der Prozessorzuteilung in UNIX besteht im Wesentlichen aus zwei Komponenten – dem *Swapper* und dem *Scheduler*. Der Swapper lagert Prozesse, die momentan nicht ausgeführt werden sollen, auf den Plattenspeicher aus und liest dafür andere Prozesse in den Hauptspeicher ein. Die Vorgehensweise eines UNIX-Swappers wird in Unterkapitel 5.2 im Zusammenhang mit der Speicherverwaltung besprochen.

Der Scheduler verwaltet die Prozessprioritäten, indem er sie zu bestimmten Zeitpunkten neu berechnet, und bestimmt, welcher der ausführungsbereiten Prozesse jeweils auf dem Prozessor ausgeführt wird. Er stützt sich dabei auf die Prioritätseinträge der einzelnen Prozesse in der Prozesstabelle. Außerdem gibt es (wie im vorigen Abschnitt unter dem Stichwort „Multilevel Queueing“ dargestellt) für jede Priorität eine Warteschlange, die die Prozesse mit dieser Priorität enthält (siehe Abb. 3.14).

UNIX teilt die Prozessprioritäten in zwei *Prioritätsklassen* ein: Prozesse im User Mode haben Prioritätswerte über einem bestimmten Basiswert B_u , Prozesse im Kernel Mode haben Prioritätswerte darunter (siehe Abb. 3.14). Die Kernel-Mode-Prioritäten beziehen sich nur auf blockierte Prozesse (Ausnahme: Swapper) und werden durch die Ereignisse bestimmt, auf die die Prozesse warten (Details siehe später). Je kleiner der Prioritätswert ist, desto höher ist die Priorität, d.h. desto größer ist die Chance des Prozesses, den Prozessor zugeteilt zu bekommen.

Der UNIX-Scheduler wählt zur Ausführung jeweils den ersten Prozess in der Warteschlange aus, die unter den nichtleeren Schlangen die höchste Priorität hat. Dabei werden natürlich nur Prozesse berücksichtigt, die unmittelbar ausführungsbereit sind, die sich also im Hauptspeicher befinden und nicht blockiert sind.

Eine Neuauswahl des auszuführenden Prozesses, also eine Unterbrechung des gerade laufenden Prozesses und gegebenenfalls ein Umschalten des Prozessors zu einem neuen Prozess, findet zu den folgenden Zeitpunkten statt:

- Der aktuell ausgeführte Prozess blockiert sich oder terminiert.
- Das Zeitquantum des aktuell ausgeführten Prozesses (typischerweise 100 ms) ist abgelaufen.

Dieser Punkt betrifft allerdings nur Prozesse, die sich gerade im User Mode befinden. Ein Prozess im Kernel Mode wird bei Zeitablauf nicht unterbrochen. Damit wird die „Integrität“ (= Unverletzlichkeit) der Datenstrukturen des Kerns sichergestellt: Ein Prozess, der auf den Kerndaten arbeitet, braucht nicht zu befürchten, dass ihn ein anderer Prozess unterbricht und dann die Daten verändert. (In neueren UNIX-Versionen sind allerdings, zumindest bei Erreichen von bestimmten *Preemption Points* im Kern, Unterbrechungen gestattet, um das Umschalten nicht übermäßig lange zu verzögern.)

- Der aktuell ausgeführte Prozess kehrt aus dem Kernel Mode (also aus einem Systemaufruf oder der Behandlung eines so genannten Interrupts – siehe Unterkapitel 3.3) in den User Mode zurück.

Im Grunde genommen umfasst dieser Punkt auch den ersten Fall (Ablauf eines Zeitquantums). Beim Ablauf des Quantums kommt nämlich von der Systemuhr ein Signal (ein so genanntes „Interruptsignal“), worauf ein „Interrupthandler“ (= Kernfunktion zur Behandlung des Unterbrechungsereignisses) ausgeführt wird.

Der vom Prozessor verdrängte Prozess wird an das Ende seiner Prioritätswarteschlange oder, falls er sich blockiert hat, in die entsprechende Schlange für Kernel-Mode-Prozesse eingefügt. Innerhalb der einzelnen Prioritätsschlangen wird also das Round-Robin-Prinzip benutzt.

Die Prioritäten für die Prozesse im User Mode werden jede Sekunde neu berechnet (oder ein wenig später, falls der aktuell ausgeführte Prozess zur vollen Sekunde keine Unterbrechung durch die Systemuhr zulässt). Die neue Priorität hängt dabei, wie bereits angedeutet, wesentlich von der kürzlich verbrauchten Prozessorzeit ab: Jeder Prozess P_i hat zwei Einträge in der Prozesstabelle, nämlich $CPU_verbrauch(P_i)$, der die (gewichtete) bisherige Prozessorbelastung durch P_i angibt, und $prio(P_i)$, die aktu-

elle Priorität von P_i . Für Prozesse im User Mode gilt allgemein: Je größer $\text{CPU_verbrauch}(P_i)$ ist, desto größer ist auch $\text{prio}(P_i)$, desto niedriger ist also die Priorität.

Bei einer Neuberechnung der Priorität wird zunächst die gewichtete Prozessorbelastung aktualisiert, was beispielsweise anhand der folgenden Formel geschieht:

$$\begin{aligned} \text{CPU_verbrauch}(P_i)_{\text{neu}} \\ = \text{CPU_verbrauch}(P_i)_{\text{alt}} / \text{decay} \\ + \text{Prozessorbelastung in der vergangenen Sekunde} \end{aligned}$$

Der zweite Summand in der Formel wird in Zeiteinheiten angegeben, z.B. in Millisekunden. Er besagt, wie lange der Prozess in der vergangenen Sekunde den Prozessor besessen hat.

decay ist ein Wert, mit dem die weiter zurückliegende Prozessorbelastung gewichtet wird und der für alle Prozesse gleich ist. In der Formel wird also die bisherige Prozessorbelastung durch eine Gewichtungskonstante dividiert, und anschließend wird dazu die Belastung addiert, die in der letzten Sekunde beobachtet wurde. decay sollte dabei größer als 1 sein (ein typischer Wert ist 2), damit die Prozessorbelastung in der unmittelbaren Vergangenheit stärker ins Gewicht fällt als weiter zurückliegende Belastungen.

Anschließend wird die Priorität von P_i neu berechnet:

$$\begin{aligned} \text{prio}(P_i)_{\text{neu}} \\ = B_u + \text{norm} * \text{CPU_verbrauch}(P_i)_{\text{neu}} + \text{nice}(P_i) \end{aligned}$$

B_u , die Basispriorität, und norm , der Normalisierungsfaktor, sind jeweils für alle Prozesse gleich; sie dienen dazu, die Prioritäten auf einen bestimmten Wertebereich abzubilden. $\text{nice}(P_i)$ ist eine P_i zugeordnete Variable, mit der die Priorität zusätzlich beeinflusst werden kann. Normale Benutzer dürfen diesen Variablen nur nichtnegative Werte zuweisen, d.h. die Priorität freiwillig verschlechtern und somit „nice“ (nett) zu anderen Benutzern sein. Der Systemadministrator darf auch negative Werte zuweisen. In den meisten Fällen hat die nice -Variable aber den Wert 0.

Hat sich die Priorität eines Prozesses geändert, so wird sein PCB in die entsprechende neue Warteschlange eingefügt (siehe Abb. 3.14).

Die Priorität eines Prozesses im Kernel Mode hängt nicht von der Prozessorbelastung ab: Geht ein Prozess vom User Mode in den Kernel Mode über, so behält er zunächst seine User-Mode-Priorität. Diese ist zwar möglicherweise recht niedrig, spielt aber hier sowieso keine Rolle, da mit dem Übergang in den Kernel Mode der Prozess nicht mehr durch andere Prozesse unterbrochen werden kann (wie oben dargestellt). Der Prozess gibt also den Prozessor frühestens bei seiner Rückkehr in den User Mode wieder ab oder dann, wenn er sich innerhalb der Kernfunktion selbst blockiert.

Ein Prozess im Kernel Mode erhält nur eine neue Priorität, wenn er sich blockiert. Es handelt sich dann um eine Kernel-Mode-Priorität, die höher ist als alle User-Mode-Prioritäten. Wie hoch sie ist, hängt von dem Ereignis ab, auf das der Prozess wartet: Wartet er beispielsweise auf das Einlesen eines Plattenblocks und belegt damit das wertvolle Betriebsmittel „Plattenspeicher“, so erhält er eine hohe Priorität, damit er den

Plattenspeicher möglichst rasch wieder freigibt (siehe Abb. 3.14). Dagegen hat ein Prozess, der auf eine Benutzereingabe von der Tastatur wartet, eine niedrigere Priorität, da um ein Benutzerterminal weniger Prozesse konkurrieren als um den Plattenspeicher.

Mit der Rückkehr in den User Mode wird dem Prozess wieder eine User-Mode-Priorität zugewiesen.

Die meisten modernen UNIX-Systeme (z.B. UNIX System V Release 4, AIX, Solaris) unterstützen neben den dynamischen Prioritäten auch statische Prioritäten, d.h. Prioritäten, die sich nicht ändern. Sie sind in der Prioritätsordnung hoch angesiedelt und sorgen somit dafür, dass dringende Prozesse bei Bedarf bevorzugt ausgeführt werden können.

Scheduling in Linux

Das Scheduling in Linux basiert auf drei Schedulingklassen, die durch POSIX 1003.1b [Gall95] festgelegt werden: Prozesse in den Klassen `SCHED_FIFO` und `SCHED_RR` sind „Realzeitprozesse“ (d.h. zeitkritische, dringende Prozesse), Prozesse in der Klasse `SCHED_OTHER` sind „Time-Sharing-Prozesse“. Time-Sharing-Prozesse werden nur ausgeführt, wenn kein Realzeitprozess ausführungsbereit ist.

Ein Realzeitprozess P_i besitzt eine feste „Realzeitpriorität“ $rt_priority(P_i)$, die größer als 0 ist und vom Scheduler nicht geändert wird. Es wird jeweils derjenige der ausführungsbereiten Realzeitprozesse ausgewählt, der die höchste Realzeitpriorität besitzt. Bei gleicher Priorität werden Prozesse in der Klasse `SCHED_FIFO` nach dem FIFO-Verfahren ausgeführt, d.h. der älteste Prozess erhält den Vorzug. In der Klasse `SCHED_RR` wird das Round-Robin-Verfahren benutzt, so dass hier Prozesse mit derselben Priorität gleichmäßig vorangetrieben werden.

Die Realzeitpriorität von Time-Sharing-Prozessen ist auf 0 festgelegt, so dass diese stets nachrangig gegenüber Realzeitprozessen behandelt werden. Das Scheduling innerhalb der Klasse der Time-Sharing-Prozesse beruht auf so genannten „Kreditpunkten“, die die Prozesse ansammeln und für ihre Ausführung benutzen können. Es wird jeweils derjenige ausführungsbreite Prozess ausgewählt, der momentan die meisten Kreditpunkte besitzt. Das Kreditpunktekonto eines Prozesses verringert sich bei seiner Ausführung ständig – durch Besitz des Prozessors werden also Punkte verbraucht.

Haben schließlich alle ausführungsbereiten Prozesse keine Kreditpunkte mehr, so werden neue Punkte vergeben. Jeder Time-Sharing-Prozess P_i erhält (auch wenn er momentan nicht ausführungsbereit ist) neue Kreditpunkte nach der folgenden Formel:

$$\text{counter}(P_i)_{\text{neu}} = \text{counter}(P_i)_{\text{alt}} / 2 + \text{priority}(P_i)$$

Die Anzahl der Kreditpunkte, die ein Prozess neu bekommt, wird durch seine „Priorität“ $\text{priority}(P_i)$ (rechter Summand) festgelegt – ein Wert, der mit der oben angeführten Realzeitpriorität $rt_priority(P_i)$ nicht verwechselt werden darf. Er kann benutzt werden, um bestimmten Prozessen einen Vorteil zu geben – ähnlich wie mit dem nice-Wert in UNIX System V. Beispielsweise können so interaktive Benutzerprozesse gegenüber Hintergrundprozessen bevorzugt werden.

Prozesse können zudem einen Teil ihrer alten Punkte „in die nächste Runde retten“ (linker Summand). Dies betrifft vor allem Prozesse, die sich oft im Wartezustand befinden und damit den Prozessor kaum belasten – insbesondere interaktive Benutzerprozesse. Sie sammeln so ein Punktekonto an und sind damit später, wenn sie wieder ausführungsbereit werden, gegenüber Prozessen mit viel Prozessorbelastung im Vorteil.

Die Auswahl des auszuführenden Prozesses und die Neuberechnung der Kreditpunkte sind auch in Linux Aufgabe des Schedulers. Er ist als Sammlung von Kernfunktionen implementiert, die – wie in UNIX – u.a. bei Timer-Interrupts und bei Übergang des laufenden Prozesses in den Wartezustand aufgerufen werden.

3.3 Traps und Interrupts

Während einer Programmausführung können Ereignisse eintreten, die eine spezielle Reaktion durch das System erfordern. Solche Ereignisse heißen Traps oder Interrupts und werden außerhalb des gerade laufenden Programms behandelt.

3.3.1 Grundkonzepte

Ein Ereignis kann „synchron“ oder „asynchron“ zur Ausführung des aktuellen Programms eintreten. *Synchrone Ereignisse*, so genannte *Traps*, werden durch das gerade ausgeführte Programm verursacht, befinden sich also auch in einem zeitlichen Zusammenhang („synchron“) mit dem Programm. Beispiele für Traps sind ein Überlauf bei einer arithmetischen Operation, eine Division durch 0 oder ein Fehler bei einem Speicherzugriff (z.B. falscher Adressbereich oder unzulässige Art des Zugriffs). Auch der Moduswechsel zwischen User Mode und Kernel Mode ist ein Trap-Ereignis, wie schon im Zusammenhang mit UNIX-Systemaufrufen besprochen.

Traps werden nicht durch das Benutzerprogramm, sondern durch die Hardware oder das Mikroprogramm des Systems entdeckt. Das hat zwei Vorteile: Erstens werden der Programmieraufwand und die Ausführungszeit eingespart, die bei einer expliziten Abfrage von Fehlermöglichkeiten durch das Programm anfielen. Zweitens ist es das System selbst, das auf das Ereignis reagiert. Das ist sicherer, als wenn man dem Benutzerprogramm die Behandlung kritischer Fehlerfälle überließe.

Asynchrone Ereignisse, so genannte *Interrupts*, stehen mit der aktuellen Programmausführung nicht unmittelbar in Zusammenhang, treten also „asynchron“ zum Programm auf. Beispiele für Interrupts sind ein Spannungsabfall im System, eine Meldung eines E/A-Geräts oder der Systemuhr sowie das Eintreffen eines Signals von einem anderen Prozess (z.B. durch die UNIX-Schnittstellenfunktion `kill()`). Auch bei Interrupts ist das System für Erkennung und Reaktion zuständig.

Interrupts können insbesondere bei der Kommunikation mit externen Geräten effizient eingesetzt werden: Sie verursachen hier weit weniger Aufwand als ein *Polling*, bei

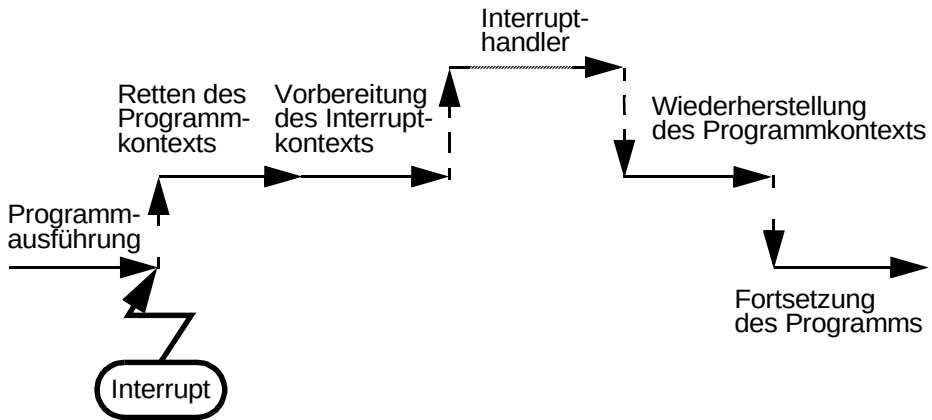


Abb. 3.15 zeitlicher Ablauf einer Interruptbehandlung

dem das Betriebssystem in bestimmten Zeitabständen die Geräte abfragt, ob etwas zur Bearbeitung ansteht.

Die Behandlung einer Trap oder eines Interrupts läuft allgemein in den folgenden Stufen ab (siehe Abb. 3.15): Die aktuelle Programmausführung wird unterbrochen. Eventuell wird ihr Kontext gerettet (zumindest die Inhalte des Befehlszähler- und des Prozessorstatus-Registers) und ein Interruptkontext eingerichtet – also ein Kontext, in dem die Behandlung des Ereignisses stattfinden soll. Eine Funktion, die spezifisch für das Ereignis ist (*Interrupthandler*, *Interrupt Service Routine*), wird ausgeführt. Anschließend wird das unterbrochene Programm an der Unterbrechungsstelle in seinem ursprünglichen Kontext fortgesetzt. Eine Fortsetzung des Programms ist natürlich nicht möglich, wenn das Ereignis ein nichtbehebbarer Programmfehler war, wie z.B. eine Division durch 0.

Interrupts und Traps spielen in modernen Betriebssystemen eine zentrale Rolle, da die Aktionen des Systems zumeist durch sie ausgelöst werden. Man spricht daher auch von „Interrupt-getriebenen“ Systemen.

3.3.2 Implementierungsaspekte

Das Interruptkonzept kann nur mit Unterstützung der Prozessorhardware realisiert werden: Der Prozessor besitzt einen speziellen *Interrupteingang*, bei dessen Aktivierung er die aktuelle Programmausführung unterbricht. Zudem ist er in der Lage, eine genauere Identifikation des Ereignisses vom Bus zu lesen und zu verarbeiten.

Im Detail sieht das Zusammenspiel zwischen Hardware und Betriebssystemsoftware bei der Interruptbehandlung folgendermaßen aus: Hat der Prozessor ein Interruptsignal über seinen Interrupteingang empfangen, so liest er zunächst eine genauere Identifikation des Ereignisses (den so genannten *Interruptvektor*) vom Bus und rettet die aktuellen Inhalte des Befehlszähler- und des Statusregisters. Er benutzt dann den Interruptvektor als Index in eine *Interrupttabelle*, wo er die Startadresse des zugehörigen Inter-

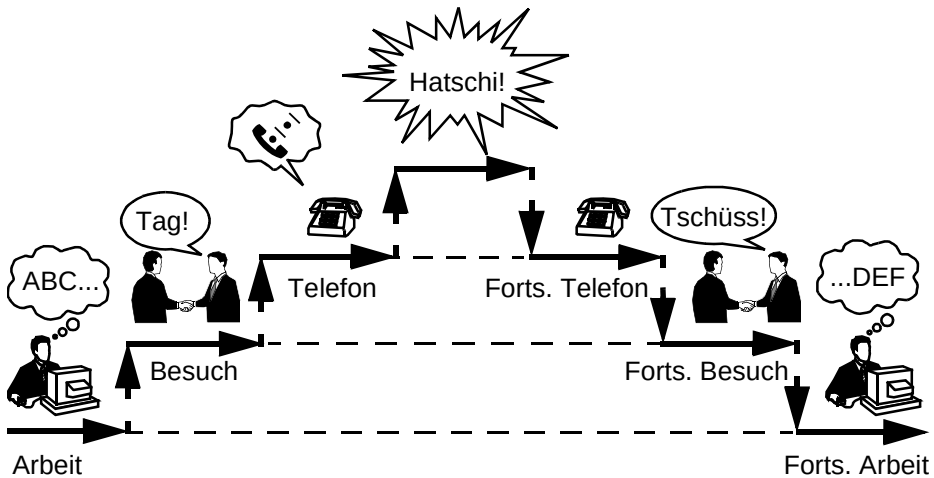


Abb. 3.16 geschachtelte Interrupts – ein Analogbeispiel

rupthandlers findet. Diese Startadresse wird in den Befehlszähler geladen und somit die Kontrolle an die Software des Interrupthandlers übergeben.

Sofern ein Kontextwechsel gewünscht wird, rettet der Interrupthandler den restlichen Programmkontext und initialisiert einen Interruptkontext. Er beschafft dann nähere Informationen über das Ereignis (z.B. verursachendes Gerät, Zustand dieses Geräts) und führt Aktionen zur Reaktion auf das Ereignis aus. Schließlich kehrt er mit einem speziellen Maschinenbefehl (z.B. RTI = return from interrupt), der Befehlszähler- und Statusregister mit den geretteten Werten lädt und ggf. den alten Kontext wiederherstellt, in das unterbrochene Programm zurück.

3.3.3 Schachtelung von Interruptbehandlungen

Offensichtlich können während der Behandlung einer Trap oder eines Interrupts neue Ereignisse eintreten, auf die ebenfalls reagiert werden muss. Zudem sind die Ereignisse möglicherweise unterschiedlich wichtig, so dass es manchmal sinnvoll sein kann, die Ausführung eines Interrupthandlers zu unterbrechen, um zunächst auf ein dringenderes Ereignis zu reagieren. Natürlich soll die Behandlung des unwichtigeren Ereignisses später wieder aufgenommen werden. Abbildung 3.16 illustriert ein entsprechendes Beispiel aus dem täglichen Leben.

Interruptprioritäten

Um Ereignisse unterschiedlicher Dringlichkeit bearbeiten zu können, ist in der Hardware ein *Prioritätenmechanismus für Interrupts* implementiert. Jedem Ereignis ist eine Priorität zugeordnet, die angibt, wie eilig seine Behandlung ist. Im Unterschied zu den

Prozessprioritäten bezeichnet hier ein hoher Prioritätswert im Allgemeinen eine große Dringlichkeit. „Normale“ Programme, die keine Interrupthandler sind, haben die Interruptpriorität 0.

Grundlage der prioritätengesteuerten Interruptbehandlung ist ein *Unterbrechungsspeicherregister*, das angibt, für welche Prioritäten Interrupts zur Bearbeitung anstehen. Das Register enthält für jede Priorität ein Bit. Tritt ein Interrupt mit einer bestimmten Priorität ein, so wird das entsprechende Bit auf 1 gesetzt; beginnt die Ausführung des zugehörigen Interrupthandlers, so wird es auf 0 zurückgesetzt.

Es wird jeweils das Ereignis mit der höchsten Priorität behandelt. Tritt also während der Ausführung eines Handlers ein Ereignis mit niedrigerer (oder auch gleicher) Priorität ein, so wird es registriert, indem das entsprechende Registerbit gesetzt wird, und nach der Beendigung der aktuellen Handlerausführung seinerseits behandelt (es sei denn, es stehen dann noch andere höherpriorie Ereignisse an). Hat dagegen das neue Ereignis eine höhere Priorität, so wird der aktuelle Handler unterbrochen und eine Beschreibung seines aktuellen Zustands (insbesondere der Befehlszähler) auf einem Interruptstack abgelegt. Der dringendere Handler für das neue Ereignis wird vollständig ausgeführt und danach der unterbrochene Handler wieder aufgenommen. Wegen des Stacks können Ausführungen von Interrupthandlern prinzipiell beliebig tief geschachtelt werden – genauso wie der Aufrufstack eines Programms Funktionsaufrufe mit beliebiger Schachteltiefe ermöglicht.

Da das Unterbrechungsspeicherregister pro Interruptpriorität nur ein Bit enthält, können Interrupts verloren gehen, wenn rasch hintereinander mehrere Ereignisse mit derselben Priorität eintreten. In der Praxis kommt das aber nur selten vor.

Die folgende Tabelle demonstriert an einem Beispiel den zeitlichen Ablauf bei der Behandlung einer Interruptfolge. Es werden drei Prioritäten angenommen. Der mit * gekennzeichnete Interrupt geht verloren.

Ereignis	behandelter Interrupt	Unterbr.speicherreg. (I_1, I_2, I_3)
Benutzerprog. läuft	-	0 0 0
I_3 tritt ein	I_3	0 0 0
I_2 tritt ein	I_3	0 1 0
I_1 tritt ein	I_3	1 1 0
I_3 beendet	I_2	1 0 0
I_1 (neu) tritt ein *	I_2	1 0 0
I_2 beendet	I_1	0 0 0
I_1 beendet	-	0 0 0

Maskierung von Interrupts

Zusätzlich zum Unterbrechungsspeicherregister gibt es ein *Unterbrechungsmaskierungsregister*. Es erlaubt, das Auftreten von Interrupts einer bestimmten Priorität zu ignorieren. Pro Priorität ist ein Bit vorhanden. Steht es auf 0, so wird auf eintreffende Interrupts dieser Priorität nicht reagiert. Allerdings kann ein solcher Interrupt im Unterbrechungsspeicherregister registriert werden, um seine Behandlung nachholen zu können, wenn das Bit im Maskierungsregister wieder auf 1 gesetzt wird.

Linux benutzt die Maskierung von Interrupts, um die Behandlung von „schnellen“ Interrupts zu beschleunigen: Bei der Ausführung des Handlers für einen solchen Interrupt werden alle anderen Interrupts gesperrt; der Handler hat dabei nur eine kurze Laufzeit. Die Behandlung „langsamer“ Interrupts mit längeren Laufzeiten kann dagegen unterbrochen werden.

Bei der Programmierung von Interrupt-handlern muss streng darauf geachtet werden, dass ihre Ausführungsdauer sehr kurz ist. Dadurch soll sichergestellt werden, dass weitere wartende Interrupts, die ja sehr zeitkritisch sind, nicht zu lange auf ihre Behandlung warten müssen. Zudem verringert sich die Gefahr, dass Interrupts verloren gehen.

3.4 Anforderungen der Praxis

In vielen praktischen Anwendungsgebieten kommt es darauf an, dass das System möglichst immer verfügbar ist und zudem bei der Ablieferung seiner Ergebnisse eine vorgegebene Frist einhält – man denke beispielsweise an die Steuerung von Fahrzeugen oder Produktionsanlagen. Hierfür sind zwei Themenbereiche wichtig, nämlich Realzeit und Fehlertoleranz.

3.4.1 Realzeit

Realzeitsysteme werden in zeitkritischen Anwendungsbereichen eingesetzt, so (wie erwähnt) in der Prozess- und Anlagensteuerung sowie zur Verarbeitung und Übertragung zeitabhängiger Daten, wie beispielsweise Audio- und Videoströme. Von Realzeitsystemen wird allgemein gefordert, dass sie innerhalb einer vorgegebenen Zeitspanne auf bestimmte Ereignisse reagieren. Dabei ist möglicherweise nicht von vornherein bekannt, wann diese Ereignisse eintreten. Man unterscheidet zwischen harten und weichen Realzeitanforderungen:

- Bei einer *harten Realzeitanforderung* ist das Ergebnis völlig unbrauchbar, wenn es zu spät eintrifft. Dies ist z.B. meist dann der Fall, wenn schnell auf eine Ausnahmesituation reagiert werden muss – ein Extremfall ist hier die Notabschaltung eines Kernkraftwerks.

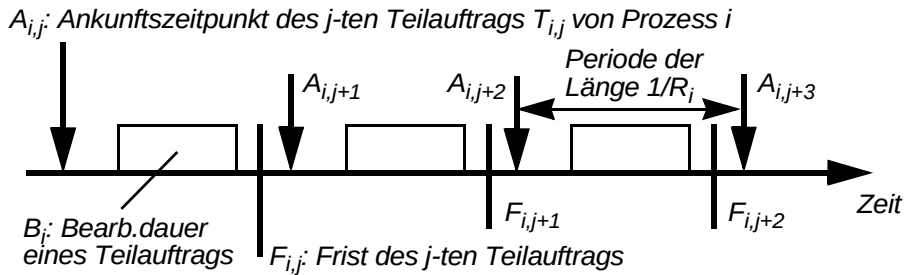


Abb. 3.17 periodisches Prozessmodell

- Bei einer *weichen Realzeitanforderung* wird das Ergebnis graduell immer weniger wert, je später es vorliegt. Ein Beispiel ist hier die Übertragung eines Videostroms: Ein zu spät eintreffendes Einzelbild führt zum „Ruckeln“ bei der Ausgabe, was aber toleriert wird, sofern die Störung nicht zu stark ist und nicht zu häufig auftritt.

Verfahren zum Realzeitscheduling

In Realzeitsystemen spielen CPU-Schedulingverfahren eine zentrale Rolle: Der Prozessor muss den Prozessen so zugeteilt werden, dass keiner von ihnen seine *Frist* (engl. *deadline*) überschreitet.

Der einfachste Ansatz ist hier, zeitkritischen Prozessen eine hohe statische Priorität zu geben – je dringender, desto höher. Sie werden damit gegenüber den anderen Prozessen bevorzugt, so dass die Chance steigt, dass sie mit ihrer Aufgabe rechtzeitig fertig werden. Linux mit seinen Realzeitprioritäten (siehe Abschnitt 3.2.3) unterstützt diesen Ansatz; Ähnliches findet sich in anderen UNIX-Implementationen und in Windows NT.

Die Zuteilung von hohen Prioritäten allein ist jedoch nur ein „Best-Effort“-Ansatz: Das System tut sein „Bestmögliches“, kann aber nicht hundertprozentig garantieren, dass die geforderten Fristen stets eingehalten werden. Es sind also weitere Mechanismen erforderlich, um entsprechende *Realzeitgarantien* abgeben zu können.

Realzeitgarantien lassen sich relativ leicht für ein System berechnen, in dem die Prozesse periodisch, also in festen Zeitabständen ausgeführt werden. Solche periodischen Prozesse findet man beispielsweise in einem Multimediacomputer, in dem Einheiten eines Datenstroms zu bearbeiten sind – z.B. ein Videostrom mit 25 Einzelbildern pro Sekunde, die im festen Abstand von 40 Millisekunden eintreffen.

Abbildung 3.17 illustriert das Verhalten eines periodischen Prozesses P_i : P_i zerfällt in Teilaufträge $T_{i,j}$, die mit einer festen Rate R_i im System eintreffen – also R_i Teilaufträge pro Zeiteinheit. Der zeitliche Abstand zwischen je zwei benachbarten Ankunftszeitpunkten $A_{i,j}$ und $A_{i,j+1}$ beträgt konstant $1/R_i$ Zeiteinheiten und wird als *Periode* bezeichnet. Jeder Teilauftrag $T_{i,j}$ benötigt den Prozessor für eine feste Bearbeitungszeit B_i . Gefordert wird, dass $T_{i,j}$ zum Zeitpunkt $F_{i,j}$ (der Frist) vollständig ausgeführt ist.

In einem System, in dem mehrere periodische Prozesse vorhanden sind, sind u.a. die folgenden beiden Schedulingstrategien möglich:

- *Fristenbasiertes Scheduling* (engl. *earliest deadline first*): Es wird jeweils der Prozess ausgeführt, dessen Frist unter den Fristen aller Prozesse am nächsten liegt.
- *Ratenmonotones Scheduling* (engl. *rate-monotonic scheduling*): Es wird jeweils der Prozess mit der höchsten Rate, d.h. mit der kürzesten Periode ausgeführt.

Für diese Schedulingstrategien lassen sich die folgenden Aussagen machen [LiLa73]:

- In einem System seien n periodische Prozesse P_i ($1 \leq i \leq n$) vorhanden mit Raten R_i und Bearbeitungsdauern B_i je Teilauftrag. Für die Frist $F_{i,j}$ eines Teilauftrags gelte $F_{i,j} = A_{i,j+1}$.
Unter *fristenbasiertem* unterbrechendem Scheduling werden alle Fristen eingehalten, wenn gilt: $\sum_{i=1..n} B_i * R_i \leq 1$.
- Unter denselben Voraussetzungen und *ratenmonotonem* unterbrechendem Scheduling werden die Fristen eingehalten, wenn gilt: $\sum_{i=1..n} B_i * R_i \leq \ln(2)$, mit $\ln(2) \approx 0,69$.

Fallen also die Fristen jeweils mit den Ankunftszeitpunkten der nächsten Teilaufträge zusammen, so hält sie fristenbasiertes Scheduling stets garantiert ein – selbst bei einer Systemauslastung von 100%. Bei ratenmonotonem Scheduling darf dagegen eine Auslastungsgrenze von ca. 69% nicht überschritten werden, um eine solche Garantie abgeben zu können. Ratenmonotones Scheduling ist also restriktiver als fristenbasiertes, hat aber den Vorteil, dass hier die Prioritäten fest bleiben, solange sich die Raten nicht ändern. Es lässt sich also mit Hilfe fester Realzeitprioritäten (wie z.B. in Linux) wesentlich leichter implementieren als fristenbasiertes Scheduling, wo die Prioritäten – je nach Nähe der jeweiligen Fristen – kurzfristig dynamisch wechseln.

In Systemen mit nichtperiodischem Verhalten lassen sich die genannten Strategien nur sehr eingeschränkt einsetzen. Im Fall von weichen Realzeitanforderungen kann man hier mit Hilfe stochastischer Methoden Wahrscheinlichkeiten für die Einhaltung der Fristen berechnen. Im Fall harter Forderungen, wo „einhundertprozentige“ Garantien verlangt werden, macht man „Worst-Case-Betrachtungen“: Es wird für den schlimmstmöglichen Fall (was das Eintreten kritischer Ereignisse betrifft) ein Ausführungsplan erstellt, der die Reihenfolge der Reaktionen festlegt und an dem somit ablesbar ist, wann die Ausführungen spätestens abgeschlossen sein werden.

Allgemeine Eigenschaften von Realzeitsystemen

Ein Betriebssystem sollte, neben dem Realzeitscheduling, eine Reihe weiterer Eigenschaften besitzen, um eine zeitgerechte Reaktion auf Ereignisse zu ermöglichen. Wir fassen die wichtigsten dieser Eigenschaften im folgenden Katalog zusammen:

- Wie im vorigen Teilabschnitt erörtert, sollten hohe Prozessprioritäten statisch, also fest zuteilbar sein. Damit lassen sich zeitkritische Prozesse gegenüber sonstigen Pro-

zessen bevorzugen, ohne dass andere Kriterien wie beispielsweise die Prozessorbelastung (siehe UNIX) einen Einfluss haben. In Multiprozessorsystemen können ein oder mehrere Prozessoren ausschließlich für hochpriorie Realzeitprozesse verwendet werden.

- Die Funktionen des Betriebssystemkerns sollten unterbrechbar sein, um den Prozessor auch dann zu zeitkritischen Prozessen umschalten zu können, wenn sich ein anderer Prozess gerade im Kern befindet.
- Die Umschaltzeit zwischen Prozessen sollte kurz sein, um die Ausführung eines zeitkritischen Prozesses sehr schnell beginnen zu können.
- Die Reaktionszeit auf einen Interrupt, also die Zeitspanne zwischen dem Eintreffen des Signals und dem Start des Interrupthandlers, sollte kurz sein.
- Zeitgeber („Timer“) mit einer feinen Auflösung sollten implementiert sein, um Aktionen zeitlich exakt ausführen und überwachen zu können. Erforderlich ist eine Auflösung von einer Millisekunde oder möglichst noch kleiner.
- Zeitkritischer Code und zeitkritische Daten sollten fest im Hauptspeicher abgelegt werden können („*code / data pinning*“). Im Unterschied zu „normalem“ Code bzw. „normalen“ Daten können sie also nicht auf den Plattenspeicher verdrängt werden, so dass keine Zeitverluste durch Aus- und Einlagern auftreten (siehe hierzu auch Kapitel 5).
- Das „Plattenlayout“, d.h. die Positionierung der Dateien auf der Platte, sollte optimiert werden, um rasche Dateizugriffe zu ermöglichen (siehe hierzu auch Abschnitte 5.1.3 und 6.1.4).
- Ein Realzeitsystem sollte insgesamt nur schwach ausgelastet sein, damit bei Bedarf Betriebsmittel in ausreichender Zahl vorhanden sind und zeitkritische Prozesse sich nicht gegenseitig behindern.

Realzeitunterstützung in modernen Betriebssystemen

Moderne Betriebssysteme, wie z.B. verschiedene UNIX-Implementationen oder Windows NT / 2000, berücksichtigen einige oder alle der Forderungen des Katalogs und unterstützen damit die Durchsetzung von weichen Realzeitanforderungen. POSIX 1003.1b (früher: POSIX.4) standardisiert eine entsprechende Erweiterung der Programmierschnittstelle [Gall95].

Im Fall von harten Realzeitanforderungen wird in der Praxis dagegen spezielle Anwendungs- und Betriebssystemsoftware auf dedizierter Hardware eingesetzt, und es wird durch eine „Worst-Case“-Prozessplanung, wie zuvor dargestellt, die Einhaltung der Fristen garantiert.

3.4.2 Fehlertoleranz

Ein System heißt *fehlertolerant*, wenn es selbst dann noch korrekte Ergebnisse liefert, wenn ein Teil seiner Komponenten nicht mehr korrekt oder überhaupt nicht mehr arbeitet. Fehlertoleranz beruht auf *Redundanz*, d.h. auf mehrfachem Vorhandensein von Hard- und Softwarekomponenten sowie auf Daten mit Fehlererkennungs- und -korrekturcodes.

Bezogen auf die Ausführung von Prozessen bedeutet Redundanz, dass relevante Hardwarebausteine, wie insbesondere Prozessoren oder Speicher, in mehreren Exemplaren implementiert sind. Bei *passiver Redundanz* springt eine Reservekomponente erst bei Ausfall der Hauptkomponente ein („*stand-by*“). Bei *aktiver Redundanz* arbeiten mehrere Komponenten nebenläufig auf denselben Eingabedaten, und ihre Resultate werden anschließend miteinander verglichen. Unterscheiden sie sich, so kann das am häufigsten gemeldete Resultat weiterverwendet werden („*voting*“). Entsprechende Techniken werden beispielsweise in der Raumfahrt eingesetzt.

Redundanz auf Softwareebene bedeutet, dass mehrere Softwarepakete auf dieselben Eingabedaten angewandt und ihre Ergebnisse miteinander verglichen werden, wobei wiederum das Voting-Prinzip benutzt wird. Die Softwarepakete sollten so unabhängig wie möglich voneinander sein um zu verhindern, dass derselbe Fehler in allen Programmen auftritt. Insbesondere sollten sie von verschiedenen Programmerteams erstellt worden sein. Diese Vorsichtsmaßnahme ist allerdings kein Allheilmittel, denn es gibt einige typische Fehler, die jeder Programmierer „gern“ macht und die daher mit hoher Wahrscheinlichkeit in mehreren Programmen auftreten. Sinnvoll wäre es also, die Korrektheit der Software streng formal nachzuweisen, was allerdings recht kompliziert ist.

3.5 Übungsaufgaben

1. Wissensfragen

- a.) Was gehört zu einem Prozess?
- b.) Was ist der Unterschied zwischen Prozessen und Threads?
- c.) Wie werden Prozesse innerhalb des Betriebssystems dargestellt, und wie werden sie identifiziert?
- d.) Was macht die UNIX-Funktion `fork()` und welchen Wert (genauer: welche Werte) gibt sie zurück?
- e.) Welche zwei Klassen von Schedulingstrategien unterscheidet man?
- f.) Was ist der Unterschied zwischen festen und dynamischen Prioritäten?
- g.) Was ist die Gemeinsamkeit von Traps und Interrupts, was ist der Unterschied?
- h.) Was ist der Unterschied zwischen harten und weichen Realzeitanforderungen?

2. Funktionen der UNIX-C-Schnittstelle zur Prozessverwaltung

Zur Lösung der folgenden Aufgaben können Sie auch den Anhang benutzen, in dem die Funktionen der UNIX-C-Schnittstelle näher beschrieben werden.

a.) Betrachten Sie den folgenden Programmausschnitt:

```
int p;
if ((p=fork())!=0) {
    printf("Ich bin der Sohn\n");
    printf("Meine PID ist %d\n",p);
}
else {
    printf("Ich bin der Vater\n");
    printf("PID meines Sohns ist %d\n",p);
}
```

- Sind die Aussagen „Ich bin der Sohn“ und „Ich bin der Vater“ im ersten bzw. dritten `printf()`-Aufruf so korrekt? Begründung!
- Ist die Verwendung von `p` im zweiten bzw. vierten `printf()`-Aufruf so korrekt? Begründung!

b.) Betrachten Sie nun das folgende Programm:

```
main() {
    int i=10;
    if (fork()==0)
        while (1) {
            sleep(2);
            i=i+10;
        }
    sleep(2);
    printf("i = %d",i);
}
```

Ist die Bildschirmausgabe des Programms eindeutig? Wenn nein: Warum nicht?
Wenn ja: Wie lautet sie?

c.) Betrachten Sie jetzt dieses C-Programm:

```
main() {
    int i, status;
    for (i=0;i<3;i++)
        if (fork()==0) {
            sleep(i+1);
            printf("Sohn Nr. %d wird jetzt fertig.\n",i);
        }
```

```

    exit(0);
}
wait(&status);
printf("Hier ist der Vater.\n");
printf("Alle meine Söhne sind fertig\n");
}

```

- Welchen Effekt hat die for-Schleife?
- Ist die Aussage der printf()-Meldungen so korrekt? Wenn nein, wie müsste man das Programm korrigieren?

d.) Betrachten Sie nun den folgenden Programmausschnitt:

```

int n=3;
...
for (i=0;i<n;i++)
    if (fork()==0) {
        printf("Hallo, hier bin ich!\n");
        exit(0);
    }
printf("Gute Nacht!\n");
sleep(3600);

```

- Wie oft wird „Hallo, ...“ auf den Bildschirm ausgegeben und wie oft „Gute Nacht“?
- Streichen Sie jetzt den exit()-Aufruf. Warum werden die Texte nun öfter ausgegeben?
- Warum ist also das exit() im Programm wichtig in Hinblick auf die Arbeitsfähigkeit des gesamten Systems? (Hinweis: Überlegen Sie sich, was für große Werte von n passiert!). Was kann das Betriebssystem tun um zu verhindern, dass Benutzer mit solchen Programmen Probleme verursachen?

e.) Gegeben ist das folgende einfache Problem: Ein UNIX-Vaterprozess soll in einer for-Schleife zehn Sohnprozesse erzeugen. Jeder dieser Sohnprozesse soll nur eine einzige Operation ausführen, nämlich den Wert ausgeben, den der Schleifenzähler bei seiner Erzeugung hatte. Die Reihenfolge der Ausgaben ist dabei beliebig. Es wird die folgende Lösung vorgeschlagen:

```

main() {
    int i;
    for (i=0;i<10;i++)
        if (fork()==0)
            printf("Schleifenzähler = %d\n",i); }

```

Ist diese Lösung korrekt? Wenn nein: Wie müsste sie korrigiert werden, um den gewünschten Effekt zu erzielen?

3. Strategien zum Prozessorscheduling

In dieser Aufgabe geht es um das Scheduling der CPU. Betrachtet werden die folgenden Strategien:

- FCFS: First Come First Served
- FP_nu: feste Prioritäten, nicht unterbrechend
- FP_u: feste Prioritäten, unterbrechend
- RR_100: Round Robin, Zeitscheibe = 100 Millisekunden (ms)

Gegeben sind 4 Prozesse mit Ankunftszeiten (in ms nach Zeitpunkt 0), Ausführungsdauern (in ms) und Prioritäten gemäß der folgenden Tabelle:

Prozessname	Ankunftszeit	Ausführungsdauer	Priorität
A	0	150	4
B	50	200	3
C	80	300	2
D	160	100	1

Es gilt Folgendes:

- Je größer die Prioritätszahl, desto niedriger ist die Dringlichkeit.
 - Bei RR_100 werden in einem Zyklus die Prozesse in der Reihenfolge A, B, C, D ausgeführt. Ein Umschalten zu einem anderen Prozess ist bei RR_100 nur am Ende einer Zeitscheibe oder Terminierung eines Prozesses möglich.
 - Bei jedem Wechsel der CPU zu einem anderen Prozess, auch beim Start des ersten Prozesses, fällt eine Umschaltzeit von 1 ms an. Wird bei RR_100 der laufende Prozess am Ende einer Zeitscheibe unterbrochen, dann aber unmittelbar fortgesetzt, fällt keine Umschaltzeit an. Die Zeitscheibe steht immer voll für die Ausführung zur Verfügung; die Umschaltungen finden gesondert statt.
- a.) Geben Sie für jede der vier Strategien an, wann die CPU welchen Prozess bearbeitet und wann umgeschaltet wird.
- b.) Berechnen Sie, für jede der Strategien gesondert, die folgenden Werte:
- Aufenthaltsdauern der einzelnen Prozesse (= Zeitdauer von der jeweiligen Ankunft bis zur Fertigstellung) in ms.
 - Gesamtdauer der Umschaltvorgänge, absolut in ms und als prozentualer Anteil an der Gesamtdauer (Gesamtdauer = Dauer vom Zeitpunkt 0 bis zur Fertigstellung des letzten Auftrags).

4. Prozessorscheduling unter Round Robin

Betrachten Sie die Round-Robin-Strategie. Es sollen folgende Vereinbarungen gelten:

- z bezeichnet die maximale Länge einer Zeitscheibe. „Maximale Länge“ bedeutet dabei: Wird ein Prozess vor Ende seiner Zeitscheibe fertig, so wird sofort zum nächsten Prozess umgeschaltet, wobei eine neue Zeitscheibe beginnt. Ansonsten bekommt er aber die volle Zeitscheibe. z bezeichnet die reine Bearbeitungszeit; Prozessumschaltungen finden also außerhalb dieser Zeit statt.
- u bezeichne die Zeitdauer, um die CPU von einem Prozess zu einem anderen umzuschalten. Auch für das Aufsetzen des ersten Prozesses auf anfangs freier CPU wird eine Zeitdauer u benötigt. Wird in zwei aufeinander folgenden Zeitscheiben derselbe Prozess bearbeitet, so fällt keine Umschaltung und damit auch kein Zeitaufwand an.
- t_p bezeichne den Gesamtbedarf des Prozesses P an reiner Bearbeitungszeit, d.h. ohne Umschaltzeiten.

a.) Stellen Sie eine allgemeine Formel auf, die für einen Prozess P in Abhängigkeit von t_p und z angibt, wie viele Zeitscheiben (auch angebrochene) er benötigt.

b.) Lässt sich eine entsprechende einfache Formel aufstellen, die die Anzahl der Umschaltungen zu P berechnet? Wenn ja, schreiben Sie sie hin; wenn nein, begründen Sie, warum nicht.

c.) Stellen Sie eine allgemeine Formel für den Abgangszeitpunkt des Prozesses $P_{\min}$ auf, der die kürzeste Zeitanforderung $t_{\min}$ aller Prozesse hat. Der Abgangszeitpunkt ist der Zeitpunkt, zu dem die Bearbeitung des Prozesses fertig ist. Zur Vereinfachung nehmen wir an, dass die Zeitanforderungen der Prozesse durch die Zeitscheibenlänge ganzzahlig teilbar sind, dass alle Prozesse (n an der Zahl) zum Zeitpunkt 0 erstmals zur Bedienung anstehen und dass $P_{\min}$ in einem Zyklus jeweils als Erster bedient wird.

d.) Es seien im System die Prozesse A, B und C vorhanden mit $t_A=100$, $t_B=60$ und $t_C=25$. Im Zyklus werden die Prozesse in der Reihenfolge A, B, C ausgeführt. Die Umschaltzeit betrage $u=5$.

Betrachten Sie drei verschiedene Zeitscheibenlängen: $z_1=50$, $z_2=25$ und $z_3=5$.

Geben Sie für z_1 und z_2 jeweils an, wann die CPU welchen Prozess bedient und wann Umschaltungen stattfinden.

Geben Sie, für z_1 , z_2 und z_3 getrennt, jeweils die Abgangszeitpunkte der drei Prozesse und das arithmetische Mittel dieser drei Zeitpunkte (die „mittlere Verweilzeit“) an.

e.) In der vorherigen Teilaufgabe stellten Sie fest, dass die mittlere Verweilzeit bei z_2 gegenüber der bei z_1 sinkt und die bei z_3 gegenüber der von z_2 steigt. Wodurch erklären sich diese Effekte jeweils?

f.) Wieso sollte in einem interaktiven System mit vielen Benutzern und zusätzlichen Hintergrundprozessen die Zeitscheibe relativ kurz sein?

5. Prozessorscheduling unter UNIX

a.) Wozu dient in UNIX der `nice`-Wert? Warum dürfen normale Benutzer die `nice`-Werte ihrer Prozesse nur auf nichtnegative Werte setzen?

b.) Betrachten Sie zwei UNIX-Prozesse A und B. Einer der Prozesse sei die Shell eines langsamen Benutzers, und der andere führe ein Programm aus, das zunächst einige wenige Daten von einer Datei einliest und dann damit eine komplizierte Berechnung ausführt, die mehrere Sekunden dauert.

Die beobachteten Prioritätswerte von Prozess A während fünf aufeinander folgender Sekunden seien 60, 100, 108, 112, 114, und die Priorität von Prozess B sei während desselben Zeitraums konstant 60. Ein hoher Zahlenwert soll dabei eine niedrige Priorität anzeigen, also eine geringe Chance, die CPU zu bekommen.

Welcher der beiden Prozesse A und B ist höchstwahrscheinlich die Shell und welcher das Berechnungsprogramm? Begründen Sie Ihre Antwort!

c.) Betrachten Sie den folgenden Programmrahmen: Ein Vaterprozess wartet hier auf ein Signal. Trifft es ein, so wird ein Sohn erzeugt, der darauf innerhalb einer bestimmten Frist reagieren muss.

```
main() {
    while (1) {
        ... /* Warten, bis Signal eintrifft. */
        if (fork()==0) {
            ...
            /* Der Sohn führt hier ein Programm aus,
               das 10 s reine CPU-Zeit benötigt
               und innerhalb von 11 s fertig sein muss. */
            exit(0);
        }
    }
}
```

Kann man erwarten, dass auf einem UNIX-System, in dem nur das in Abschnitt 3.2.3 dargestellte Scheduling benutzt wird, die im Kommentar formulierte Forderung eingehalten wird? Wenn ja, warum; wenn nein, was benötigt man beim Scheduling noch zusätzlich?

d.) Gegeben sind drei Prozesse P_1 , P_2 und P_3 mit den folgenden Anfangsprioritäten (zum Zeitpunkt 0 = Systemstartzeit) und `nice`-Werten:

	P_1	P_2	P_3
Anfangspriorität	70	80	60
<code>nice</code> -Wert	10	20	0

Der CPU-Zeit-Verbrauch sei in den ersten vier Sekunden wie folgt (Werte in ms):

	P_1	P_2	P_3
1. Sekunde	320	120	200
2. Sekunde	80	180	380
3. Sekunde	40	200	320
4. Sekunde	0	480	520

Die Konstantenwerte sind $\text{decay} = 2$, $B_u = 60$ und $\text{norm} = 0.125$.

Geben Sie an, welche Werte sich für den gewichteten CPU-Zeit-Verbrauch $\text{CPU_verbrauch}(P_i)$ und die Priorität $\text{prio}(P_i)$ nach der 1., 2., 3. und 4. Sekunde ergeben. Benutzen Sie bei den Berechnungen die Zeitwerte in ms.

Weitere Aufgaben zur Prozessbenutzung unter UNIX finden sich im Aufgabenteil des Anhangs.

4 Prozesssynchronisation und -kommunikation

Im vorigen Kapitel ging es um relativ einfache Aspekte der Verwaltung von Prozessen und Threads: Es wurde diskutiert, wie diese Aktivitäten intern dargestellt werden und wie ihnen der Prozessor zugeteilt wird. Ein Betriebssystem muss sich aber nicht nur um diese hardwarenahe Verwaltung kümmern, sondern es muss das „Zusammenleben“ der Aktivitäten auch auf höherer Ebene regeln. Dazu gehören die *Synchronisation* von Prozessen / Threads, also die Durchsetzung von zeitlichen Bedingungen bei ihrer Ausführung, und die *Kommunikation*, also der Datenaustausch zwischen Prozessen / Threads.

Aus Gründen der Einfachheit werden wir im Folgenden nur von *Prozesssynchronisation* / -kommunikation sprechen; die Aussagen gelten aber zumeist auch für Threads.

4.1 Prozesssynchronisation

4.1.1 Problemstellung

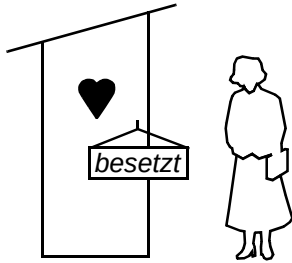
Prozesse werden nebenläufig zueinander ausgeführt. Sie können völlig unabhängig voneinander arbeiten, also nichts miteinander zu tun haben und sich in keiner Weise gegenseitig beeinflussen (natürlich bis auf ihre Konkurrenz um den Prozessor), oder sie können gewisse Abhängigkeiten voneinander haben. Diese Abhängigkeiten können unterschiedlicher Natur sein. Beispielsweise kann es erforderlich sein, dass die Prozesse Daten austauschen, um weiterarbeiten zu können, oder sie können um exklusiv benutzbare Betriebsmittel konkurrieren. Auch hier bietet die Alltagserfahrung eine Fülle von Beispielen (siehe Abb. 4.1, Teil a): Von mehreren Leuten, die eine Telefonzelle (oder eine andere kleine Räumlichkeit) benutzen möchten, kann dies nur einer gleichzeitig. Bei einem Staffellauf muss der zweiten Läufer einer Mannschaft auf den ersten Läufer warten.

Solche Abhängigkeiten haben einen sachlichen, aber insbesondere auch einen zeitlichen Aspekt. Sie führen also zu zeitlichen Einschränkungen, die bei der Prozessausführung beachtet werden müssen – so genannten *Synchronisationsbedingungen*. Man unterscheidet zwischen zwei allgemeinen Arten solcher Bedingungen (siehe auch Abb. 4.1, Teil b):

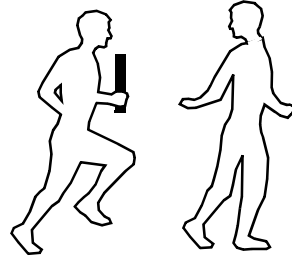
- Der *wechselseitige Ausschluss* (auch *Sperrsynchronisation* genannt) ist auf exklusiv benutzbare Betriebsmittel zurückzuführen – also beispielsweise Speicherbereiche, in

a.) Alltagsbeispiele zur Synchronisation:

wechselseitiger Ausschluss:

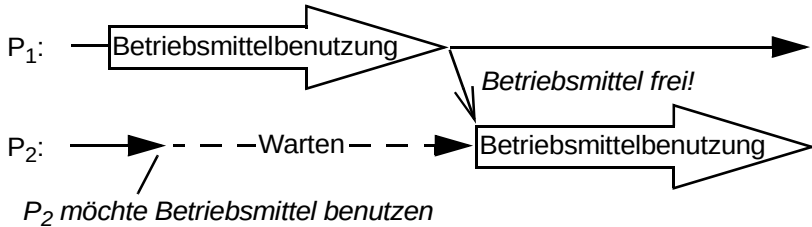


Reihenfolgebedingung:

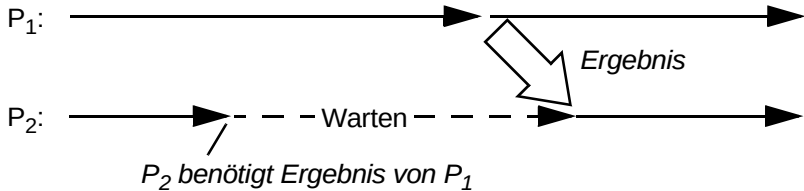


b.) synchronisiertes zeitliches Verhalten:

wechselseitiger Ausschluss:



Reihenfolgebedingung:

**Abb. 4.1** Synchronisation: wechselseitiger Ausschluss und Reihenfolgebedingung

die geschrieben werden soll, oder Drucker. Hier darf höchstens ein Prozess gleichzeitig auf das Betriebsmittel zugreifen oder, wie es allgemeiner formuliert wird, in einem *kritischen Abschnitt* (engl. *critical section*) bezüglich dieses Betriebsmittels tätig sein. Ein zweiter Prozess, der ebenfalls in einen entsprechenden kritischen Abschnitt eintreten möchte, muss warten, bis der erste Prozess den kritischen Abschnitt wieder verlassen hat. Das Telefonzellen-Beispiel von oben gehört zu der Kategorie „wechselseitiger Ausschluss“.

- Bei der *Reihenfolgebedingung* (auch *Kooperation* oder *Zustandssynchronisation* genannt) wartet ein Prozess auf die Tätigkeit, ein Signal oder ein Ergebnis eines anderen

Prozesses. Er kann nicht fortgesetzt werden, solange der andere Prozess nicht das Gewünschte geliefert hat. Die Staffelläufer sind ein Beispiel hierfür.

Prozesssynchronisation ist die Durchsetzung von Synchronisationsbedingungen bei der nebenläufigen Ausführung von Prozessen. Eine solche Synchronisation geschieht nicht von allein, sondern muss durch *Synchronisationsmechanismen* explizit durchgesetzt werden.

4.1.2 Einfache Synchronisationsmechanismen

Zur Durchsetzung von Synchronisationsbedingungen kann das Betriebssystem verschiedene Mechanismen bereitstellen, die Programmierer bei der Programmierung benutzen können.

Interruptsperrung

Die Sperrung von Interruptsignalen ist ein primitiver Mechanismus, um den wechselseitigen Ausschluss sicherzustellen: Bevor ein Prozess in einen kritischen Abschnitt eintritt, setzt er alle Bits des Unterbrechungsmaskierungsregisters auf 0 und „sperrt“ damit sämtliche Interrupts. Der Prozess kann also nicht unterbrochen werden, während er im kritischen Abschnitt aktiv ist. Insbesondere haben Interruptsignale der Uhr, mit denen Prozesswechsel eingeleitet werden, keine Wirkung. Beim Verlassen des kritischen Abschnitts setzt der Prozess das Maskierungsregister wieder in seinen ursprünglichen Zustand zurück.

Trotz oder gerade wegen seiner Einfachheit hat dieser Ansatz gravierende Nachteile:

- Die Sperrzeit und damit der Aufenthalt im kritischen Abschnitt dürfen nur kurz sein, denn die Behandlung von Interrupts, die während der Sperrzeit eintreffen, muss rasch nachgeholt werden. Zudem ist bei langer Sperrzeit die Gefahr groß, dass Interrupts durch Überschreibungen im Speicherregister verloren gehen.
- Es ist nicht ausgeschlossen, dass ein Benutzerprogramm im kritischen Abschnitt „abstürzt“, also durch einen Fehler vorzeitig terminiert, oder dass es in eine Endlosschleife gerät. Die Interruptsperrung wird in diesen Fällen nicht wieder aufgehoben, was zu einer Blockade des gesamten Systems führt.
- Die Sperrung von Interrupts ist nur eine Lösung für Ein-Prozessor-Systeme. In Systemen mit mehreren Prozessoren werden mehrere Prozesse echt nebenläufig ausgeführt, so dass sie auch bei gesperrten Interrupts gleichzeitig in einem kritischen Abschnitt aktiv sein können.
- Der Ansatz ist nur geeignet zur Durchsetzung des wechselseitigen Ausschlusses. Reihenfolgeprobleme wie bei der Zustandssynchronisation oder komplexere Synchronisationsaufgaben können damit nicht gelöst werden.

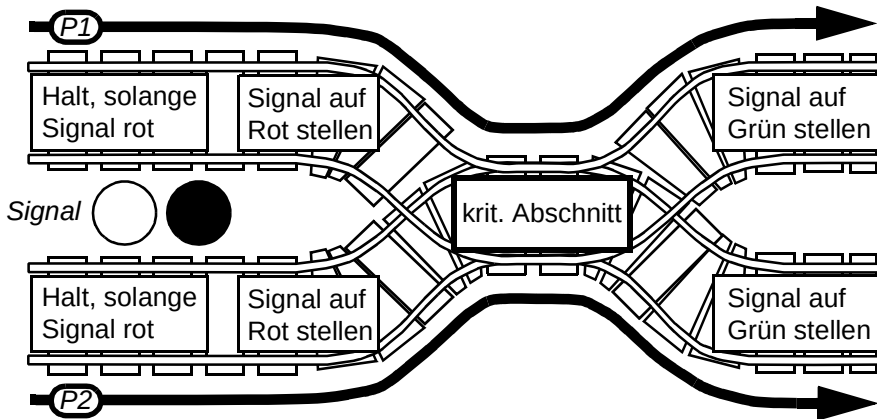


Abb. 4.2 Analogbeispiel zu TEST_AND_SET

Die Sperrung von Interrupts sollte also nur innerhalb des Betriebssystemkerns (also in „vertrauenswürdigen“ Code) verwendet werden, um die Ausführung kurzer kritischer Abschnitte zu schützen.

Spinlocks durch Maschinenbefehle oder Lock Files

Der zweite Lösungsansatz soll mit dem Beispiel in Abbildung 4.2 verdeutlicht werden. Es zeigt eine zweigleisige Eisenbahnstrecke, die an einem Engpass eingleisig geführt wird („kritischer Abschnitt“). Kommt ein Zug (P1 oder P2) von links auf einer der beiden Strecken an, so prüft er zunächst, ob ein zentrales Eingangssignal auf Rot steht. Wenn ja, so hält er an; wenn nein, so fährt er weiter und schaltet dabei das Signal auf Rot um. Damit soll erreicht werden, dass ein Zug auf der anderen Strecke wartet, solange sich der erste Zug im eingleisigen Abschnitt befindet. Nach Verlassen des kritischen Abschnitts wird das Signal wieder auf Grün gestellt.

Entscheidend ist, dass die Prüfung des Signals und das Rotsetzen in einem zusammenhängenden Schritt ausgeführt werden: Es darf nicht geschehen, dass ein Zug das grüne Signal sieht und weiterfährt, aber noch vor der Umschaltung ein weiterer Zug auf der zweiten Strecke kommt, ebenfalls das Signal noch grün sieht und auch weiterfährt.

In Rechensystemen kann dieser Ansatz zur Durchsetzung des wechselseitigen Ausschlusses benutzt werden. Hier wird für die „Signaloperationen“ (d.h. die Prüfung und das Umschalten) ein spezieller Maschinenbefehl (oder, wie in Windows NT, eine entsprechende Schnittstellenfunktion) zur Verfügung gestellt. Dieser Befehl, nennen wir ihn TEST_AND_SET, ist „atomar“. Er kann also während seiner Ausführung nicht unterbrochen werden, was mit Bussperren durch die Hardware sichergestellt wird. Die Wirkungsweise des Befehls wird durch den folgenden Pseudo-C-Code definiert:

```
bool TEST_AND_SET(bool *signal_rot) {
    if (*signal_rot==FALSE) {
```

```

    *signal_rot=TRUE;
    return FALSE; }
else
    return TRUE; }

```

`bool` bezeichnet dabei einen Typ für aussagenlogische Variablen mit dem Wertebereich {TRUE, FALSE}.

`TEST_AND_SET` arbeitet auf einer aussagenlogischen Variablen (im Code: `bool *signal_rot`), die insbesondere als *Sperrvariable* (engl. *lock variable*) für exklusiv benutzbare Betriebsmittel eingesetzt werden kann: Jedem solchen Betriebsmittel ist eine eigene Sperrvariable zugeordnet. Sie hat genau dann den Wert TRUE, wenn das Betriebsmittel gerade von einem Prozess benutzt wird und somit für andere Prozesse „gesperrt“ ist. Alle Prozesse, die um das Betriebsmittel konkurrieren, verwenden die Sperrvariable um festzustellen, ob sie in ihren kritischen Abschnitt eintreten können, und um das Betriebsmittel für sich zu „reservieren“.

Diese Prüfung und Reservierung geschehen genau wie im Eisenbahnbeispiel: `TEST_AND_SET` überprüft zunächst, ob das Betriebsmittel „frei“, also nicht gesperrt ist (`Test *signal_rot==FALSE`). Ist das der Fall, so wird das Betriebsmittel gesperrt (`*signal_rot=TRUE`) und dem Prozess gemeldet, dass er in seinen kritischen Abschnitt eintreten kann (`return FALSE`). Der Prozess stellt also fest, dass das Betriebsmittel frei ist und sperrt es gleichzeitig für andere Prozesse. Ist dagegen das Betriebsmittel bereits gesperrt, so wird das dem Prozess gemeldet (`return TRUE`).

Exklusiv benutzbare Betriebsmittel können also dadurch geschützt werden, dass jeder der konkurrierenden Prozesse den folgenden so genannten *Spinlock* durchführt (Angabe wieder als Pseudo-C-Code):

```

extern bool lock; /* Verweis auf globale Sperrvar. */
...
/* Kreisen in einer while-Schleife,
   bis TEST_AND_SET() den Wert FALSE liefert: */
while (TEST_AND_SET(&lock));
< kritischer Abschnitt >
/* Freigabe des Betriebsmittels: */
lock = FALSE;

```

Man beachte das Semikolon am Ende der `while`-Zeile: Ein Prozess kreist so lange in einer `while`-Schleife mit leerem Körper, bis er das Betriebsmittel für sich belegen kann. Damit erklärt sich auch der Name „Spinlock“: „to spin“ = „drehen“, „kreisen“.

Derselbe Effekt wie mit dem Maschinenbefehl `TEST_AND_SET` kann mit einem so genannten *Lock File* erreicht werden: Ein Prozess, der in einen kritischen Abschnitt eintreten möchte, versucht zuvor, eine Datei mit einem festgelegten Namen zu erzeugen. Gelingt ihm dies, weil die Datei zuvor noch nicht existierte, so durchläuft er den kritischen Abschnitt und löscht die Datei anschließend wieder. Schlägt die Erzeugung jedoch fehl, da es die Datei schon gibt (und somit ein anderer Prozess im kritischen Abschnitt ist), so läuft der Prozess zunächst nicht weiter, sondern probiert es später noch einmal. Das Ganze funktioniert natürlich nur, wenn die Operation zur Dateierzeugung

atomar ist. Unter UNIX / Linux lassen sich Lock Files mit Hilfe der Schnittstellenfunktionen `open()` (mit den Optionen `O_CREAT` und `O_EXCL`) und `unlink()` realisieren (siehe Anhang).

Ein Problem der Spinlocks ist ihre mögliche Unfairness: Es ist nicht sichergestellt, dass jeweils der am längsten wartende Prozess das Betriebsmittel erhält. Stattdessen findet stets ein „Wettrennen“ (engl. *race* oder *race condition*) der konkurrierenden Prozesse statt. Im Extremfall kann es sogar zum „Verhungern“ (engl. *starvation*) eines Prozesses kommen, wenn er nämlich ständig von anderen Prozessen „überholt“ wird. Allerdings tritt dieses Phänomen in der Praxis nur relativ selten auf.

Das wesentlich größere Problem ist das Kreisen der Prozesse in den while-Schleifen. Durch dieses „aktive Warten“, d.h. das fortwährende Testen der Sperrvariablen, wird unnötig viel Prozessorzeit verbraucht. Günstiger ist es, wartende Prozesse zu blockieren und sie erst dann wieder zu aktivieren, wenn sich der Wert der Sperrvariablen ändert. Der Lösungsansatz, der im folgenden Abschnitt geschildert wird, beruht auf diesem Prinzip.

4.1.3 Semaphore

Semaphore wurden bereits 1965, also in der Frühzeit der Informatik, durch den Niederländer E.W. Dijkstra eingeführt [Dijk65]. Das Wort „Semaphor“ kommt aus dem Griechischen und bedeutet in seiner wörtlichen Übersetzung so viel wie „Zeichenträger“. Ein Semaphor ist ein Sperrmechanismus, der Prozesse unter bestimmten Bedingungen blockiert und so sehr vielseitig zur Durchsetzung von Synchronisationsbedingungen eingesetzt werden kann.

Funktionsprinzip

Die Wirkungsweise von Semaphoren lässt sich gut an einem Beispiel aus der verflochtenen DDR illustrieren. Dort war bekanntlich vieles reglementiert, so auch der Zugang zu den Geschäften. Um ein Gedränge zu vermeiden, war die Anzahl der Kunden im Laden beschränkt. Durchgesetzt wurde dies mit einem Stapel von Einkaufskörben am Eingang. Jeder Kunde musste beim Betreten des Ladens einen Einkaufskorb nehmen, ihn mit sich führen und beim Verlassen wieder abgeben. Waren keine Körbe vorhanden, so musste er warten, bis ein anderer Kunde das Geschäft verließ und seinen Korb wieder abgab. Je nach Beliebtheit der angebotenen Waren bildete sich so an der Tür eine mehr oder weniger lange Schlange, denn es konnten ja nie mehr Leute im Geschäft sein, als Körbe vorhanden waren.

Dieses „Korbstapelprinzip“ wird auch bei Semaphoren benutzt: Ein Semaphor besteht im Wesentlichen aus zwei Datenstrukturen / Variablen (siehe Abb. 4.3): Eine ganzzahlige nichtnegative *Zählvariable* `COUNT` gibt an, wie viele Prozesse den Semaphor momentan ohne Blockierung „passieren“ (siehe unten) dürfen. Ihr Wert entspricht im einführenden Beispiel der Anzahl der Körbe, die sich momentan auf dem Stapel befinden. Daneben gibt es einen *Warteraum* für Prozesse, die darauf warten, den Sema-

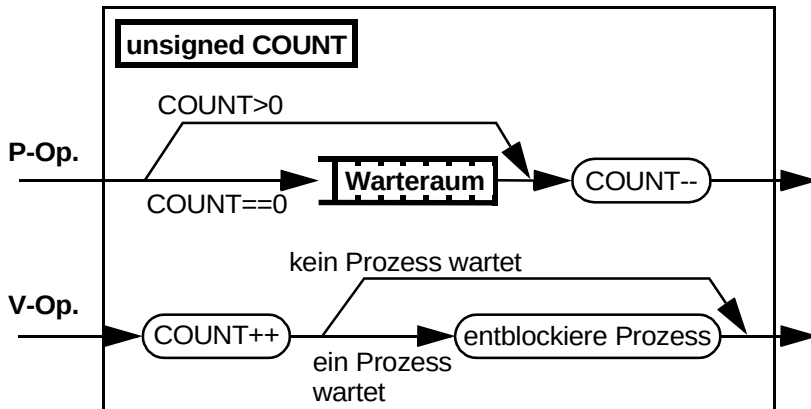


Abb. 4.3 Aufbauschema eines Semaphors

phor passieren zu dürfen. Diese Prozesse warten „passiv“, d.h. sie sind im Zustand „blockiert“ und werden durch das Betriebssystem in den Zustand „bereit“ überführt, wenn der Semaphor den Weg freigibt. Die Ordnung in diesem Warteraum ist vorzugsweise „First In First Out“, um möglichst fair zu sein (in UNIX ist dies allerdings nicht der Fall – siehe später). Im Beispiel entspricht dem Warteraum die Schlange der Kunden.

Ein Semaphor bietet drei Zugriffsoperationen an:

- Initialisierung:

```
SEM.INIT(unsigned int init_wert) {
    SEM.COUNT = init_wert; }
```

(Lies SEM.INIT als „Operation INIT auf dem Semaphor SEM“ und SEM.COUNT als „Variable COUNT des Semaphors SEM“.) Durch die Operation wird die Zählvariable mit einem nichtnegativen Anfangswert initialisiert. Im Beispiel entspricht das dem Herausstellen eines Stapels von Körben bei Ladenöffnung.

- Passieren:

```
SEM.P() {
    if (SEM.COUNT == 0)
        < blockiere >
    SEM.COUNT = SEM.COUNT-1; }
```

Die Operation prüft zunächst, ob der Wert der Zählvariablen gleich 0 ist („ob auf dem Stapel kein Korb mehr vorhanden ist“). Ist das der Fall, so blockiert sich der Prozess („der Kunde stellt sich in die Schlange“). Anderenfalls wird der Variablenwert unmittelbar um 1 erniedrigt („ein Korb wird entnommen“). Man beachte, dass auch ein zunächst blockierter Prozess nach seiner späteren Entblockierung die Variable

erniedrigt („ein Kunde, der zunächst in der Schlange gewartet hat, nimmt später einen Korb“).

Der Name der Operation P kommt vom niederländischen „passeeren“ (passieren).

- Freigeben:

```
SEM.V() {
    SEM.COUNT = SEM.COUNT+1;
    if ( < SEM-Warteraum ist nicht leer > )
        < entblockiere einen wartenden Prozess >
}
```

Die Operation erhöht zunächst den Wert der Zählvariablen um 1 („gibt einen Korb auf den Stapel zurück“). Ist der Warteraum des Semaphors nicht leer („Kunden warten“), so wird ein wartender Prozess entblockiert („ein wartender Kunde kann sich jetzt einen Korb holen“). Wie bereits erwähnt, setzt der entblockierte Prozess die P-Operation, in der er blockiert worden war, fort und senkt dabei den Wert der Zählvariablen wieder.

Der Name der Operation V kommt vom niederländischen „vrijgeven“ (freigeben).

Die Operationen sind atomar, also nichtunterbrechbar, was mit Hilfe des TEST_AND_SET-Maschinenbefehls oder Interruptsperrern sichergestellt werden kann. Neben diesen drei Operationen gibt es im Allgemeinen noch Funktionen zum Erzeugen und Löschen eines Semaphors sowie zur Abfrage seines aktuellen Werts.

Anwendungsbeispiele

Semaphore sind ein universelles Mittel zur Durchsetzung von Synchronisationsbedingungen. Drei Beispiele sollen wichtige Anwendungsmöglichkeiten demonstrieren:

- Beispiel 1: wechselseitiger Ausschluss (gemäß Abb. 4.1)

Erzeugung eines Semaphors S_WA.

Initialisierung: S_WA.INIT(1) ;

```
Prozess 1:    S_WA.P() ;
              < kritischer Abschnitt >
              S_WA.V() ;
```

Prozess 2: wie Prozess 1

Prozess 1 und 2 konkurrieren um ein exklusiv benutzbares Betriebsmittel. Sie müssen vor Eintritt in ihren jeweiligen kritischen Abschnitt eine P-Operation auf einem Semaphor S_WA ausführen, also versuchen, den Wert von S_WA.COUNT um 1 zu sen-

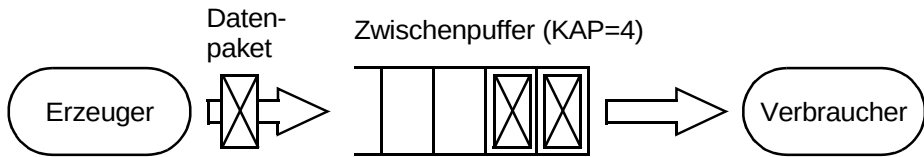


Abb. 4.4 Erzeuger-Verbraucher-System mit beschränktem Zwischenpuffer

ken. Die V-Operation am Ende des kritischen Abschnitts erhöht den Variablenwert wieder um 1. Da `S_WA.COUNT` mit 1 initialisiert wurde, kann nie mehr als ein Prozess in seinem kritischen Abschnitt aktiv sein (man mache sich das an einem Geschäft klar, in dem nur ein Einkaufskorb vorhanden ist).

- Beispiel 2: Reihenfolgebedingung (gemäß Abb. 4.1),
z.B. einfache Erzeuger-Verbraucher-Beziehung

Erzeugung eines Semaphors `S_REIHE`.

Initialisierung: `S_REIHE.INIT(0);`

Erzeuger: `< Erzeugen von Daten >`
 `S_REIHE.V();`

Verbraucher: `S_REIHE.P();`
 `< Verbrauchen der Daten >`

Der Erzeuger generiert Daten, die vom Verbraucher weiterverarbeitet werden. Der Verbraucher kann nicht ausgeführt werden, bevor die Daten vorliegen. Das wird dadurch sichergestellt, dass der Verbraucher als Erstes eine P-Operation auf einem Semaphor `S_REIHE` ausführt. Da `S_REIHE` mit 0 initialisiert wurde, kann der Verbraucher die P-Operation erst dann verlassen, wenn der Erzeuger seine V-Operation ausgeführt hat. Das wiederum geschieht erst nach der Erzeugung der Daten. Die Vorgehensweise kann mit der bei einem Stafettenlauf verglichen werden, bei dem ein Stab vom Vorgänger an den Nachfolger weitergegeben wird.

- Beispiel 3: Erzeuger-Verbraucher-Beziehung mit beschränktem Zwischenpuffer

Im System gemäß Beispiel 2 werden die Daten direkt vom Erzeuger an den Verbraucher übertragen. Das führt zu einer engen zeitlichen Kopplung, da der Verbraucher blockiert ist, bis ihm der Erzeuger eine Dateneinheit übergibt. Dies ist insbesondere in zyklischen Systemen nachteilig, wo Erzeuger und Verbraucher in Endlosschleifen Datenpakete erzeugen bzw. verbrauchen, wobei zeitliche Schwankungen auftreten.

Man kann die zeitliche Kopplung lockern und damit das System flexibler machen, indem man einen Zwischenpuffer einsetzt. Dieser Puffer kann eine begrenzte Anzahl von Datenpaketen, nämlich `KAP` Stück aufnehmen (siehe Abb.4.4).

Es muss nun dafür Sorge getragen werden, dass der Verbraucher bei einem leeren Puffer blockiert wird, dass der Erzeuger bei einem vollen Puffer blockiert wird und dass der Zugriff auf den Puffer wechselseitig ausgeschlossen ist. Eine mögliche Lösung sieht wie folgt aus:

Erzeugung dreier Semaphore: `S_LEER`, `S_VOLL`, `S_WA`.

Initialisierung: `S_LEER.INIT(0);`
`S_VOLL.INIT(KAP);`
`S_WA.INIT(1);`

Erzeuger:

```
while (TRUE) {
    < Datenpaket erzeugen >
    SEM_VOLL.P();
    SEM_WA.P();
    < Datenpaket einfügen >
    SEM_WA.V();
    SEM_LEER.V();
}
```

Verbraucher:

```
while (TRUE) {
    SEM_LEER.P();
    SEM_WA.P();
    < Datenpaket entnehmen >
    SEM_WA.V();
    SEM_VOLL.V();
    < Datenpaket verbrauchen >
}
```

Die Semaphore `S_LEER` und `S_VOLL` werden gegenläufig zueinander eingesetzt: `S_LEER` zählt die Anzahl der Datenpakete, die sich momentan im Puffer befinden. Er wird vom Erzeuger in einer V-Operation hochgezählt und vom Verbraucher in einer P-Operation heruntergezählt. Ist der Puffer leer, so wird der Verbraucher vor der Entnahme blockiert. Umgekehrt zählt `S_VOLL` die Anzahl der freien Plätze im Puffer, wird vom Verbraucher in einer V-Operation hochgezählt und vom Erzeuger in einer P-Operation heruntergezählt. Ist der Puffer voll, so wird der Erzeuger vor dem Einfügen blockiert. `S_WA` setzt den wechselseitigen Ausschluss durch.

Inbesondere das dritte Beispiel zeigt, dass Semaphore recht flexibel eingesetzt werden können, dass die Programmierung mit ihnen aber nicht immer ganz einfach ist. Insbesondere die Wahl des Initialisierungswerts und die Platzierung der P- und V-Operationen sind mögliche Fehlerquellen. Im Laufe der Zeit wurden daher abstraktere, möglichst „narrensichere“ Konzepte entwickelt. Ein Beispiel hierfür sind *Monitore*, auf die wir im übernächsten Abschnitt noch kurz eingehen werden.

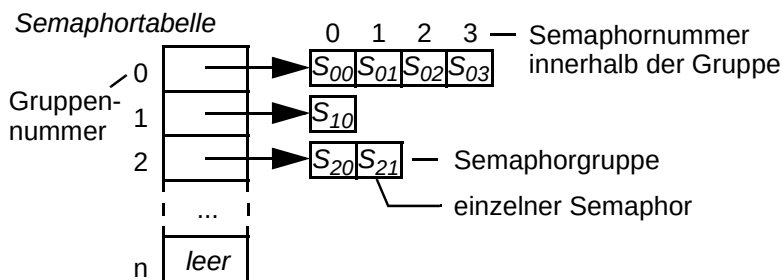


Abb. 4.5 Semaphortabelle in UNIX / Linux

Semaphore in UNIX / Linux

Die Semaphore in UNIX / Linux sind eine Verallgemeinerung der Dijkstra-Semaphore: In UNIX kann die Zählvariable bei den P- und V-Operationen auch um mehr als 1 gesenkt bzw. erhöht werden. Darüber hinaus können mehrere Operationen auf verschiedenen Semaphoren „atomar“ (= „unteilbar“) durchgeführt werden, also z.B. auf zwei Semaphoren jeweils eine P-Operation. Das bedeutet, dass die Werte der betroffenen Zählvariablen nur dann geändert werden, wenn *alle* beteiligten P-Operationen nicht blockieren. Ansonsten bleiben sämtliche Variablenwerte zunächst unverändert, und der Prozess blockiert.

Zur Realisierung des Konzepts führen der UNIX- und der Linux-Kern eine *Semaphortabelle*, die Verweise auf Arrays von Semaphoren enthält (siehe Abb. 4.5). Jedes Array beschreibt eine Gruppe von Semaphoren, die durch den Index der Semaphortabelle identifiziert wird. Einzelne Semaphore können über den Tabellenindex und ihre Position innerhalb der Gruppe (beginnend bei 0) angesprochen werden.

Atomare Operationen auf mehreren Semaphoren sind nur dann möglich, wenn die Semaphore derselben Gruppe angehören. Daneben kann aber auch, wie bei Dijkstra, auf einzelnen Semaphoren gearbeitet werden.

UNIX / Linux bietet drei Bibliotheksfunktionen zur Benutzung von Semaphoren (Darstellung gemäß System V, die POSIX-Schnittstelle differiert hiervon):

- Die Funktion `semget()` (Prototyp: `int semget(long key, int count, int flag)`) dient dazu, eine neue Semaphorgruppe zu erzeugen oder Zugriff auf eine bereits vorhandene Gruppe zu erhalten. Ihr Rückgabewert ist ein Index der Semaphortabelle, also die Identifikation der neuen oder der nunmehr zugreifbaren Semaphorgruppe, oder eine -1 im Fehlerfall.

Der Parameter `key_t` ist dabei ein externer Schlüssel (nicht zu verwechseln mit dem intern verwendeten Index der Semaphortabelle). Dies ist eine Nummer, auf die sich beispielsweise zwei Programmierer mündlich verständigt haben, damit sich ihre Prozesse über einen gemeinsamen Semaphor synchronisieren können. Anstelle einer solchen Nummer kann auch die Konstante `IPC_PRIVATE` übergeben werden. Sie wird bei der Erzeugung von Semaphoren benutzt und stellt sicher, dass nicht versehentlich auf eine bereits bestehende Semaphorgruppe Bezug genommen wird (siehe

Beispiele unten). Die externen Schlüssel bereits existierender Semaphorgruppen können mit Hilfe des UNIX-Kommandos `ipcs` ermittelt werden.

Der Parameter `count` gibt die Anzahl der Semaphore in der Gruppe an, und `flag` legt fest, wie `semget()` ausgeführt werden soll. Wenn beispielsweise das Flag `IPC_CREAT` gesetzt ist, erzeugt `semget()` eine neue Semaphorgruppe; wenn es nicht gesetzt ist, gibt `semget()` einen Zugriff auf eine bereits existierende Gruppe. Das Flag `IPC_CREAT` kann mit anderen Flags verknüpft werden sowie mit weiteren Bits, die (wie bei den Zugriffsrechten für Dateien) Rechte für Semaphorzugriffe festlegen.

Zwei Anwendungsbeispiele zeigen die Verwendung von `semget()`: Mit

```
semid = semget(IPC_PRIVATE, 1, IPC_CREAT | 0777);
```

wird eine neue Semaphorgruppe erzeugt. Sie enthält nur einen Semaphore und ist für alle Prozesse zugreifbar (Zugriffsrechte oktal 777). Der durch `semget()` gelieferte Tabellenindex wird in einer Variablen `semid` gespeichert und steht so für spätere Zugriffe auf den Semaphore zu Verfügung. Mit

```
semid1 = semget(75, 2, IPC_CREAT | 0770);
```

erzeugt ein Prozess eine Gruppe von zwei Semaphoren, auf die nur der Besitzer (also der Benutzer, für den der Prozess ausgeführt wird) und seine Benutzergruppe zugreifen dürfen. Er gibt dabei den (im Prinzip willkürlich gewählten) externen Schlüssel 75 an. Führt nun ein anderer Prozess

```
semid2 = semget(75, 2, 0770);
```

aus, so erhält er damit Zugriff auf die erzeugten Semaphore, da `semget()` den entsprechenden internen Tabellenindex zurückgibt und somit anschließend `semid1` und `semid2` identische Werte haben.

- Die Funktion `semctl()` (Prototyp: `int semctl(int id, int num, int cmd, union semun arg)`) ermöglicht verschiedene Kontrolloperationen auf Semaphoren. Dazu gehören das Setzen und Auslesen der Semaphorewerte sowie das Löschen der Semaphore.

Der Parameter `id` gibt den Tabellenindex der Semaphorgruppe an. `num` identifiziert einen Semaphore innerhalb der Gruppe, falls durch die Operation ein bestimmter Semaphore angesprochen wird. `cmd` ist das auszuführende Kommando, beispielsweise `SETALL` oder `GETALL`, um alle Semaphorewerte zu setzen bzw. auszulesen, `SETVAL` und `GETVAL`, um den Wert eines bestimmten Semaphors zu setzen bzw. zu lesen, sowie `IPC_RMID` zur Löschung der Semaphorgruppe. Im Parameter `arg` werden Werte übergeben, die für das Kommando erforderlich sind; seine genaue Form hängt von dem benutzten Kommando ab (siehe auch Beispiele).

Drei Beispiele zeigen Verwendungsmöglichkeiten von `semctl()`: Mit

```
unsigned short initarray[2];
initarray[0] = initarray[1] = 1;
```

```
semid = semget(IPC_PRIVATE, 2, IPC_CREAT | 0777);
semctl(semid, 0, SETALL, initarray);
```

werden zwei Semaphore neu erzeugt und jeweils mit 1 initialisiert (unter manchen UNIX-Versionen, wie beispielsweise Solaris, ist die Parameterübergabe bei `semctl()` etwas komplizierter – siehe Anhang). Mit

```
unsigned short outarray[2];
...
semctl(semid, 0, GETALL, outarray);
printf("Semaphorwerte: %d %d\n",
        outarray[0], outarray[1]);
```

werden die aktuellen Werte der Semaphore auf den Bildschirm ausgegeben. Mit

```
semctl(semid, 0, IPC_RMID, 0);
```

wird die Semaphoregruppe gelöscht. Der Parameter `num` ist in allen Beispielen irrelevant; zudem ist im letzten Beispiel kein Parameter `arg` erforderlich.

- Mit der Funktion `semop()` (Prototyp `int semop(int id, struct sembuf *oplist, int num)`) können eine oder mehrere P- und V-Operationen ausgeführt werden. Handelt es sich um mehrere Operationen, so findet die Ausführung, wie oben beschrieben, atomar statt. Die betroffenen Semaphore müssen alle derselben Gruppe angehören.

Der Parameter `id` ist der Tabellenindex der Semaphoregruppe. `oplist` verweist auf einen Array, in dem eine oder mehrere P- und V-Operationen beschrieben sind. Jeder Eintrag des Arrays definiert eine dieser Operationen und bezieht sich auf einen einzelnen Semaphore der Gruppe. Der Eintrag hat das folgende Format:

```
struct sembuf {
    short sem_num;
    short sem_op;
    short sem_flg; }
```

`sem_num` ist dabei die Nummer des Semaphors innerhalb seiner Gruppe. `sem_op` legt die auszuführende Funktion fest: Ist der Wert größer als 0, so handelt es sich um eine (verallgemeinerte) V-Operation. `sem_op` wird dann zum Wert des Semaphors addiert, und alle Prozesse, die auf eine Erhöhung des Werts warten, werden entblockiert. Es tritt hier also eine „Race Condition“ für die bisher blockierten Prozesse ein, bei der im Allgemeinen nur einer von ihnen (nämlich der schnellste) seine P-Operation beenden kann und die anderen wieder blockiert werden. Ist `sem_op` kleiner als 0, so handelt es sich um eine (verallgemeinerte) P-Operation. `sem_op` wird zum Wert des Semaphors addiert (d.h. der Semaphorewert gesenkt), sofern dieser dadurch nicht negativ wird. Würde der Semaphorewert negativ, so wird er nicht verändert und der Prozess blockiert. Im Spezialfall „`sem_op = 0`“ wird der Prozess blockiert, bis

der Semaphor den Wert 0 erreicht hat. Mit `sem_flg` können Flags zur Steuerung der Ausführung der Operation gesetzt werden.

Der dritte Parameter, `num`, nennt die Anzahl der Operationen in `oplist`.

Wir geben wiederum zwei Beispiele an, die die Benutzung von `semop()` demonstrieren. Mit

```
struct sembuf sem_v;
sem_v.sem_num = 0;
sem_v.sem_op  = 1;
sem_v.sem_flg = 0;
semop(semid, &sem_v, 1);
```

wird eine V-Operation auf dem ersten Semaphor der Gruppe `semid` ausgeführt. Mit

```
struct sembuf sem_p[2];
sem_p[0].sem_num = 0;
sem_p[1].sem_num = 1;
sem_p[0].sem_op  = sem_p[1].sem_op  = -1;
sem_p[0].sem_flg = sem_p[1].sem_flg = 0;
semop(semid, sem_p, 2);
```

werden jeweils eine P-Operation auf dem ersten und dem zweiten Semaphor der Gruppe `semid` ausgeführt. Dies geschieht atomar nach dem „Alles-oder-nichts-Prinzip“: Die Semaphorwerte werden nur dann gesenkt, wenn sie zuvor beide größer als 0 sind, der Prozess also auf keinem der beiden Semaphore blockiert. Ist mindestens einer der Semaphorwerte gleich 0, so wird der Prozess blockiert, und die Semaphorwerte bleiben beide unverändert.

Ein UNIX/Linux-Programm, das Semaphore benutzt, kann beispielsweise folgendermaßen aufgebaut sein: Ein Vaterprozess erzeugt einen oder mehrere Semaphore oder Semaphoregruppen mit

```
semidx=semget(IPC_PRIVATE, ..., IPC_CREAT|0700)
        (x=1, 2, ...)
```

initialisiert sie mit

```
semctl(semidx, 0, SETALL, ...)
```

und startet dann mit

```
fork()
```

zwei oder mehr Sohnprozesse. Die Sohnprozesse „erben“ die Semaphor-Identifikatoren `semidx` vom Vater, können also auf die Semaphore zugreifen und sich mit

```
semop(semidx, ...)
```

über sie synchronisieren. Nach der Terminierung der Söhne löscht der Vaterprozess mit

```
semctl(semidx, 0, IPC_RMID, 0)
```

die Semaphore. Das Löschen der Semaphore ist wichtig, da sich sonst im System alte, nicht mehr gebrauchte Semaphore ansammeln, bis schließlich keine neuen Semaphore mehr erzeugt werden können. Bricht das Programm vorzeitig ab, so sollte der Benutzer mit dem Shellkommando `ipcs` feststellen, ob noch Semaphore vorhanden sind, und diese gegebenenfalls mit dem Shellkommando `ipcrm` löschen.

Weitere Programmbeispiele finden sich bei den Übungsaufgaben am Ende dieses Kapitels und im Anhang.

4.1.4 Weitere Techniken zur Synchronisation

Neben den bisher besprochenen Techniken gibt es eine Reihe weiterer Synchronisationsmechanismen, die hier kurz dargestellt werden sollen:

Signale

Signale können Reihenfolgebedingungen durchsetzen: Der Nachfolgeprozess wartet, bis bei ihm ein Signal des Vorgängerprozesses eingetroffen ist. In UNIX / Linux beispielsweise gibt es hierfür die Funktionen `pause()` bzw. `kill()`. Nachteilig ist hier allerdings, dass `pause()` in jedem Fall vor `kill()` aufgerufen werden muss, da das Signal sonst verloren geht. Zudem richtet sich ein Signal i.a. an einen bestimmten Prozess; ist nicht bekannt, wer der Nachfolger ist, wird die Programmierung schwierig.

Events

Ein *Event* gibt an, ob ein bestimmtes Ereignis eingetreten ist: Ein Event-Objekt befindet sich in einem der beiden Zustände „nicht signalisiert“ oder „signalisiert“, wobei im Zustand „nicht signalisiert“ beliebig viele Prozesse auf ihm warten können. Ein Prozess kann das Eintreten des Ereignisses anzeigen, indem er den Zustand zu „signalisiert“ ändert. Wahlweise können hierdurch entweder nur einer oder alle wartenden Prozesse entblockiert werden; in letzterem Fall kann das Event-Objekt anschließend automatisch oder durch die explizite Aktion eines Prozesses auf „nicht signalisiert“ zurückgesetzt werden. Events mit den geschilderten Wahlmöglichkeiten gibt es z.B. in Windows NT.

Mutexe

Mutexe (von „*Mutual Exclusion*“ = wechselseitiger Ausschluss) dienen zum Schutz kritischer Abschnitte, in denen jeweils nur ein Prozess gleichzeitig aktiv sein darf. Sie werden so angewandt wie der Semaphor im Beispiel zum wechselseitigen Ausschluss, sind

aber einfacher als Semaphore, da sie nur die Zustände „belegt“ und „nicht belegt“ (entsprechend den Semaphorewerten 0 bzw. 1) annehmen können. Ein Prozess verlangt Eintritt in den kritischen Abschnitt, indem er eine „Wait“-Funktion auf dem zugehörigen Mutex aufruft; er wird dabei gegebenenfalls blockiert. Er verlässt den Abschnitt unter Aufruf der entsprechenden „Signal“-Funktion.

Windows NT enthält Mutexe zur Synchronisation verschiedener Tasks und außerdem einfachere *Critical Sections* zur Synchronisation von Threads innerhalb derselben Task. Die Programmiersprache Java bietet die Möglichkeit, Anweisungsfolgen mit dem Schlüsselwort `synchronized` als kritische Abschnitte zu kennzeichnen.

Monitore

Monitore sind höhersprachliche Programmkonstrukte, die eine weniger fehleranfällige Programmierung als mit Semaphoren ermöglichen sollen. Sie wurden Anfang der siebziger Jahre von P. Brinch Hansen [Brin73] und C. A. R. Hoare [Hoar74] eingeführt.

Ein Monitor ist ein „Objekt“ – also eine Einheit, die Daten und zugehörige Zugriffsoperationen umfasst, wobei die Daten ausschließlich über diese Operationen benutzt werden können. Zu jedem Zeitpunkt darf höchstens ein Prozess auf dem Monitor arbeiten; der Zugriff auf das Objekt ist also wechselseitig ausgeschlossen. Darüber hinaus kann man komplexere Synchronisationsbedingungen für die Zugriffe formulieren, wie z.B. Reihenfolgebeziehungen. Hierzu dienen *Bedingungsvariablen* (engl. *condition variables*), die ähnlich eingesetzt werden, wie es bei Events skizziert wurde.

Ein Monitor-Konzept wird z.B. durch die objektorientierte Programmiersprache Java unterstützt. Methoden auf Objekten können hier mit dem Schlüsselwort `synchronized` versehen werden. Es können nie zwei Threads gleichzeitig auf demselben Objekt dieselbe oder auch verschiedene als `synchronized` gekennzeichnete Methoden ausführen. Java-Threads können mit der Funktion `wait()` auf Ereignisse warten und sie durch `notify()` oder `notifyall()` anzeigen. Bei `notify()` wird nur einer der wartenden Threads entblockiert, bei `notifyall()` werden alle geweckt.

Atomare Transaktionen

Atomare Transaktionen sind insbesondere im Zusammenhang mit der Arbeit auf großen Datenbeständen (z.B. Datenbanken) interessant, auf die eine Vielzahl von Prozessen zugreifen können. Ein ungeschützter Zugriff kann zu Inkonsistenzen der Daten führen – erstens durch gleichzeitige Schreiboperationen mehrerer Prozesse, zweitens durch den vorzeitigen Abbruch eines Prozesses inmitten seiner Aktionen.

Durch Transaktionen werden solche Probleme vermieden: Eine Transaktion ist eine atomare Aktion, d.h. ihre Ausführung ist mit der Ausführung anderer Transaktionen wechselseitig ausgeschlossen. Im System selbst kann zwar aus Effizienzgründen real doch eine gleichzeitige Ausführung stattfinden, dann stellt das System aber sicher, dass es hierdurch zu keinen Störungen kommt. Zudem wird über die Schritte einer Transaktion Buch geführt, so dass bei Bedarf (z.B. im Fehlerfall) ein *Rollback* stattfinden kann: Die Transaktion wird abgebrochen, und der vorherige Zustand wird wiederhergestellt.

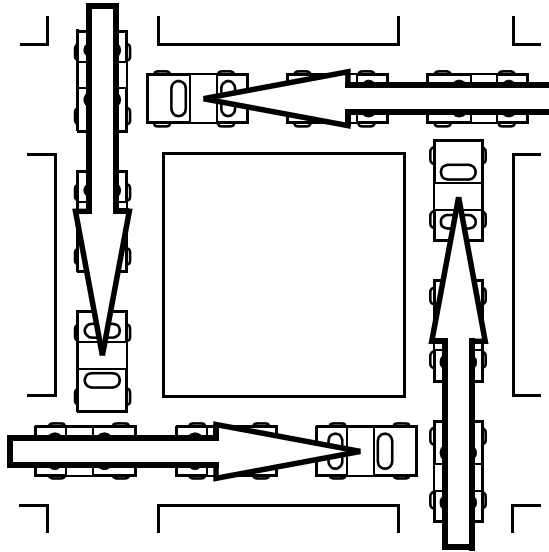


Abb. 4.6 Deadlock im Straßenverkehr

4.1.5 Deadlocks

Bei der Synchronisation kann es zu Situationen kommen, in denen sich Prozesse „verklemmt“ haben, d.h. sich gegenseitig so blockieren, dass keiner von ihnen weiterlaufen kann. Das Betriebssystem muss mit solchen *Deadlock*-Situationen umgehen können.

Deadlock-Begriff

In einem Multiprogramming-System gibt es exklusiv benutzbare Betriebsmittel, um die die Prozesse konkurrieren. Diese Konkurrenz kann kritisch werden, wenn mehrere Prozesse gleichzeitig mehrere dieser Betriebsmittel belegen möchten. Es kann dann nämlich dazu kommen, dass jeder Prozess schon einige der benötigten Betriebsmittel besitzt und zusätzlich noch weitere haben möchte, die jedoch schon einem anderen Prozess zugeteilt sind. Da erstens kein Prozess seine Betriebsmittel wieder hergeben und zweitens nicht auf die zusätzlichen Betriebsmittel verzichten kann oder möchte, blockieren sich die Prozesse gegenseitig. Eine solche Blockade bezeichnet man als *Deadlock* (offizielle deutsche Übersetzung: „*Verklemmung*“).

Zu einem Deadlock kann es beispielsweise im morgendlichen Berufsverkehr kommen. In Abbildung 4.6 blockiert jede der vier Autoschlangen eine Straßenkreuzung und benötigt, um weiterfahren zu können, freie Fahrt auf einer zweiten Kreuzung, die jedoch von einer anderen Schlange blockiert wird.

Bei der Benutzung von Semaphoren kann Programmcode der folgenden Form einen Deadlock verursachen:

```
Initialisierung: SEM_A.INIT(1);
                SEM_B.INIT(1);
```

```
Prozess 1:      SEM_A.P();
                SEM_B.P();
                < kritischer Abschnitt >
                SEM_B.V();
                SEM_A.V();
```

```
Prozess 2:      SEM_B.P(); /* Beachte: andere Reihenfolge */
                SEM_A.P(); /*           als bei Prozess 1. */
                < kritischer Abschnitt >
                SEM_A.V();
                SEM_B.V();
```

Hier schützen SEM_A und SEM_B jeweils ein exklusiv benutzbares Betriebsmittel. Die Prozesse 1 und 2 wollen in einen kritischen Abschnitt eintreten, in dem sie diese beiden Betriebsmittel benutzen. Wird Prozess 1 nach seiner ersten P-Operation unterbrochen und dann Prozess 2 ausgeführt, so belegen beide schließlich je eines dieser Betriebsmittel und warten in ihren zweiten P-Operationen jeweils auf die Freigabe des anderen.

Allgemein gesagt ist ein Deadlock also ein Zustand, in dem eine Menge von Prozessen existiert, von denen jeder auf die Aktion eines anderen wartet, selbst aber nichts tut. Bezieht man diese Definition speziell auf exklusiv benutzbare Betriebsmittel, so wartet jeder der Prozesse auf die Freigabe eines Betriebsmittels, das ein anderer belegt.

Damit es im System überhaupt zu einem Deadlock kommen kann, müssen dort sämtliche der folgenden vier Bedingungen erfüllt sein:

- Die betroffenen Betriebsmittel sind exklusiv benutzbar, d.h. sie können höchstens einem Prozess gleichzeitig zugeteilt sein.
- Die betroffenen Betriebsmittel sind nicht entziehbar, d.h. sie können einem Prozess nicht zwischenzeitlich weggenommen und später wieder zugeteilt werden.
- Prozesse, die schon Betriebsmittel belegen, können noch weitere anfordern.
- Es kann einen Zyklus von Prozessen $P_1, \dots, P_n$ geben, so dass jeweils P_i ein Betriebsmittel anfordert, das P_{i+1} belegt ($1 \leq i \leq n-1$; P_n fordert ein Betriebsmittel, das P_1 belegt – siehe den „Belegungs-Anforderungs-Graphen“ in Abb. 4.7).

Trifft eine dieser notwendigen Bedingungen nicht zu, so kann kein Deadlock auftreten.

Deadlock-Verhinderung

Um das Deadlock-Problem zu lösen, kann man versuchen, Deadlocks „strukturell unmöglich“ zu machen, d.h. das System so zu konstruieren, dass Deadlocks generell nicht auftreten können. Dies bezeichnet man als *Deadlock-Verhinderung* (engl. *deadlock*

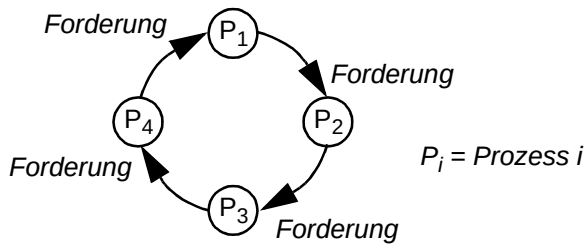


Abb. 4.7 Deadlock: Zyklus von Prozessen im „Belegungs-Anforderungs-Graphen“

prevention). Ein möglicher Ansatzpunkt sind die vier Bedingungen von oben. Lässt sich eine von ihnen ausschließen, so werden damit Deadlocks effektiv verhindert.

Wenn man sich die einzelnen Bedingungen ansieht, so stellt man Folgendes fest: Auf den wechselseitigen Ausschluss kann man nicht allgemein verzichten, da einige wichtige Betriebsmittel von Natur aus exklusiv benutzbar sind (siehe Kapitel 1). Ebenso sind einige Betriebsmittel generell nicht entziehbar (siehe ebenfalls Kapitel 1), so dass auch die zweite Bedingung keinen Ansatzpunkt bietet.

Etwas hoffnungsvoller sieht es bei der dritten Bedingung aus, denn im Prinzip lässt sich darauf verzichten, dass ein Prozess weitere Betriebsmittel anfordert, wenn er schon welche besitzt. Man muss hierfür „nur“ verlangen, dass jeder Prozess schon bei seinem Start alle Betriebsmittel belegt, die er je benötigen wird. In der Praxis ist aber auch dieser Ansatz problematisch. So werden damit Betriebsmittel weitaus länger belegt, als es eigentlich nötig wäre, so dass andere Prozesse unnötig lange warten müssen. Zudem ist beim Prozessstart oft noch nicht bekannt, was später einmal gebraucht wird.

Damit bleibt nur noch die vierte Bedingung, also der Versuch zu verhindern, dass ein „Belegungs-Anforderungs-Zyklus“ wie in Abbildung 4.7 entsteht. Hierfür kann man folgendermaßen vorgehen: Man teilt die Betriebsmittel in mehrere Klassen $K_1, K_2, \dots, K_m$ ein und definiert auf diesen Klassen eine Ordnung: $K_1 < K_2 < \dots < K_m$. Betriebsmitelanforderungen sind nur in aufsteigender Rangfolge zugelassen, d.h. ein Prozess, der Betriebsmittel der Klasse K_i besitzt, darf nur Betriebsmittel aus Klassen $K_j, j > i$, fordern. Man kann sich leicht klar machen, dass mit dieser Einschränkung kein Zyklus entstehen kann. In der Praxis wird man versuchen, knappe Betriebsmittel in „hohe“ Klassen einzuordnen um sicherzustellen, dass sie nicht lange belegt werden. Dass sich aber eine allseits befriedigende Klasseneinteilung finden lässt, ist nicht sicher.

Deadlock-Vermeidung

Ein alternativer Lösungsansatz ist die *Deadlock-Vermeidung* (engl. *deadlock avoidance*). Hier sind zwar, von der allgemeinen Systemkonstruktion her, Deadlocks prinzipiell möglich, die Ausführung der Prozesse wird aber so überwacht, dass kein Deadlock-Zustand erreicht wird.

Die Deadlock-Vermeidung basiert auf dem Begriff des *sicheren Zustands*. Ein Zustand heißt sicher, wenn aus ihm alle Prozesse zu Ende geführt werden können, ohne dass ein Deadlock auftritt. Genauer gesagt: Aus einem sicheren Zustand gibt es mindes-

tens einen Weg (eine „Ausführungsfolge“) zur Deadlock-freien Beendigung aller Prozesse. Umgekehrt ist ein Zustand unsicher, wenn er selbst zwar vielleicht noch kein Deadlock-Zustand ist, aber zu einem Deadlock-Zustand hinführt.

Um diese Begriffe zu veranschaulichen, betrachten wir noch einmal das zweite Deadlock-Beispiel von oben:

```
Prozess 1:  SEM_A.P() ;  
           SEM_B.P() ;  
           < kritischer Abschnitt >  
           ...
```

```
Prozess 2:  SEM_B.P() ;  
           SEM_A.P() ;  
           < kritischer Abschnitt >  
           ...
```

Der Zustand vor Beginn der Ausführung beider Prozesse ist sicher. Eine mögliche Ausführungsfolge ist nämlich, beide Prozesse hintereinander jeweils vollständig zu Ende auszuführen. Anders sieht es aus, wenn beide Prozesse jeweils ihre erste P-Operation ausgeführt haben und vor der zweiten Operation stehen. Der Zustand ist zwar noch kein Deadlock, da die Prozesse noch nicht blockiert sind, aber unsicher, da ab hier ein Deadlock unvermeidlich ist. Der eigentliche Deadlock wird dann erreicht, wenn die Prozesse jeweils in ihrer zweiten P-Operation blockieren.

Das Ziel muss also sein, unsichere Zustände zu vermeiden: Aus einem sicheren Zustand darf (durch Belegung neuer Betriebsmittel) nur dann in einen Nachfolgezustand übergegangen werden, wenn dieser ebenfalls sicher ist.

Es existieren zwar Verfahren, die die Sicherheit von Zustandsübergängen überprüfen – so beispielsweise der klassische *Bankiersalgorithmus* (nachzulesen z.B. in [Tan92]), bei dem der „Bankier“ (= das Betriebssystem) nur so viele Kredite vergibt (= Betriebsmittel belegen lässt), dass er sie auch später wieder sicher zurückbekommt (= dass die Prozesse sauber zu Ende laufen können, ohne sich zu blockieren). Solche Verfahren können aber in der Praxis nur mit starken Einschränkungen eingesetzt werden, so dass wir sie hier nicht näher betrachten.

Deadlock-Behandlung in der Praxis

In der Praxis ist also eine absolute Garantie, dass kein Deadlock auftritt, kaum möglich. Man konzentriert sich daher meist darauf, ein Betriebssystem so zu konstruieren, dass Deadlocks mit einer möglichst geringen Wahrscheinlichkeit eintreten. Wenn damit das Auftreten eines Deadlocks ähnlich unwahrscheinlich wird wie sonstige Systemabstürze, ist dieser „Vogel-Strauß-Algorithmus“ [Tan92] vertretbar.

Zur Behebung eines Deadlocks wird im einfachsten Fall das System neu gestartet. Verfeinerte Verfahren versuchen, diesen Neustart zu vermeiden, indem sie nur einzelne Prozesse abbrechen oder sogar Prozesse einige Schritte zurückversetzen („Process Rollback“) und ihnen dabei belegte Betriebsmittel entziehen.

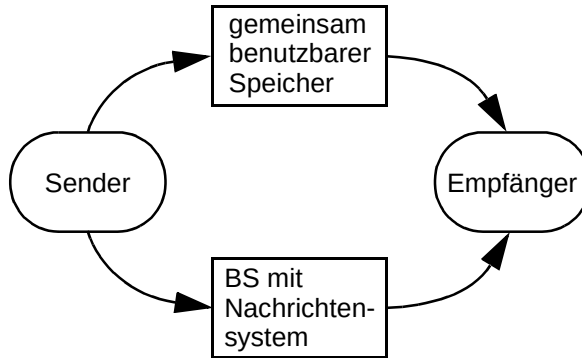


Abb. 4.8 Alternativen zur Informationsübertragung zwischen Sender und Empfänger

Allerdings sollte beachtet werden, dass Deadlocks in manchen Systemen unter keinen Umständen akzeptiert werden können. So dürfen beispielsweise Realzeitsysteme, die innerhalb einer bestimmten Frist auf kritische Ereignisse reagieren müssen, nicht in einen Deadlock geraten. Das „Restrisiko“ eines Deadlocks kann also nicht immer ignoriert werden. Zudem wird die Deadlock-Gefahr um so größer, je mehr Prozessoren echt nebenläufig arbeiten, je mehr unterschiedliche Betriebsmittel verwaltet werden müssen und je rascher Belegungen und Freigaben von Betriebsmitteln aufeinander folgen – eine Tendenz, die in modernen Systemen zu beobachten ist [Dei90].

4.2 Prozesskommunikation

Im vorigen Unterkapitel ging es um Prozesssynchronisation, also um die zeitliche Koordination der Ausführung mehrerer Prozesse. Um sich zu koordinieren zu können, müssen Prozesse miteinander kommunizieren, also Informationen austauschen. Beim Spinlock beispielsweise benutzen die Prozesse eine gemeinsame Sperrvariable um festzustellen, ob ein bestimmtes Betriebsmittel gerade frei ist. Auch die Zählvariable eines Semaphors dient zur Übermittlung von Informationen zwischen Prozessen.

Allerdings ist die Information, die so übertragen wird, auf den Synchronisationszweck beschränkt. Es sollen nun Techniken diskutiert werden, mit denen Prozesse *beliebige* Informationen austauschen können, also Techniken zur *Prozesskommunikation* (engl. *interprocess communication, IPC*). Hierbei wird zwischen zwei Ansätzen unterschieden (siehe Abb. 4.8):

Erstens können Prozesse Informationen über *gemeinsam benutzbare Speicherbereiche* (engl. *shared memory*) übertragen: Der „Sendeprozess“ schreibt Daten in einen bestimmten Speicherbereich; der „Empfängerprozess“ liest sie daraus. Hierbei stellt das Betriebssystem nur den Speicher zur Verfügung; um die Verwaltung (insbesondere die Synchronisation) müssen sich die Prozesse selbst kümmern.

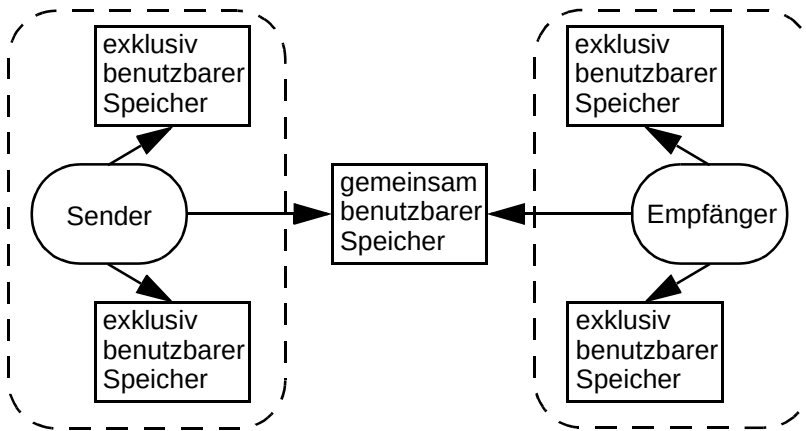


Abb. 4.9 gemeinsam benutzbarer Speicher vs. exklusiv benutzbarer Speicher

Zweitens können Prozesse zur Informationsübertragung ein *Nachrichtensystem* (engl. *message system*) benutzen. Hier übernimmt das Betriebssystem sämtliche Verwaltungsaufgaben und stellt Systemfunktionen für die Kommunikation bereit. Der Sender schickt also seine Daten durch einen einfachen Funktionsaufruf ab; der Empfänger nimmt sie entgegen, indem er ebenfalls eine Funktion aufruft.

4.2.1 Shared Memory

Grundprinzipien

Der Erste der beiden Ansätze wird auch mit dem Begriff „*speicherbasierte Kommunikation*“ bezeichnet. Es gibt hier bestimmte Speicherbereiche, auf die mehrere Prozesse direkt, d.h. durch Angabe einer Speicheradresse in ihrem Code zugreifen können. Man sagt, dass diese Speicherbereiche „im Adressraum mehrerer Prozesse liegen“, und nennt sie *Shared-Memory-Segmente*. Um einen Speicherbereich zu Shared Memory zu machen, muss eine bestimmte Betriebssystemfunktion aufgerufen werden, die ihn entsprechend registriert. Geschieht das nicht, so ist der Speicherbereich exklusiv benutzbar; dies ist der Standardfall (siehe Abb. 4.9).

Sender- und Empfängerprozesse müssen ihre Kommunikation über Shared Memory selbst koordinieren: Sie müssen sicherstellen, dass ihre Speicherzugriffe wechselseitig ausgeschlossen sind und dass der Empfänger nichts aus dem Speicher liest, bevor der Sender etwas hineingeschrieben hat. Außerdem müssen sie auf die richtige Typisierung der Daten achten: So darf es beispielsweise nicht vorkommen, dass der Sender eine Reihe von Ganzzahlwerten in den Speicher schreibt und anschließend der Empfänger die gelesenen Bytes als Gleitkommawerte interpretiert.

Shared Memory kann, außer zur Kommunikation, auch zur Einsparung von Speicherplatz dienen: Code, der von mehreren Prozessen ausgeführt wird, braucht nicht als Kopie bei jedem Prozess vorzuliegen, sondern muss nur einmal in einem Shared-Memory-

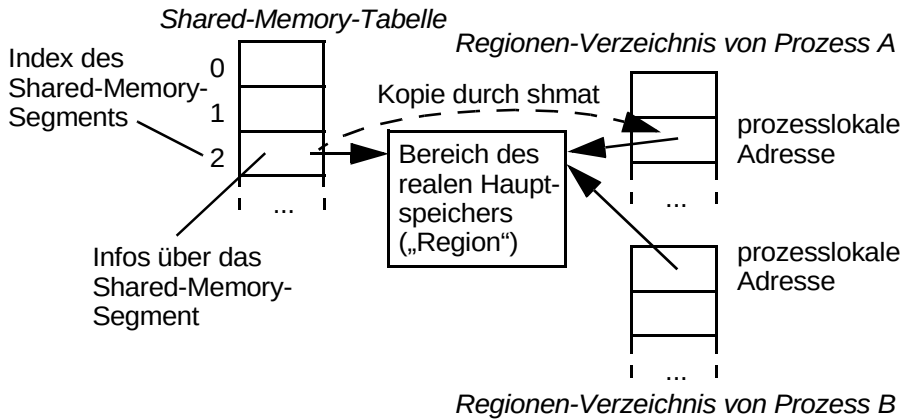


Abb. 4.10 Shared Memory in UNIX / Linux

Segment vorhanden sein, auf das alle Prozesse zugreifen können. Die *Shared Libraries* in UNIX und die *Dynamic Link Libraries (DLLs)* in Windows-Betriebssystemen beruhen auf diesem Prinzip. Da auf Programmcode nur lesend zugegriffen wird, müssen die Zugriffe nicht synchronisiert werden.

Shared Memory in UNIX / Linux

In UNIX / Linux basiert die Implementation von Shared Memory auf ähnlichen Datenstrukturen und Funktionen wie bei Semaphoren. Eine *Shared Memory Table* führt Informationen über die momentan vorhandenen Shared-Memory-Segmente, wie z.B. Anfangsadresse, Größe, Besitzer und Zugriffsrechte. Ein Shared-Memory-Segment wird über seinen Tabellenindex identifiziert und entspricht einer Region im Speicher. Abbildung 4.10 stellt den Sachverhalt schematisch dar, wobei der allgemeine Begriff „Regionenverzeichnis“ für Per Process Region Tables in UNIX System V bzw. die Liste der *vm_area_struct*-Strukturen in Linux steht (siehe Abschnitt 3.1.4).

UNIX / Linux bietet vier Schnittstellenfunktionen zur Arbeit mit Shared Memory (Darstellung gemäß System V, die POSIX-Schnittstelle differiert hiervon):

- Die Funktion `shmget()` (Prototyp: `int shmget(long key, int size, int flag)`) dient dazu, ein neues Shared-Memory-Segment zu erzeugen oder Zugriff auf ein bereits vorhandenes Segment zu erhalten. Ihr Rückgabewert ist ein Index der Shared Memory Table, also der interne Identifikator des Segments, oder eine -1 im Fehlerfall.

Der Parameter `size` gibt die Größe des Segments in Bytes an; die beiden anderen Parameter haben dieselbe Aufgabe wie bei `semget()`.

Nach der Ausführung von `shmget()` besitzt der Prozess zwar einen Identifikator für das Shared-Memory-Segment, kann darauf jedoch noch nicht zugreifen. Hierzu muss er erst die Funktion `shmat()` ausführen.

- Mit der Funktion `shmat()` („shared memory attach“, Prototyp: `char *shmat(int id, char *addr, int flag)`) kann ein Prozess ein Shared-Memory-Segment in seinen Adressraum einbinden, es also für sich zugreifbar machen. Hierfür legt die Funktion (unter UNIX System V) einen Eintrag in der Per Process Region Table des Prozesses an (siehe Abb. 4.10). Darüber hinaus wird mit der Ausführung von `shmat()` realer Speicher für das Segment belegt, sofern nicht schon ein anderer Prozess `shmat()` für dieses Segment aufgerufen hat. Bei `shmget()` wurde noch kein Speicher reserviert.

Der Parameter `id` ist der Identifikator des Shared-Memory-Segments. Mit `flag` können bestimmte Eigenschaften festgelegt werden, beispielsweise, dass der Prozess anschließend nur lesend auf das Segment zugreifen darf. `addr` ist eine Adresse innerhalb des Adressraum des Prozesses: Der Prozess schlägt vor, das Segment an diese Adresse zu binden, woran sich das Betriebssystem aber nicht halten muss. Hat dieser Parameter den Wert 0, so wird kein solcher Vorschlag gemacht.

Der Rückgabewert von `shmat()` ist die (Anfangs)adresse im Adressraum des Prozesses, an die das Shared-Memory-Segment tatsächlich gebunden wurde. Der Prozess kann anschließend über diese Adresse auf das Segment zugreifen:

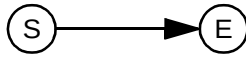
```
int shmidx, *pint;
shmidx = shmget(IPC_PRIVATE, 1024, IPC_CREAT | 0777);
pint = (int *) shmat(shmidx, 0, 0);
*pint = 256;
```

Das Beispielsprogramm erzeugt zunächst ein Shared-Memory-Segment mit einer Größe von einem Kilobyte. Der Aufruf von `shmat()` liefert dann eine Adresse, über die auf das Segment zugegriffen werden kann. Sie wird, nach einer Typkonversion, der Zeigervariablen `pint` zugewiesen. Mit der letzten Anweisung wird der Ganzzahlwert 256 in die ersten 4 Bytes des Shared Memory geschrieben (unter der Annahme, dass ein Integer 4 Bytes belegt).

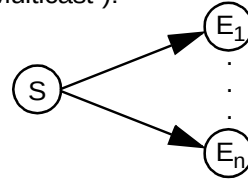
- Die Funktion `shmdt()` („shared memory detach“, Prototyp: `int shmdt(char *addr)`) entfernt ein Shared-Memory-Segment aus dem Adressraum des aufrufenden Prozesses. Hierbei wird (unter UNIX System V) der zugehörige Eintrag in der Per Process Region Table des Prozesses gelöscht.
- Mit `shmctl()` (Prototyp: `int shmctl(int id, int cmd, struct shmidx *buf)`) können verschiedene Steuerungsfunktionen auf Shared-Memory-Segmenten ausgeführt werden. Die Parameter `id` und `cmd` identifizieren das Segment und das auszuführende Kommando. `buf` überträgt Werte, die zur Ausführung des Kommandos gebraucht werden. Für das Kommando `IPC_RMID`, das ein Shared-Memory-Segment löscht, ist `buf` 0.

Ein Programm mit mehreren Prozessen, die über Shared Memory kommunizieren, kann nach demselben „Rezept“ wie für UNIX-Semaphore angegeben aufgebaut werden (siehe Ende von Abschnitt 4.1.3). Konkrete Programmbeispiele finden sich in den Übungsaufgaben am Ende dieses Kapitels und im Anhang.

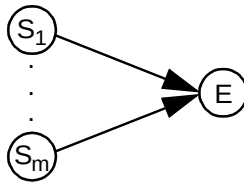
a.) 1:1 („Unicast“):



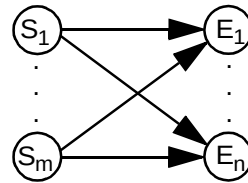
b.) 1:n („Multicast“):



c.) m:1 (z.B. „Client-Server“):



d.) m:n (z.B. „Client-Server“):

**Abb. 4.11** Sender-Empfänger-Beziehungen

4.2.2 Nachrichtenbasierte Systeme: allgemeine Prinzipien

Nachrichtenbasierte Systeme ermöglichen eine komfortablere Kommunikation als Shared Memory. Hier bietet das Betriebssystem Funktionen, mit denen Prozesse Nachrichten absenden und empfangen können. Das zugehörige Stichwort ist „*nachrichtenbasierte Kommunikation*“ (engl. *message-based communication*).

Der erste Vorteil gegenüber der speicherbasierten Kommunikation ist, dass hier das Betriebssystem die Benutzerprozesse von allen Verwaltungsaufgaben entlastet – insbesondere von Synchronisationsproblemen. Zweitens kann die nachrichtenbasierte Kommunikation auch in verteilten Systemen eingesetzt werden, wo kein gemeinsamer Hardwarespeicher vorhanden ist, sondern die Informationen über ein Kommunikationsnetz übertragen werden müssen.

Zu Letzterem ist allerdings ergänzend zu sagen, dass es in modernen verteilten Systemen auch das Konzept des *Distributed Shared Memory (DSM)* gibt, das durch die Hardware unterstützt wird. Hier sind die Daten (genauer: die Seiten eines seitenorientierten virtuellen Speichers – siehe Kapitel 5) auf die einzelnen Rechnerknoten verteilt und werden bei einem Zugriff automatisch auf den zugreifenden Knoten gebracht. Aus Sicht der Software ist also im gesamten Rechnernetz ein einheitlicher globaler Speicher vorhanden; die Datentransporte bleiben ihr verborgen.

Sender-Empfänger-Beziehungen

Bei der Übertragung von Nachrichten stehen Prozesse in einer Sender-Empfänger-Beziehung zueinander. Je nach der Anzahl der Sender und Empfänger, die an der Kommunikation beteiligt sind, unterscheidet man die folgenden Beziehungen (siehe Abb. 4.11):

- 1:1-Beziehung (*Unicast*): Ein Sender, ein Empfänger.
- 1:n-Beziehung (*Multicast*): Ein Sender, mehrere Empfänger. Hier wird dieselbe Nachricht an mehrere Empfänger übertragen. Sollen *alle* existierenden Prozesse den Multicast empfangen, so spricht man von *Broadcast*.
- m:1-Beziehung: Mehrere Sender, ein Empfänger. Ein Beispiel für diese Beziehung sind *Client-Server-Anwendungen*, bei denen ein Diensteanbieter (Server) die Aufträge mehrerer Kunden (Clients) entgegennehmen kann.
- m:n-Beziehung: Mehrere Sender, mehrere Empfänger. Ein Beispiel für diese Beziehung sind Client-Server-Anwendungen mit mehreren Diensteanbietern.

Um eine Nachricht abzusenden, ruft der Sendeprozess eine Sendefunktion (im Folgenden *Send-Funktion* genannt) auf; um eine Nachricht zu empfangen, ruft der Empfängerprozess eine Empfangsfunktion (*Receive-Funktion*) auf. Die Nachrichtenübertragung kann dabei entweder direkt vom Sender zum Empfänger erfolgen oder über einen Zwischenspeicher, eine so genannte *Mailbox* oder einen *Port* (siehe Abb. 4.12).

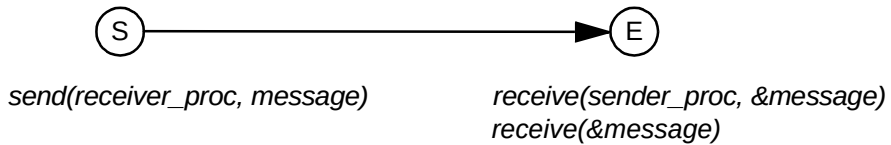
Direkte Übertragung, Mailboxen und Ports

Bei der *direkten Übertragung* hat die Send-Funktion die allgemeine Form `send(receiver_process, message)`. Der Sender gibt also den Empfängerprozess oder (bei Multicast) die Gruppe der Empfängerprozesse an sowie die Nachricht, die übertragen werden soll. Die Empfangsfunktion hat die allgemeine Form `receive(sender_process, &message)`. Der Parameter `sender_process` benennt den Absender, von dem die Nachricht stammen soll; Nachrichten von anderen Prozessen werden hierdurch also nicht empfangen. Prinzipiell kann auch eine Gruppe von möglichen Absendern angegeben oder überhaupt kein bestimmter Absender festgelegt werden (`receive(&message)`). `&message` ist ein Rückgabeparameter, in dem die empfangene Nachricht zurückgegeben wird.

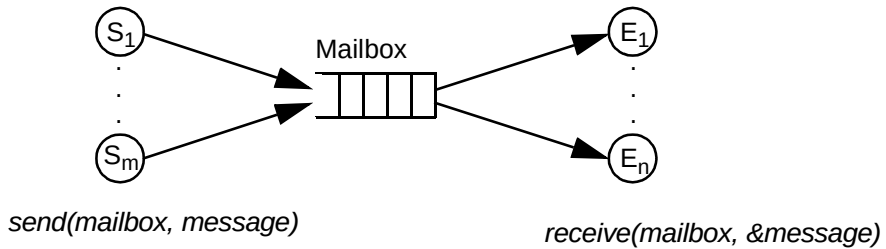
Bei der Kommunikation über eine *Mailbox* wird kein bestimmter Empfänger oder Sender angegeben, sondern stattdessen die Mailbox, in die die Nachricht geschickt bzw. aus der sie abgeholt werden soll: Die Funktionen lauten allgemein `send(mailbox, message)` und `receive(mailbox, &message)`. Man beachte, dass (zumindest bei diesen allgemein formulierten Operationen) der Sender keinen Einfluss darauf hat, welcher Empfänger die Nachricht aus der Mailbox abholt, und der Empfänger keinen Einfluss darauf, von welchem Sender die Nachricht stammt. Allerdings lässt sich dies durch eine Erweiterung der Nachrichten um eine Sender- und Empfängerangabe oder durch die Zuordnung der einzelnen Mailboxen zu bestimmten Sendern und Empfängern erreichen. Unabhängig davon ist es sowieso meist sinnvoll, in oder mit der Nachricht den Absender anzugeben.

Realisiert wird die Nachrichtenübertragung folgendermaßen: Bei direkter Übertragung wird die Nachricht unmittelbar aus einem Speicherbereich des Senders in einen Speicherbereich des Empfängers geschrieben. Bei der Benutzung einer Mailbox wird

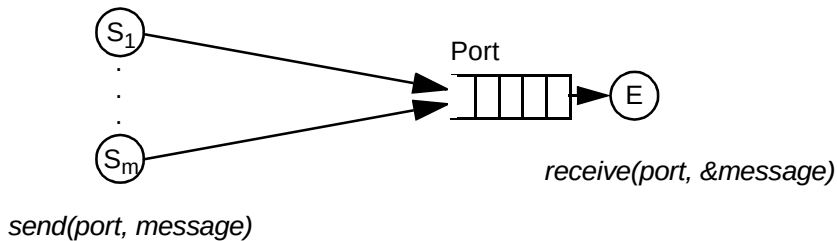
a.) direkt:



b.) indirekt über „Mailbox“:



c.) indirekt über „Port“:

**Abb. 4.12** Arten der Nachrichtenübertragung

die Nachricht zunächst in eine Warteschlange (eben die Mailbox) kopiert, für die das Betriebssystem einen Speicherbereich bereitstellt. Aus dieser Warteschlange kann der Empfänger sie dann abholen. In einem verteilten System wird die Nachricht über ein Kommunikationsnetz zwischen verschiedenen Rechnern übertragen.

Der Begriff „Mailbox“ bezeichnet im Allgemeinen eine Warteschlange, auf die mehrere Sender und mehrere Empfänger zugreifen dürfen (m:n-Kommunikation). Eine Mailbox, auf die nur ein einziger Empfänger zugreifen darf (m:1-Kommunikation), wird *Port* genannt. Ein Port ist also einem bestimmten Empfänger fest zugeordnet (siehe Abb. 4.12).

Ports werden insbesondere bei der Client-Server-Kommunikation eingesetzt (siehe Abb. 4.13): Ein Server ist, wie bereits gesagt, ein Diensteanbieter, an den Client-Prozesse Aufträge richten können. Beispielsweise nimmt ein *Print Server* Druckaufträge entgegen, und ein *File Server* ermöglicht Zugriffe auf Dateien. Hierzu stellt der Server einen Port (oder auch mehrere) bereit, in den die Clients ihre Aufträge eingeben können.

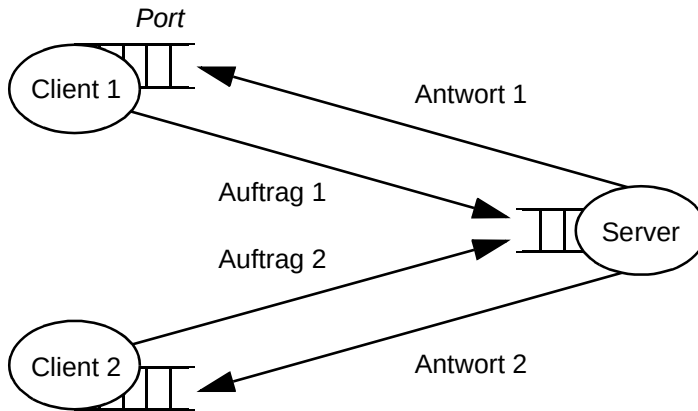


Abb. 4.13 Client-Server-Kommunikation

Nach der Bearbeitung eines Auftrags gibt der Server seine Antwort in einen Port des Clients zurück. Er identifiziert dabei den richtigen Antwort-Port anhand einer Absenderangabe im Auftrag.

Zeit und Richtung der Kommunikation

Bei der Ausführung von Kommunikationsoperationen sind zudem zeitliche Aspekte wichtig. Man unterscheidet hier zwischen *synchroner* und *asynchroner* Kommunikation: Die synchrone Kommunikation bewirkt eine (mehr oder weniger enge) zeitliche Beziehung zwischen Sender und Empfänger, die asynchrone nicht. Konkret bedeutet das, dass ein Prozess bei der Ausführung einer Send- oder Receive-Funktionen blockiert werden kann („synchron“) bzw. dass er dabei nie blockiert wird („asynchron“).

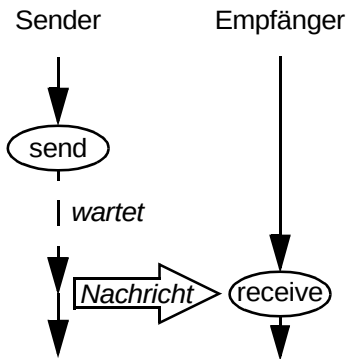
Bezogen auf die einzelnen Funktionen sieht das wie folgt aus (siehe Abb. 4.14): Beim blockierenden Senden (engl. *blocking send*) wird der Sender so lange in der Send-Funktion blockiert, bis der Empfänger empfangsbereit ist; erst dann findet die Übertragung statt. Beim nichtblockierenden Senden (engl. *non-blocking send*) dagegen legt der Sender seine Nachricht in einem bestimmten Speicherbereich ab, wo sie der Empfänger sofort oder später abholen kann, und fährt unmittelbar mit seiner Ausführung fort.

Der Empfänger wartet beim blockierenden Empfang (engl. *blocking receive*) so lange, bis er eine Nachricht des Senders erhält. Beim nichtblockierenden Empfang (engl. *non-blocking receive*) liest er eine Nachricht des Senders nur dann, wenn sie bereits vorliegt. Ansonsten verzichtet er auf die Nachricht und setzt seine Ausführung fort.

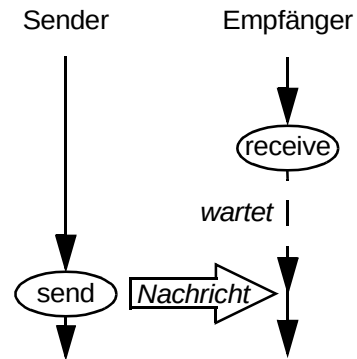
Sind sowohl die Send- als auch die Receive-Funktion blockierend, so spricht man von einem *Rendezvous*.

Schließlich wird eine Kommunikationsbeziehung auch noch durch die Übertragungsrichtung charakterisiert. Bei der *unidirektionalen* Kommunikation werden Nachrichten nur in einer Richtung geschickt; die Rollen des Senders und des Empfängers sind also fest verteilt. Bei der *bidirektionalen* Kommunikation werden Nachrichten in beiden

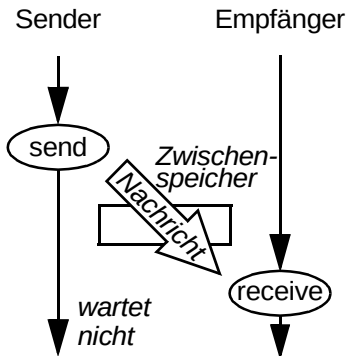
a.) blocking send:



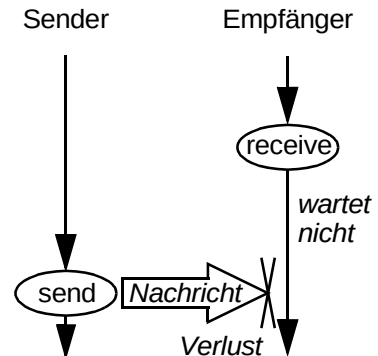
b.) blocking receive:



c.) non-blocking send:



d.) non-blocking receive:

**Abb. 4.14** Zeitverhalten bei der Kommunikation

Richtungen übertragen; die Rollen des Senders und Empfängers wechseln also dynamisch. Anstelle der Begriffe uni- und bidirektional werden auch die Bezeichnungen *simplex* bzw. *duplex* verwendet.

In den folgenden drei Abschnitten werden anhand von UNIX / Linux drei Ansätze für die nachrichtenbasierte Kommunikation dargestellt, nämlich *Pipes*, *Message Queues* und *Sockets*. Dieselben oder ähnliche Mechanismen finden sich aber auch in anderen Systemen, insbesondere in Windows NT.

4.2.3 Pipes

Der einfachste nachrichtenbasierte Kommunikationsmechanismus sind *Pipes* (siehe Abb. 4.15). Sie übertragen – ähnlich einer Rohrleitung – einen Strom von Bytes in FI-FO („First In First Out“)-Reihenfolge zwischen Prozessen. Pipes werden u.a. an den

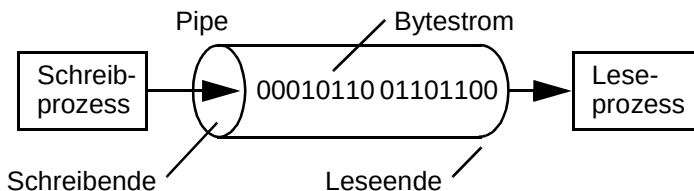


Abb. 4.15 Pipes in UNIX / Linux

Programmierschnittstellen von UNIX / Linux und Windows NT unterstützt, aber – mit dem Pipeoperator | – auch an deren Benutzerschnittstellen. Zum Schreiben in und Lesen aus Pipes werden im Allgemeinen dieselben Operationen wie bei „normalen“ Dateien benutzt, so dass Pipes als spezielle Dateien in das Dateisystem eingebettet sind.

UNIX/Linux-Pipes sind, in ihrer einfacheren Form, unidirektional, d.h. es gibt ein festgelegtes „Schreibende“ und ein „Leseende“. Daneben gibt es aber mit den so genannten *Stream Pipes* auch bidirektionale Pipes. Die Aufnahmekapazität von Pipes ist begrenzt (mindestens 4 KByte), so dass sie meist vollständig im Hauptspeicher gehalten werden können.

Man unterscheidet zwischen unbenannten und benannten Pipes: Auf eine *unbenannte Pipe* (engl. *unnamed pipe*) können nur der erzeugende Prozess und seine Nachkommen zugreifen; anderen Prozessen ist die Pipe nicht bekannt. Das folgende Beispiel zeigt, wie man solche Pipes erzeugt und benutzt:

```

main() {
    char buffer[5]; /* Puffer für empfangene Daten. */
    int fd[2];      /* Deskriptoren für Leseende (fd[0])
                    und Schreibende (fd[1]) der Pipe. */
    /* Erzeugung einer unbenannten Pipe: */
    pipe(fd);
    if (fork()==0) {
        /* Sohnprozess als Schreiber: */
        /* Lesedeskriptor schließen, da nicht benötigt: */
        close(fd[0]);
        /* Bytes in die Pipe schreiben: */
        write(fd[1], "TEST", 5);
        ...
        exit(0); }
    /* Vaterprozess als Leser: */
    /* Schreibdeskriptor schließen, da nicht benötigt: */
    close(fd[1]);
    /* Bytes aus der Pipe lesen: */
    read(fd[0], buffer, 5);
    printf("Gelesen: %s\n", buffer);
    ...
}

```

Der Vaterprozess erzeugt durch die Funktion `pipe()` eine unbenannte Pipe und erhält über den Parameter `fd` zwei Dateideskriptoren zurück: `fd[0]` dient zum Zugriff auf das Lesende der Pipe, `fd[1]` zum Zugriff auf das Schreibende. Anschließend wird ein Sohnprozess erzeugt, der Daten in die Pipe schreiben soll. Vater und Sohn schließen jeweils ihren Schreib- bzw. Lesedeskriptor, da sie ihn nicht benötigen (zur näheren Erläuterung der Dateioperationen siehe Kapitel 6 oder Anhang). Der Sohn überträgt dann mit `write()` einen Text von fünf Bytes in die Pipe, der vom Vater durch `read()` ausgelesen wird. Mit der Terminierung der Prozesse werden ihre noch offenen Dateien (und damit auch die Pipe) automatisch geschlossen. Da danach kein Prozess mehr auf die Pipe zugreifen kann, wird sie ebenfalls automatisch gelöscht.

Eine *benannte Pipe* (engl. *named pipe*, auch *FIFO* genannt) hat einen Namen, der im Dateibaum verzeichnet ist. Sie kann daher von beliebigen Prozessen geöffnet und zur Kommunikation benutzt werden (entsprechende Rechte vorausgesetzt). Das folgende Beispiel zeigt, wie das geht:

```
Prozess 1:  #include <fcntl.h>
            main() {
                int fd; /* Deskriptor für die Pipe */
                /* Erzeugen einer benannten Pipe
                   namens PIPE_1: */
                mkfifo("PIPE_1", 0666);
                /* Öffnen der Pipe zum Schreiben: */
                fd=open("PIPE_1", O_WRONLY);
                /* Schreiben in die Pipe: */
                write(fd, "TEST", 5);
                ...
            }
```

```
Prozess 2:  #include <fcntl.h>
            main() {
                char buffer[5];
                int fd;
                /* Öffnen der Pipe zum Lesen: */
                fd=open("PIPE_1", O_RDONLY);
                /* Lesen aus der Pipe: */
                read(fd, buffer, 5);
                printf("Gelesen: %s\n", buffer);
                ...
                /* Löschen der Pipe: */
                unlink("PIPE_1");
            }
```

Prozess 1 erzeugt durch `mkfifo()` in seinem Arbeitsverzeichnis eine Pipe namens `PIPE_1`, auf der alle Prozesse Lese- und Schreibrechte haben. Anschließend öffnet er die Pipe für Schreibzugriffe (Flag `O_WRONLY`) und schreibt in sie fünf Bytes. Von Prozess 2 wird angenommen, dass er dasselbe Arbeitsverzeichnis hat und erst nach der Er-

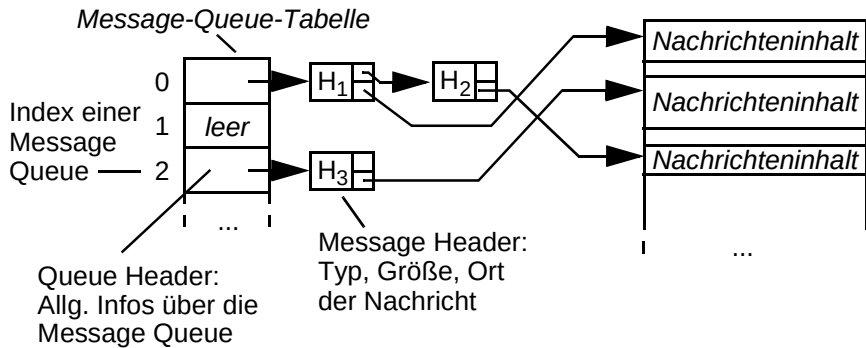


Abb. 4.16 Message Queues in UNIX / Linux

zeugung der Pipe startet. Er öffnet die Pipe für Lesezugriffe (Flag `O_RDONLY`) und liest aus ihr fünf Bytes. Schließlich wird die Pipe durch `unlink()` explizit gelöscht – geschähe dies nicht, so bliebe die Pipe auch dann noch bestehen, wenn sie von allen Prozessen (per `close()` oder bei Prozessterminierung) geschlossen wird.

Zum Abschluss noch zwei weitere Bemerkungen zu Pipes:

- Es können durchaus mehrere Prozesse gleichzeitig eine Pipe zum Lesen oder Schreiben geöffnet haben. Sie müssen sich dann aber selbst geeignet synchronisieren.
- Ein Prozess, der das Leseende einer Pipe öffnet, wird im Normalfall so lange blockiert, bis die Pipe auch zum Schreiben geöffnet wird, und umgekehrt. Die Daten sollen nämlich so schnell wie möglich an den Empfänger weitergegeben und dabei nur möglichst kurz in der Pipe zwischengespeichert werden.

Weitere Informationen zu UNIX/Linux-Pipes finden sich im Anhang.

4.2.4 Message Queues

Message Queues ermöglichen eine Datenübertragung über Mailboxen oder Ports. Die Grenzen zwischen den einzelnen Nachrichten bleiben dabei erhalten – im Unterschied zu Pipes, wo ja die Nachrichten zu einem unstrukturierten Bytestrom zusammengefasst werden. Neben UNIX / Linux unterstützt u.a. Windows NT mit seinen *Mailslots* dieses Konzept.

Die Implementationsprinzipien für UNIX/Linux-Message Queues ähneln denen von Semaphoren und Shared Memory: Die zentrale Datenstruktur ist wiederum ein Array, der Informationen über sämtliche existierenden Message Queues (= Mailboxen) enthält (siehe Abb. 4.16). Jeder Array-Eintrag beschreibt eine Queue, für die er die folgenden Angaben speichert: Verweise auf Anfang und Ende einer Liste von Nachrichten(-köpfen), aktuelle Länge der Liste (Anzahl der Nachrichten und Anzahl der Bytes), maximale Listenlänge (Bytes) sowie Informationen über die letzten Operationen, die auf der

Queue stattgefunden haben. Der Eintrag, der über den Array-Index identifiziert wird, wird *Queue Header* genannt.

Ein Queue Header verweist auf eine verkettete Liste. Sie enthält Informationen über die Nachrichten, die sich momentan in der Message Queue befinden, d.h. die von einem Sender abgeschickt, aber noch nicht von einem Empfänger abgeholt wurden. Die Listeneinträge sind nicht die Nachrichten selbst, sondern nur *Nachrichtenköpfe* (engl. *message header*). Sie enthalten den Typ der Nachricht (siehe unten), ihre Länge und einen Zeiger auf den Speicherbereich, in dem der eigentliche Inhalt der Nachricht steht. Die Liste wird nach dem FIFO-Prinzip verwaltet.

UNIX / Linux bietet vier Schnittstellenfunktionen zur Benutzung von Message Queues (Darstellung gemäß System V, die POSIX-Schnittstelle differiert hiervon):

- Die Funktion `msgget()` (Prototyp: `int msgget(long key, int flag)`) erzeugt eine neue Message Queue oder gibt Zugriff auf eine bereits existierende Queue. Sie geht wie `semget()` und `shmget()` vor.
- Mit `msgsnd()` (Prototyp: `int msgsnd(int msgqid, struct msgbuf *msg, int count, int flag)`) können Nachrichten in eine Message Queue eingetragen werden. Der Parameter `msgqid` ist der Identifikator dieser Queue.

Der Parameter `msg` ist ein Zeiger auf eine Struktur, in der an erster Stelle ein Ganzzahlwert („Typ“ der Nachricht) steht, gefolgt von weiteren Daten beliebiger Typen („Nutzdaten“ = eigentlicher Inhalt der Nachricht). Der Sender kann den Typ der Nachricht beliebig setzen (Ganzzahlwert außer 0), und er kann sich mit dem Empfänger frei darüber verständigen, wie die einzelnen Typwerte zu interpretieren sind. Das Betriebssystem gibt hier also keine Bedeutungen vor.

`count` gibt die Länge der Nutzdaten in Bytes an, und `flag` legt u.a. die Reaktion bei Speichermangel in der Queue fest: Hier kann gewählt werden, ob der Prozess blockieren oder mit einer Fehlermeldung aus der Funktion zurückkehren soll.

Bei der Ausführung von `msgsnd()` belegt der UNIX-Kern Speicherplatz und kopiert die Nachricht dorthin. Er fügt einen entsprechenden Nachrichtenkopf an das Ende der Queue-Header-Liste an und entblockiert alle Prozesse, die auf das Eintreffen einer solchen Nachricht warten (falls vorhanden; siehe unten: Funktion `msgrcv()`). Das führt gegebenenfalls zu einer Race Condition, bei der nur ein Prozess (der schnellste) die Nachricht empfängt und alle anderen wieder blockieren.

- Mit `msgrcv()` (Prototyp: `int msgrcv(int msgqid, struct msgbuf *msg, int maxcount, int type, int flag)`) können Nachrichten aus der Queue `msgqid` gelesen werden. `msg` verweist auf eine Struktur, in die die empfangene Nachricht eingetragen werden soll, und `maxcount` ist die maximale Byteanzahl der Nutzdaten in `msg`. `type` legt den Typ fest, den die empfangene Nachricht haben soll. Wird hier eine 0 angegeben, so liefert die Funktion eine Nachricht mit einem beliebigen Typ. `flag` beschreibt u.a. die gewünschte Reaktion, wenn keine passende Nachricht vorliegt.

Zur Ausführung von `msgrcv()` wird die Message Queue (d.h. ihre Queue-Header-Liste) von vorn nach hinten nach einer Nachricht des gewünschten Typs durchsucht. Die erste passende Nachricht wird aus der Liste ausgefügt und in den

Speicherbereich kopiert, den der Empfänger angegeben hat. Liegt keine solche Nachricht vor, so wird (je nach Wert von `flag`) der Empfänger blockiert (blocking receive), oder er kehrt mit einer Fehlermeldung zurück (non-blocking receive).

Die Reihenfolge, in der Nachrichten empfangen werden, ist also prinzipiell FIFO. Ein prioritätengesteuertes Empfangen („wichtigste Nachricht zuerst“) ist zwar möglich, indem man den Nachrichtentyp zur Festlegung der Priorität benutzt, aber recht umständlich zu programmieren.

- Die Funktion `msgctl()` (Prototyp: `int msgctl(int id, int cmd, struct msqid_ds *buf)`) ermöglicht, analog zu `semctl()` und `shmctl()`, verschiedene Steuerungsoperationen auf Message Queues. Dazu gehören die Ausgabe von Informationen über den aktuellen Zustand der Queue (Kommando `IPC_STAT`) und die Löschung der Queue (Kommando `IPC_RMID`).

Ein Programm mit mehreren Prozessen, die über Message Queues kommunizieren, kann nach demselben „Rezept“ wie für UNIX-Semaphore angegeben aufgebaut werden (siehe Ende von Abschnitt 4.1.3). Konkrete Programmbeispiele finden sich im Anhang und bei den Übungsaufgaben am Ende dieses Kapitels.

4.2.5 Sockets

Mit der BSD-Version von UNIX wurde ein weiterer Kommunikationsmechanismus eingeführt, nämlich die so genannten *Sockets*. Sie sind heute ein Standard für UNIX/Linux-Systeme, werden aber auch von anderen Betriebssystemen, wie z.B. Windows NT mit seinen *Winsocks*, und von Programmiersprachen, wie z.B. Java, unterstützt. Damit können auch Anwendungen miteinander kommunizieren, die unter unterschiedlichen Betriebssystemen laufen.

Grundprinzipien

Sockets ermöglichen sowohl eine lokale Kommunikation zwischen Prozessen auf demselben Rechner als auch eine Kommunikation zwischen verschiedenen Rechnern über ein Kommunikationsnetz. Mit Sockets kann man also Kommunikationsprotokolle in das Betriebssystem einbinden, wie z.B. insbesondere TCP/IP für die Benutzung des Internets. Die Einbindung von Protokollen ist ein erster Schritt hin zur „Netzfähigkeit“ eines Betriebssystems (siehe hierzu auch Kapitel 8 und 9).

Eine *Socket* (auf Deutsch wörtlich: „Steckdose“) ist ein Datenendpunkt in einem Kommunikationssystem, in etwa vergleichbar mit einer Telefonanschlussdose. Sie besteht aus mehreren Schichten (siehe Abb. 4.17): Der *Socket-Kopf* ist die Betriebssystemschnittstelle, die nach außen Schnittstellenfunktionen für die Kommunikation anbietet und sie auf die darunter liegenden internen Kommunikationsmechanismen abbildet. Oberhalb des Socket-Kopfs setzen Sender- und Empfängerprozesse auf. Die darunter liegende *Protokollschicht* implementiert ein Kommunikationsprotokoll (z.B. TCP/IP oder UDP/IP – siehe Kapitel 8) und regelt damit die Ende-zu-Ende-Datenüber-

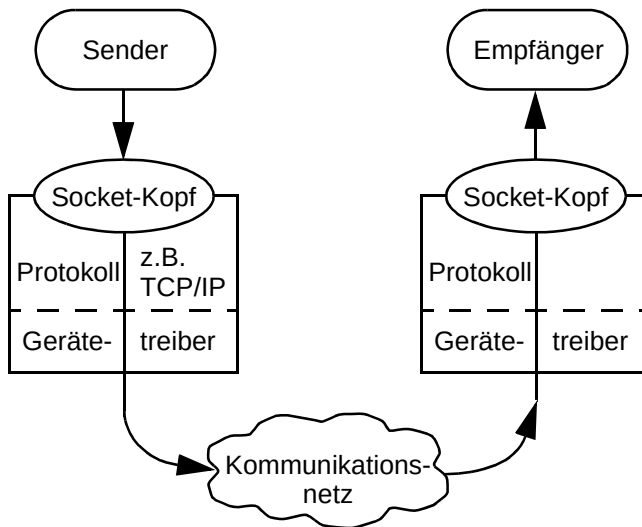


Abb. 4.17 Schichten bei Sockets

tragung zwischen den Datenendpunkten. Diese Übertragung kann über ein Kommunikationsnetz stattfinden, kann aber auch lokal sein. Noch tiefer liegen der *Gerätetreiber*, der den Zugriff auf das Netz steuert, und das *Kommunikationsnetz* selbst, auf dem die Datenübertragung real stattfindet.

Sockets mit gleichen Kommunikationseigenschaften, wie z.B. identischen Adressformaten für Datenendpunkte, bilden eine *Socket Domain*. Wichtig sind hier unter UNIX die *UNIX System Domain* mit Sockets für die lokale Kommunikation auf einer Maschine sowie die *Internet Domain* für die Kommunikation über das Internet.

Zudem kann man mehrere *Socket-Typen* unterscheiden. So gibt es beispielsweise den *Stream-Typ* für eine verbindungsorientierte, zuverlässige Kommunikation und den *Datagramm-Typ* für eine verbindungslose, nicht zuverlässige Kommunikation. „Verbindungslos“ bedeutet, dass eine Nachricht vom Sender zum Empfänger geschickt wird, ohne zuvor eine Verbindung zwischen den beiden Partnern aufzubauen. Eine „verbindungsorientierte“ Kommunikation wird dagegen durch einen solchen Verbindungsaufbau eingeleitet. Bei einer „nicht zuverlässigen“ Kommunikation sind Übertragungsfehler, wie z.B. der Verlust von Nachrichten, nicht ausgeschlossen; „zuverlässige“ Kommunikationsmechanismen nehmen eine Fehlerkorrektur vor. Nähere Informationen zum Thema Kommunikation enthält Kapitel 8.

Sockets in UNIX / Linux

Die Kommunikation über Sockets geht insbesondere in UNIX / Linux nach dem Client-Server-Prinzip vor sich (siehe Abb. 4.18): Zunächst eröffnet ein Server eine Socket und bietet sie anderen Prozessen (Clients) an (Schnittstellenfunktionen `socket()`, `bind()` und `listen()`; Details der Funktionen siehe weiter unten). Ein Client, der

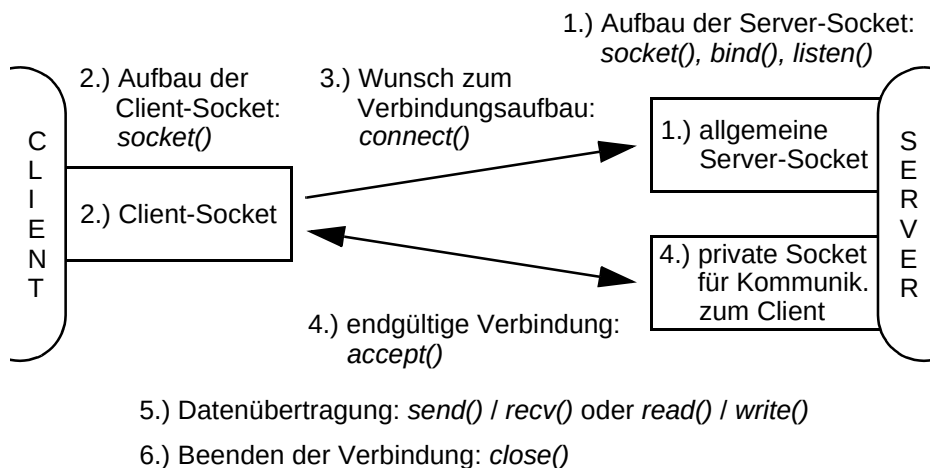


Abb. 4.18 Sockets und Socket-Operationen mit ihrer zeitlichen Reihenfolge

mit dem Server in Verbindung treten möchte, eröffnet eine eigene Socket (Funktion `socket()`) und sendet an die Server-Socket einen Wunsch zum Verbindungsaufbau (`connect()`). Der Server akzeptiert dies und erzeugt dabei eine neue, private Verbindung zu der Client-Socket mit einer neuen Server-Socket (`accept()`). Die folgende Kommunikation zwischen Client und Server findet über diese Verbindung statt (`read()` / `write()` oder `send()` / `recv()`). Zur Beendigung der Kommunikation werden die Sockets geschlossen und die Verbindung abgebaut (`close()`).

Im Folgenden sollen anhand eines Beispielsprogramms (angelehnt an [Bach86]) die genaue Vorgehensweise bei der Kommunikation und die dafür benötigten Schnittstellenfunktionen besprochen werden. Das Programm realisiert eine einfache Sender-Empfänger-Beziehung zwischen einem Client und einem Server, die beide auf demselben Rechner laufen. Ein etwas ausführlicheres Beispiel findet sich im Anhang.

- Server als Empfänger:

```
#include <sys/socket.h>
main() {
    int sd_acc, sd_comm, addr_len;
    char buf[256];
    struct sockaddr server_addr, client_addr;
    sd_acc = socket(AF_UNIX, SOCK_STREAM, 0);
    server_addr.sa_family = AF_UNIX;
    strcpy(server_addr.sa_data, "Socket_1");
    bind(sd_acc, &server_addr, sizeof(struct sockaddr));
    listen(sd_acc, 1);
    sd_comm = accept(sd_acc, &client_addr, &addr_len);
    read(sd_comm, buf, sizeof(buf));
    < Weiterverarbeitung der empfangenen Nachricht. >
```

```

close(sd_acc);
close(sd_comm);
}

```

- Client als Sender:

```

#include <sys/socket.h>
main() {
    int sd_out, error;
    struct sockaddr server_addr;
    sd_out = socket(AF_UNIX, SOCK_STREAM, 0);
    server_addr.sa_family = AF_UNIX;
    strcpy(server_addr.sa_data, "Socket_1");
    error = connect(sd_out,
                   &server_addr, sizeof(struct sockaddr));
    if (error == -1) exit(-1);
    write(sd_out, "Hallo", 6);
    close(sd_out);
}

```

Wir betrachten zuerst die Operationen des Servers als Empfänger: Der Server erzeugt zunächst mit der Schnittstellenfunktion `socket()` eine neue Socket und speichert ihren Deskriptor in der Variablen `sd_acc` ab. Die `socket()`-Funktion hat allgemein die Form `sd = socket(format, type, protocol)`. Der Parameter `format` legt dabei die Socket Domain fest (`AF_UNIX` für die UNIX Domain, `AF_INET` für die Internet Domain), `type` den Socket-Typ (`SOCK_STREAM` für den Stream-Typ, `SOCK_DGRAM` für den Datagramm-Typ) und `protocol` das gewünschte darunter liegende Kommunikationsprotokoll (ist hier der Wert 0, so wählt das System selbst ein geeignetes Protokoll aus). Der Rückgabewert von `socket` ist der *Socket-Deskriptor*, über den in den folgenden Aufrufen auf die Socket zugegriffen werden kann.

Im nächsten Schritt wird mit `bind()` der „externe“ Name „Socket_1“ an die Socket gebunden. Die Socket tritt unter diesem Namen im Dateisystem auf, so dass andere Prozesse ihn benutzen können, um Kommunikationsverbindungen zu der Socket aufzubauen. Die allgemeine Form der Funktion ist `bind(sd, address, length)`, wobei `sd` der Deskriptor der Socket ist. Der Parameter `address` ist ein Zeiger auf eine Struktur, die die Domain sowie den externen Socketnamen angibt. Das Format des Namens hängt vom Kommunikationsprotokoll und der Socket Domain ab. In der UNIX-Domain sind Zeichenketten zugelassen, die den Bildungsregeln für Dateinamen gehorchen; in der Internet-Domain wird die Internet-Adresse des Empfänger-Systems und zusätzlich eine Identifikation der Socket auf dem Empfänger-System (vornehmlich die TCP-Portangabe) verwendet (siehe hierzu z.B. [Stev97] oder [FiMü99]). Der Parameter `length` gibt die Länge des Namens an.

Mit dem folgenden Aufruf von `listen()` zeigt der Server an, dass er nun bereit ist, eingehende Verbindungswünsche für die neue Socket entgegenzunehmen. `listen(sd, qlength)` stellt einen Speicherbereich bereit, in dem maximal `qlength` Anmeldungen auf ihre weitere Bearbeitung warten können.

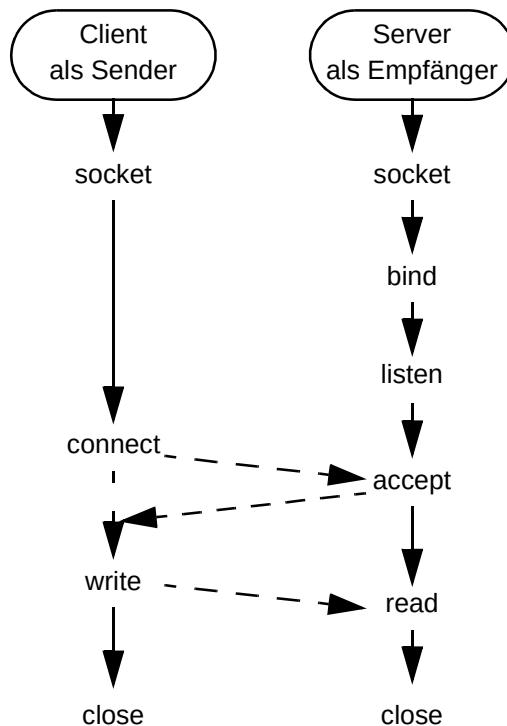


Abb. 4.19 zeitlicher Ablauf der Socket-Kommunikation

Ein solcher Verbindungswunsch kommt vom Client-Prozess, der hier als Sender auftritt. Er hat mit `socket()` eine eigene Socket `sd_out` erzeugt und möchte nun eine Verbindung zwischen ihr und der Socket des Servers aufbauen. Hierzu ruft er `connect()` auf, das die allgemeine Form `connect(sd, address, length)` hat. Der Parameter `address` hat dasselbe Format wie bei `bind()`: Er identifiziert die Socket eines anderen Prozesses, mit der die lokale Socket `sd` verbunden werden soll. `length` ist die Länge des Namens. Der Rückgabewert der Funktion ist eine 0 bei erfolgreichem Verbindungsaufbau und eine -1 bei einem Fehlschlag.

Nur bei Stream-Sockets wird im darunter liegenden Netz eine Verbindung aufgebaut. Bei Datagramm-Sockets wird lediglich ein Vermerk im Betriebssystemkern gespeichert, der angibt, wohin später die Daten gesendet werden sollen.

Der Server akzeptiert eintreffende Verbindungswünsche mit der Funktion `accept()`. `nsd = accept(sd, address, addrlen)` blockiert den aufrufenden Prozess, bis ein Verbindungswunsch für die Socket `sd` vorliegt. Trifft ein solcher Wunsch ein, so wird dem Prozess über den Rückgabeparameter `address` Domain und (falls vorhanden) Name der Socket des Partners mitgeteilt und über `length` dessen Länge. Die Funktion `accept()` erzeugt darüber hinaus eine neue Socket, über die im Folgenden die Kommunikation mit dem Partner stattfindet. Die ursprüngliche Socket `sd` bleibt somit für im Folgenden eingehende Verbindungswünsche frei. Der Rückgabewert von `accept()` ist der Deskriptor der neuen Socket.

Man beachte den Unterschied zwischen `listen()` und `accept()`: `listen()` erzeugt lediglich einen Warteraum für eingehende Verbindungswünsche, die erst durch `accept()` akzeptiert werden.

Die eigentliche Datenübertragung findet mit den Funktionen `send()` und `recv()` oder (als Alternative, die nur für verbindungsorientierte Kommunikation möglich ist) `write()` und `read()` statt. `count = write(sd, msg, length)` sendet über die Socket `sd` die Daten, auf die der Pointer `msg` verweist. Der Parameter `length` gibt die Länge dieses Datenfeldes an (in Bytes). Der Rückgabewert der Funktion ist die Anzahl der tatsächlich gesendeten Daten. Für verbindungslose Kommunikation kann die Funktion `sendto()` benutzt werden, deren Parameter zusätzlich die Empfängeradresse angeben.

`count = read(sd, buf, length)` empfängt Daten von der Socket `sd`. Sie werden in den Speicherbereich übertragen, auf den der Pointer `buf` verweist. Über den Parameter `length` gibt der aufrufende Prozess die maximale Anzahl der Daten an, die er so empfangen kann (in Bytes). Der Rückgabewert der Funktion ist die Anzahl der tatsächlich empfangenen Daten. Für verbindungslose Kommunikation kann die Funktion `recvfrom()` benutzt werden, die zusätzliche Parameter für die Senderadresse hat.

Zum Abschluss wird die Verbindung mit `close()` wieder abgebaut; die Sockets werden geschlossen.

Abb. 4.19 fasst den zeitlichen Ablauf einer Socket-Kommunikation zusammen.

4.3 Übungsaufgaben

1. Wissensfragen

- Welche zwei Klassen von Synchronisationsbedingungen unterscheidet man?
- Welche Nachteile hat ein Spinlock gegenüber einem Semaphor?
- Welche drei Operationen auf einem Semaphor gibt es? (allgemein, nicht nur UNIX!)
- Welchen Vorteil bieten UNIX-Semaphorgruppen?
- Was ist ein Deadlock?
- Welche zwei Arten der Kommunikation unterscheidet man?
- Was ist der Unterschied zwischen einer Mailbox und einem Port?

2. Prozesssynchronisation I

Lösen Sie die folgenden Synchronisationsaufgaben:

- In der Hochschulbibliothek gibt es von Buch A drei Exemplare sowie von Buch B und Buch C je zwei Exemplare. Vor der Diplomarbeit muss ein Diplomand von jedem dieser Bücher je ein Exemplar ausleihen, nach der Diplomarbeit gibt er sie wieder zurück.

- b.) Ein Produktionsbetrieb benötigt, bevor er ein Produkt herstellen kann, von Zulieferer A und Zulieferer B je ein Teil. Produktionsbetrieb und Zulieferer sollen in Endlosschleifen laufen.
- c.) Beim *Leser-Schreiber-Problem* gibt es eine Anzahl von Lese- und Schreibprozessen, die auf einen gemeinsamen Speicher zugreifen. Während die Leser unbeschränkt nebenläufig auf dem Speicher arbeiten können, ist der Zugriff eines Schreibers gegen Zugriffe der anderen Schreiber und der Leser wechselseitig ausgeschlossen. Es sollen hier drei Leser und zwei Schreiber vorhanden sein.

Die Lösung soll im Stil so aussehen wie die drei allgemeinen (also noch nicht UNIX-spezifischen) Anwendungsbeispiele aus Abschnitt 4.1.3: Es muss angegeben werden, welche Semaphore es gibt und wie sie initialisiert werden. Zudem müssen die Prozesse mit ihren Ausführungsschritten skizziert werden.

3. Prozesssynchronisation II

Betrachten Sie eine Schwimmerstaffel mit drei Schwimmern, bei der der zweite Schwimmer erst starten darf, wenn der erste fertig ist, und der dritte erst starten darf, wenn der zweite fertig ist. Vorgeschlagen wird die folgende Lösung:

Initialisierung: `SEM.INIT(0);`

Erster Schwimmer: `< Schwimmen >`
 `SEM.V();`

Zweiter Schwimmer: `SEM.P();`
 `< Schwimmen >`
 `SEM.V();`

Dritter Schwimmer: `SEM.P();`
 `< Schwimmen >`

`SEM.P()` / `SEM.V()` bezeichnet dabei eine P- bzw. eine V-Operation auf dem Semaphore `SEM` mit Dekrement bzw. Inkrement 1.

- a.) Ist die angegebene Lösung korrekt? Begründen Sie Ihre Antwort und skizzieren Sie gegebenenfalls, wie das Programm korrigiert werden muss.
- b.) Erweitern Sie das Programm so, dass die Schwimmer anschließend duschen gehen. Da nur eine Dusche vorhanden ist, soll sichergestellt sein, dass höchstens ein Schwimmer gleichzeitig duscht.
Wie sieht die Lösung aus, wenn es zwei Duschen gibt?
- c.) Erweitern Sie Ihre Lösung so, dass ein Wärter zuerst von allen drei Schwimmern je einen Euro kassiert und dann die Dusche aufschließt.

4. Prozesssynchronisation in UNIX I

Betrachten Sie das folgende Teilstück eines C-Programms auf einem UNIX-System:

```
int semid;
unsigned short initarray[5] = {1,0,1,1,1};
struct sembuf semp[3];
semid = semget(IPC_PRIVATE,5,IPC_CREAT|0777);
semctl(semid,5,SETALL,initarray);
semp[0].sem_num=1;
semp[1].sem_num=3;
semp[2].sem_num=4;
semp[0].sem_op=semp[1].sem_op=semp[2].sem_op=-1;
semp[0].sem_flg=semp[1].sem_flg=semp[2].sem_flg=0;
semop(semid,semp,3);
```

- a.) Was bewirken die Aufrufe von `semget()` und `semctl()`, wie sie in diesem Programm verwendet werden?
- b.) Was ist der Effekt des `semop()`-Aufrufs erstens für den ausführenden Prozess und zweitens für die Werte der Zählvariablen der Semaphore? (Wir gehen davon aus, dass andere Prozesse zwischenzeitlich keine `semop()`-Aufrufe gemacht haben.)
- c.) Was wäre der Effekt, wenn der `semop()`-Aufruf folgendermaßen aussähe:
`semop(semid,&semp[1],2);`?

Hinweis: Nehmen Sie die Funktionsbeschreibungen im Anhang zu Hilfe!

5. Prozesssynchronisation in UNIX II

Setzen Sie Ihre Lösung aus Aufgabe 2.b in ein UNIX-Programm um.

6. Deadlocks

Wie könnte bei der Problemstellung von Aufgabe 2.a ein Deadlock auftreten? Wie lassen sich hier Deadlocks verhindern, vermeiden oder aufheben? Umgangssprachliche Antworten genügen.

7. Prozesskommunikation in UNIX I

Betrachten Sie den folgenden Ausschnitt aus einem Programm, in dem zwei Prozesse kommunizieren: Der Sohnprozess übergibt einen Wert (in diesem Fall die Konstante 10) an den Vaterprozess, der ihn auf den Bildschirm ausgibt.

```

int id, *p;
...
id = shmget(IPC_PRIVATE, sizeof(int), IPC_CREAT|0777);
if (fork()==0) {
    p = (int *) shmat(id, 0, 0);
    *p = 10;
    exit(0); }
p = (int *) shmat(id, 0, 0);
printf("%d\n", *p);

```

- a.) Wie nennt man die Technik zur Datenübergabe, die hier benutzt wird?
- b.) Welche andere Techniken der Datenübergabe gibt es in UNIX? Was ist ihr Vorteil gegenüber der hier verwendeten?
- c.) Ist im gegebenen Programm die Synchronisation von Vater- und Sohnprozess korrekt? Wenn ja: Warum? Wenn nein: Warum nicht, und wie könnte das Programm korrigiert werden?

8. Prozesskommunikation in UNIX II

Schreiben Sie ein Programm mit Funktionen der UNIX-C-Schnittstelle, in dem ein Erzeuger-Prozess zehn Nachrichten in eine Message Queue schickt, die von einem Verbraucher-Prozess ausgelesen werden. Jede Nachricht soll einen `int`- und einen `float`-Wert enthalten, die vom Verbraucher auf den Bildschirm ausgegeben werden.

Weitere Aufgaben zur Prozesssynchronisation und -kommunikation unter UNIX finden sich im Aufgabenteil des Anhangs.

5 Speicherhierarchie

Neben der Zuteilung des Prozessors ist die Organisation des Speichers eine zweite wichtige Verwaltungsaufgabe des Betriebssystems. Die Komponenten des real vorhandenen Speichers, wie z.B. Cache, Hauptspeicher und Magnetplatte, differieren stark voneinander: Ihre Speicherkapazitäten und Zugriffszeiten unterscheiden sich um Größenordnungen. Man kann sagen, dass die Speichereinheiten „unterschiedlich weit vom Prozessor entfernt liegen“, so dass also ein Speicher mit mehreren Stufen, eine so genannte *Speicherhierarchie* (engl. *storage hierarchy*), besteht.

In diesem Kapitel wird zunächst eine kurze Übersicht über die Komponenten dieses Speichersystems gegeben. Anschließend steht die Frage im Mittelpunkt, wie die beiden zentralen Bestandteile der Hierarchie, nämlich Hauptspeicher und Plattenspeicher, gemeinsam effizient verwaltet werden können. Die Diskussion konzentriert sich dabei auf den Bedarf der Prozesse nach direkt zugreifbarem Speicher, um dort ihre Daten und ihren Programmcode unterbringen zu können. Dateien und das Dateisystem werden im folgenden Kapitel besprochen.

5.1 Komponenten der Speicherhierarchie

Abbildung 5.1 skizziert die typische Speicherstruktur eines Rechensystems: Der *Hauptspeicher*, der zur Zentraleinheit gehört, ist mit dem Prozessor über einen Bus verbunden und somit rasch zugreifbar. Der *Plattenspeicher* als Komponente der Peripherie wird über eine E/A-Schnittstelle angesprochen. Er realisiert erstens das Dateisystem und nimmt zweitens Speicherbereiche von Prozessen auf, die momentan keinen Platz im Hauptspeicher finden, dient also zur Erweiterung des Hauptspeichers. Ein *Cache* ist ein relativ kleiner, sehr schnell zugreifbarer Speicher, der an den Bus angeschlossen ist (*Off-Chip Cache*) oder sogar im Prozessor selbst (*On-Chip Cache*) liegt. Daneben gibt es noch weitere Arten von Speichern wie beispielsweise *Disketten*, *CD-ROMs* und *Magnetbänder* mit entsprechenden Laufwerken, die aber hier nicht betrachtet werden sollen.

Anzumerken ist, dass die Abbildung die reale Situation etwas vereinfacht darstellt. Im Allgemeinen sind Prozessor und Hauptspeicher an einen „CPU Memory Bus“ angeschlossen, während es für Peripheriegeräte einen oder mehrere gesonderte Busse (z.B. PCI-Bus) gibt. Die Busse sind über spezielle Elektronik (so genannte „Controller“) miteinander verknüpft.

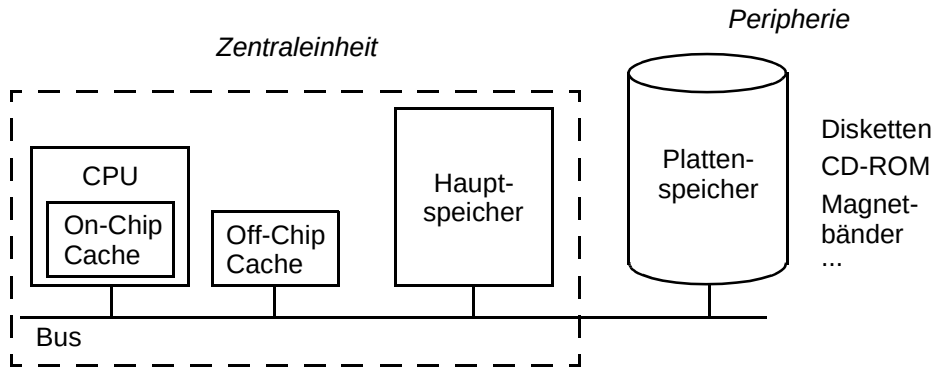


Abb. 5.1 Struktur des Realspeichers

5.1.1 Hauptspeicher

Der Hauptspeicher (engl. *main memory*, auch *RAM = random access memory*) dient als „Arbeitsspeicher“ für die Prozesse, indem er ihre Daten und ihren Programmcode aufnimmt. Der Betriebssystemkern liegt dabei im Allgemeinen vollständig im Hauptspeicher, während Prozesse ganz oder teilweise auf den Plattenspeicher verdrängt werden können.

Struktur

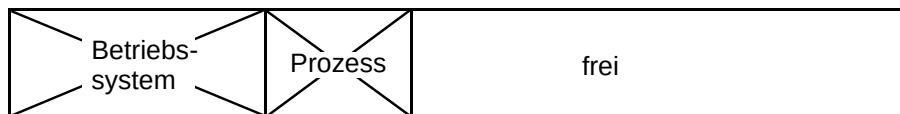
Der Hauptspeicher ist „wortorganisiert“: Seine logischen Einheiten sind *Wörter*, die aus einigen wenigen Bytes bestehen (z.B. vier Bytes pro Wort) und auf die einzeln direkt zugegriffen werden kann. Dabei werden Ganzzahladressen benutzt. Hauptspeicherzugriffe sind zwar relativ schnell (Zugriffsdauer im Nanosekundenbereich), aber langsamer als die Verarbeitungsoperationen des Prozessors.

Hauptspeicherwörter können nicht nur die eigentlichen Nutzdaten, sondern auch zusätzliche FehlererkennungsCodes (engl. *error detection codes, EDC*) oder Fehlerkorrekturcodes (engl. *error correction codes, ECC*) enthalten, mit denen Speicherfehler erkannt und korrigiert werden können. Das einfachste Beispiel eines fehlererkennenden Codes ist das *Parity Bit*: Ein solches Bit ist einer Gruppe von Nutzdatenbits (z.B. einem Byte oder Wort) zugeordnet und wird immer so gesetzt, dass die Anzahl der Einsen in den Nutzdaten und im Parity Bit gerade ist. Bei der Verfälschung eines Bits (also beim „Umkippen“ von 0 auf 1 oder umgekehrt) wird diese Eigenschaft verletzt, so dass der Fehler erkannt werden kann.

Beispiele für fehlerkorrigierende Codes, bei denen Fehler nicht nur erkannt, sondern auch korrigiert werden können, sind der *Hamming Code* und der *Reed Solomon Code*. Die leistungsfähigeren dieser Codes können nicht nur Ein-Bit-Fehler, sondern auch Mehr-Bit-Fehler korrigieren. Sie werden vor allem auch auf externen Speichermedien, wie z.B. CD-ROMs, angewandt.

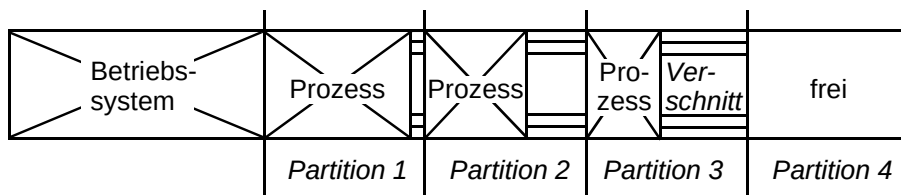
a.) Monoprogramming: nur ein Prozess

Hauptspeicher



b.) Multiprogramming: mehrere Prozesse

b.1.) feste Hauptspeicher-Partitionen einheitlicher Größe



b.2.) feste Hauptspeicher-Partitionen unterschiedlicher Größen

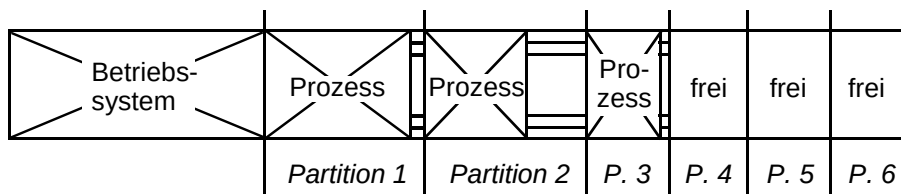


Abb. 5.2 feste Hauptspeicherpartitionen

Arbeitsspeicher für Prozesse

Damit ein Prozess ausgeführt werden kann, müssen Programmcode und Daten im Hauptspeicher liegen. Wir gehen zunächst von der vereinfachenden Annahme aus, dass sich Code und Daten eines Prozesses *vollständig* im Hauptspeicher befinden müssen und dass zudem der Speicherbereich eines Prozesses zusammenhängen, also aus einem Stück bestehen muss. In den Unterkapiteln 5.2 und 5.3 werden wir von diesen Forderungen abgehen und zeigen, wie ein Prozess auf mehrere Bereiche in Haupt- und Plattenspeicher verteilt werden kann.

Im einfachsten Fall, nämlich in einem Monoprogramming-System, ist nur ein Anwendungsprozess vorhanden, der sich den Hauptspeicher mit dem Betriebssystem teilt (siehe Abb. 5.2, Teil a). Hier ist keine aufwendige Speicherverwaltung nötig: Das Betriebssystem belegt einen bestimmten Teil des Speichers, und der Prozess kann über den Rest frei verfügen.

Anders sieht es im Fall von Multiprogramming aus, da hier mehrere Prozesse Hauptspeicherplatz beanspruchen. Das Betriebssystem muss jedem Prozess bei seinem Start einen Hauptspeicherbereich (eine so genannte *Hauptspeicher-Partition*) zuteilen, die

mindestens so viele Bytes umfasst, wie der Prozess fordert. Die Partition wird bei der Terminierung des Prozesses wieder freigegeben und kann anschließend anderen Prozessen zugeteilt werden. Ist keine passende Partition vorhanden, so muss der Prozess warten, oder er wird ganz abgewiesen.

Feste Partitionen

Bei Verwendung von *festen Partitionen* wird der Speicher bei Systemstart in mehrere Bereiche unterteilt, deren Zahl und Größe anschließend nicht mehr geändert wird (siehe Abb. 5.2, Teil b). Die Partitionen können dabei alle dieselbe Größe oder auch unterschiedliche Größen haben. Im Fall von identischen Größen wird einem Prozess bei seinem Start eine beliebige freie Partition zugeteilt – welche, ist gleichgültig. Da ein Prozess in den seltensten Fällen exakt so viele Bytes benötigt, wie eine Partition enthält, bleibt meist ein Teil der Partition ungenutzt. Er kann jedoch nicht von anderen Prozessen verwendet werden, da nur ganze Partitionen zugeteilt werden dürfen. Es entsteht also ein *Verschnitt*. Umgekehrt kann ein Prozess nicht mehrere Partitionen erhalten, so dass die Größe der Partition die Größe von Prozessen beschränkt.

Um das Verschnittproblem zu verringern, kann man die festen Partitionen verschieden groß machen. Hier wird einem Prozess jeweils die Partition zugeteilt, die unter allen freien Partitionen „am besten passt“, bei der also am wenigsten Verschnitt anfällt.

Variable Partitionen

Wesentlich flexibler als das starre Konzept der festen Partitionen ist der Ansatz mit *variablen Partitionen*. Hier kann die Speicheraufteilung dynamisch geändert und somit dem aktuellen Bedarf der Prozesse angepasst werden. Insbesondere kann der Rest einer Partition, den ein Prozess nicht benötigt, einem anderen Prozess zugeteilt werden, und es können benachbarte Partitionen zu einer großen Partition zusammengefasst werden. Der Speicher besteht also logisch aus einer Folge von belegten und freien Bereichen, deren Lage, Größe und Anzahl sich dynamisch ändern kann (siehe Abb. 5.3).

Die Speicherverwaltung stützt sich hier auf eine doppelt verkettete *Freibereichsliste*. Sie enthält für jede freie Partition einen Eintrag, der ihre Anfangsadresse und Länge angibt. Die Einträge sind nach aufsteigenden Anfangsadressen geordnet.

Soll für einen Prozess Speicherplatz belegt werden, so wird in der Freibereichsliste ein geeigneter Bereich gesucht. Enthält dieser genau so viele Bytes, wie der Prozess benötigt, so wird er belegt und der entsprechende Eintrag aus der Liste entfernt. Ist er größer, so wird dem Prozess ein genau passendes Anfangsstück des Freibereichs zugeteilt. Der noch freie Rest bleibt verfügbar; hierzu wird der Listeneintrag entsprechend geändert, indem eine neue Basisadresse eingesetzt und die Längenangabe verringert wird.

Gibt ein Prozess seinen Speicher wieder frei, so wird in die Freibereichsliste ein entsprechender Eintrag eingefügt. Grenzt der neue Freibereich an einen oder sogar zwei bereits bestehende Freibereiche, so werden sie zu einem zusammenhängenden Freibereich „verschmolzen“, der durch einen einzigen Eintrag in der Liste repräsentiert wird.

Zur Suche nach einem passenden Freibereich bei der Speicherbelegung gibt es verschiedene Strategien. Beispielsweise durchsucht *First Fit* die Liste von vorn nach hin-

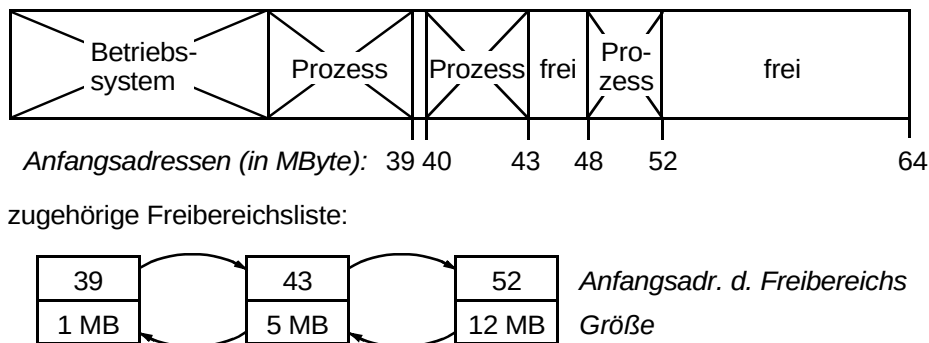


Abb. 5.3 variable Hauptspeicherpartitionen

ten und teilt Speicher aus dem ersten Bereich zu, der mindestens so groß wie gefordert ist. *Best Fit* wählt unter allen freien Bereichen denjenigen, der „am besten passt“, d.h. bei dem nach der Belegung am wenigsten freie Bytes übrig bleiben.

Verschnittprobleme

Auch bei variablen Partitionen können Verschnittprobleme auftreten. Es ist nämlich möglich, dass zwar insgesamt noch genügend Bytes frei sind, um die Forderung eines Prozesses zu erfüllen, dass sich diese Bytes aber über mehrere getrennte Freibereiche verteilen, von denen jeder zu klein für den Prozess ist. Um die Forderung dennoch erfüllen zu können, muss dann eine aufwendige *Speicherkompaktierung* durchgeführt werden, d.h. es müssen belegte Bereiche verschoben werden, um Freibereiche zusammenfassen zu können.

Man beachte, dass das Verschnittproblem hier etwas anders aussieht als bei den festen Partitionen. Dort konnten freie Bytes innerhalb einer insgesamt belegten Partition nicht mehr zugeteilt werden, hier können Bytes außerhalb der belegten Partitionen nicht vergeben werden. Man spricht im ersten Fall von *interner Fragmentierung* und im zweiten Fall von *externer Fragmentierung*.

Das Verschnittproblem tritt besonders bei der Best-Fit-Strategie auf, da hier bei der Belegung oft nur kleine Freibereiche übrig bleiben, die für weitere Forderungen zu kurz sind. Aus diesem Grund ist Best Fit in der Praxis weniger effizient als First Fit. Um dem Verschnittproblem beizukommen, kann man daher auch die *Worst-Fit-Strategie* anwenden, bei der jeweils Speicherplatz aus dem größten Freibereich zugeteilt wird.

Adressverwaltung

Ein weiteres Problem bei der Speicherbelegung ist die Adressverwaltung: Die Maschinenbefehle im Code eines Prozesses arbeiten auf Daten, die in Hauptspeicherzellen abgelegt sind, und müssen dazu die Zellen adressieren. Dabei ist zu berücksichtigen, dass ein Prozess bei aufeinander folgenden Ausführungen in unterschiedlichen Bereichen

des Hauptspeichers liegen kann – je nachdem, wo ihm gerade Speicherplatz zugeteilt wurde. Der Code kann also nicht ohne weiteres absolute Hauptspeicheradressen verwenden.

Zur Lösung dieses Problems könnte man bei jeder Neuplatzierung des Prozesses alle Adressen in seinem Code ändern. Dies ist aber in der Praxis bei weitem zu aufwendig. Praktikabler ist die Verwendung eines speziellen *Basisregisters*, das jeweils mit der Anfangsadresse des Speicherbereichs des Prozesses besetzt wird. Hier ist eine Adresse im Prozesscode nur der so genannte *Offset* zu dieser Basisadresse, gibt also die Position einer Speicherzelle relativ zur Anfangsadresse des Speicherbereichs an (Zahlenbeispiel: Basisadresse = 4500, Offset einer Prozessvariablen = 100 $\Rightarrow$ reale Adresse der Variablen = 4600). Bei der Prozessausführung werden diese Offsets jeweils automatisch durch die Hardware zur Basisadresse addiert.

In Unterkapitel 5.3 werden wir unter dem Stichwort „virtueller Speicher“ eine Verallgemeinerung dieses Ansatzes besprechen.

5.1.2 Cache

Cache-Speicher (deutsche Übersetzung: „Pufferspeicher“) sind vom Prozessor aus direkt zugreifbar und ähneln damit dem Hauptspeicher. Sie haben aber eine wesentlich kürzere Zugriffszeit als dieser, da schnellere Hardwarebausteine verwendet werden, die zudem näher am Prozessor liegen. Ein *On-Chip Cache* befindet sich sogar auf demselben Chip wie der Prozessor, ein *Off-Chip Cache* ist dagegen an den Bus angeschlossen. Caches sind relativ klein, denn ihre Hardware ist aufwendig und teuer, und zudem ist, im Fall von On-Chip Caches, nur beschränkt Platz vorhanden. On-Chip Caches sind typischerweise einige wenige Kilobyte groß, Off-Chip Caches mehrere hundert Kilobyte bis zu mehreren Megabyte. Cache-Speicher können in einer Hierarchie angeordnet werden; man spricht dann von *Level-1*-, *Level-2*- und (manchmal) *Level-3-Caches*.

Funktionsweise

Ein Cache enthält Kopien der Hauptspeicherbereiche, die momentan am häufigsten benutzt werden. Bei jedem Hauptspeicherzugriff wird geprüft, ob die gewünschten Bytes im Cache liegen. Ist das der Fall, kann unmittelbar, d.h. sehr schnell, aus dem Cache gelesen oder in den Cache geschrieben werden. Der Cache ist trotz seiner geringen Größe sehr effizient, da Programme meist recht lange auf einer eng begrenzten Menge von Daten und Code arbeiten, die größtenteils im Cache gespeichert werden können. So liegen die „Trefferquoten“ in Caches meist weit über 90 Prozent.

Neben den Lese- und Schreibzugriffen des Prozessors gibt es zwei weitere Cache-Operationen: Hauptspeichereinträge können in den Cache kopiert („geladen“) werden. Das wird zumeist bei einem *Zugriffsfehler* gemacht, d.h. wenn der gewünschte Eintrag im Cache nicht vorgefunden wird. Um im Cache Platz zu schaffen, können Einträge aus ihm entfernt („verdrängt“) werden. Üblicherweise sind das die Einträge, die am längsten nicht mehr benutzt wurden. Die Operationen arbeiten auf Speicherblöcken, wobei ein Block einige wenige Wörter des Hauptspeichers umfasst.

Organisationsformen

Die genaue Vorgehensweise beim Laden und Verdrängen hängt von der Organisationsform des Caches ab (siehe Abb. 5.4):

- In einem *voll-assoziativen Cache* kann ein Hauptspeicherblock an jeder beliebigen Stelle stehen, ein Block also beim Laden an jede freie Cache-Position gebracht werden. Neben dem Blockinhalt wird dabei die Nummer des Blocks gespeichert (sie ergibt sich unmittelbar aus der zugehörigen Hauptspeicheradresse – z.B. durch das Abschneiden der letzten zwei Adressbits, wenn ein Block vier Wörter enthält). Darüber hinaus gibt ein *Valid Bit* an, ob die Cache-Position momentan gültige Daten enthält.

Möchte ein Prozess auf eine bestimmte Hauptspeicheradresse zugreifen, so werden alle Cache-Positionen anhand der gespeicherten Blocknummern überprüft, ob dort der gewünschte Eintrag steht. Dies geschieht an allen Positionen gleichzeitig, wofür spezielle Hardware (ein so genannter *Assoziativspeicher*) benutzt wird.

Voll-assoziative Caches sind sehr flexibel, aber wegen der komplexen Hardware teuer.

- In einem *Direct-Mapped Cache* kann jeder Block nur an genau einer bestimmten Position stehen. Die entsprechende Cache-Adresse ergibt sich dabei unmittelbar aus der Hauptspeicheradresse des Blocks, z.B. als die Restklasse der Blocknummer bezüglich der Cache-Größe (Rechenbeispiel: Bei der Blocknummer 55 und einer Cache-größe 8 ist die Restklasse 7). Offensichtlich werden so eine Vielzahl von Hauptspeicherblöcken auf dieselbe Cache-Position abgebildet.

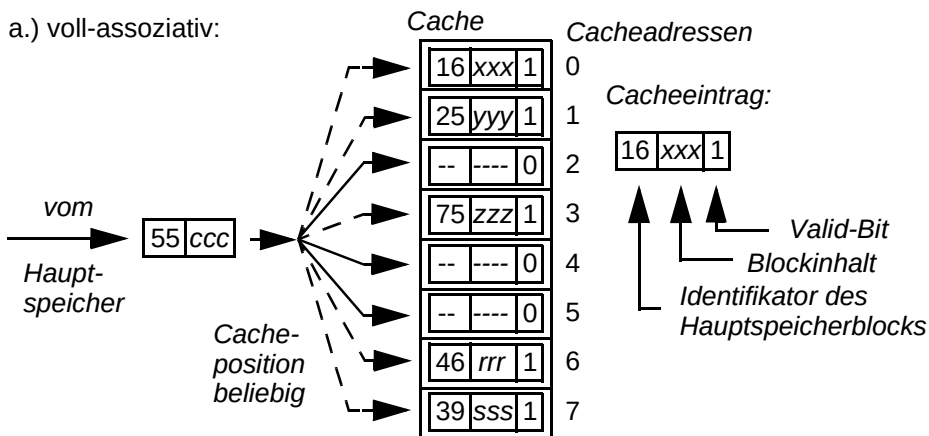
Nicht nur beim Laden, sondern auch beim Lesen und Schreiben von Daten wird die Cache-Adresse unmittelbar aus der Hauptspeicheradresse der Daten abgeleitet. Allerdings muss dabei auch überprüft werden, ob sich dort tatsächlich der gewünschte Block und nicht ein anderer befindet. Hierzu steht an der Position die Blocknummer oder eine entsprechende andere Information. Auch hier ist ein *Valid Bit* vorhanden.

Mit dieser direkten Adressierung ist der Direct-Mapped Cache weniger aufwendig zu realisieren, da kein teurer Assoziativspeicher benötigt wird. Diese Vorteile werden erkaufte durch eine geringere Flexibilität und damit eine niedrigere Trefferquote. Es kann hier nämlich beim Laden eines Blocks vorkommen, dass ein anderer aus seiner Position verdrängt werden muss, obwohl noch andere Stellen frei sind.

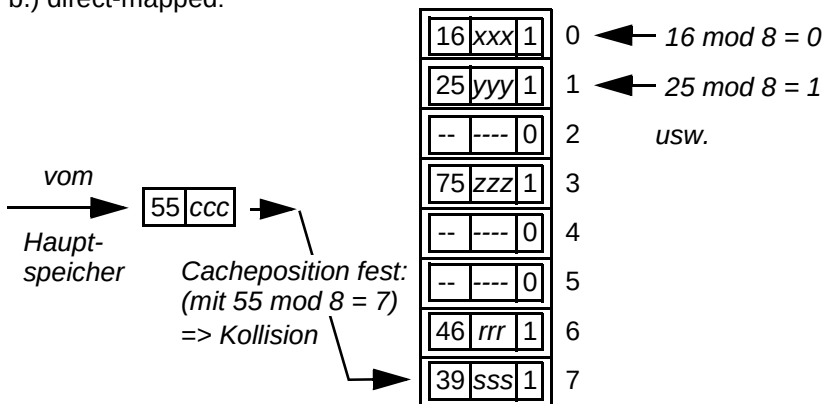
- Der *Mengen-assoziative Cache* ist eine Mischform aus den beiden anderen Ansätzen. Hier wird jedem Block eine Menge von Cache-Positionen eindeutig zugeordnet, wobei sich die Menge ähnlich wie oben durch Restklassenbestimmung ergibt. Innerhalb dieser Menge kann der Block dann an einer beliebigen Stelle gespeichert werden.

Wird in den Cache geschrieben, so muss auch der zugehörige Hauptspeichereintrag geändert werden. Bei einem *Write-Through Cache* erfolgt die Änderung sofort, bei einem *Copy-Back Cache* erst dann, wenn der Block aus dem Cache verdrängt wird. Offensichtlich wird bei einem Copy-Back Cache tendenziell seltener auf den Hauptspeicher zugegriffen als bei einem Write-Through Cache. Dafür können bei Copy-Back Cache- und Hauptspeichereinträge voneinander differieren, was für E/A-Operationen in den bzw. aus dem Hauptspeicher ungünstig sein kann.

a.) voll-assoziativ:



b.) direct-mapped:



c.) Mengen-assoziativ:

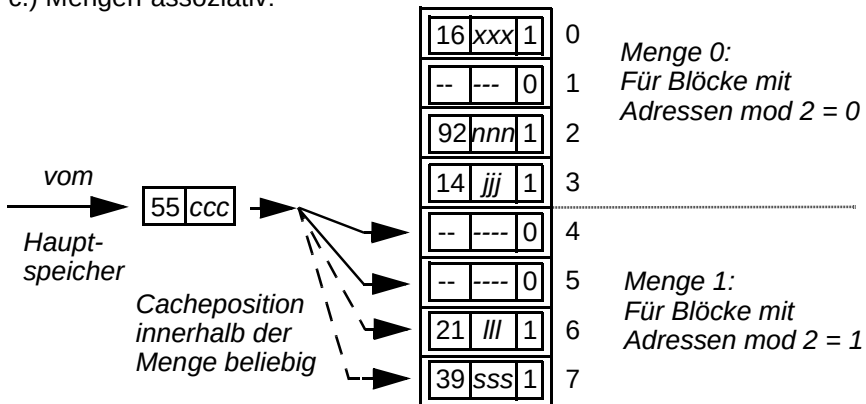


Abb. 5.4 Cacheorganisation

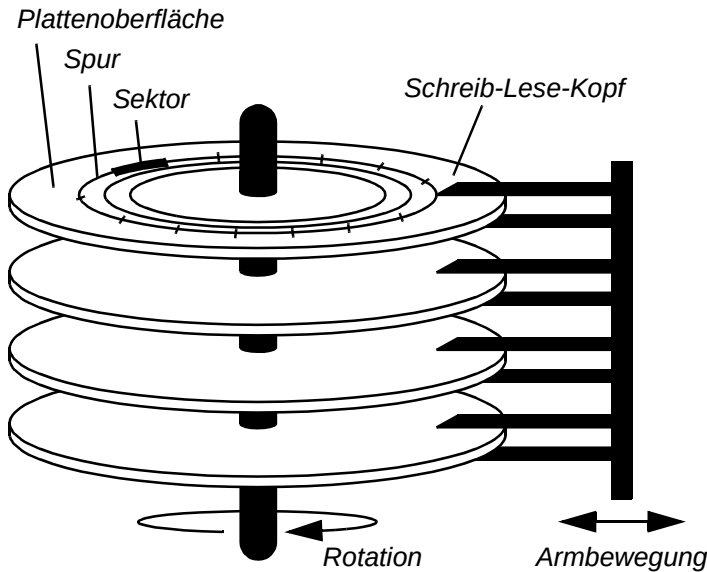


Abb. 5.5 Organisation des Plattenspeichers

5.1.3 Plattenspeicher

Der Plattenspeicher (*Magnetplatte*, *Festplatte*, engl. *magnetic disk*, *hard disk*) ist wesentlich größer als der Hauptspeicher. Er kann damit als Ergänzung oder Erweiterung des Hauptspeichers genutzt werden und wird daher auch verallgemeinernd als *Hintergrundspeicher* oder *Sekundärspeicher* (engl. *secondary storage*) bezeichnet. Zugriffe auf den Plattenspeicher sind jedoch erheblich langsamer als Hauptspeicherzugriffe; ihre Dauer liegt im Millisekundenbereich. Zudem können Daten auf der Platte nicht direkt vom Prozessor verarbeitet werden, sondern müssen dazu zuerst in den Hauptspeicher geladen werden.

Struktur

Wie Abbildung 5.5 zeigt, ist der Sektor (engl. *sector*) die kleinste adressierbare Einheit eines Plattenspeichers. Sektoren haben eine konstante, einheitliche Länge – typischerweise 512 Bytes. Bei einem Plattenzugriff werden meist mehrere zusammenhängende Sektoren zu einem *Cluster* zusammengefasst und gemeinsam gelesen oder geschrieben.

Sektoren werden gemäß der Hardwarestruktur des Plattenspeichers adressiert: Der Speicher besteht aus mehreren magnetischen Scheiben, die auf einer Achse stecken und jeweils zwei *Oberflächen* haben. Jede Oberfläche enthält mehrere konzentrische *Spuren* (engl. *tracks*), die wiederum in die Sektoren unterteilt sind. Eine Sektoradresse besteht also aus drei Komponenten: der Nummer der Oberfläche, der Nummer der Spur und der Nummer des Sektors innerhalb der Spur. Nach außen bietet der Plattencontrol-

ler allerdings im Allgemeinen einen eindimensionalen Adressraum von aufsteigenden *Blocknummern* an, die intern umgerechnet werden. Bei Plattenzugriffen spricht man daher von *Plattenblöcken* und nicht von Sektoren.

Zugriffe

Die Achse des Plattenspeichers rotiert mit einer konstanten Geschwindigkeit (meist 7200 Umdrehungen pro Minute oder höher). An einem Arm, der quer zu den Spuren bewegt werden kann, sind *Schreib-Lese-Köpfe* angebracht – pro Oberfläche einer. Über die Schreib-Lese-Köpfe können Daten auf die Plattenoberfläche geschrieben oder von ihr gelesen werden. Bei einem Zugriff auf einen bestimmten Sektor wird der entsprechende Schreib-Lese-Kopf ausgewählt, der Arm auf die gesuchte Spur platziert und abgewartet, bis sich der Sektor unter dem Arm hinwedgeht.

Ein Zugriff auf einen Sektor dauert mit mehreren Millisekunden relativ lange, da drei recht zeitaufwendige Schritte hintereinander ausgeführt werden müssen: Der Arm muss auf die richtige Spur positioniert, die Ankunft des gesuchten Sektors muss abgewartet und die Bytes müssen sequentiell übertragen werden. Die Zugriffszeit ist also die Summe der Zeit für die Positionierung des Arms, der *Rotationsverzögerung* (= Wartezeit auf den Sektor) und der eigentlichen Übertragungsdauer. Die ersten beiden Summanden (und damit auch die gesamte Zugriffszeit) sind nicht konstant: Die Positionierungszeit wird bestimmt von der Anzahl der Spuren, die überquert werden müssen; dazu kommt noch ein konstanter Zeitaufwand zum Starten und Stoppen des Arms. Die Rotationsverzögerung hängt ab von der Position des Sektors auf der Spur und der Umdrehungsgeschwindigkeit der Platte.

Buffer Cache

Um Plattenzugriffe zu beschleunigen, stellen die meisten Betriebssysteme im Hauptspeicher (oder auch die Platte selbst in einem eigenen RAM) einen so genannten *Buffer Cache* bereit. Er enthält die Inhalte der Plattenblöcke, die momentan am meisten benötigt werden. Da der Buffer Cache wesentlich rascher zugreifbar ist als der rotierende Magnetspeicher, wird hierdurch die mittlere Zugriffszeit für Plattenblöcke erheblich gesenkt. Der Buffer Cache ist nicht zu verwechseln mit dem Prozessor-Cache aus Abschnitt 5.1.2!

Plattenscheduling

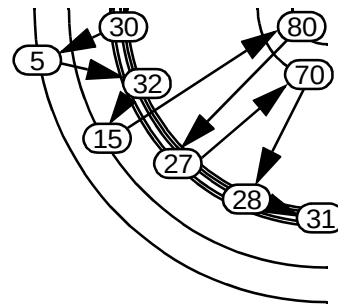
An einem Plattenspeicher steht häufig eine Warteschlange von Aufträgen zur Bearbeitung an. Eine Anforderung muss also oft die Erledigung mehrerer anderer Aufträge abwarten, bevor sie selbst an die Reihe kommt. Um die so entstehenden Wartezeiten nicht übermäßig lang werden zu lassen, benötigt man effiziente Strategien zum *Plattenscheduling*. Sie sollen die anstehenden Aufträge in einer Reihenfolge bearbeiten, bei der der Gesamtaufwand möglichst gering ist und gleichzeitig Aufträge „fair“ behandelt werden. Hierfür sind die folgenden Algorithmen denkbar (siehe Abb. 5.6):

Ausgangsspur: 30

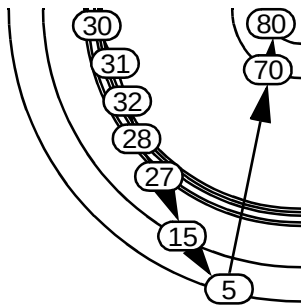
Spuren der Aufträge
(in Ankunftsreihenfolge):

5, 32, 15, 80, 27, 70, 28, 31

a.) First Come First Served:



b.) Shortest Seek Time First:



c.) Fahrstuhl (SCAN):

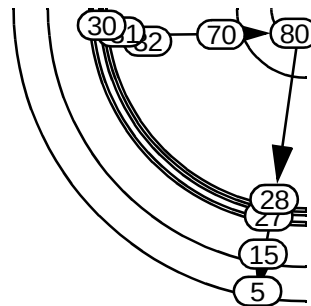


Abb. 5.6 klassische Verfahren zum Plattenscheduling

- *First Come First Served (FCFS)* bearbeitet die Aufträge in der Reihenfolge, in der sie eingetroffen sind. Diese Vorgehensweise erscheint zwar fair, kann aber zu einem unnötig großen Aufwand für die Positionierungen des Arms führen. Es ist hier nämlich nicht ausgeschlossen, dass unmittelbar aufeinander folgende Aufträge jeweils weit auseinanderliegende Spuren betreffen, so dass der Arm weite Wege zurücklegen muss.
- *Shortest Seek Time First (SSTF)* bearbeitet jeweils den „nächstliegenden“ Auftrag. Das ist unter allen Aufträgen derjenige, dessen Spur der Spur am nächsten liegt, auf der der Arm gerade steht; für ihn ist die Positionierungszeit am kürzesten. Diese Vorgehensweise senkt zwar den Aufwand für Armbewegungen recht effektiv, ist aber nicht fair: Bei hohem Anforderungsaufkommen, d.h. beim Eintreffen immer neuer Aufträge nahe der aktuellen Spur, werden weit entfernt liegende Spuren nur sehr spät oder, im Extremfall, überhaupt nicht erreicht. Es kommt also zum *Verhungern* (engl. *starvation*) von Aufträgen.
- Der *Fahrstuhl-Algorithmus* (engl. *elevator algorithm*, *SCAN algorithm*) geht so vor wie manche Aufzugssteuerungen: Er bewegt den Arm so lange in dieselbe Richtung

(also von innen nach außen oder umgekehrt), wie Aufträge für Spuren in dieser Bewegungsrichtung vorliegen, und überquert dabei Spur für Spur. Trifft er dabei auf eine Spur, für die ein Auftrag vorhanden ist, so wird dieser bearbeitet. Gibt es keine Aufträge in der aktuellen Richtung mehr, so kehrt der Arm um und arbeitet in der Gegenrichtung weiter. Offensichtlich sind hier, im Gegensatz zu SSTF, die Wartezeiten der Aufträge beschränkt – das Verfahren zeigt also eine bessere Fairness.

Eine Variante des Algorithmus setzt nach dem Erreichen der innersten gewünschten Spur den Arm unmittelbar nach außen zurück und arbeitet erneut von außen nach innen. Die Fairness ist hier noch besser als bei der beidseitig arbeitenden Version, da eine Anforderung maximal nur einen Durchgang des Arms über die Spuren und nicht zwei abwarten muss.

In der Praxis wird der Fahrstuhl-Algorithmus dem SSTF-Verfahren vorgezogen. FCFS spielt keine praktische Rolle.

Realzeitaspekte

In Realzeitsystemen (siehe Abschnitt 3.4.1) kommt es auch beim Plattenscheduling darauf an, Fristen einzuhalten. Hierzu sind die oben angeführten Verfahren gar nicht bzw. (Fahrstuhl-Algorithmus) nur bedingt geeignet. An ihrer Stelle kann, wie bei der Prozessorzuteilung, ein *fristenbasiertes Scheduling* eingesetzt werden: Es wird vorausgesetzt, dass jeder Auftrag eine Frist hat, d.h. einen Zeitpunkt, zu dem er spätestens fertiggestellt sein muss. Fristenbasiertes Scheduling bearbeitet die Aufträge in aufsteigender Reihenfolge ihrer Fristen (siehe Abb. 5.7, Teil a).

Bei fristenbasiertem Scheduling kann es allerdings, wie bei FCFS, zu einem übermäßigen Springen des Plattenarms kommen. Es sollten daher besser Aufträge mit zeitlich benachbarten Fristen zu einer Gruppe zusammengefasst werden. Innerhalb einer Gruppe kann dann der Fahrstuhlalgorithmus angewandt werden; die Fristen werden benutzt, um die Reihenfolge der Gruppen untereinander zu bestimmen (siehe Abb. 5.7, Teil b).

In speziellen Anwendungen kann alternativ die *Balanced-Buffers-Strategie* eingesetzt werden (siehe Abb. 5.7, Teil c). Hier wird angenommen, dass von der Platte nebenläufig mehrere Dateien gelesen werden, die jede für sich gesehen sequentiell, d.h. von vorn nach hinten durchlaufen werden. Dies ist z.B. beim Abspielen von Multimediadateien (also Audio- oder Videostreamen) der Fall. Jeder Datei wird im Buffer Cache des Hauptspeichers ein eigener Pufferbereich fester Größe zugeteilt, der mit einer unteren und einer oberen Füllmarke (engl. *low* bzw. *high water mark*) versehen ist. Unterschreitet der Füllstand des Puffers die untere Marke, droht er also leer zu laufen, so werden von der Platte bevorzugt Blöcke der zugehörigen Datei gelesen, um den Puffer wieder aufzufüllen. Umgekehrt werden Puffer mit einem Füllstand oberhalb der oberen Marke vorerst nicht mehr bedient. Liegen alle Puffer innerhalb ihrer Füllgrenzen, kann ein klassisches Verfahren wie z.B. der Fahrstuhlalgorithmus benutzt werden.

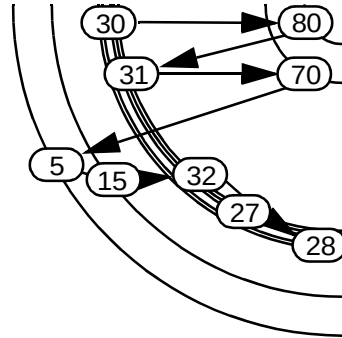
Neben dem Scheduling der Auftragsbearbeitung ist auch die Platzierung der Daten auf der Platte wichtig: Dateien sollten möglichst zusammenhängend abgespeichert werden, um lange Suchzeiten zu vermeiden. Wir werden hierauf in Abschnitt 6.1.4 zurückkommen.

a.) Fristenbasiertes Scheduling (reine Form):

Ausgangsspur: 30

Spuren der Aufträge (mit Fristen):

5, 32, 15, 80, 27, 70, 28, 31
 (21, 30, 22, 10, 31, 13, 34, 12)



b.) Fristenbasiertes Scheduling (mit Gruppenbildung):

Ausgangsspur: 30

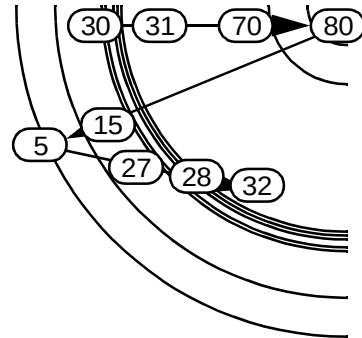
Spuren der Aufträge (mit Fristen):

5, 32, 15, 80, 27, 70, 28, 31
 (21, 30, 22, 10, 31, 13, 34, 12)

Gruppe I: Spuren 31, 70, 80
 (Fristen 12, 13, 10)

Gruppe II: Spuren 15, 5
 (Fristen 22, 21)

Gruppe III: Spuren 27, 28, 32
 (Fristen 31, 34, 30)



c.) Balanced-Buffers-Strategie zum Abspielen von Audio- und Videoströmen:

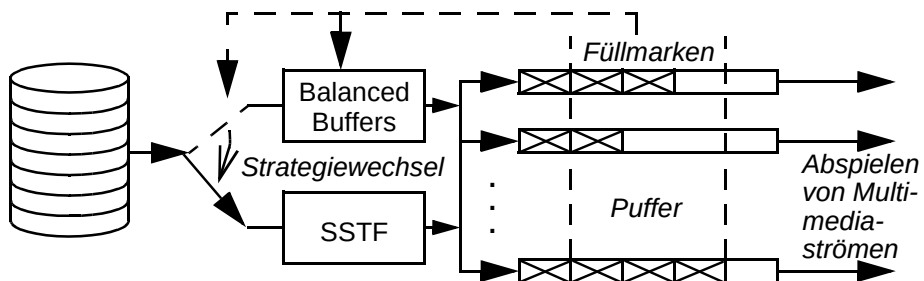


Abb. 5.7 Strategien zum Plattenscheduling mit Berücksichtigung von Realzeitforderungen

5.2 Swapping

Prozesse haben einen großen Speicherbedarf – insbesondere wenn sie moderne Standardsoftware ausführen. Der Hauptspeicher ist im Allgemeinen zu klein für alle aktiven Prozesse, so dass zusätzlich der Plattenspeicher herangezogen werden muss, um Daten und Code der Prozesse aufzunehmen. Hierbei können entweder ganze Prozesse oder auch nur Teile von ihnen auf die Platte „ausgelagert“ werden. Zunächst soll unter dem Stichwort *Swapping* die Auslagerung vollständiger Prozesse diskutiert werden.

5.2.1 Allgemeines Schema

In einem Swappingssystem steht ein Prozess entweder vollständig im Hauptspeicher oder vollständig auf dem Plattenspeicher. Der Plattenspeicher stellt hierzu (meist getrennt vom Dateisystem) einen speziellen Bereich bereit, den so genannten *Swap Space* (siehe Abb. 5.8). Alternativ kann für das Swapping eine normale Datei verwendet werden, was aber für Zugriffe weniger effizient ist.

Prozesse können aus dem Hauptspeicher in den Swap Space *ausgelagert* und später von dort wieder in den Hauptspeicher *eingelagert* werden. Ein Prozess wird ausgelagert, um im Hauptspeicher Platz für andere Prozesse zu schaffen oder wenn er dort selbst momentan nicht mehr genügend Platz hat. Hierfür werden seine Daten und sein Programmcode vollständig in den Swap Space gebracht und anschließend seine Hauptspeicherbereiche freigegeben. Im Hauptspeicher verbleiben dann nur noch einige Basisinformationen über den Prozess, beispielsweise in UNIX sein Prozesstabelleneintrag. Ein Prozess kann natürlich nicht ausgeführt werden, solange er ausgelagert ist.

Um einen Prozess einzulagern, muss zunächst ein ausreichend großer Bereich des Hauptspeichers bereitgestellt werden (oder mehrere Bereiche, falls ein Prozess wie in UNIX über mehrere Regionen verfügt). Ist nicht genügend Platz frei, so müssen zuvor andere Prozesse ausgelagert werden. Anschließend werden die Daten und der Code des Prozesses aus dem Swap Space in den Hauptspeicherbereich eingelesen, so dass der Prozess wieder ausführbar wird.

Zur Verwaltung des Hauptspeichers, also zur Belegung und Freigabe von Hauptspeicherbereichen, können die Verfahren aus Abschnitt 5.1.1 benutzt werden. Auch der Swap Space mit seinen Freibereichen kann auf diese Weise verwaltet werden. Darüber hinaus muss das Swappingssystem Buch über die aktuellen Standorte der Prozesse führen, was z.B. in der Prozesstabelle geschehen kann.

5.2.2 Swapping in UNIX System V

Swapping wurde und wird beispielsweise im klassischen UNIX System V benutzt. Im Folgenden sollen für diese UNIX-Version drei Problemfelder näher betrachtet werden, nämlich die Verwaltung der Bereiche des Plattenspeichers sowie die Auslagerung und die Einlagerung von Prozessen. Die Darstellung orientiert sich dabei an [Bach 86].

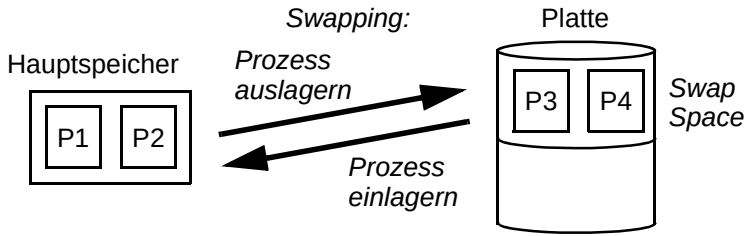


Abb. 5.8 allgemeines Schema beim Swapping

Verwaltung des Plattenspeichers

UNIX System V stellt auf dem Plattenspeicher einen oder sogar mehrere Swap Spaces bereit, die unabhängig vom Dateisystem verwaltet werden. Wird ein Prozess ausgelagert, so ordnet ihm das Betriebssystem dynamisch einen passenden Bereich des Swap Spaces zu. „Dynamisch“ bedeutet dabei, dass dem Prozess bei jeder Auslagerung ein Bereich neu zugeteilt wird, den er bei seiner Einlagerung wieder freigibt. Solange sich ein Prozess im Hauptspeicher befindet, wird für ihn also kein Platz im Swap Space bereitgehalten.

Der Swap-Space-Bereich eines Prozesses ist zusammenhängend, im Unterschied zu seinem Hauptspeicheranteil, der ja in mehrere Regionen zerfällt. Der Grund hierfür ist, dass das Ein- und Auslagern bei einem zusammenhängenden Plattenbereich schneller ablaufen kann als bei einer Aufteilung auf mehrere Bereiche.

Die Freibereiche des Swap Space werden durch eine so genannte *Map* beschrieben, d.h. einen Array, der eine sortierte Freibereichsliste realisiert (siehe schematische Darstellung in Abb. 5.9). Er liegt im Hauptspeicher und ist somit rasch zugreifbar. Die Map enthält für jeden Freibereich einen Eintrag mit seiner Basisadresse auf der Platte und seiner Länge, die in Plattenspeicherblöcken angegeben wird. Speicheranforderungen werden nach der First-Fit-Strategie erfüllt, wobei der Array entsprechend aktualisiert wird (gegebenenfalls durch Entfernen des Map-Eintrags und „Zusammenschieben“ der restlichen Einträge). Bei einer Speicherfreigabe wird die Map so aktualisiert, wie es weiter oben im Zusammenhang mit der Freibereichsliste diskutiert wurde.

Darüber hinaus implementiert UNIX Funktionen zur Behandlung des Verschnittproblems im Swap Space.

Auslagerung

Ein Prozess wird ausgelagert, wenn für ihn selbst oder einen anderen Prozess nicht mehr genügend Hauptspeicherplatz vorhanden ist. Das trifft in den folgenden Fällen zu:

- Ein Prozess, der durch `fork()` neu entsteht, wird sofort ausgelagert, wenn er nicht im Hauptspeicher untergebracht werden kann. Allerdings ist seine baldige Wiedereinlagerung wahrscheinlich, da er im Zustand „bereit“ ist und das Swapping-Verfahren bevorzugt ausführungsbereite Prozesse einlagert (siehe unten).

Freispeicher-Map:

Anfang	Länge
2	1
5	2
10	1
19	3

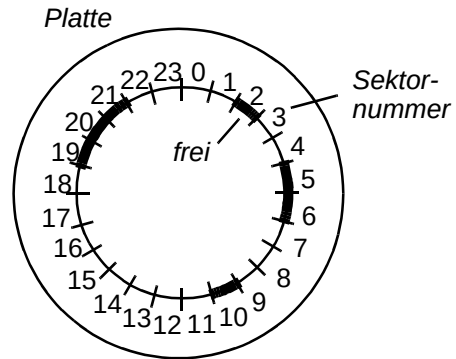


Abb. 5.9 Freispeicher-Map für den „UNIX Swap Space“

- Ein Prozess, der seinen Hauptspeicherplatz erweitern möchte, jedoch keinen zusätzlichen Speicher bekommen kann, wird sofort ausgelagert.
- Soll ein „wichtiger“ Prozess aus dem Swap Space eingelagert werden und ist für ihn kein Hauptspeicherplatz vorhanden, so wird für ihn ein anderer, „unwichtiger“ Prozess verdrängt.

Bei einer Auslagerung werden zunächst sämtliche Regionen des Prozesses in den Swap Space gebracht, sofern nicht noch andere Prozesse auf sie zugreifen. In der Prozesstabelle und der Regionentabelle wird vermerkt, wo im Swap Space die Bestandteile des Prozesses stehen. Anschließend werden die Hauptspeicherbereiche des Prozesses freigegeben.

Je nach Implementation können die Operationen zur Auslagerung entweder durch den Prozess selbst oder durch einen anderen speziellen Prozess, nämlich den *Swapper*, ausgeführt werden.

Einlagerung

Für das Einlagern von Prozessen ist der Swapper-Prozess zuständig, der die Prozessnummer 0 trägt. Er wird stets im Kernel Mode ausgeführt und hat die höchste Priorität aller Prozesse, erhält also sofort den Prozessor, wenn er ausführungsbereit wird. Der Swapper wird beim Hochfahren des Systems gestartet und führt anschließend eine Endlosschleife mit den folgenden Operationen aus:

1. Warte, bis im Swap Space ein Prozess im Zustand „bereit“ vorhanden ist.
2. Setze P_rein = „derjenige der bereiten Prozesse, der am längsten im Swap Space steht“.

3. Wenn genug Hauptspeicherplatz für P_{rein} vorhanden ist:
 - 3.1. Lagere P_{rein} ein.
 - 3.2. Gehe zum Schleifenanfang 1. zurück.
4. Wenn nicht genug Hauptspeicherplatz für P_{rein} vorhanden ist:
 - 4.1. Wenn ein blockierter Prozess im Hauptspeicher vorhanden ist:
Setze P_{raus} = „derjenige der Prozesse im Hauptspeicher im Zustand 'blockiert', der die größte Summe 'Prioritätswert + Zeit im Hauptspeicher' hat“.
 - 4.2. Wenn kein blockierter Prozess im Hauptspeicher vorhanden ist:
Setze P_{raus} = „derjenige der Prozesse im Hauptspeicher, der die größte Summe 'Zeit im Hauptspeicher + nice-Wert' hat“.
 - 4.3. Lagere P_{raus} aus.
 - 4.4. Gehe zum Schleifenanfang 1. zurück.

Um zu verhindern, dass Prozesse nach ihrer Einlagerung sofort wieder ausgelagert werden (oder umgekehrt), beachtet der Swapper die folgende Randbedingung: Ein ein- oder ausgelagerter Prozess kann nicht vor Ablauf von 2 Sekunden wieder aus- bzw. eingelagert werden. In einem solchen Fall blockiert sich der Swapper bis zum Ablauf der Zwei-Sekunden-Frist, es sei denn, es gibt jeweils noch andere Kandidaten zum Ein- oder Auslagern.

Der System-V-Swapper bemüht sich also, ausführungsbereite Prozesse im Hauptspeicher zu halten, während blockierte Prozesse mit niedriger Priorität in den Swap Space gebracht werden. Es gibt allerdings bei diesem Algorithmus mehrere Probleme. So geht beispielsweise die Größe der Hauptspeicherbereiche, die die Prozesse benötigen, nicht in die Auslagerungsentscheidung ein. Es kann also sein, dass ein Prozess ausgelagert wird, obwohl er gar nicht genügend Platz für den einzulagernden Prozess freigibt. Außerdem können Prozesse wieder ausgelagert werden, die seit ihrer Einlagerung noch nicht weiter ausgeführt wurden – die Einlagerung war also vergeblich. Schließlich kann sogar unter sehr speziellen Bedingungen ein Deadlock auftreten [Bach86].

Probleme mit Swapping

Allgemein lässt sich feststellen, dass das Swapping zwar einige, aber nicht alle Probleme des beschränkten Hauptspeicherplatzes löst. Ein „Problemfall“ sind beispielsweise Systeme, in denen die Prozesse viele E/A-Operationen ausführen und daher häufig blockieren. Hier ist es sinnvoll, möglichst viele Prozesse im Hauptspeicher zu halten, damit wenigstens einer von ihnen ausführungsbereit ist. Möglicherweise ist der Hauptspeicher aber zu klein, um alle diese Prozesse aufzunehmen. Im Extremfall, nämlich bei Prozessen mit vielen Daten und großen Programmen, ist der Hauptspeicher eventuell sogar für einen einzelnen Prozess zu klein, der dann überhaupt nicht ausgeführt werden kann.

5.3 Virtueller Speicher

Swapping löst, wie wir gesehen haben, die betrachtete Problemstellung nur unvollständig. Bessere Möglichkeiten bietet hier ein Konzept, bei dem ein Prozess auch nur teilweise im Hauptspeicher stehen kann, während sein Rest auf dem Plattenspeicher liegt. Es handelt sich dabei um das Konzept des *virtuellen Speichers*: Hier wird Prozessen und ihren Maschinenprogrammen die Existenz eines großen, nach Art des Hauptspeichers zugreifbaren Speichermediums vorgespiegelt, das dann durch Betriebssystem und Hardware auf der Basis des realen Haupt- und Plattenspeichers realisiert wird.

5.3.1 Grundlegende Überlegungen

Bislang wurde ohne weitere Diskussion angenommen, dass Prozesse in ihren Programmen reale Hauptspeicheradressen benutzen: Der Code eines Prozesses greift also auf die Zellen des Hauptspeichers unmittelbar über deren Hardwareadressen zu (siehe Abb. 5.10). Dieser „naive“ Ansatz führt aber, insbesondere in einem Multiprogramming-System, zu schwerwiegenden Problemen:

Probleme mit realer Hauptspeicheradressierung

Die Schwierigkeiten bei der Verwendung realer Hauptspeicheradressen in Maschinenprogrammen lassen sich wie folgt aufschlüsseln:

- Ein Prozess muss bei seiner Ausführung vollständig im Hauptspeicher liegen, da er aus seinem Code heraus keine anderen Speichermedien direkt ansprechen kann. Das führt zu denselben Problemen, wie sie oben bei Swapping diskutiert wurden.
- Bei der Erstellung des Maschinencodes für einen Prozess ist im Allgemeinen noch nicht bekannt, in welchen Hauptspeicherbereich der Prozess später einmal geladen wird. Die Adressen im Code müssen also später beim Einlesen in den Hauptspeicher angepasst werden.
- Mit realen Hauptspeicheradressen kann ein Prozess auf beliebige Speicherzellen zugreifen, also auch Bereiche anderer Prozesse lesen oder verändern.

Für das zweite und dritte Problem gibt es praktikable Lösungen: Die Adressumrechnung wird vermieden durch den Einsatz eines Basisregisters, relativ zu dem adressiert wird (siehe Abschnitt 5.1.1). Der Zugriff auf fremde Bereiche wird verhindert mit Hilfe eines Längenregisters, das die Größe des Hauptspeicherbereichs eines Prozesses angibt. Zusammen mit dem Basisregister legt es also die Grenzen des Prozesses fest. Bei jedem Speicherzugriff wird durch die Hardware überprüft, dass diese Grenzen nicht überschritten werden

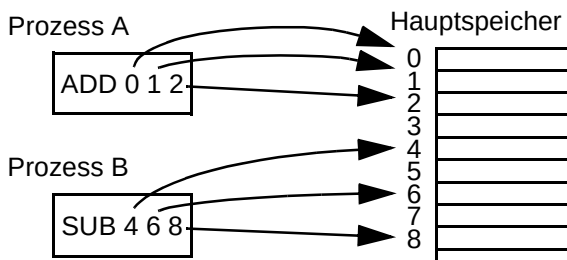


Abb. 5.10 Benutzung realer Hauptspeicheradressen

Das erste Problem bleibt aber bestehen: Die Beschränkungen durch die zu geringe Hauptspeichergroße können nur schwer überwunden werden, wenn die reale Hauptspeicheradressierung beibehalten wird. Zwar könnte hier eine *Overlay-Technik* benutzt werden, bei der ein Prozess selbst Teile seines Speichers lädt und verdrängt. Sie führt allerdings ebenfalls zu zahlreichen Schwierigkeiten und ist daher heute nicht mehr Stand der Technik.

Anforderungen

Wegen der genannten Schwierigkeiten ist es sinnvoll, von der Verwendung realer Adressen im Prozesscode wegzukommen. Mit anderen Worten: Die Vorstellung, die ein Prozess von seinem Speicher hat, sollte von den Realitäten der Speicherhierarchie aus Haupt- und Plattenspeicher so weit wie möglich und sinnvoll entkoppelt werden. Ein Prozess soll auf einem Speicher arbeiten können, der seinen Bedürfnissen entspricht, ohne sich mit den Beschränkungen der realen Hardware befassen zu müssen. Ein solcher Speicher soll die folgenden Ansprüche erfüllen:

- Der Speicher soll so groß sein, dass alle Komponenten des Prozesses, also alle seine Daten und sein vollständiger Programmcode, hineinpassen.
- Der Speicher soll direkt zugreifbar sein, d.h. der Prozess soll in seinem Programmcode die Speicherinhalte unmittelbar ansprechen und bearbeiten können. Er soll sie also nicht zuvor in einen anderen, dafür besser geeigneten Speicher transportieren müssen, wie es z.B. beim Auslesen von Daten aus einer Datei nötig ist.
- Beim Speicherzugriff soll ein einheitliches („homogenes“) Adressierungsschema verwendet werden. Der Prozess soll sich also nicht um unterschiedliche Adressformate wie z.B. bei Haupt- und Plattenspeicher kümmern müssen.
- Andere Prozesse sollen nicht auf den Speicher eines Prozesses zugreifen können – es sei denn, der „Eigentümer“ hätte es ihnen explizit erlaubt (Stichwort „Shared Memory“).

Ein Analogbeispiel

Vor der detaillierten Diskussion eines Realisierungsansatzes betrachten wir ein Analogbeispiel, um das Problem und den Weg zur Lösung zu veranschaulichen (siehe Abb. 5.11): In diesem Beispiel besitzen drei Reisende jeweils eine Anzahl von Koffern, die sie von Zeit zu Zeit in einer Gepäckaufbewahrung abstellen möchten – entweder alle oder nur einen Teil davon. Die Koffer sind nummeriert und sollen anhand ihrer Nummer wiedergefunden werden können (Abb. 5.11, oben).

Man könnte hier für die Gepäckablage einen abschließbaren Raum bereitstellen, in dem mehrere Fächer zur Aufnahme jeweils eines Koffers vorhanden sind (Abb. 5.11, Mitte). Für den Raum ist nur ein Schlüssel vorhanden, damit nur jeweils ein Kunde auf die dort abgestellten Koffer (nämlich seine eigenen) zugreifen kann. Dieser Ansatz bringt jedoch ähnliche Probleme wie die oben besprochenen mit sich: Erstens kann nicht mehr als ein Kunde gleichzeitig den Gepäckraum benutzen. Zweitens muss der Kunde sich selbst merken, wo er welchen Koffer abgelegt hat. Drittens kann der Kunde nicht alle seine Gepäckstücke unterbringen, wenn er mehr Koffer hat, als Fächer vorhanden sind.

Sinnvoll ist daher, einen Verwalter für den Gepäckraum einzustellen, der den Kunden das Einordnen und Suchen der Koffer abnimmt (Abb. 5.11, unten): Er nimmt Koffer von Kunden entgegen, ordnet sie in freie Fächer ein und vermerkt in einer Tabelle, wo er welches Gepäckstück abgelegt hat. Sind alle Fächer belegt, so kann er Koffer in ein externes Zweitlager bringen, um Platz zu schaffen. Beim Abholen muss der Kunde nur seine eigene Koffernummer nennen, worauf der Verwalter in der Tabelle nachsieht und dann das Gepäck von der richtigen Stelle holt.

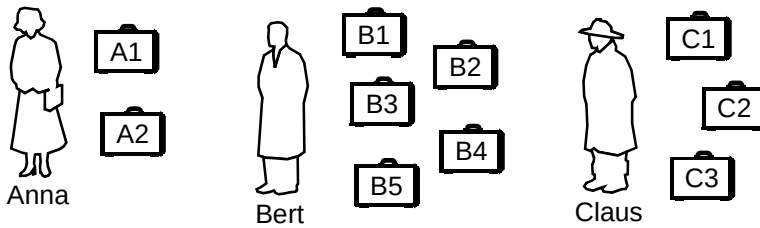
Offensichtlich hat dieser Ansatz mehrere Vorteile: Da ein großes Zweitlager vorhanden ist, kann ein Kunde (fast) unbegrenzt viele Koffer abgeben. Der Kunde braucht sich nicht um die interne Speicherorganisation zu kümmern, sondern muss sich nur seine eigenen Koffernummern merken. Schließlich können mehrere Kunden den Gepäckraum gleichzeitig benutzen, da der Verwalter einen Koffer nur seinem Besitzer aushändigt (es sei denn, im Einzelfall ist etwas anderes vereinbart).

In einem Rechensystem entsprechen die Gepäckfächer dem Hauptspeicher und das Zweitlager dem Plattenspeicher. Als „Verwalter“ gibt es Dienste des Betriebssystems und der Hardware zur Speicherverwaltung. Die Kunden mit ihren Koffern schließlich sind die Prozesse mit ihren Daten und ihrem Code, die im Speicher untergebracht werden müssen. Sie haben also dabei die Vorstellung eines Speichers, der unbegrenzt groß ist, in dem Daten über hardwareunabhängige Adressen (die „Koffernummern“) adressiert werden können und in dem die Daten verschiedener Prozesse gegenseitig geschützt sind.

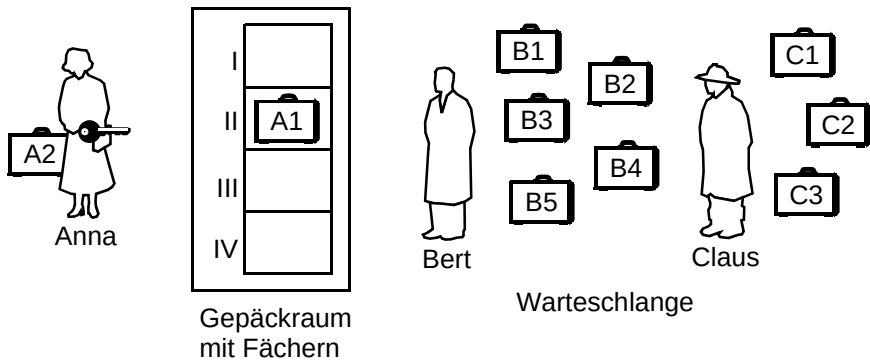
Der Begriff des virtuellen Speichers

Ein Speicher, wie er im letzten Teilabschnitt geschildert wurde, erfüllt also die Bedürfnisse der Prozesse, ist aber nicht unmittelbar real in Hardware vorhanden. Es handelt sich somit um einen *virtuellen Speicher*, also einen Speicher, der nur in der Vorstellung existiert. Die Aufgabe des Betriebssystems liegt nun darin, den virtuellen Speicher mit

Reisende mit Koffern:



Lösung 1: Gepäckraum mit einem Schlüssel



Lösung 2: Gepäckraum mit Verwalter

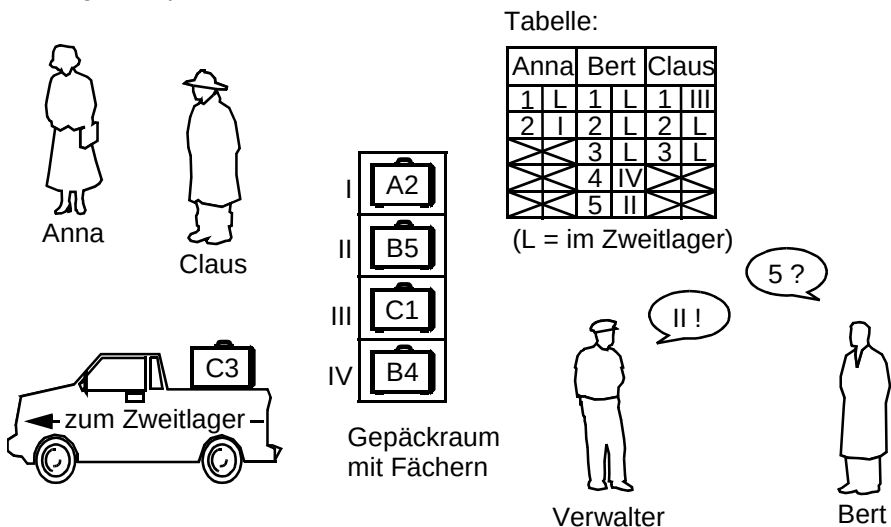


Abb. 5.11 virtueller Speicher: ein Analogbeispiel

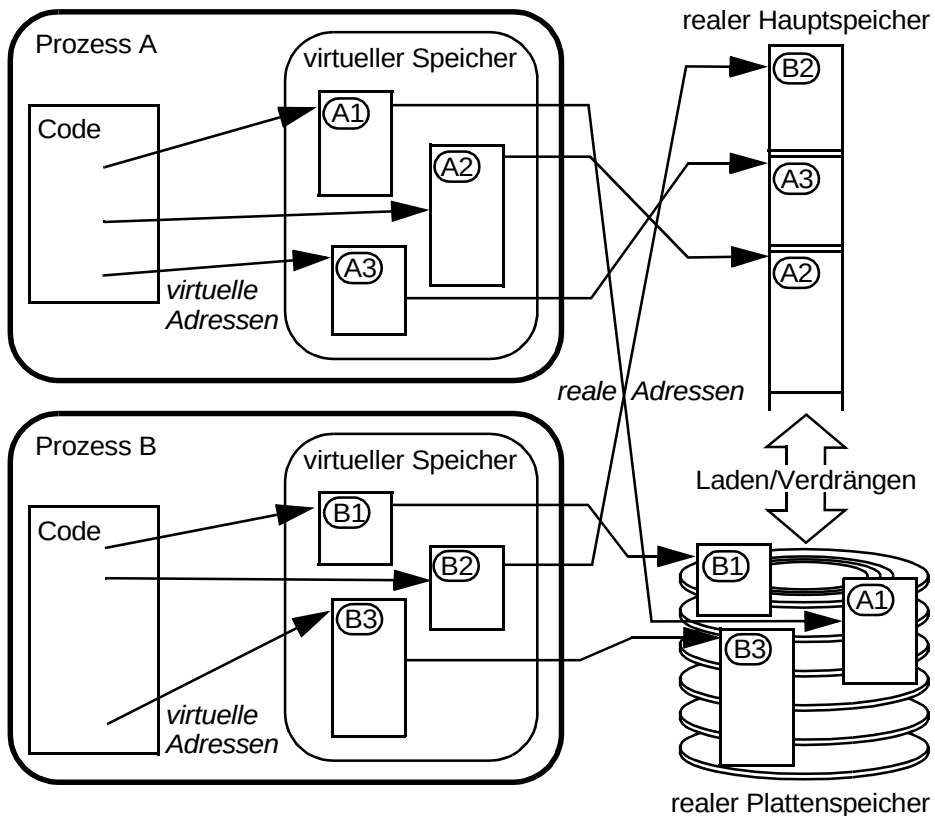


Abb. 5.12 virtueller Speicher

Hilfe der tatsächlich vorhandenen Speicherbetriebsmittel zu realisieren. Es muss also – wie im Beispiel der Verwalter – Mechanismen bereitstellen, die den Prozessen die Existenz eines solchen virtuellen Speichers vorspiegeln und sich dabei auf die reale Speicherhierarchie stützen (siehe Abb. 5.12).

Die allgemeine Vorgehensweise sieht dabei folgendermaßen aus: Der Code und die Daten eines Prozesses werden real teilweise im Haupt- und teilweise auf dem Plattenspeicher abgelegt, wobei nur die Teile im Hauptspeicher gehalten werden, mit denen der Prozess gerade arbeitet. Es befinden sich jeweils mehrere Prozesse gleichzeitig im Hauptspeicher und können somit unmittelbar ausgeführt werden.

Jedem Prozess wird auf dieser Basis ein eigener virtueller Speicher geboten, auf den nur er zugreifen darf, nicht jedoch andere Prozesse (Ausnahme: Shared Memory). Der virtuelle Speicher ist wesentlich größer als der Hauptspeicher und wird im Maschinenprogramm direkt über so genannte *virtuelle Adressen* adressiert.

Zur Realisierung dieses virtuellen Speichers teilt das Betriebssystem, wie gesagt, jedem Prozess Bereiche des Haupt- und des Plattenspeichers zu, in denen seine Speichereinträge real untergebracht werden. Greift der Prozess auf seinen Speicher zu, so wird die dabei benutzte virtuelle Adresse dynamisch auf eine *reale Adresse* abgebildet, d.h.

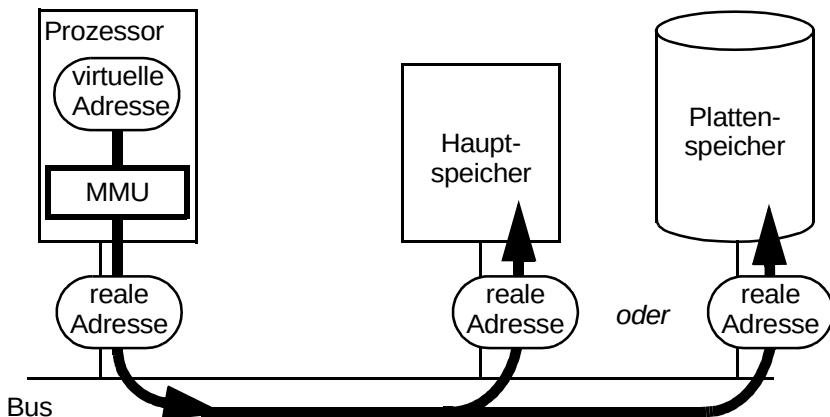


Abb. 5.13 Memory Management Unit (MMU)

in eine Hauptspeicher- oder Plattenadresse umgerechnet. „Dynamisch“ bedeutet, dass die Umrechnung erst zum Zeitpunkt des Zugriffs stattfindet. Das Resultat kann von Mal zu Mal unterschiedlich ausfallen – je nachdem, wo sich die Daten gerade befinden. Eine virtuelle Adresse ist also nicht fest an eine bestimmte Speicherstelle gebunden.

Die Adressumwandlung wird durch die Hardware der *Memory Management Unit* (MMU) vorgenommen, die auf dem Pfad zwischen dem Prozessor und dem Speicher liegt (siehe Abb. 5.13). Befinden sich dabei die gewünschten Einträge nicht im Hauptspeicher, so liest sie das Betriebssystem von der Platte in den Hauptspeicher ein, um sie so zugreifbar zu machen. Um Platz zu schaffen, müssen eventuell andere Hauptspeichereinträge auf die Platte verdrängt werden. Der Algorithmus zur Umrechnung der Adressen stellt sicher, dass sich aus einer virtuellen Adresse nur reale Adressen für solche Speicherbereiche ergeben, die dem ausführenden Prozess „gehören“.

Zur Organisation des virtuellen Speichers gibt es einerseits den *segmentorientierten Ansatz*, bei dem der Speicher in Einheiten unterschiedlicher Größen aufgeteilt ist, und andererseits den *seitenorientierten Ansatz*, bei dem alle Speichereinheiten gleich lang sind. Zunächst soll der segmentorientierte Ansatz besprochen werden.

5.3.2 Segmentorientierter Speicher

Beim segmentorientierten Ansatz besteht der virtuelle Speicher eines Prozesses aus einer Anzahl unterschiedlich großer Einheiten, den so genannten *Segmenten* (siehe Abb. 5.14). Die Aufteilung des Speichers in Segmente geschieht nach logischen Gesichtspunkten; z.B. kann ein Prozess ein Code-, ein Daten- und ein Stacksegment besitzen, oder auch jeweils mehrere davon. Jedes Segment des virtuellen Speichers wird auf einen zusammenhängenden Realspeicherbereich mit entsprechender Länge abgebildet, der entweder im Haupt- oder auf dem Plattenpeicher steht. Heutige Prozessorhardware ermöglicht einem Prozess die Benutzung mehrerer tausend Segmente mit einer jeweiligen Größe im Gigabytebereich.

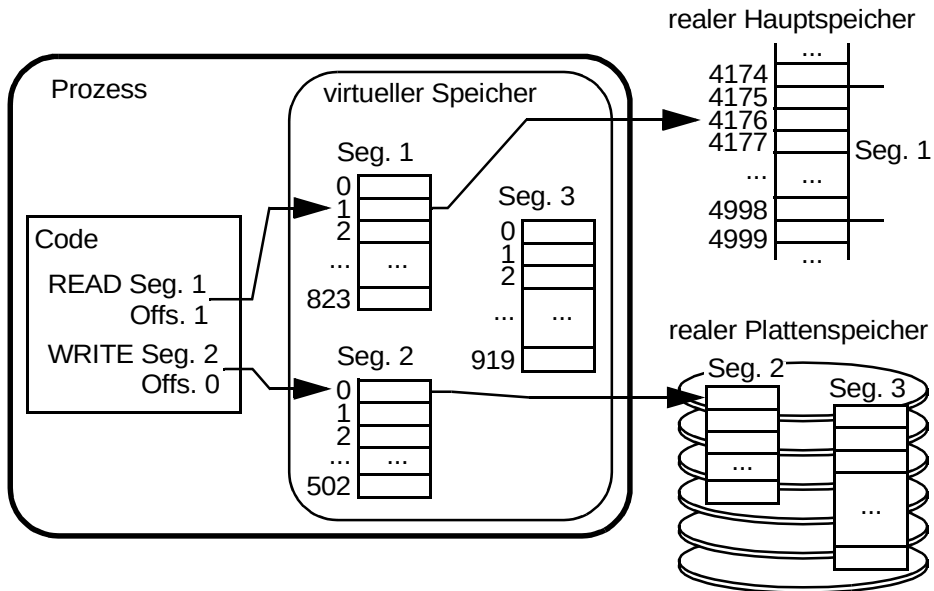


Abb. 5.14 Schema des segmentorientierten virtuellen Speichers

Adressumrechnung

Eine virtuelle Adresse besteht hier aus einer Segmentnummer und einem Offset (= „Positionsnummer“ eines Speicherworts innerhalb des Segments). Der Prozess gibt damit an, auf welches seiner Segmente und auf welche Position innerhalb dieses Segments er zugreifen möchte.

Die Umrechnung von virtuellen in reale Adressen stützt sich auf *Segmenttabellen* (engl. *segment table*, siehe Abb. 5.15). Jeder Prozess besitzt eine eigene Tabelle, die – ähnlich einem Array – über die Segmentnummern indiziert wird. Die Segmenttabelle eines Prozesses enthält für jedes seiner Segmente einen eigenen Eintrag mit den folgenden Informationen:

- Reale Anfangsadresse des Segments (im Haupt- oder auf dem Plattenspeicher).
- Länge des Segments.
- *Valid Bit* (gibt an, ob sich das Segment momentan auf dem Plattenspeicher oder im Hauptspeicher befindet).
- Zugriffsrechte (z.B. Berechtigungen zum Lesen und Schreiben des Segmentinhalts).

Zur Adressberechnung werden die folgenden Schritte ausgeführt: Über die Segmentnummer wird in der Tabelle der Eintrag für das Segment ermittelt. Anhand des Valid Bits wird festgestellt, ob sich das Segment momentan auf dem Plattenspeicher befindet.

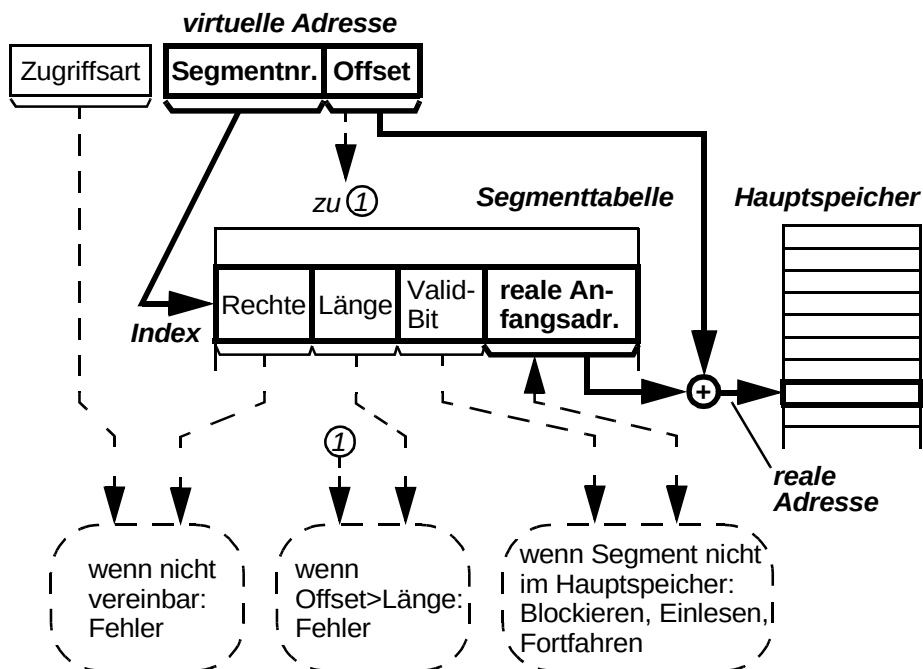


Abb. 5.15 Adressberechnung bei segmentorientiertem Speichern

Ist das der Fall, so tritt ein Trap-Ereignis (ein so genannter *Segmentfehler*, engl. *segment fault*) ein: Der Prozess wird blockiert, und der Trap Handler (ein Unterprogramm des Betriebssystems) liest das Segment in den Hauptspeicher ein. Eventuell müssen zuvor andere Segmente aus dem Hauptspeicher verdrängt werden, um Platz zu schaffen. Anschließend wird die neue Hauptspeicheradresse des Segments in die Segmenttabelle eingetragen, das Valid Bit gesetzt und der Prozess entblockiert. Zudem wird geprüft, ob der angegebene Offset kleiner als die Länge des Segments ist und ob der Zugriff mit den gegebenen Rechten vereinbar ist. Ist eine dieser Bedingungen verletzt, so wird (ebenfalls durch ein Trap-Ereignis) der Zugriff und dabei zumeist auch der ganze Prozess abgebrochen. Ansonsten wird die reale Hauptspeicheradresse berechnet, indem der Offset auf die Anfangsadresse des Segments, die sich aus der Tabelle ergibt, aufaddiert wird. Mit ihr wird dann auf den Hauptspeicher zugegriffen.

Um die Adressberechnung zu beschleunigen, kann der Prozessor *Segmentregister* bereitstellen, die schneller zugreifbar sind als die Tabelle im Hauptspeicher. Sie nehmen die Tabelleneinträge für die Segmente auf, die momentan in Benutzung sind.

Man beachte, dass dieselbe virtuelle Adresse auf unterschiedliche reale Adressen abgebildet wird, wenn verschiedene Prozesse sie benutzen (Ausnahme wiederum: Shared Memory). Die Prozesse arbeiten nämlich auf jeweils ihren eigenen Segmenttabellen, in denen unter derselben Segmentnummer im Allgemeinen verschiedene reale Adressen eingetragen sind (siehe Abb. 5.16). Dies ist auch unbedingt erforderlich, da ja die Speicherbereiche der Prozesse voneinander getrennt sein sollen.

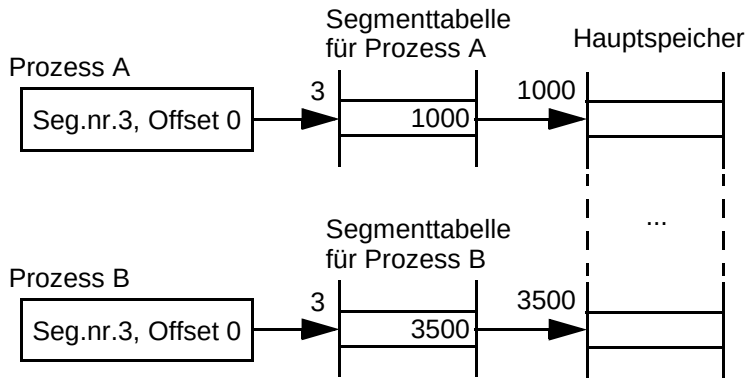


Abb. 5.16 unterschiedliche Segmenttabellen für unterschiedliche Prozesse

Vor- und Nachteile des segmentorientierten Ansatzes

Der segmentorientierte Ansatz hat eine Reihe von Vorteilen. So entsprechen die Segmente unmittelbar den logischen Speichereinheiten eines Prozesses, was die Programmierung erleichtert. Segmente, und damit die logischen Einheiten, können individuell durch Zugriffsrechte geschützt und einzeln auf den Plattenspeicher verdrängt werden. Da jeder Prozess seine eigene Segmenttabelle und damit auch seine eigenen Segmente besitzt, sind die Prozesse wirksam voneinander abgeschottet. Sie können jedoch auch leicht auf bestimmten Segmenten gemeinsam arbeiten, indem deren Tabelleneinträge in mehrere Segmenttabellen kopiert werden oder, alternativ, indirekt über eine gemeinsame Shared Segment Table zugegriffen wird. Schließlich können Segmente dynamisch wachsen und schrumpfen und so dem aktuellen Bedarf angepasst werden.

Leider gibt es aber auch mehrere Nachteile: Um ein Segment abzuspeichern zu können, wird ein zusammenhängender Speicherbereich gebraucht, der möglicherweise recht groß ist. Das erschwert insbesondere die Verwaltung des Hauptspeichers, denn möglicherweise müssen Daten umgespeichert werden, um ausreichend große Bereiche zu schaffen. Darüber hinaus treten Verschnittprobleme auf, da die Segmente unterschiedlich groß sind. Schließlich muss bei einem Zugriff das vollständige Segment im Hauptspeicher stehen, obwohl vielleicht nur ein kleiner Teil davon benötigt wird. Das führt bei großen Segmenten zu einem unnötig hohen Bedarf an Hauptspeicherplatz und zu einem unnötig großen Zeitaufwand für Einlagerungen.

5.3.3 Seitenorientierter Speicher (Paging)

Der segmentorientierte Ansatz mit Speicherbereichen unterschiedlicher Größen ist zwar vorteilhaft für die Programmierung, aber aufwendig für die Speicherverwaltung. Beim seitenorientierten Ansatz, der nun vorgestellt wird, sind die Vor- und Nachteile genau umgekehrt verteilt. Hier ist der virtuelle Speicher in eine Vielzahl gleich großer Einheiten unterteilt, die auf reale Speicherbereiche derselben Länge abgebildet werden.

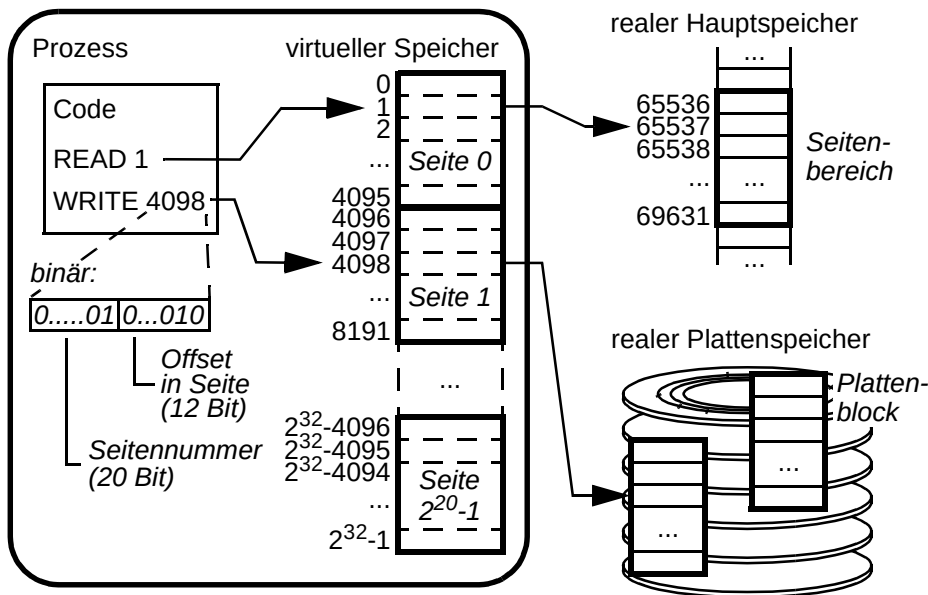


Abb. 5.17 Schema des seitenorientierten Speichers

Die Benutzung von Bereichen identischer Länge vereinfacht die Speicherverwaltung erheblich.

Grundprinzipien des seitenorientierten Speichers

Wie Abbildung 5.17 zeigt, besteht ein seitenorientierter virtueller Speicher aus einer Menge von Speicherwörtern, die „linear“ (also in einer Folge) angeordnet und von 0 beginnend durchnummeriert sind. Die virtuelle Adresse eines Worts ist seine Positionsnummer in dieser Folge – also analog der Adresse einer realen Hauptspeicherzelle. In vielen der heutigen Prozessoren und Betriebssysteme ist eine solche Adresse 32 Bit lang (u.a. daher rührt der Begriff „32-Bit-Prozessor“ bzw. „32-Bit-Betriebssystem“), so dass 4 GByte große virtuelle Speicher adressiert werden können. Altsysteme wie MS-DOS und Intel-Prozessoren bis zum 80286 adressierten nur mit 16 Bit, während nunmehr 64-Bit-Systeme verstärkt im Kommen sind.

Die Folge der Wörter des virtuellen Speichers wird in *Seiten* gleicher, konstanter Länge unterteilt, weshalb man in diesem Zusammenhang auch von *Paging* spricht. Eine virtuelle Seite wird in einem *Seitenbereich* des Hauptspeichers (manchmal auch *Kachel*, eng. *tile* genannt) oder in einem Block des Plattenspeichers realisiert. Deren Größe entspricht der Seitenlänge oder ist ein ganzzahliges Vielfaches davon.

Die Seitenlänge ist systemabhängig: Ein typischer Wert sind 4 KByte, es sind aber auch andere, insbesondere größere Werte möglich. In jedem Fall aber die Länge eine Zweierpotenz, da das für die Adressberechnung vorteilhaft ist. Die Bitdarstellung einer virtuellen Adresse zerfällt dann nämlich in zwei Teile: Die erste Gruppe von Bits gibt

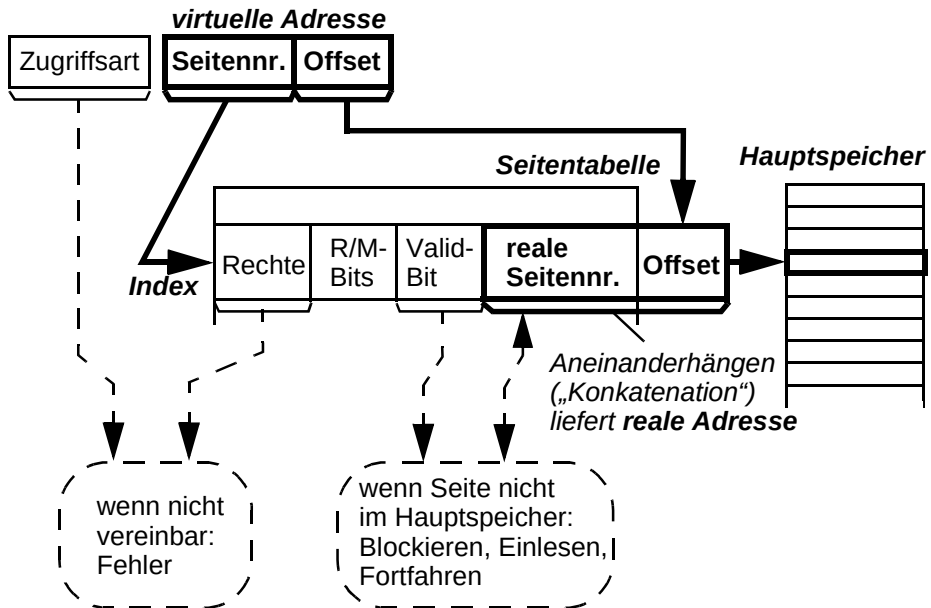


Abb. 5.18 Adressberechnung bei seitenorientierten Speichern

die Nummer der Seite an, die zweite Gruppe den Offset, also die Position innerhalb der Seite (siehe Abb. 5.17).

Adressumrechnung

Wie bei Segmenten stützt sich auch beim Paging die Adressberechnung auf eine Tabelle, die hier *Seitentabelle* (engl. *page table*) genannt wird (siehe Abb. 5.18). Jeder Prozess besitzt wiederum seine eigene Tabelle, die über die Seitennummer indiziert wird und für jede seiner Seiten einen eigenen Eintrag mit den folgenden Informationen enthält:

- Anfangsadresse im realen Speicher (Nummer des Seitenbereichs im Hauptspeicher oder Adresse des Plattenblocks).
- Valid Bit (gibt an, ob die Seite momentan im Hauptspeicher oder auf der Platte liegt).
- Zugriffsrechte (wie bei Segmenten).
- Bits für die Speicherverwaltung. Zu diesen Bits gehören das *Referenced Bit* (*R-Bit*), das bei einem Lese- oder Schreibzugriff auf die Seite von der Hardware auf 1 gesetzt wird, und das *Modified Bit* (*M-Bit*), das bei einem Schreibzugriff auf 1 gesetzt wird. Wir werden im nächsten Abschnitt darauf zurückkommen.

Die Adressberechnung geht hier ähnlich vor wie im segmentorientierten System: Mit der Seitennummer wird in der Tabelle der zugehörige Eintrag ermittelt. Liegt die Seite auf dem Plattenspeicher, was mit dem Valid Bit festgestellt wird, so tritt ein Trap-Ereignis ein, ein so genannter *Seitenfehler* (engl. *page fault*). Hierauf wird mit dem Einlesen der Seite (unter Änderung des Tabelleneintrags) reagiert. Gegebenenfalls muss zuvor eine andere Seite aus dem Hauptspeicher verdrängt werden, um Platz zu schaffen. Zudem wird geprüft, ob der Zugriff mit den gegebenen Rechten vereinbar ist. Ist das nicht der Fall, so wird der Zugriff und zumeist auch der ganze Prozess abgebrochen. Ansonsten ergibt sich die reale Hauptspeicheradresse, indem an die Bits der realen Seitennummer, die in der Tabelle stehen, die Bits des Offsets angehängt werden (= „Konkatenation“). Mit dieser Adresse wird dann auf den Hauptspeicher zugegriffen.

Auch hier wird im Allgemeinen dieselbe virtuelle Adresse auf unterschiedliche reale Adressen abgebildet, wenn sie von verschiedenen Prozessen benutzt wird. Zur Speicherung der Seitentabelle gibt es mehrere Möglichkeiten:

- Im einfachsten Fall liegt die Seitentabelle vollständig im Hauptspeicher, wobei ihre Anfangsadresse in einem Prozessorregister, dem *Page Table Base Register*, abgelegt ist. Bei einem Speicherzugriff kann die Hardware aus dem Registerwert, der Seitennummer und der Länge der Tabelleneinträge die Position des entsprechenden Tabelleneintrags ermitteln. Aus Aufwandsgründen ist dieser einfache Ansatz aber kaum praktikabel – siehe übernächster Absatz.
- Um die Adressberechnung zu beschleunigen, können häufig benötigte Tabelleneinträge in speziellen Registern (einem so genannten *Translation Lookaside Buffer [TLAB oder TLB]*) gespeichert werden, die wesentlich schneller zugreifbar sind als der Hauptspeicher. Typische „Trefferquoten“ liegen bei modernen Prozessoren mit bis zu mehreren hundert solcher Register zwischen 80 und 98 Prozent. Das bedeutet, dass bei den weitaus meisten Speicherzugriffen die Adressberechnung ohne die „langsamere“ Seitentabelle auskommt.
- Der Speicherbedarf einer Seitentabelle ist meist so groß, dass sie nicht als Ganzes in den Hauptspeicher passt. Ein virtueller Speicher mit 4 GByte umfasst bei einer Seitengröße von 4 KByte insgesamt 2^{20} Seiten. Die Seitentabelle hätte also, wenn man sie nach dem in Abbildung 5.18 dargestellten einfachen Schema realisieren würde, insgesamt über eine Million Einträge – für einen einzigen Prozess! Solche Größenordnungen überfordern den Hauptspeicher, und es bleibt nur die Möglichkeit, die große Seitentabelle in eine Vielzahl kleinerer Tabellen aufzuspalten, die teilweise auf den Plattenspeicher verdrängt werden können. Die Adressberechnung durchläuft stufenweise mehrere dieser Tabellen, bis schließlich die reale Adresse gefunden wird (siehe Abb. 5.19). Heutige Prozessoren und auch Linux unterstützen eine Adressumrechnung mit bis zu vier Stufen. Die Mehrstufigkeit bringt zwar im Prinzip einen erheblichen Zeitaufwand mit sich, der aber in der Praxis durch die Benutzung eines Translation Lookaside Buffers aufgefangen werden kann.
- Eine aufwendige, aber sehr effiziente Lösung für das Zeitproblem bei mehrstufigen Seitentabellen ist der zusätzliche Einsatz einer *invertierten Seitentabelle*. Diese Ta-

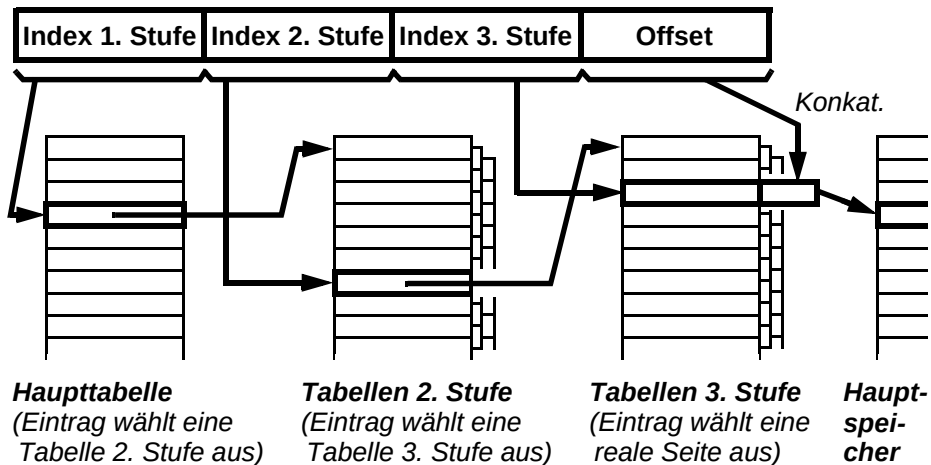
virtuelle Adresse

Abb. 5.19 mehrstufige Adressübersetzung bei seitenorientiertem Speichern

belle wird am günstigsten in Hardware durch einen Assoziativspeicher realisiert und enthält für jeden Seitenbereich *des Hauptspeichers* einen Eintrag, ist also wesentlich kleiner als die herkömmliche Seitentabelle. Der *i*-te Eintrag enthält dabei die (virtuelle) Adresse der virtuellen Seite, die momentan im *i*-ten Seitenbereich des Hauptspeichers steht. Dazu kommen noch die Rechte- und Verwaltungsbits dieser Seite sowie ein Valid Bit. Bei einem Speicherzugriff wird durch eine schnelle Suche im Assoziativspeicher festgestellt, ob und wo sich die gewünschte virtuelle Seite im Hauptspeicher befindet. Auf die herkömmliche Seitentabelle wird nur dann zugegriffen, wenn die Seite nicht im Hauptspeicher steht. Das ist zwar aufwendiger, dann aber auch unkritisch, da ja sowieso ein Plattenzugriff zum Einlesen der Seite erforderlich ist.

Vor- und Nachteile des seitenorientierten Ansatzes

Der Vorteil des seitenorientierten Ansatzes liegt, wie bereits gesagt, in der einfachen Speicherverwaltung. Alle Seiten haben eine einheitliche Länge, so dass ein Verschnittproblem wie bei Segmenten nicht auftritt. Insbesondere kann jede beliebige Seite aus dem Hauptspeicher verdrängt werden, um Platz für eine beliebige andere einzulagernde Seite zu schaffen. Der Zugriff auf den Plattenspeicher wird dadurch erleichtert, dass seine Blocklänge der Seitengröße entspricht oder ein Vielfaches davon ist. Die relativ kleine Seitengröße ermöglicht es zudem, gezielt nur die benötigten Daten in den Hauptspeicher zu laden. Da jeder Prozess seine eigene Seitentabelle besitzt, sind auch hier die Prozesse voneinander abgeschottet; es ist aber auch Sharing von Seiten durch Kopieren ihrer Tabelleneinträge in mehrere Tabellen möglich.

Ein Nachteil des seitenorientierten Ansatzes ist, dass die Struktur des virtuellen Speichers nicht den logischen Aufbau des Prozesses widerspiegelt. Unter anderem erschwert das, im Vergleich zu Segmenten, die Identifikation, den Schutz und das Sharing individueller logischer Einheiten. Zweitens kann Speicherplatz nur in ganzen Seiten zugeteilt werden, so dass ein Prozess eventuell mehr Bytes belegen muss, als er eigentlich benötigt. Da aber Seiten relativ klein sind, ist dieses Verschnittproblem nicht sehr kritisch. Das dritte und für die Implementierung größte Problem ist der hohe Speicherbedarf für die Seitentabellen und die möglicherweise zeitaufwendige Adressberechnung. Entsprechende Lösungen wurden bereits angesprochen.

5.3.4 Pagingstrategien

Das Betriebssystem verschiebt beim Paging ständig Seiten in der Speicherhierarchie: Es lädt Seiten in den Hauptspeicher, um sie zugreifbar zu machen (insbesondere zur Reaktion auf Seitenfehler), und verdrängt Seiten auf den Plattenspeicher, um im Hauptspeicher Platz zu schaffen. Dabei muss es nach drei Strategien vorgehen:

- *Ladestrategie*: Sie legt fest, zu welchem Zeitpunkt welche Seiten in den Hauptspeicher geladen werden.
- *Verdrängungsstrategie*: Sie bestimmt, welche Seiten verdrängt werden, wenn Hauptspeicherplatz benötigt wird.
- *Speicherzuteilungsstrategie*: Sie entscheidet darüber, wie viele Seitenbereiche des Hauptspeichers den einzelnen Prozessen zugeteilt werden.

Ladestrategien

Bei den Ladestrategien unterscheidet man zwischen *Demand Paging* und *Prepaging*. Demand Paging, was in etwa „Laden auf Anforderung“ bedeutet, lädt eine Seite nur dann, wenn sie unmittelbar benötigt wird, wenn also bezüglich dieser Seite ein Seitenfehler aufgetreten ist.

Prepaging lädt, wie der Name schon sagt, Seiten auch vorzeitig, d.h. auch dann, wenn auf sie nicht unmittelbar zugegriffen werden soll. Das vorzeitige Laden einer Seite ist natürlich spekulativ, denn im Allgemeinen ist nicht bekannt, ob sie später tatsächlich gebraucht wird. Es kann sich aber dann lohnen, wenn das Laden mehrerer Seiten in einem gemeinsamen Ladevorgang weniger aufwendig ist als mehrere Ladevorgänge für einzelne Seiten. Das ist beispielsweise dann der Fall, wenn die geladenen Seiten auf dem Plattenspeicher unmittelbar hintereinander liegen. Zudem kann beim Start eines Prozesses seine gesamte „Arbeitsumgebung“ („Working Set“, siehe unten) durch Prepaging eingelesen werden. Geschieht das nicht, so kommt es in der Anlaufphase des Prozesses zu zahlreichen Unterbrechungen durch Seitenfehler.

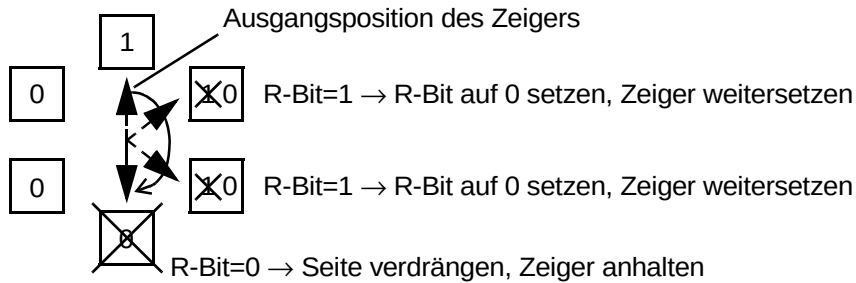


Abb. 5.20 Clock-Algorithmus zur Approximation des LRU-Verfahrens

Verdrängungsstrategien

Soll eine Seite in den Hauptspeicher geladen werden und ist dort kein Seitenbereich mehr frei, so muss anhand einer Verdrängungsstrategie eine Seite ausgewählt werden, die dann auf den Plattenspeicher ausgelagert wird und somit Platz freigibt. Hierbei sind unter anderem die folgenden Strategien denkbar:

- *First In First Out (FIFO)* verdrängt die Seite, die am längsten im Hauptspeicher steht. Das ist zwar einfach zu implementieren, bringt aber das Problem mit sich, dass dabei auch Seiten ausgelagert werden können, die immer noch stark in Benutzung sind. Tanenbaum vergleicht das mit einem Supermarkt, der sein Sortiment erweitern will und dafür aus Platzmangel das Mehl aus den Regalen entfernt, weil das schon seit über einhundert Jahren verkauft wird [Tan92]. Das charakterisiert die Qualität von FIFO recht treffend.
- *Least Recently Used (LRU)* verdrängt die Seite, für die der letzte Zugriff am längsten zurückliegt. Das hat den Vorteil, dass die Seiten, die in der aktuellen Ausführungsphase gebraucht werden, im Hauptspeicher verbleiben. Man kann nämlich recht gut von der Vergangenheit auf die Zukunft schließen: Prozesse verhalten sich meist „lokal“, d.h. arbeiten über einen längeren Zeitraum hinweg auf derselben, relativ kleinen Menge von Seiten. Somit wird eine Seite, auf die lange nicht zugegriffen wurde, mit hoher Wahrscheinlichkeit auch in der näheren Zukunft nicht gebraucht.
Der Nachteil ist, dass LRU in seiner reinen Form praktisch nicht zur Verwaltung eines virtuellen Speichers genutzt werden kann: LRU verlangt nämlich, dass Buch über die Zugriffszeitpunkte der einzelnen Seiten geführt wird. Bei jedem Speicherzugriff die Uhr abzufragen und die Uhrzeit z.B. in der Seitentabelle zu speichern, ist aber viel zu aufwendig.
- Der *Clock-Algorithmus* ist eine Annäherung an das „reine“ LRU-Verfahren, die leicht zu implementieren ist. Er stützt sich auf die Referenced Bits (R-Bits) in der Seitentabelle, die jeweils bei Zugriffen auf die zugehörigen Seiten auf 1 gesetzt werden. Das Verfahren legt für die Seiten eine zyklische Reihenfolge fest (siehe Abb. 5.20) und benutzt einen „Zeiger“, der jeweils auf eine der Seiten verweist – wie beim Zif-

ferblatt einer Uhr. Tritt ein Seitenfehler ein, so durchwandert der Zeiger die Seiten im Uhrzeigersinn. Solange er dabei auf Seiten mit gesetzten R-Bits trifft, löscht er diese Bits, d.h. setzt sie auf 0 zurück, und geht weiter. Die erste gefundene Seite, deren R-Bit nicht gesetzt ist, wird verdrängt. Der Zeiger bleibt an dieser Stelle stehen und setzt beim nächsten Seitenfehler von dort aus seinen Weg fort. Offensichtlich wurde die verdrängte Seite während des gesamten letzten Zyklus des Zeigers, also für relativ lange Zeit nicht gebraucht.

- Neben den drei genannten Verfahren gibt es noch eine Reihe weiterer Strategien. So verdrängt beispielsweise *Least Frequently Used (LFU)* die Seite, auf die am seltensten zugegriffen wurde.

In realen Systemen wird eine zur Verdrängung ausgewählte Seite nicht sofort auf die Platte übertragen, sondern zunächst im Hauptspeicher einem „Pool“ freier oder freier werdender Seitenbereiche zugeteilt und erst später real ausgelagert. Die zu ladende Seite wird dann in einen anderen Bereich aus dem Pool gebracht, der bereits frei ist. Das entzerzt Lade- und Verdrängungsvorgänge und ermöglicht außerdem, bei Bedarf Seiten rasch wieder verfügbar zu machen, sofern sie sich noch im Pool befinden.

Zudem wird der Inhalt einer Seite nur dann auf die Platte geschrieben, wenn ihr Modified Bit gesetzt ist. Ist das nicht der Fall, so kann sie im Hauptspeicher unmittelbar überschrieben werden, da sie seit ihrem letzten Einlesen nicht geändert wurde und somit ihre Kopie auf dem Plattenspeicher noch aktuell ist. Das setzt natürlich voraus, dass für alle Seiten im Hauptspeicher Kopien auf der Platte existieren.

Speicherzuteilungsstrategien

Die Speicherzuteilungsstrategie entscheidet darüber, wie viele Seitenbereiche des Hauptspeichers den einzelnen Prozessen zugeteilt werden, also wie viele seiner Seiten ein Prozess gleichzeitig im Hauptspeicher haben kann. Einerseits sollte ein Prozess nicht mehr Hauptspeicherplatz besitzen, als er wirklich braucht, da der Speicher ein knappes Betriebsmittel ist. Andererseits sollten aber mindestens alle die Seiten im Hauptspeicher stehen, auf denen der Prozess gerade arbeitet. Ansonsten kommt es nämlich zu vielen dicht aufeinander folgenden Seitenfehlern mit Ein- und Auslagerungen, so dass der Prozess immer wieder blockiert wird und auf Seiten warten muss. Man bezeichnet das mit dem Begriff „*Thrashing*“ (auf Deutsch etwa „Vorwärtsquälen“).

Nicht sehr sinnvoll ist, einem Prozess einen Hauptspeicherbereich fester, unveränderlicher Größe zuzuteilen, denn der Bedarf des Prozesses ändert sich im Allgemeinen während seiner Ausführung. Die Zuteilungsstrategie sollte daher versuchen, die Speichergröße dynamisch dem aktuellen Bedarf anzupassen.

Ein klassisches dynamisches Zuteilungsverfahren ist die *Working-Set-Strategie* nach P. J. Denning [Denn68]. Sie versucht, für einen Prozess genau die Seiten im Hauptspeicher zu halten, auf denen er gerade arbeitet. Der *Working Set* $WS(t, \delta)$ eines Prozesses zum Zeitpunkt t mit der *Fenstergröße* δ ist definiert als die Menge aller Seiten, die er bei den letzten δ Speicherzugriffen angesprochen hat. Abbildung 5.21 illustriert diese Definition anhand der Seitenzugriffsfolge eines Prozesses.

Zugriffsfolge eines Prozesses (Nummern virtueller Seiten):

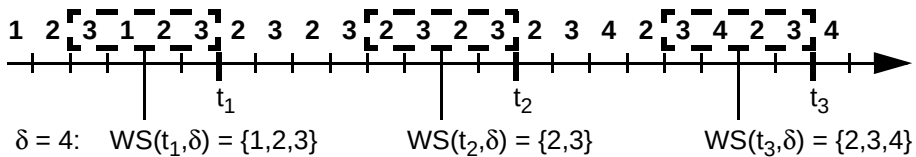


Abb. 5.21 Working Sets

Die Working-Set-Strategie hält zu jedem Zeitpunkt t genau die Seiten aus $WS(t, \delta)$ im Hauptspeicher. Sie schließt also, wie LRU, vom vergangenen Verhalten des Prozesses auf das zukünftige. Kritisch ist dabei die Wahl von δ : Ist es zu klein, so erfasst $WS(t, \delta)$ nicht alle Seiten, die der Prozess gerade benutzt; ist es wesentlich zu groß, so liegen in $WS(t, \delta)$ auch Seiten, die momentan nicht mehr gebraucht werden.

Wie LRU kann auch das reine Working-Set-Verfahren nur sehr aufwendig implementiert werden. Eine Annäherung ist unter Benutzung der Referenced Bits möglich. Dabei werden in festen Zeitabständen die Seiten mit nicht gesetzten Bits verdrängt und die Bits aller verbleibenden Seiten auf 0 zurückgesetzt. Damit verbleiben genau die Seiten im Hauptspeicher, auf die während des letzten Zeitabschnitts zugegriffen wurde.

Eine Alternative zum Working-Set-Verfahren ist die *Page-Fault-Frequency-Strategie*, die ebenfalls den Hauptspeicherbereich eines Prozesses dynamisch seinem Bedarf anpasst. Sie vergrößert den Bereich, wenn die Seitenfehlerrate des Prozesses einen oberen Schwellwert übersteigt, und verkleinert ihn, wenn die Fehlerrate einen unteren Schwellwert unterschreitet.

5.3.5 Speicherorganisation im praktischen Einsatz

Paging vs. Segmentierung

Blickt man noch einmal auf die beiden Organisationsformen des virtuellen Speichers zurück, so stellt man Folgendes fest: Das Paging realisiert einen linearen Adressraum, der zwar wesentlich größer als der des Hauptspeichers ist, aber dieselbe Struktur aufweist. Das hauptsächliche „Verdienst“ des Paging-Prinzips ist also, jedem Prozess gewissermaßen einen eigenen großen virtuellen Hauptspeicher zur Verfügung zu stellen. Die Segmentierung dagegen unterstützt die Aufteilung von Programmen und Daten in logische Einheiten mit unabhängigen Adressbereichen, die individuell geschützt oder gemeinsam benutzt werden können.

Beide Ansätze haben ihre Stärken: Paging ist für die Verwaltung vorteilhaft, da alle Verwaltungseinheiten dieselben Längen haben und daher beliebig austauschbar sind. Segmentierung reflektiert, wie gesagt, besser die logische Struktur von Daten und Programmen.

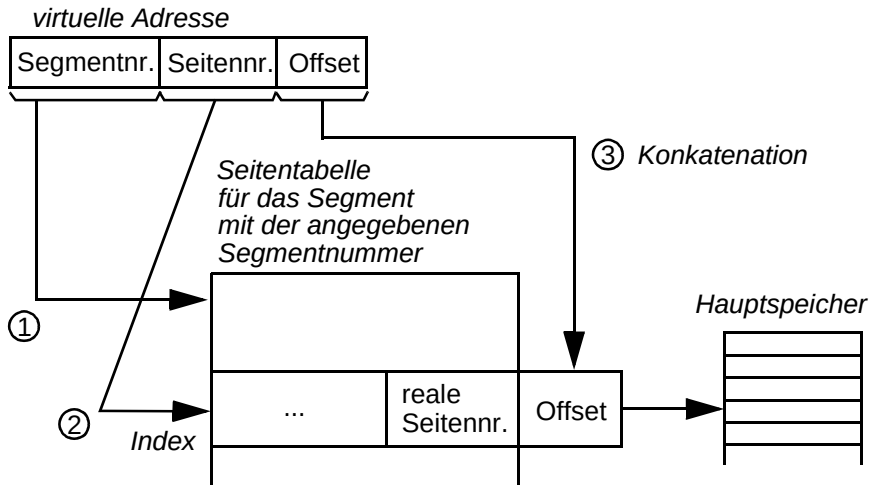


Abb. 5.22 hybride Speicherorganisation

Hybride Formen

In modernen Prozessorarchitekturen werden der segment- und der seitenorientierte Ansatz zu hybriden Formen (= Mischformen) kombiniert, um von den Vorteilen beider zu profitieren. Hier besteht der virtuelle Speicher eines Prozesses aus Segmenten, die jeweils in Seiten unterteilt sind. Virtuelle Adressen bestehen aus (mindestens) drei Teilen, nämlich der Segmentnummer, der Nummer einer Seite innerhalb des Segments und einem Offset in die Seite (siehe Abb. 5.22).

Speicherzugriffe laufen in drei Stufen ab: Im ersten Schritt wird mit der Segmentnummer die Seitentabelle für das Segment ermittelt, denn jedes Segment hat hier seine eigene Seitentabelle. Die Seitennummer wird im zweiten Schritt als Index in diese Tabelle benutzt und liefert so die Nummer des zugehörigen Seitenbereichs im Hauptspeicher. Der dritte Schritt hängt an diese Adresse den Offset an und liefert so die reale Adresse.

Diese Mischform aus Segmentierung und Paging kombiniert offensichtlich die Vorteile beider Ansätze, was allerdings durch einen höheren Aufwand bei der Adressberechnung erkauft wird. Außerdem ist die Zahl der Seitentabellen recht groß, da für jedes Segment eine gesonderte Tabelle geführt wird.

Ein Anwendungsbeispiel ist die Intel-Prozessorreihe ab dem 80386-Prozessor. Der 80386, als erstes Modell dieser Reihe, unterstützte bereits bis zu 2^{14} Segmente, die jeweils bis zu 4 GByte groß sein können und jeweils in Seiten zu 4 KByte unterteilt sind. Ein Speicherzugriff läuft über eine Segment- und eine zweistufige Seitentabelle. Spätere Modelle ermöglichen noch größere Segmente. Bei ihnen ist zudem das Paging abschaltbar, so dass wahlweise reine Segmentierung, reines Paging und die Mischform möglich sind.

Swapping vs. virtueller Speicher

Ein virtueller Speicher bietet, wie bereits diskutiert, eine größere Flexibilität als Swapping, verlangt dafür aber eine ausgefeiltere Hardwareunterstützung. Diese Beobachtung steht im Einklang damit, dass Swapping der ältere der beiden Ansätze ist: Ältere Systeme, wie z.B. UNIX-Versionen vor BSD 4.0 oder Windows 3.1, benutzten ausschließlich diese Technik zur Speicherverwaltung. Neuere Systeme, insbesondere moderne UNIX-Implementationen und Windows NT, stützen sich dagegen auf eine fortgeschrittenere Hardware und können somit einen virtuellen Speicher realisieren.

Modifizierte Swapping-Konzepte finden sich aber immer noch: So werden beispielsweise in vielen UNIX-Implementationen Prozesse vollständig ausgelagert, wenn zu viele Prozesse im Hauptspeicher liegen und damit einzelnen Prozessen zu wenig Hauptspeicherplatz bleibt.

Linux verzichtet allerdings ganz auf das Swapping und stützt sich ausschließlich auf einen seitenorientierten virtuellen Speicher. Am Rande sei bemerkt, dass zwar in manchen Publikationen über Linux, wie z.B. [Rus99], von „Swapping“ die Rede ist. Dies ist aber eine sprachliche Unschärfe; gemeint ist das hier besprochene Paging.

5.4 Übungsaufgaben

1. Wissensfragen

- a.) Welche Unterschiede bestehen zwischen Haupt- und Plattenspeicher?
- b.) Warum ist ein Cache, trotz seiner geringen Größe, vorteilhaft?
- c.) Was ist der Unterschied zwischen Swapping und Paging?
- d.) Wozu dient die MMU?
- e.) Was ist der Unterschied zwischen einem segmentorientierten und einem seitenorientierten virtuellen Speicher?
- f.) Wozu dient eine Verdrängungsstrategie?
- g.) Wie können die Vorteile segmentorientierter und seitenorientierter Speicher kombiniert werden?

2. Plattenscheduling

An einem Plattenspeicher liegen momentan Anforderungen für die folgenden Spuren vor (in der Reihenfolge ihres Eintreffens): 55, 90, 40, 85, 30, 20, 100.

Der Plattenarm mit dem Schreib-Lese-Kopf befindet sich gerade auf der Spur 50, und seine aktuelle Bewegungsrichtung (für den Fahrstuhl-Algorithmus) ist in Richtung der Spuren mit niedrigeren Nummern.

Geben Sie für die drei Strategien *First Come First Served*, *Shortest Seek Time First* und *Fahrstuhl* jeweils die Reihenfolge an, in der die Anforderungen bearbeitet werden. Berechnen Sie außerdem die Gesamtzahl der dabei überquerten und besuchten Spuren.

3. Hauptspeicherverwaltung

Betrachten Sie die Freispeicherverwaltung eines Hauptspeichers mit variablen Partitionen, die eine Folge von Anforderungen nach Hauptspeicherplatz bedienen muss. Eine Anforderung trifft zu einem bestimmten Zeitpunkt ein und gibt die Größe des benötigten Speichers an. Sofern ein geeigneter Speicherbereich frei ist, erhält sie diesen unmittelbar zugeteilt und belegt ihn für eine bestimmte Verweildauer. Anschließend gibt sie ihn wieder frei.

Die folgende Tabelle definiert eine Folge von Anforderungen. Angegeben sind jeweils die Nummer der Anforderung, der Zeitpunkt ihres Eintreffens (in Sekunden nach dem Systemstart), die Zeitdauer, für die der Speicher belegt werden soll („Verweildauer“, in Sekunden), sowie die Größe des geforderten Speichers (in MByte):

Anforderungsnr.	Zeitpunkt des Eintreffens	Verweildauer	Größe
I	0	4	8
II	1	20	1
III	2	2	3
IV	3	20	1
V	6	20	2
VI	7	20	2
VII	9	20	4
VIII	11	20	3

Der Hauptspeicher hat die Größe 16 MByte und ist zu Beginn leer.

- Beschreiben Sie, jeweils gesondert für die Zuteilungsstrategien *First Fit* und *Best Fit*, das Aussehen des Hauptspeichers zu den Zeitpunkten 5, 8 und 12.
- Berechnen Sie für jede der beiden Strategien und jeden der oben angegebenen Zeitpunkte gesondert die mittlere Länge der Freibereiche im Hauptspeicher.

4. Segmentorientierter virtueller Speicher

Betrachten Sie einen segmentorientierten virtuellen Speicher. Der Hauptspeicher umfasst insgesamt 1000 Bytes (damit die Zahlen nicht zu groß werden) und ist momentan wie folgt belegt:

S ₁ (150)	S ₃ (180)	F (100)	S ₄ (200)	F (200)	S ₆ (120)	F (50)
----------------------	----------------------	---------	----------------------	---------	----------------------	--------

„S_n“ steht für Segment Nummer n, „F“ für einen Freibereich, und die Zahl in Klammern bezeichnet die Länge des Bereichs in Bytes.

- a.) Vervollständigen Sie auf der Grundlage der Zeichnung die folgende Segmenttabelle. Die Anfangsadresse ist, je nach aktuellem Aufenthaltsort des Segments, entweder eine Byteadresse im Hauptspeicher oder eine Plattenblocknummer. Die kleinste Hauptspeicheradresse ist 0.

Segmentnr.	Anfangsadresse	Länge	Zugriffsrechte
S ₁			Lesen / Schreiben
S ₂	Plattenblock X	40	Nur Schreiben
S ₃			Nur Lesen
S ₄			Lesen / Schreiben
S ₅	Plattenblock Y	110	Lesen / Schreiben
S ₆			Lesen / Schreiben

- b.) Geben Sie für die unten aufgelistete Folge von Lese- und Schreibzugriffen mit virtuellen Adressen jeweils die zugehörige reale Adresse (also berechnete Bytenummer im Hauptspeicher) an – auch, wenn der Zugriff unzulässig ist. Geben Sie zusätzlich an, ob der Zugriff überhaupt zulässig ist – wenn nein, warum nicht. Sollte das Segment auf dem Plattenspeicher stehen, so laden Sie das Segment gemäß der Best-Fit-Strategie in den Hauptspeicher und geben die dann berechnete reale Hauptspeicheradresse an.

Zugriffsfolge: Lies Segment 4, Byte 0; Schreibe Segment 6, Byte 90;
 Schreibe Segment 3, Byte 78; Schreibe Segment 2, Byte 35;
 Lies Segment 1, Byte 155; Lies Segment 5, Byte 80.

- c.) Was für Angaben muss ein Prozess dem Betriebssystem liefern, wenn er ein neues Segment erzeugen möchte? Welche Schritte muss dann das Betriebssystem im Einzelnen ausführen, um das Segment zu erzeugen und dem Prozess einen Zugriff darauf zu ermöglichen?

5. Seitenorientierter virtueller Speicher

Betrachten Sie einen seitenorientierten virtuellen Speicher mit einer Seitengröße von 4096 Bytes. Gegeben ist der folgende Ausschnitt der Seitentabelle eines Prozesses:

Nummer der virt. Seite	0	1	2	3	4	5	6
reale Anfangsadresse	12288	P3	32768	0	8192	P12	16384

„Pn“ bedeutet „Plattenblock n“, die anderen Adressen sind Bytenummern im Hauptspeicher.

a.) Gegeben sind die folgenden virtuellen Adressen:

2000, 21000, 14000, 9000, 5000, 18000.

Geben Sie für diese Adressen die zugehörigen realen Hauptspeicheradressen an (falls sich die Seite im Hauptspeicher befindet) bzw. die Nummer des Plattenblocks und den Offset (falls die Seite momentan auf dem Plattenspeicher steht). Der Offset ist die Bytenummer innerhalb des Plattenblocks.

b.) Es wird nun die virtuelle Seite Nr. 2 auf den Plattenblock Nr. 23 verdrängt. Sie gibt damit einen Seitenbereich des Hauptspeichers frei, in den nun die virtuelle Seite Nr. 5 geladen wird. Wo und wie ändert sich die Seitentabelle? Welche der virtuellen Adressen aus Teilaufgabe a.) werden nun auf neue reale Hauptspeicheradressen oder Plattenblöcke abgebildet? Geben Sie die neuen Adressen bzw. Blocknummern und Offsets an!

c.) Kann ein reines Swapping-System einen Prozess enthalten, dessen Daten wie durch die Tabelle angegeben im Speicher stehen? Warum oder warum nicht?

6. Strategien zur Speicherverwaltung

a.) Betrachten Sie einen seitenorientierten virtuellen Speicher mit insgesamt 5 virtuellen Seiten. Im Hauptspeicher stehen insgesamt 3 reale Seitenbereiche zur Verfügung. Ein Prozess greift auf seine Seiten in der folgenden Reihenfolge zu (virtuelle Seitennummern): 1, 2, 3, 2, 1, 4, 2, 1, 5, 2. Wir nehmen an, dass der Hauptspeicher zu Beginn leer ist.

Wie viele Seitenfehler treten unter der FIFO-Verdrängungsstrategie auf und wie viele unter LRU?

Geben Sie außerdem für die einzelnen Zugriffszeitpunkte an, welche Seiten sich gerade im Hauptspeicher befinden.

b.) Gegeben ist ein Prozess, der auf drei verschiedene virtuelle Seiten in der folgenden Reihenfolge zugreift: 1 2 3*1 2 3 2 3 3 3.

- Geben Sie die acht Working Sets an, die ab der durch * markierten Stelle jeweils unmittelbar vor den einzelnen Zugriffen bestehen („unmittelbar vor“ bedeutet, dass der Working Set als Letztes den vorherigen Zugriff, jedoch noch nicht den aktuellen Zugriff berücksichtigen soll). Die Fenstergröße sei $\delta = 3$.
- Was folgt daraus für die Anzahl der Seitenfehler, die unter der Working-Set-Strategie ab dem Zeitpunkt * auftreten?
- Berechnen Sie die mittlere Größe der acht Working Sets.
- Wie viele Seitenbereiche des Hauptspeichers müssten dem Prozess mindestens zugeteilt werden, um unter LRU dieselbe Fehlerrate zu erreichen? Vergleichen Sie diesen Wert mit der oben berechneten mittleren Working-Set-Größe.

6 Dateisystem und Ein-/Ausgabe

Die Realisierung eines Dateisystems und der Zugriff auf E/A-Geräte sind zwei wichtige Aufgabenbereiche des Betriebssystems (E/A = Ein-/Ausgabe, auch I/O = Input/Output). Diese beiden Aspekte haben mehr Gemeinsamkeiten, als es zunächst den Anschein hat: Der Plattenspeicher, auf dem das Dateisystem liegt, ist über eine E/A-Schnittstelle an die Zentraleinheit angeschlossen und wird somit auf dieselbe Weise angesteuert wie die Geräte zur Kommunikation mit der Außenwelt. Zudem sind auf Dateien prinzipiell dieselben Operationen wie auf E/A-Geräten möglich, nämlich das Öffnen und Schließen für Datenübertragungen sowie das Lesen und Schreiben von Daten. Ein Betriebssystem wie UNIX behandelt daher auf Benutzerebene und an der System-schnittstelle E/A-Geräte und Dateien gleich.

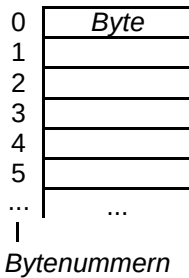
6.1 Dateisysteme

6.1.1 Grundlegende Begriffe

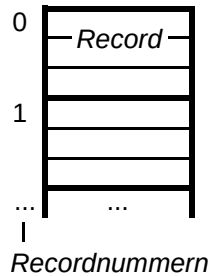
Würden Daten und Code nur im Hauptspeicher oder in einem virtuellen Speicher abgelegt, so gäbe es zwei Probleme: Erstens sind solche Speicherinhalte im Allgemeinen eng an einen Prozess gebunden, können also nur von diesem einen Prozess zugegriffen werden (Ausnahme, wie so oft: Shared Memory) und gehen mit der Terminierung des Prozesses verloren. Zweitens ist zwar der virtuelle Speicher eines Prozesses recht groß, aber für manche Anwendungen, wie z.B. große Datenbanken, möglicherweise noch zu klein.

Diese Probleme werden bekanntlich durch die Benutzung von *Dateien* (engl. *files*) gelöst. Formal betrachtet ist eine Datei eine Zusammenfassung von Informationen, die für längere Zeit auf einem externen Datenträger gespeichert werden und von dort beliebig oft zur Verarbeitung herangezogen werden können. Dateien werden also im Allgemeinen in Speicherbereichen eines Peripheriegeräts, z.B. Sektoren eines Plattenspeichers, oder eines auswechselbaren Speichermediums, z.B. Sektoren einer Diskette oder Blöcken eines Magnetbands, abgelegt. Es ist allerdings auch möglich, eine Datei temporär im Hauptspeicher zu halten, um Zugriffe zu beschleunigen. Die Lebensdauer einer Datei ist unabhängig von der Lebensdauer einzelner Prozesse. Eine Datei wird über *Dateioperationen* benutzt, wie z.B. Erzeugen und Löschen, Öffnen und Schließen sowie Lesen, Schreiben, Ausführen und Suchen.

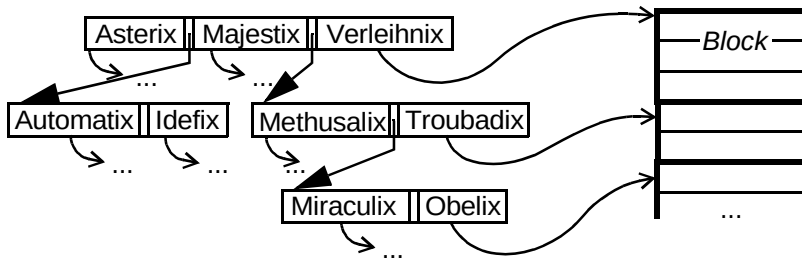
a.) byteorientiert:



b.) recordorientiert:



c.) baumorientiert (B-Baum):

**Abb. 6.1** interne Dateistruktur

Die vorhandenen Dateien werden in einem *Dateisystem* (engl. *file system*) organisiert, das allen Prozessen zur Verfügung steht (was allerdings nicht besagt, dass jeder Prozess auch auf jede Datei zugreifen darf). Aufgabe eines Dateisystems ist, über die vorhandenen Dateien Buch zu führen und dabei insbesondere den logischen Einheiten, aus denen eine Datei aufgebaut ist, reale Einheiten der Speicherhardware zuzuordnen. Zudem muss das reale Speichermedium verwaltet werden, wozu Operationen zur Belegung, Lokalisierung und Freigabe von Speicherbereichen gehören.

6.1.2 Logische Strukturen

Als Erstes soll der logische Aufbau der einzelnen Dateien und des gesamten Dateisystems besprochen werden.

Interne Dateistruktur

Dateien bestehen, wie bereits gesagt, aus Informationseinheiten, auf die individuell zugegriffen werden kann. Zur Organisation dieser Einheiten gibt es verschiedene Möglichkeiten (siehe Abb. 6.1):

- Beim *byte-orientierten Ansatz* ist die Datei eine einfache Folge von Bytes, auf die unter Angabe der Byteposition einzeln zugegriffen werden kann. Jede weitere Strukturierung und die Interpretation der einzelnen Bytes ist Sache des zugreifenden Programms. Schon MS-DOS realisierte byte-orientierte Dateien, und heutzutage benutzen unter anderem Windows NT und UNIX / Linux dieses Konzept.

Vorteilhaft ist hier die große Flexibilität, da durch die Datei keine bestimmte Struktur vorgeschrieben wird. Das Fehlen jeglicher Struktur ist aber auch ein Nachteil, da so sämtliche „Verantwortung“ beim Programm liegt und damit Fehlzugriffe und -interpretationen möglich sind.

- *Record-orientierte Dateien* enthalten eine Folge von Byte-Blöcken („Records“) einheitlicher Länge, die jeweils eine Gruppe zusammengehöriger Daten enthalten. Auf die Daten wird blockweise zugegriffen. Die ursprüngliche Motivation für diesen Ansatz war, dass früher Daten von Lochkarten eingelesen und auf Zeilendrucker ausgegeben wurden, die jeweils eine konstante Zeilenlänge (80 bzw. 132 Zeichen) hatten. Beispielsweise verwendete das historische Betriebssystem CP/M dieses Strukturierungsprinzip.

Heutzutage ist die Dateiorganisation durch Records veraltet, da das Schema zu starr ist. Im Prinzip könnten zwar Datenbestände wie z.B. Adressenlisten, deren Einträge identische Längen haben, in record-orientierten Dateien abgelegt werden. Für sie ist aber der nächste Ansatz geeigneter.

- Eine *baumorientierte Datei* besteht aus Blöcken unterschiedlicher Längen, denen jeweils ein *Schlüssel* zugeordnet ist. Ein Schlüssel ist ein Eintrag des Blocks, mit dem der Block unter allen Blöcken eindeutig identifiziert werden kann – wie beispielsweise ein Autokennzeichen das zugehörige Fahrzeug eindeutig bestimmt.

Verweise auf die Blöcke sind in einem B-Baum angeordnet, der anhand der Schlüsselwerte sortiert ist und somit einen raschen Zugriff auf bestimmte Blöcke erlaubt (siehe Abb. 6.1). Ein B-Baum ist ein Baum, dessen Knoten jeweils eine aufsteigend geordnete Liste enthalten. Zwischen den Elementen der Liste können Kanten zu Söhnen des Knotens stehen. Dabei darf die Liste eines Sohns nur solche Einträge enthalten, die gemäß der Ordnung größer sind als das Element der Vaterliste, das links von der Kante steht, und kleiner als das Element der Vaterliste, das rechts davon steht.

Die baumorientierte Organisation ist also insbesondere für Großrechner mit umfangreichen Datenbeständen / Datenbanken sinnvoll.

Neben der Organisationsform können Dateien auch anhand der Zugriffsmöglichkeiten klassifiziert werden:

- In einer *sequentiellen Datei* kann auf eine bestimmte Informationseinheit nur zugegriffen werden, indem die Datei von vorn nach hinten durchsucht wird. Das entspricht Dateien, die auf Magnetbändern abgelegt sind, da diese nur sequentiell abgespult werden können.
- Bei *direkt zugreifbaren Dateien* ist eine Informationseinheit unmittelbar durch Angabe ihrer Position zugänglich, d.h. ohne dass andere Einheiten durchlaufen werden

müssen. Beispielsweise sind Dateien auf Plattenspeichern oder Disketten direkt zugreifbar.

- *Indexsequentielle Dateien* sind eine Mischform aus den beiden geschilderten Ansätzen. Hier wird zunächst eine Indexliste nach einem bestimmten Indexbegriff (Schlüssel) sequentiell durchsucht. Der so gefundene Listeneintrag enthält eine Positionsangabe, mit der dann direkt auf den gewünschten Block zugegriffen werden kann.

Dateinamen und -attribute

Dateien in einem Dateisystem werden über *Namen* identifiziert. Zusätzlich sind ihnen *Attribute* zugeordnet, d.h. Zusatzinformationen, die bei Dateizugriffen benötigt werden.

Dateinamen haben sinnvollerweise die Form „Basisname.Erweiterung“. Die Erweiterung (engl. *extension*) sagt dabei etwas über die Art der gespeicherten Daten aus, z.B. TXT für eine Textdatei, EXE für eine ausführbare Datei, C für eine Datei mit C-Quellcode usw. In manchen Betriebssystemen gibt es Einschränkungen bezüglich der Länge von Namen und der benutzbaren Zeichen.

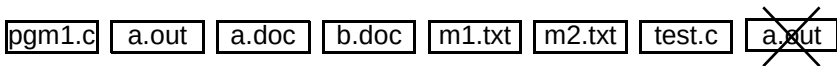
Zu den typischen Dateiattributen gehören der Erzeuger oder Besitzer der Datei, ihr Typ (z.B. einfache Datei mit ASCII- oder Binäreinträgen, Verzeichnisdatei, Geräte-datei), die Zeitpunkte der Erzeugung und des letzten Lese- oder Schreibzugriffs, die Dateigröße, die Zugriffsrechte (z.B. Lesen, Lesen und Schreiben, Ausführen) sowie aktuelle Zugriffsbeschränkungen („Locks“).

Struktur des Dateisystems

Da in einem Rechensystem meist sehr viele Dateien vorhanden sind, muss die Menge der Dateien geeignet in einem *Dateisystem* organisiert werden. Im allereinfachsten Fall ist diese Organisation „flach“ (siehe Abb. 6.2, Teil a). Das bedeutet, dass die Dateien nicht in einer Struktur angeordnet sind, sondern nur als unstrukturierte Ansammlung vorliegen und beispielsweise in einer langen Liste verzeichnet sind. Dieser primitive Ansatz hat offensichtlich eine Reihe von Nachteilen: Je mehr Dateien vorhanden sind, desto unübersichtlicher ist das „Dateisystem“. Zudem müssen alle Dateien unterschiedliche Namen haben, da es keine anderen Unterscheidungsmöglichkeiten gibt. Schließlich gibt es noch Schutzprobleme, da die Dateien der einzelnen Benutzer (und die des Betriebssystems) nicht getrennt voneinander gehalten werden können.

Aufgrund dieser Nachteile werden Dateien schon seit langem hierarchisch angeordnet. Hierbei können mehrere Dateien zu einer Gruppe zusammengefasst und in einem *Verzeichnis* (*Katalog*, *Directory*) abgelegt werden. Ein Verzeichnis ist im Prinzip nichts anderes als eine spezielle Datei, in der Informationen über die Dateien des Verzeichnisses gespeichert sind – insbesondere ihre Namen mit den Dateiattributen und Informationen zur Lokalisierung der Dateien auf der Platte. Mehrere Verzeichnisse wiederum können, eventuell mit weiteren einfachen Dateien, in einem übergeordneten Verzeichnis stehen, was beliebig oft „nach oben“ fortgesetzt werden kann. Es entsteht also ein *Dateibaum* (siehe Abb. 6.2, Teil b), in dem die inneren Knoten Verzeichnisse darstellen

a.) flaches Dateisystem:



b.) hierarchisches Dateisystem (Dateibaum):

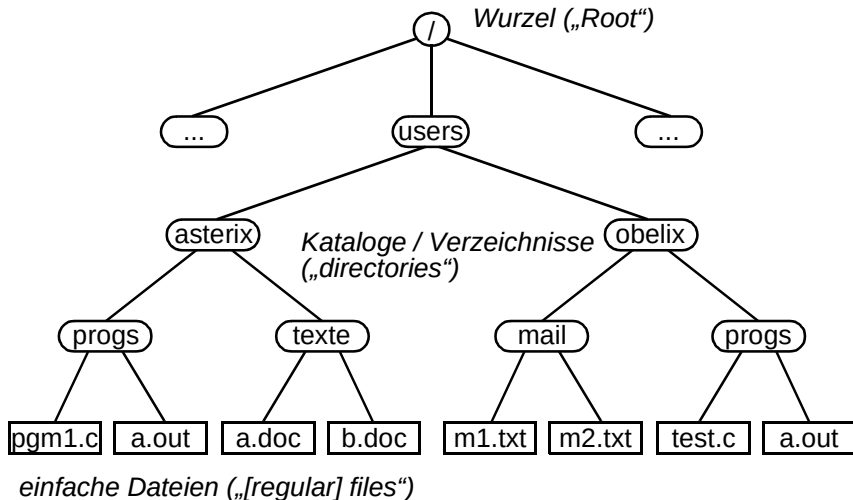


Abb. 6.2 flaches und hierarchisches Dateisystem

und die Blattknoten einfache Dateien (*[regular] files*) oder auch (leere) Kataloge. Je nach Organisation der Speichergeräte können in einem Rechnersystem auch mehrere Dateibäume vorhanden sein, z.B. pro Plattenspeicher und pro Diskettenlaufwerk ein eigener.

Dateien werden anhand ihrer Position im Dateibaum identifiziert. Das geschieht entweder *absolut* durch die Angabe des Pfads von der Wurzel des Baums zum Dateiknoten oder *relativ* durch einen Pfad, der vom Arbeitsverzeichnis des Prozesses ausgeht. Dazu kommt gegebenenfalls noch die Identifikation des Baums selbst, wie z.B. an der MS-DOS-Benutzerschnittstelle die Laufwerksangabe.

Die strenge Baumstruktur kann durch Verwendung von *Links* durchbrochen werden. Ein Link verknüpft einen Dateinamen, der in einem Verzeichnis steht, mit einer bereits bestehenden Datei in einem anderen Verzeichnis. Damit kann dieselbe Datei aus zwei verschiedenen Verzeichnissen unter zwei Namen angesprochen werden. Links ersparen dem Benutzer die mühselige Angabe oft langer Pfadnamen oder den Wechsel in ein anderes Verzeichnis. Unter UNIX / Linux werden Links mit Hilfe des `ln`-Befehls gesetzt.

Die Baumstruktur überwindet also effektiv die Nachteile der flachen Organisationsform: Die Menge der Dateien ist übersichtlicher gegliedert, und es besteht größere Freiheit bei der Namensgebung, da nur Dateien im selben Verzeichnis unterschiedliche Namen haben müssen. Zudem lassen sich Gruppen von Dateien leicht schützen, indem der Zugriff auf ihre jeweiligen Kataloge kontrolliert wird.

Operationen

Dateien mit ihren Inhalten und das Dateisystem können dynamisch geändert werden. Hierzu stellt ein Betriebssystem typischerweise die folgenden Operationen für einfache Dateien und Verzeichnisse zur Verfügung:

- Erzeugen („create“), Löschen („delete“), Kopieren („copy“)
- Öffnen („open“), Schließen („close“)
- Positionieren des Lese-/Schreibzeigers („seek“)
- Lesen („read“), Schreiben („write“), Anhängen („append“)
- Umbenennen („rename“)
- Attribute setzen / lesen („set / get attributes“)
- Setzen / Löschen von Links („link“ / „unlink“)

In vielen Betriebssystemen kann sich zudem ein Prozess einen exklusiven Zugriff auf eine Datei sichern, indem er die gesamte Datei oder bestimmte Bereiche für andere Prozesse „sperrt“ (*file locking* bzw. *record locking*). POSIX beispielsweise bietet hierfür die Funktion `fcntl()`.

Zur Vereinfachung und Beschleunigung des Dateizugriffs können Kopien von Dateien als *Memory-Mapped Files* im Hauptspeicher abgelegt werden. Auf den Dateieinträgen wird dann mit normalen Hauptspeicheroperationen gearbeitet, wobei Änderungen auf der Platte nachgetragen werden. Öffnen mehrere Prozesse eine solche Datei, so können sie darüber kommunizieren – ähnlich wie beim Shared Memory. Zur Realisierung bieten verschiedene UNIX-Versionen (z.B. System V, Solaris) die Funktion `mmap()`. Auch Windows NT unterstützt dieses Konzept.

6.1.3 Realisierung von Verzeichnissen

Ein System zur längerfristigen Speicherung von Daten kann aus zwei verschiedenen Blickwinkeln betrachtet werden. Aus der Sicht des Benutzers stehen die Daten in einer Menge von Dateien, die über Dateinamen und -pfade identifiziert werden können. Das entspricht also dem oben geschilderten Dateisystem. Aus einer hardwarenahen Sicht werden die Daten in Plattenblöcken gespeichert, auf die über Hardwareadressen zugegriffen werden kann.

Das Betriebssystem hat die Aufgabe, diese beiden Sichten zusammenzubringen. Es muss dazu erstens Informationen über die einzelnen Dateien führen, also insbesondere Dateinamen bestimmten Plattenblöcken zuordnen und Dateiattribute speichern. Dies geschieht über eine Sammlung von Verzeichnissen, die in diesem Abschnitt besprochen werden. Zweitens muss das Betriebssystem eine effiziente Speicherverwaltung für den

Suche nach dem Plattenblock für /users/obelix/progs/test.c:

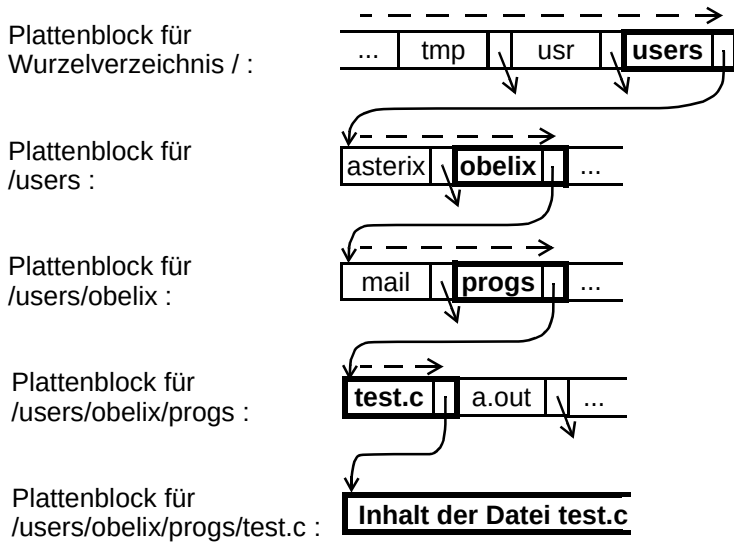


Abb. 6.3 interne Struktur eines Verzeichnisbaums

realen Plattenspeicher implementieren, also beispielsweise die Daten so auf der Platte platzieren, dass ein rascher Zugriff möglich ist. Dies ist unter dem Stichwort „Plattenorganisation“ Thema des nächsten Abschnitts.

Informationen über Dateien

Informationen über Dateien werden, wie bereits gesagt, in Verzeichnissen abgelegt. Ein Verzeichnis ist eine spezielle Datei, in der eine Menge von Paaren steht (siehe Abb. 6.3). Jedes Paar repräsentiert eine Datei in dem Verzeichnis: Die erste Komponente des Paares ist der Dateiname, die zweite Komponente enthält Informationen über die Datei – also die Dateiattribute und insbesondere (siehe Abbildung) den Standort der Datei im realen Speicher. Ebenso wie einfache Dateien werden Verzeichnisse mit ihren Einträgen in Blöcken des Plattenspeichers abgelegt.

Wie die Menge der Paare abgespeichert wird, hängt von der Implementation des Betriebssystems ab. Es können hier Arrays mit Einträgen fester Länge oder verkettete Listen und Baumstrukturen (z.B. B-Bäume) mit Einträgen möglicherweise unterschiedlicher Längen verwendet werden. Die Art der Realisierung hat unmittelbaren Einfluss auf die Flexibilität der Verzeichniseinträge und den Suchaufwand zum Auffinden eines bestimmten Eintrags. Linux legt beispielsweise Verzeichniseinträge in einer einfach verketteten Liste ab, die nur relativ langsam durchlaufen werden kann, bietet aber zusätzlich einen *Directory Cache*, in dem häufig gebrauchte Verzeichniseinträge rasch zugreifbar sind.

Verzeichniseintrag:

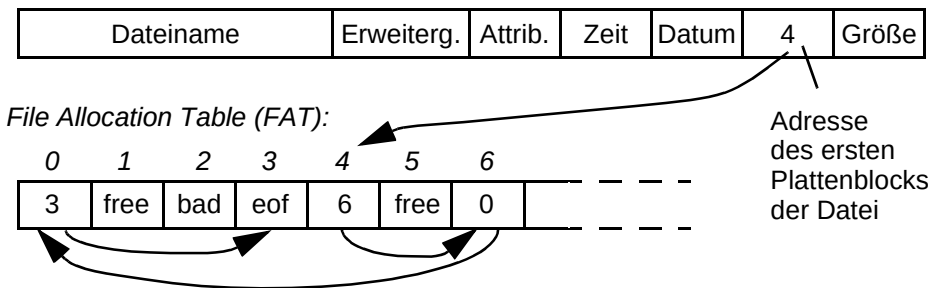


Abb. 6.4 FAT-basiertes Dateisystem: Verzeichniseintrag und File Allocation Table

Eine Datei wird lokalisiert, indem die Verzeichnisliste nach dem Dateinamen durchsucht wird. Wird statt eines einfachen Dateinamens ein ganzer Pfad angegeben, so muss hintereinander mit den einzelnen Pfadb Bestandteilen in mehreren Verzeichnissen gesucht werden, also vom Wurzelverzeichnis ausgehend der Dateibaum „top-down“ durchlaufen werden. Abbildung 6.3 stellt diesen Suchvorgang schematisch dar.

Zur Speicherung der Informationen über die Dateien gibt es zwei Möglichkeiten: Erstens können alle Informationen direkt in der Verzeichnisdatei abgelegt werden. Zweitens kann die Verzeichnisdatei lediglich Verweise auf andere Speicherbereiche enthalten, in denen dann die eigentlichen Informationen stehen.

FAT-Dateisystem

Das *FAT-Dateisystem* geht auf MS-DOS zurück und wird heute – neben anderen Dateisystemen – von den modernen Windows-Betriebssystemen und optional von Linux unterstützt. Wie Abbildung 6.4 zeigt, liegt hier die Dateiinformation direkt im Verzeichnis: Neben dem Basisnamen der Datei und der Erweiterung enthält der Verzeichniseintrag also Dateiattribute, die Eigenschaften der Datei festlegen, Uhrzeit und Datum der letzten Änderung der Datei, die Adresse des ersten Plattenblocks und die Länge der Datei.

Der Verzeichniseintrag enthält eine einzige Plattenadresse, nämlich die des ersten Blocks der Datei. Eine einzelne Adresse reicht zum Zugriff auf die gesamte Datei nur dann aus, wenn die Daten zusammenhängend in aufeinander folgenden Plattenblöcken abgespeichert sind. Im Allgemeinen steht eine Datei jedoch nicht in zusammenhängenden Blöcken, da das für die Speicherverwaltung die bekannten Verschnittprobleme mit sich bringt; sie kann vielmehr beliebig über die Platte „verstreut“ sein. Also muss für jede Datei eine ganze Liste geführt werden, in der die Plattenblöcke der Datei einzeln verzeichnet sind. Die Liste kann beliebig lang sein und passt daher nicht in einen Verzeichniseintrag konstanter Länge, wie er hier vorliegt.

Die Lösung dieses Problems sieht wie folgt aus: Die Adresse des ersten Plattenblocks verweist gleichzeitig auf einen Eintrag in einer globalen *File Allocation Table (FAT)*

(siehe Abb. 6.4). Die FAT wird anhand von Plattenblocknummern indiziert und enthält für jeden Block (oder bei den heute gebräuchlichen großen Platten für jedes „Cluster“ = Gruppe von Blöcken) einen Eintrag: Ist der Block momentan nicht belegt, so wird dies durch einen bestimmten Code (FREE) vermerkt. Dasselbe gilt, wenn der Block wegen einer Beschädigung der Magnetplatte unbrauchbar ist (BAD). Ist der Block momentan einer Datei zugeordnet, so enthält sein FAT-Eintrag die Nummer (und damit auch den FAT-Index) des nächsten Blocks der Datei oder den Code EOF (= „end of file“), falls es der letzte Dateiblock ist. Die FAT enthält also für jede Datei eine verkettete Liste, die die Nummern aller Plattenblöcke angibt, in denen die Datei gespeichert ist – und dies in der richtigen Reihenfolge. Wo diese Listen beginnen, wird, wie dargestellt, durch die Verzeichniseinträge der Dateien definiert.

Inode-basiertes Dateisystem in UNIX / Linux

Stehen die Informationen über die Dateien direkt im Verzeichnis, so hat das den Vorteil, dass sie unmittelbar (also nicht indirekt über einen Verweis) zugreifbar sind. Allerdings können damit Links nur schwer realisiert werden, da die Dateiinformation einem bestimmten Namen in einem bestimmten Verzeichnis fest zugeordnet sind. UNIX und Linux, wo solche Links eine wichtige Rolle spielen, gehen daher einen alternativen Weg, indem sie ein „Inode-basiertes Dateisystem“ benutzen.

In einem solchen Dateisystem steht im Verzeichniseintrag, neben dem Dateinamen, nur ein Zeiger auf eine gesonderte Datenstruktur. Diese befindet sich an anderer Stelle und enthält die eigentlichen Dateiinformationen (siehe Abb. 6.5). Für jede Datei gibt es eine eigene solche Struktur, die *Inode* (index node = „Indexknoten“) genannt wird.

Im klassischen UNIX System V umfasste der Verzeichniseintrag 14 Byte für den Dateinamen und zwei Byte für den Verweis auf den Inode. In heutigen UNIX-Systemen ist die Einschränkung bezüglich der Länge von Dateinamen aufgehoben: Beispielsweise erlauben BSD-UNIX und Linux Dateinamen mit bis zu 255 Zeichen. Auch ist der Bereich für Inode-Nummern größer, so in Linux vier Bytes.

Ein Inode in einem UNIX- oder Linux-System enthält typischerweise u.a. die folgenden Einträge: Typ der Datei (u.a. einfache Datei, Verzeichnis, E/A-Gerät), Zugriffsrechte (aufgeschlüsselt für Besitzer, Gruppe und andere), einen *Referenzzähler* (= Anzahl der bestehenden Links für diese Datei), *UID* und *GID* (user / group identifier = Identifikation des Besitzers und der Gruppe), Länge der Datei in Bytes, Zeitpunkte der Erstellung, der letzten Änderung und des letzten Zugriffs sowie, last but not least, Verweise auf die Plattenblöcke der Datei. Je nach System können weitere Informationen hinzukommen. Die meisten Inode-Einträge lassen sich durch die Schnittstellenfunktionen `stat()` und `fstat()` abfragen.

Die Verweise auf die Plattenblöcke sind komplexer organisiert als im FAT-Dateisystem: Der Inode enthält mehrere solche Verweise, von den die ersten *m* unmittelbar die Plattenadressen der ersten *m* Blöcke der Datei angeben, also „Direktverweise“ sind. Der genaue Wert von *m* ist implementationsabhängig: Im klassischen System V war *m*=10, im heutigen Linux beispielsweise ist *m*=12.

Der *m*+1. Verweis zeigt auf einen so genannten „*Indirektionsblock* erster Stufe“, in dem Verweise auf die nächsten *n* Blöcke der Datei stehen. Auch *n* hängt von der Im-

Verzeichniseintrag:

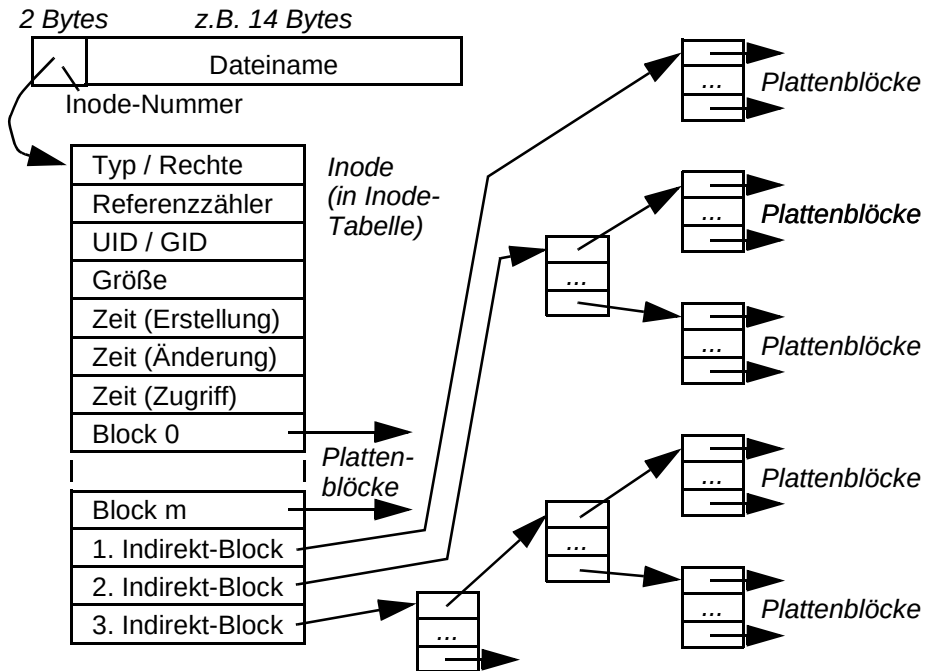


Abb. 6.5 Inode-basiertes Dateisystem: Verzeichniseintrag und Inode

plementation ab; ein Minimalwert ist 256. Auf diese Blöcke kann also nicht direkt aus dem Inode, sondern nur mit einem Zwischenschritt über den Indirektionsblock zugegriffen werden. Der $m+2$. Verweis gibt einen „Indirektionsblock zweiter Stufe“ an, der auf n weitere Indirektionsblöcke verweist, die wiederum jeweils die Adressen von n Dateiblöcken enthalten. Über die zweite Stufe können also maximal n^2 Dateiblöcke zugegriffen werden. Sollte dies immer noch nicht ausreichen, so kann mit dem $m+3$. Verweis ein „Indirektionsblock dritter Stufe“ angesprochen werden, mit dem dreifach indirekt auf die übrigen Dateiblöcke (höchstens n^3) zugegriffen wird.

Man beachte den Vorteil gegenüber dem FAT-Ansatz: In der FAT muss bei jedem Blockzugriff die Verkettung von vorn durchlaufen werden, was insbesondere bei großen Dateien zeitaufwendig ist. Bei der Verwendung von Inodes der hier geschilderten Form sind dagegen bei kleinen Dateien alle Blöcke direkt zugreifbar, und auch Zugriffe auf große Dateien können wegen der baumartigen Verweisstruktur rasch durchgeführt werden.

Die Inodes aller Dateien liegen gemeinsam in einer *Inode-Tabelle*, die im Plattenspeicher realisiert ist. Beim Öffnen einer Datei wird ihr Inode in den Hauptspeicher kopiert, um rascher zugreifbar zu sein. Einer der Inodes ist als *Wurzel-Inode* (engl. *root inode*) ausgezeichnet; er repräsentiert das Wurzelverzeichnis, bei dem die Suche über einen Dateipfad beginnt (siehe auch Abb. 6.3). Die Inode-Nummer im Verzeichniseintrag einer Datei (Abb. 6.5 oben) ist der Index in die Inode-Tabelle.

Beim Inode-basierten Ansatz sind die Vor- und Nachteile genau umgekehrt verteilt wie beim FAT-basierten Schema: Dateizugriffe verlaufen indirekt über den Inode, was den Aufwand etwas erhöht. Dafür stehen aber die Informationen über eine Datei an einer zentralen Stelle, womit beispielsweise Links leicht eingerichtet werden können: Hierzu muss in die betroffenen Verzeichniseinträge einfach dieselbe Inode-Nummer eingesetzt werden.

Links, die auf der Mehrfachverwendung von Inode-Nummern beruhen, werden übrigens als *Hard Links* bezeichnet. Daneben gibt es *Soft Links* / *symbolische Links*. Das sind Hilfsdateien, in denen die Pfadnamen der eigentlichen Dateien eingetragen sind.

Ext2fs und VFS in Linux

Das Standarddateisystem von Linux ist *ext2fs* – „ext“ steht für „extended“, da es sich um die Erweiterung des Dateisystems des experimentellen MINIX-Betriebssystems handelt, „2“ für die aktuelle zweite Version und „fs“ für „file system“.

Ext2fs ist im Prinzip so strukturiert wie das klassische UNIX-Dateisystem – sowohl, was sein Erscheinungsbild gegenüber dem Benutzer, als auch, was die interne Realisierung betrifft. Zudem bietet es einige zusätzliche Eigenschaften, wie z.B. Verzeichniseinträge unterschiedlicher Längen, lange Dateinamen mit bis zu 255 Zeichen und die Unterstützung unterschiedlicher Plattenblockgrößen.

In einem Linux-System kann aber nicht nur *ext2fs* verwendet werden, sondern es lassen sich auch anders strukturierte Dateisysteme integrieren – so insbesondere FAT-basierte Ansätze. Ein *Virtual File System (VFS)* verbirgt dabei „nach oben“ die unterschiedlichen Eigenschaften dieser Systeme und bietet so eine einheitliche Benutzer- und Programmierschnittstelle. Hierzu werden drei Typen von Objekten definiert, die ein Dateisystem und seinen Inhalt beschreiben:

- Ein *File-System-Objekt* (ein so genannter *Super Block*) charakterisiert das Dateisystem als Ganzes, indem es seine charakteristischen Eigenschaften definiert – wie z.B. seinen Typ, seine Blockgröße, seinen Wurzel-Inode, mögliche Zugriffsrechte usw. Die Informationen über die registrierten Dateisysteme werden in einer verketteten Liste gehalten, zu der neue Systeme dynamisch hinzugefügt werden können.
- Ein *Inode-Objekt* beschreibt eine einzelne Datei in einem Dateisystem. Neben allgemeinen Informationen über die Datei (ähnlich wie im vorangehenden Teilabschnitt beschrieben) enthält er Einträge, die für das jeweilige Dateisystem spezifisch sind und unterstützt so einen Dateizugriff über dieses System.

Die Inode-Objekte der existierenden Dateien stehen in einer doppelt verketteten zyklischen Liste und zusätzlich in einer Hashtabelle, um Zugriffe zu beschleunigen. Sie werden über Paare aus Dateisystemnummern und Inode-Nummern innerhalb des Dateisystems identifiziert.

- Ein *File-Objekt* enthält Informationen über eine geöffnete Datei, wie z.B. Zugriffsrechte und Position des Lese-/Schreibzeigers. Wir werden auf File-Objekte im folgenden Abschnitt noch kurz zurückkommen.

Für jedes registrierte Dateisystem gibt es Tabellen, durch die den allgemeinen Linux-Schnittstellenfunktionen entsprechende Funktionen, die für das Dateisystem spezifisch sind, zugeordnet werden – z.B. für das Erzeugen und Löschen von Dateien und für Les- und Schreibzugriffe. Der Aufruf einer Linux-Funktion wird über eine solche Tabelle automatisch in den Aufruf einer Funktion des Dateisystems umgesetzt. Ein Benutzer oder Programmierer kann so die ihm bekannten Linux-Dienste und -Funktionen verwenden, ohne sich um die unterschiedlichen Eigenschaften der realen Dateisysteme kümmern zu müssen.

NTFS in Windows NT / 2000

Zum Abschluss dieses Abschnitts soll noch kurz auf *NTFS* („NT File System“) eingegangen werden, das – neben anderen Dateisystemen – von Windows NT und Windows 2000 implementiert wird. NTFS basiert auf einer *Master File Table (MFT)* – einem Array, der über Dateinummern indiziert wird und für jede Datei mindestens einen *File Record* mit Informationen über diese Datei enthält. Die MFT entspricht also der Inode-Tabelle von UNIX; allerdings sind die Einträge mit 1 KByte pro Datei wesentlich größer, und zudem kann eine Datei mehrere MFT-Einträge haben.

Ein File Record speichert, ähnlich einem Eintrag der Inode-Tabelle, die Standardattribute einer Datei, wie Zugriffsrechte und Zeitstempel, sowie Informationen über die Nutzdaten. Bei kleinen Dateien stehen die Daten im File Record selbst, also in der MFT.

Bei größeren Dateien enthält der File Record die Adressinformationen, in welchen Plattenblöcken sich die Nutzdaten befinden. Diese Adressinformationen werden in einer kompakten Form abgespeichert, die Speicherplatz spart und eine rasche Adressberechnung erlaubt: Es wird nicht, wie bei FAT oder Inode, jeder Plattenblock einzeln aufgeführt, sondern es werden nur die Anfangsadressen und Längen zusammenhängender Bereiche des Plattenspeichers angegeben, die zusammen die Datei bilden. Wir betrachten zur Illustration ein Beispiel: Stehen die Daten einer Datei auf der Platte in den Clustern 1000 bis 1006 und 2000 bis 2004 (in dieser Reihenfolge), so enthält der File Record nur die zwei Einträge (0,1000,7) und (7,2000,5). Dies bedeutet: Die 7 logischen Dateicluster 0-6 stehen in den realen Plattenclustern 1000-1006, die 5 logischen Dateicluster 7-11 in den realen Clustern 2000-2004.

Die MFT enthält insbesondere File Records für Dateien, in denen Verwaltungsinformationen für das Dateisystem stehen. Dazu gehören das Wurzelverzeichnis, der Frei-/Belegt-Status der realen Plattencluster, ein Verzeichnis unbrauchbarer Cluster, eine Liste verwendbarer Dateiattribute sowie eine Log-Datei, in der über die stattfindenden Operationen Buch geführt wird.

Gegenüber dem ursprünglichen Microsoft-Dateisystem, dem FAT-basierten Ansatz von MS-DOS, hat NTFS eine Reihe von Vorteilen:

- Die Benutzung langer Dateinamen (bis zu 255 Zeichen) wird unterstützt. Dabei kann der gesamte Unicode-Zeichensatz benutzt werden. Unicode ist ein standardisierter 16-Bit-Code zur Darstellung von Zeichen. Er umfasst die Zeichensätze sämtlicher bedeutender Sprachen, u.a. auch Chinesisch und Japanisch.

- Dateiattribute und Schutzmöglichkeiten wurden wesentlich erweitert. U.a. können Zugriffsrechte differenziert nach einzelnen Benutzern und Operationen vergeben werden.
- Adressinformationen werden effizienter gespeichert, so dass auch auf große Dateien rasch zugegriffen werden kann.
- Große Verzeichnisse werden nicht als Listen, sondern als B-Bäume abgelegt. Sie sind daher effizienter zugreifbar.
- Eine „Recover-Funktionalität“, die sich auf ein Transaktionskonzept (siehe Abschnitt 4.1.4) und das oben erwähnte Log-File stützt, erlaubt die Behebung von Fehlern. Zudem werden System- und (optional) Nutzdaten redundant abgelegt, so dass verloren gegangene Daten wiederhergestellt werden können.
- Dateiinhalte können komprimiert werden, wodurch Plattenplatz eingespart wird.

Nachteilig bei NTFS ist, dass für die Verwaltungsinformationen sehr viel Platz benötigt wird – je nach Dateisystem 1-5 MByte. Für kleinere Dateisysteme (bis zu 400 MByte) wird daher die Benutzung des FAT-Ansatzes empfohlen, der von Windows NT und Windows 2000 ebenfalls unterstützt wird; für Disketten ist nichts anderes möglich.

6.1.4 Plattenorganisation

Dateieinträge stehen auf dem realen Plattenspeicher und werden, wie im vorigen Abschnitt dargestellt, über Verzeichnisse zugegriffen. Wie effizient ein solcher Zugriff verläuft, hängt von der Organisation des Plattenspeichers ab. Hierbei sind insbesondere die folgenden Fragestellungen interessant: Wie sieht die logische Gesamtstruktur des Speichers aus? Wie werden die Schnittstellenfunktionen für Dateizugriffe unterstützt? Wie geht die Freispeicherverwaltung des Plattenspeichers vor sich? Wie können Plattenzugriffe beschleunigt und der Datendurchsatz erhöht werden?

Logische Struktur

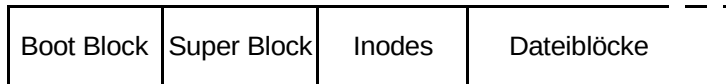
Ein Plattenspeicher kann in mehrere logische *Partitionen* unterteilt werden, von denen jede ein gesondertes Dateisystem enthält. Dies hat folgende Vorteile:

- Die einzelnen Dateisysteme werden nicht zu groß, was die Verwaltung vereinfacht.
- Die einzelnen Partitionen können unterschiedlich parametrisiert sein (z.B. verschiedene Blockgrößen).
- Fehler bleiben meist auf eine Partition beschränkt.

a.) FAT-System:



b.) UNIX:



c.) Linux (ext2fs):

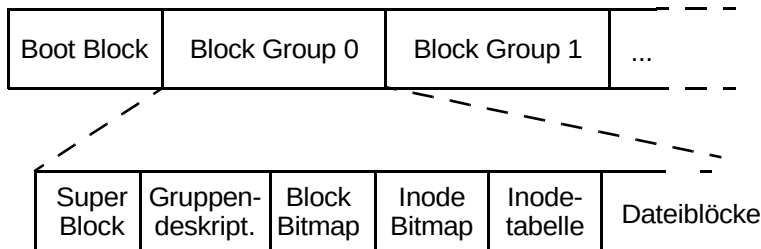


Abb. 6.6 Plattenorganisation

- In jeder Partition kann ein eigenes Betriebssystem installiert werden, so dass der Computer z.B. wahlweise unter Windows NT oder Linux betrieben werden kann.

Sind mehrere Plattenspeicher (oder andere Sekundärspeicher) vorhanden, so enthält jeder von ihnen mindestens ein eigenes Dateisystem. Diese Systeme können in einen globalen Dateibaum integriert werden, indem sie mit ihren Wurzeln an bestimmte Stellen des Dateibaums eingehängt (auf Neudeutsch: „gemountet“) werden. In UNIX / Linux gibt es hierfür den `mount`-Befehl.

Drei Beispiele sollen die logische Struktur von Plattenspeichern in heutigen Betriebssystemen illustrieren (siehe Abb. 6.6). Dargestellt wird jeweils eine Partition des Plattenspeichers; es können aber durchaus mehrere dieser Partitionen vorhanden sein:

- Im FAT-basierten Dateisystem steht ganz vorne der *Boot Sector*, der das Programm zum Starten des Systems (den so genannten *Boot Code*) enthält sowie Parameter des Dateisystems, wie z.B. die Blockgröße, die Clustergröße, die Größe des Wurzelverzeichnisses usw. Daran schließt sich die FAT an, für die optional ein Duplikat angelegt werden kann. Ein Duplikat ist sinnvoll, da bei einer Zerstörung der FAT das ganze Dateisystem verloren ist. Hinter der FAT stehen das Wurzelverzeichnis und die übrigen Dateiblöcke.
- Im klassischen UNIX System V folgt auf den Boot Block, der hier nur den Boot Code enthält, der *Super Block* mit Informationen über das Dateisystem, wie z.B. die Länge

der Inode-Tabelle, die Gesamtzahl der Blöcke, ihre Größe, die maximale Gesamtzahl der Dateien und ein Anfangsstück der Liste der freien Plattenblöcke. Im Anschluss daran werden die Inode-Tabelle und die eigentlichen Dateiblöcke gespeichert.

- In Linux mit ext2fs kann, als Weiterentwicklung des UNIX-Schemas, eine Partition in mehrere *Blockgruppen* unterteilt sein. Jede Blockgruppe enthält zuerst den Super Block, in dem auch hier Informationen über das gesamte Dateisystem stehen – z.B. die Blockgröße sowie die Anzahl der vorhandenen und der freien Blöcke und Inodes. Dem Super Block folgen Gruppendeskriptoren, deren Einträge die einzelnen Blockgruppen beschreiben. Ein Gruppendeskriptor gibt insbesondere die Positionen der Block Bitmap, der Inode Bitmap und der Inode-Tabelle sowie die Anzahl der freien Blöcke und Inodes an. Daran schließen sich Block Bitmap und Inode Bitmap mit den Verzeichnissen der freien Blöcke bzw. Inodes sowie die Inode-Tabelle und die eigentlichen Dateiblöcke an.

Die Unterteilung einer Partition in Blockgruppen ermöglicht schnellere Dateizugriffe. Linux bemüht sich nämlich, Datenblöcke in derselben Blockgruppe wie der zugehörige Inode unterzubringen und so die Plattenfragmentierung niedrig zu halten. Zudem ergibt sich eine höhere Sicherheit gegen Datenverluste, da jede Blockgruppe eine Kopie des Super Blocks und der Deskriptoren aller Blockgruppen enthält.

Dateizugriffe

Die Funktionen zur Dateibenutzung, die in Abschnitt 6.1.2 angeführt wurden, müssen mit Hilfe der realen Plattenoperationen implementiert werden. Wir illustrieren am Beispiel von UNIX System V, wie die Operationen „Öffnen“, „Dateizugriff“ und „Schließen“ im Einzelnen vorgehen.

Grundlage der Dateioperationen in UNIX ist ein System von Tabellen im Hauptspeicher (siehe Abb. 6.7). Jeder Prozess verfügt über eine eigene *User File Descriptor Table*. Sie gibt die Dateien an, die momentan für den Prozess „geöffnet“ sind, d.h. auf die der Prozess momentan lesend oder schreibend zugreifen kann. Die Tabelle wird indiziert über so genannte *Dateideskriptoren* (siehe unten) und enthält Verweise auf Einträge in der File Table.

Die *File Table* ist eine globale Tabelle, die allen Prozessen zur Verfügung steht. Ihre Einträge geben Zugriffsrechte für geöffnete Dateien an und definieren Positionen von Lese-/Schreibzeigern. Referenzzähler geben die Anzahl der Verweise an, die in User File Descriptor Tables auf die jeweiligen Einträge existieren. Pointer in den Einträgen verweisen auf Einträge der Inode Table im Hauptspeicher.

Die *Inode Table* im Hauptspeicher enthält für jede geöffnete Datei die Kopie ihres Eintrags in der Inode Table auf dem Plattenspeicher. Darüber hinaus gibt jeweils ein Referenzzähler an, wie viele Einträge der File Table auf diesen Eintrag verweisen. Auch die Inode Table kann von allen Prozessen benutzt werden.

Lese-/Schreibzeiger für die geöffneten Dateien werden also in der File Table geführt, die zwischen den prozessspezifischen Tabellen und der globalen Inode Table liegt. Das gibt zwei (oder auch mehr) Prozessen die Wahlmöglichkeit, auf eine Datei entweder unter verschiedenen Zeigern zuzugreifen (= verschiedene File-Table-Einträge) oder ei-

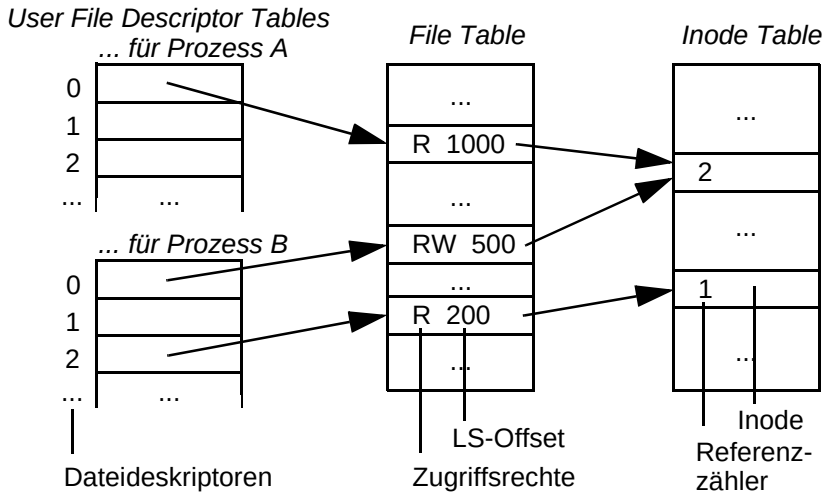


Abb. 6.7 Tabellen zur Dateibenutzung in UNIX

nen gemeinsamen Zeiger zu benutzen (= ein gemeinsamer File-Table-Eintrag). Insbesondere haben nach der Ausführung von `fork()` der Vater- und der Sohnprozess zwar getrennte User File Descriptor Tables, deren Einträge aber zunächst auf jeweils dieselben Positionen der File Table und damit dieselben Lese-/Schreibzeiger verweisen.

Dateien werden mit der Schnittstellenfunktion `creat()` (Prototyp: `int creat(char *filename, int mode)`) erzeugt. Der Parameter `filename` gibt dabei den Namen der neuen Datei an (wobei auch ein ganzer Pfad möglich ist) und `mode` die Zugriffsrechte. Bei der Ausführung wird ein Eintrag im entsprechenden Verzeichnis erzeugt, ein Inode auf der Platte belegt und initialisiert sowie Einträge in den drei Hauptspeichertabellen angelegt. Plattenblöcke werden noch nicht zugeteilt, da die Datei bei ihrer Erzeugung leer ist. Die Funktion gibt einen Dateideskriptor zurück, mit dem später auf die Datei zugegriffen werden kann. Er ist, wie schon gesagt, der Index des zugehörigen Eintrags in der User File Descriptor Table.

Die Schnittstellenfunktion `open()` (Prototyp: `int open(char *filename, int flags, int mode)`) „öffnet“ Dateien, d.h. bereitet nachfolgende Dateizugriffe vor. Existiert die Datei noch nicht, so wird sie analog zu `creat()` neu angelegt. Der Parameter `flags` gibt unter anderem an, welche Zugriffe auf der geöffneten Datei möglich sein sollen (z.B. nur Lesen oder Lesen und Schreiben); die beiden anderen haben dieselbe Bedeutung wie bei `creat()`. Bei der Ausführung wird zunächst überprüft, ob die Inode Table im Hauptspeicher schon einen Eintrag für diese Datei enthält. Ist das nicht der Fall, so wird ein neuer Eintrag angelegt, in den die entsprechenden Werte aus der Inode Table des Plattenspeichers kopiert werden. Anschließend werden Einträge in der File Table und der User File Descriptor Table initialisiert. Der Rückgabewert ist, wie bei `creat()`, ein Dateideskriptor. Man beachte, dass eine Datei nach ihrer Erzeugung durch `creat()` bereits geöffnet ist. Ein gesonderter Aufruf von `open()` ist also nicht nötig.

Daten können mit den Funktionen `read()` und `write()` aus geöffneten Dateien gelesen bzw. in sie geschrieben werden (Prototypen: `int read(int fd, char *buffer, int size)` bzw. `int write(int fd, char *buffer, int size)`). Der Parameter `fd` ist der Dateideskriptor, das Feld `buffer` ist das Ziel bzw. die Quelle der Daten, und `size` gibt die Länge von `buffer`, also die Anzahl der zu übertragenden Daten bzw. der Daten, die maximal empfangen werden können, an (in Bytes). Bei einem Zugriff wird zunächst die Berechtigung anhand der Rechtebits in der File Table geprüft. Anschließend werden der Plattenblock und die Position innerhalb des Blocks berechnet, auf die zugegriffen werden soll. Sie ergibt sich aus dem Lese-/Schreibzeiger, den Angaben im Inode zu den Dateiblöcken sowie der Blocklänge. Aus dieser Position wird dann gelesen bzw. in sie geschrieben, wobei ggf. ein neuer Plattenblock angelegt wird, wenn die Schreibposition jenseits des bisherigen Dateiendes liegt. Der Lese-/Schreibzeiger kann mit der Funktion `lseek()` neu positioniert werden.

Mit `close()` (Prototyp: `int close(int fd)`) wird eine Datei wieder geschlossen. Dabei werden die Einträge in den drei Hauptspeichertabellen wieder gelöscht; die Einträge in der File Table und der Inode Table allerdings nur, wenn nicht noch weitere Verweise auf sie existieren.

Eine Datei wird automatisch gelöscht, wenn kein entsprechender Verzeichniseintrag mehr existiert. Dabei werden ihre Plattenblöcke und ihr Inode auf der Platte wieder freigegeben. Die Schnittstellenfunktion zum Löschen von Verzeichniseinträgen heißt `unlink()` (Prototyp: `int unlink(char *filename)`).

Eine Zusammenstellung der wichtigsten Dateifunktionen mit Anwendungsbeispielen befindet sich im Anhang.

Die Vorgehensweise von VFS in Linux ist der des klassischen UNIX sehr ähnlich: Jeder Prozess besitzt ein Verzeichnis der von ihm geöffneten Dateien. Dieses Verzeichnis enthält für jede dieser Dateien einen Verweis auf ein File-Objekt (siehe voriger Abschnitt), in dem u.a. der Lese-/Schreibzeiger, die aktuellen Zugriffsrechte und ein Verweis auf den zugehörigen Inode stehen. Der Inode in der Inode-Tabelle enthält dann Informationen zur Lokalisierung der Daten auf der Platte.

Freispeicherverwaltung und Platzierung von Dateien

Die Freispeicherverwaltung eines Plattenspeichers kann sich auf eine so genannte *Bitmap* stützen, d.h. einen Array, in dem jedem Plattenblock ein Bit zugeordnet ist. Dieses Bit hat den Wert 0 genau dann, wenn der Block momentan frei ist. Linux verwendet diesen Ansatz. Auch in der FAT findet sich im Prinzip dieses Schema, da hier jeder Block einen Eintrag hat, aus dem geschlossen werden kann, ob er momentan frei ist. Anstelle der Bitmap kann auch eine verkettete Freibereichsliste verwendet werden, in der z.B. in jedem freien Block ein Zeiger auf den nächsten freien Block gespeichert ist.

Bei der Zuteilung von freien Blöcken zu einer Datei ist es wichtig, die *Plattenfragmentierung* zu minimieren. Die Datei sollte nicht über die Platte verstreut sein, sondern möglichst zusammenhängend abgelegt werden, um beim Durchlaufen der Datei die Suchzeiten gering zu halten. Insbesondere Realzeitsysteme erfordern eine derartige Speicherung von Dateien. Ein Beispiel ist hier ein Multimediasystem, in dem Video- und Audio-Dateien kontinuierlich abgespielt werden sollen. Diese Dateien sollten zu-

sammenhängend auf der Platte liegen, da der Übergang des Plattenarms zu einer weiter entfernt liegenden Spur zu einer Verzögerung der Ausgabe führen würde.

Entsprechende Ansätze zur Plattenspeicherverwaltung finden sich in vielen Betriebssystemen: Linux versucht, die Einträge einer Datei und den zugehörigen Inode in nahe beieinander liegenden Plattenblöcken unterzubringen. Solaris arbeitet mit zusammenhängenden „Clustern“ von 56 KByte, wann immer das möglich ist. Im Großrechnerbetriebssystem VM/CMS werden Dateien generell nur zusammenhängend abgespeichert.

Erhöhung des Durchsatzes

Wie schon in Kapitel 5 erwähnt, können Zugriffe auf das Dateisystem durch einen *Buffer Cache (Platten-Cache)* beschleunigt werden. Das ist ein Bereich des Hauptspeichers, in dem Kopien der am häufigsten benutzten Plattenblöcke stehen. UNIX beispielsweise benutzt einen Cache, der nach der LRU-Verdrängungsstrategie verwaltet wird; aber auch schon das alte MS-DOS implementierte ein solches Konzept.

Ebenfalls durchsatzsteigernd wirkt die zusammenhängende Speicherung von Dateien, wie sie im vorigen Teilabschnitt dargestellt wurde: Je weniger der Plattenarm bewegt werden muss, desto mehr Zeit bleibt für die Übertragung von Nutzdaten.

Schließlich lässt sich ein höherer Datendurchsatz mit Hilfe der *RAID-Technik* erreichen, die im folgenden Abschnitt besprochen wird.

6.1.5 Fehlertoleranz

Zur Vermeidung von Datenverlusten müssen Dateisystem und Plattenspeicher fehlertolerant ausgelegt sein, d.h. es muss auch bei einem Ausfall eines Teils der Speicherbereiche noch ein Zugriff auf die gespeicherten Daten möglich sein. Dies wird durch die redundante Führung von Hardwarekomponenten und Daten erreicht, wie es beispielsweise bei der *RAID-Technik* geschieht.

RAID

Die Abkürzung RAID steht für „Redundant Array of Inexpensive Disks“. Hier ist das Dateisystem nicht auf einer großen (teuren) Magnetplatte gespeichert, sondern auf mehreren kleinen (billigen) Platten, auf die nebenläufig zugegriffen werden kann. Bei jeder Datei verteilt sich der Inhalt auf mehrere dieser Platten; es findet also ein so genanntes „Striping“ statt, durch das – unter Ausnutzung der Nebenläufigkeit – der Datendurchsatz erhöht wird.

Durch Hinzunahme von Redundanzinformationen, wie z.B. fehlererkennende Parity Bits oder fehlerkorrigierende Hamming und Reed Solomon Codes, lassen sich Plattenfehler und -ausfälle kompensieren. Je nach Verteilung und Art der Daten und Korrekturcodes unterscheidet man sieben verschiedene RAID-Stufen RAID 0 bis RAID 6 (siehe Abb. 6.8). In der Praxis werden meist RAID 1 (auch manchmal RAID 10 genannt) oder RAID 5 verwendet – z.B. optional in Windows NT.

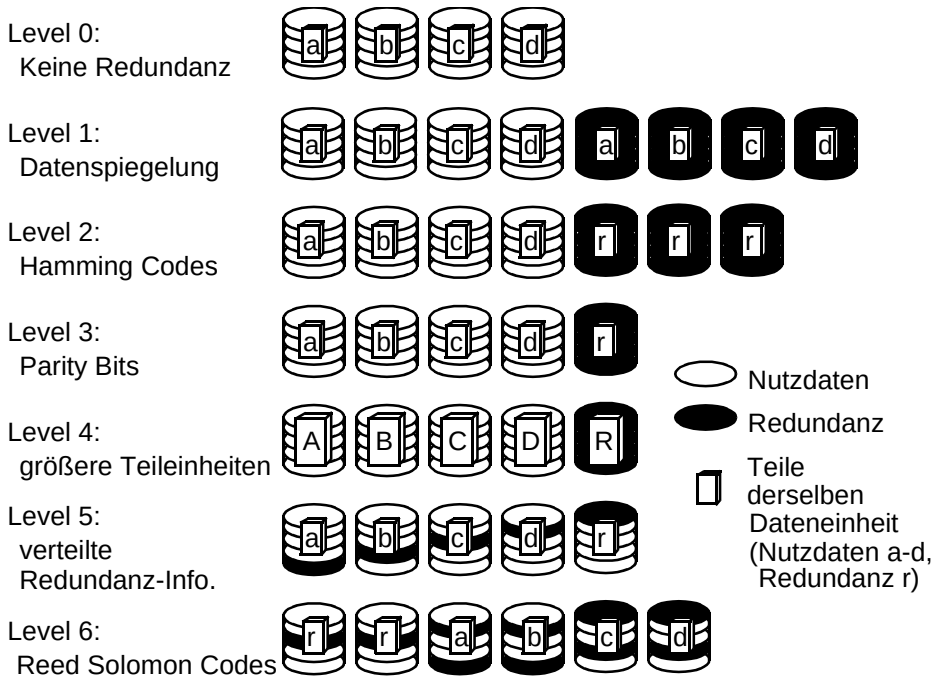


Abb. 6.8 RAID: Redundant Array of Inexpensive Disks

Backups

Größere Datenverluste bei Totalausfall der Magnetplatte(n) lassen sich dadurch vermeiden, dass *Sicherungskopien* (engl. *backups*) von Dateien auf Disketten, Magnetbändern oder Plattenlaufwerken anderer Rechner abgelegt werden. Ein solcher Backup sollte in regelmäßigen, nicht zu großen Zeitabständen erstellt werden. Dabei kann man entweder einen *vollständigen Backup* anlegen, also alle Dateien speichern, was sehr zeitaufwendig sein kann, oder einen *inkrementellen Backup* erzeugen, d.h. nur die Dateien kopieren, die seit der letzten Sicherung geändert wurden. Inkrementelle Backups wurden schon von MS-DOS unterstützt: Jede Datei besitzt ein *Archive Bit*, das bei einer Änderung der Datei automatisch gesetzt wird. Bei einer Datensicherung brauchen dann nur Dateien mit gesetztem Bit berücksichtigt zu werden.

Weitere Maßnahmen

Weitere Maßnahmen zur Vermeidung von Datenverlusten sind beispielsweise die Buchführung über fehlerhafte Platten- oder Diskettenblöcke sowie Mechanismen zur Überprüfung der Konsistenz des Dateisystems. UNIX führt beispielsweise beim Systemstart einen *Block Consistency Check* und einen *File Consistency Check* durch. Der Block Consistency Check vergleicht die Liste der freien Plattenblöcke mit den Blöcken,

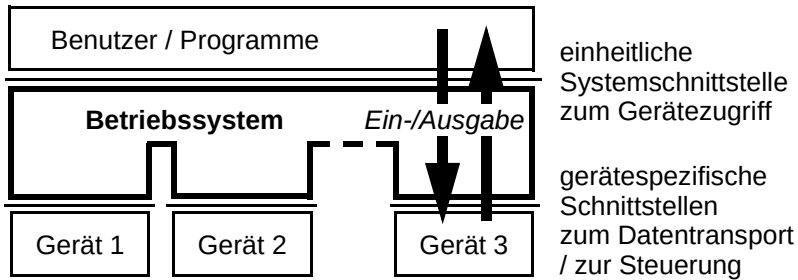


Abb. 6.9 Position des Betriebssystems bei der Ein-/Ausgabe

auf die die Inodes verweisen, und stellt so unter anderem fest, ob ein Block gleichzeitig als frei und belegt markiert ist. Der File Consistency Check vergleicht für jede Datei den Referenzzähler in ihrem Inode mit der Anzahl der Einträge für diese Datei in den Verzeichnissen.

6.2 Ein-/Ausgabe

Unter dem Stichwort *Ein-/Ausgabe* (E/A, *Input/Output*, I/O) versteht man den Datentransport zwischen der Zentraleinheit des Systems, also Prozessor und Hauptspeicher, und ihrer Peripherie (= Umwelt). Diese Definition umfasst auch Zugriffe auf den Plattenspeicher und damit auf das Dateisystem. Die Ein-/Ausgabe findet mit Hilfe von E/A-Geräten statt, die über eine spezielle Elektronik an die Zentraleinheit angeschlossen sind. Die Aufgabe des Betriebssystems ist dabei, einerseits die Geräte über ihre Schnittstellen gerätespezifisch zu steuern und andererseits den Prozessen gegenüber eine Schnittstelle mit einheitlichen, geräteunabhängigen E/A-Systemfunktionen zu bieten (siehe Abb. 6.9).

6.2.1 E/A-Hardware

Zu den E/A-Geräten gehören

- Speicher (z.B. Plattenspeicher, Magnetbandgerät, CD-ROM-Laufwerk),
- Eingabegeräte (z.B. Tastatur, Maus, Sensor),
- Ausgabegeräte (z.B. Bildschirm, Drucker, Roboterarm) sowie im weiteren Sinne auch
- Kommunikationsnetze, die in Kapitel 8 etwas näher betrachtet werden.

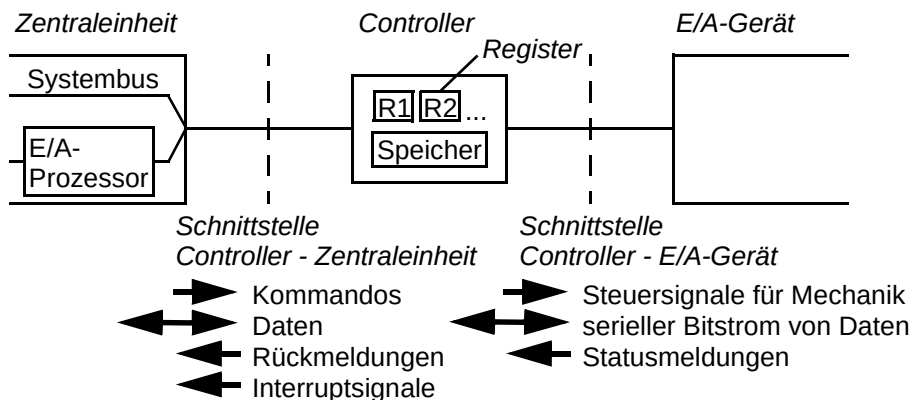


Abb. 6.10 Position und Aufgaben des Controllers

Geräten der ersten drei Klassen ist gemeinsam, dass sie aus zweierlei Teilen bestehen, nämlich aus mechanischen und aus elektronischen Komponenten. Bei einem Magnetplattenspeicher beispielsweise handelt es sich bei den mechanischen Komponenten um den Arm mit den Schreib-/Leseköpfen, den Antriebsmotor, die Scheiben mit ihren magnetischen Oberflächen usw.

Die mechanischen Teile interessieren uns hier nicht weiter. Wir setzen stattdessen bei den elektronischen Komponenten an, die die Mechanik des Geräts steuern und daher als *Controller* bezeichnet werden. Sie bilden nach außen die Schnittstelle zur Zentraleinheit und sind damit das Bindeglied zwischen der Software des Betriebssystems und der realen Hardware. Deswegen ist auch der Begriff *Adapter* gebräuchlich.

Controller

Ein Controller befindet sich entweder auf einer Karte, die in einem „Slot“ der Zentraleinheit steckt, oder direkt im E/A-Gerät, wie es beispielsweise bei modernen PC-Festplatten der Fall ist. Er besitzt zwei Schnittstellen (siehe Abb. 6.10): Die erste Schnittstelle dient zur Kommunikation mit dem Prozessor oder dem Hauptspeicher und ist entweder direkt mit dem Systembus (meist bei PCs und Workstations) oder mit speziellen E/A-Prozessoren (insbesondere bei Großrechnern) verbunden. Die zweite Schnittstelle führt zum eigentlichen E/A-Gerät, das über eine Steckdose an der Außenseite des Computers oder über interne Leitungen angeschlossen ist. Je nach Leistungsfähigkeit des Controllers können über die Geräteschnittstelle auch mehrere E/A-Geräte gesteuert werden. Ist die Schnittstelle standardisiert (z.B. SCSI u.a. für Festplatten, VGA für Monitore), so ist der Anschluss von Geräten verschiedener Hersteller möglich.

Zwischen Controller und Gerät werden Signale unterschiedlicher Typen übertragen: Steuersignale vom Controller an das Gerät lösen Aktionen aus, wie z.B. die Positionierung des Arms eines Magnetplattenspeichers. Statusmeldungen des Geräts informieren den Controller über den Zustand des Geräts und eventuell aufgetretene Fehler. Daten

können, je nach Art des E/A-Geräts, in einer oder in beiden Richtungen zwischen Controller und Gerät fließen. Sie werden als serieller Bitstrom (also als Folge einzelner Bits) übertragen, der beispielsweise mit einer Präambel (= „Einleitung“) beginnt, mit einer bestimmten Anzahl von Nutzdaten fortgesetzt wird (z.B. 4096 Bit, entsprechend einem Plattensektor von 512 Byte) und mit Codes zur Fehlererkennung und -korrektur endet.

Der Controller verfügt über einen internen Speicher, in dem er die Daten „puffern“, d.h. zwischenspeichern kann. Daten, die vom Gerät eintreffen, müssen also nicht unmittelbar Bit für Bit an die Zentraleinheit weitergegeben werden, sondern können zunächst „aufgesammelt“ und anschließend in größeren Einheiten weitertransportiert werden. Der Controller nimmt zudem eine Fehlerprüfung vor und führt gegebenenfalls eine Fehlerbehandlung durch.

Die Kommunikation zwischen dem Controller und der Zentraleinheit stützt sich auf eine Reihe von *Controller-Registern*. Über sie kann einerseits das Betriebssystem Kommandos an den Controller übergeben, z.B. die Aufforderung zu einer Schreib- oder Leseoperation und die Angabe entsprechender Speicheradressen. Andererseits kann der Controller hier Rückgabewerte (wie Status- und Fehlermeldungen) an das Betriebssystem übertragen.

Das Betriebssystem kann auf die Controller-Register direkt zugreifen. Es spricht sie über spezielle E/A-Adressen an oder, alternativ, über „normale“ Hauptspeicheradressen, die durch die Hardware auf die Register abgebildet werden (ein so genannter „memory-mapped I/O“). Der Controller informiert das Betriebssystem über bestimmte Ereignisse, wie beispielsweise den Abschluss einer E/A-Operation; dies geschieht über Interruptsignale.

Im Prinzip könnten die ein- oder auszugebenden Daten ebenfalls über die Controller-Register transportiert werden, was aber viel zu aufwendig wäre: Der Controller müsste nämlich eingelesene Daten jeweils in den Registern ablegen und dann ein Interruptsignal an den Prozessor schicken. Der aktuelle Prozess würde unterbrochen, und der Prozessor müsste einen Interrupthandler ausführen, der die Daten in den Hauptspeicher überträgt. Da die Register klein sind (genauso wie der Controller-Speicher, der alternativ zu den Registern benutzt werden könnte), träten solche Interrupts in rascher Folge auf. Der Aufwand für die Ausgabe wäre ähnlich hoch.

Direct Memory Access

Wesentlich weniger „störend“ ist eine Technik, die mit *Direct Memory Access (DMA)* bezeichnet wird. Hier gibt das Betriebssystem dem Controller über die Register die Anfangsadresse und die Länge eines Hauptspeicherbereichs an, aus dem bzw. in den Daten transportiert werden sollen. Der Controller transferiert dann die Daten direkt über den Bus aus dem bzw. in den Hauptspeicher und informiert anschließend das Betriebssystem mit einem Interrupt über das Ende der Operation. Der Prozessor ist also durch den eigentlichen Datentransport nicht betroffen und kann während dieser Zeit die Benutzer- und Systemprozesse weiter ausführen. Prozessor und Controller müssen lediglich sicherstellen, dass sie nicht gleichzeitig auf den Bus zugreifen. Das geschieht mit Bussperren, die durch die Hardware gesteuert werden.

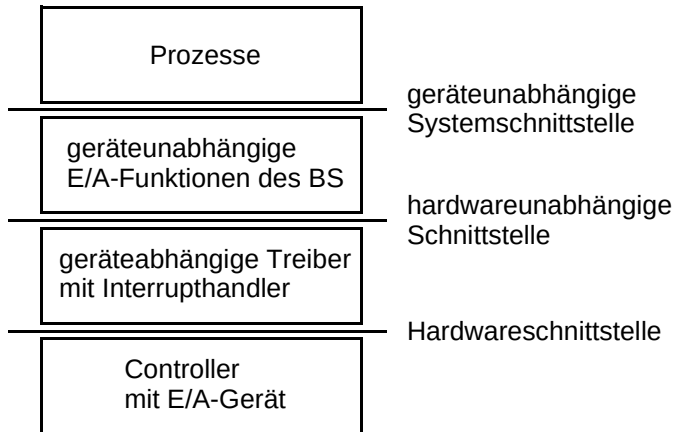


Abb. 6.11 Hierarchie der E/A-Hard- und Software

6.2.2 E/A-Software

Der Controller bietet also gegenüber der Zentraleinheit eine Hardwareschnittstelle, auf der die Software des Systems aufsetzen kann. Diese Software ist hierarchisch aufgebaut (siehe Abb. 6.11).

Treiber

Die unterste Komponente der Softwarehierarchie ist der *(Geräte-)Treiber* (engl. *device driver*), der die gerätespezifische Steuerung des Controllers übernimmt. „Gerätespezifisch“ bedeutet dabei, dass die Treibersoftware die Schnittstelle des Controllers und seine Funktionalität genau „kennt“ und in der Lage ist, mit dem Controller über die Register und Interruptsignale entsprechend zu kommunizieren. Für jedes Gerät, oder zumindest jeden Gerätetyp, existiert ein eigener Treiber. Im Einzelnen hat der Treiber die folgenden Aufgaben:

- Initialisierung des E/A-Geräts beim Systemstart.
- Vorbereitung einer E/A-Operation und Aktivierung des Geräts für den Datentransport.
- Reaktion auf Geräte-Interrupts.
- Fehlerbehandlung.

Ein Treiber kann entweder als ein eigener Prozess oder als eine Sammlung betriebssysteminterner Funktionen realisiert werden, die beispielsweise durch Interrupthandler aufgerufen werden können.

Geräteunabhängige E/A-Software

Der Treiber bietet nach oben eine geräteunabhängige Schnittstelle, auf der die nächste Stufe der Software aufsetzen kann. Dies ist die *geräteunabhängige E/A-Software*, also Code innerhalb des Betriebssystems, der zwar zur Ausführung von E/A-Operationen dient, aber nicht mehr die speziellen Eigenschaften der einzelnen Geräte berücksichtigen muss. Sie dient beispielsweise dazu, über logische Geräteadressen (z.B. externe Gerätenamen) die entsprechenden Treiber anzusprechen, Geräte vor unberechtigtem Zugriff zu schützen und Daten auf ihrem Weg zwischen den Prozessen und den Treibern zwischenspeichern. Die Software bietet nach oben, d.h. an der Betriebssystemschnittstelle, einen Satz von E/A-Systemfunktionen, mit denen Prozesse Daten einlesen oder ausgeben können.

Auch oberhalb der Systemschnittstelle kann Code vorhanden sein, der speziell zur Durchführung von E/A-Operationen dient. Beim *Druckerspooling* beispielsweise übernimmt ein *Spoolingdaemon* (ein spezieller Prozess) die Druckaufträge aller Prozesse und steuert als Einziger den Drucker selbst an.

Zeitlicher Ablauf einer E/A-Operation

Abbildung 6.12 illustriert den zeitlichen Ablauf einer E/A-Operation: Ein Benutzerprozess ruft über die Systemschnittstelle eine Operation auf einem bestimmten E/A-Gerät auf. Die geräteunabhängige Software des Betriebssystems identifiziert und aktiviert daraufhin den zuständigen Treiber, der seinerseits die entsprechenden Controller-Register setzt und somit eine Geräteoperation startet. Der Treiber blockiert sich dabei bis zur Rückmeldung des E/A-Geräts. Der Controller führt mit dem eigentlichen Gerät die Operation aus, besetzt seine Register mit Rückgabewerten und „weckt“ den Treiber durch ein Interruptsignal. Der Treiber wertet die Rückgabewerte aus und reagiert entsprechend, z.B. durch die Behandlung eines gemeldeten Fehlers, durch die Bereitstellung weiterer auszugebender Daten oder durch eine Fertigmeldung an den aufrufenden Prozess. Der Treiber steuert also den Controller möglicherweise mehrfach an, bis der E/A-Vorgang abgeschlossen ist. Schließlich kehrt die Ausführung über die geräteunabhängige E/A-Software zum aufrufenden Prozess zurück, wobei Rückgabewerte übergeben werden.

6.2.3 E/A-Konzept in UNIX / Linux

Die Grundidee bei der Ein-/Ausgabe in UNIX und Linux ist, die E/A-Geräte vollständig in das Dateisystem zu integrieren: Geräte werden als „*special files*“ angesehen, also als Dateien eines speziellen Typs. Dabei wird zwischen *zeichenorientierten Geräten* (engl. *character devices*, wie z.B. die Tastatur) und *blockorientierten Geräten* (engl. *block devices*, wie z.B. die Magnetplatte) unterschieden. Wie schon früher erwähnt, enthält der Dateibaum im Verzeichnis `/dev` Einträge für die E/A-Geräte. Für sie gelten dieselben Schutzkonzepte wie bei einfachen Dateien und Verzeichnissen, und sie können vom Benutzer mit denselben Kommandos wie die übrigen Dateien angesprochen werden.

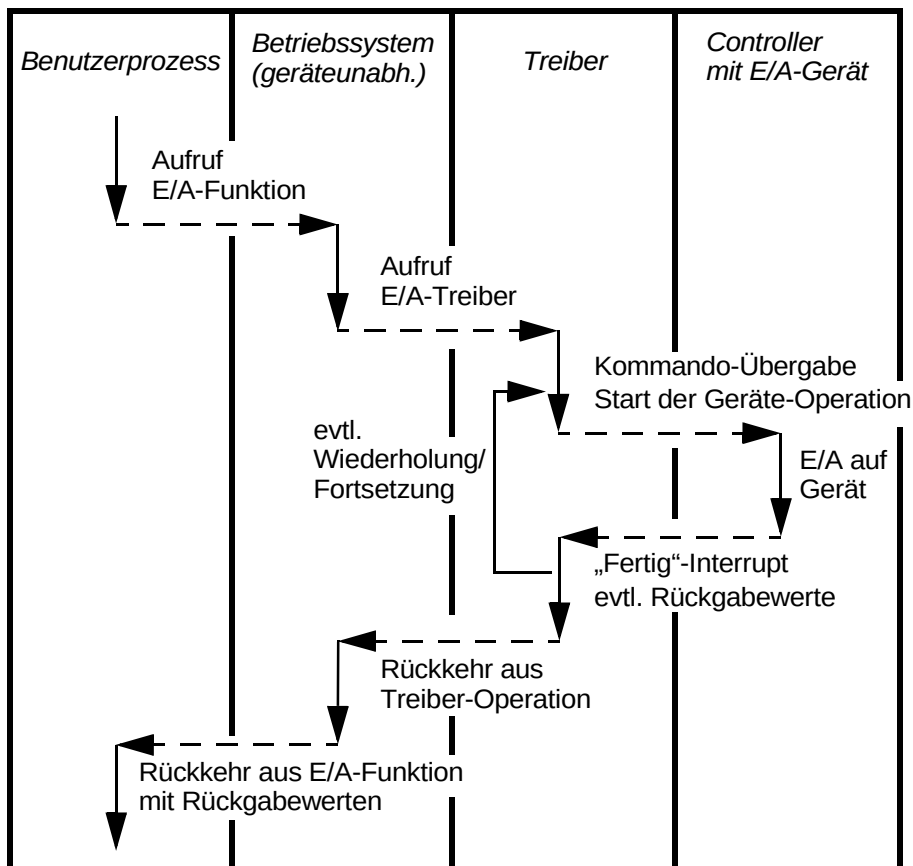


Abb. 6.12 zeitlicher Ablauf einer E/A-Operation

Auch die Schnittstellenfunktionen zum Zugriff auf „normale“ Dateien und auf E/A-Geräte sind im Wesentlichen dieselben, nämlich `open()`, `close()`, `read()` und `write()`. Für E/A-Geräte gibt es zusätzlich die Funktion `ioctl()`, mit der in gewissen Grenzen eine Gerätesteuerung möglich ist – z.B. durch das Setzen von Geräteparametern oder durch Operationen auf der Warteschlange der E/A-Aufträge. Die Ein- und Ausgabe ist jeweils Bytestrom-orientiert, d.h. die übertragenen Daten werden als unstrukturierte Folge von Bytes angesehen – gleichgültig, ob eine normale Datei oder ein E/A-Gerät betroffen ist.

Innerhalb des UNIX- und des Linux-Kerns sind E/A-Geräte ebenfalls in das Dateisystem eingegliedert, indem für sie spezielle Inodes verwendet werden (siehe Abb. 6.13). Verzeichnisse können Verweise auf „normale“ Datei-Inodes enthalten oder Verweise auf *Geräte-Inodes*. In Geräte-Inodes stehen, anstelle einer Blockliste, zwei Identifikatoren, nämlich die *Major Device Number* und die *Minor Device Number*. Die Major Device Number ist ein Index in eine *Device Switch Table* (bzw. *Function Table* in Linux), die Informationen über die einzelnen E/A-Geräte enthält. So identifiziert ein

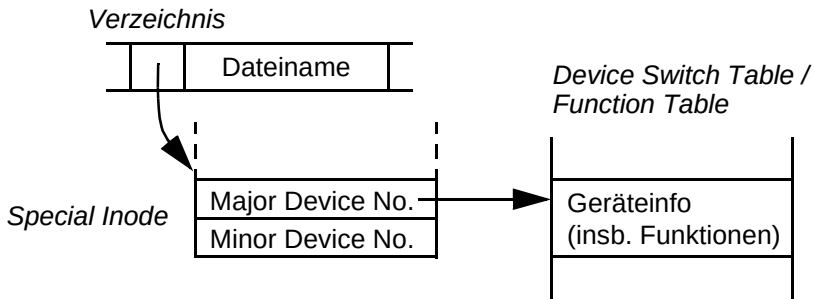


Abb. 6.13 Einbindung der E/A-Geräte in das UNIX/Linux-Dateisystem

Eintrag dieser Tabelle insbesondere die gerätespezifischen Funktionen, die den Schnittstellenfunktionen `open()`, `close()` usw. entsprechen – für jedes Gerät werden somit die geräteunabhängigen auf die geräteabhängigen Funktionen abgebildet. Mehreren Geräten desselben Typs kann derselbe Tabelleneintrag zugeordnet werden. Sie haben also dieselbe Major Device Number und werden anhand der Minor Device Number unterschieden.

6.3 Übungsaufgaben

1. Wissensfragen

- Was ist eine byte-orientierte Datei?
- Was ist ein Link?
- Welcher Hauptunterschied besteht zwischen dem FAT-Dateisystem und dem Inode-basierten Dateisystem von UNIX?
- Was leistet das Virtual File System (VFS) von Linux?
- Was ist RAID?
- Wozu dient ein Controller und wozu ein Treiber?

2. FAT-basiertes Dateisystem

Gegeben ist ein sehr vereinfachter Ausschnitt eines FAT-basierten Dateiverzeichnisses, bei dem nur die Dateinamen und jeweils rechts davon die Nummern der zugehörigen ersten Plattenblöcke angegeben sind:

A.TXT	5	B.EXE	12	C.DOC	16	D.COM	4	E.HTM	11
-------	---	-------	----	-------	----	-------	---	-------	----

Außerdem ist ein Anfangsausschnitt der File Allocation Table gegeben (erste Zeile: reale Plattenblocknummern, zweite Zeile: FAT-Einträge):

0	1	2	3	4	5	6	7	8	9	10	...
3	eof	bad	21	17	13	1	free	10	eof	6	...

...	11	12	13	14	15	16	17	18	19	20	21
...	19	18	0	eof	free	8	9	14	eof	free	eof

- a.) Geben Sie für die fünf Dateien des Verzeichnisses jeweils die zugehörigen Plattenblöcke an, und zwar in der Reihenfolge, in der die Daten der Datei abgespeichert sind.
- b.) Betrachten Sie die folgenden Paare aus Dateinamen und Bytenummern (gezählt vom Dateianfang): (B.EXE, 1998), (B.EXE, 2137), (E.HTM, 2137), (A.TXT, 7512). Geben Sie jeweils die Nummer des realen Plattenblocks an, in dem sich das Byte befindet, sowie den Offset (= die Bytenummer) innerhalb des Plattenblocks. Die Länge eines Blocks beträgt 2 KByte (= 2^{11} Byte). Bytenummern beginnen bei 0.
- c.) Die Datei A.TXT wird nun um zwei Blöcke verkürzt, d.h. ihre beiden letzten Blöcke werden freigegeben. Anschließend wird die Datei B.EXE um einen Block verlängert, wobei von den freien Blöcken der am weitesten links stehende belegt wird. Wie sieht die FAT anschließend aus? Ändert sich auch der oben angegebene Ausschnitt des Verzeichnisses?

3. Inode-basiertes Dateisystem

Gegeben sind drei Dateien, die jeweils in mehreren Blöcken der Festplatte gespeichert sind. Im Einzelnen handelt es sich dabei um die folgenden Dateien und Blocknummern:

- A.HTM: 2, 7, 3, 1
- B.TXT: 20, 11, 5, 17, 29, 19, 12, 8, 4, 30, 10, 9, 6, 13
- C.TAR: 35, 14, 15, 16, 21, 22, 23, 24, 25, 26, 27, 28

Hierbei ist die Reihenfolge der Blocknummern wichtig; der erste Block von A.HTM steht also in Plattenblock Nr. 2, der zweite Block in Plattenblock Nr. 7 usw. Die Länge eines Blocks beträgt 2 KByte (= 2^{11} Byte).

- a.) Gehen Sie von der in Abb. 6.5 dargestellten Struktur mit $m=10$ aus. Skizzieren Sie hierfür das interne Aussehen eines Verzeichnisses, das diese drei Dateien enthält, und außerdem das Aussehen der zugehörigen Inodes. Die Skizze soll insbesondere detailliert angeben, wo die Dateinamen stehen und wie ihnen die Plattenblocknummern zugeordnet sind. Setzen Sie zudem an die passende Stelle einen passenden Wert für die Dateigröße in Bytes ein.

- b.) Gegeben sind die folgenden Paare aus Dateinamen und Byte-nummern (gezählt vom Dateianfang):
(A.HTM, 600), (A.HTM, 2048), (B.TXT, 6000), (B.TXT, 20000), (C.TAR, 2047),
(C.TAR, 7100)

Geben Sie jeweils die Nummer des realen Plattenblocks an, in dem sich das Byte befindet, sowie der Offset (= die Byte-nummer) innerhalb des Plattenblocks. Byte-nummern beginnen bei 0.

- c.) Dauert für alle Byte-nummern einer UNIX-Datei die Ermittlung des zugehörigen Plattenblocks und Offsets gleich lang? Warum oder warum nicht?

4. Vergleich FAT- und Inode-basiertes Dateisystem

- a.) Sind Dateizugriffe unter dem UNIX-Dateisystem oder unter dem FAT-Dateisystem schneller – insbesondere auf Blöcke, die weit hinten in einer großen Datei liegen? Warum?
- b.) In welchem Fall ist der Schaden möglicherweise größer – beim Verlust eines Inodes oder beim Verlust eines Teilstücks der FAT mit derselben Länge? Warum?
- c.) Warum sind Links beim Inode-basierten Dateisystem wesentlich besser zu realisieren als beim FAT-basierten?

7 Sicherheit

Von einem Rechensystem wird gefordert, dass es zuverlässig und korrekt funktioniert. Beispielsweise soll die Hardware stets betriebsbereit sein, die Software soll fehlerfreie Ergebnisse liefern, auf fremde Daten soll nicht ohne Erlaubnis des Besitzers zugegriffen werden können, und ein normaler Benutzer soll sicherheitskritische Operationen nicht uneingeschränkt ausführen dürfen. Da nicht ohne weiteres vorausgesetzt werden kann, dass das System tatsächlich so arbeitet wie gewünscht, müssen besondere Mechanismen implementiert werden, die solche Forderungen durchsetzen. Forderungen und Mechanismen lassen sich unter zwei Schlagworten zusammenfassen, nämlich *Fehlertoleranz* (engl. *fault tolerance*) und *Sicherheit* (engl. *security*).

Ein System ist fehlertolerant, wenn es auch dann noch korrekt arbeitet, wenn ein Teil seiner Hard- und Softwarekomponenten ausgefallen ist oder fehlerhafte Resultate liefert. Die ersten beiden Forderungen, die oben als Beispiele aufgeführt sind, fallen unter diesen Begriff. Fehlertoleranz wurde schon in früheren Kapiteln angesprochen.

Die dritte und vierte Beispielforderung sind dem Bereich der Sicherheit zuzuordnen, der in diesem Kapitel diskutiert wird und der ein wichtiges Merkmal eines Betriebssystems ist. Vom US-Verteidigungsministerium wurde 1985 ein Schema („*Orange Book*“) erarbeitet, mit dem Betriebssysteme danach klassifiziert werden können, wie gut sie bestimmte Anforderungen erfüllen [DoD85]. Die Skala reicht von Systemen, die überhaupt keine Sicherheit bieten, bis zu Systemen, deren Sicherheit formal nachgewiesen wurde. Die heute weit verbreiteten Betriebssysteme rangieren hier relativ weit unten.

7.1 Problematik

Die Daten, die im System gespeichert sind, und die Fähigkeit des Systems, seine Funktionen korrekt auszuführen, sind auf unterschiedliche Weise bedroht. Erstens können unbeabsichtigte Ereignisse eintreten, wie höhere Gewalt, menschliches Versagen oder Hardware- und Softwarefehler. Um ein System gegen solche Ereignisse zu wappnen, muss es – wie bereits angesprochen – fehlertolerant ausgelegt sein.

Die zweite Art von Bedrohungen geht von böswilligen Eindringlingen („*Crackern*“) aus, also Personen oder ihren Programmen / Prozessen, die auf dem System unberechtigte Aktionen ausführen und das System möglicherweise zerstören wollen. Die Möglichkeiten dieser Eindringlinge sind vielfältig (siehe Abb. 7.1). Sie können beispiels-

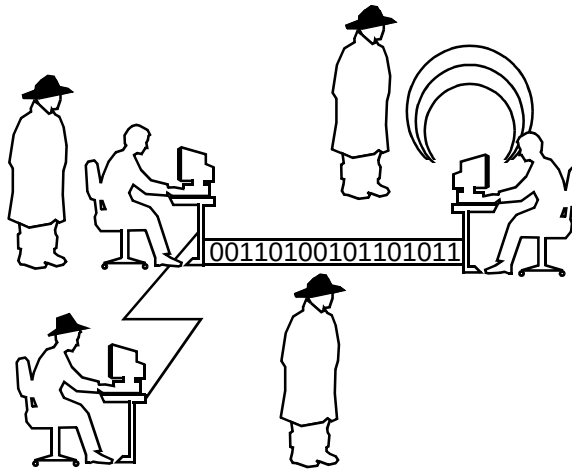


Abb. 7.1 Angriffsmöglichkeiten für Hacker und Cracker

weise versuchen, sich unberechtigt einen physischen oder elektronischen Zugang zum System zu verschaffen, gespeicherte fremde Daten zu lesen oder gar zu verändern, Datenübertragungen über einen Kanal mitzuhören oder Abstrahlungen von Geräten zu messen. Das grundlegende Problem ist also, zunächst einmal alle Möglichkeiten eines Eindringlings zu erkennen, um dann Mechanismen realisieren zu können, die alle diese Bedrohungen nachweislich abwehren.

Ein möglicher, etwas akademischer Ansatz beruht auf einem Modell des Systems, das drei Klassen von Komponenten umfasst:

- Eine Menge von passiven *Objekten* O enthält alle Systemkomponenten, die zu schützen sind. Für jedes Objekt ist eine Menge von Zugriffsoperationen definiert, mit denen auf dem Objekt gearbeitet werden kann.
- Eine Menge von aktiven *Subjekten* S enthält alle Aktivitätsträger, die auf die Objekte zugreifen.
- Eine Menge von *Zugriffsrechten* legt für jedes Paar aus einem Subjekt $s \in S$ und einem Objekt $o \in O$ fest, welche Zugriffsoperationen s auf o ausführen darf.

Im UNIX-Dateisystem sind beispielsweise die Benutzer oder ihre Prozesse die Subjekte, die Dateien sind die Objekte mit Lese-, Schreib- und Ausführungsoperationen, und die individuellen Zugriffsrechte werden durch die neun Schutzbits der einzelnen Dateien festgelegt.

Auf der Grundlage eines solchen Modells muss das Betriebssystem dann so konstruiert werden, dass die Zugriffsrechte durchgesetzt werden. Dabei ist sicherzustellen (möglichst unter Beweis), dass ein Subjekt s genau dann eine Zugriffsoperation auf einem Objekt o ausführen kann, wenn s ein entsprechendes Zugriffsrecht an o hat.

Die fundamentale Schwierigkeit bei diesem Ansatz ist, dass jedes Modell eine Vereinfachung des realen Systems darstellt und damit nicht alle möglichen Zugriffswege erfasst. Kritisch sind hier insbesondere so genannte *verdeckte Kanäle*, bei denen Informationen über Wege übertragen werden, die dafür eigentlich nicht vorgesehen sind. Beispiele für verdeckte Kanäle sind die folgenden:

- Plattenblöcke oder Hauptspeicherbereiche, die freigegeben wurden, können noch Daten des Vorbesitzers enthalten, so dass sie der nachfolgende Besitzer lesen kann. Sie sollten also überschrieben werden – auf einem Magnetplattenspeicher vorzugsweise mehrfach hintereinander mit unterschiedlichen Bitmustern, um die Magnetisierung möglichst umfassend zu ändern [BSI00]. Der Schreib-Lese-Kopf der Platte trifft die Spur nämlich meist nicht exakt, so dass eventuell Teile der alten Magnetisierung erhalten bleiben. Dies wurde beispielsweise dafür ausgenutzt, um Dateien des Bundeskanzleramts, die im Herbst 1998 gelöscht wurden, wiederherzustellen (was übrigens in den USA nicht passieren könnte, da dort bei einem Präsidentenwechsel die alte Mannschaft die Amtsräume radikal leerräumt [Tag01] und insbesondere die Computer ohne Festplatten zurücklässt).
- Beliebige Informationen können übertragen werden, indem man sie durch eine Bitfolge aus Nullen und Einsen codiert und diese Folge dann in irgendwelche Aktivitätsmuster (beispielsweise in Phasen mit wenigen und mit vielen Zugriffen auf die Festplatte) umsetzt, die dann von außen abgehört werden können (wenig Aktivität = 0, viel Aktivität = 1). Cracker können so unter Umgehung der normalen Ein-/Ausgabe Informationen nach außen transferieren.
Ähnlich ging man übrigens, in Ermangelung einer besseren Technik, in Fernsehshows der sechziger Jahre vor. Zuschauer konnten über bestimmte Fragen abstimmen, indem sie die Wasserspülung betätigten oder es bleiben ließen. Nachfragen bei den Wasserwerken lieferten dann das Abstimmungsergebnis.
- Bei Chipkarten ist es gelungen, die (geheime) Befehlsfolge des ausgeführten Programms zu rekonstruieren, indem man den Verlauf der Stromaufnahme beobachtete [Schm98]. Die Chipkarte könnte so nachgebildet werden, wodurch sicherheitskritische Manipulationen möglich wären.

Auf die Objekte eines Systems kann also nicht nur über ihre im Modell erfassten Zugriffsooperationen, sondern auch über andere, „unerlaubte“ Wege zugegriffen werden. Daher darf man sich nicht ausschließlich auf formale Sicherheitsmodelle verlassen, sondern es sollten schon bei der Konstruktion des Systems u.a. bezahlte (oder anders motivierte) Hacker eingesetzt werden: Sie versuchen praktisch, die Sicherheitsmaßnahmen zu überwinden, und liefern so Hinweise auf Schwachstellen, die dann beseitigt werden müssen. Man kann sich dabei auch auf Erfahrungen mit anderen Systemen stützen, die in allgemeinen Listen potentieller Schlupflöcher verzeichnet sind. Mittlerweile gibt es sogar Programme, die alle möglichen Schwachstellen systematisch durchtesten. So etwas ist natürlich eine zweischneidige Sache, denn auch böswillige Cracker dürften eine solche Software gern benutzen [auf eine genaue Quellenangabe verzichten wir an dieser Stelle lieber].

Man beachte hier übrigens den feinen Unterschied zwischen den Bezeichnungen „Hacker“ und „Cracker“: Beide versuchen, in ein System einzudringen – der Hacker allerdings eher aus „sportlichen“ Motiven und um möglicherweise dem Systembetreiber oder der Öffentlichkeit Gefahren aufzuzeigen, der Cracker dagegen aus böswilliger Absicht, um Daten auszuspionieren oder das System zu zerstören. Als Hackervereinigung hat sich beispielsweise der Chaos Computer Club hervorgetan [CCC01]; Beispiele für Cracking entnehme man der aktuellen Tagespresse.

Es zeigt sich also, dass die Sicherheitsproblematik sehr vielschichtig ist. Es lassen sich aber grob zwei Bereiche der Sicherheit unterscheiden, die zu bearbeiten sind:

- Die *externe Sicherheit* bezieht sich auf den Schutz gegen unberechtigte Eindringlinge von außen. Sie wird an der Außengrenze des Systems mit *Zugangskontrollen* durchgesetzt und stellt sicher, dass alle diejenigen abgewiesen werden, die keinerlei Arbeitsberechtigung auf dem System haben.
- Die *interne Sicherheit* befasst sich mit dem Schutz gegen Benutzer, die zwar prinzipiell im System arbeiten dürfen, aber bestimmte Aktionen nicht ausführen sollen. Sie wird innerhalb des Systems durchgesetzt, indem für die einzelnen Objekte *Zugriffskontrollen* durchgeführt werden.

Für beide Arten der Sicherheit ist an zentraler Stelle das Betriebssystem verantwortlich, wobei es sich vielfach auf Hardwaremechanismen stützt.

7.2 Externe Sicherheit

Ziel der externen Sicherheit ist, den berechtigten Benutzern – und nur diesen! – den Zugang zum System zu gewähren. Ein Benutzer ist zugangsberechtigt, wenn es im System irgendeine Tätigkeit gibt, die er ausführen darf. Wir werden in diesem Unterkapitel zunächst grundlegende Techniken der externen Sicherheit angeben und anschließend zwei Ansätze etwas detaillierter betrachten, nämlich wissensbasierte Mechanismen (insbesondere Passwörter) und biometrische Verfahren.

7.2.1 Grundlegende Techniken

Um die externe Sicherheit durchzusetzen, muss zunächst einmal sichergestellt werden, dass das System resistent gegen Cracking ist, dass ein möglicher Benutzer sich also nur auf dem dafür vorgesehenen Weg Zugang zum System verschaffen kann. Dies entspricht einem Gelände, das nur am Eingangstor, nicht aber an anderer Stelle betreten werden kann. Auf dieser Grundlage muss dann beim Eintritt einer Person in das System eindeutig festgestellt werden, wer diese Person ist – ähnlich der Ausweiskontrolle beim Betreten eines Geländes.

Schutz gegen Cracking

Wie im vorigen Unterkapitel dargestellt, ist die Verhinderung von Cracker-Angriffen ein vielschichtiges Problem, dem auf mehreren Ebenen begegnet werden muss:

- *Bauliche Maßnahmen* müssen sicherstellen, dass sich niemand unberechtigt physischen Zugriff zum System verschaffen oder das System von außen abhören kann. So sollten beispielsweise Zentraleinheit und Peripheriespeicher in einem abgeschlossenen Raum untergebracht werden, zu dem nur der Systemverwalter Zutritt hat. Wichtige Datenbänder und Disketten sollten in Panzerschränken aufbewahrt werden. Ein Metalkäfig bietet Schutz gegen das Austreten „kompromittierender“ Strahlung, die z.B. von Bildschirmen mit Kathodenstrahlröhre abgegeben wird (siehe Abb. 7.1).

- Die Verbindung zum externen Datennetz / Internet muss so geschützt werden, dass von außen kein unberechtigter elektronischer Zugriff möglich ist. Ein *Firewall-Rechner* sollte den Fluss der Daten in den und aus dem sensitiven Bereich kontrollieren.

In Ergänzung der Firewall sollte Überwachungssoftware, ein so genanntes *Intruder Detection System (IDS)* oder *Intruder Response System (IRS)*, eingesetzt werden. Es hält nach Mustern typischer Cracker-Angriffe Ausschau, benachrichtigt gegebenenfalls den Systemverwalter oder ergreift (im Falle eines IRS) selbst Gegenmaßnahmen (siehe hierzu z.B. [Snor01] oder [CERT01]).

Rechner mit besonders sensiblen Daten und Aufgaben sollten überhaupt nicht an ein extern zugreifbares Netz angeschlossen werden.

- Hardware und Betriebssystem müssen so ausgelegt sein, dass ein Systemzugang nur über eine festgelegte *Login-Prozedur* möglich ist. Diese Prozedur, die von jedem zur Anmeldung am System benutzt werden muss, darf nicht umgangen oder überlistet werden können und muss die Identität des Benutzers eindeutig verifizieren.

Die Praxis zeigt, dass es vielfältige Möglichkeiten gibt, den vorgesehenen Login-Vorgang „auszutricksen“. Eine häufig angewandte Technik ist die Eingabe von überlangen Zeichenketten, um so einen Überlauf des Eingabepuffers zu provozieren und Daten in nachfolgende kritische Speicherbereiche zu schreiben. Beim Entwurf der Login-Prozedur muss an alle diese Möglichkeiten gedacht werden, um sie effektiv zu verhindern.

Als weitere Sicherheitsmaßnahme kann für jeden Benutzer der Zeitpunkt des letzten erfolgreichen Einloggens und die Anzahl der Fehlversuche aufgezeichnet und beim nächsten Einloggen angezeigt werden. Der Benutzer kann dann feststellen, ob jemand anderes unter seinem Namen aktiv war oder werden wollte. Um ein „Herumprobieren“ durch Cracker zu verhindern, kann nach mehreren Fehlversuchen der Login-Vorgang abgebrochen und eventuell die Benutzererkennung gesperrt werden.

Authentifizierung

Möchte sich eine Person in ein System einloggen, so muss eine *Zugangskontrolle* vorgenommen werden. Dabei wird festgestellt, ob diese Person tatsächlich die ist, als die sie sich ausgibt – die Person wird *authentifiziert*. Eine Authentifizierung ist unbedingt

erforderlich, denn insbesondere die internen Schutzmaßnahmen müssen sich darauf verlassen können, dass der Benutzer wirklich derjenige ist, als der er sich angemeldet hat (siehe Unterkapitel 7.3).

Beim Einloggen müssen also der Name des Benutzers oder eine andere Benutzerkennung sowie ein Beweis für die Richtigkeit des Namens eingegeben werden. Für diesen Beweis gibt es verschiedene Möglichkeiten:

- *Tokenbasierte Verfahren:* Hier präsentiert der Benutzer einen physischen Gegenstand (ein *Token*), der seinem Besitzer die gewünschten Zugriffsrechte gewährt – beispielsweise einen Schlüssel, eine Magnet- oder eine Chipkarte. Im System muss eine entsprechende Hardwarekomponente vorhanden sein, die das Token prüft.

Bei tokenbasierten Verfahren besteht die Gefahr, dass der Benutzer sein Token an einen Cracker verliert. Sie sollten daher nur zusammen mit anderen Methoden eingesetzt werden, insbesondere mit wissensbasierten Verfahren (siehe nächster Punkt). Wir werden tokenbasierte Verfahren im Folgenden nicht näher betrachten.

- *Wissensbasierte Verfahren:* Hier gibt der Benutzer eine Information in das System ein, die nur ihm bekannt ist – beispielsweise ein Passwort oder als Spezialfall davon eine PIN (= Persönliche Identifikationsnummer, wie von Bankkarten her bekannt). Die Betriebssystemsoftware prüft, ob die eingegebene Information mit gespeicherten Daten, die diesem Benutzer zugeordnet sind, in Einklang steht.

Wissensbasierte Verfahren sind Thema des folgenden Abschnitts.

- *Biometrische Verfahren:* Hier werden unveränderliche und eindeutige körperliche Kennzeichen des Benutzers (z.B. Fingerabdruck, Augeniris oder -hintergrund) aufgenommen und mit im System gespeicherten Daten verglichen. Im Prinzip könnten diese Merkmale auch benutzt werden, um den Benutzer zu erkennen (also zu *identifizieren*), so dass eine gesonderte Eingabe des Benutzernamens entfiel.

Biometrische Verfahren werden in Abschnitt 7.2.3 behandelt.

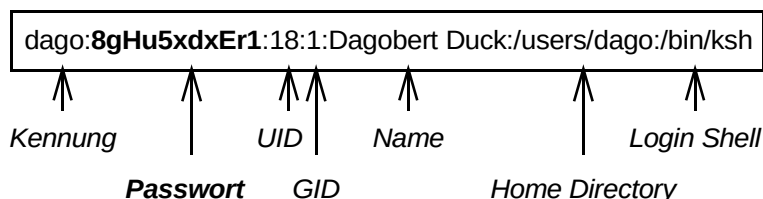
Von Authentifizierungsverfahren wird erwartet, dass sie mit hoher Zuverlässigkeit arbeiten: Erstens sollen sie unberechtigten Personen stets den Zugang zum System verweigern, zweitens sollen sie berechtigten Benutzern den Zugang stets ermöglichen.

7.2.2 Wissensbasierte Authentifizierung

Das heutige Standardverfahren zur wissensbasierten Authentifizierung beruht auf geheimen *Passwörtern*: Ein Benutzer vereinbart ein Passwort, das in der *Passwortdatei* des Systems abgelegt wird. Beim Einloggen tippt er sein Passwort ein, und das Betriebssystem vergleicht es mit dem Dateieintrag. Wurde ein falsches Wort eingegeben, so wird der Benutzer abgewiesen.

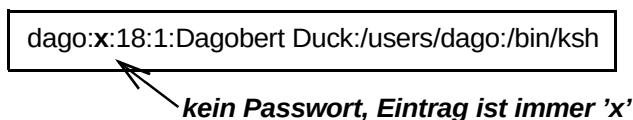
Abbildung 7.2, Teil a, zeigt als Beispiel einen Eintrag der klassischen UNIX-Passwortdatei `/etc/passwd`. Er enthält einige Informationen über einen Benutzer, insbesondere sein Passwort in verschlüsselter Form. Beim Einloggen wird das eingegebene Passwort ebenfalls verschlüsselt und die beiden verschlüsselten Wörter miteinander

a.) Passwortdatei `/etc/passwd` in klassischen UNIX-Systemen:



b.) Passwortdateien `/etc/passwd` und `/etc/shadow` in Linux:

`/etc/passwd` (öffentlich):



`/etc/shadow` (gesperrt):

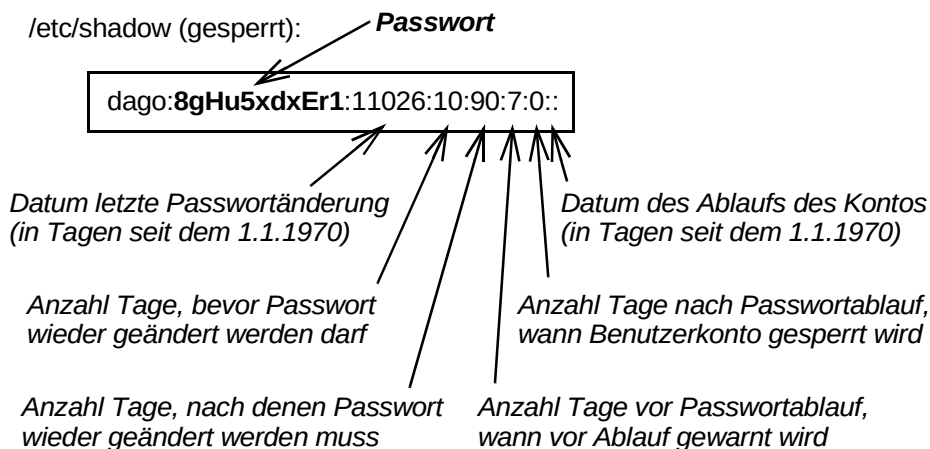


Abb. 7.2 Einträge der UNIX- und Linux-Passwortdateien

verglichen. Das ist fast so effizient wie ein Vergleich der Klartexte, da die Verschlüsselungsfunktion einfach zu berechnen ist, und hat zudem den Vorteil, dass die Datei nicht unmittelbar zur Schwachstelle wird: Ein Cracker kann mit dem dortigen Passworteintrag direkt nichts anfangen, da das Eintippen des verschlüsselten Passworts nichts nützt und eine Entschlüsselung einen unrealistisch hohen Aufwand erfordert.

Allerdings könnte ein Cracker versuchen, eine Reihe von möglichen Klartextpasswörtern jeweils zu verschlüsseln und sie mit den Passworteinträgen zu vergleichen (der so genannte „Wörterbuchangriff“). Um dies zu verhindern, speichern neuere UNIX-Versionen die Passworteinträge in einer Datei, auf die nur der Systemverwalter zugrei-

fen darf. Solaris und Linux beispielsweise führen hierfür die Datei `/etc/shadow`, die zudem Festlegungen und Informationen bezüglich der Gültigkeitsdauer von Passwörtern und Benutzerkonten enthält (siehe Abb. 7.2, Teil b).

Um das Erraten von Passwörtern zu erschweren, kann das System bestimmte Mindestanforderungen stellen, z.B. eine minimale Länge festsetzen und fordern, dass im Passwort mindestens eine Ziffer oder ein Sonderzeichen vorkommt. Zudem kann es die Benutzer zwingen, ihre Passwörter jeweils nach einer bestimmten Zeit zu ändern (siehe `shadow`-Datei). Der Benutzer sollte seinerseits bei der Wahl seiner Passwörter sehr vorsichtig sein – am sichersten sind hier Kunstwörter, die in keinem Wörterbuch zu finden sind und zudem mehrere Ziffern und / oder Sonderzeichen enthalten. Eine gute Methode ist hier, die Anfangsbuchstaben der Wörter einer leicht zu merkenden Wortfolge zu verwenden. Zum Beispiel liefert „Jim Knopf und die Wilde 13“ das Passwort „JKudW13“.

Trotz aller Vorgaben besteht bei der Verwendung von Passwörtern immer die Gefahr, dass zu einfache Wörter verwendet werden, die leicht zu erraten sind. Zudem notieren trotz aller Warnungen Benutzer gern ihre Passwörter, denn der menschlichen Merkfähigkeit sind Grenzen gesetzt – insbesondere bei zu komplizierten Wörtern oder wenn zu viele zu merken sind (man denke an die heute übliche Flut von Plastikkarten, von denen jede eine eigene PIN hat).

Besonders kritisch ist die Übertragung von Passwörtern über Datennetze, die heute noch vielfach im Klartext geschieht. Um hier einen Missbrauch nach Abhören zu verhindern, sollten Einmal-Passwörter (z.B. Transaktionsnummern TANs) verwendet werden, die nur für einen Vorgang Gültigkeit haben.

Eine Alternative zu Passwörtern, die allerdings in der Praxis kaum benutzt wird, ist ein Frage- und Antwortspiel: Das System stellt dem Benutzer eine Frage, dessen Antwort nur ihm bekannt ist (etwa von der Art: „Welchen Spitznamen hatte in der 9. Klasse der Mathelehrer?“). Grundlage ist hierfür ein Katalog aus Fragen und Antworten, die der Benutzer bei seiner ersten Anmeldung eingegeben hat.

7.2.3 Biometrische Authentifizierung

Bei wissensbasierten Verfahren besteht erstens die Gefahr, dass ein Cracker sich das Passwort eines Benutzers verschafft, und zweitens, dass ein Benutzer sein Passwort vergisst. Das System wird dann eine unberechtigte Person einlassen bzw. einen berechtigten Benutzer abweisen. Wünschenswert ist also ein Ansatz, bei dem besser gewährleistet ist, dass sich niemand den Authentifizierungsbeweis einer anderen Person verschaffen kann und dass jeder seinen eigenen Beweis stets zur Verfügung hat. Biometrische Verfahren, die sich auf körperliche Merkmale der Benutzer stützen, sind hierfür prinzipiell gut geeignet. Wir geben im Folgenden eine kurze Übersicht über die Forderungen an solche Verfahren sowie über verschiedene praktische Ansätze.

An das körperliche Merkmal, das durch ein biometrisches Authentifizierungsverfahren überprüft wird, werden nach [JaHP00] die folgenden allgemeinen Anforderungen gestellt:

- *Universalität*: Jede Person muss ein solches Merkmal besitzen.
- *Eindeutigkeit*: Keine zwei Personen dürfen in dem Merkmal übereinstimmen.
- *Permanenz*: Das Merkmal darf sich nicht im Laufe der Zeit von selbst verändern oder durch gezielte Manipulation verändern lassen.
- *Abfragbarkeit*: Das Merkmal muss leicht aufgenommen und quantifiziert werden können.

Damit ein entsprechendes Verfahren praktisch einsetzbar ist, muss darüber hinaus noch das Folgende gelten:

- *Zuverlässigkeit*: Das Verfahren muss das Merkmal einer Person mit der erforderlichen Genauigkeit analysieren können. Das bedeutet, dass es zwei Personen anhand ihrer Merkmale eindeutig unterscheiden können muss. Außerdem wird eine sehr geringe Fehlerrate verlangt: Einerseits dürfen keine unberechtigten Personen zugelassen werden, andererseits darf berechtigten Benutzern der Zugang nicht verweigert werden.
- *Leistung*: Das Verfahren muss schnell und mit möglichst wenig Ressourcen (z.B. Eingabegeräte, Speicher, Prozessorleistung) arbeiten.
- *Akzeptanz*: Die Anwendung des Verfahrens darf für Benutzer nicht unangenehm oder mit größerem Aufwand verbunden sein.
- *Resistenz gegen Betrugsversuche*: Das Verfahren darf nicht umgangen werden können, insbesondere nicht durch Präsentation eines gefälschten Merkmals.

Unter anderem können die folgenden Merkmale zur Authentifizierung dienen [JaHP00]:

- *Gesicht (Aussehen)*: Menschen erkennen einander insbesondere an ihren Gesichtern. Ein automatisiertes Authentifizierungsverfahren könnte dies auch versuchen, indem es die Lage und Form einiger typischer Gesichtsmarkmale (Augen, Nase, Lippen usw.) analysiert. Es ist jedoch noch offen, ob dies mit der erforderlichen Zuverlässigkeit möglich ist, zumal das Aussehen u.a. stark von der Beleuchtung und dem Blickwinkel abhängt.
- *Gesicht (Wärmeverteilung)*: Die Blutgefäße, die das Gesicht durchlaufen, geben Wärme durch die Haut ab und erzeugen so ein charakteristisches Wärmeverteilungsmuster, das durch eine Wärmebildkamera aufgenommen werden kann. Auch hier hängt das Merkmal stark vom Blickwinkel der Aufnahme ab. Zudem wird die Wärmeverteilung von veränderlichen Faktoren wie emotionalem Befinden und Körpertemperatur beeinflusst.

- *Auge (Retina)*: Der Augenhintergrund (die „Retina“) wird von Blutgefäßen durchzogen, die ein eindeutiges und stabiles Muster darstellen. Um dieses aufzunehmen, muss die zu prüfende Person allerdings eine bestimmte Haltung einnehmen, und es muss ein Lichtstrahl (möglicherweise Infrarot) ins Auge geleitet werden. Zudem ist die benötigte Hardware relativ aufwendig.
- *Auge (Iris)*: Die Iris, also der farbige Teil des Auges um die Pupille, zeigt ein charakteristisches Farbmuster, das nach bisherigen Erkenntnissen bei jedem Menschen eindeutig ist. Eine Aufnahme ist leichter möglich als bei der Retina. Zwar verlangen die heutzutage vorhandenen Aufnahmegeräte eine beträchtliche Kooperation des Benutzers, und sie sind teuer, es wird jedoch an benutzerfreundlicheren und kostengünstigeren Geräten gearbeitet.
- *Fingerabdruck*: Die Struktur der Hautleisten auf den Fingerkuppen ist bei den einzelnen Menschen verschieden, so dass die Untersuchung von Fingerabdrücken ein praktikables Authentifizierungsverfahren ist und bei Polizei und Justiz seit langem angewandt wird. Probleme liegen im Rechenaufwand, der Universalität, da ein (kleiner) Teil der Bevölkerung aus unterschiedlichen Gründen keine geeigneten Hautstrukturen hat, und möglicherweise bei der Akzeptanz, da das Abnehmen von Fingerabdrücken mit Kriminalität verbunden wird.
- *Geometrie der Hand*: Die Maße der menschlichen Hand, insbesondere ihre Gesamtform sowie Länge und Breite der Finger, sind ein individuelles Merkmal, das leicht abgefragt werden kann. Nachteilig ist allerdings, dass sich die Handformen zweier Personen nicht in jedem Fall so stark unterscheiden, dass eine eindeutige Unterscheidung möglich wäre. Zudem ändert sich die Form der Hand im Laufe des Lebens.
- *Unterschrift*: Eine Unterschrift wird im Geschäftsleben als Autorisation durch eine bestimmte Person akzeptiert. Bei der Prüfung der Unterschrift werden statische Merkmale überprüft (also ihre Form und evtl. Größe); daneben sind aber auch dynamische Merkmale (z.B. Druck und Geschwindigkeit beim Schreiben) wichtig. Für die Authentifizierung erscheint die Unterschrift aber als weniger gut geeignet, da sie auch bei derselben Person nie ganz gleich aussieht, sich im Laufe der Zeit verändert und zudem leicht gefälscht werden kann.
- *Stimme*: Jede Person besitzt ein individuelles Stimmuster, das sich hauptsächlich aus den physischen Merkmalen von Mund, Kehlkopf usw. ergibt. Die Unterscheidbarkeit zweier Menschen ist hier aber nicht groß genug, das Stimmuster kann sich im Laufe der Zeit ändern, und die Aufnahme einer Stimmprobe bei der Authentifizierung wird durch verschiedene Randbedingungen (z.B. Hintergrundgeräusche) stark beeinflusst.

In [Tan92] werden noch weitere Möglichkeiten angeführt, die aber entweder schmerzhaft oder unappetitlich sind und daher – Stichwort „Akzeptanz“ – für den praktischen Einsatz ausscheiden.

Die folgende Tabelle gibt an, wie gut die dargestellten Verfahren die anfangs genannten Forderungen erfüllen [JaHP00]:

	Univer- salität	Eindeu- tigkeit	Perma- nenz	Abfrag- barkeit	Zuverl. / Leistg.	Akzep- tanz	Resis- tenz
Gesichtsform	+	-	o	+	-	+	-
Gesichtswärme	+	+	-	+	o	+	+
Retina	+	+	o	-	+	-	+
Iris	+	+	+	o	+	-	+
Fingerabdruck	o	+	+	o	+	o	+
Handgeometrie	o	o	o	+	o	o	o
Unterschrift	-	-	-	+	-	+	-
Stimme	o	-	-	o	-	+	-

(+ = hoch, o = mittel, - = niedrig)

Biometrische Verfahren werden bereits heute in Zugangskontrollen für Gebäude und Gelände benutzt; eine Einführung im Bankwesen (insbesondere bei Geldautomaten) und elektronischen Handel ist im Gespräch. Die allgemeine Einsatz im Computerbereich setzt die Verbreitung von preiswerter und zuverlässiger Überprüfungshardware voraus.

7.3 Interne Sicherheit

Generell wird bei der Durchsetzung der internen Sicherheit vorausgesetzt, dass die externe Sicherheit gewährleistet ist, d.h. dass nur zugangsberechtigte, authentifizierte Benutzer im System aktiv sind. Diese Benutzer haben unterschiedliche Rechte, so z.B. unterschiedliche Zugriffsrechte für die einzelnen Dateien und unterschiedliche Ausführungsrechte für bestimmte Operationen oder Programme. Mechanismen der internen Sicherheit müssen diese Rechte durchsetzen: Sie müssen sicherstellen, dass einerseits kein Benutzer eine unberechtigte Aktion ausführen kann und andererseits keinem Benutzer eine berechtigte Aktion verwehrt wird.

In heutigen Systemen basiert die interne Sicherheit auf relativ grob strukturierten „Schutzringen“ und zusätzlich auf feineren Mechanismen wie „Zugriffskontrolllisten“ und „Capabilities“. Diese Techniken sollen im Folgenden dargestellt werden.

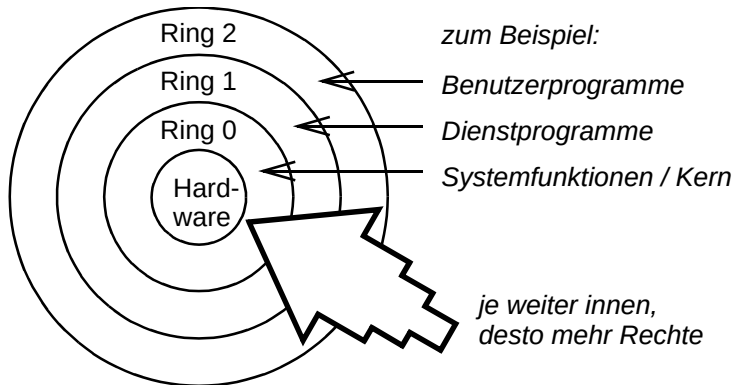


Abb. 7.3 Schutzringe

7.3.1 Schutzringe

Eine Möglichkeit zur Durchsetzung der internen Sicherheit sind die so genannten *Schutzringe* (engl. *rings of security*). Ausgangspunkt sind hier die Operationen des Systems, z.B. die Systemfunktionen des Betriebssystems, Dienstprogramme oder Benutzerprogramme. Diese Operationen werden in Sicherheitsklassen eingeteilt. Je höher die Sicherheitsklasse ist, desto mehr Rechte hat die Operation, z.B. Zugriffsrechte auf bestimmte Datenstrukturen des Betriebssystems oder Ausführungsrechte für bestimmte Maschinenbefehle. Den Sicherheitsklassen entsprechen verschiedene Modi, in denen der Prozessor einen Prozess ausführen kann.

Bildlich kann man sich die Sicherheitsklassen als eine Folge von Ringen vorstellen, die sich um die Hardware legen (siehe Abb. 7.3): Je näher sich ein Ring an der Hardware befindet, desto größer sind die Privilegien der Operationen in diesem Ring. UNIX / Linux verfügt (ebenso wie Windows NT / 2000) bekanntlich über nur zwei Ringe, die dem Kernel Mode für privilegierte Betriebssystemfunktionen und dem User Mode für Benutzerprogramme entsprechen. Intel-Prozessoren unterstützen hardwaremäßig vier Schutzringe (die von OS/2 voll ausgenutzt werden), und MULTICS, der UNIX-Vorläufer, hatte sogar 64 Sicherheitsklassen.

Der Wechsel eines Prozesses in einen höherprivilegierten Schutzring wird streng kontrolliert. Wie in Abschnitt 2.2.3 diskutiert, ist in UNIX ein Übergang nur durch ein Trap-Ereignis möglich, bei dem der Prozess durch das Setzen bestimmter Prozessorbits „hochgestuft“ wird und gleichzeitig in eine vorgegebene Funktion verzweigt, z.B. eine Systemfunktion des Kerns.

Schutzringe allein reichen zur Durchsetzung eines Sicherheitskonzepts nicht aus, da es nur recht wenige von ihnen gibt, demgegenüber aber viele Objekte existieren, die individuell geschützt werden müssen. Ein Extremfall ist UNIX / Linux mit seinen zwei Ringen, wo der Zugriff auf eine Vielzahl von Dateien kontrolliert werden muss. Daher sind zusätzlich objektspezifische *Zugriffskontrollen* erforderlich.

a.) Definition der Rechte mit einer Zugriffskontrollmatrix:

	Datei A	Datei B	Datei C
Prozess 1	-	R W	R X
Prozess 2	R W	R	-

(R = Read, W = Write, X = eXecute)

b.) Durchsetzung der Rechte:

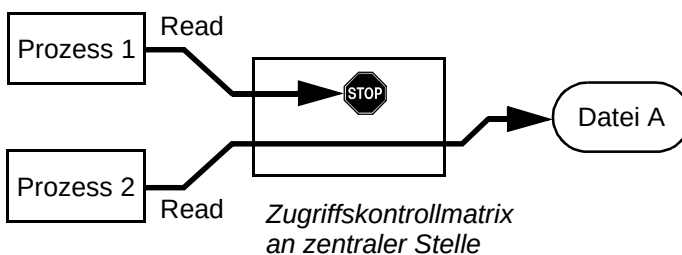


Abb. 7.4 Zugriffskontrollmatrix

7.3.2 Zugriffskontrollen

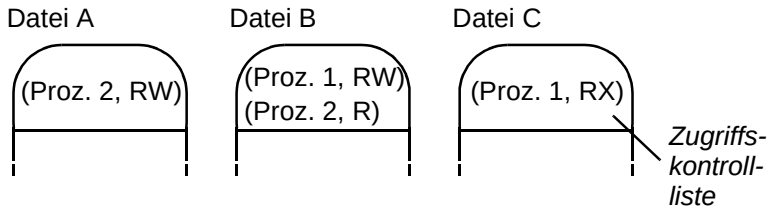
Die Kontrolle der Zugriffe auf einzelne Komponenten des Systems beruht auf dem Modell, das zu Anfang dieses Kapitels eingeführt wurde. Es enthält eine Menge S von Aktivitätsträgern oder Subjekten (z.B. Prozesse), eine Menge O von passiven Objekten (Betriebsmittel wie z.B. Speicherbereiche, Dateien oder Geräte) und für jedes Subjekt $s \in S$ und jedes Objekt $o \in O$ die „Zugriffsrechte“ von s an o , also die Menge der Operationen, die s auf o durchführen darf. Das System muss diese Zugriffsrechte durchsetzen, wozu insbesondere die effiziente Speicherung und Verwaltung eines Verzeichnisses für die Rechte gehört.

Zugriffskontrollmatrix

Allgemein lassen sich die Zugriffsrechte in einer *Zugriffskontrollmatrix* (engl. *access control matrix*) abspeichern, deren Zeilen den Subjekten und deren Spalten den Objekten entsprechen. Der Eintrag in Zeile s und Spalte o nennt die Operationen, die s auf o ausführen darf. Die Matrix wird an einer zentralen Stelle gespeichert und liefert bei Objektzugriffen die Information, ob der Zugriff zulässig ist. Abbildung 7.4 stellt als Beispiel eine Kontrollmatrix für ein System mit zwei Prozessen und drei Dateien dar.

Der Nachteil einer solchen Matrix ist, dass sie einerseits eine beträchtliche Größe hat, da es sehr viele Subjekte und Objekte gibt, aber andererseits recht „schwach besetzt“ ist, da in realen Systemen (fast) jedes Subjekt an den meisten Objekten keine Rechte hat. Daher sind effizientere Arten der Speicherung nötig.

a.) Definition der Rechte mit Zugriffskontrolllisten:



b.) Durchsetzung der Rechte:

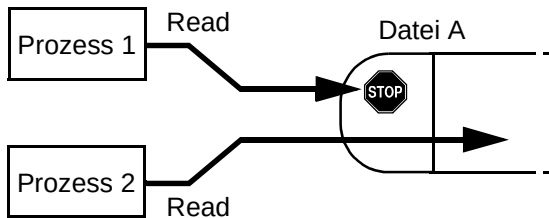


Abb. 7.5 Zugriffskontrolllisten (ACLs)

Zugriffskontrolllisten

Ein möglicher Ansatz ist die Benutzung von *Zugriffskontrolllisten* (engl. *access control lists, ACLs*), bei dem die Zugriffskontrollmatrix gewissermaßen spaltenweise abgespeichert wird: Für jedes Objekt wird eine Liste geführt, die angibt, welches Subjekt welche Operationen auf diesem Objekt ausführen darf. Bei jedem Zugriff wird die Berechtigung anhand der Liste überprüft. Da Subjekte, die keine Rechte an dem Objekt haben, in der Liste nicht aufgeführt werden, wird erheblich weniger Speicher benötigt als bei der Matrixdarstellung. Abbildung 7.5 gibt für die Dateien aus dem vorangehenden Beispiel die Zugriffskontrolllisten an.

Die Schutzbits für UNIX/Linux-Dateien entsprechen eingeschränkten Zugriffskontrolllisten, da sie die Rechte nicht für jeden Benutzer einzeln, sondern für Benutzergruppen festlegen. Windows NT / 2000 implementiert hier volle Zugriffskontrolllisten, in denen Rechte individuell festgelegt werden können und zudem nach mehr Zugriffsoperationen differenziert wird als bei UNIX. Auch Linux sieht in den Inodes seines ext2-Dateisystems Zugriffskontrolllisten vor, die aber momentan noch nicht genutzt werden.

Capabilities

Die Zugriffskontrollmatrix kann auch zeilenweise abgespeichert werden. Jeder Prozess besitzt dann eine Liste mit den Objekten, auf die er zugreifen darf, und seinen Rechten an diesen Objekten (siehe Abb. 7.6). Ein Eintrag in der Liste wird *Capability* (deutsch wörtlich „Fähigkeit“, sinngemäß „Berechtigung“) genannt.

Definition und Durchsetzung der Rechte mit Capabilities:

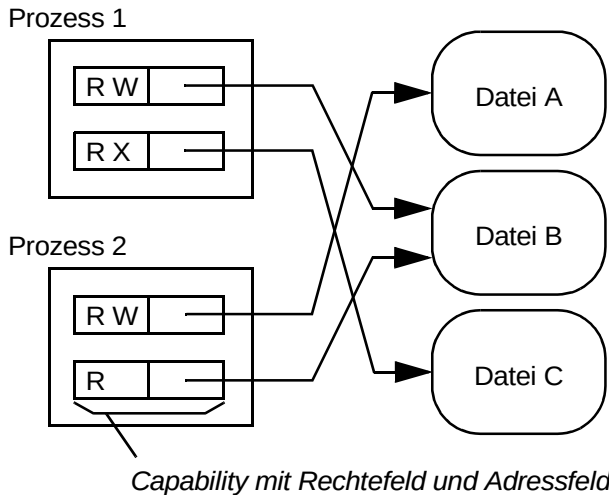


Abb. 7.6 Capabilities

Eine Capability enthält mindestens zwei Komponenten: Das *Objektfeld* oder *Adressfeld* besagt, auf welches Objekt sich die Capability bezieht und wo das Objekt zu finden ist, und das *Rechtefeld* gibt die zulässigen Operationen auf dem Objekt an. Das Rechtefeld ist im Allgemeinen als Bitfeld realisiert, das für jede mögliche Operation ein eigenes Bit enthält. Von den Bits sind genau diejenigen auf 1 gesetzt, die zulässigen Operationen entsprechen.

Eine Capability ist gewissermaßen ein Schlüssel für ein Objekt: Ein Objektzugriff ist nur möglich, wenn eine entsprechende Capability präsentiert wird. Dabei wird durch die Systemhardware überprüft, ob die erforderlichen Rechte vorliegen. Für ein Objekt können durchaus mehrere Capabilities existieren, die unterschiedliche Rechteeinträge haben und an verschiedenen Orten stehen können – z.B. bei mehreren Prozessen jeweils eine.

Fundamental für die Sicherheit ist der Schutz der Capabilities: Es muss sichergestellt werden, dass nur die Hardware und das Betriebssystem sicherheitskritische Operationen auf Capabilities ausführen dürfen, nicht jedoch benutzerdefinierter Programmcode. Sicherheitskritisch sind das Erzeugen einer neuen Capability (außer das exakte Kopieren einer bereits vorhandenen) und die Erweiterung der Rechte in einer Capability. Als nicht sicherheitskritisch wird das Kopieren einer Capability angesehen, die der Prozess bereits besitzt, sowie das Einschränken der Rechte in einer Capability. Nach [Tan92] gibt es insbesondere die folgenden Möglichkeiten, diese Forderungen durchzusetzen:

- In einer *Tagged Architecture* wird der Schutz der Capabilities durch Hardware sichergestellt: Hier hat jedes Wort des realen Speichers an einer festgelegten Position ein Bit, das genau dann den Wert 1 hat, wenn dort momentan eine Capability gespeichert ist. Das Setzen dieses Bits und die Änderung von Worten mit gesetztem Bit ist nur

möglich, wenn sich der Prozessor im privilegierten Ausführungsmodus befindet, also eine Funktion des Betriebssystems ausführt.

- Manche Betriebssysteme, wie beispielsweise Mach, speichern *Capability-Listen* für die einzelnen Prozesse in ihrem Kern und schützen sie so vor unberechtigten Manipulationen. Ein Benutzerprozess kann eine Capability nur indirekt durch Angabe eines Listenindex benutzen; die eigentlichen Operationen auf der Capability führt das Betriebssystem aus. Auch Windows NT implementiert für die Objekte seines Kerns ein solches Konzept.
- In anderen Systemen, wie dem experimentellen Betriebssystem *Amoeba*, werden Capabilities nur in *verschlüsselter Form* an die Benutzerprozesse ausgegeben. Bei einem Objektzugriff wird die Capability entschlüsselt und ihre Gültigkeit überprüft. Nur sehr wenige der hier möglichen Bitkombinationen sind gültige verschlüsselte Capabilities, so dass die Gefahr des unberechtigten „Zusammenbastelns“ einer Capability gering ist.

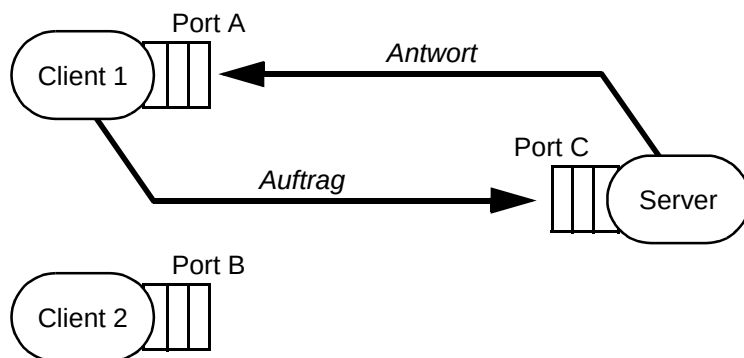
Capabilities ermöglichen eine dezentrale Führung der Schutzinformationen (wenn nicht gerade, wie in Mach, die Capabilities „künstlich“ an einer zentralen Stelle zusammengehalten werden): Jeder Prozess verwaltet seine Capabilities selbst, kann sie im Prinzip an beliebigen Stellen seines Adressraums speichern, Kopien mit gleichen oder eingeschränkten Rechten herstellen und an andere Prozesse weitergeben. Erzeugt beispielsweise ein Prozess ein neues Objekt, so übergibt ihm das Betriebssystem eine zugehörige Capability, die beliebige Zugriffe auf dieses Objekt erlaubt. Der Prozess kann dann, durch Weitergabe von Kopien, selbst bestimmen, wer wie auf das Objekt zugreifen darf.

Diese große Flexibilität hat natürlich auch Nachteile. Es kann nämlich nicht zentral kontrolliert werden, welche Prozesse an welcher Stelle Capabilities für ein bestimmtes Objekt gespeichert haben. Damit ist beispielsweise die Rücknahme von einmal gewährten Rechten schwierig. Außerdem wird nicht ohne weiteres bemerkt, wenn für ein Objekt keine Capabilities mehr existieren und es somit gelöscht werden kann.

Für diese Probleme gibt es allerdings Lösungen. Beispielsweise kann man Capabilities mit Versionsnummern versehen und zum Rückruf von Rechten jeweils Capabilities mit einer neuen Nummer ausgeben, wodurch die alten Capabilities ungültig werden. Alternativ kann man lediglich Capabilities für ein zentrales „Indirektionsobjekt“ ausgeben, das seinerseits einen Zeiger auf das eigentliche Objekt enthält (ähnlich wie ein zentraler Schlüsselkasten). Rechte werden entzogen, indem der Zeiger im Indirektionsobjekt gelöscht wird, so dass die Capabilities der Prozesse dann „ins Leere“ verweisen. Nachteilig ist in beiden Fällen aber, dass Rechte an einem Objekt nicht einzeln, sondern nur alle auf einmal zurückgenommen werden können.

Um zu erkennen, wann ein Objekt gelöscht werden kann, kann für das Objekt ein Referenzzähler geführt werden. Er wird beim Kopieren einer zugehörigen Capability um 1 erhöht und beim Löschen einer Capability um 1 gesenkt. Erreicht er den Wert 0, so kann auch das Objekt gelöscht werden.

Prozesse mit Ports:



Betriebssystem-Kern:

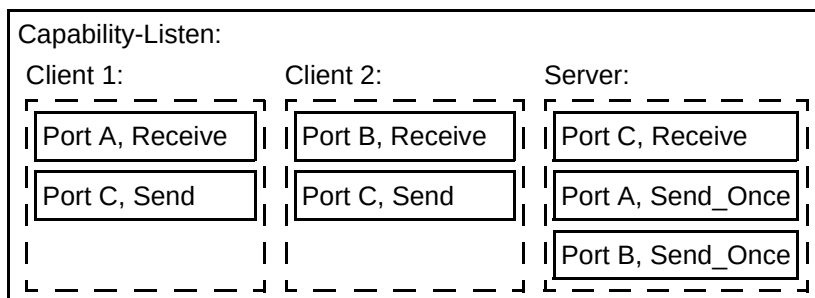


Abb. 7.7 Capabilities im Betriebssystem „Mach“

Capabilities in Mach

Ein Betriebssystem, das sich stark auf Capabilities stützt, ist *Mach*. Mach verwendet Capability-geschützte Ports zur Prozesskommunikation und unterstützt damit Client-Server-Konzepte, die insbesondere für den Einsatz in Multiprozessorsystemen und verteilten Systemen gedacht waren. Mach selbst hat es nie über ein Nischendasein hinaus gebracht, hatte aber Einfluss auf die Entwicklung weiterer Betriebssysteme, unter anderem Windows NT. Zudem lohnt es sich wegen seiner interessanten Architektur, sich zumindest ein wenig mit Mach zu befassen (siehe auch Kapitel 9):

Jeder Mach-Prozess besitzt eine Liste mit Capabilities für alle Ports, auf die er zugreifen darf. Die Capability-Listen liegen im Kern und werden durch diesen verwaltet (siehe Abb. 7.7). In einer Capability können die folgenden drei Rechte definiert werden:

- *Receive*: Recht, Nachrichten aus dem Port zu empfangen. Für einen Port darf nur jeweils *ein* Prozess das Empfangsrecht besitzen, wie es ja allgemein für Ports gilt.

- *Send*: Recht, Nachrichten in den Port zu senden. Für einen Port dürfen beliebig viele Prozesse das Senderecht besitzen.
- *Send-Once*: Recht, eine einzige Nachricht in den Port zu senden. Dieses Recht wird Servern zugestanden, um nach der Auftragsbearbeitung eine Antwort in einen Client-Port zu schicken.

Abbildung 7.7 illustriert die Situation, nachdem zwei Clients jeweils einen Auftrag an einen Server übergeben haben: Die Clients besitzen jeweils das Senderecht für den Auftrags-Port des Servers sowie Empfangsrechte für ihre jeweiligen Antwort-Ports. Der Server darf Nachrichten aus seinem Auftrags-Port empfangen sowie jeweils genau eine Antwort in die Antwort-Ports der Clients senden.

Capability-Listen und Ports sind eng an die Prozesse gebunden. Terminiert ein Prozess, so werden seine Capability-Liste gelöscht und ebenso alle Ports, für die er ein Empfangsrecht besitzt. Zudem werden sämtliche Capabilities für diese Ports, die sich in anderen Listen befinden, entfernt.

7.4 Weitere Gebiete

Aus Platzgründen können hier nicht alle Aspekte und Methoden der Systemsicherheit im Detail behandelt werden. Es sollen aber noch zwei wichtige Punkte angerissen werden, nämlich die Kryptologie sowie Gefahren durch Viren und ähnliche Software.

7.4.1 Kryptologie

Die *Kryptologie* befasst sich mit der Verschlüsselung von Daten. Ihr Ziel ist, dass ein Eindringling selbst dann keine Informationen erhalten kann, wenn er auf einen Datenträger zugreift oder eine Datenübertragung abhört. Wir beschränken uns hier auf eine sehr grobe Übersicht über das Gebiet. Eine detaillierte Darstellung kryptologischer Verfahren findet sich beispielsweise in [Bau95] – zusammen mit vielen amüsanten Anekdoten aus gar nicht so amüsanten (Kriegs-)zeiten.

Grundlegendes Modell

Abbildung 7.8 skizziert das grundlegende Modell der Kryptologie: Ein *Sender* „verschlüsselt“ einen *Klartext*, indem er daraus mit Hilfe einer *Verschlüsselungsfunktion* einen *Kryptotext* berechnet (englische Begriffe: *plaintext*, *encryption function* bzw. *cyphertext*). Er schickt diesen Kryptotext dann über einen *Kanal* an den *Empfänger*, der ihn „entschlüsselt“, d.h. daraus mit Hilfe einer *Entschlüsselungsfunktion* (engl. *decryption function*) den Klartext zurückberechnet. Ein *Spion* (engl. *eavesdropper*), der die Übertragung abhört, erhält nur den Kryptotext, den er nicht verstehen kann.

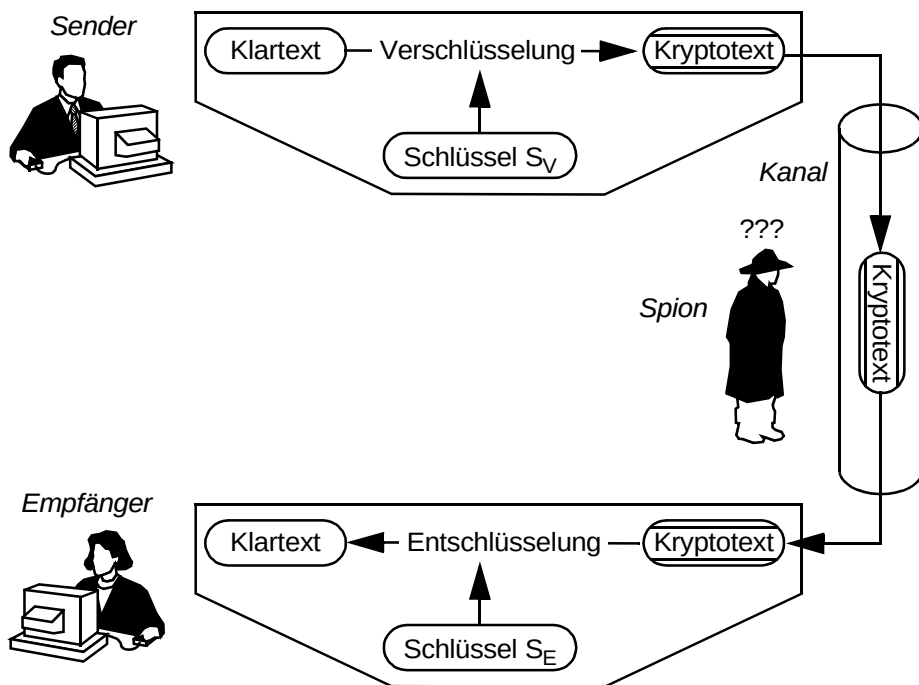


Abb. 7.8 grundlegendes Modell der Kryptologie

Der Begriff „Kanal“ ist dabei sehr allgemein zu verstehen: Es kann sich dabei nicht nur um eine reale Kommunikationsverbindung handeln, sondern beispielsweise auch um eine Magnetplatte, auf der Daten gespeichert sind.

Zur Ver- und Entschlüsselung werden im Allgemeinen *Schlüssel* (engl. *keys*) benutzt. Grob gesprochen, sind Schlüssel (S_V bzw. S_E in der Abbildung) Parameter für die Ver- und Entschlüsselungsfunktionen und bestimmen so die Datentransformation wesentlich mit. Es wird gefordert, dass Nachrichten effizient ver- bzw. entschlüsselt werden können, wenn der oder die Schlüssel bekannt sind, dass aber die Entschlüsselung (praktisch) unmöglich ist, wenn der entsprechende Schlüssel nicht bekannt ist. Die Funktionen selbst müssen also nicht geheim gehalten werden – lediglich der oder die Schlüssel.

Allgemein unterscheidet man zwei Klassen von Verschlüsselungsalgorithmen, nämlich *symmetrische* und *asymmetrische Verfahren*.

Symmetrische Verfahren

Bei symmetrischen Verfahren (*Verfahren mit geheimem Schlüssel*, engl. *symmetric encryption schemes* bzw. *private key encryption*) wird für die Ver- und die Entschlüsselung derselbe Schlüssel benutzt; es ist also $S_V = S_E$. Nachteilig ist hierbei, dass der Schlüssel streng vertraulich bleiben muss und insbesondere ein sicherer, nicht abhörbarer Kanal benötigt wird, um ihn zwischen Sender und Empfänger zu übertragen.

Klassische Beispiele für symmetrische Verfahren sind *IDEA (International Data Encryption Algorithm)* und insbesondere der im kommerziellen Umfeld sehr weit verbreitete *DES (Data Encryption Standard)*. DES wurde von IBM in den siebziger Jahren entwickelt und ist seit 1977 US-Standard. Heutzutage ist Standard-DES wegen seiner kurzen Schlüssel von 56 Bits nicht mehr sicher („If your secret is worth more than 8 cents, don't encrypt it with DES.“ [WeLu99]): Mit heutiger Hardware lassen sich alle möglichen Schlüssel durchprobieren und somit codierte Texte relativ rasch entschlüsseln.

Um solchen Angriffen zu begegnen, wird DES in der Praxis bisweilen dreimal hintereinander auf den Klartext bzw. die daraus resultierenden Kryptotexte angewandt („Triple-DES“). Das erhöht zwar die Sicherheit, verlängert aber die Zeit zur Verschlüsselung einer Nachricht erheblich. IDEA benutzt übrigens 128-Bit-Schlüssel.

Wegen der kritischen Schwäche von DES wurde Ende der neunziger Jahre nach einem Nachfolgeverfahren gesucht, um es dann unter dem Namen *AES (Advanced Encryption Standard)* zu standardisieren [NIST01]. In einem Wettbewerb, in dem übrigens das Verfahren *MAGENTA* (!) eines großen deutschen Telekommunikationsunternehmens bezüglich Sicherheit und Laufzeit sehr schwach abschnitt [WeLu99], setzte sich das *Rijndael*-Verfahren der belgischen Forscher J. Daemen und V. Rijmen durch.

Das neue Standardverfahren AES benutzt Schlüssel der Längen 128, 192 und 256 Bits und ist damit erheblich sicherer als DES. Ein weiterer Vorteil von AES ist, dass es in einem öffentlichen Verfahren ermittelt wurde, während der Entwurfsprozess von DES geheim gehalten und zudem von der NSA (National Security Agency der USA) beeinflusst wurde. So konnte der Verdacht nie ausgeräumt werden, dass DES geheime „Hintertüren“ enthält, die den Entwicklern das Knacken von Nachrichten erleichtern.

Asymmetrische Verfahren

Asymmetrische Verfahren (*Verfahren mit öffentlichen Schlüsseln*, engl. *asymmetric encryption schemes* bzw. *public key encryption*) benutzen für die Ver- und Entschlüsselung zwei verschiedene Schlüssel; es ist also $S_V \neq S_E$. S_E kann aus S_V nicht berechnet werden (in der Praxis heißt das: nur mit unvertretbar hohem Aufwand). Vor der Datenübertragung wählt der Empfänger ein geeignetes Paar solcher Schlüssel aus, gibt S_V öffentlich bekannt und hält S_E geheim. Ein (beliebiger) Sender kann nun S_V zur Verschlüsselung seiner Nachricht benutzen, die nur durch den Empfänger entschlüsselt werden kann, da nur er S_E kennt.

Asymmetrische Verfahren können auch verwendet werden, um den Absender und die Nachricht zu authentisieren (Stichwort *digitale Unterschrift* oder *digitale Signatur*; engl. *digital signature*). Der Absender berechnet zunächst aus seiner Nachricht einen so genannten *Digest* oder *Hash*, d.h. ein für sie charakteristisches Bitmuster. Er kann hierfür ein Standardverfahren wie z.B. *MD5 (Message Digest No. 5)* oder *SHA (Secure Hash Algorithm)* benutzen. Anschließend verschlüsselt er den Digest mit seinem eigenen geheimen Schlüssel und schickt das Resultat zusammen mit der Nachricht an den Empfänger. Dieser entschlüsselt den Digest mit dem öffentlichen Schlüssel des Absenders und berechnet außerdem den Digest der empfangenen Nachricht. Stimmen beide Resultate überein, so ist offensichtlich der Absender der, als der er sich ausgibt, und die Nachricht ist unverfälscht.

Kritisch ist bei asymmetrischen Verfahren natürlich die Echtheit der öffentlichen Schlüssel: Es darf niemand anders, also kein Spion, unter dem Namen des Empfängers einen Schlüssel S_V bekannt geben, da er dann Nachrichten an den Empfänger mit seinem eigenen zugehörigen Schlüssel S_E entschlüsseln könnte. Um solchen Angriffen zu begegnen, werden zentrale vertrauenswürdige Authentifizierungsstellen (*Trust Center*) eingesetzt, die die Echtheit von Schlüsseln bestätigen. Ein weiteres Problem bei asymmetrischen Verfahren ist, dass sie komplexer und somit aufwendiger zu implementieren und auszuführen sind als symmetrische Verfahren – die Laufzeiten unterscheiden sich in der Praxis um den Faktor 1000 oder mehr!

Asymmetrische Verfahren wurden erstmals in den siebziger Jahren entwickelt, so z.B. 1978 das *RSA-Verfahren*, das nach seinen Erfindern R. L. Rivest, A. Shamir und L. Adleman benannt ist. Die Sicherheit von RSA basiert darauf, dass es zwar relativ einfach ist, zwei große Primzahlen zu finden und miteinander zu multiplizieren, dass es aber sehr schwierig (d.h. für praktische Zwecke unmöglich) ist, das resultierende Produkt ohne Kenntnis eines Faktors wieder aufzuspalten. Bei RSA wählt jeder Teilnehmer zunächst zwei große Primzahlen p und q und berechnet ihr Produkt $n=p*q$. „Groß“ bedeutet, dass die Binärdarstellung von n 768, 1024 oder sogar 2048 Bit lang ist [RePo99]. Anschließend ermittelt er aus p , q , und n geeignete Werte für S_V und S_E und gibt S_V und n bekannt. Die Verschlüsselung geschieht mit Hilfe von S_V und n , die Entschlüsselung mit S_E und n (Details siehe z.B. [Bau95] oder [RePo99]). Ein Spion kennt zwar S_V und insbesondere n , könnte den Code aber nur knacken, wenn ihm auch die Primfaktoren p und q bekannt wären.

PGP

Das bei Privatanwendern beliebte *PGP (Pretty Good Privacy)*-Programm realisiert eine hybride Verschlüsselung (also eine Mischform), die standardisierte Verfahren wie MD5, RSA und IDEA kombiniert: Ein Paar aus öffentlichem und geheimem Schlüssel S_V und S_E wird zur Ver- bzw. Entschlüsselung eines weiteren Schlüssels S' verwendet. S' steuert ein symmetrisches Verfahren, mit dem dann die eigentliche Nachricht verschlüsselt und entschlüsselt wird. Eine asymmetrische Verschlüsselung auch der Nachricht selbst würde zu viel Zeit kosten (siehe oben).

Die Vorgehensweise ist im Einzelnen wie folgt: Der Absender erzeugt zunächst einen zufälligen Schlüssel S' und verschlüsselt damit anhand des symmetrischen Verfahrens seine Nachricht. Anschließend verschlüsselt er mit S_V anhand des asymmetrischen Verfahrens S' zu S'' und verschickt die verschlüsselte Nachricht zusammen mit S'' an den Empfänger. Dieser entschlüsselt zunächst S'' zu S' , wobei er das asymmetrische Verfahren mit S_E verwendet. Danach entschlüsselt er die eigentliche Nachricht mit S' anhand des symmetrischen Verfahrens.

Neben der Verschlüsselung selbst ermöglicht PGP die Signierung und entsprechende Prüfung von Nachrichten und komprimiert zudem die übertragenen Daten. Zur Verwaltung von Schlüsselsammlungen werden so genannte „Key Rings“ (entsprechend den Ringen von Schlüsselbunden) bereitgestellt.

Mehr zu PGP findet man beispielsweise in [Tan96] oder unter [PGP01].

Steganographie

Im Zusammenhang mit der Verschlüsselung sind *steganographische Verfahren* interessant, bei denen Geheimdaten in einer großen Menge offener, „harmloser“ Daten versteckt werden. Ein Beispiel ist die Speicherung und Übertragung von Informationen in Bilddateien: Der Wert jedes Punkts („Pixels“) eines digitalisierten Bilds wird bei hoher Farbauflösung durch drei Byte codiert, die seinen Rot-, Grün- und Blauanteil angeben. Man könnte hier die geringstwertigen Bits der Blau-Bytes mit den Bits der zu versteckenden Nachricht überschreiben – eine Änderung, die bei der Betrachtung des Bildes nicht vom normalen Rauschen zu unterscheiden ist. Bei einer Bildauflösung von beispielsweise 768 mal 1024 Pixeln ergibt sich so ein „Stauraum“ von immerhin 96 KByte.

Für einen Überwacher wird es damit sehr schwierig herauszufinden, ob und wo Geheiminformationen vorhanden sind. Damit laufen übrigens auch staatliche Bemühungen ins Leere, durch den Zwang zur Hinterlegung von Schlüsseln Zugriff auf verschlüsselte Daten zu bekommen: Kriminelle werden zwar brav Schlüssel anmelden, dann aber ihre geheimen Daten steganographisch verstecken und so an Überwachungsinstitutionen vorbeischleusen.

7.4.2 Viren und Würmer

Im Zuge der fortschreitenden Vernetzung von Computern nimmt die Bedrohung durch bössartige Software immer stärker zu. Zu dieser Software gehören Viren, Würmer und Trojanische Pferde.

Viren

Ein *Virus* ist Programmstück, das sich in einem anderen, an sich harmlosen Programm befindet und bei dessen Ablauf ausgeführt wird. Es kopiert sich dann in andere Programme hinein, vervielfältigt sich also, und führt zugleich schädliche Operationen aus.

Im Prinzip ist der Code eines Virus folgendermaßen aufgebaut: Ein *Erkennungsteil* stellt fest, ob ein anderes Programm bereits „infiziert“ ist, d.h. das Virus bereits enthält. Nur nichtinfizierte Programme werden befallen. Ein *Infektionsteil* kopiert das Virus in das zu befallende Programm hinein, und zwar meist an den Anfang (oder zumindest mit einem Sprungbefehl vom Anfang aus), um beim Aufruf des Programms sofort aktiv zu werden. Der *Funktionsteil* führt eine schädliche Funktion aus, wie z.B. das Löschen von Daten, was entweder sofort oder erst nach einer längeren Zeitdauer oder bei einem bestimmten Ereignis geschehen kann.

Früher befielen Viren nur Dateien mit Maschinencode, nicht jedoch Dateien mit Daten. Leider ist jedoch in den letzten Jahren die Trennlinie zwischen Programmen und Daten immer unschärfer geworden. So enthalten z.B. MS-Office-Dokumente Makros, die beim Öffnen der Datei ausgeführt werden. Ein immer „beliebterer“ Angriff ist, Dokumente mit Makroviren zu verseuchen und per Mail zu verschicken. Liest der Empfänger die Mail, wird das Virus aktiv. Die heutigen Viren sind in ihrer Mehrzahl solche Makroviren.

Um sich vor Viren zu schützen, sollte man die folgenden Punkte beachten:

- Es sollte möglichst nur „vertrauenswürdige“ Software installiert und ausgeführt werden. Einen gewissen Vertrauensschutz bieten originalverpackte Produkte eines Herstellers, obwohl auch hier keine hundertprozentige Virenfreiheit garantiert ist.
- Auf andere Programmdateien – insbesondere aus dem Internet heruntergeladene – sollten *Virencanner* (spezielle Programme zur Virenerkennung) angewandt werden. Sie können allerdings nur nach bereits bekannten Viren suchen und sind somit bei neuen Viren wenig wirksam; eine Aktualisierung in kurzen Zeitabständen ist hier also wichtig ([Nach97] skizziert zwar Möglichkeiten, auch mit unbekannten Viren umzugehen, die aber recht zeitaufwendig sind).
- Um die Ausbreitung von Viren zu behindern, sollten Verzeichnisse mit Programmdateien für Schreibzugriffe gesperrt werden. Es ist allerdings nicht gesagt, dass das Virus diesen Schreibschutz nicht wieder aufhebt.
- Verdächtig aussehende Mails sollten nicht geöffnet, sondern sofort gelöscht werden.
- Schließlich sollte in kurzen Zeitabständen eine Datensicherung erfolgen, um das System im Notfall ohne größere Verluste neu installieren zu können.

Würmer

Würmer können sich wie Viren selbst kopieren und sich dabei auch über Kommunikationsnetze in andere Rechner fortpflanzen. Im Unterschied zu Viren sind sie eigenständige Programme. Von der Grundidee her waren Würmer für nützliche Tätigkeiten vorgesehen, wie z.B. zur Netzwartung, können aber auch für bösartige Zwecke verwendet werden (man beachte die Parallelen zur Kernspaltung, bei der eine Kettenreaktion ebenfalls zu schädlichen Resultaten führen kann). Ein klassisches Paradebeispiel ist hier der *Internet-Wurm* aus dem Jahre 1988, der Tausende von Rechnern durch Prozessorüberlastung in die Knie zwang. Der Täter, ein Informatikstudent und Sohn eines bekannten Experten für Computer-Sicherheit, wurde gefasst und zu einer hohen Strafe verurteilt.

Trojanische Pferde

Neben Viren und Würmern gibt es noch *Trojanische Pferde*. Das sind Programme, aus deren Namen und Beschreibung der arglose Benutzer vermuten muss, dass sie eine nützliche Funktion erbringen. Beim Aufruf des Programms wird aber nicht nur diese Funktion ausgeführt (wenn überhaupt), sondern es finden noch weitere, schädliche Operationen statt.

Ein Beispiel hierfür ist der Angriff auf T-Online-Nutzer vom Frühjahr 1998. Hier boten zwei Cracker im Internet kostenlose Software an, die dann auf den Benutzerrechnern Passwörter, die auf der Festplatte abgespeichert waren, ausspähte und an die Cra-

cker zurückmeldete. Die Benutzer traf hierbei natürlich eine Mitschuld, denn man sollte keine Passwörter unverschlüsselt auf der Platte ablegen.

Als Trojanische Pferde bezeichnet man auch Programmstücke, die im System „schlafen“, bis ein bestimmtes Passwort eingegeben wird oder ein bestimmtes Ereignis eintritt. Solche Programme werden bereits bei der Entwicklung oder bei der Wartung des Systems „eingepflanzt“ und bei „Bedarf“ (z.B. wenn der Programmierer seine Firma im Unfrieden verlassen hat) aktiviert.

7.5 Übungsaufgaben

1. Wissensfragen

- a.) Was ist der Unterschied zwischen externer und interner Sicherheit?
- b.) Wozu dient eine Authentifizierung? Welche Möglichkeiten gibt es dafür?
- c.) Welches Modell kann zur Beschreibung von Forderungen der internen Sicherheit benutzt werden? Welche drei prinzipiellen Möglichkeiten gibt es zu ihrer Durchsetzung?
- d.) Was ist der prinzipielle Unterschied zwischen symmetrischer und asymmetrischer Verschlüsselung?
- e.) Warum ist das Verschlüsselungsverfahren DES nicht mehr sicher?
- f.) Was ist ein Trojanisches Pferd?

2. Methoden der internen Sicherheit I

Betrachten Sie ein System mit einer „Fließbandverarbeitung“, bei der die Datenübergabe über Ports stattfindet: Prozess 1 entnimmt Eingabedaten aus Port A, verarbeitet sie und fügt entsprechende Zwischenergebnisse in Port B ein. Prozess 2 entnimmt diese Zwischenergebnisse aus Port B, verarbeitet sie weiter und fügt die Endergebnisse in Port C ein. Prozess 3, ein Überwachungsprozess, darf zwar die Inhalte sämtlicher Ports betrachten, aber nichts aus den Ports entnehmen.

- a.) Was sind in dem oben beschriebenen System die Subjekte, was sind die Objekte? Was für drei Operationen sind auf den Objekten *prinzipiell* möglich?
- b.) Welche Rechte müssen die Subjekte an den Objekten *mindestens* haben, damit das System funktioniert? Geben Sie Ihre Antwort auf drei verschiedene Arten, nämlich durch eine Zugriffskontrollmatrix, mit Zugriffskontrolllisten sowie mit Capabilities.

3. Methoden der internen Sicherheit II

Gegeben ist die folgende Zugriffskontrollmatrix (R = Lesen, W = Schreiben):

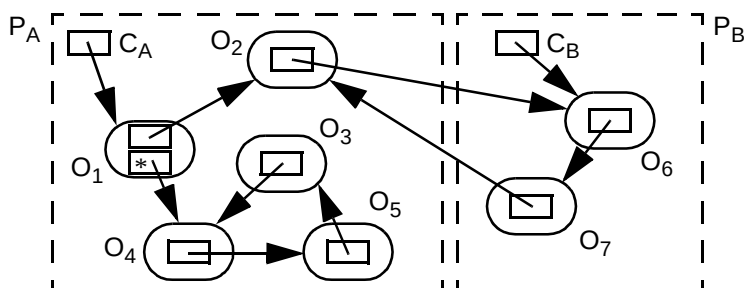
	Port A	Port B	Port C	Port D
P ₁	-	-	-	R
P ₂	W	-	R	W
P ₃	R	W	W	W
P ₄	W	R	-	W

Die Matrix stellt die Rechte eines Client-Server-Systems dar, in dem zwei Clients und ein Server vorhanden sind. Clients schicken an den Server Aufträge und erhalten Antworten. Darüber hinaus gibt einen vierten Prozess („Log-Prozess“), an den Meldungen geschickt werden können; er zeichnet diese auf, ohne eine Rückmeldung zu geben. Alle Prozesse kommunizieren über Ports.

- Welchem Prozess „gehört“ welcher Port, und welcher Prozess hat welche Funktion? Zeichnen Sie als Antwort eine Skizze, aus der die Zuordnung der Ports zu den Prozessen ersichtlich wird und in der die möglichen Nachrichtenflüsse durch Pfeile angegeben sind.
- Stellen Sie die gegebenen Rechte durch Zugriffskontrolllisten und Capabilities dar.

4. Capabilities

Zugriffsrechte auf Objekte können mit Capabilities realisiert werden. Da Capabilities ihrerseits in Objekten gespeichert werden können, ergibt sich somit als Zugriffsstruktur ein Graph. In diesem Graphen verläuft eine Kante von einem Objekt O_i zu einem Objekt O_j genau dann, wenn in O_i eine Capability für O_j gespeichert ist. Ein solcher Graph ist hier dargestellt (wobei die genauen Zugriffsrechte nicht angegeben sind, da sie hier nicht interessieren):



In dem hier beschriebenen System gibt es zwei Prozesse, P_A und P_B . P_A besitzt die Objekte O_1 bis O_5 , P_B die Objekte O_6 und O_7 . Da die Prozesse irgendwann einmal dem jeweils anderen Prozess eine Capability für eines ihrer Objekte übergeben haben, können sie auch auf Objekte des anderen Prozesses zugreifen.

Jeder Prozess besitzt eine „Basis-Capability“ C_A bzw. C_B . Er kann auf ein Objekt O nur zugreifen, indem er von seiner Basis-Capability ausgeht und versucht, einen Pfad zu diesem Objekt zu finden. Auf andere Objekte kann er nicht zugreifen.

- a.) P_A darf momentan auf O_6 zugreifen. Wie könnte P_B ihm dieses Recht entziehen, ohne dabei auf die Objekte von P_A zuzugreifen?
- b.) Was ergibt sich für eine Situation bezüglich der Objekte O_3 , O_4 und O_5 , wenn die mit * bezeichnete Capability gelöscht würde? Was ist allgemein das Problem, wenn immer wieder neue Objekte erzeugt (und dabei Hauptspeicherbereiche belegt) würden, die anschließend stets in dieselbe Situation wie O_3 , O_4 und O_5 geraten?
- c.) Wie könnte das Betriebssystem vorgehen, um mit der in Aufgabe b.) auftretenden Situation fertig zu werden? Nützen „Referenzzähler“ etwas, d.h. Zähler, die angeben, wie viele Capabilities für ein bestimmtes Objekt noch vorhanden sind?

5. Kryptographie: zwei primitive Verfahren

Versuchen Sie, die beiden folgenden Kryptotexte zu entschlüsseln:

- EJFTFS DPEF LBOO TFIS MFJDIU HFLOBDLU XFSEFO.
- UN AIEK NULO GIUY EUNNUOU GUOSMLOUV
MYN HIUNU UIVSMTLUV LIUO.

Es wurden zwei unterschiedliche Verschlüsselungsverfahren benutzt. Welche?

8 Verteilte Systeme: Grundlagen

Alle Betrachtungen in den Kapiteln 1 bis 7 bezogen sich im Wesentlichen auf ein lokales Ein-Prozessor-System, also ein System, in dem alle Aktivitäten um einen einzigen Prozessor konkurrieren und in dem alle Betriebsmittel zentral verwaltet werden. Problematisch ist hierbei, dass der Einzelprozessor zu einem Engpass des Systems werden kann. Zudem sind viele Anwendungen von Natur aus dezentral – man denke beispielsweise an ein Buchungssystem für Reisebüros oder ein Kontenführungsprogramm für Bankfilialen. Es ist also sinnvoll, echt nebenläufige Systeme mit mehreren Prozessoren einzuführen, bei denen die Prozessoren und andere Betriebsmittel räumlich verteilt, also direkt bei den Anwendern angesiedelt sind.

Im Unterschied zu einer „eng gekoppelten“ lokalen Hardware ist hier die Hardware „lose gekoppelt“: Die Funktionseinheiten liegen mehr oder weniger weit auseinander, und die Kommunikation zwischen ihnen verläuft über ein Datenkommunikationsnetz, das im Vergleich zum lokalen Bus schmalbandig und langsam ist. Die Probleme, zu denen diese lose Kopplung führt, sollten aber dem Programmierer und Benutzer gegenüber so weit wie möglich verborgen bleiben. Es muss also Systemsoftware bereitgestellt werden, die auf Basis der lose gekoppelten Hardware ein virtuelles eng gekoppeltes System realisiert – d.h. die dem Programmierer und Anwender vorspiegelt, dass er auf einer eng gekoppelten Hardware arbeitet. Man spricht in diesem Zusammenhang auch von der Realisierung eines *verteilten Systems*.

In diesem Kapitel sollen die Grundlagen solcher verteilter Systeme besprochen werden: Es werden zunächst die Begriffe der engen und losen Kopplung näher beleuchtet und die Vorteile und Probleme verteilter Systeme aufgezeigt. Anschließend wird eine Übersicht über die wichtigsten Aspekte der Datenkommunikation gegeben. Das nachfolgende Kapitel befasst sich dann mit Betriebssystemtechniken zur Realisierung von Prozessen, Dateisystemen usw., die auf dieser Basis eingesetzt werden.

8.1 Eng und lose gekoppelte Systeme

In Systemen mit mehreren Prozessoren spielt die Nebenläufigkeit eine noch stärkere Rolle als bei einem Ein-Prozessor-System. Die Art der Nebenläufigkeit hängt allgemein davon ab, wie viele Prozessoren vorhanden sind und wie eng sie miteinander gekoppelt sind. Wir stellen daher zunächst die verschiedenen Typen der Nebenläufigkeit einander gegenüber und erörtern ihren Zusammenhang mit der Art der Hardwarekopplung. Anschließend übertragen wir den Begriff der Kopplung auf die Softwareebene

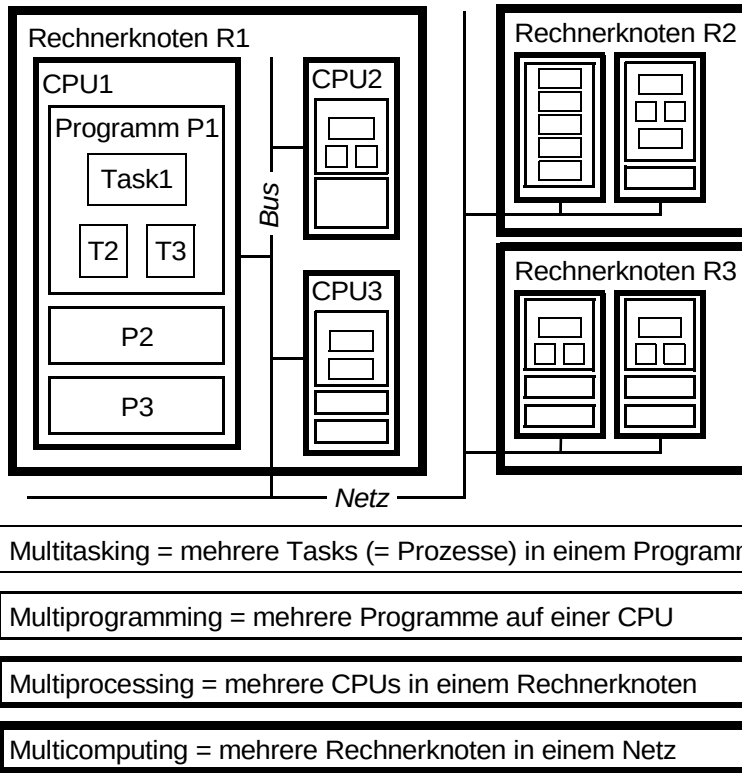


Abb. 8.1 Arten der Nebenläufigkeit

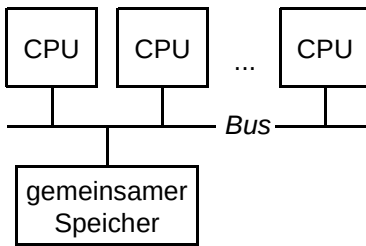
und erörtern, welchen Rolle das Betriebssystem in diesem Zusammenhang spielt, welche Vorteile ein eng gekoppeltes Softwaresystem bietet und welche Probleme bei seiner Realisierung auftreten.

8.1.1 Arten der Nebenläufigkeit

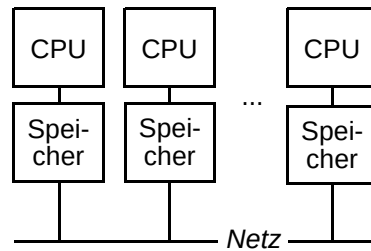
Multitasking, Multiprogramming, Multiprocessing und Multicomputing

Allgemein lassen sich mehrere Arten der Nebenläufigkeit unterscheiden (siehe Abb. 8.1): Auf einem Ein-Prozessor-System ist nur eine *Pseudo-Nebenläufigkeit* möglich, da zu jedem Zeitpunkt nur ein Prozess auf dem Prozessor ausgeführt werden kann. Durch das rasche Hin- und Herschalten des Prozessors zwischen den Aktivitäten wird jedoch den Benutzern eine nebenläufige Ausführung vorgespiegelt. Man kann hier zwischen *Multitasking* und *Multiprogramming* unterscheiden: Beim Multitasking zerfällt ein Programm in mehrere nebenläufige Ausführungseinheiten (z.B. in UNIX und Linux realisiert durch `fork()` oder auch `clone()`), bei Multiprogramming werden mehrere eigenständige Programme bearbeitet.

a.) enge Kopplung:



b.) lose Kopplung:

**Abb. 8.2** Arten der Hardwarekopplung

Systeme mit mehreren Prozessoren führen Prozesse *echt nebenläufig* aus. Beim *Multiprocessing* befinden sich mehrere Prozessoren in einem Rechner und kommunizieren über einen gemeinsamen Speicher und einen schnellen Bus. Man spricht hier auch von einem *Multiprozessorsystem*.

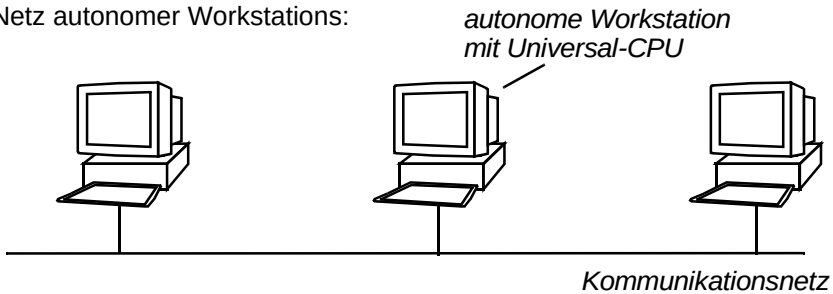
Beim *Multicomputing* besteht das System aus mehreren autonomen *Rechnerknoten*, also eigenständigen Computern mit einem oder mehreren Prozessoren, die über ein „langsames“ Kommunikationsnetz miteinander verbunden sind. „Langsam“ bedeutet dabei erstens, dass die Zeitdauer der Ende-zur-Ende-Übertragung eines Bits relativ hoch ist, und vor allem zweitens, dass die Übertragungsrate in Bits pro Sekunde relativ niedrig ist.

Multiprocessing und Multicomputing entsprechen unterschiedlich starken Kopplungen der Hardware (siehe Abb. 8.2). Beim Multiprocessing ist die Hardware *eng gekoppelt* (engl. *closely coupled*), da die Prozessoren über eine schnelle Kommunikationsverbindung verfügen und auf einem gemeinsamen Speicher arbeiten (allerdings besitzen die einzelnen Prozessoren zumeist noch lokale Caches, auf die nur sie zugreifen können). Multicomputing findet auf einer *lose gekoppelten* (engl. *loosely coupled*) Hardware statt, bei der die Prozessoren eigene Speicher besitzen und mit Hilfe von Nachbarn kommunizieren, die über ein Kommunikationsnetz verschickt werden.

Systemkonfigurationen für das Multicomputing

Für das Multicomputing sind verschiedene Systemkonfigurationen denkbar: Erstens können hier mehrere Workstations zusammengeköpelt werden, die jeweils einen (Universal-)Prozessor enthalten und somit (von der Hardware her) im Wesentlichen auch allein arbeitsfähig sind (siehe Abb. 8.3, Teil a). Die Workstations können homogen oder auch heterogen sein, d.h. vom selben Typ oder auch von unterschiedlichen Typen. An jeder dieser Workstations ist höchstens ein Benutzer aktiv. Die Vernetzung durch ein lokales Netz (LAN, siehe Unterkapitel 8.2) wird z.B. benutzt für die Kommunikation zwischen den Benutzern, zur Realisierung eines gemeinsamen Dateisystems oder zur Verlagerung von Aufträgen auf gerade unbenutzte Stationen. Beispiele für eine solche Konfiguration sind Gruppen von PCs unter Windows NT / 2000 oder *Cluster* von Sun-Workstations unter Solaris.

a.) Netz autonomer Workstations:



b.) Graphikterminals mit zentraler Universal-CPU oder CPU-Pool:

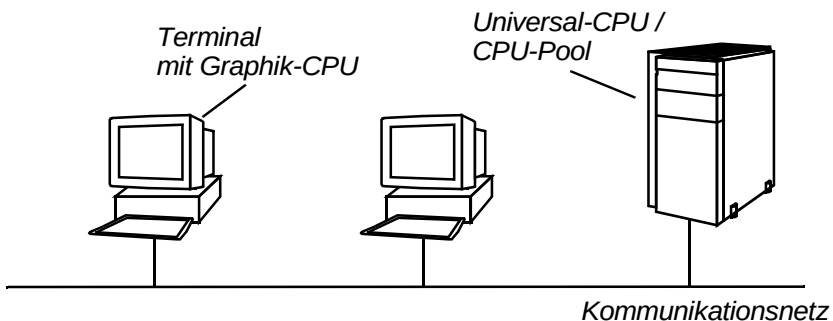


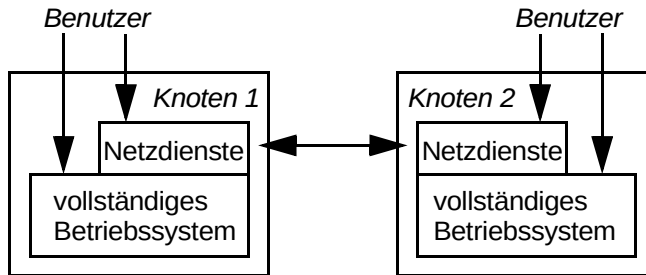
Abb. 8.3 Hardwarekonfigurationen bei Multicomputing

Zweitens ist es möglich, die Verarbeitungskapazität an einer zentralen Stelle zu bündeln (siehe Abb. 8.3, Teil b). Jeder Benutzer verfügt dabei über ein grafikfähiges Terminal, das nur einen speziellen Graphikprozessor für die Ausgabe und einen Hauptspeicher, jedoch keinen Universalprozessor und vielfach auch keine eigene Platte besitzt. Der oder die Universalprozessoren stehen zentral in einem *Processor Pool* und kann bzw. können den Benutzern je nach Bedarf zugeteilt werden – nur sie sind in der Lage, beliebige Programme auszuführen. Die Vernetzung wird hier also genutzt zur Übertragung von Benutzeraufträgen und Antworten zwischen Terminals und Universalprozessoren. Ein Beispiel für ein solches System sind Workstations, an die mehrere grafikfähige *XStations* angeschlossen sind. Auch das experimentelle verteilte System *Amoeba* [Tan92] basiert auf einer solchen Hardware.

Ein Multicomputing-System des ersten (und teilweise auch des zweiten) Typs hat die Eigenschaften, die in der Einführung gewünscht wurden: Es besitzt mehrere Prozessoren, die sich zusammen mit anderen Betriebsmitteln an räumlich getrennten Stellen befinden. Seine Komponenten können eigenständig arbeiten, aber auch kooperieren, indem sie über ein Kommunikationsnetz Nachrichten austauschen.

Ein verteiltes System ist jedoch mehr als ein bloßes Multicomputing-System, wie sich im nächsten Abschnitt zeigen wird.

a.) lose gekoppelte Software:



b.) eng gekoppelte Software:

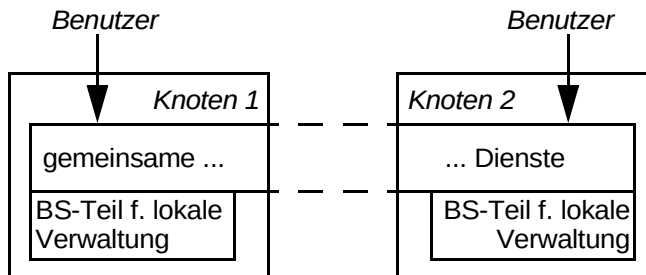


Abb. 8.4 Kopplung der Software

8.1.2 Betriebssystemklassen

Im vorigen Abschnitt wurden nebenläufige Systeme anhand von Hardwareaspekten klassifiziert. Lose gekoppelte Multicomputing-Systeme können aufgrund ihrer Softwareeigenschaften weiter untergliedert werden, wobei insbesondere das Betriebssystem von Interesse ist. Auch hier kann zwischen enger und loser Kopplung unterschieden werden.

Lose gekoppelte Software

Bei einer losen Kopplung, also bei *lose gekoppelter Software* auf lose gekoppelter Hardware, besitzt jeder Rechnerknoten sein eigenes vollständiges Betriebssystem (siehe Abb. 8.4, Teil a), wobei sogar auf verschiedenen Knoten unterschiedliche Betriebssysteme laufen können. Es gibt zudem zusätzliche maschinenübergreifende Dienste, die auf den lokalen Betriebssystemen aufsetzen und ein Arbeiten im Netz ermöglichen. Typische Dienste sind hier (wie z.B. unter Solaris oder als Internet-Dienste zu finden):

- die Ausführung von Betriebssystemkommandos auf anderen Rechnern des Netzes (Kommando `rsh` = remote shell),

- das Einloggen auf andere Rechner vom eigenen Terminal aus (Kommando `rlogin` = remote login bzw. Internet-Dienst Telnet),
- der Transfer von Dateien über das Netz (Kommando `rcp` = remote copy bzw. Internet-Dienst FTP = file transfer protocol) sowie
- die Bereitstellung von Dateibäumen durch File-Server-Knoten, die auf jedem Rechner ganz oder teilweise in den lokalen Dateibaum eingebunden werden können.

Ein Betriebssystem mit solchen Diensten heißt *Netzbetriebssystem* (engl. *network operating system*). Bei Netzbetriebssystemen „sieht“ der Benutzer die Verteiltheit, denn die Benutzerschnittstelle unterscheidet zwischen Kommandos, die sich auf den lokalen Rechner beziehen, und Kommandos für andere Rechner sowie zwischen Zugriffen auf lokale und auf entfernt liegende Dateien.

Eng gekoppelte Software

Bei einer engen Kopplung, also bei *eng gekoppelter Software* auf lose gekoppelter Hardware, gibt es ein gemeinsames Betriebssystem für alle Computer im Netz – d.h. alle Rechnerknoten führen zusammen die Funktionen eines globalen Betriebssystems aus (siehe Abb. 8.4, Teil b). Dieses Betriebssystem verwaltet die Komponenten im gesamten Netz, wofür es wie folgt organisiert ist: Auf jedem Knoten ist eine Kopie eines grundlegenden Betriebssystemkerns vorhanden, der lokale Basisdienste (z.B. Speicher- und Prozessorverwaltung) sowie Mechanismen zur Kommunikation mit anderen Knoten enthält. Zusätzlich gibt es gemeinsame höhere Dienste (z.B. Dateisystem, Druckausgabe, Verfahren zur Verteilung der Arbeitslast), die nicht auf jedem Knoten, sondern nur auf bestimmten Servern implementiert sind. Diese Dienste stehen aber überall zur Verfügung, da sie über das Netz vom jeweiligen Server angefordert werden können.

Der Anspruch bei diesem Ansatz ist, dem Benutzer gegenüber die Verteiltheit zu verbergen, d.h. ihm ein mächtiges System vorzuspiegeln, das ihm lokal und allein zur Verfügung steht (ein „virtuelles Ein-Prozessor-System“). Im Unterschied zu Netzbetriebssystemen unterscheidet die Benutzerschnittstelle also nicht zwischen lokalen und entfernt liegenden Diensten. Ein entsprechendes Betriebssystem heißt *verteiltes Betriebssystem* (engl. *distributed operating system*), das aus einer lose gekoppelten Hardware ein eng gekoppeltes Benutzersystem macht.

Neben Netzbetriebssystemen und verteilten Betriebssystemen gibt es noch *Multi-prozessor-Betriebssysteme* zur Steuerung einer (lokalen) Rechnerhardware mit mehreren Prozessoren. Sie werden jedoch in diesem Kapitel nicht näher betrachtet. Hier soll nur darauf hingewiesen werden, dass moderne Betriebssysteme wie Solaris und Windows NT / 2000 Multiprozessorhardware unterstützen.

In einem verteilten System ist die Realisierung der Software im Allgemeinen schwieriger als die der Hardware. Im Folgenden werden wir uns mit dieser Software näher befassen und dabei insbesondere auf Betriebssystemaspekte eingehen, da das Betriebssystem hier eine der wichtigsten Komponenten ist. Neben dem Betriebssystem (siehe Abb. 8.5) gehören zur Software eines verteilten Systems ein *Anwendungssystem* (d.h.

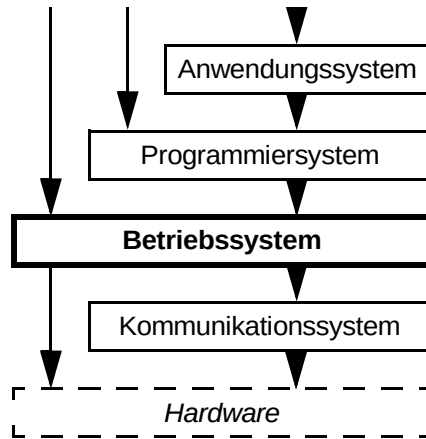


Abb. 8.5 Softwarekomponenten eines verteilten Systems

Programme zur Lösung verteilter Anwendungsprobleme wie z.B. eine verteilte Kontenverwaltung), ein *Programmierersystem* (d.h. Compiler und Laufzeitsystem zur Programmierung und Ausführung des Anwendungssystems) sowie ein *Kommunikationssystem* für einen rechnerübergreifenden Nachrichtenaustausch (siehe Unterkapitel 8.2).

8.1.3 Ziele, Vorteile und Probleme verteilter Systeme

Bei der Implementierung eines verteilten Systems versucht man, eine Reihe von Zielen zu erreichen, die für den Programmierer und Anwender vorteilhaft sind. Dabei stellen sich aber eine ganze Reihe von Realisierungsproblemen.

Ziele

Das wichtigste Ziel ist die so genannte *Transparenz*. Dieser Begriff besagt, dass der Benutzer des verteilten Systems die räumliche Verteiltheit nicht „sieht“, sondern dass ihm (wie schon weiter oben gesagt) ein einheitliches Ein-Prozessor-System vorgespiegelt wird. Transparenz bezieht sich auf verschiedene Aspekte, zum Beispiel den Ort der Datenspeicherung und der Prozessausführung oder die Anzahl der Kopien einer Datei, die auf verschiedenen Knoten geführt werden. Der Benutzer soll beispielsweise auf Daten zugreifen können, ohne wissen zu müssen, auf welchem Rechnerknoten sie sich gerade befinden. Transparenz soll an der Benutzeroberfläche und möglichst auch an der Systemschnittstelle realisiert werden, wobei letzteres wesentlich schwieriger ist.

Weitere Ziele sind die *Erweiterbarkeit (Skalierbarkeit)*, die sicherstellen soll, dass das System bei Bedarf leicht auf ein Vielfaches seiner bisherigen Knotenanzahl vergrößert werden kann, die *Flexibilität*, d.h. eine leichte Veränder- oder Ersetzbarkeit einzelner Systemkomponenten, sowie die *Zuverlässigkeit*, wozu Fehlertoleranz und auch

Systemsicherheit gehören. Last but not least sollte das verteilte System auch eine *Leistung* erbringen, die besser als die eines lokalen Systems ist.

Vorteile

Die genannten Ziele weisen bereits auf einige Vorteile verteilter Systeme hin. Dazu gehört insbesondere die Dezentralisierung von Betriebsmitteln: Wenn gewünscht, können Daten und Verarbeitungskapazität „vor Ort“ gehalten werden, wodurch verteilte Anwendungen unmittelbar unterstützt werden. Außerdem können die Benutzer der einzelnen Knoten über das Netz miteinander kommunizieren.

In einem verteilten System können Server eingerichtet werden, die bestimmte Dienste erbringen (z.B. ein Dateisystem bereithalten oder Druckaufträge entgegennehmen). Diese Dienste müssen damit nicht auf jedem einzelnen Knoten implementiert sein, können aber von überallher angesprochen werden. Man spricht in diesem Zusammenhang auch von einem *Daten-, Funktions- und Betriebsmittelverbund*.

Ein weiterer Vorteil ist die Fehlertoleranz: Beim Ausfall eines Knoten steht nicht unbedingt das gesamte System still, sondern es kann möglicherweise mit den restlichen Komponenten weiterarbeiten. Zudem kann in einem verteilten System die Arbeitslast auf mehrere Knoten verteilt werden (Stichwort *Lastverbund* oder *Leistungsverbund*), wobei bestimmte Anwendungen möglicherweise *parallelisiert*, d.h. in mehreren Teilen echt nebenläufig ausgeführt werden können. Zu den Vorteilen gehört schließlich noch die schrittweise Erweiterbarkeit und Anpassbarkeit des Systems.

Probleme

Die Vorteile eines verteilten Systems ergeben sich allerdings nicht von selbst, sondern sie müssen recht mühsam erarbeitet werden. Das erste Problem bei der Implementierung liegt darin, dass in einem verteilten System Hard- und Software wesentlich komplexer sind als in einem Ein-Prozessor-System. Insbesondere sind nebenläufige Algorithmen schwer zu entwickeln und zu implementieren, wobei insbesondere die Verifikation (d.h. der Nachweis der Korrektheit) sowie die Fehlersuche schwierig sind.

Das zweite Problem ist die räumliche Verteiltheit des Systems: Der Transport einer Information über das Kommunikationsnetz dauert wesentlich länger als über einen lokalen Bus, und die beschränkte Übertragungskapazität des Netzes kann zu einem Engpass werden. Die Übertragungsverzögerungen verursachen insbesondere eine „Unschärfe“ im System. Das bedeutet, dass an keiner einzelnen Stelle eine vollständige aktuelle Information z.B. über den globalen Systemzustand vorliegt und eine gemeinsame Zeitbasis nicht als gegeben vorausgesetzt werden kann.

Die räumliche Verteiltheit erschwert außerdem das Auffinden von Betriebsmitteln, also von Daten, Code und Geräten, und macht die Synchronisation von Prozessen sowie die Deadlock-Behandlung zu einem größeren Problem. Die Systemsicherheit ist schwieriger durchzusetzen, da kritische Systemkomponenten nicht an einer zentralen Stelle geschützt werden können und zudem die Übertragungsstrecken ein potentieller Angriffspunkt sind.

Ein weiteres Problem ist, dass die oben geforderte Skalierbarkeit nicht leicht realisiert werden kann. Insbesondere muss hierfür auf zentralisierte Steuerungskomponenten (wie z.B. Verzeichnisdaten und -funktionen) so weit wie möglich verzichtet werden, da sie bei einer Systemerweiterung leicht zum Engpass werden können.

Schließlich kann die Dezentralisierung von Betriebsmitteln auch eine Dezentralisierung von Verantwortung bedeuten: Jeder lokale Benutzer ist „sein eigener Systemverwalter“, was die Übersichtlichkeit und Steuerbarkeit des Systems gefährdet. Um Wildwuchs und Fehlermöglichkeiten einzudämmen und zudem den Administratoren der Endgeräte eine Arbeitslast abzunehmen, können Verwaltungsaufgaben an einer zentralen Stelle konzentriert werden, was aber wiederum Engpassgefahren auf den Plan ruft. Der am weitesten gehende Vorschlag in dieser Richtung ist die Einführung eines *Network Computers (NC)*. Er ist im Wesentlichen ein grafikfähiges Endgerät, das alles über das Netz von einem zentralen Server bezieht – also mehr oder weniger eine Rückkehr zum Großrechner mit seinen „dummen“ Terminals.

Fazit

Die Programmierung von System- und Anwendungssoftware für verteilte Systeme ist also nicht ganz leicht. Während die vergleichsweise einfachen Netzbetriebssysteme schon seit vielen Jahren auf dem Markt vertreten sind, sind echte verteilte Systeme immer noch Gegenstand intensiver Forschungs- und Entwicklungsarbeiten. Erste verteilte Betriebssysteme kamen erst in der jüngeren Vergangenheit auf, wobei es sich zunächst hauptsächlich um Experimentalsysteme handelte.

Kommerzielle Betriebssysteme bewegen sich ebenfalls langsam in Richtung verteilter Systeme. Beispiele sind hier die UNIX-Version Solaris von Sun und die Windows-Linie von Microsoft, die von Version zu Version in dieser Richtung ausgebaut werden. Wir werden im folgenden Kapitel auf entsprechende Sun/Solaris-Mechanismen sowie auf Dienste von Windows 2000 eingehen.

Zudem werden mittlerweile unter dem Stichwort „Middleware“ Sammlungen von Diensten angeboten, die für verteilte Betriebssysteme typisch sind und zur Realisierung verteilter Anwendungen benutzt werden können. Beispiele sind hier *OSF/DCE* und *OMG CORBA*, die ebenfalls in Kapitel 9 näher besprochen werden.

8.2 Grundlagen der Datenkommunikation

Ein verteiltes System basiert auf einem Kommunikationssystem, über das Nachrichten und damit Informationen zwischen den beteiligten Rechnerknoten übertragen werden. In diesem Unterkapitel sollen daher knapp die wichtigsten Grundlagen der Datenkommunikation dargestellt und anhand einiger praktischer Beispiele illustriert werden. Details zum Thema finden sich beispielsweise in [Tan96] und [Com99].

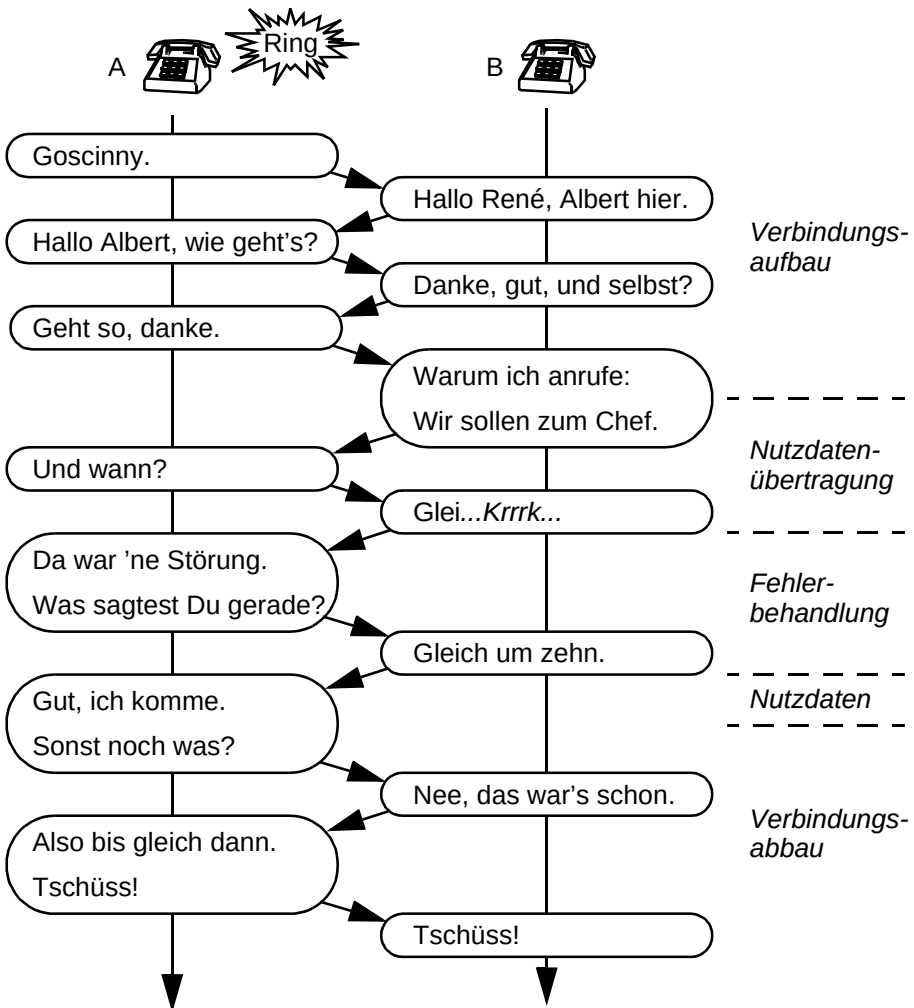


Abb. 8.6 Beispiel für einen Nachrichtenaustausch

8.2.1 Nachrichten und Protokolle

Abbildung 8.6 zeigt den Ablauf eines Telefongesprächs als ein einfaches Beispiel für einen Nachrichtenaustausch. Eine Nachricht ist hier ein Satz oder Satzfragment in gesprochener deutscher Sprache, das von dem einen Gesprächspartner (dem *Sender*) zum anderen Partner (dem *Empfänger*) übertragen wird. Man kann hier unterscheiden zwischen Nachrichten, die Nutzdaten (= die „eigentlichen“ Informationen) enthalten, und Nachrichten, die zur Steuerung des Kommunikationsflusses dienen. Als Medium für die Kommunikation wird die Telefonleitung mit elektrischen oder optischen Signalen bzw. die Luft mit Schallwellen benutzt.

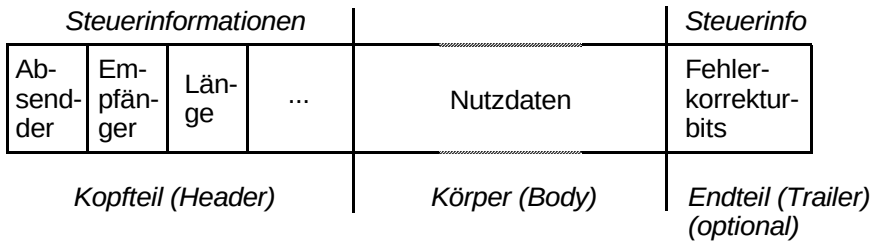


Abb. 8.7 schematischer Aufbau eines Pakets

Die Kommunikation unterliegt einer Reihe von Regeln, die durch allgemeine Konvention festgelegt sind und beim Gespräch mehr oder weniger unbewusst befolgt werden. Sie bestimmen insbesondere

- die benutzte Sprache,
- den zeitlichen Ablauf des Gesprächs (Gesprächsbeginn = „Verbindungsaufbau“, Hauptteil = „Nutzdatenübertragung“, Gesprächsende = „Verbindungsabbau“),
- die Form und die Abfolge der Steuernachrichten (Meldung mit Namen, Begrüßung mit Nennung des anderen Namens usw.) und
- die Behandlung von Übertragungsfehlern.

Die Datenkommunikation in Rechnernetzen beruht auf denselben Prinzipien: Nachrichten werden zu Datenpaketen zusammengefasst und übertragen, wobei die Vorgehensweise durch eine Menge von Regeln, ein so genanntes Protokoll, festgelegt wird:

- Eine *Nachricht* ist eine Folge von Zeichen (z.B. Bits / Bytes), die *Informationen* darstellen („codieren“). Die Art der Codierung ist per Konvention festgelegt. Die Zeichen werden zu *Paketen* zusammengefasst, die neben den *Nutzdaten* auch *Steuerinformationen* enthalten (siehe Abb. 8.7) und vom Kommunikationssystem als Einheiten aufgefasst werden. Die entsprechenden englischen Begriffe lauten *message*, *information*, *packet*, *payload* und *control information*.
- Ein *Kommunikationsprotokoll* (kurz: *Protokoll*, engl. *communication protocol*) ist eine Vereinbarung über den geordneten Ablauf der Kommunikation. Insbesondere legt es Aussehen und Bedeutung der Pakete (Datenpakete und reine Steuerpakete) fest und bestimmt zulässige Abfolgen beim Austausch der Pakete.

Geschichtete Protokolle

Wie Betriebssysteme, so müssen auch Kommunikationssysteme den großen Abstand überbrücken, der zwischen der real vorhandenen Hardware und der Anwendungsschnittstelle mit ihren benutzerfreundlichen Diensten besteht. Dies geschieht nach dem-

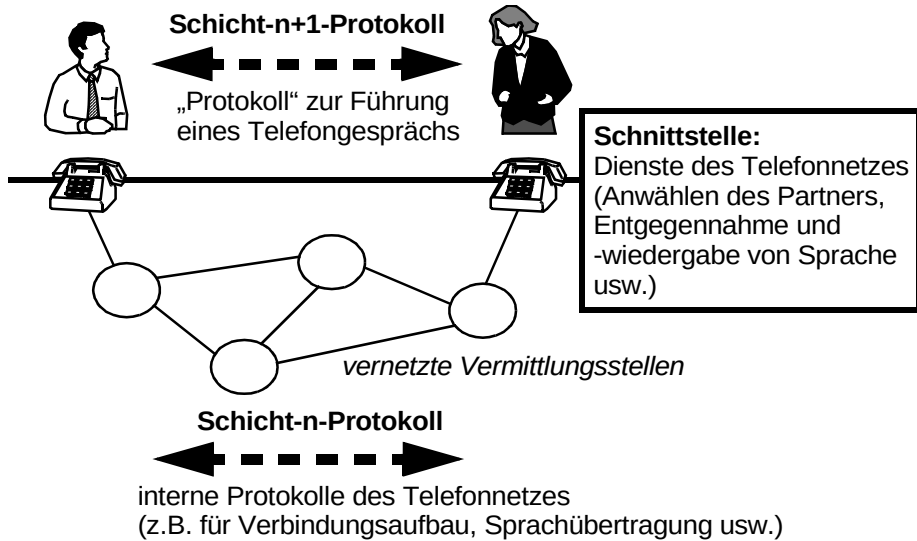


Abb. 8.8 Beispiel für geschichtete Protokolle

selben Prinzip wie bei hierarchisch aufgebauten Betriebssystemen: Es werden mehrere Protokollschichten realisiert, die aufeinander aufbauen. Bei unserem Telefonbeispiel sieht das (stark vereinfacht) wie in Abbildung 8.8 dargestellt aus:

Neben dem oben skizzierten „Protokoll“, das die Führung eines Telefongesprächs zwischen zwei Menschen regelt, gibt es im Telefonnetz weitere Protokolle, die das interne Zusammenspiel der Systemkomponenten festlegen – wie beispielsweise das technische Verhalten der Vermittlungsstellen beim Aufbau einer Verbindung. Diese Protokolle stellen nach oben an einer Schnittstelle Dienste bereit, z.B. die Möglichkeit zum Anwählen des Partners und die Ein- und Ausgabe von Sprache. Bei der Schnittstelle handelt es sich um den Telefonapparat. Die Einheiten der Schicht darüber (hier die Gesprächspartner) nutzen diese Dienste, um damit ihre eigene Kommunikation anhand ihres Telefongesprächs-Protokolls zu realisieren. Sie müssen dazu aber nicht die darunter liegenden Protokolle der Vermittlungsstellen in ihren Einzelheiten kennen.

Abbildung 8.9 zeigt allgemein, wie Kommunikationssysteme mit *geschichteten Protokollen* (engl. *layered protocols*) organisiert sind: Funktionseinheiten der Schicht $n+1$ kommunizieren mit Einheiten derselben Schicht auf anderen Knoten (so genannten *Peers*) und halten sich dabei an das Protokoll der Schicht $n+1$. Zur Realisierung dieser Kommunikation benutzen sie *Dienste* (engl. *services*), die von Funktionseinheiten der darunter liegenden Schicht n an so genannten *Service Access Points* (SAPs) angeboten werden. Die SAPs bilden also die Schnittstelle zwischen Schicht n und Schicht $n+1$. Die Einheiten der Schicht $n+1$ „sehen“ nur ihre peers, ihr Schicht- $n+1$ -Protokoll und die SAPs an der Schnittstelle, nicht jedoch die Komponenten unterhalb der Schnittstelle.

Meist sind mehr als nur zwei Protokolle übereinander geschichtet, so dass sich ein *Protokollstack* (deutsch *Protokollstapel*) ergibt. Die „virtuelle Kommunikation“ (also das, was Schicht- k -Komponenten „sehen“) spielt sich hier jeweils direkt zwischen

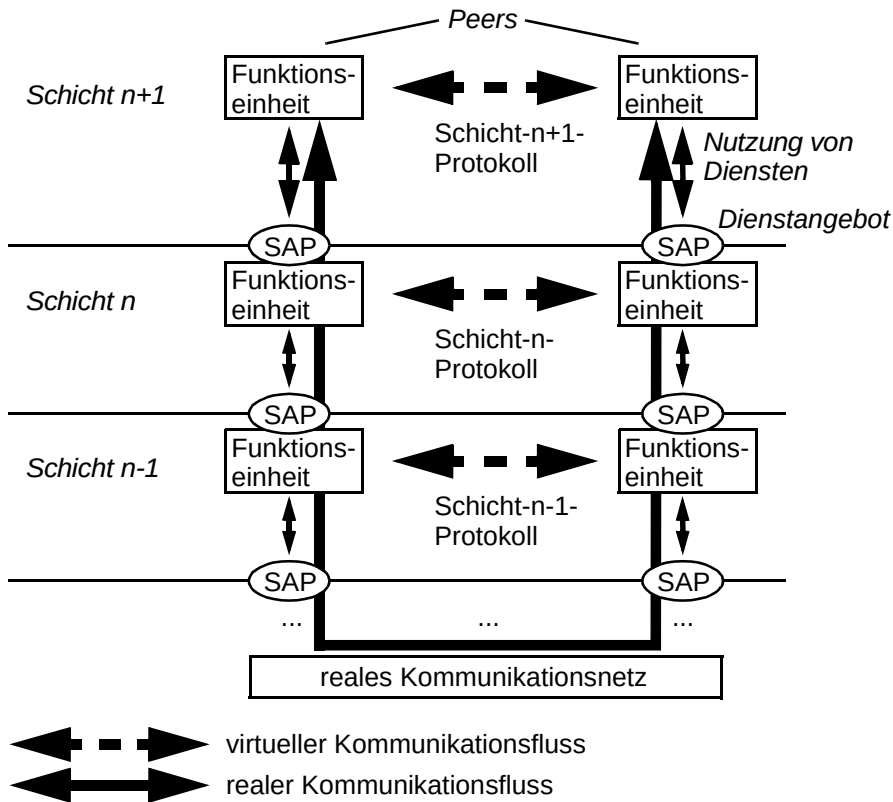


Abb. 8.9 geschichtete Protokolle („Protokollstack“)

Schicht-k-Einheiten über das Schicht-k-Protokoll ab. Der reale Kommunikationsfluss durchläuft dagegen alle darunter liegenden Schichten und das reale Netz.

Ein „Wermutstropfen“ bei diesem Ansatz ist, dass der Aufwand für den Durchlauf eines Pakets durch mehrere Schichten zu Leistungseinbußen führen kann. In der Praxis wird daher bisweilen versucht, die Bearbeitung zu beschleunigen, indem man die Funktionalitäten verschiedener Schichten in einer Komponente zusammenfasst.

Arten der Kommunikation

Die Dienste, die in realen Kommunikationssystemen angeboten werden, sind sehr vielfältig. Man unterscheidet hier u.a. zwischen *verbindungsorientierten* und *verbindungslosen Diensten* (engl. *connection-oriented* bzw. *connectionless services*). Bei verbindungsorientierten Diensten wird, wie im Telefonnetz, vor der Nutzdatenübertragung eine Verbindung zwischen den Kommunikationspartnern aufgebaut und anschließend wieder abgebaut. Bei verbindungslosen Diensten werden, wie bei der Briefpost, Nutzdatenpakete ohne vorherigen Verbindungsaufbau losgeschickt und anhand des Adress-eintrags im Paket an den Empfänger weitergeleitet.

Ferner unterscheidet man zwischen *Punkt-zu-Punkt-Kommunikation* (die Nachricht wird an einen bestimmten Empfänger geschickt), *Multicast* (an eine Empfängergruppe) und *Broadcast* (an alle Netzteilnehmer).

8.2.2 ISO/OSI-Referenzmodell für geschichtete Protokolle

Für Systeme, die mit geschichteten Protokollen arbeiten, hat die *International Organization for Standardization (ISO)* ein so genanntes *Referenzmodell* entwickelt, nämlich das bekannte *OSI-Modell*. OSI steht für *Open Systems Interconnection* und formuliert damit das Ziel, einen Standard für die Verbindung „offener“ Systeme zu bieten – also Systeme, die nicht nur mit Systemen desselben Herstellers, sondern auch mit anderen kommunizieren können. Der Begriff „Referenzmodell“ bedeutet (zumindest aus heutiger Sicht), dass ein Klassifikationsschema gegeben wird, in das Protokolle anhand ihrer Funktionalität und der angebotenen Dienste eingeordnet werden können. OSI war zudem ursprünglich als allgemeiner Standard für Dienste, die an den Schnittstellen der Schichten angeboten werden, und zugehörige Protokolle gedacht. Dieses Ziel wurde aber wegen des raschen Aufkommens von Nicht-OSI-Protokollen (insbesondere der Internet-Protokolle) nicht erreicht.

Schichten des OSI-Modells

Das OSI-Modell unterscheidet sieben Schichten (siehe Abb. 8.10):

- Die unterste Schicht 1 (*Physical Layer, Bitübertragungsschicht*) setzt auf dem realen Übertragungsmedium (z.B. Kupferkabel oder Glasfaser) auf und realisiert die Übertragung eines Bitstroms mit Hilfe elektrischer oder optischer Signale.
- Schicht 2 (*Data Link Layer, Sicherungsschicht*) zerfällt in zwei Teilschichten: Die untere, der *Media Access Control (MAC) Sublayer*, regelt im Fall mehrerer konkurrierender Stationen den Zugang zum Übertragungsmedium, während die obere, der *Logical Link Control (LLC) Sublayer*, das Verschicken von Paketen ermöglicht und dabei Mechanismen zur Fehlerbehandlung bietet.
- Protokolle der Schicht 3 (*Network Layer, Vermittlungsschicht*) dienen zur Wegesuche im Kommunikationsnetz (zum so genannten „Routing“), indem sie Pakete vom Sender über möglicherweise mehrere Zwischenknoten zum Empfänger weiterreichen. Zudem sollen sie auf Paketstaus innerhalb des Netzes reagieren.
- Schicht 4 (*Transport Layer, Transportschicht*) ermöglicht eine „Ende-zu-Ende-Kommunikation“, also eine Datenübertragung zwischen den Kommunikationspartnern an den Endpunkten des Netzes. Zudem können Schicht-4-Protokolle Mechanismen zur Fehlerbehandlung realisieren sowie eine Flusskontrolle, die verhindert, dass mehr Daten in das Netz geschickt werden, als es verkraften kann.

Schicht 7: Application Layer (Anwendungsschicht)	problemnahe Dienste (Mail, Zugriff auf entfernte Dateisysteme, ...)
Schicht 6: Presentation Layer (Datendarstellungsschicht)	Anpassung der Datencodierung
Schicht 5: Session Layer (Kommunikationssteuerungsschicht)	„Sitzungen“ zur Prozess-Prozess-Koop. mit evtl. mehreren Verbindungen
Schicht 4: Transport Layer (Transportschicht)	Datentransport zwischen Endsystemen, Fehlerbehandlung, Flusskontrolle
Schicht 3: Network Layer (Vermittlungsschicht)	Bestimmung eines Kommunikationswegs im Netz („Routing“), Behandlung von Datenstaus
Schicht 2: Data Link Layer (Sicherungsschicht)	Logical Link Control (LLC) Media Access Control (MAC) Datenpakete mit Fehlerbehandlung Zugangsregelung zum Übertr.medium
Schicht 1: Physical Layer (Bitübertragungsschicht)	Bitstrom auf dem physischen Übertragungsmedium

Abb. 8.10 ISO/OSI-Referenzmodell für Protokolle zur Datenkommunikation

- Schicht 5 (*Session Layer, Kommunikationssteuerungsschicht*) enthält Mechanismen zur Steuerung von „Sitzungen“ der Kommunikationspartner, in deren Verlauf möglicherweise mehrere Transportverbindungen auf- und abgebaut werden. Beispielsweise können in Dialoganwendungen hier die Datenübertragungen, die in den beiden Richtungen verlaufen, synchronisiert werden.
- Schicht 6 (*Presentation Layer, Datendarstellungsschicht*) befasst sich im weitesten Sinne mit der Syntax der zu übertragenden Daten. Eine herausragende Aufgabe ist hier die Transformation der Daten von bzw. in eine plattformunabhängige Codierung. Damit soll eine korrekte Interpretation der Bytes beim Empfänger sichergestellt werden. *ASN.1 (Abstract Syntax Notation-1)* mit ihren *BER (Basic Encoding Rules)* ist ein entsprechender Standard für die Datendarstellung.
- Schicht 7 (*Application Layer, Anwendungsschicht*) enthält schließlich anwendungsnahe Protokolle, beispielsweise zum Dateitransfer oder zum Übertragen von Mails.

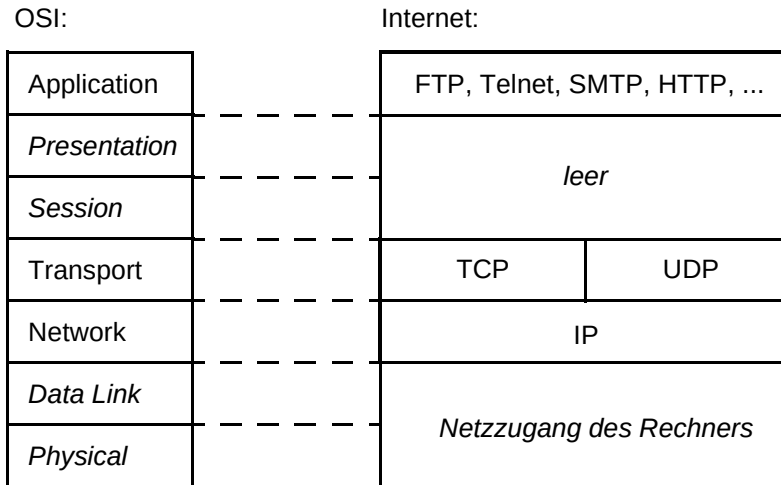


Abb. 8.11 grobe Einordnung des Internet-Protokollstacks in das OSI-Modell

Man bezeichnet die Schichten 5 bis 7 auch als anwendungsorientierte Schichten und die Schichten 1 bis 4 als transportorientierte Schichten.

Schichten im Internet

Im *Internet* sind im Wesentlichen drei Schichten des OSI-Modells von Interesse (siehe Abb. 8.11 für eine grobe Zuordnung): Schicht 7 bietet anwendungsorientierte Internetdienste und enthält entsprechende Protokolle, wie z.B. *FTP* (*File Transfer Protocol*) für Dateitransfers, *Telnet* zum Einloggen auf entfernt liegende Rechner, *SMTP* (*Simple Mail Transfer Protocol*) für den Mail-Verkehr und *HTTP* (*HyperText Transfer Protocol*) zum Zugriff auf WWW-Seiten. Schicht 4 umfasst zwei Transport-Protokolle, nämlich das verbindungsorientierte *TCP* (*Transmission Control Protocol*) und das verbindungslose *UDP* (*User Datagram Protocol*). In Schicht 3 befindet sich das Herzstück des Internets, nämlich *IP* (*Internet Protocol*). *IP* setzt seinerseits auf ein (im Prinzip beliebiges) Protokoll auf, das den Zugriff auf das Kommunikationsmedium regelt. Wir werden hierauf sowie auf *TCP* und *IP* später noch zurückkommen.

Abbildung 8.12 illustriert am Beispiel einer Internet-Mail die Vorgehensweise beim Senden und Empfangen von Daten (wie man sie analog in allen anderen Protokollstacks findet): Beim Sender durchwandern die Nutzdaten die Schichten von oben nach unten. Dabei fügt jede Schicht einen „Header“ und gegebenenfalls einen „Trailer“ hinzu, die Informationen für das Protokoll dieser Schicht enthalten. Wird das Paket zu lang, so wird es in mehrere Teile aufgespalten. Nach der Datenübertragung über das reale Netz durchlaufen das oder die Pakete auf dem Empfängerrechner den Protokollstack in umgekehrter Reihenfolge. Dabei werden Header und Trailer entfernt und Teilpakete wieder zusammengefasst.

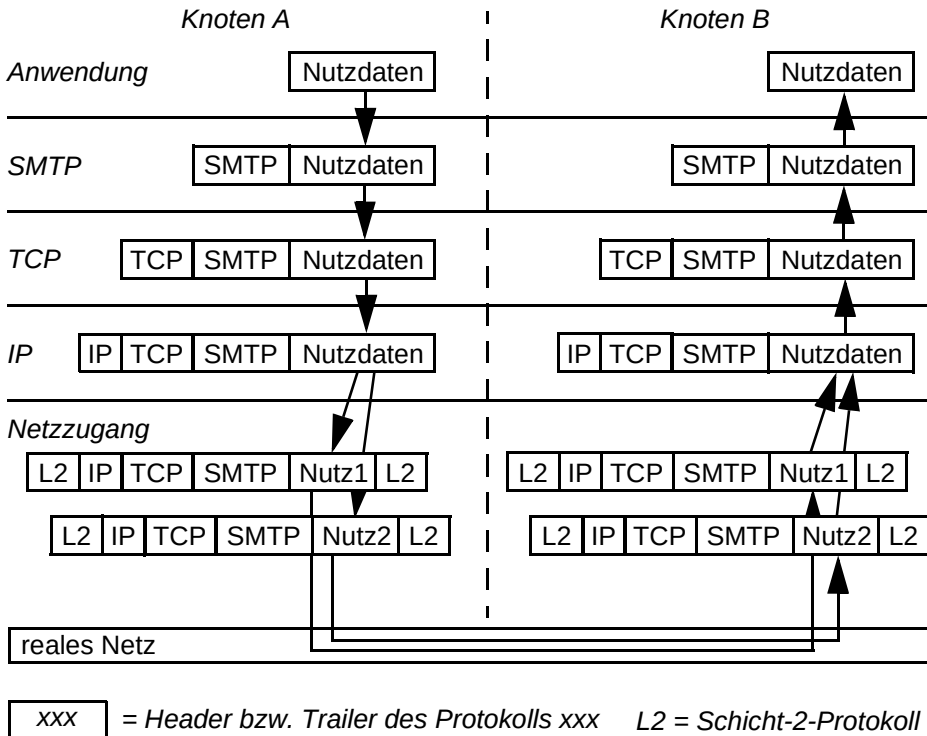


Abb. 8.12 Datenübertragung über den Internet-Protokollstack (Beispiel: Mail)

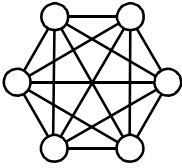
8.2.3 Bitübertragungsschicht

Übertragungsmedien

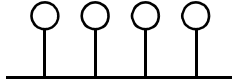
Kommunikationsnetze können verschiedene Übertragungsmedien nutzen, die sich in den Kosten, der Übertragungsrate (d.h. Maximalzahl der Bits pro Sekunde, engl. *bits per second* / *bps*, die übertragen werden können), der Reichweite und der Fehleranfälligkeit unterscheiden:

- *Elektrische Kabel* gibt es in verschiedenen Ausführungen. Der älteste Ansatz ist die Telefonverkabelung mit Leitungen aus zwei verdrehten Kupferkabeln (engl. *twisted pair*). Weniger störänfällig und besser hinsichtlich der Übertragungsrate sind *Koaxialkabel*.
- In *optischen Übertragungsmedien* (unter den Bezeichnungen *Glasfaserkabel*, *Lichtwellenleiter (LWL)* und *Fiber* bekannt) erfolgt die Datenübertragung durch kurze Lichtimpulse. Sie ermöglichen wesentlich höhere Übertragungsraten und haben niedrigere Fehlerraten als elektrische Kabel.

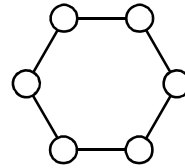
a.) vollst. Vernetzung:



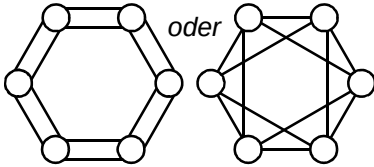
b.) Bus:



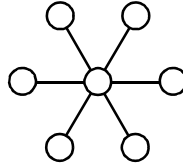
c.) Ring (einfach):



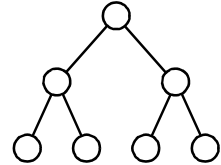
d.) Ring (doppelt):



e.) Stern:



f.) Baum:

**Abb. 8.13** Netztopologien

- Neben den kabelgebundenen gibt es auch *drahtlose Netze*, die als Übertragungsmedium Funkwellen, infrarotes Licht oder auch Laser benutzen.

Da die Reichweite der Signale jeweils begrenzt ist, müssen bei größeren Entfernungen so genannte *Repeater* zur Signalverstärkung eingesetzt werden.

Netztopologie

Für die Struktur eines Netzes (die *Netztopologie*) gibt es verschiedene Möglichkeiten (siehe Abb. 8.13):

- Die *vollständige Vernetzung*, bei der jeder Knoten mit jedem anderen direkt verbunden ist, ermöglicht unter allen Topologien den größten Datendurchsatz. Sie ist dafür aber auch am aufwendigsten. Zur Kosteneinsparung können einzelne Verbindungen, die weniger häufig benötigt werden, weggelassen werden.
- Bei einem Netz mit *Busstruktur* sind alle Knoten direkt an ein gemeinsames Übertragungsmedium angeschlossen, „hören“ also sämtliche Sendungen unmittelbar mit. Um Störungen zu vermeiden, werden Protokolle zur Zugangsregelung benötigt, wie sie im folgenden Abschnitt besprochen werden.
- In einem *Ring* sind die Rechnerknoten zyklisch angeordnet, wobei jeder Knoten nur mit seinen beiden Nachbarn direkt verbunden ist. Sendungen zwischen Knoten, die weiter voneinander entfernt liegen, müssen also über die dazwischenliegenden Knoten weitergereicht werden. Aus Fehlertoleranzgründen kann der Ring auch doppelt angelegt sein.

- Bei einem *sternförmigen Netz* gibt es eine zentrale Station, die Sendungen aller anderen Stationen entgegennimmt und direkt an die Empfänger weiterleitet.
- *Baumförmige Strukturen* eignen sich zur Realisierung von hierarchischen Netzen, die beispielsweise aus persönlichen Workstations, Abteilungsrechnern und Großrechnern bestehen.

Diese Topologien lassen sich zu Mischformen kombinieren. So kann man z.B. mehrere sternförmig organisierte Subnetze aufbauen und deren zentrale Knoten vollständig miteinander vernetzen.

8.2.4 Klassifizierung von Netzen

Netze können anhand ihrer Ausdehnung klassifiziert werden:

- Ein *lokales Netz (Local Area Network, LAN)* beschränkt sich meist auf ein oder einige wenige Gebäude und hat eine maximale Ausdehnung von einigen wenigen Kilometern. Es dient zur Verbindung von Computern in derselben Firma / Institution oder Abteilung.
- Ein *Metropolitan Area Network (MAN)* verbindet Computer in einem größeren Gebiet (typischerweise einer Großstadt), basiert aber im Wesentlichen auf derselben Netztechnologie wie ein LAN.
- Ein *Weitverkehrsnetz (Wide Area Network, WAN)* überspannt eine wesentlich größere Fläche, z.B. ein ganzes Land oder einen oder mehrere Kontinente. WANs dienen insbesondere als „Backbone“ zur Verbindung einzelner LANs, um den dort angeschlossenen Rechnern eine Weitverkehrskommunikation (z.B. im Internet) zu ermöglichen.

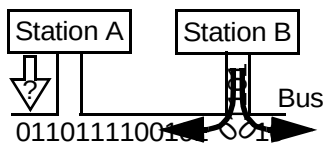
LANs, MANs und WANs unterscheiden sich zudem in den Verfahren zur Regelung des Netzzugangs.

8.2.5 Medienzugangsschicht

Sind in einem Netz mehrere Knoten an dasselbe Kommunikationsmedium angeschlossen, so muss der Zugang dazu geregelt werden, d.h. es muss jeweils entschieden werden, welche von mehreren sendewilligen Stationen tatsächlich senden darf. Das *Institute of Electrical and Electronics Engineers (IEEE)* hat seiner *802er-Serie* eine Reihe von Verfahren standardisiert, die später von der ISO in die *Serie 8802* übernommen wurden. Von diesen Standards sind *802.3* mit dem Ethernet und *802.5* mit dem Token Ring am weitesten verbreitet.

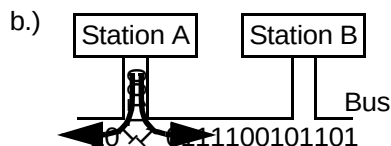
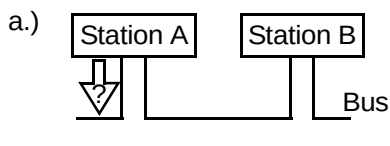
I.) Datenübertragung ohne Kollision:

1.) Sendeversuch bei belegtem Bus:



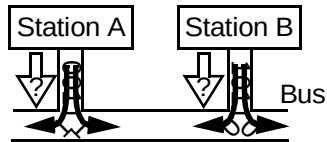
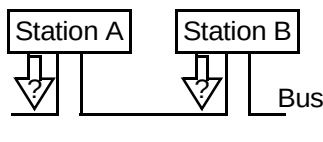
A sendet nicht, sondern wartet und versucht es später erneut.

2.) Sendeversuch bei freiem Bus:

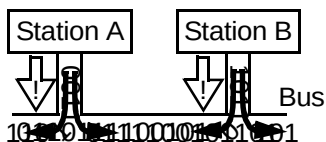


II.) Datenübertragung mit Kollision:

1.) gleichzeitiges Abhören des Busses: 2.) gleichzeitiges Senden:



3.) Entdecken der Kollision:



4.) Abbruch der Sendungen:

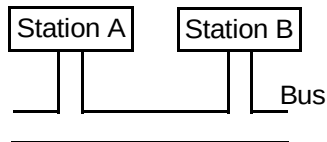


Abb. 8.14 CSMA/CD-Verfahren im Ethernet

Ethernet

Das *Ethernet* ist ein LAN mit Bustopologie. In seiner klassischen Form ermöglicht es Übertragungsraten von 10 MBit/s; im Laufe der Zeit kamen Versionen mit Raten von 100 MBit/s (*Fast Ethernet*) und sogar 1 GBit/s (*Gigabit Ethernet*) hinzu. Ethernet-Datenpakete können maximal 1500 Byte Nutzdaten enthalten.

Der Netzzugang im klassischen Ethernet und im Fast Ethernet wird nach dem *CSMA/CD-Verfahren* (*Carrier Sense Multiple Access with Collision Detection*) geregelt (siehe Abb. 8.14): Wenn eine Station senden möchte, so hört sie zunächst das Medium ab, ob es gerade durch eine andere Station belegt ist („Carrier Sense“). Ist das der Fall, so ver-

sucht sie es nach einer gewissen Wartezeit noch einmal („Multiple Access“); stellt sie dagegen fest, dass das Medium frei ist, so beginnt sie zu senden. Dabei kann es vorkommen, dass zwei Stationen zu (fast) demselben Zeitpunkt sendewillig werden, beide das Medium frei vorfinden und beide mit dem Senden beginnen. Das führt zu einer „Kollision“ auf dem Netz mit Zerstörung der Pakete. Die Stationen stellen dies jedoch fest, da sie das Medium weiter abhören („Collision Detection“), und brechen ihre Sendung ab. Jede Station wartet für eine bestimmte Zeit und versucht es dann erneut. Die Wartezeiten der Stationen werden zufällig gewählt, damit es nicht zwangsläufig zu einer neuen Kollision kommt.

Im Gigabit Ethernet wird nicht CSMA/CD, sondern ein anderes Verfahren benutzt, bei dem ein zentraler Knoten („Hub“) die Zugangsregelung und Weitergabe der Pakete übernimmt.

Token Ring

Im *Token Ring*, ebenfalls einem LAN, sind die Rechnerknoten ringförmig miteinander verbunden. Der klassische Token Ring ermöglicht Übertragungsraten von bis zu 40 MBit/s; üblich waren und sind in der Praxis 4 und 16 MBit/s. Später wurden zwar auch schnellere Versionen (100 Mbit/s) realisiert; es ist jedoch festzustellen, dass der Token Ring hinsichtlich der Datenraten gegenüber dem Ethernet ins Hintertreffen geraten ist.

Der Token Ring arbeitet mit einem speziellen Paket, dem *Token*, das zyklisch von Station zu Station weitergereicht wird und entweder „frei“ oder „besetzt“ ist (siehe Abb. 8.15). Eine Station darf nur senden, wenn sie das freie Token besitzt. Dazu hängt sie ihre Daten an das Token an (das damit besetzt wird) und schickt sie auf das Netz. Das Paket wird dann von Station zu Station weitergesendet, wobei es der oder die Empfänger vom Netz kopieren, und erreicht schließlich wieder den Sender. Dieser entfernt das Paket vom Netz, erzeugt ein freies Token und gibt es an seine Nachbarstation weiter. Pakete im 16-Mbit/s-Token-Ring können maximal ca. 18 KByte Nutzdaten enthalten, d.h. das Token muss spätestens nach dieser Datenmenge weitergegeben werden.

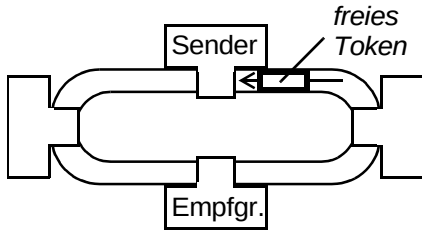
FDDI

Ein weiteres Token-gestütztes LAN ist *FDDI* (*Fiber Distributed Data Interface*). Es hat eine Übertragungsrate von 100 MBit/s und wird im *Backbone*-Bereich eingesetzt, d.h. zur Verbindung mehrerer LANs. An den Schnittstellen zwischen dem FDDI-Ring und den einzelnen LANs werden hier (wie auch zur direkten Verbindung von Ethernets, Token Rings usw.) spezielle Rechner, so genannte *Bridges*, eingesetzt.

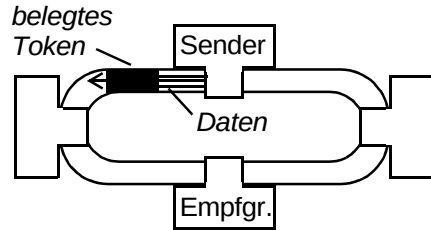
8.2.6 Vermittlungs- und Transportschicht

Hauptaufgabe der Vermittlungsschicht ist das „Routing“, d.h. das Weiterleiten von Paketen über Vermittlungsknoten innerhalb des Netzes (die so genannten *Router*). Die darüber liegende Transportschicht verbirgt dagegen die interne Struktur des Netzes und

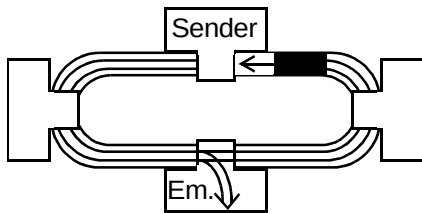
- 1.) freies Token
kommt beim Sender an:



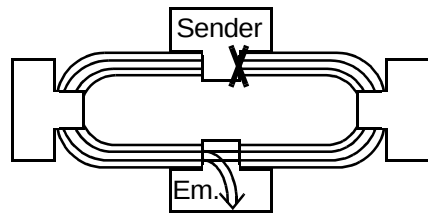
- 2.) Sender ändert Token in „besetzt“
und hängt Daten an:



- 3.) Stationen leiten Daten weiter,
Empfänger kopiert sie:



- 4.) Sender nimmt zurückkehrende
Daten vom Ring:



- 5.) Sender setzt freies Token
auf den Ring:

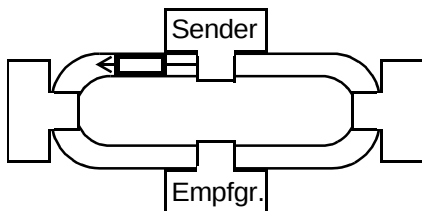


Abb. 8.15 Token-gestütztes Zugriffsverfahren im Token Ring

ermöglicht damit eine transparente Ende-zu-Ende-Kommunikation zwischen Endstationen. Die bekanntesten Protokolle dieser Schichten sind *IP* (*Internet Protocol*) und *TCP* (*Transmission Control Protocol*), auf denen die Datenübertragung im Internet beruht.

Struktur des Internets

Wie Abbildung 8.16 zeigt, besteht das Internet aus einer Vielzahl von Subnetzen, also z.B. LANs, die miteinander verbunden sind. Diese Subnetze sind autonom, d.h. sie sind allein arbeitsfähig und werden dezentral verwaltet.

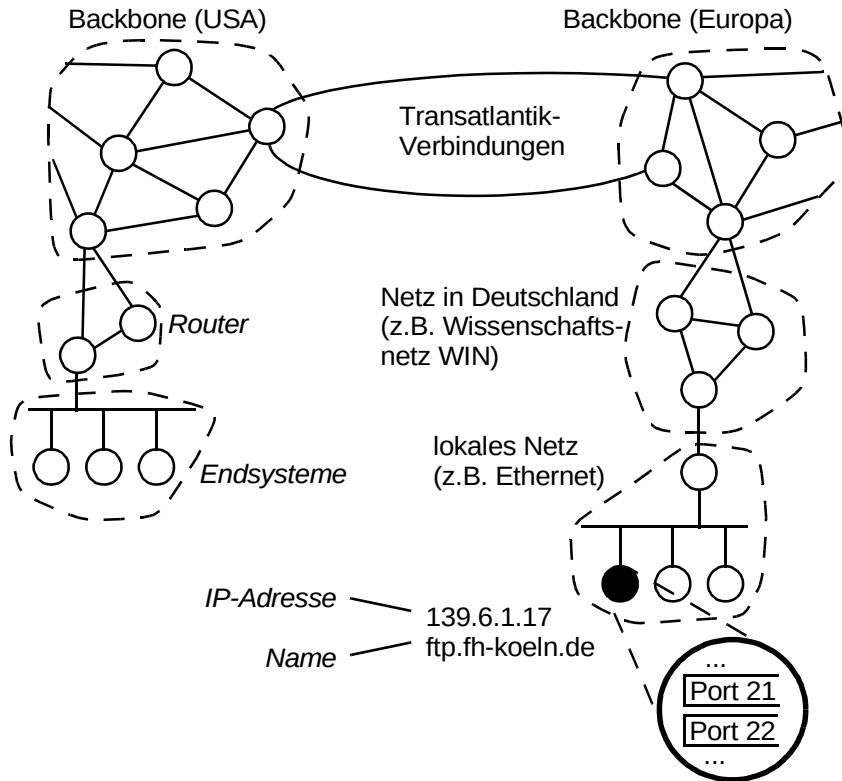


Abb. 8.16 Aufbau und Adressstruktur des Internets

Die Kommunikation im Internet stützt sich auf global eindeutige *Internet-Adressen* (*IP-Adressen*): Jeder Rechner, der an das Internet angeschlossen ist, besitzt eine solche Adresse. In der aktuellen IP-Version Nr. 4 besteht sie aus vier ganzen Zahlen, die jeweils zwischen 0 und 255 liegen – sie lautet also z.B. 139.6.1.17. Die IP-Adresse setzt sich zusammen aus der Nummer des Subnetzes, die von einer zentralen Stelle vergeben wird, und einer Rechnernummer innerhalb des Subnetzes, für deren Vergabe der lokale Netzverwalter zuständig ist.

Da solche Nummern für Anwender etwas unhandlich sind, hat jeder Rechner zudem noch einen symbolischen Namen. Er ist hierarchisch aufgebaut und gibt so den Rechner selbst und seine Einordnung in das Internet an. Der Name „ftp.fh-koeln.de“ besagt z.B., dass der Rechner „ftp“ in der Fachhochschule Köln in Deutschland angesiedelt ist. Verwendet ein Benutzer einen solchen Namen, so wird dieser durch das *Domain Name System (DNS)* auf die zugehörige numerische IP-Adresse abgebildet. DNS wird realisiert durch eine Vielzahl von kooperierenden Servern, die jeweils eine Tabelle mit einem Teil des Namensraums und den entsprechenden IP-Adressen enthalten.

Ein Rechnerknoten bietet unter derselben IP-Adresse meist mehrere Dienste an. Sie werden durch *Portnummern* unterschieden, die mit 16 Bit codiert werden und zusammen mit der IP-Adresse einen Dienst im Internet eindeutig identifizieren. Portnummern

IP-Adr. Source IP-Adr. Destination weitere Info	Portnr. Source Portnr. Destination weitere Info	...	...
<i>IP- Paketkopf</i>	<i>TCP- Paketkopf</i>	<i>Paketkopf höheres Protokoll</i>	<i>Nutzdaten</i>

Abb. 8.17 IP-Paket

aus dem Bereich bis 255 sind bestimmten Diensten fest zugeordnet („well-known ports“): Port 21 dient z.B. dem FTP-Verkehr, Port 23 dem Telnet-Dienst (für ein Verzeichnis dieser Nummern siehe [RFC1700]). Ports werden durch Sockets realisiert.

IP

IP ist, wie schon gesagt, für die Weiterleitung der Pakete über die Router des Netzes zuständig. Der Kopf eines IP-Pakets enthält dazu insbesondere die IP-Adressen seines Absenders („Source“) und seines Empfängers („Destination“) – siehe Abb. 8.17. Ein IP-Router stellt anhand der Zieladresse fest, an welchen benachbarten Router das Paket weitergegeben werden soll. Dabei stützt er sich auf seine *Routing-Tabelle*, deren Eintragungen je nach Nähe zum Ziel unterschiedlich differenziert sind. So könnte beispielsweise die Tabelle eines Routers in den USA festlegen, dass alle Pakete der Form 139.x.y.z an denselben Router in Frankfurt weitergegeben werden sollen, während die Tabelle in einem Netzserver in der Fachhochschule Köln für die Adressen 139.6.1.1, 139.6.1.2 usw. jeweils unterschiedliche Empfängerrechner angibt.

Die Routing-Tabellen werden durch ständigen Informationsaustausch zwischen den Routern auf dem aktuellen Stand gehalten. Auf diese Weise ist es möglich, rasch auf Veränderungen im Netz, wie lokale Überlastungen oder Leitungsausfälle, zu reagieren. Dies macht die Robustheit des Internets aus. Gleichzeitig kann es aber dazu führen, dass zwei Pakete, die ein Sender unmittelbar aufeinander folgend an denselben Empfänger schickt, unterschiedliche Wege nehmen, damit unterschiedlich lange Übertragungszeiten benötigen und sogar eventuell in verkehrter Reihenfolge eintreffen.

Benutzer können den Weg eines Pakets verfolgen, indem sie beispielsweise unter UNIX das `traceroute`-Kommando benutzen. Man kann sich so den Spaß machen festzustellen, welche Route etwa ein Paket nach Tokio oder Australien nimmt.

TCP und UDP

Das Transmission Control Protocol TCP, das auf IP aufsetzt, unterstützt die verbindungsorientierte Kommunikation zwischen zwei Endstationen. Vor der eigentlichen Datenübertragung wird also eine Verbindung aufgebaut und anschließend wieder abgebaut, wofür TCP Protokollmechanismen bereitstellt. Der Kopf eines TCP-Pakets ent-

hält die Portnummer des Senders und des Empfängers (siehe Abb. 8.17); letztere wird beim Empfänger nach Entfernen des vorgeschalteten IP-Kopfs benutzt, um die Daten an den richtigen Dienst weiterzuleiten. Im Gegensatz zu IP, das Multicast-Kommunikation unterstützt, gibt es in TCP nur Punkt-zu-Punkt-Verbindungen. Zu den Aufgaben von TCP gehören insbesondere

- das Zerlegen von größeren Paketen aus höheren Protokollschichten in mehrere TCP-Pakete (sie dürfen nicht größer als 64 KByte sein) und das Wiederaussetzen nach der Übertragung,
- die Wiederherstellung der richtigen Paketreihenfolge nach der Übertragung,
- die Drosselung des Senders bei Überlastung des Netzes („Flusskontrolle“) und
- die erneute Übertragung von Paketen („Retransmission“) bei Fehlern, insbesondere Paketverlusten.

UDP (User Datagram Protocol) ist das verbindungslose Gegenstück zu TCP. Es ermöglicht das Versenden von Paketen (so genannten Datagrammen) ohne vorherigen Verbindungsaufbau und ist auch sonst wesentlich einfacher strukturiert als TCP.

ATM

Neben der Internet-Protokollfamilie ist noch kurz *ATM (Asynchronous Transfer Mode)* zu erwähnen, das Übertragungsraten von 155 MBit/s, 622 MBit/s oder sogar Werte im Gigabit-Bereich erreicht und dessen zentrales Protokoll auf den OSI-Schichten 2 und 3 angesiedelt ist. ATM operiert, anstelle von größeren Paketen, mit kleinen Zellen (Länge: 53 Byte), die in rascher Folge über die Übertragungsleitungen geschickt werden. Schnelle „Switches“ an den Endpunkten der Verbindungen sind in der Lage, Zellen verschiedener Verbindungen zu unterscheiden und sie geeignet weiterzuleiten.

Als Einsatzgebiet von ATM gilt hauptsächlich das Breitband-ISDN, das wesentlich höhere Bitraten als das heutige ISDN ermöglichen soll. Es ist auch möglich, ATM im Backbone- und sogar im LAN-Bereich einzusetzen. Allerdings stößt es hier auf die Konkurrenz der neuen und preiswerteren Ethernet-Entwicklungen.

8.3 Übungsaufgaben

1. Wissensfragen

- a.) Was ist der Unterschied zwischen Multitasking, Multiprogramming, Multiprocessing und Multicomputing?
- b.) Auf welche zwei Arten kann Multicomputing realisiert werden?

- c.) Was ist der Unterschied zwischen einem Netzbetriebssystem und einem verteilten (Betriebs-)System?
- d.) Wonach kann man Netze zur Datenkommunikation klassifizieren?
- e.) Wofür steht die Abkürzung CSMA/CD?
- f.) Was ist ein Protokoll?
- g.) Welche Internet-Protokolle gibt es und wozu dienen sie?

2. Netztopologie

Betrachten Sie Netze mit vollständiger Vernetzung, Stern- und Bustopologie. Was sind deren Vor- und Nachteile hinsichtlich Verkabelungsaufwand, Durchsatz und Störanfälligkeit?

3. Ethernet vs. Token Ring

- a.) Welches der beiden Verfahren CSMA/CD und tokenbasierter Zugriff ist günstiger bei niedrigem Verkehrsaufkommen (d.h. die Stationen möchten nur selten senden), welches bei hohem Aufkommen (d.h. oft wollen zwei oder mehr Stationen senden)?
- b.) Realzeitanwendungen, wie z.B. die Übertragung von Multimediadaten, erfordern die Einhaltung von Zeitschranken bei der Datenübertragung und Garantien bezüglich der bereitgestellten Übertragungskapazität. Wer bietet hierfür eine bessere Grundlage – der Token Ring oder das Ethernet?
- c.) Beim klassischen Token Ring setzt die sendende Station ein neues freies Token erst dann wieder auf den Ring, wenn das gesendete Paket vollständig zurückgekehrt ist. Gibt es hier eine effizientere Möglichkeit?

4. Internet

Im Internet werden zunehmend multimediale Anwendungen, wie z.B. die Internet-Telefonie, realisiert. Solche Anwendungen stellen Realzeitanforderungen, d.h. sie verlangen, dass Daten innerhalb einer festgelegten Zeitspanne übertragen werden und dass das Netz eine bestimmte Übertragungskapazität bereitstellt. Welche Probleme ergeben sich hier mit den vorhandenen Internet-Protokollen?

9 Verteilte Systeme: Techniken

Nachdem das vorangehende Kapitel die Grundlagen verteilter Systeme behandelte, sollen nun einige Betriebssystemtechniken in verteilten Systemen diskutiert werden. Im Einzelnen geht es um die Organisation des Dateisystems, die Kommunikation zwischen Rechnerknoten auf höherer Ebene sowie die Verwaltung von Prozessen. Außerdem werden einige Systembeispiele besprochen. Aus Platzgründen ist die Darstellung an manchen Stellen nur übersichtsartig; für ausführlichere Informationen sollte man daher weiterführende Literatur heranziehen (z.B. [Web98] oder [Tan92]).

9.1 Dateisystem

Als ersten Themenbereich verteilter Betriebssysteme betrachten wir den „Datenverbund“, also die Nutzung eines gemeinsamen Dateisystems für das ganze Netz. Die Realisierung und Markteinführung sind auf diesem Gebiet sehr weit fortgeschritten, denn Netzbetriebssysteme mit einem globalen Dateisystem sind schon seit den achtziger Jahren im Einsatz. Zu diesen Systemen gehören das *Network File System (NFS)* der Firma Sun und das *Andrew File System (AFS)* der Carnegie Mellon University. *Novell Netware*, das klassische *Windows for Workgroups* und *Windows NT / 2000* ermöglichen ebenfalls Dateizugriffe über ein Netz.

9.1.1 Grundlegende Ansätze

Frühere Kapitel befassten sich mit einem „lokalen“ Dateisystem. Es war auf einer „lokalen“ Platte innerhalb eines Rechners realisiert und nur für diesen Rechner zugänglich. Im Folgenden geht es zusätzlich um die Benutzung *entfernt liegender Dateien* (engl. *remote files*). Das heißt, dass Dateien nunmehr auch auf Platten anderer Rechnerknoten stehen und über das Kommunikationsnetz zugegriffen werden können (siehe Abb. 9.1).

In einem solchen System kann es sein, dass viele Rechnerknoten ihre lokalen Platten nur sehr eingeschränkt benutzen – beispielsweise lediglich für das Paging, temporäre Dateien und eventuell Dateien mit Betriebssystem-Code. Alle anderen Dateien werden auf Platten anderer Knoten gespeichert, die damit zu *File Servern* werden. Im Extremfall besitzt ein Computer überhaupt keine lokale Platte, ist also eine *Diskless Workstation*.

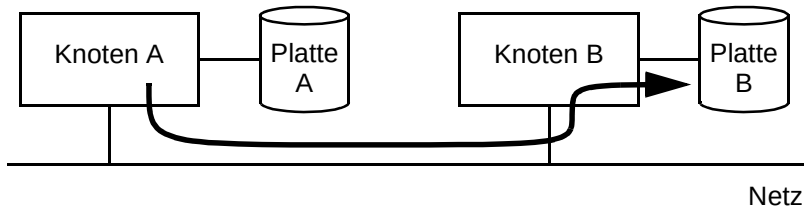


Abb. 9.1 Zugriff auf eine entfernte Datei

Das Ziel bei der Realisierung eines solchen verteilten Dateisystems ist, dass alle Knoten auf sämtliche Dateien zugreifen können, unabhängig davon, wo diese Dateien liegen. Ein Zugriff auf entfernt liegende Dateien sollte mit denselben Befehlen möglich sein wie ein Zugriff auf lokale Dateien.

Abbildung 9.2, Teil a, skizziert die Grundstruktur eines verteilten Dateisystems: Im Rechnernetz gibt es einen oder mehrere *File-Server-Rechner*, die auf ihren Festplatten jeweils Teile des Dateisystems speichern und nach außen anbieten. Client-Rechner können diese Dateien benutzen, indem sie entsprechende Anforderungen an den betreffenden Server senden. Der Server greift dann auf seine Platte zu und schickt eine Antwort zurück – z.B. bei einem Lesezugriff die gewünschten Daten.

Zur Identifikation der Dateien sollten möglichst systemweit bekannte, einheitliche Namen benutzt werden, die unabhängig vom Ort der Dateien sind. Das Betriebssystem erkennt, ob ein Zugriff einer lokalen oder einer entfernt liegenden Datei gilt, und setzt ihn gegebenenfalls in eine Anforderung an den entsprechenden Server um. Unter dieser Voraussetzung entsteht also ein verteiltes Dateisystem, das für alle Rechnerknoten im Netz einheitlich aussieht und zudem „ortstransparent“ ist, da ein Benutzer nicht wissen muss, wo die Dateien gespeichert sind. In der Praxis muss man allerdings manchmal Abstriche von diesen Eigenschaften machen, wie sich später zeigen wird.

Um ein verteiltes Dateisystem zu realisieren, werden zwei Dienste benötigt:

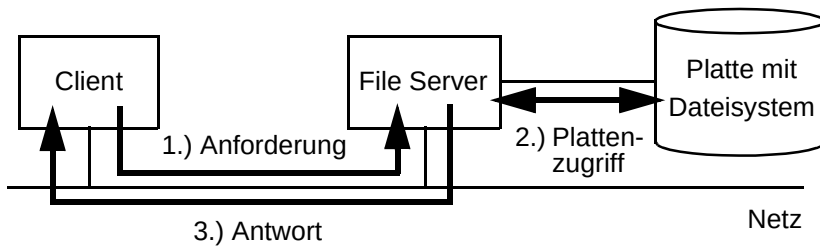
- Der *File Service* (Dateidienst) ist, wie bereits geschildert, für die Speicherung der Dateien zuständig und ermöglicht Operationen auf einzelnen Dateien. Er wird realisiert durch File-Server-Knoten.
- Ein *Directory Service* (Verzeichnisdienst) organisiert einen *Namensraum* zur eindeutigen Identifizierung der Dateien und bietet damit einen *Name Service* (Namensdienst). Zudem stellt er die Informationen bereit, die zum Auffinden der Dateien benötigt werden. Er wird realisiert durch Directory- bzw. Name-Server-Knoten.

9.1.2 Strukturen und Zugriffe

Ein Dateiverzeichnis in einem verteilten System hat im Wesentlichen dieselben Aufgaben wie ein Verzeichnis im lokalen System:

- Die Menge der Dateien muss strukturiert werden, z.B. durch einen Dateibaum.

a.) Dateizugriff über einen File Server:



b.) Dateizugriff mit vorherigem Zugriff auf einen Directory Server:

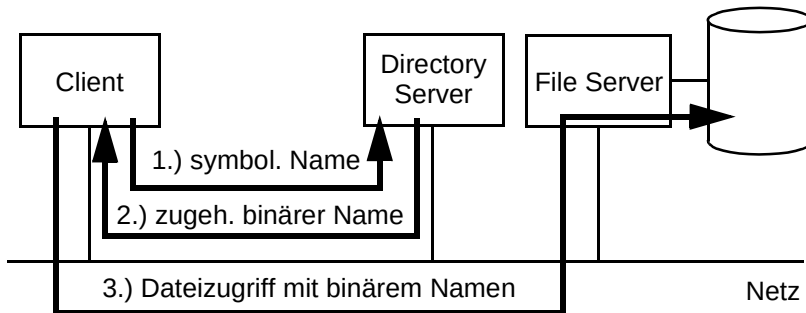


Abb. 9.2 Dateizugriffe über File Server und Directory Server

- Externe („symbolische“) Namen müssen auf interne („binäre“) Namen abgebildet werden, um Dateien intern lokalisieren und verwalten zu können – ähnlich wie es bei der Zuordnung von Pfadnamen zu Inode-Nummern geschieht.
- Es müssen Dateiattribute mit Informationen über die Dateien geführt werden.

Im verteilten System sind die zugehörigen Mechanismen allerdings schwieriger zu realisieren als bei lokaler Dateiverwaltung. Sie werden, wie bereits gesagt, durch Directory Server implementiert. Es können mehrere Directory Server vorhanden sein, die jeweils Teilm Informationen über das Dateisystem enthalten.

Schritte eines Dateizugriffs

Mit der Trennung von Directory und File Service besteht ein Dateizugriff aus zwei Stufen (siehe Abb. 9.2, Teil b): Im ersten Schritt schickt ein Client einen symbolischen Dateinamen an den Directory Server und erhält von diesem den zugehörigen binären Namen, also unter anderem die Information, auf welchem File Server sich die Datei befindet. Der Client nimmt also den Directory Server insbesondere auch als Name Server in Anspruch. Im zweiten Schritt greift er dann mit dem binären Namen über den entsprechenden File Server auf die Datei zu.

Directory Server und File Server sind zwar logisch voneinander getrennt, können aber durchaus auf demselben Rechnerknoten liegen. Während eine räumliche Trennung eine größere Flexibilität erlaubt (z.B. die Führung der Namensinformationen nahe bei den Clients), kann hierdurch der Aufwand bei einem Dateizugriff steigen. Das ist insbesondere dann der Fall, wenn die Namensinformationen auf mehrere Directory Server verteilt sind und sich somit der Client durch mehrere Server „hindurchfragen“ muss.

Während damit das Zusammenspiel zwischen Client und Directory Server in groben Zügen klar ist, sind konkrete Fragen zur Realisierung noch offen: Wie sehen symbolische Dateinamen überhaupt aus, wie werden sie erzeugt und in einem Verzeichnis registriert, und wie werden symbolische auf binäre Namen abgebildet?

Symbolische Dateinamen

Ein externer (symbolischer) Dateiname ist im Allgemeinen eine Pfadangabe in einem baumartig strukturierten Namensraum, wie er von lokalen Systemen her bekannt ist. In einem verteilten System kann man Dateibäume wie folgt organisieren (siehe Abb. 9.3):

- Die einfachste Möglichkeit ist, auf jedem Knoten einen *eigenen lokalen Dateibaum* zu führen, der unabhängig von den Bäumen anderer Knoten ist. Lokale Dateien werden, wie bisher, durch ihre Pfade im lokalen Dateibaum identifiziert. Zugriffe auf Dateien anderer Knoten sind möglich, indem im symbolischen Namen zuerst der Name des Rechnerknotens und dann der Pfad der Datei in dessen Baum angegeben wird. UNIX unterstützt beispielsweise in seinem `rcp`-Kommando (= remote copy) diesen Ansatz. Ein weiteres Beispiel ist Windows NT mit der Möglichkeit, auf „Netzlaufwerke“, d.h. Ressourcen entfernter Rechnerknoten, zuzugreifen.

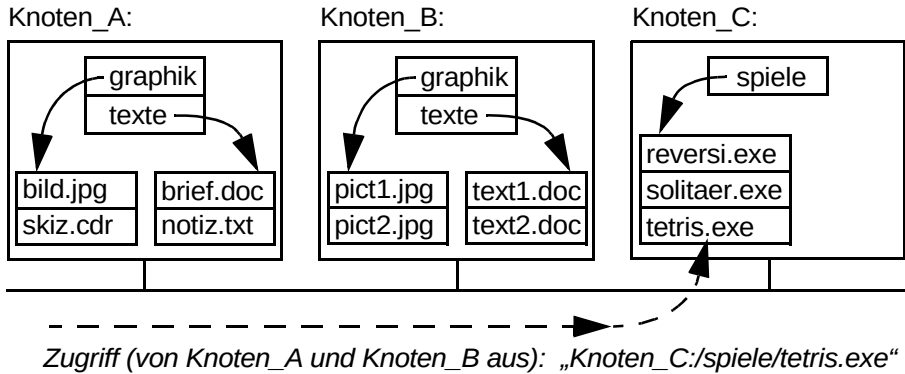
Nachteilig ist hier die fehlende Ortstransparenz: Ein Benutzer muss wissen, auf welchem Knoten eine bestimmte Datei liegt, da er diese Information für den symbolischen Namen benötigt. Insbesondere erschwert das die Verlagerung von Dateien auf andere Knoten.

- Der zweite Ansatz, das so genannte *Mounting*, beruht auf File-Server-Knoten, die ihre Dateibäume den anderen Knoten ganz oder teilweise anbieten. Clients können diese (Teil-)Bäume in ihre lokalen Dateibäume „einbinden“ (*mounten* = „anmontieren“), also in ihre Verzeichnisse Verweise auf entfernt liegende Bäume eintragen. Hierfür wird ein `mount`-Kommando des Betriebssystems benutzt (etwa der Form „`mount <externer (Unter-)Baum> <lokales Verzeichnis>`“), das, grob gesagt, einen Link auf einen anderen Rechner anlegt. Nach seiner Ausführung kann also unter Angabe eines lokalen Pfads auf entfernt liegende Dateien zugegriffen werden. Beispielsweise geht Sun NFS, das weiter unten besprochen wird, so vor.

Nachteilig ist hier, dass jeder Client selbst das Einbinden vornehmen muss, wodurch sich auf den einzelnen Clients unterschiedliche Verzeichnisstrukturen ergeben können und die Gefahr eines Wildwuchses im Netz wächst.

- Der dritte und anspruchsvollste Ansatz realisiert einen *globalen Dateibaum*, der systemweit von allen Knoten aus gleich aussieht. Wie bisher werden zwar die einzelnen

Möglichkeit 1: lokaler Dateibaum für jeden Knoten



Möglichkeit 2: „Mounting“ von Verzeichnissen eines Servers

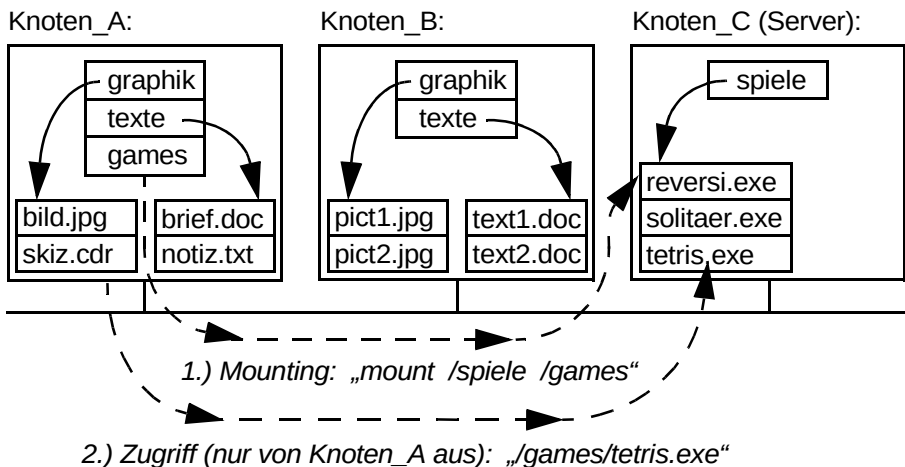


Abb. 9.3a Organisation verteilter Dateisysteme

Dateien auf den lokalen Knoten gespeichert, die Verwaltung des darüber liegenden Dateibaums ist jetzt aber Sache des verteilten Betriebssystems – konkret des oder der Directory Server. Während also bei den beiden anderen Ansätzen Directory Server eine untergeordnete oder gar keine Rolle spielen, kommen sie hier voll zum Einsatz. Das Andrew File System AFS (siehe Ende dieses Abschnitts) ist ein Beispiel für ein solches System.

Wie binäre Dateinamen konkret aussehen, hängt vom jeweiligen System ab. Wir werden hierauf im nächsten Teilabschnitt noch zurückkommen.

Möglichkeit 3: globaler Dateibaum

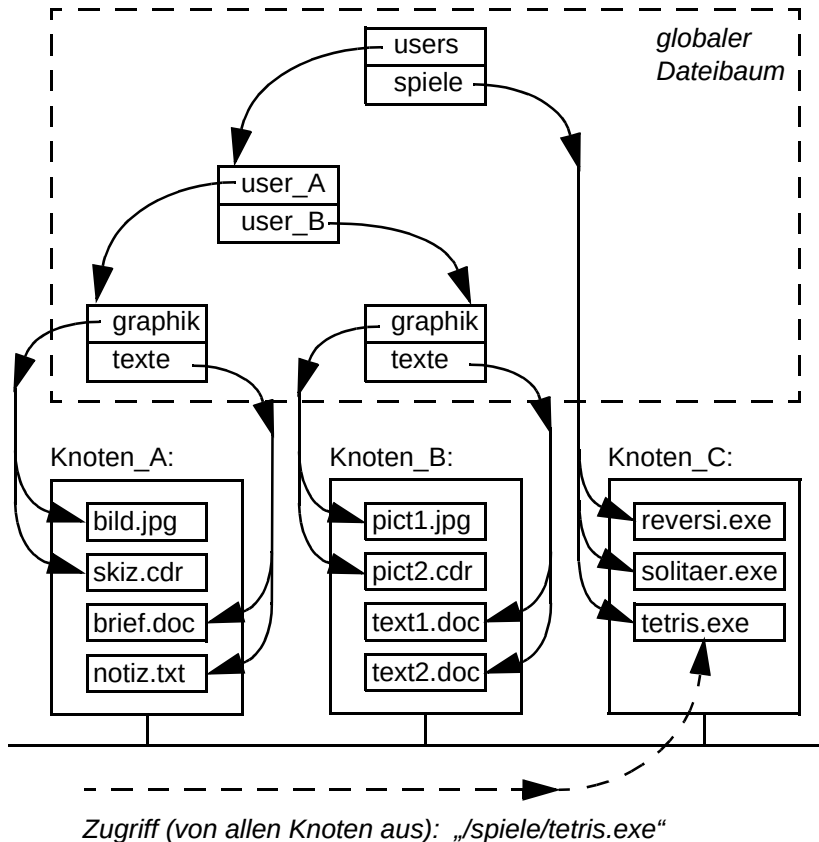


Abb. 9.3b Organisation verteilter Dateisysteme

Name Server

Dateinamen werden erzeugt und registriert mit Hilfe eines Directory Servers, der hierfür *Name-Server-Dienste* anbietet. Solche Name Services gibt es nicht nur für Dateien, sondern generell für alle global verfügbaren Objekte in einem verteilten System.

Wie ein Name Server bei der Namensvergabe vorgeht, ist in Abbildung 9.4 skizziert: Ein Prozess erzeugt eine Datei (oder allgemein: ein Objekt), gibt ihm einen symbolischen Namen (sofern er die Regeln zur Erzeugung solcher Namen kennt) und wendet sich dann an den Name Server. Dieser prüft, ob der Name korrekt ist, oder er bildet bei Bedarf selbst einen solchen Namen. Anschließend registriert er in einer Tabelle den symbolischen Namen zusammen mit einem zugehörigen binären Namen, der Zugriffsinformationen für das Objekt enthält oder auf solche Informationen verweist. Der Name Server bestätigt dem Prozess die Registrierung oder gibt eine Fehlermeldung.

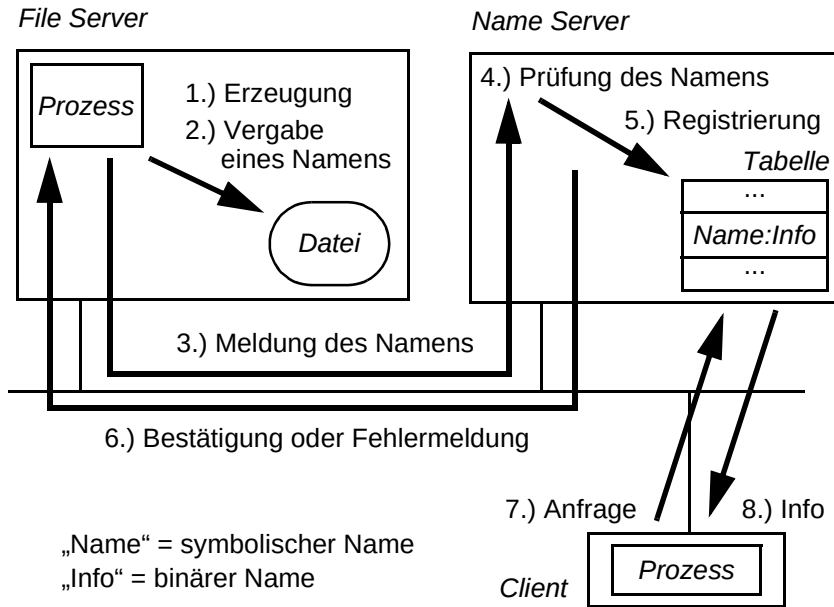


Abb. 9.4 Aktionen des Name Service

Nachdem der Name erzeugt und registriert wurde, kann er verteilt werden: Clients fragen beim Name Server an, wo bestimmte Daten abgelegt sind oder wer einen bestimmten Dienst anbietet, und der Name Server erteilt die gewünschten Informationen.

Damit verläuft die Abbildung von symbolischen auf binäre Dateinamen ähnlich wie in lokalen Systemen: Der Directory Server führt Listen mit Paaren aus symbolischen und binären Namen, die er bei einer Anfrage des Clients durchsucht. Der binäre Name, der sich dabei ergibt, kann beispielsweise ein Paar aus einem Server-Identifikator und der Inode-Nummer auf diesem Server sein. Statt der Server-Identifikation kann auch ein Verweis auf einen Eintrag einer Indirektionstabelle gespeichert werden. Erst dieser Eintrag gibt den entsprechenden Server an; er kann leicht geändert werden, wenn (z.B. bei Ausfall des ursprünglichen Servers) ein anderer Server angesprochen werden soll. Zusätzlich kann der binäre Name wie eine Capability Zugriffsberechtigungen festlegen.

9.1.3 Caching und Replikation

Neben den grundlegenden Diensten, wie File und Directory Service, sind in einem verteilten Dateisystem weitere Techniken wichtig. Dazu gehört das *Caching*, mit dem der lange Weg vom Client zu den Server-Dateien (bestehend aus Plattenzugriff und Netzübertragung) verkürzt werden soll (siehe Abb. 9.5). Hierbei werden Kopien von häufig gebrauchten Dateiblöcken „näher“ beim Client gespeichert, wofür es im Wesentlichen zwei Möglichkeiten gibt:

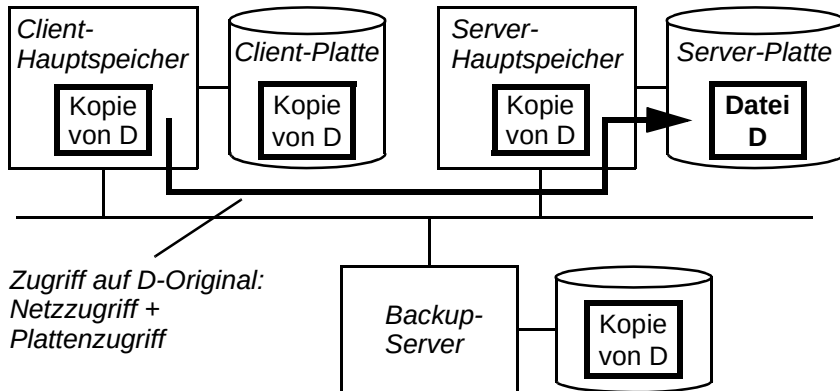


Abb. 9.5 Datei-Caching und Replikation

Wie in einem lokalen System können Dateiblöcke im Hauptspeicher des Servers abgelegt werden. Das ist leicht zu implementieren, hat aber den Nachteil, dass weiterhin eine zeitaufwendige Netzübertragung zum Client nötig ist. Alternativ können daher Blöcke (oder sogar ganze Dateien) in den Hauptspeicher oder auf die Platte des Clients kopiert werden, wo sie für diesen rasch zugreifbar sind. Problematisch ist hier aber die Erhaltung der *Dateikonsistenz*, wenn mehrere Clients gleichzeitig dieselbe Datei bearbeiten: Schreibzugriffe eines Client haben nämlich nur unmittelbare Auswirkungen auf seine eigene lokale Kopie, nicht jedoch auf die Kopien der anderen Clients, so dass sich unterschiedliche Dateiversionen ergeben. Zur Sicherstellung der Konsistenz, d.h. zum Abgleich der einzelnen Kopien, gibt es spezielle Mechanismen, die beispielsweise Informationen über Änderungen verteilen.

Ebenso wichtig wie das Caching ist die *Replikation*, bei der Kopien einer Datei auf mehreren Servern abgelegt werden (siehe Abb. 9.5). Erstens erhöht sich hierdurch die Fehlertoleranz des Systems gegenüber Server-Ausfällen, zweitens kann die Last der Dateizugriffe auf mehrere Knoten verteilt werden, und drittens verkürzen sich die Zugriffswege der Clients, da sie jeweils den nächstliegenden Server ansprechen können. Auch hier werden Techniken zur Erhaltung der Konsistenz benötigt.

9.1.4 Beispiele verteilter Datei- und Namensdienste

Verteilte Dateisysteme und Namensdienste spielen in Rechnernetzen eine zentrale Rolle. In der Praxis werden entsprechende Produkte schon seit vielen Jahren eingesetzt.

Sun NFS

Das *Network File System (NFS)* der Firma Sun ist Bestandteil von *ONC+* (ONC = Open Network Computing) [Sun01]. Es erweitert die Sun-Betriebssysteme SunOS und Solaris zu Netzbetriebssystemen, kann auch in den UNIX-Versionen vieler anderer Herstel-

ler eingesetzt werden und ist als Internet-Dienst standardisiert (siehe [RFC1094], [RFC1813] und [RFC2624]). Die Hardwarebasis von NFS ist eine Anzahl von Workstations, die miteinander vernetzt sind. Die Workstations können heterogen sein, d.h. es können neben Sun-Computern auch Rechner anderer Hersteller eingebunden werden, und zudem müssen Clients nicht unbedingt unter Solaris laufen. Nicht jeder Rechner muss eine eigene Platte besitzen, da NFS Diskless Stations unterstützt.

NFS realisiert eine Client-Server-Architektur mit „Mounting“: File Server *exportieren* ihre Dateibäume (oder nur Teile davon), stellen sie also nach außen zur Verfügung. Nur explizit exportierte Bäume können außerhalb eines Servers benutzt werden. Clients machen sie sich zugreifbar, indem sie sie an beliebige Stellen ihrer lokalen Dateibäume einbinden. Ein Extremfall sind hier die Diskless Stations, die lokal nur ein Wurzelverzeichnis, aber keine Dateien besitzen, also alles von den Servern importieren. Im Prinzip kann jeder Knoten ein Client, ein Server oder auch beides sein.

Zur Kommunikation zwischen Clients und Servern gibt es zwei Protokolle, das *Mounting-Protokoll* und das *Zugriffs-Protokoll*: Das Mounting-Protokoll wird zum Einbinden von Dateibäumen benutzt. Hier sendet ein Client einem Server einen Server-Pfadnamen und gibt damit einen (Teil-)Baum an, den er bei sich einbinden möchte. Voraussetzung dafür ist, dass der Server diesen Teilbaum explizit exportiert, d.h. dessen Wurzelverzeichnis in die Datei `/etc/exports` (oder eine ähnliche Datei anderen Namens) eingetragen hat. Der Server gibt dem Client einen *File Handle* zurück, der unter anderem die Inode-Nummer des gewünschten Verzeichnisses im Server-Dateisystem enthält und im Folgenden für Zugriffe auf Server-Dateien benutzt werden kann.

Für das Einbinden hat der Client drei Möglichkeiten: Erstens kann es „per Hand“ geschehen, d.h. durch die Eingabe eines `mount`-Kommandos über die Tastatur. Zweitens können Dateibäume beim Hochfahren des Systems automatisch eingebunden werden, was durch `mount`-Befehle in der Kommandodatei `/etc/rc` (oder Einträge einer ähnlichen Datei anderen Namens) gesteuert wird. Drittens ist ein *Automounting* möglich, das das Einbinden erst bei einem Dateizugriff versucht. Diese Technik ermöglicht eine gewisse Fehlertoleranz, da ein solcher Versuch hintereinander bei mehreren Servern gemacht werden kann, also auch beim Ausfall eines Servers noch zum Erfolg führt.

Für Zugriffe auf Dateien und Verzeichnisse wird das Zugriffsprotokoll verwendet. Die meisten UNIX-Systemaufrufe zur Dateibenutzung werden durch NFS auf dieses Protokoll abgebildet und damit automatisch über das Netz umgelenkt, falls erforderlich. Das Protokoll besteht aus drei Schichten (siehe Abb. 9.6): Der *System Call Layer* bietet nach oben eine Schnittstelle mit Systemfunktionen an und bildet Pfadnamen, die an diese Funktionen übergeben werden, auf *Vnodes* ab. Ein Vnode (= virtual inode) ist entweder ein lokaler Inode, falls sich die betreffende Datei auf dem Client-Knoten befindet, oder ein File Handle für eine Datei auf einem Server. Eine Ebene tiefer stellt das *Virtual File System (VFS)* fest, um welche Art von Vnode es sich handelt, und gibt die Anforderung dann entweder an das lokale Dateisystem oder an den darunter liegenden *NFS Client* weiter. Der schickt sie dann über das Netz an den entsprechenden Server. Auf der Server-Seite nimmt der *NFS Server* die Anforderung entgegen und übergibt sie an das VFS des Servers, das sie wie eine lokale Anforderung behandelt. Die Antwort wird auf demselben Weg zurückübertragen. Durch diese Vorgehensweise erreicht man also an der Systemschnittstelle eine Ortstransparenz, da erst das VFS lokale und entfernt liegende Dateien unterscheidet.

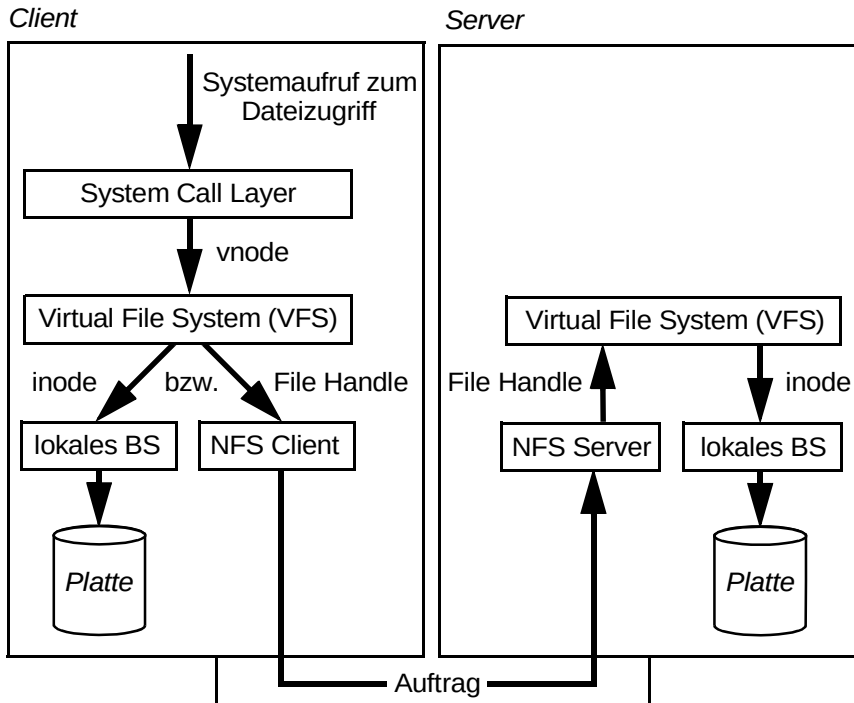


Abb. 9.6 NFS-Zugriffsprotokoll (nach [Tan92])

NFS beschleunigt Dateizugriffe durch ein Caching von Dateiblöcken in Client und Server. Um die Gefahr von Inkonsistenzen zu verringern, überträgt ein Client in festgelegten Zeitabständen (typischerweise alle 30 Sekunden) sämtliche geänderten Dateiblöcke an den Server. Zudem fragt er beim Öffnen einer Datei, die sich in seinem Cache befindet, beim Server nach, ob seine Kopie noch aktuell ist – wenn nein, aktualisiert er sie. Zwar sind diese Maßnahmen kein vollkommener Schutz gegen Inkonsistenzen, machen sie aber weniger wahrscheinlich und führen nur zu einem relativ geringen Mehraufwand.

Weiterhin wird die Effizienz dadurch gesteigert, dass Daten in größeren Blöcken transportiert werden und dass nach dem Eintreffen eines Blocks bereits der nächste Block angefordert wird („read ahead“). Durch diese Techniken werden Diskless Stations ebenso effizient wie Workstations mit einer eigenen Platte, vorausgesetzt dass das Netz nicht überlastet ist.

Zu den weiteren NFS-Diensten gehört die gemeinsame Benutzung von Dateien (engl. *file sharing*), die sich automatisch ergibt, wenn mehrere Clients dieselbe Datei bei sich einbinden. Mit den Kommandos `rsh` und `rlogin` können Benutzer Aktionen auf entfernt liegenden Rechnern ausführen bzw. sich dort einloggen. Der *Network Information Service* (NIS, früher *Yellow Pages* genannt) bietet eine Sammlung von symbolischen Service-Namen und den zugehörigen Netzadressen an.

Active Directory Service in Windows 2000

Microsoft ergänzt seine Windows-Betriebssysteme fortlaufend um Dienste zum Einsatz in Rechnernetzen. So wurden als integraler Bestandteil von Windows 2000 die Verzeichnisdienste *Active Directory Services (ADS)* eingeführt (im Folgenden dargestellt nach [Gall00]). Die ADS sollen Systemverwalter und Benutzer bei der Buchführung über und beim Auffinden von Ressourcen in einem Rechnerverbund unterstützen. Sie ersetzen damit das *Domänen-Modell* von Windows NT, das auf die Verwaltung von Benutzerkonten und Zugriffsrechten in einem Rechnernetz beschränkt war.

Mit ADS wird versucht, eine gewisse Ordnung in die Vielzahl von Objekten zu bringen, die es in einem Rechnerverbund gibt. Unter einem Objekt wird dabei im Prinzip eine beliebige Komponente des Systems verstanden, also beispielsweise eine Datei mit ihrem Inhalt und ihren Eigenschaften, ein Benutzer mit seinen Rechten und sonstigen Attributen oder ein Router mit seinen Diensten zur Datenübertragung. Ziel ist, die verschiedenartigen Anwendungen und Dienste des Rechnerverbunds (z.B. Mail, File Service, WWW-Dienste) durch einen einheitlichen Verzeichnisdienst zu unterstützen.

Die Active Directory Services bieten die Möglichkeit, Objekten symbolische Namen zu geben. Der Namensraum ist an den des Internets angelehnt (siehe Abschnitt 8.2.6): Er ist baumartig aufgebaut und erlaubt die Realisierung von mehrstufigen hierarchischen Domänen (*Domains*), die wiederum in Organisationseinheiten (*Organizational Units, OUs*) untergliedert sein können. Wie im Internet bildet ein *Domain Name System (DNS)* Namen auf Adressen ab, so dass Objekte lokalisiert werden können.

Objekte können nicht nur über ihre Namen zugegriffen werden, sondern es sind auch Suchoperationen wie in einer Datenbank möglich. Ein *Index Server* unterstützt eine indizierte Suche im Objektraum und liefert seine Ergebnisse in Form einer HTML-Seite, die mit Hilfe eines Browsers betrachtet werden kann. Dabei wird kein Unterschied zwischen der Darstellung von lokalen und entfernt liegenden Ressourcen gemacht.

Ein wichtiger Anwendungsbereich von ADS ist das verteilte Dateisystem von Windows 2000. Unter anderem bietet ADS Mechanismen zur Dateireplikation: Von einer Datei oder einem Verzeichnis können an verschiedenen Orten im Netz gleichwertige Kopien gespeichert sein. Ein *Change Journal* führt Buch über die Änderungen an den Kopien – sowohl bezüglich der Inhalte als auch der Attribute. Die Einträge des Journals sind mit eindeutigen *Update Sequence Numbers (USNs)* versehen, die in aufsteigender Reihenfolge vergeben werden und somit ermöglichen, die Ereignisse in eine zeitliche Reihenfolge zu bringen. Mit Hilfe des Change Journals kann ein *Replication Server* die Kopien einer Datei oder eines Verzeichnisses auf einen einheitlichen aktuellen Stand bringen.

Die Effizienz des Replikationsdienstes hängt von der Häufigkeit ab, mit der Dateiänderungen vorgenommen werden. Nach [Gall00] wird angenommen, dass nur ein Prozent aller Dateizugriffe Änderungen verursachen, also 99 Prozent Lesezugriffe sind, über die nicht Buch geführt werden muss. Daten, die oft geändert werden, werden in einem gesonderte *Volatile Storage* abgelegt, auf den andersartige Replikationsmechanismen angewandt werden. Wie die tatsächliche Leistung in der Praxis aussieht, wird sich allerdings noch zeigen müssen.

Weitere Beispiele

Neben den aufgeführten Beispielen gibt es noch eine Reihe weiterer verteilter Dateisysteme und Verzeichnisdienste. Hierzu gehört das schon genannte *Andrew File System* (AFS). Es hat zum Ziel, ein weltweit verfügbares, leicht erweiterbares Dateisystem für eine große Anzahl von Clients zu schaffen und dabei einen globalen, für alle Clients identisch aussehenden Dateibaum zur Verfügung zu stellen. Es stützt sich stark auf Caching-Techniken, um die Server eines solch großen Systems zu entlasten. AFS war und ist recht erfolgreich: Bereits 1994 waren 130 Institutionen in zehn Ländern durch AFS miteinander verbunden, so dass schon damals ein einheitlicher Dateinamensraum für ca. 1000 Server und 20000 Clients existierte [SpSa96].

Das *Remote File System* (RFS) ist eine UNIX-System-V-Erweiterung der Firma AT&T. *Novell Netware* ist ein Netzbetriebssystem, das verteilte Dateisysteme unterstützt und hierfür unter anderem den Verzeichnisdienst *Network Directory Service* (NDS) bereitstellt.

9.2 Kommunikation und Kooperation

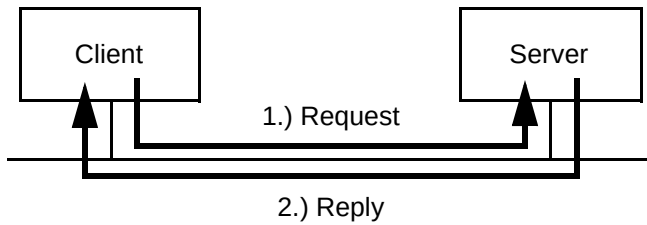
Damit Rechnerknoten bzw. Prozesse in einem verteilten System kooperieren können, müssen Kommunikationsmechanismen als Basisdienst bereitgestellt werden. Die Kommunikation in einem verteilten System kann sich nicht auf einen gemeinsamen, rasch zugreifbaren Hardwarespeicher stützen wie in Ein-Prozessor-Systemen, sondern es müssen Nachrichten über ein Kommunikationsnetz ausgetauscht werden (siehe aber die Diskussion in Abschnitt 4.2.2 unter dem Stichwort *Distributed Shared Memory*).

Im Folgenden sollen zwei Ansätze für den Nachrichtenaustausch betrachtet werden, die auf unterschiedlichen Abstraktionsniveaus angesiedelt sind: Erstens ein *allgemeines Client-Server-Modell* auf der Basis von send- und receive-Operationen, zweitens die *Remote-Procedure-Call-Technik* als eine komfortable Realisierungsmöglichkeit für Client-Server-Strukturen. Im Mittelpunkt werden dabei betriebssystemspezifische Aspekte stehen, die sich auf gegebene Kommunikationsnetze und -protokolle stützen (siehe hierzu Unterkapitel 8.2).

9.2.1 Client-Server mit Request-Reply

Client-Server ist ein allgemeines Modell für die Kooperation in einem verteilten System: Hier wird das System als eine Menge kooperierender Prozesse aufgefasst, von denen einige als Server bestimmte Dienste anbieten (z.B. File Server, Name Server, Print Server, Database Server) und andere als Clients diese Dienste in Anspruch nehmen. Die Rollen sind dabei aber nicht streng getrennt: Ein Prozess kann durchaus Server für bestimmte Dienste und gleichzeitig Client für andere Dienste sein. Client-Server-Strukturen wurden in den vorangehenden Kapiteln schon mehrfach angesprochen.

a.) einfacher Request-Reply-Vorgang:



b.) Request-Reply mit Benutzung eines Name Servers:

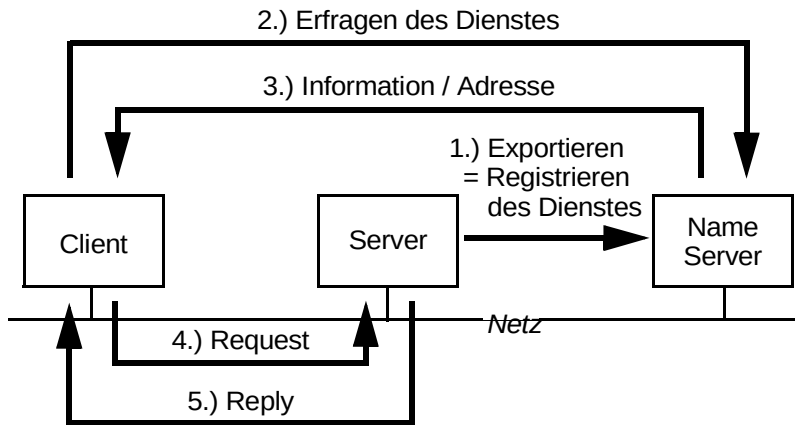


Abb. 9.7 Request-Reply-Protokoll

Request-Reply-Protokoll

Zur Implementierung von Client-Server-Systemen kann ein einfaches *Request-Reply-Protokoll* verwendet werden, das nur aus zwei Schritten besteht (siehe Abb. 9.7, Teil a): Ein Client schickt eine Anforderung (engl. *request*) an einen Server, womit er einen bestimmten Dienst anfordert. Der Server bearbeitet den Auftrag und schickt dann seine Antwort (engl. *reply*) zurück.

Der Vorteil dieses Ansatzes liegt in seiner Einfachheit und Effizienz: Wie früher besprochen, werden nur die beiden Systemaufrufe `send()` und `receive()` benötigt, und durch den Verzicht auf einen komplexeren Protokoll-Stack wird der Zeitaufwand gering gehalten.

Name Service

Eine zentrale Frage beim Client-Server-Modell ist, woher der Client weiß, wie er den Server adressieren soll. Am komfortabelsten ist hier der Einsatz eines *Name Servers*, also eines speziellen Rechnerknotens oder Prozesses, der ein Verzeichnis der angebo-

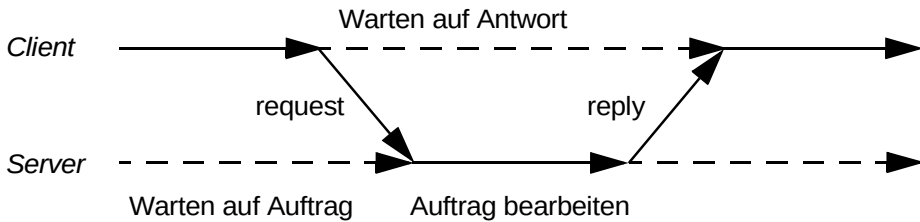


Abb. 9.8 zeitliches Verhalten von Request-Reply

tenen Dienste führt (siehe Abb. 9.7, Teil b). Wie schon in Abschnitt 9.1.2 für verteilte Dateisysteme dargestellt, lässt jeder Server seine Dienste bei einem Name Server registrieren, „exportiert“ sie also. Benötigt ein Client einen bestimmten Dienst, so wendet er sich an den Name Server, der ihm die Adresse eines Servers zurückgibt. Zur eigentlichen Auftragsbearbeitung wird dann, wie oben dargestellt, das Request-Reply-Protokoll verwendet.

Die Benutzung eines Name Servers hat mehrere Vorteile: Er erhöht die Transparenz im System, denn ein Client muss nicht von vornherein wissen, wo ein Server für einen bestimmten Dienst liegt. Zudem unterstützt er die Flexibilität, da seine Einträge leicht dynamisch geändert werden können, was insbesondere bei Server-Ausfällen wichtig ist. Schließlich können Sicherheitsmechanismen implementiert werden, die alle unberechtigten Anfragen zurückweisen. Nachteilig ist der zusätzliche Zeitaufwand für die Kommunikation mit dem Name Server. Zudem ist der Name Server eine zentrale Systemkomponente, macht das System also fehleranfällig und ist zudem ein möglicher Engpass. Dem kann allerdings durch Benutzung mehrerer Name Server mit replizierten Informationen begegnet werden.

Mit dem Request-Reply-Protokoll lässt sich also das Client-Server-Modell effizient implementieren. Allerdings ist das Abstraktionsniveau recht niedrig, so dass die Programmierung mit Protokollaspekten befrachtet wird. Im Folgenden soll daher ein abstrakterer Ansatz betrachtet werden.

9.2.2 Remote Procedure Call

Abbildung 9.8 zeigt das zeitliche Verhalten bei der Kooperation zwischen Client und Server: Der Client schickt einen Auftrag mit Parametern an den Server, wartet seine Erledigung ab und arbeitet dann mit den Resultaten weiter. Vergleicht man das mit einem Funktionsaufruf in einem (lokalen) Programm, so erkennt man deutlich eine Gemeinsamkeit: In beiden Fällen wird das aufrufende Programm nicht weiter ausgeführt, bis die gerufene Funktion beendet ist. Es ist also möglich und sinnvoll, den Aufruf von Server-Diensten auch syntaktisch an einen lokalen Funktionsaufruf anzulehnen, ihn nämlich in der Form `funktionsname(parameterliste)` zu beschreiben. Damit wird auf ein Programmkonstrukt zurückgegriffen, das von Ein-Prozessor-Systemen her vertraut ist und somit die Programmierung für verteilte Systeme erleichtert. Man bezeichnet diese Technik mit *Remote Procedure Call* (RPC, *Fernaufruf*).

RPC-Semantik

Das Ziel bei der Implementierung der RPC-Technik ist, nicht nur die Syntax, sondern auch die Semantik eines Fernaufrufs, d.h. seine Auswirkungen und Möglichkeiten, der eines lokalen Aufrufs anzugleichen. Hierbei gibt es jedoch mehrere Probleme:

- Ein RPC wird im Adressraum des Servers ausgeführt. Dieser liegt im Allgemeinen auf einem anderen Rechnerknoten als der Adressraum des Clients, so dass die Adressräume vollständig voneinander getrennt sind. Das erschwert die Übergabe von Referenzparametern (= Pointern / Adressen).
- Server und Client sind möglicherweise heterogen, haben also eine unterschiedliche Hardware, und codieren damit dieselben Werte durch unterschiedliche Bitmuster. Eine Parameterübergabe durch einfaches Kopieren der Bits ist dann nicht möglich.
- Im verteilten Fall können Fehler auftreten, die im lokalen Fall keine Rolle spielen – beispielsweise der Ausfall des Servers oder Störungen bei der Datenübertragung. Solche Fehler müssen geeignet behandelt werden.

In der Praxis gibt es verschiedene Lösungsansätze für diese Probleme, die aber alle nicht die volle Semantik eines lokalen Funktionsaufrufs durchsetzen. Sie sollen im Folgenden nicht näher betrachtet werden (siehe hierzu z.B. [Web98] oder [Tan92]). Wir beschäftigen uns stattdessen mit zwei allgemeinen Aspekten der RPC-Implementierung, nämlich der technischen Realisierung eines Fernaufrufs und dem Einbinden von Server-Funktionen beim Client.

Schritte beim RPC

Abbildung 9.9 illustriert die Schritte, die bei einem Fernaufruf ausgeführt werden. Ein Client ruft eine entfernte Funktion syntaktisch wie eine lokale Funktion auf, indem er den Funktionsnamen und eine Liste aktueller Parameter angibt. Durch den Aufruf wird ein *Procedure Stub* (wörtliche Übersetzung: „Prozedurstumpf“) ausgeführt, der in einer lokalen Bibliothek des Clients steht.

Für jede Server-Funktion, die der Client benutzen darf, gibt es einen eigenen Stub. Der Stub ist zuständig für die Kommunikation mit dem Server und verbirgt so dem Client-Programm gegenüber, dass es sich um den Aufruf einer entfernt liegenden Funktion handelt. Er erzeugt eine Request-Nachricht, die unter anderem die Identifikation der Funktion und die Parameter in einem bestimmten Format enthält, und schickt sie über das darunter liegende Transportsystem an den Server-Knoten.

Das Transport-System des Servers reicht die Nachricht an einen Server-Stub (manchmal auch *Skeleton* genannt) weiter, der sie „auspackt“ und lokal die gewünschte Server-Funktion aufruft. Nach Beendigung der Funktionsausführung erzeugt der Server-Stub eine Reply-Nachricht, die u.a. die Ergebnisse enthält, und sendet sie über das Transportsystem an den Client-Stub. Dieser packt die Ergebnisse aus und übergibt sie – wie bei der Wertrückgabe einer lokalen Funktion – an das Client-Programm, das dann weiterlaufen kann.

Client:

Server:

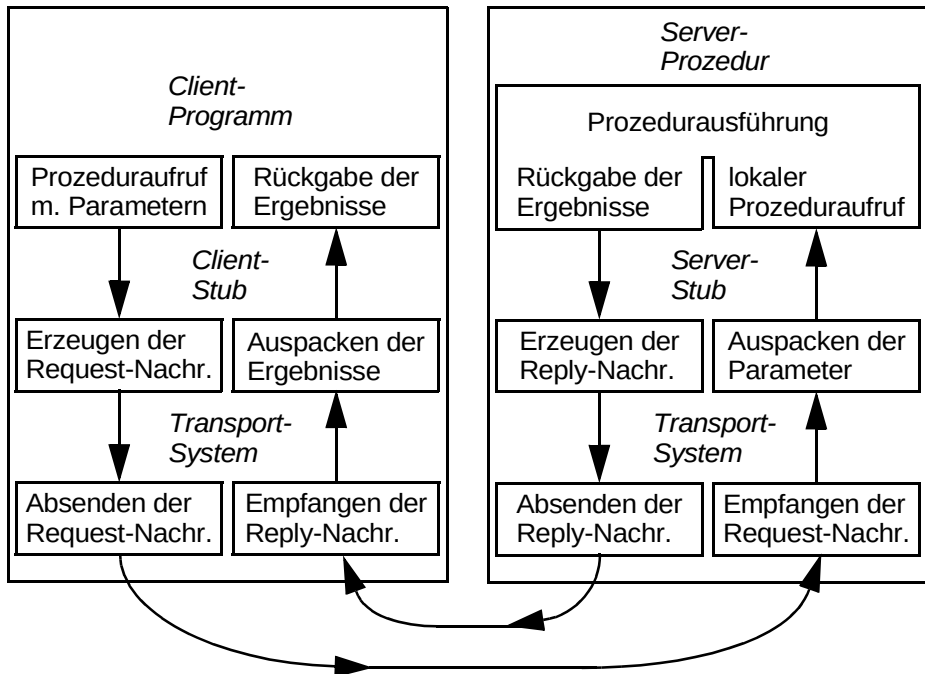


Abb. 9.9 Schritte beim Fernaufruf (RPC)

Um eine Server-Funktion aufrufen zu können, muss ein Client sie zuvor bei sich „einbinden“. Hierzu fragt der Client bei einem Name Server an, wo ein bestimmter Dienst angeboten wird, und erhält von diesem einen *Handle* mit den benötigten Informationen. Der Client generiert aus diesem Handle einen Stub und fügt ihn in seine Bibliothek ein, wo er für die folgenden Fernaufrufe bereitsteht.

9.2.3 Sun RPC

Heutzutage sind eine Reihe kommerzieller RPC-Implementationen verfügbar, wie z.B. in *DCE* und *CORBA* (siehe Abschnitte 9.4.2 bzw. 9.4.3), Microsoft's *DCOM* (Distributed Component Object Model), *Java RMI* (Remote Method Invocation) und *Sun RPC*.

Sun RPC (auch *ONC RPC* oder *TI-RPC* genannt) ist ein Bestandteil des *Sun ONC+* (*Open Network Computing*) und bildet unter anderem die Grundlage für Sun NFS. Es besteht aus zwei Bibliotheken, nämlich der *RPC-Bibliothek* für die eigentlichen RPC-Mechanismen und der *XDR-Bibliothek* zur Umwandlung von Daten aus oder in eine maschinenunabhängige Darstellung (XDR = External Data Representation). Damit ist Sun RPC auch in heterogenen Netzen einsetzbar (siehe auch [RFC1831] und [RFC1833]).

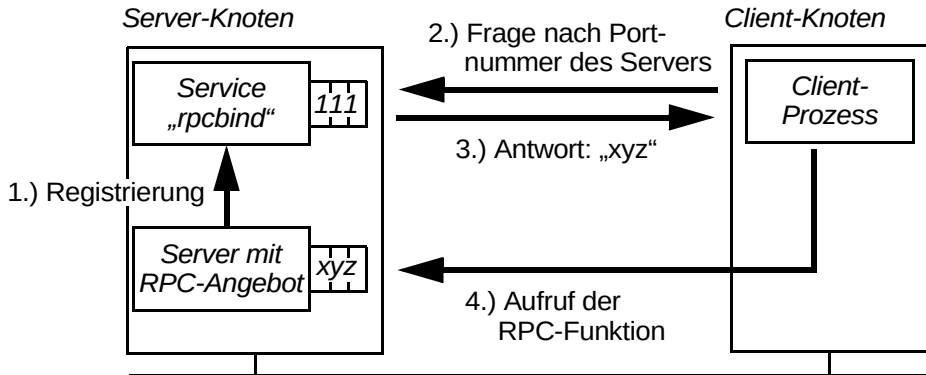


Abb. 9.10 Vorgehensweise beim Sun RPC

Sun RPC kann im OSI-Protokollstack den Ebenen 5 und 6 (Session Layer bzw. Presentation Layer) zugeordnet werden. Es setzt auf vorhandene Kommunikationsprotokolle wie z.B. TCP/IP oder UDP/IP auf, die ihrerseits beispielsweise über LAN kommunizieren.

RPC-Bibliothek

In der RPC-Bibliothek gibt es verschiedene Gruppen von Funktionsschnittstellen, mit denen der RPC auf unterschiedlichen Abstraktionsniveaus benutzt werden kann. Von diesen ist das *Simplified Interface* die abstrakteste Schnittstelle: Es generiert automatisch Stubs und ruft XDR-Transformationsfunktionen auf. Die anderen, tiefer liegenden Schnittstellenbürden dem Programmierer mehr Detailarbeit auf, bieten dafür aber mehr Auswahl- und Einstellmöglichkeiten.

Bei Benutzung des Simplified Interface „registriert“ ein Server seinen Dienst mit Hilfe der Funktion `rpc_reg()` (macht den Dienst also allgemein bekannt) und ruft dann die Funktion `svc_run()` auf. Hier blockiert er so lange, bis ein Auftrag für ihn eintrifft. Dieser Auftrag wird dann bearbeitet, anschließend auf die nächste Anforderung gewartet usw. Ein Client kann den Dienst mit Hilfe der Funktion `rpc_call()` aufrufen. `rpc_call()` erwartet als Parameter u.a. den Namen des Rechnerknotens, auf dem der Server läuft, die Identifikation des Serverprogramms, die Eingabeparameter für das Serverprogramm sowie die Angabe der Adresse, an der das Resultat abgelegt werden soll.

Client und Server werden mit Hilfe des Services `rpcbind` (früher *Portmapper*) zusammengeführt (siehe Abb. 9.10). Hier registriert der Server seinen Dienst unter Angabe seiner Portnummer. Der Client kann dann über den Port 111, der fest diesem Service zugeordnet ist, die Portnummer des Servers erfragen und damit anschließend seinen Auftrag an die richtige Stelle senden.

rpcgen

Die Programmierung einer Sun-RPC-Anwendung wird durch das `rpcgen`-Kommando erleichtert, das an der Benutzerschnittstelle aufgerufen werden kann. Es erhält als Eingabe eine Datei mit der Namenserverweiterung `.x`, die den Prototypen (= die Schnittstelle) der Server-Prozedur, Struct-Typen für deren Parameter und Rückgabewert sowie Verwaltungsnummern enthält. Als Beschreibungssprache für diese Informationen wird das C/C++-ähnliche *RPCL (RPC Language)* verwendet. `rpcgen` generiert daraus automatisch mehrere Dateien mit C-Quellcode. Dazu gehören eine Header-Datei mit Prototypen für die Server-Prozedur und Konversionsfunktionen, Struct-Typen für Parameter und Rückgabewert sowie Konstanten für die Verwaltungsnummern. Darüber hinaus werden Dateien mit Funktionscode für die Konversionsfunktionen, Client- und Server-Stubs sowie die Server-Schleife zum Empfang von Aufträgen erzeugt. Der Programmierer selbst muss, neben der RPCL-Datei, lediglich die aufzurufende Server-Prozedur und das aufrufende Client-Programm selbst erstellen.

Eine detaillierte Beschreibung des Sun RPC mit Programmbeispielen findet sich beispielsweise in [Web98] oder unter [Sun01].

9.3 Prozessverwaltung

Die Verwaltung der Prozesse in einem verteilten System weist einige Charakteristika auf, die zusätzlich zu denen in einem lokalen System beachtet werden müssen. Hierzu gehören insbesondere die Verteilung der Aktivitäten auf die einzelnen Rechnerknoten, ihre Synchronisation und der Einsatz der Threading-Technik.

9.3.1 Lastverteilung und Fehlertoleranz

In einem verteilten System muss festgelegt werden, welche Rechnerknoten welche Aufgaben übernehmen sollen. Allgemein gesprochen bedeutet das, dass die Prozesse des Betriebssystems und der Anwendungen auf die einzelnen Knoten verteilt werden müssen. Man sagt, dass Betriebssystem und Anwendungen auf dem realen Netz *konfiguriert* werden, und spricht in diesem Zusammenhang auch von *Lastverteilung*.

Bei der Konfigurierung sind Randbedingungen zu beachten, die sich aus den Eigenschaften der Prozesse ergeben. So kann beispielsweise ein Druckerserver ausschließlich auf einem Knoten ausgeführt werden, der einen Drucker besitzt. Wichtig ist zudem die Effizienz. So sollte die Arbeitslast möglichst gleichmäßig auf die Rechnerknoten verteilt werden, und die Kommunikationsbeziehungen zwischen den Knoten sollten minimiert werden, um möglichst wenige Daten über das schmalbandige Netz übertragen zu müssen.

Die Konfigurierung kann entweder statisch beim Programmstart erfolgen, oder es kann die Zuordnung von Prozessen zu Knoten während der Programmausführung dy-

namisch geändert werden. Bei einer dynamischen Änderung spricht man von *Prozessmigration*, d.h. der Verschiebung von Prozessen zwischen den Knoten. Eine Konfigurierung kann entweder „per Hand“ durch den Benutzer oder Systemverwalter durchgeführt werden (z.B. mit den Kommandos `rsh` und `rlogin`) oder automatisch durch das System.

Mechanismen zur automatischen Konfigurierung existieren heutzutage insbesondere für Multiprozessorsysteme. Verfahren zur Verteilung der Last in Rechnernetzen sind dagegen noch relativ stark beschränkt. Beispielsweise versteht Windows 2000 unter dem Begriff „Lastverteilung“ lediglich die Möglichkeit, eingehende Aufträge (z.B. Anfragen an einen WWW-Server) gleichmäßig auf mehrere Server zu verteilen, die alle denselben Dienst anbieten.

Das Vorhandensein mehrerer Rechnerknoten unterstützt auch bei der Prozessausführung, ähnlich wie bei der Datenhaltung, eine verbesserte *Fehlertoleranz*. Dieselbe Aktivität kann nebenläufig auf mehreren Knoten ausgeführt werden, oder es können Aktivitäten bei Ausfall eines Knotens auf einen anderen verlagert werden (aktive bzw. passive Redundanz – siehe Abschnitt 3.4.2). Der „Cluster Service“ von Windows 2000 bietet beispielsweise mit seinen „Fail-Over Groups“ die Möglichkeit, Reservekomponenten zu installieren, die im Fehlerfall die ausgefallene Funktionalität übernehmen können.

9.3.2 Synchronisation

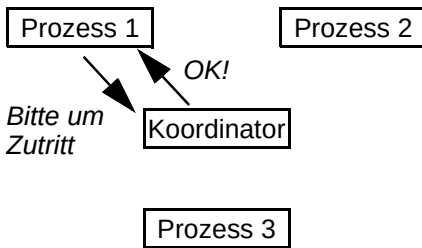
Synchronisationsmechanismen setzen zeitliche Bedingungen bei der nebenläufigen Ausführung von Prozessen durch. Sie sind in verteilten Systemen schwieriger zu implementieren als in Ein-Prozessor-Systemen: Erstens steht kein gemeinsamer Speicher zur Verfügung, so dass insbesondere Semaphore nicht benutzt werden können. Zweitens gibt es aufgrund der „Unschärfe“ des Systems auf keinem Rechnerknoten die vollständige aktuelle Information über die Zustände der einzelnen Prozesse. Im Folgenden sollen einige Probleme bei der Synchronisation kurz angesprochen werden.

Uhrensynchronisation

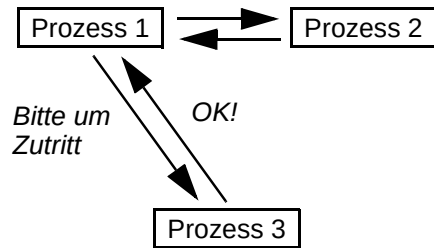
Ein Grundproblem in einem verteilten System ist, dass keine globale Zeitbasis (= einheitliche Uhrzeit) existiert, auf die sich die einzelnen Knoten und Prozesse stützen könnten. Zwar besitzt jeder Knoten eine lokale Uhr, aber die Uhren verschiedener Knoten zeigen möglicherweise unterschiedliche Zeiten an und haben zudem nicht unbedingt alle dieselbe Geschwindigkeit. Damit sind Zeitmarken, wie sie beispielsweise unter den UNIX-Dateiattributen zu finden sind, nicht ohne weiteres verlässlich, und Vorher-Nachher-Beziehungen sind schwer feststellbar.

Um die lokalen Uhren zu synchronisieren, also auf dieselbe Zeit einzustellen, können entweder Funkempfänger zum Auffangen eines Zeitsignals oder ein spezielles Netzwerkprotokoll, wie zum Beispiel *NTP (Network Time Protocol)* [RFC1305], eingesetzt werden. Bei einem Zeitsignal von einem terrestrischen Sender (= Sendemast auf der Erde) kön-

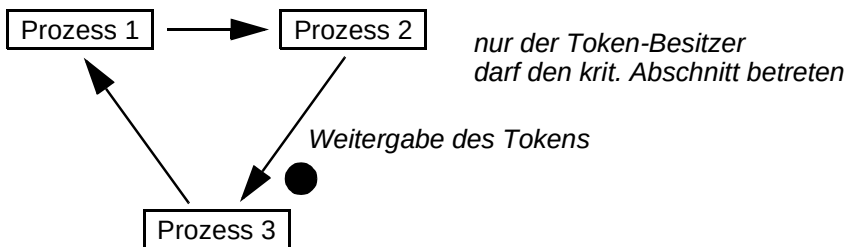
a.) zentrales Verfahren:



b.) dezentrales Verfahren:



c.) Token-gestütztes Verfahren:

**Abb. 9.11** wechselseitiger Ausschluss in verteilten Systemen

nen die Uhren einander auf ca. ± 10 ms angenähert werden, bei einem satellitengestützten Sender auf ca. $\pm 0,5$ ms oder noch besser [Tan92]. Die Genauigkeit, die mit einem Netzprotokoll erzielt werden kann, ist abhängig von der Varianz der Übertragungsverzögerungen des Netzes. NTP erreicht in der Praxis eine Genauigkeit im unteren Millisekunden- oder sogar im Mikrosekundenbereich ([Web98], [Gall00]).

Anstatt die absoluten Uhrzeiten zu synchronisieren, kann man sich auch darauf beschränken, eine *relative Zeitbasis* zu erzeugen. Diese Zeitbasis gibt nicht die absoluten Zeitpunkte bestimmter Ereignisse (z.B. Dateizugriffe) an, aber zumindest ihre zeitliche Reihenfolge, was für viele Anwendungen ausreichend ist. Zur Feststellung solcher Vorher-Nachher-Beziehungen gibt es praktikable Ansätze, z.B. das Verfahren von Lamport ([Lam78], siehe auch [Web98]).

Wechselseitiger Ausschluss

Ein wichtiges Problem der Prozesssynchronisation ist der wechselseitige Ausschluss, der sicherstellen soll, dass nur ein Prozess gleichzeitig in einem kritischen Abschnitt aktiv ist. Hierfür gibt es in verteilten Systemen mehrere Lösungsansätze (siehe Abb. 9.11): In einem zentralen Verfahren muss ein Prozess bei einem festgelegten zentralen Koordinator-Prozess anfragen, ob er den kritischen Abschnitt betreten darf, und sich nach dem Verlassen des Bereichs bei ihm wieder abmelden. Bei einem dezentralen Verfahren geht die Anfrage an alle anderen Prozesse. Bei einem Token-Verfahren wird eine

Marke (das „Token“) unter den Prozessen zyklisch weitergegeben, und nur der Markenbesitzer darf in den kritischen Abschnitt eintreten.

In diesem Zusammenhang ist auch der Begriff der *Transaktion* interessant (siehe auch Abschnitt 4.1.4): Eine Transaktion ist eine Operation, die entweder vollständig oder überhaupt nicht ausgeführt wird. Sie kann zudem während ihrer Ausführung nicht unterbrochen werden, ist also atomar. Ein Transaktionskonzept ist wichtig zur Realisierung verteilter Datenbanken, bei denen die Konsistenz der Einträge sichergestellt werden muss.

Election-Verfahren

Wichtig sind schließlich auch noch *Election-Verfahren*. Mit ihnen wird genau ein Prozess ausgewählt, der eine bestimmte Aufgabe übernehmen soll – zum Beispiel die Rolle eines Koordinators für kritische Abschnitte oder des Überwachers bestimmter Vorgänge im System. Zu berücksichtigen ist hier die Fehlertoleranz: Fällt der bisherige Koordinator / Überwacher aus, so müssen sich die verbleibenden Prozesse dynamisch auf einen neuen einigen. Hierfür gibt es recht effiziente Algorithmen. Beispielsweise können sich die Prozesse gegenseitig ihre Prozessidentifikatoren und Netzadressen mitteilen, und der Prozess mit dem numerisch höchsten Wert „gewinnt“.

9.3.3 Threading

Wie in Abschnitt 3.1.2 diskutiert wurde, können mit Threads mehrere Aktivitäten im selben Kontext ausgeführt werden und so gemeinsame Daten nutzen. Dieses Konzept kann u.a. in Client-Server-Strukturen vielfältig angewandt werden, die in verteilten Systemen häufig auftreten: Jeder Client-Auftrag kann im Server durch einen eigenen Thread bearbeitet werden, so dass einerseits die Nebenläufigkeit und andererseits der Zugriff auf gemeinsame Server-Daten realisiert wird.

Zur näheren Illustration betrachten wir einen File-Server: Der Server nimmt jeweils den Auftrag eines Clients entgegen, fordert von der Platte die benötigten Plattenblöcke an und blockiert, bis die Blöcke im Buffer Cache vorliegen. Erst dann kann er den Auftrag weiter ausführen. Um die Bearbeitung mehrerer wartender Aufträge zu beschleunigen, ist es vorteilhaft, mehrere nebenläufige Prozesse im Server einzusetzen. Während ein Teil der Prozesse blockiert und auf das Eintreffen von Blöcken wartet, können die übrigen bereits die nächsten Aufträge entgegennehmen. Alle diese Prozesse müssen allerdings auf dieselben Datenstrukturen zugreifen, wie zum Beispiel den Auftrags-Port oder den Buffer Cache. Zwar könnte man dies durch Sharing-Mechanismen erreichen, die aber explizit ausprogrammiert werden müssten. Eleganter ist hier der Einsatz von Threads.

Ein naher Zusammenhang besteht zudem zwischen dem Thread-Konzept und Multiprozessorssystemen: Hier können Threads auf den einzelnen Prozessoren echt nebenläufig ausgeführt werden, wobei ihr Kontext u.a. durch den gemeinsam benutzbaren Hauptspeicher realisiert wird. Die enge (Software-)Kopplung der Threads entspricht also der engen (Hardware-)Kopplung des Multiprozessor-systems.

9.4 Systembeispiele

Im Bereich der verteilten Systeme ist unter anderem die *Open Group* (früher *Open Software Foundation – OSF*) aktiv, ein Zusammenschluss führender Hersteller auf dem Gebiet der Informationstechnik. Das Ziel der Open Group ist, herstellerunabhängige Systemsoftware bereitzustellen, auf deren Basis dann „offene“ (also nicht-plattformspezifische) verteilte Anwendungsumgebungen realisiert werden können. Zu den Produkten gehören unter anderem das Betriebssystem *OSF/1* sowie *OSF/DCE* (*Distributed Computing Environment*), das eine Sammlung von Diensten für verteilte Systeme ist. Einschränkung ist zu sagen, dass OSF/1 seit 1995 nicht mehr weiterentwickelt wird.

Daneben gibt es die *Object Management Group* (OMG), ebenfalls ein Zusammenschluss von IT-Herstellern, die ihrerseits mit *CORBA* (*Common Object Request Broker Architecture*) eine Kollektion von Diensten für verteilte objektorientierte Systeme standardisiert hat. Auch hier ist die Offenheit der Dienste ein wichtiges Ziel.

9.4.1 Mach und OSF/1

OSF/1 basiert auf dem schon mehrfach genannten Betriebssystem Mach, dessen Entwicklung Mitte der achtziger Jahre an der Carnegie Mellon University (Pittsburgh, USA) begann. Von seiner Geschichte her ist Mach ein Multiprozessor-Betriebssystem, das auch als Basis für verteilte Betriebssysteme geeignet ist. Es ist zwar eine der wenigen Hochschulentwicklungen, die Produktreife erreicht haben; der durchschlagende Markterfolg ist ihm allerdings verwehrt geblieben.

Wie schon in Abschnitt 1.3.1 besprochen, wird Mach durch seinen Microkernel charakterisiert: Im Kern von Mach befinden sich nur einige wenige Basisdienste, während alle anderen Dienste durch Serverprozesse implementiert sind, die im User Mode oberhalb des Kerns laufen. Diese Aussage gilt allerdings nur für die reine Form von Mach. In kommerziellen Mach-Implementationen umfasst der Kern mehr Funktionalität, da die Kommunikation mit den Server-Prozessen relativ aufwendig ist und zu Leistungseinbußen führt.

Zu den Basisdiensten und -konzepten des Kerns gehören Threads und Tasks, ein virtueller Speicher, ein Kommunikationsmechanismus mit Ports und Nachrichten sowie eine *transparente Netzerweiterung*. Aufgrund seines Portkonzepts kann Mach sowohl auf Ein-Prozessor-Systemen als auch auf Multiprozessorsystemen und verteilten Systemen implementiert werden. Die transparente Netzerweiterung im Kern ist dabei zuständig für die Prozesskommunikation über ein Netz.

Die Mach-Version 2.5 bildet die Basis für OSF/1. Zusätzlich zu den Mach-Komponenten enthält OSF/1 ein *Virtual File System* (VFS), das die Dateisysteme von UNIX System V, BSD 4.4 sowie NFS unterstützt, einen *Logical Volume Manager* zur Verwaltung von Festplatten mit dynamischen Partitionen und Replikationen, ein *Security System* mit Zugriffskontrolllisten für Dateien und Verzeichnisse, sowie das *Pthreads*-Paket, das einen komfortablen Zugriff auf die Threadfunktionen des Mach-Kerns erlaubt.

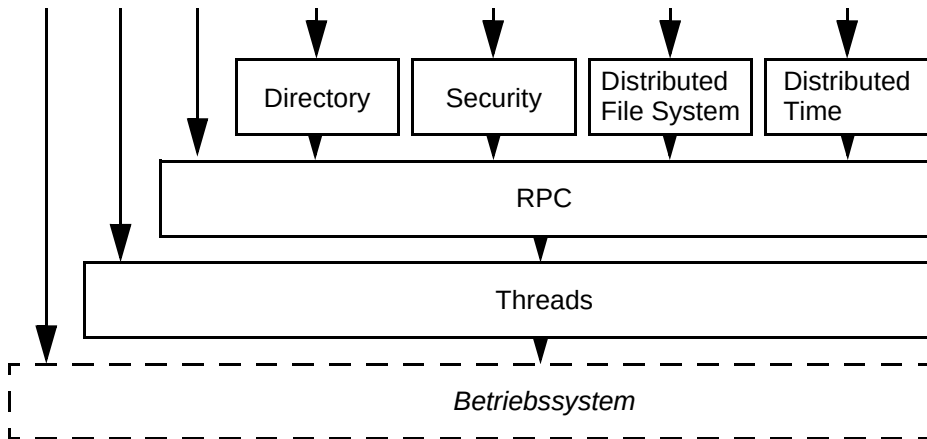


Abb. 9.12 DCE-Dienste

Mach hatte auch Einfluss auf die Entwicklung von Windows NT: Windows NT hat (zumindest in seiner ursprünglichen Fassung) ebenfalls einen recht kleinen Kern, lagert höhere Betriebssystemaufgaben in gesonderte Subsysteme aus und bietet oben Schnittstellen verschiedener Betriebssysteme an (siehe Abschnitt 1.3.1). Zudem bietet es mit seinen *Local Procedure Calls* auch einen portbasierten Kommunikationsmechanismus.

9.4.2 OSF / DCE

OSF/DCE (OSF Distributed Computing Environment) ist eine Sammlung von Diensten für verteilte Softwaresysteme. Es setzt voraus, dass die speziellen Eigenschaften der Hardware durch einen (eventuell abgespeckten) Betriebssystem-Kern verborgen werden. Hierzu können beispielsweise OSF/1, aber auch UNIX, Windows XX oder andere Betriebssysteme verwendet werden. Auf diesem Kern setzt DCE mit seinen Diensten auf. Diese sind hardware- und betriebssystemübergreifend kompatibel und unterstützen so verteilte Anwendungen auf heterogenen Systemen.

Solche Dienste, die oberhalb des Betriebssystems angesiedelt, aber keine eigentlichen Anwendungsprogramme sind, werden auch als *Middleware* bezeichnet. Die Middleware dient gewissermaßen als Verteilungsplattform für verteilte Anwendungen und versucht, eine möglichst weit gehende Transparenz gegenüber der physischen Verteiltheit und Heterogenität zu erreichen.

Die DCE-Middleware enthält insbesondere folgende Dienste (siehe auch Abb. 9.12):

- Ein *Thread Service* bildet die Basis von DCE. DCE setzt also nicht voraus, dass darunter liegende Betriebssystem über Threads verfügt, sondern bietet sein eigenes Thread Package. Der Thread Service ist als User-Level-Bibliothek realisiert, die auf dem POSIX-Standard basiert und u.a. verschiedene Synchronisations- und Schedulingmechanismen bietet.

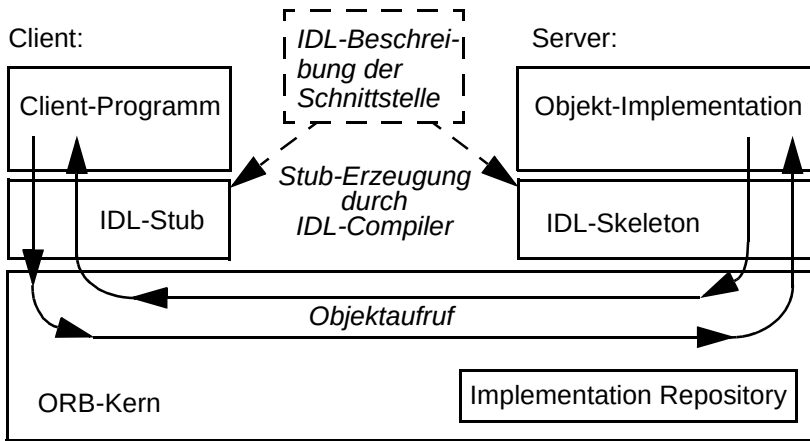
- Oberhalb der Threads befindet sich der *RPC Service*, der als Kommunikationsmechanismus in DCE dient. Er ermöglicht unter anderem die automatische Generierung von Stubs aus Prozedurbeschreibungen in der Definitionssprache *IDL (Interface Definition Language)* und die Übergabe von Referenzparametern.
- Der *Directory Service* setzt, wie die übrigen Dienste, auf Threads und RPC auf. Er implementiert eine verteilte Namensverwaltung und wird insbesondere für die Lokalisierung von RPC-Diensten und von Dateien im verteilten Dateisystem (siehe unten) benutzt.
- Der *Security Service* enthält Mechanismen für Authentifizierung, Zugriffskontrollen und Verschlüsselung und basiert auf dem *Kerberos*-System des *Massachusetts Institute of Technology (MIT)*. In Kerberos meldet sich ein Benutzer einmalig beim Einloggen mit Kennung und Passwort bei einem *Authentication Service AS* an und erhält von diesem ein Ticket zum Zugriff auf einen *Ticket Granting Service TGS*. Dieses Ticket ist zwar zeitbeschränkt, aber meist während dieser gesamten Benutzersitzung gültig. Mit dem Ticket kann sich der Benutzer-Client dann wiederholt an den TGS wenden. Der TGS liefert ihm dabei jeweils ein Ticket zum Zugriff auf einen Dienstanbieter, mit dem dann der Client über RPC entsprechende Dienste aufrufen kann. Aus Sicherheitsgründen werden die übertragenen Daten verschlüsselt.
In DCE kann der Benutzer zwischen unterschiedlichen Häufigkeiten der Authentifizierung wählen (z.B. nur einmal pro Kommunikationsvorgang oder für jedes einzelne Paket). Zudem ist die Verschlüsselung optional.
- Das *Distributed File System (DFS)* realisiert, wie der Name schon sagt, ein verteiltes Dateisystem. Es ist aus dem *Andrew File System (AFS)* entstanden und ermöglicht außerdem die Einbindung weiterer Dateisysteme, wie z.B. die von UNIX und Windows. Es wird ergänzt durch den *Diskless Support Service* mit Funktionen zur Einbindung von plattenlosen Rechnerknoten und die *PC Integration*, die DFS-Zugriffe von PC-Betriebssystemen und UNIX aus ermöglicht.
- Der *Time Service* dient zur Uhrensynchronisation.

9.4.3 OMG CORBA

OMG CORBA, die „Common Object Request Broker Architecture“ der Object Management Group, ist eine Middleware für verteilte objektorientierte Software. CORBA unterstützt heterogene verteilte Systeme, d.h. die beteiligten Rechnerknoten können sich bezüglich ihrer Hardware, ihres Betriebssystems und der verwendeten Programmiersprachen unterscheiden. Dienste können durch Objekte bereitgestellt und nach dem Modell der Client-Server-Kommunikation aufgerufen werden (siehe Abb. 9.13, Teil a).

Ein Objekt ist ein Programmmodul, auf das nur über eine wohldefinierte Schnittstelle zugegriffen werden kann, dessen interne Daten also „gekapselt“ sind. Die Schnittstelle ist dabei eine Sammlung von Funktionsprototypen und eventuell Attributen, d.h. Werten, die im Objekt gespeichert sind. Ein Dienst in CORBA wird also durch einen Funk-

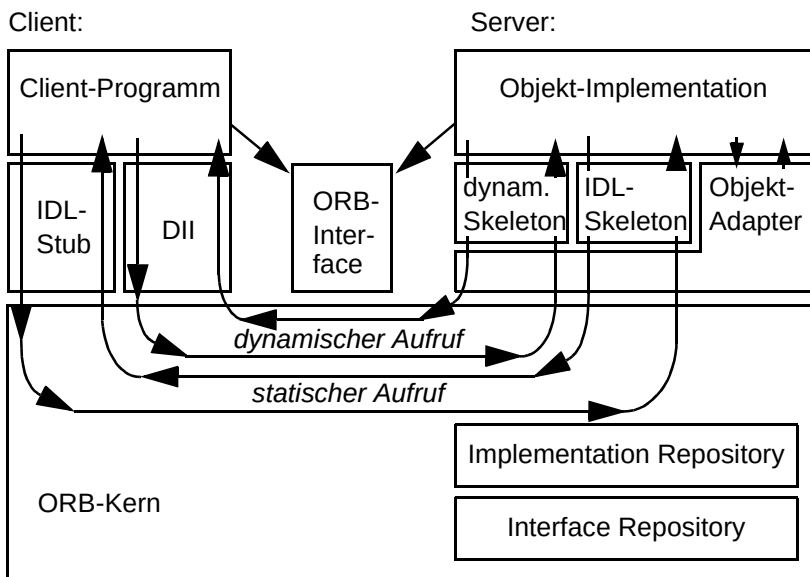
a.) vereinfachtes Bild (nur statische Objektaufrufe):



IDL = Interface Definition Language

ORB = Object Request Broker

b.) vollständiges Bild (statische und dynamische Objektaufrufe):



DII = Dynamic Invocation Interface

Abb. 9.13 CORBA: Common Object Request Broker Architecture

tionsprototypen in einer Objektschnittstelle bereitgestellt. Dabei werden die Objektschnittstellen in der Sprache *IDL* (*Interface Definition Language*) beschrieben, die eine C++-artige Syntax hat. Die Objekte in einem CORBA-System können auf beliebigen Rechnerknoten stehen und in verschiedenen Sprachen implementiert sein (z.B. C, C++, Java).

Der Aufruf von Diensten geschieht, wie gesagt, nach dem Client-Server-Prinzip: Ein Client übergibt über einen *Stub* seinen Auftrag, der den Server dann über einen *Skeleton* (einen Server-Stub) erreicht. Stubs und Skeleton sind Programmstücke in der jeweils zugrunde liegenden ausführbaren Programmiersprache. Sie werden mit Hilfe eines IDL-Compilers automatisch aus der abstrakten IDL-Beschreibung generiert.

Die Vermittlung zwischen Client und Server geschieht durch einen *Object Request Broker (ORB)*, der ein *Implementation Repository* enthält. Server registrieren dort ihre Dienste, indem sie eine Beschreibung der erforderlichen Aufrufkommandos, Aufrufparameter, Zugriffsrechte und weitere Informationen ablegen. Beim Aufruf eines Dienstes leitet der ORB den Client-Auftrag an das zugehörige Server-Objekt weiter, wobei gegebenenfalls die Datendarstellung konvertiert wird. Auch die Rückgabe des Ergebnisses läuft über den ORB.

Die bislang geschilderte Vorgehensweise bei einem Aufruf wird als „statischer Objektaufwurf“ bezeichnet: Hier muss schon zur Übersetzungszeit des Clients festgelegt werden, welcher Server-Dienst angesprochen wird. Daneben gibt es „dynamische Objektaufrufe“ (siehe Abb. 9.13, Teil b): Hier bestimmt der Client erst zur Laufzeit das aufzurufende Objekt. Er stützt sich dabei auf das *Interface Repository*, das IDL-Spezifikationen von Objektschnittstellen enthält, und benutzt zum Aufruf das *Dynamic Invocation Interface (DII)*.

Client und Server können Dienste, die der ORB selbst anbietet, über ein *ORB Interface* nutzen. Ein *Object Adapter* auf Server-Seite dient u.a. zur Aktivierung des Servers und seiner Objekte bei Objektaufrufen und zur Überprüfung von Zugriffsberechtigungen.

In einem großen, heterogenen System existieren im Allgemeinen mehrere ORBs, die Aufträge und Antworten ihrer Clients und Server untereinander austauschen. Jeder ORB muss für diese Kommunikation ein *General Inter-ORB Protocol (GIOP)* implementieren. Ein Standard ist hier das *Internet Inter-ORB Protocol (IIOP)*, das auf TCP/IP basiert und von jeder ORB-Implementation unterstützt werden muss.

Neben dem ORB stellt CORBA weitere Komponenten bereit: Die *Object Services* enthalten elementare betriebssystemnahe Dienste (z.B. zur Sicherheit, Uhrensynchronisation, Ereignisbehandlung), die *Common Facilities* komplexere, allgemein anwendbare Dienste (z.B. graphische Oberflächen, Datenbanken, Gerätesteuerung), die *Domain Services* standardisierte Lösungen für spezielle Anwendungen (z.B. Software für das Finanzwesen) und die *Application Objects* Anwendungsprogramme außerhalb des eigentlichen CORBA-Standards.

CORBA stellt einen abstrakteren Ansatz dar als das nicht objektorientierte DCE. DCE kann daher als ein Mechanismus für die Zusammenarbeit zwischen ORBs eingesetzt werden. In Konkurrenz zu CORBA steht Microsoft mit seinem *DCOM* (*Distributed Component Object Model*).

X-Server:

z.B. Terminal mit Graphik-Prozessor

X-Client:

z.B. zentraler Universal-Prozessor
mit Anwendungsprogramm
oder Betriebssystem

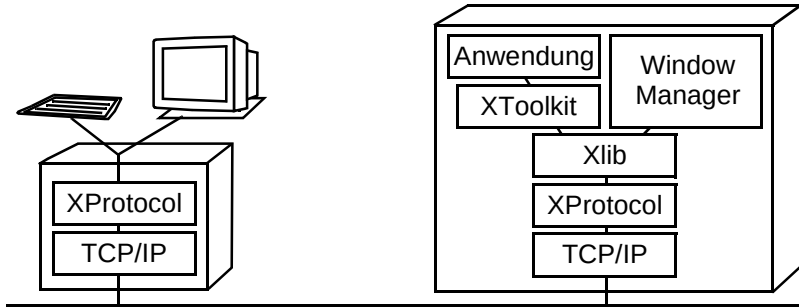


Abb. 9.14 Client-Server-Struktur in X-Window

Mittlerweile gibt es eine ganze Reihe von CORBA-Implementationen. Insbesondere für Studienzwecke ist *MICO* interessant – eine CORBA-Freeware, die an der Universität Frankfurt/Main ihren Ursprung hat und inzwischen für eine Reihe von Betriebssystemplattformen verfügbar ist [MICO01].

9.4.4 X-Window

Der CORBA-Kern stellt Mechanismen bereit, mit denen Programmobjekte die Dienste anderer, entfernt liegender Objekte nutzen können. Etwas Ähnliches ist auch auf einer höheren Ebene sinnvoll: Anwender sollen die Möglichkeit haben, an ihrer Benutzeroberfläche nicht nur die Aktivitäten ihres lokalen Rechnerknotens zu steuern und zu beobachten, sondern auch die Aktivitäten entfernt liegender Knoten. Entsprechende Dienste werden u.a. von *X-Window* bereitgestellt.

Unter *X-Window* geschieht der Zugriff auf eine Benutzeroberfläche nach dem Client-Server-Prinzip (siehe Abb. 9.14). Ein *XClient* ist dabei ein normales Anwendungsprogramm oder das Betriebssystem selbst, das z.B. einen Window Manager zur Steuerung einer fensterorientierten Oberfläche besitzt. Der *XClient* wickelt seine Ein- und Ausgaben über einen *XServer* ab. Der *XServer* ist eine eigenständige Einheit, die Ausgabeaufträge eines *XClient* annimmt bzw. eingegebene Daten an ihn weiterliefert und seinerseits die E/A-Geräte ansteuert. Der *XClient* führt also seine Ein- und Ausgaben nicht selbst durch, sondern lässt das den *XServer* tun.

Der „Clou“ bei *X-Window* ist, dass der *XServer* nicht auf demselben Rechner laufen muss wie der *XClient*: Der Benutzer kann also seinen lokalen Rechner (*XServer*) für Ein- und Ausgaben einsetzen, während die eigentliche Programmausführung auf einem anderen Rechner (*XClient*) stattfindet. E/A-Aufträge werden dann über das Kommuni-

kationsnetz weitergeleitet, wozu das *XProtocol* verwendet wird, das seinerseits auf TCP aufsetzt.

Im Extremfall hat der XServer nur einen E/A-Prozessor, jedoch keinen Hauptprozessor zur Ausführung von Programmen. Das ist beispielsweise bei *XStations* der Fall, die als Arbeitsplatzrechner nur für die Ein-/Ausgabe zuständig sind, während die Programme auf zentralen Stationen ausgeführt werden.

X-Window stellt mit der *Xlib* eine Bibliothek mit Funktionen zur Ansteuerung der Oberfläche bereit. Diese Funktionen können über ihre C-Prototypen in ein Anwendungsprogramm eingebunden werden, sind allerdings von einem recht niedrigen Abstraktionsniveau, also protokollnah. Daher gibt es zusätzlich das *XToolkit*, das anwendungsnahe Unterstützung bei der Programmierung von Fensteroberflächen (Menüs, Dialogboxen, Buttons, Scrollbars, ...) bietet.

X-Window unterstützt damit die beiden Anforderungen, die bezüglich der Benutzeroberfläche an ein Betriebssystem gestellt werden: Erstens soll dem Benutzer eine ansprechende und leistungsfähige Bedienoberfläche geboten werden, so dass Betriebssystemfunktionen komfortabel benutzt werden können. Zweitens sollen Schnittstellen und Programmierwerkzeuge für den Anwendungsprogrammierer bereitgestellt werden, so dass auch Anwendungsprogramme die Graphikoberfläche ansteuern können.

Als Alternative zu X-Window gibt es das *Common Desktop Environment CDE*, das eine gemeinsame Aktivität von IBM, HP, SunSoft und Novell zur Standardisierung der UNIX-Oberfläche ist. *KDE* und *Gnome* sind entsprechende Oberflächen für Linux.

9.5 Übungsaufgaben

1. Wissensfragen

- a.) Warum benutzt man in einem verteilten System Server?
- b.) Was ist der Unterschied zwischen Directory Server und File Server?
- c.) Was bedeutet der Begriff Mounting?
- d.) Für welchen Begriff steht die Abkürzung RPC? Was ist damit gemeint?
- e.) Was geschieht bei der Uhrensynchronisation?
- f.) Was bedeutet der Begriff Middleware? Nennen Sie zwei Beispiele!
- g.) Wozu dient X-Window?

2. Verteilte Dateisysteme

Gegeben sind vier Gruppen von Dateien:

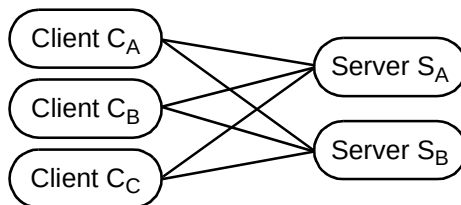
- Datenbank: datbank.exe (1,4 MB), mensaplan.dat (100 KB), prüfungen.dat (200 KB)
- Praktikumsdateien: compi.exe (500 KB), pr1.c (10 KB), pr2.c (20 KB), pr3.c (60 KB)
- Internet-Zugriff: browser.exe (400 KB), ftp.exe (250 KB), telnet.exe (150 KB)
- Textverarbeitung: texttool.exe (750 KB), text1.doc (100 KB), text2.doc (120 KB)

Im Computernetz gibt es drei Fileserver, nämlich Knoten_A mit einer Plattengröße von 1 MByte, Knoten_B mit 1,5 MByte und Knoten_C mit 2 MByte. Zudem gibt es einen Client-Knoten, der auf diese Dateien zugreifen will. Auf seiner Platte steht ein Dateibaum mit dem Hauptverzeichnis „users“ und den Unterverzeichnissen „user1“ und „user2“, in denen die einzelnen Benutzer ihre Dateien abspeichern.

- Verteilen Sie die Dateien sinnvoll auf die einzelnen Server.
- Skizzieren Sie für jede der drei Möglichkeiten, die in Abschnitt 9.1.2 dargestellt wurden, das Aussehen der Dateibäume.
- User1 greift nun vom Client aus auf die Datei prakt1.c zu. Geben Sie für jeden der drei Ansätze den verwendeten Dateinamen/-pfad und gegebenenfalls erforderliche Zusatzoperationen an.

3. Client-Server: Optimierung von Zugriffszeiten

In einer Firma wurde ein Rechnernetz mit fünf Knoten installiert. Zwei Knoten sollen als Server eingesetzt werden, die drei übrigen Knoten sind Clients. Jeder Client ist mit jedem Server über eine eigene Leitung direkt verbunden.



In diesem Netz sollen nun vier Dienste D_1 - D_4 durch die Server angeboten und durch die Clients genutzt werden. Eine Voruntersuchung hat ergeben, wie viele Male pro Sekunde die einzelnen Clients die einzelnen Dienste aufrufen werden (absolute Anzahl):

	D_1	D_2	D_3	D_4
C_A	2	6	1	1
C_B	1	2	4	5
C_C	2	2	1	6

Zudem ist für jeden Dienst die Ausführungszeit des Servers pro Aufruf bekannt:

	D_1	D_2	D_3	D_4
Ausführungszeit in ms	60	70	30	50

- a.) Verteilen Sie die Dienste so auf die beiden Server, dass beide möglichst ähnlich ausgelastet sind und keiner überlastet ist. Die Gesamtausführungszeiten pro Sekunde für die einzelnen Server sollen also möglichst nahe beieinander liegen und dürfen 1000 ms nicht übersteigen.

Geben Sie als Antwort an, welche Dienste auf dem einen und welche auf dem anderen Server installiert werden sollen. Nehmen Sie aber noch keine konkrete Zuordnung zu S_A und S_B vor (siehe hierzu Aufgabe b.)!

- b.) Berücksichtigen Sie nun, dass die Laufzeiten auf den einzelnen Verbindungen unterschiedlich sind. Die Tabelle gibt die Kommunikationszeit zwischen den einzelnen Clients und Servern pro Auftragsbearbeitung an (Werte in ms):

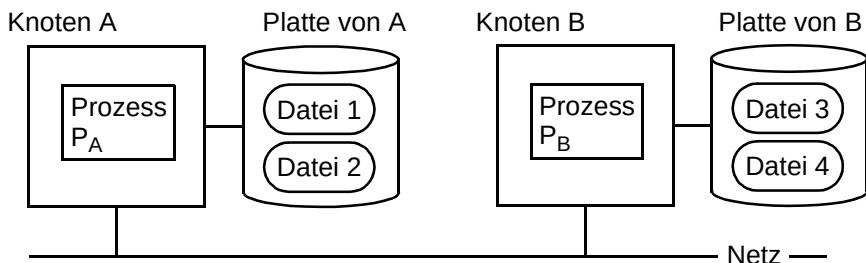
	S_A	S_B
C_A	15	5
C_B	5	10
C_C	5	15

Ordnen Sie nun die beiden Gruppen von Diensten, die Sie in Teilaufgabe a.) gebildet haben, so S_A und S_B zu, dass der Kommunikationsaufwand möglichst gering wird. Begründen Sie Ihre Antwort durch Angabe von Zahlenwerten!

- c.) In Teilaufgabe a.) wurde auf eine möglichst gleichmäßige Auslastung geachtet. Ist es möglich, den Kommunikationsaufwand weiter zu senken, wenn man dafür eine ungleichmäßige Verteilung in Kauf nimmt? Achtung: Kein Server darf überlastet werden! Begründen Sie auch hier Ihre Antwort mit einem Zahlenwert!

4. Client-Server: Replikation

Gegeben ist die folgende Konfiguration:



Die Anzahl der Zugriffe der beiden Prozesse auf die einzelnen Dateien ist durch folgende Tabelle gegeben (Anzahl der Zugriffe *pro Minute*):

	Datei 1	Datei 2	Datei 3	Datei 4
P_A , lesend	20	5	4	3
P_B , lesend	8	30	1	4
P_A , schreibend	2	3	2	18
P_B , schreibend	2	1	15	2

Ein lokaler Dateizugriff (= Zugriff auf die Platte des eigenen Knotens) dauert L ms, ein entfernter Zugriff (= Zugriff auf die Platte des anderen Knotens) E ms. Der Einfachheit halber nehmen wir an, dass L und E konstant sind.

- a.) Berechnen Sie die „Gesamtkosten“ κ pro Minute, wobei κ die Summe der Dauer sämtlicher Dateizugriffe in einer Minute ist. Stellen Sie dazu zunächst eine Formel auf, die die Variablen L und E und die konkreten Werte aus der Tabelle enthält. Werten Sie anschließend die Formel für $L = 15$ ms und $E = 40$ ms aus.
- b.) Wir nehmen nunmehr an, dass sämtliche Dateien repliziert sind, d.h. jede der beiden Platten Kopien aller vier Dateien enthält. Berechnen Sie hierfür ebenfalls die Gesamtkosten pro Minute (wieder zuerst die Formel und dann die Auswertung für $L = 15$ ms und $E = 40$ ms). Beachten Sie dabei Folgendes:
 - Jeder Lesezugriff erfordert nur den Zugriff auf die lokale Platte.
 - Jeder Schreibzugriff erfordert sowohl einen Zugriff auf die lokale als auch auf die entfernte Platte, da beide Dateikopien aktualisiert werden müssen. Die Gesamtdauer hierfür ist gleich dem Maximum von L und E , da beide Zugriffe nebenläufig ausgeführt werden können.
- c.) Lohnt sich (für $L = 15$ ms und $E = 40$ ms) die Replikation aller vier Dateien? Wenn ja, warum? Wenn nein, welche Datei würde man besser nicht replizieren? Bei der Antwort „nein“ genügt die Angabe einer Datei (inkl. einer anschaulichen Begründung *und* einer Begründung durch einen Zahlenwert).
- d.) Gehen Sie nun davon aus, dass Sie nur die Alternative haben, entweder alle vier Dateien oder gar keine zu replizieren.
 - Ab welchem Verhältnis E/L lohnt sich die Replikation (auf der Basis der gegebenen Tabelle und der Formeln aus a. und b.)? Was gilt *hier* also allgemein?
 - Gilt diese allgemeine Aussage auch für beliebige Zugriffshäufigkeiten? Warum oder warum nicht?

Anhang: UNIX-C-Schnittstelle

A.1 Einführende Bemerkungen

Im Folgenden werden die am häufigsten verwendeten Funktionen der UNIX/Linux-C-Schnittstelle mit ihren wichtigsten Eigenschaften beschrieben. Die Dokumentation orientiert sich dabei an der System V Interface Definition SVID bzw. (für die Socket-Funktionen) am BSD UNIX. Ein vollständiges Verzeichnis der Funktionen mit einer detaillierteren Beschreibung findet sich in der Spezialliteratur, ebenso bezüglich des POSIX-Standards – siehe die Literaturempfehlungen. Details zu Bibliotheksfunktionen lassen sich auch mit dem `man`-Kommando abrufen. Bei ihm muss man übrigens manchmal, wenn es ein Benutzerkommando gleichen Namens gibt, explizit die Kapitelnummer der Dokumentation angeben – beispielsweise `man 2 kill`.

C-Programme können die Funktionen der UNIX-Systemschnittstelle (also die UNIX-Systemaufrufe) benutzen, indem sie entsprechende Bibliotheksfunktionen mit einer C-Schnittstelle aufrufen (siehe Abschnitt 2.2.3). Viele dieser Funktionen stehen in der C-Standardbibliothek: Sie werden durch den Compiler automatisch eingebunden, müssen also weder im Programm noch bei Aufruf des Compilers explizit angegeben werden.

Zugehörige Variablentypen, Konstanten (= symbolische Werte) und die Prototypen weiterer Funktionen werden in den Dateien der Verzeichnisse `/usr/include` sowie `/usr/include/sys` definiert. Diese Dateien („Header Files“) haben Namen, die auf `.h` enden. Sie müssen in C-Programme explizit durch `#include` eingebunden werden. Im `include`-Statement wird dabei der Dateiname (evtl. mit Vorsatz `sys/`) zweckmäßigerweise durch `<>` geklammert, wodurch der Compiler die Datei im Allgemeinen automatisch im Verzeichnis `/usr/include` sucht (Beispiele: `#include <errno.h>` oder `#include <sys/types.h>`).

Wenn im Folgenden nicht anders angegeben, liefern die Funktionen als Rückgabewert 0 (bei fehlerfreier Ausführung) oder -1 (im Fehlerfall). Ein genauer numerischer Fehlercode steht dann in der globalen Variablen `errno`, die als externe Variable deklariert werden muss. Die Datei `/usr/include/errno.h` enthält diese Deklaration und bindet darüber hinaus die Datei `/usr/include/sys/errno.h` ein, in der die Fehlercodes symbolisch (d.h. durch die Angabe von Konstantennamen) definiert werden. Die Funktion `perror()` kann benutzt werden, um eine entsprechende Fehlermeldung in Klartext auszugeben. Ein Beispiel illustriert die Vorgehensweise:

```
#include <signal.h>
/* Definiert Konstanten zur Benutzung beim Aufruf
   der unten verwendeten Funktion kill(). */
```

```

#include <errno.h>
    /* Definiert Fehlercodes
       und bindet die Variable errno ein. */
...
int err; /* zur Aufnahme des Fehlercodes. */
...
err = kill(3,SIGKILL);
    /* Versucht, Prozess 3 zu löschen. */
if (err == -1) {
    printf("Fehler Nr. %d\n",errno);
    /* Gibt die Fehlernummer aus. */
    perror("");
    /* Gibt die Fehlermeldung im Klartext aus. */
}

```

In den Beispielen dieses Texts werden die Fehlerabfragen aus Platzgründen meist weglassen. In realen Programmen sollte man sie aber unbedingt durchführen!

Abschließend noch eine Anmerkung zu den zu benutzenden Datentypen: UNIX definiert in seiner Datei `sys/types.h` eine Reihe von Typen, wie z.B. `pid_t` für Prozessnummern, `uid_t` für Benutzernummern, `mode_t` für Dateitypen usw., die als Parameter- und Rückgabetypen der Funktionen verwendet werden. Dahinter „verbergen“ sich aber unmittelbar klassische C-Ganzzahltypen, wie ein Blick in `types.h` zeigt. Aus Gründen der besseren Lesbarkeit werden in diesem Buch die C-Typen angegeben. In der Praxis, insbesondere für Übungszwecke, ist dies fast immer unproblematisch. In Zweifelsfällen empfiehlt sich jedoch die Einbindung dieser Header-Datei und die Verwendung der Systemdatentypen. Nähere Informationen liefern beispielsweise [Stev92], [GuOb95] oder das `man`-Kommando.

A.2 Dateiverwaltung

chdir:

Prototyp:	<code>int chdir(char *pathname);</code>
Effekt:	Ändert das aktuelle Verzeichnis des ausführenden Prozesses.
Parameter:	<code>char *pathname</code> Name oder Pfad des neuen Verzeichnisses.
Beispiel:	<pre>chdir("/sesame/bibo"); /* Ändert das aktuelle Verzeichnis zu "/sesame/bibo". */</pre>

close:

Prototyp: `int close(int file_desc);`

Effekt: Schließt eine Datei.

Parameter: `int file_desc` Deskriptor der zu schließenden Datei.

Beispiel:

```
int fd;
fd = open("Nichts", O_RDONLY, 0);
close(fd); /* Öffnet die Datei "Nichts" und
           schließt sie gleich wieder. */
```

creat:

Prototyp: `int creat(char *pathname, int mode);`

Effekt: Erzeugt eine neue Datei. Falls die Datei bereits existiert, wird ihr Inhalt gelöscht.

Parameter: `char *pathname` Name oder Pfad der neuen Datei.

`int mode` Bitmuster, das die Zugriffsrechte für die neue Datei festlegt. Die Positionen und Bedeutungen der Bits sind dieselben wie in der Ausgabe des Kommandos `ls -l` (Rechte für Besitzer, Gruppe und andere Benutzer).

Rückgabe: Dateideskriptor für folgende Dateizugriffe oder -1 bei Fehler.

Beispiel:

```
int fd1, fd2; /* Dateideskriptoren */
fd1 = creat("Public", 0777);
/* Erzeugt die Datei "Public" mit sämt-
   lichen Rechten für alle Benutzer.
   0777 ist der Oktalwert 777,
   also das Bitmuster 11111111. */
fd2 = creat("Secret", 0700);
/* Erzeugt die Datei "Secret", auf die
   nur der Besitzer zugreifen darf. */
```

lseek:

Prototyp: `long lseek(int file_desc,
 long offset, int origin);`

Effekt: Positioniert den Lese-Schreib-Zeiger einer Datei.

Parameter: `int file_desc` Deskriptor der Datei.

`long offset` Wird auf einen Basiswert addiert und bestimmt so die neue (Byte-)Position des Lese-Schreib-Zeigers. `offset` kann auch negativ sein. Der Basiswert wird durch `origin` bestimmt.

`int origin` Bestimmt zusammen mit `offset` die neue (Byte-)Position des Lese-Schreib-Zeigers:
 0: LS-Zeiger = `offset`
 1: LS-Zeiger = LS-Zeiger (alt) + `offset`
 2: LS-Zeiger = Dateilänge + `offset`

Rückgabe: Neue (Byte-)Position des Lese-Schreib-Zeigers oder -1 bei Fehler.

Beispiel:

```
int fd;    /* Dateideskriptor */
int pos;   /* Zeiger-Position */
< Hier Öffnen einer Datei
  und Zuweisung des Deskriptors an fd. >
pos = lseek(fd,6L,0);
      /* Versetzt den Zeiger auf Byte Nr. 6. */
pos = lseek(fd,0L,1);
      /* Liefert aktuelle Zeigerposition und
         lässt dabei den Zeiger unverändert. */
pos = lseek(fd,-20L,2);
      /* Versetzt den Zeiger 20 Bytes vor das
         Dateiende. */
```

mknod:

Prototyp: `int mknod(char *pathname, int mode, int dev);`

Effekt: Erzeugt eine Datei beliebigen Typs, insbesondere eine Pipe, ein Verzeichnis oder eine Gerätedatei.

Includes: `#include <sys/stat.h>`

Parameter: `char *pathname` Name oder Pfad
 der Datei oder des Verzeichnisses.

`int mode` Bitmuster, das die Art der neuen Datei und die Zugriffsrechte bestimmt. U.a. stehen dabei `S_IFDIR` für ein Verzeichnis und `S_IFIFO` für eine Pipe. Weitere 9 Bits legen die Zugriffsrechte fest, ihre Positionen und Bedeutungen sind dieselben wie in der

Ausgabe des Kommandos `ls -l` (Rechte für Besitzer, Gruppe und andere Benutzer).

`int dev` Für eine Gerätedatei die „Major Device Number“ und die „Minor Device Number“.

Beispiel: `mknod("Directory", S_IFDIR|0777, 0);`
 `/* Erzeugt das Verzeichnis "Directory"`
 `mit sämtlichen Rechten für alle`
 `Benutzer. */`

open:

Prototyp: `int open(char *pathname, int flag, int mode);`

Effekt: Öffnet eine Datei für Zugriffe.

Includes: `#include <fcntl.h>`

Parameter: `char *pathname` Name oder Pfad der zu öffnenden Datei.

`int flag` Bitmuster, das festlegt, wie die Datei geöffnet werden soll. Das Bitmuster ist eine Oder-Verknüpfung (Operator `|`) u.a. aus den folgenden Bits (wobei von den ersten drei Bits genau eines ausgewählt werden muss):

- `O_RDONLY` Nur Lesezugriff.
- `O_WRONLY` Nur Schreibzugriff.
- `O_RDWR` Lese- und Schreibzugriff.
- `O_APPEND` Folgende Schreibzugriffe nur am Dateiende.
- `O_CREAT` Erzeugt Datei, falls noch nicht vorhanden.
- `O_EXCL` Führt zu Fehlermeldung, falls `O_CREAT` gesetzt ist und die Datei schon existiert (u.a. für Lock Files nützlich).
- `O_TRUNC` Löscht Dateiinhalt.

`int mode` Bitmuster für Zugriffsrechte der neuen Datei, falls `O_CREAT` gesetzt ist (siehe `creat()`).

Rückgabe: Dateideskriptor für nachfolgende Dateizugriffe oder -1 bei Fehler.

Beispiel: `#include <fcntl.h>`
 `int fd1, fd2, fd3;`
 `/* Für die Dateideskriptoren */`

```

fd1 = open("Daten",O_RDWR|O_TRUNC,0);
/* Öffnet die Datei "Daten" zum Lesen und
   Schreiben und löscht dabei ihren
   Inhalt. fd1 enthält anschließend den
   Dateideskriptor. */
fd2 = open("Archiv",O_WRONLY|O_APPEND,0);
/* Öffnet die Datei "Archiv" nur zum
   Schreiben, wobei neue Daten ans Ende
   der Datei geschrieben werden. */
fd3 = open("Neu",O_RDWR|O_CREAT|O_EXCL,0777);
/* Erzeugt die Datei "Neu", wobei alle
   Benutzer alle Rechte haben, und öffnet
   sie zum Lesen und Schreiben. Fehler-
   code -1 in fd3, wenn die Datei schon
   existiert. */

```

read:

Prototyp: int read(int file_desc,
 char *buf, unsigned size);

Effekt: Liest Daten aus einer Datei.

Parameter: int file_desc Deskriptor der Datei.

 char *buf Zeiger auf Speicherbereich zur Aufnahme
 der gelesenen Daten.

 unsigned size Maximale Anzahl der gelesenen Daten (in
 Bytes).

Rückgabe: Tatsächliche Anzahl der gelesenen Daten (in Bytes) oder -1 bei
 Fehler.

Beispiel: int fd; /* Dateideskriptor */
 char buf[10]; /* Speicherbereich für
 eingelesene Daten */
 < Hier Öffnen einer Datei
 und Zuweisung des Deskriptors an fd. >
 read(fd,buf,10);
 /* Liest bis zu 10 Bytes nach buf,
 beginnend beim aktuellen LS-Zeiger. */

unlink:

Prototyp: int unlink(char *pathname);

Effekt: Löscht eine Datei oder einen Link.

Parameter: `char *pathname` Name oder Pfad der zu löschenden Datei.

Beispiel: `unlink("/home/obelix/kleinbonum.txt");`

write:

Prototyp: `int write(int file_desc,
 char *buf, unsigned size);`

Effekt: Schreibt Daten in eine Datei.

Parameter: `int file_desc` Deskriptor der Datei.

`char *buf` Zeiger auf auszugebende Daten.

`unsigned size` Anzahl der auszugebenden Daten (in Bytes).

Rückgabe: Anzahl der tatsächlich geschriebenen Daten (in Bytes) oder -1 bei Fehler.

Beispiel:

```
int fd;          /* Dateideskriptor */
int i;           /* Laufvariable */
char buf[10];    /* Auszugebende Daten */
< Hier Öffnen einer Datei
  und Zuweisung des Deskriptors an fd. >
for (i=0;i<10;i++) buf[i] = 'x';
write(fd,buf,10);
/* Schreibt eine Folge von 10 'x' in die
   Datei, Start beim akt. LS-Zeiger. */
```

A.3 Prozessverwaltung

execv:

Prototyp: `int execv(char *pathname, char *argv[]);`

Effekt: Lässt den aufrufenden Prozess ein neues Programm ausführen.

Parameter: `char *pathname` Name oder Pfad der Datei, in der das auszuführende Programm steht.

`char *argv[]` Zeiger auf ein Feld, in dem die Argumente für das aufgerufene Programm stehen. Der letzte Zeiger in `argv[]` muss der Null-Zeiger sein!

Beispiel:

```
#include <strings.h>
/* Zum Einbinden von strcpy() */
char *parameter[3];
/* Parameter für Programmaufruf */
/* Bereitstellung von Speicherplatz für den
Argumentvektor: */
parameter[0] = malloc(20*sizeof(char));
parameter[1] = malloc(10*sizeof(char));
/* Setzen der Argumente: Erster Parameter ist
stets der Programmname, zweiter Parameter
ist hier "Hallo". */
strcpy(parameter[0], "guten_morgen");
strcpy(parameter[1], "Hallo");
parameter[2] = NULL;
/* Aufruf des Programms in der ausführbaren
Datei "guten_morgen": */
execv("guten_morgen", parameter);
```

exit:

Prototyp: `void exit(int status);`

Effekt: Beendet den ausführenden Prozess und liefert den „Exit-Status“ (d.h. eine OK- oder Fehlermeldung) an den Vaterprozess zurück.

Parameter: `int status` Zurückzuliefernder Status-Wert (0 = OK).

Beispiel:

```
exit(0);
/* Fehlerfreie Terminierung des Prozesses. */
```

fork:

Prototyp: `int fork();`

Effekt: Erzeugt eine genaue Kopie des aufrufenden Prozesses und startet deren Ausführung. Der aufrufende Prozess wird damit der „Vaterprozess“ des neu erzeugten „Sohnprozesses“.

Rückgabe: Der Vaterprozess erhält die PID (= Prozessidentifikationsnummer) des Sohnprozesses (oder -1 bei Fehler) zurück, der Sohnprozess eine 0. Anhand dieser Unterscheidung können Vater- und Sohnprozess in verschiedene Programmstücke verzweigen.

Beispiel:

```
int err; /* Für Fehlercode */
int status;
/* Für Rückkehrstatus des Sohns */
char *parameter[10];
/* Parameter für das Programm des Sohns */
```

```

/* Erzeugen des Sohns und unmittelbare Ver-
   zweigung in verschiedene Codestücke: */
if (fork() == 0) {
    /* Sohnprozess: Führt Prog. "guten_morgen"
       aus und terminiert dann. */
    < Hier Setzen der Parameter
       wie im Beispiel zu execv(). >
    err = execv("guten_morgen",parameter);
    exit(err);
    /* exit() ist wichtig, denn sonst würde
       der Sohn im Vatercode weiterlaufen. */
} /* Ende des Sohncodes */
/* Vater wartet auf Terminierung des Sohns: */
wait(&status);
/* status erhält den Rückgabestatus, der
   mit Makros in sys/wait.h oder wait-
   status.h ausgewertet werden kann. */

```

getpid:

Prototyp: `int getpid();`

Effekt: Liefert die PID (= Prozessidentifikationsnummer) des aufrufenden Prozesses oder -1 bei Fehler.

Beispiel:

```

if (getpid() == 0)
    printf("Ich bin der Swapper-Prozess\n");
if (getpid() == 1)
    printf("Ich bin der Init-Prozess\n");

```

getppid:

Prototyp: `int getppid();`

Effekt: Liefert die PID (= Prozessidentifikationsnummer) des Prozesses zurück, der der Vater des aufrufenden Prozesses ist, oder -1 bei Fehler.

Beispiel: `printf("PID meines Vaters: %d\n",getppid());`

kill:

Prototyp: `int kill(int pid, int sig);`

Effekt: Schickt ein Signal an einen (oder auch mehrere) Prozesse.

Includes: `#include <signal.h>`

Parameter:	pid	PID des Empfänger-Prozesses.
	sig	Signalnummer (siehe <code>signal.h</code>). Das Signal <code>SIGKILL</code> (Signal Nr. 9) bewirkt (Berechtigung vorausgesetzt) eine Terminierung des Prozesses, die dieser nicht verhindern („abfangen“) kann. Alle anderen Signale, insbesondere die benutzerdefinierbaren Signale <code>SIGUSR1</code> und <code>SIGUSR2</code>, können „abgefangen“ werden, d.h. sie terminieren den Empfänger nicht, sondern bewirken bei ihm die sofortige Ausführung eines „Signal Handlers“. Dies ist eine beliebige benutzerdefinierte Funktion, die zuvor durch Aufruf von <code>signal()</code> an das Signal gebunden wurde (siehe Bsp. 2).

Beispiel 1:

```
#include <signal.h>
...
kill(1234, SIGKILL);
/* Terminiert Prozess Nr. 1234. */
```

Beispiel 2:

```
#include <signal.h>
int sohn_pid;
void sighand() {
    /* Signal Handler (beliebige C-Funktion).
       Wird durch signal() an Signal
       SIGUSR1 gebunden (siehe unten) und dann
       bei Eintreffen dieses Signals ausge-
       führt. Fehlt diese Bindung, so wird
       der empfangende Prozess durch das
       Signal terminiert. */
    signal(SIGUSR1, &sighand);
    /* Nach Empfang eines Signals muss die
       Bindung jeweils erneuert werden, was
       - wie hier - am besten im Signal Hand-
       ler selbst geschieht. Anschließend
       beliebiger Programmcode, z.B.: */
    printf("Signal SIGUSR1 empfangen.\n\n");
}
main() {
    signal(SIGUSR1, &sighand);
    /* Bindung des Signalhandlers
       an das Signal SIGUSR1. */
    if ( (sohn_pid = fork()) != 0 ) {
        /* Vaterprozess: */
        printf("Vater gestartet.\n\n");
```

```

sleep(2);
/* Stellt sicher, dass der Sohn pause()
   vor dem kill() des Vaters ausführt. */
printf("Vater sendet SIGUSR1 an Sohn.\n");
kill(sohn_pid, SIGUSR1);
} /* Ende des Vaters. */
else { /* Sohnprozess: */
printf("Sohn wartet auf Vater-Signal\n\n");
pause();
/* Sohn wartet auf ein beliebiges Signal.
   Wenn Signal SIGUSR1 eintrifft, wird
   sighand() ausgeführt. Sohn läuft
   dann in den folgenden Zeilen weiter. */
printf("Weiter im Sohnprogramm.\n\n");
} /* Ende des Sohns. */
} /* Ende des Programms. */

```

pause:

Prototyp: `int pause();`

Effekt: Prozess blockiert, bis ein Signal von einem anderen Prozess kommt.

Beispiel: Siehe `kill()`.

sleep:

Prototyp: `unsigned sleep(int sec);`

Effekt: Prozess blockiert für eine bestimmte Zeit.

Parameter: `int sec` Dauer des Blockierens in Sekunden.

Beispiel: `sleep(5); /* blockiert 5 Sek. */`

time:

Prototyp: `long time(long *tloc);`

Effekt: Gibt die aktuelle Uhrzeit an.

Parameter: `long *tloc` Zeiger auf Variable, in der die Uhrzeit zurückgegeben wird. Hier kann auch NULL übergeben werden: Dann gibt nur der Rückgabewert der Funktion die Uhrzeit an.

Rückgabe: Uhrzeit in Sekunden seit dem 1.1.1970, 0 Uhr, oder -1 bei Fehler.

times:

Prototyp: `long times(struct tms *tbuffer);`

Effekt: Misst die CPU-Zeit, die vom Prozess und seinen Söhnen verbraucht wurde (Details siehe unter „Parameter“).

Includes: `#include <sys/times.h>`

Parameter: `struct tms *tbuffer`

Zeiger auf eine Strukturvariable, in der Informationen über Prozessorzeiten zurückgegeben werden. `*tbuffer` ist vom Typ

```
struct tms {
    time_t tms_utime;
    time_t tms_stime;
    time_t tms_cutime;
    time_t tms_cstime; }
```

`time_t` ist ein Ganzzahltyp, zum Beispiel `long`. Die Komponenten der Struktur haben die folgenden Bedeutungen:

`tms_utime` CPU-Zeit, die der aufrufende Prozess im User Mode verbraucht hat.

`tms_stime` CPU-Zeit, die der aufrufende Prozess im Kernel Mode verbraucht hat.

`tms_cutime` CPU-Zeit, die die bereits terminierten Söhne des Prozesses im User Mode verbraucht haben.

`tms_cstime` CPU-Zeit, die die bereits terminierten Söhne des Prozesses im Kernel Mode verbraucht haben.

Alle Zeiten werden in „Clock Ticks“ angegeben, d.h. die zugrunde liegende Einheit ist die Anzahl der Signale, die die Systemuhr pro Sekunde liefert. Zur Umrechnung in Sekunden kann die Konstante `CLK_TCK` benutzt werden, die die Anzahl der Clock Ticks pro Sekunde angibt (zugreifbar mit `#include <sys/time.h>`). Der Wert von `CLK_TCK` ist systemabhängig.

Rückgabe: Verstrichene Zeit seit dem Start des Systems oder -1 bei Fehler.

Beispiel:

```
#include <sys/time.h>
#include <sys/times.h>
float a=1.23, b=4.56, c;
int i;
```

```

struct tms anfang, ende;
times(&anfang);
for (i=0;i<10000000;i++) c=a*b;
times(&ende);
printf("Zeit für die Multiplikationen:\n");
printf("User Mode: %u s., Kernel Mode: %u s.",
      (ende.tms_utime-anfang.tms_utime)/CLK_TCK,
      (ende.tms_stime-anfang.tms_stime)/CLK_TCK);

```

wait:

Prototyp: `int wait(int *wait_stat);`

Effekt: Prozess blockiert, bis ein beliebiger seiner Sohnprozesse terminiert. Falls es beim Aufruf schon einen terminierten Sohnprozess gibt, der noch nicht durch ein `wait()` „entgegengenommen“ wurde (ein so genannter „Zombie“), so kehrt die Funktion sofort zurück.

Parameter: `int *wait_stat`

Zeiger auf Variable, in der Terminierungsstatus des Sohns zurückgegeben wird. Dieser Status enthält u.a. den Wert, den der Sohn seinem `exit()`-Aufruf übergeben hat, sowie die Nummer des Signals, das den Sohn terminiert hat. Der Statuswert kann mit Hilfe von Makros in `sys/wait.h` analysiert werden.

Wird der Status nicht benötigt, so kann hier 0 als Parameterwert benutzt werden: `wait(0)`.

Rückgabe: PID des terminierten Sohns oder -1, wenn kein Sohnprozess existiert (d.h. kein Sohn in Ausführung war und alle terminierten Söhne bereits durch frühere `wait()`-Aufrufe entgegengenommen wurden).

Beispiel: Siehe Beispiel zu `fork()`.

Bemerkung: Die Funktion `waitpid()` ermöglicht das Warten auf das Ende eines *bestimmten* Prozesses: `waitpid(p, 0, 0)` blockiert den aufrufenden Prozess so lange, bis sein Sohn mit der PID `p` terminiert.

A.4 Semaphore

semctl:

Prototyp: `int semctl(int id, int num,
int cmd, union semun arg);`

Effekt:	Verschiedene Steuerungsfunktionen auf einer Semaphorgruppe.	
Includes:	<pre>#include <sys/ipc.h> #include <sys/sem.h></pre>	
Parameter:	<pre>int id</pre> <pre>int num</pre> <pre>int cmd</pre>	Interner Identifikator für die Semaphorgruppe, wie von <code>semget()</code> geliefert. Nummer eines Semaphors in der Gruppe, falls auf einem einzelnen Semaphor gearbeitet wird. Auszuführendes Kommando, z.B. <code>SETALL</code> Setzen aller Semaphorwerte <code>GETALL</code> Auslesen aller Semaphorwerte <code>SETVAL</code> Setzen eines einzelnen Sem.werts <code>GETVAL</code> Auslesen eines einzelnen Semaphorwerts (Wert wird nicht im unten angegebenen Parameter <code>arg</code> zurückgegeben, sondern als Funktionsrückgabewert) <code>IPC_RMID</code> Löschung der Semaphorgruppe (wichtig, muss stets vor Beendigung des Programms ausgeführt werden!)
	<pre>union semun arg</pre>	Parameter für das Kommando. <code>arg</code> ist vom Typ <pre>union semun { int val; struct semid_ds *buf; ushort *array; }</pre> Für <code>cmd == SETALL</code> und <code>cmd == GETALL</code> ist <code>arg</code> ein Array vom Typ <code>unsigned short</code>, das die zu setzenden Semaphorwerte enthält bzw. die aktuellen Semaphorwerte zurückgibt. Für <code>cmd == SETVAL</code> ist <code>arg</code> ein entsprechender <code>int</code>-Ausdruck. Für <code>cmd == GETVAL</code> hat <code>arg</code> keine Bedeutung (siehe oben).
Beispiel:	<pre>#include <sys/ipc.h> #include <sys/sem.h> int semid; /* Identifikator für Semaphorgruppe */ unsigned short initarray[2]; /* Semaphoreingabewerte */ unsigned short outarray[2]; /* Semaphorausgabewerte */</pre>	

```

< Hier Zuweisung einer Gruppe aus zwei
  Semaphoren an semid. >
initarray[0] = initarray[1] = 1;
semctl(semid,0,SETALL,initarray);
/* Initialisiert beide Sem. mit Wert 1. */
semctl(semid,0,GETALL,outarray);
/* Schreibt die aktuellen Semaphorwerte
  nach outarray. */
semctl(semid,0,IPC_RMID,0);
/* Löscht die Semaphorgruppe. */

```

Bemerkung: Manche UNIX-Versionen, wie z.B. Solaris, akzeptieren bei SET-ALL und GETALL keinen Array, sondern verlangen die Übergabe einer Union. Das verkompliziert die Übergabe etwas:

```

union semun para;
ushort initarray[2];
< Hier Erzeugung einer Semaphorgruppe. >
initarray[0]=initarray[1]=1;
para.array=initarray; /* Pointerzuweisung! */
semctl(semid,0,SETALL,para);

```

semget:

Prototyp: `int semget(long key, int count, int flag);`

Effekt: Erzeugt eine neue Gruppe von Semaphoren oder gibt Zugriff auf eine bereits existierende Gruppe.

Includes: `#include <sys/ipc.h>`
`#include <sys/sem.h>`

Parameter: `long key` Externer Schlüssel für die Semaphorgruppe (frei durch den Programmierer definierbar oder unter mehreren Programmierern vereinbar – entweder Schlüssel für eine bereits existierende Gruppe oder neuer Schlüssel für neue Gruppe) oder `IPC_PRIVATE`, falls auf jeden Fall eine neue Semaphorgruppe erzeugt werden soll.

`int count` Anzahl der Semaphore in der Gruppe.

`int flag` Bitmuster: Die letzten neun Bitstellen geben die Zugriffsrechte auf die neuen Semaphore an, die Stellen davor spezifizieren, wie `semget()` ausgeführt werden soll. Ist hier das `IPC_CREAT`-Bit gesetzt und `key` noch

keine Semaphorgruppe zugeordnet, so wird eine neue Gruppe erzeugt.

Rückgabe: Interner Identifikator für die Semaphorgruppe, zur Verwendung in nachfolgenden Aufrufen von `semctl()` und `semop()`, oder -1 bei Fehler.

Beispiel:

```
#include <sys/ipc.h>
#include <sys/sem.h>
int semid, semid2, semid3;
    /* Identifikatoren für Semaphorgruppen */
/* Erzeugung der Gruppen: */
semid = semget(IPC_PRIVATE, 2, IPC_CREAT|0777);
    /* Erzeugt neue Gruppe von zwei Semaphoren
       mit allen Zugriffsrechten und weist
       semid den internen Identifikator zu. */
semid2 = semget(100, 3, IPC_CREAT|0777);
    /* Erzeugt unter dem ext. Schlüssel 100
       Gruppe von drei Semaphoren mit allen
       Zugriffsrechten und weist semid2 den
       Identifikator zu. */
semid3 = semget(100, 3, 0777);
    /* Ermittelt die unter dem externen
       Schlüssel 100 erzeugte Gruppe von drei
       Semaphoren und weist semid3 den Iden-
       tifikator zu. */
```

semop:

Prototyp: `int semop(int id, struct sembuf *ops, int count);`

Effekt: Manipulationen der Semaphorwerte durch eine Gruppe von P- und V-Operationen. Führt mindestens eine der P-Operationen zur Blockierung, so wird der Prozess blockiert und vorerst keine der Operationen ausgeführt.

Includes: `#include <sys/ipc.h>`
`#include <sys/sem.h>`

Parameter: `int id` Interner Identifikator für die Semaphorgruppe, wie von `semget()` geliefert.

`struct sembuf *ops`

Zeiger auf ein Feld mit Semaphoroperationen. Die Komponenten dieses Feldes sind vom Typ `struct sembuf`:

```

struct sembuf {
    short sem_num;
    Gruppeninterne Nummer des Semaphors, auf dem die
    Operation ausgeführt werden soll.

    short sem_op;
    Auszuführende Operation:
    Falls sem_op > 0: Addiere sem_op zum Semaphor-
    Wert und entblockiere alle wartenden Prozesse, die dann
    einer nach dem anderen erneut versuchen, ihre Sema-
    phoroperation erfolgreich auszuführen.
    Falls sem_op < 0: Falls Semaphor-Wert+sem_op>=0,
    addiere sem_op zum Semaphorwert, sonst blockiere.
    Falls sem_op = 0: Blockiere, bis Semaphorwert=0 ist

    short sem_flg;
    Legt fest, wie reagiert werden soll, wenn die Operation
    nicht ausführbar ist.
}

```

```

int count      Anzahl der auszuführenden Operationen in
ops.

```

Rückgabe: -1 bei Fehler, 0 oder (in manchen Systemen) Wert größer als 0 sonst

Beispiel:

```

#include <sys/ipc.h>
#include <sys/sem.h>
int semid;
    /* Identifikator für Semaphorgruppe */
struct sembuf sem_p[2];
    /* Für P-Operationen auf den Semaphoren */
struct sembuf sem_v[2];
    /* Für V-Operationen auf den Semaphoren */
< Hier Zuweisung einer Gruppe aus zwei
  Semaphoren an semid. >
/* Vorbereitung der P-Operation:
  1.) Setzen der Nummern der betroffenen
    Semaphore: */
sem_p[0].sem_num = 0;
sem_p[1].sem_num = 1;
/* 2.) Setzen der Op. "Dekrementieren der
  Semaphorwerte": */
sem_p[0].sem_op = sem_p[1].sem_op = -1;
/* 3.) Setzen der Flags: */
sem_p[0].sem_flg = sem_p[1].sem_flg = 0;
/* Analog: Vorbereitung der V-Operation. */
sem_v[0].sem_num = 0;

```

```

sem_v[1].sem_num = 1;
sem_v[0].sem_op = sem_v[1].sem_op = 1;
sem_v[0].sem_flg = sem_v[1].sem_flg = 0;
/* P-Operation: */
semop(semid, sem_p, 2);
/* V-Operation: */
semop(semid, sem_v, 2);

```

A.5 Shared Memory

shmat:

Prototyp:	<code>char *shmat(int id, char *addr, int flag);</code>	
Effekt:	Bindet Shared-Memory-Segment an prozesslokale Adresse.	
Includes:	<pre>#include <sys/ipc.h> #include <sys/shm.h></pre>	
Parameter:	<code>int id</code>	Interner Identifikator für das Shared-Memory-Segment, wie von <code>shmget()</code> geliefert.
	<code>char *addr</code>	Vorschlag für die prozesslokale Adresse, an die das Shared-Memory-Segment gebunden werden soll. 0, wenn kein solcher Vorschlag gemacht werden soll.
	<code>int flag</code>	Spezifiziert insbesondere, ob Lese- und Schreibzugriffe oder nur Lesezugriffe erlaubt sein sollen (<code>flag = 0</code> bzw. <code>flag = SHM_RDONLY</code>).
Rückgabe:	Prozesslokale Adresse, an die das Shared-Memory-Segment tatsächlich angebunden wurde (Pointer auf <code>char</code>), oder -1 bei Fehler.	
Beispiel:	<pre>#include <sys/ipc.h> #include <sys/shm.h> int shmid; /* Identifikator für Shared-Mem.-Segment */ int *p_int; /* Ganzzahl-Pointer zum Zugriff auf das Shared-Memory-Segment */ < Hier Zuweisung eines Shared-Memory- Segments an shmid. ></pre>	

```

p_int = (int *) shmat(shmid,0,0);
/* Bindung des Segments mit Lese- und
   Schreibrechten an die Adresse p_int. */
*p_int = 1234;
/* Zuweisung des Werts 1234 an die
   erste Stelle des Segments. */
*(p_int+1) = 4711;
/* Zuweisung an die zweite Stelle des
   Segments. Vorsicht vor Längenüber-
   schreitungen! */

```

shmctl:

Prototyp:	<pre>int shmctl(int id, int cmd, struct shmid_ds *buf);</pre>						
Effekt:	Verschiedene Steuerungsfunktionen auf einem Shared-Memory-Segment.						
Includes:	<pre>#include <sys/ipc.h> #include <sys/shm.h></pre>						
Parameter:	<table border="0" style="width: 100%;"> <tr> <td style="vertical-align: top; padding-right: 20px;">int id</td> <td>Interner Identifikator für das Shared-Memory-Segment, wie von shmget() geliefert.</td> </tr> <tr> <td style="vertical-align: top; padding-right: 20px;">int cmd</td> <td>Auszuführendes Kommando, z.B. IPC_RMID: Löschung des Tabelleneintrags und auch des gesamten Segments, wenn nicht noch andere Prozesse darauf Zugriff haben (wichtig, muss stets vor Beendigung des Programms ausgeführt werden!).</td> </tr> <tr> <td style="vertical-align: top; padding-right: 20px;">struct shmid_ds *buf</td> <td>Datenstruktur, die für einige Kommandos benötigt wird. Für cmd == IPC_RMID kann 0 als Parameter verwendet werden.</td> </tr> </table>	int id	Interner Identifikator für das Shared-Memory-Segment, wie von shmget() geliefert.	int cmd	Auszuführendes Kommando, z.B. IPC_RMID: Löschung des Tabelleneintrags und auch des gesamten Segments, wenn nicht noch andere Prozesse darauf Zugriff haben (wichtig, muss stets vor Beendigung des Programms ausgeführt werden!).	struct shmid_ds *buf	Datenstruktur, die für einige Kommandos benötigt wird. Für cmd == IPC_RMID kann 0 als Parameter verwendet werden.
int id	Interner Identifikator für das Shared-Memory-Segment, wie von shmget() geliefert.						
int cmd	Auszuführendes Kommando, z.B. IPC_RMID: Löschung des Tabelleneintrags und auch des gesamten Segments, wenn nicht noch andere Prozesse darauf Zugriff haben (wichtig, muss stets vor Beendigung des Programms ausgeführt werden!).						
struct shmid_ds *buf	Datenstruktur, die für einige Kommandos benötigt wird. Für cmd == IPC_RMID kann 0 als Parameter verwendet werden.						
Beispiel:	<pre>#include <sys/ipc.h> #include <sys/shm.h> int shmid; /* Identifikator für Shared-Mem.-Segment */ < Hier Zuweisung eines Shared-Memory- Segments an shmid. > shmctl(shmid,IPC_RMID,0); /* Löscht den Tabellen-Eintrag für shmid, auch das Segment, falls keine anderen Prozesse mehr Zugriff darauf haben. */</pre>						

shmdt:

Prototyp:	<code>int shmdt(char *addr);</code>
Effekt:	Löst Bindung zwischen Shared-Memory-Segment und prozess-lokaler Adresse.
Includes:	<code>#include <sys/ipc.h></code> <code>#include <sys/shm.h></code>
Parameter:	<code>char *addr</code> Prozesslokale Adresse, an die das Segment bislang angebunden ist.
Beispiel:	<pre> #include <sys/ipc.h> #include <sys/shm.h> int shmid; /* Identifikator für Shared-Mem.-Segment */ char *addr; /* Zur Aufnahme der prozesslok. Adresse */ < Hier Zuweisung eines Shared-Memory- Segments an shmid. > addr = shmat(shmid,0,0); /* Bindet Segment an addr. */ shmdt(addr); /* Löst Verbindung wieder. */ </pre>

shmget:

Prototyp:	<code>int shmget(long key, int size, int flag);</code>
Effekt:	Erzeugt ein neues Shared-Memory-Segment oder gibt Zugriff auf bereits existierendes Segment.
Includes:	<code>#include <sys/ipc.h></code> <code>#include <sys/shm.h></code>
Parameter:	<code>long key</code> Externer Schlüssel für das Shared-Memory-Segment (frei durch Programmierer definierbar oder unter mehreren Programmierern vereinbar – entweder Schlüssel für ein bereits existierendes Segment oder neuer Schlüssel für neues Segment) oder <code>IPC_PRIVATE</code> , falls auf jeden Fall ein neues Segment erzeugt werden soll.
	<code>int size</code> Größe des Segments in Bytes.

`int flag` Bitmuster: Die letzten neun Bitstellen geben die Zugriffsrechte auf das neue Segment an, die Stellen davor spezifizieren, wie `shmget()` ausgeführt werden soll. Ist hier das `IPC_CREAT`-Bit gesetzt und `key` noch kein Segment zugeordnet, so wird ein neues Segment erzeugt.

Rückgabe: Interner Identifikator für das Shared-Memory-Segment, zur Verwendung in nachfolgenden Aufrufen von `shmctl()`, `shmat()` und `shmdt()`, oder -1 bei Fehler.

Beispiel:

```
#include <sys/ipc.h>
#include <sys/shm.h>
int shmidx;
shmidx =
    shmget(IPC_PRIVATE, 8192, IPC_CREAT|0777);
/* Erzeugt neues Shared-Memory-Segment
   der Größe 8192 Bytes mit sämtlichen
   Zugriffsrechten und weist semid den
   internen Identifikator zu. */
```

A.6 Pipes

mkfifo:

Prototyp: `int mkfifo(char *pathname, int mode);`

Effekt: Erzeugt eine benannte Pipe.

Parameter: `char *pathname` Name oder Pfad der Pipe.

`int mode` Bitmuster für Zugriffsrechte der Pipe (siehe `creat()`).

Beispiel: Prozess 1 (es wird vorausgesetzt, dass er vor Prozess 2 startet):

```
#include <fcntl.h> /* für open()-Parameter */
main() {
    int fd; /* Deskriptor für die Pipe */
    /* Erzeugen einer benannten Pipe PIPE_1
       mit Lese-Schreib-Rechten für alle: */
    mkfifo("PIPE_1", 0666);
    /* Öffnen der Pipe zum Schreiben: */
    fd=open("PIPE_1", O_WRONLY);
```

```

/* Schreiben in die Pipe: */
write(fd, "TEST", 5);
... }
Prozess 2:
#include <fcntl.h> /* für open()-Parameter */
main() {
    char buffer[5];
    int fd;
    /* Öffnen der Pipe zum Lesen: */
    fd=open("PIPE_1", O_RDONLY);
    /* Lesen aus der Pipe: */
    read(fd, buffer, 5);
    printf("Gelesen: %s\n", buffer);
    ...
    /* Löschen der Pipe: */
    unlink("PIPE_1"); }

```

open:

Prototyp: `int open(char *pathname, int flag, int mode);`

Effekt: Eine allgemeine Beschreibung der Funktion steht im Abschnitt A.2 über Dateiverwaltung. Öffnet ein Prozess eine Pipe zum Schreiben, so wird er so lange blockiert, bis die Pipe auch am Leseende geöffnet wurde. Entsprechendes gilt für das Öffnen zum Lesen. `open()` kann jedoch durch Setzen des Flags `O_NDELAY` auch nichtblockierend aufgerufen werden.

pipe:

Prototyp: `int pipe(int filedes[2]);`

Effekt: Erzeugt eine unbenannte Pipe.

Parameter: `int filedes[2]` Zur Rückgabe der Dateideskriptoren für das Leseende (`filedes[0]`) und das Schreibende der Pipe (`filedes[1]`).

Beispiel:

```

main() {
    char buffer[5]; /* Puffer für Datenempfang */
    int fd[2];
    /* Deskr. für Lese- und Schreibende */
    /* Erzeugung einer unbenannten Pipe: */
    pipe(fd);
    if (fork()==0) {
        /* Sohnprozess als Schreiber: */
        /* Lesedeskriptor schließen, da nicht

```

```

        benötigt: */
close(fd[0]);
/* Bytes in die Pipe schreiben: */
write(fd[1], "TEST", 5);
...
exit(0); }
/* Vaterprozess als Leser: */
/* Schreibdeskriptor schließen,
   da nicht benötigt: */
close(fd[1]);
/* Bytes aus der Pipe lesen: */
read(fd[0], buffer, 5);
printf("Gelesen: %s\n", buffer);
... }

```

read:

Prototyp: `int read(int file_desc,
 char *buf, unsigned size);`

Effekt: Eine allgemeine Beschreibung der Funktion steht im Abschnitt A.2 über Dateiverwaltung. `read()` liest die Daten in der Reihenfolge aus der Pipe, in der sie hineingeschrieben wurden; sie werden dadurch aus der Pipe gelöscht. Es werden höchstens `size` Bytes gelesen – sind weniger vorhanden, entsprechend weniger. Ist die Pipe leer, so blockiert die Funktion normalerweise. Wurde die Pipe am Schreibende bereits geschlossen, so entspricht das einem „End of File“, und `read()` kehrt mit dem Rückgabewert 0 zurück.

write:

Prototyp: `int write(int file_desc,
 char *buf, unsigned size);`

Effekt: Eine allgemeine Beschreibung der Funktion steht im Abschnitt A.2 über Dateiverwaltung. `write()` schreibt Daten sequentiell in eine Pipe und blockiert im Normalfall, sobald diese voll ist (eine Pipe kann nur eine beschränkte Anzahl von Daten – mindestens 4 KByte – speichern). Wurde die Pipe am Leseende bereits geschlossen, so kehrt `write()` mit einer Fehlermeldung zurück.

A.7 Message Queues

msgctl:

Prototyp: `int msgctl(int id, int cmd,
 struct msqid_ds *buf);`

Effekt: Verschiedene Steuerungsfunktionen auf einer Message Queue.

Includes: `#include <sys/ipc.h>
 #include <sys/msg.h>`

Parameter: `int id` Interner Identifikator für die Message Queue, wie von `msgget()` geliefert.

`int cmd` Auszuführendes Kommando, z.B. `IPC_RMID`: Löschung des Tabelleneintrags und der Message Queue (wichtig, muss vor Ende des Programms ausgeführt werden!).

`struct shmid_ds *buf` Datenstruktur, die für einige Kommandos benötigt wird. Für `cmd == IPC_RMID` kann 0 als Parameter verwendet werden.

Beispiel: `#include <sys/ipc.h>
 #include <sys/msg.h>
 int msgid;
 < Hier Zuweisung einer Message Queue
 an msgid. >
 msgctl(msgid,IPC_RMID,0);
 /* Löscht den Tabellen-Eintrag für msgid
 und auch zugehörige Message Queue. */`

msgget:

Prototyp: `int msgget(long key, int flag);`

Effekt: Erzeugt eine neue Message Queue oder gibt Zugriff auf eine bereits existierende Queue.

Includes: `#include <sys/ipc.h>
 #include <sys/msg.h>`

Parameter: `long key` Externer Schlüssel für die Message Queue (frei durch Programmierer definierbar oder

unter mehreren Programmierern vereinbar – entweder Schlüssel für eine bereits existierende Message Queue oder neuer Schlüssel für neue Message Queue)
oder `IPC_PRIVATE`, falls auf jeden Fall eine neue Message Queue erzeugt werden soll.

`int flag` Bitmuster: Die letzten neun Bitstellen geben die Zugriffsrechte auf die neue Message Queue an, die Stellen davor spezifizieren, wie `msgget()` ausgeführt werden soll. Ist hier das `IPC_CREAT`-Bit gesetzt und `key` noch keine Message Queue zugeordnet, so wird eine neue Message Queue erzeugt.

Rückgabe: Interner Identifikator für die Message Queue, zur Verwendung in nachfolgenden Aufrufen von `msgctl()`, `msgsnd()` und `msgrcv()`, oder -1 bei Fehler.

Beispiel:

```
#include <sys/ipc.h>
#include <sys/msg.h>
int msgid;
    /* Identifikator der Message Queue */
msgid = msgget(IPC_PRIVATE,IPC_CREAT|0777);
    /* Erzeugt neue Message Queue mit sämt-
        lichen Zugriffsrechten und weist msgid
        einen entsprechenden Ident. zu. */
```

msgrcv:

Prototyp: `int msgrcv(int id, struct msgbuf *msgp,
 int size, long type, int flag);`

Effekt: Empfängt die erste Nachricht eines bestimmten Typs aus einer Message Queue.

Includes: `#include <sys/ipc.h>`
 `#include <sys/msg.h>`

Parameter: `int id` Interner Identifikator für die Message Queue, wie von `msgget()` geliefert.

`struct msgbuf *msgp`
 Zeiger auf einen Puffer, in den die empfangene Nachricht geschrieben werden soll. Der Puffer ist in der einfachsten Form eine

Struktur aus einer Ganzzahl (= Nachrichtentyp) und einem Character-Array (= Text der Nachricht). Der entsprechende Strukturtyp ist in `msg.h` z.B. so definiert:

```
struct msgbuf {
    long mtype;
    char mtext[1]; }
```

Es ist aber möglich, Nachrichten auch anderer Längen zu empfangen. Ebenso sind Nachrichten-Strukturen möglich, die anstelle des `char`-Arrays `mtext` eine Folge aus beliebigen anderen Variablen enthalten (siehe Beispiel).

<code>int size</code>	Maximal zulässige Länge der Nachricht (= Länge von <code>*msgp</code> in Bytes, <i>ohne</i> <code>mtype</code>). Am wenigsten fehleranfällig ist die Ermittlung dieser Länge mit Hilfe der Funktion <code>sizeof()</code> .
<code>long type</code>	Gewünschter Typ der Nachricht (0 = beliebiger Typ).
<code>int flag</code>	Spezifiziert die gewünschte Reaktion, wenn momentan keine entsprechende Nachricht in der Message Queue vorhanden ist: Blockierung, wenn das <code>IPC_NOWAIT</code> -Bit nicht gesetzt ist; unmittelbare Rückkehr (mit Rückgabewert -1), wenn das <code>IPC_NOWAIT</code> -Bit gesetzt ist.

Rückgabe: Anzahl der empfangenen Bytes (ohne `mtype`) oder -1 bei Fehler.

Beispiel:

```
#include <sys/ipc.h>
#include <sys/msg.h>
int msgid;
/* Identifikator der Message Queue */
int rec;
/* Anzahl der empfangenen Bytes */
struct {
    long mtype;
    char mtext[1024];
} message1; /* für 1. Nachricht */
struct {
    long mtype;
    int wert1, wert2, wert3;
} message2; /* für 2. Nachricht */
```

```

< Hier Zuweisung einer Message Queue
  an msgid. >
rec = msgrcv(msgid,&message1,
             sizeof(message1)-sizeof(long),
             3,IPC_NOWAIT);
/* Empfängt Nachricht des Typs 3 (siehe
   mtype) aus der Message Queue und
   schreibt sie nach message1. Maximal
   zulässige Länge ist 1024 Bytes. Sofor-
   tige Rückkehr mit Rückgabewert -1,
   falls keine Nachricht des gewünschten
   Typs vorhanden ist. Ansonsten Rückgabe
   der Anzahl der empfangenen Bytes. */
msgrcv(msgid,&message2,
       sizeof(message2)-sizeof(long),0,0);
/* Empfängt Nachricht mit beliebigem
   mtype-Eintrag aus der Message Queue
   und schreibt sie nach message2.
   Es wird vorausgesetzt, dass die ge-
   sendete Nachricht drei Integerwerte
   enthält. Blockierung, falls keine
   Nachricht vorhanden ist. */

```

msgsnd:

Prototyp: int msgsnd(int id, struct msgbuf *msgp,
 int size, int flag);

Effekt: Sendet eine Nachricht in eine Message Queue.

Includes: #include <sys/ipc.h>
 #include <sys/msg.h>

Parameter: int id Interner Identifikator für die Message
 Queue, wie von msgget () geliefert.

```

struct msgbuf *msgp
    Zeiger auf die zu sendende Nachricht. Die
    Nachricht ist in im einfachsten Fall eine
    Struktur aus einer Ganzzahl (= Nachrichten-
    typ, durch Benutzer frei wählbar, muss im-
    mer ungleich 0 sein!!) und einem char-
    Array (= Text der Nachricht). Der Struktur-
    typ ist in msg.h z.B. so definiert:
    struct msgbuf {
        long mtype;
        char mtext[1]; }

```


Es ist aber möglich, Nachrichten auch anderer Längen zu verschicken und structs zu verwenden, die statt des char-Arrays mtext eine Folge aus beliebigen anderen Variablen enthalten (siehe Beispiel).

int size	Länge der Nachricht (= Länge von *msgp in Bytes, <i>ohne</i> mtype). Am wenigsten fehleranfällig ist die Ermittlung dieser Länge aus *msgp mit Hilfe der Funktion sizeof().
int flag	Spezifiziert gewünschte Reaktion, wenn momentan nicht genügend Platz in der Message Queue vorhanden: Blockierung, wenn das IPC_NOWAIT-Bit nicht gesetzt ist; unmittelbare Rückkehr mit Fehlermeldung, wenn das IPC_NOWAIT-Bit gesetzt ist.

Beispiel:

```
#include <strings.h>
#include <sys/ipc.h>
#include <sys/msg.h>
int msgid;
    /* Identifikator der Message Queue */
struct {
    long mtype;
    char mtext[10];
} message1; /* für 1. Nachricht */
struct {
    long mtype;
    int wert1, wert2, wert3;
} message2; /* für 2. Nachricht */
< Hier Zuweisung einer Message Queue
  an msgid. >
message1.mtype = 3;
    /* Setzt Nachrichtentyp. */
strcpy(message1.mtext, "Hallo");
    /* Setzt Nachrichtentext. */
msgsnd(msgid, &message1,
        sizeof(message1) - sizeof(long), 0);
    /* Schickt Nachricht "Hallo" mit Typ 3
       in die Queue. Dabei Blockierung, wenn
       nicht mehr genug Platz in der Queue. */
message2.mtype = 1;
    /* Setzt Nachrichtentyp. */
message2.wert1 = 4; /* und -werte */
```

```

message2.wert2 = 8;
message2.wert3 = 16;
msgsnd(msgid, &message2,
        sizeof(message2) - sizeof(long), 0);
/* Schickt Nachricht mit drei Integer-
   werten in die Queue. Blockierung,
   wenn nicht mehr genug Platz. */

```

A.8 Sockets

accept:

Prototyp:	<code>int accept(int sd, struct sockaddr *address, int *addrlen);</code>
Effekt:	Akzeptiert einen mit <code>connect()</code> eingehenden Verbindungsaufbauwunsch und erzeugt für die folgende Kommunikation zwischen Client und Server eine eigene Socket. Blockiert, solange kein Verbindungsaufbauwunsch vorliegt.
Includes:	<code>#include <sys/socket.h></code>
Parameter:	<code>int sd</code> Socket-Deskriptor. <code>struct sockaddr *address</code> Zeiger auf Struktur für Rückgabeparameter: Domain (siehe <code>socket()</code>), Name / Adresse der Socket des Clients. <code>int *addrlen</code> Zeiger auf Variable für Rückgabeparameter: tatsächliche Länge des Namens / der Adresse der Client-Socket.
Rückgabe:	Socket-Deskriptor für die folgende Kommunikation zwischen Server und Client oder -1 bei Fehler.
Beispiel:	Siehe unten.

bind:

Prototyp:	<code>int bind(int sd, struct sockaddr *address, int length);</code>
Effekt:	Bindet einen Namen oder eine Adresse an eine Socket.

Includes: `#include <sys/socket.h>`

Parameter: `int sd` Socket-Deskriptor.

`struct sockaddr *address`

Zeiger auf eine Struktur, die die Domain (siehe `socket()`) sowie einen Namen oder eine Adresse für die Socket angibt (Name/Adresse sind domain- und protokollspezifisch: Dateiname für UNIX-Domain, IP-Adresse und Socket-Identifikation auf dem Empfänger-System für Internet-Domain).

`int length` Länge von `struct sockaddr`.

Beispiel: Siehe unten.

close:

Prototyp: `int close(int sd);`

Effekt: Schließt eine Socket und beendet damit die Verbindung.

Parameter: `int sd` Socket-Deskriptor.

Beispiel: Siehe unten.

Bemerkung: Der Socket-Eintrag im Dateisystem bleibt auch nach Ausführung von `close()` erhalten. Er kann per `unlink()` oder – von der Benutzerschnittstelle aus – per `rm` gelöscht werden. Dies sollte unbedingt geschehen, bevor das entsprechende Programm erneut ausgeführt wird, um Fehler zu vermeiden!

connect:

Prototyp: `int connect(int sd, struct sockaddr *address, int length);`

Effekt: Baut einen Kommunikationspfad zwischen zwei Sockets auf.

Includes: `#include <sys/socket.h>`

Parameter: `int sd` Socket-Deskriptor der ersten Socket.

`struct sockaddr *address`

Zeiger auf eine Struktur, die die Domain (siehe `socket()`) sowie einen Namen oder eine Adresse für die Socket angibt (Name/Adresse sind domain- und protokollspezifisch).

zifisch: Dateiname für UNIX-Domain, IP-Adresse und Socket-Identifikation auf dem Empfänger-System für Internet-Domain).

`int length` Länge des Namens / der Adresse.

Beispiel: Siehe unten.

listen:

Prototyp: `int listen(int sd, int qlength);`

Effekt: Erzeugt eine Warteschlange für mit `connect()` eingehende Verbindungsaufbauwünsche bezüglich einer Socket.

Includes: `#include <sys/socket.h>`

Parameter: `int sd` Socket-Deskriptor.

`int qlength` Länge der Warteschlange.

Beispiel: Siehe unten.

recv, read:

Prototyp: `int recv(int sd, char *buf,
 int length, int flags);`
`int read(int sd, char *buf, int length);`

Effekt: Liest Daten von einer Socket ein. `recv()` ist die allgemeine Empfangsfunktion für Sockets, die zuvor durch `connect()` verbunden wurden. Für verbindungsorientierte Sockets (Stream-Sockets) kann hier auch der `read()`-Aufruf benutzt werden. Für verbindungslose Sockets (Datagramm-Sockets) gibt es eine Empfangsfunktion `recvfrom()`, die zusätzlich Adressparameter enthält.

Includes: `#include <sys/socket.h>`

Parameter: `int sd` Socket-Deskriptor.

`char *buf` Zeiger auf einen Speicherbereich zur Aufnahme der empfangenen Daten.

`int length` Erwartete Länge der Daten (in Bytes).

`int flags` Flags zur Steuerung des Empfangsvorgangs (z.B. um eintreffende Daten nur „anzusehen“, aber nicht aus dem Empfangsbereich zu entfernen).

Rückgabe: Anzahl der tatsächlich empfangenen Daten (in Bytes) oder -1 bei Fehler.

Beispiel: Siehe unten.

send, write:

Prototyp:

```
int send(int sd, char *buf,
          int length, int flags);
int write(int sd, char *buf, int length);
```

Effekt: Gibt Daten auf eine Socket aus. `send()` ist die allgemeine Sendefunktion für Sockets, die zuvor durch `connect()` verbunden wurden. Für verbindungsorientierte Sockets (Stream-Sockets) kann hier auch der `write()`-Aufruf benutzt werden. Für nicht verbundene verbindungslose Sockets (Datagramm-Sockets) gibt es eine Sendefunktion `sendto()`, die zusätzlich Adressparameter enthält.

Includes: `#include <sys/socket.h>`

Parameter:	<code>int sd</code>	Socket-Deskriptor.
	<code>char *buf</code>	Zeiger auf einen Speicherbereich mit den zu sendenden Daten.
	<code>int length</code>	Länge der zu sendenden Daten (in Bytes).
	<code>int flags</code>	Flags zur Steuerung des Sendevorgangs.

Rückgabe: Anzahl der tatsächlich gesendeten Daten (in Bytes) oder -1 bei Fehler.

Beispiel: Siehe unten.

socket:

Prototyp:

```
int socket(int format,
           int type, int protocol);
```

Effekt: Erzeugt eine neue Socket.

Includes: `#include <sys/socket.h>`

Parameter:	<code>int format</code>	Spezifikation der Domain (<code>AF_UNIX</code> für UNIX-, <code>AF_INET</code> für Internet-Domain).
	<code>int type</code>	Art der Kommunikation (<code>SOCK_STREAM</code> für verbindungsorientierte Kommunikation, <code>SOCK_DGRAM</code> für verbindungslose).

int protocol Verwendetes Kommunikationsprotokoll.
Wird hier der Wert 0 angegeben, so bestimmt das System selbst ein geeignetes Protokoll.

Rückgabe: Socket-Deskriptor oder -1 bei Fehler.

Beispiel: Siehe unten.

Beispiel zu Sockets:

System aus Server und Client (nach [Bach86]):

```
Server: #include <sys/socket.h>
main() {
    int sd_acc, sd_comm;
    /* Socketdeskriptoren für die listen()-
       Operation des Servers bzw. die Kommunika-
       tion zwischen Server und Client */
    char buf[256];
    /* Zur Aufnahme der Nachricht des Clients */
    struct sockaddr server_addr, client_addr;
    /* Socketadressen für die Kommunikation vom
       Client zum Server und umgekehrt */
    int addr_len;
    /* Länge von client_addr */
    sd_acc = socket(AF_UNIX, SOCK_STREAM, 0);
    /* Generierung einer Socket des Typs "Stream"
       (= verbindungsorientierte, zuverlässige
       Kommunikation) in der UNIX-Domain, Proto-
       koll wird durch das System bestimmt. */
    server_addr.sa_family = AF_UNIX;
    strcpy(server_addr.sa_data, "Socket_1");
    bind(sd_acc, &server_addr,
         sizeof(struct sockaddr));
    /* Bindet den Namen "Socket_1" an die neue
       Socket. */
    listen(sd_acc, 1);
    /* Prozess ist bereit, Verbindungsaufbau-
       wünsche über die neue Socket entgegenzu-
       nehmen. Dabei kann höchstens ein noch
       nicht akzeptierter Wunsch zwischen-
       gespeichert werden. */
    for (;;) {
        /* Endlosschleife, in der Verbindungsaufbau-
           wünsche akzeptiert werden: */
```

```

sd_comm = accept(sd_acc,&client_addr,&addr_len);
/* Akzeptiert Verbindungsaufbauwunsch eines
   Clients.
   &client_addr ist der Socket-Name, der für
   die (Rück-)Kommunikation vom Server zum
   Client benutzt werden soll; addr_len
   dessen Länge. Die folgende Kommunikation
   vom Client zum Server verläuft anschlie-
   ßend über eine neue Socket sd_comm, wäh-
   rend sd_acc für das Akzeptieren weiterer
   Verbindungsaufbauwünsche verwendet werden
   kann. */
if (fork() == 0) {
    /* Erzeugung eines Sohnprozesses, der die
       neue Verbindung bedienen kann. Der Vater-
       prozess bearbeitet in der Endlosschleife
       neue Aufbauwünsche. */
    close(sd_acc);
    /* Der Sohnprozess benötigt sd_acc nicht
       mehr, da er ja keine Aufbauwünsche akzep-
       tiert. */
    read(sd_comm,buf,sizeof(buf));
    /* Einlesen einer Nachricht des Clients. */
    < Hier Bedienung des Clients. >
    exit(0); } /* Ende des Sohnprozesses. */
/* Vaterprozess: */
close(sd_comm);
/* Der Vaterprozess benötigt sd_comm nicht
   mehr, da ja der Sohnprozess den Client
   bedient. */
} /* Ende der for-Schleife. */
} /* Ende des Server-Programms. */

```

```

Client: #include <sys/socket.h>
main() {
    int sd_out;
    /* Deskriptor für die lokale Socket, die mit
       der entfernten Socket des Servers ver-
       bunden werden soll. */
    int error;
    /* Zur Fehlerabprüfung des connect-Aufrufs */
    struct sockaddr server_addr;
    /* Zur Kommunikation vom Client zum Server */
    sd_out = socket(AF_UNIX,SOCK_STREAM,0);
    /* Generierung einer Socket des Typs "Stream"
       (= verbindungsorientierte, zuverlässige

```

```

        Kommunikation) in der UNIX-Domain, Proto-
        koll wird durch das System bestimmt. */
server_addr.sa_family = AF_UNIX;
strcpy(server_addr.sa_data, "Socket_1");
error = connect(sd_out, &server_addr,
                sizeof(struct sockaddr));
/* Verbindet die lokale Socket mit der ent-
   fernten Socket, an die der Server den
   Namen "Socket_1" gebunden hat. */
if (error == -1)
    exit(-1);
/* Terminierung, falls Verbindungsaufbau
   fehlgeschlagen. */
write(sd_out, "Hallo", 6);
/* Schickt die Nachricht "Hallo"
   an den Server. */
close(sd_out);
/* Schließt lokale Socket, da nicht mehr
   benötigt. */
} /* Ende des Client-Programms. */

```

A.9 Programmieraufgaben

1. Ausführung von Prozessen

Schreiben Sie ein C-Programm für einen Vaterprozess, der zwei Sohnprozesse startet. Sie können dabei den Code für die Söhne direkt in die entsprechenden `if`-Zweige des Programms schreiben, müssen also keine gesonderten Programmdateien für die Söhne erstellen.

Die Söhne sollen den Prozessor unterschiedlich stark belasten: Sohn 1 schläft für 15 Sekunden und tut sonst nichts. Sohn 2 führt eine Endlosschleife aus, in deren Körper zwei `int`-Zahlen addiert werden. Sonst soll er nichts tun, also insbesondere nicht schlafen. Die Additionen dienen nur dazu, eine Prozessorbelastung zu erzeugen; ihre Eingabewerte und Ergebnisse sind uninteressant und sollen auch nicht ausgegeben werden. Der Vaterprozess soll Folgendes tun:

- Ausgeben seiner eigenen Prozess-ID und der zwei IDs seiner Söhne.
- Sechsmal hintereinander:
 - Schlafen für eine Sekunde
 - Ausgabe der aktuellen Zeit
 - Ausgabe von Informationen über die aktuell von Ihnen gestarteten Prozesse, wobei insbesondere die jeweiligen Prioritäten der Söhne ausgegeben werden sollen.
- Löschen der Sohnprozesse.

Stellen Sie eine Tabelle mit den ausgegebenen Prioritäten der Sohnprozesse zu den einzelnen Zeitpunkten auf. Welchen Effekt beobachten Sie und wie erklären Sie ihn sich (unter UNIX, nicht unter Linux)? Welche Informationen, die mit der Priorität in Zusammenhang stehen, werden mit ausgegeben? Die ausgegebenen Zeitpunkte liegen nicht immer genau eine Sekunde auseinander. Wie ist das zu erklären?

Hinweise zur Programmierung:

- Zum Ausgeben der Informationen können Sie die C-Funktion `system()` zusammen mit geeigneten UNIX-Shell-Kommandos verwenden: `system(String)` interpretiert `String` als Kommando der UNIX-Benutzerschnittstelle und führt es aus.
- Um sich die Prioritäten der Prozesse anzeigen zu lassen, müssen Sie das `ps`-Kommando mit geeigneten Optionen verwenden. Eine Erläuterung dieser Optionen können Sie mit `man ps` abrufen.

2. Primitive Synchronisation von Prozessen

Hier geht es um ein Programm, das die Benutzung eines langsamen Ausgabegeräts durch zwei Prozesse simuliert. In diesem Programm erzeugt der Vaterprozess unmittelbar hintereinander zwei Sohnprozesse. Sohn1 wartet zunächst zwei Sekunden, gibt dann sechs Zeilen mit jeweils dem Inhalt „Erster“ auf den Bildschirm aus und terminiert dann. Zwischen den einzelnen Zeilenausgaben schläft er jeweils für eine Sekunde, was z.B. einem Drucker entspricht, dessen Druckzeit pro Zeile eine Sekunde beträgt. Sohn2 verhält sich ähnlich, wartet allerdings zu Beginn nicht und gibt jeweils (in Sekundenabständen) den Text „---Zweiter“ aus. Der Vater wartet auf die Terminierung beider Söhne und gibt dann eine Abschlussmeldung aus.

a.) Schreiben Sie ein entsprechendes C-Programm, wobei Sie noch keine Synchronisationsmechanismen verwenden, und führen Sie es aus.

- Welcher Prozess beginnt als Erster mit seiner Ausgabe? Warum?
- Warum erscheinen die Ausgaben der beiden Prozesse auf dem Bildschirm durchmischt?

b.) Erweitern Sie nun das Programm so, dass zunächst der erste Sohn seine Ausgabe vollständig abschließt und erst dann der zweite Sohn mit seiner Ausgabe beginnt. Sie sollen dabei die Programmstruktur aus Aufgabe a.) nicht verändern, sondern nur um Synchronisationsfunktionen ergänzen.

Diese Synchronisation soll über Signale geschehen: Der Vater startet wie bisher beide Söhne unmittelbar hintereinander. Sohn1 beginnt wie oben nach zwei Sekunden Wartezeit mit seiner Ausgabe. Währenddessen sollen der Vater und der Sohn jeweils auf ein Signal eines anderen Prozesses warten. Hat Sohn1 seine Ausgabe abgeschlossen, so schickt er ein Signal an den Vater, der dann wiederum ein Signal an Sohn2 sendet. Sohn2 beginnt daraufhin mit seiner Ausgabe.

Hinweise zur Programmierung:

- Ein Prozess findet die PID seines Vaters durch Aufruf von `getppid()`.
- Ein Prozess wartet mit Aufruf von `pause()` auf das Eintreffen eines Signals.
- Ein Signal wird mit der `kill()`-Funktion verschickt. Benutzen Sie hier das erste benutzerdefinierte Signal (d.h. die Konstante `SIGUSR1` als zweiten Parameter von `kill()`).
- Normalerweise verursacht das Eintreffen eines Signals bei einem Prozess dessen sofortige Terminierung. Um das zu vermeiden, muss das Signal abgefangen werden. Das macht man, indem man dem Signal mit Hilfe der Schnittstellenfunktion `signal()` eine C-Funktion als „Signalhandler“ zuordnet.
- Ein Beispiel für die Benutzung von `pause()`, `kill()` und `signal()` finden Sie in der Übersicht über die Schnittstellenfunktionen unter der Beschreibung von `kill()`.

3. Reihenfolge und wechselseitiger Ausschluss für zwei Druckerprozesse

Gehen Sie von Ihrer Lösung von Aufgabe 2.a aus, also dem Programm mit zwei Sohnprozessen ohne Synchronisationsmechanismen.

- a.) Führen Sie in Ihrem Programm einen Semaphor ein, der sicherstellt, dass der zweite Sohn mit der Ausgabe wartet, bis der erste alle seine Ausgaben abgeschlossen hat. Der Semaphor setzt also eine Reihenfolgebedingung durch – wie die Signale aus Aufgabe 2.b.

Der Vaterprozess muss hier den Semaphor geeignet erzeugen, initialisieren und löschen (Operationen `semget()` und `semctl()`), die Sohnprozesse müssen den Semaphor geeignet benutzen (`semop()`).

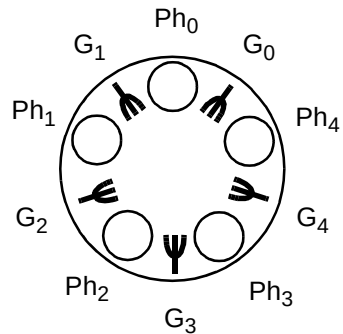
- b.) Nehmen Sie nochmals das ursprüngliche Programm von Aufgabe 2.a und betten Sie jeweils die sechs Ausgabeoperationen der Sohnprozesse in eine Endlosschleife ein. Jeder Prozess gibt also einen Block von sechs Zeilen aus (wobei zwischen den einzelnen Zeilen je eine Sekunde gewartet wird) und beginnt dann wieder von vorn. Der Vaterprozess soll die Söhne nach einer bestimmten Zeit (z.B. 30 Sekunden) beenden.

Die Reihenfolge der Ausgaben sei nun gleichgültig. Stattdessen soll sichergestellt werden, dass ein Prozess jeweils sechs Zeilen vollständig hintereinander ausgibt, ohne dass ihm dabei der andere Prozess dazwischenkommt. Führen Sie einen Semaphor ein, der diesen wechselseitigen Ausschluss sicherstellt.

Denken Sie daran, Semaphore nach ihrer Benutzung wieder zu löschen! Hierzu dient bekanntlich die Schnittstellenfunktion `semctl()` oder (nach Programmabstürzen) der Befehl `ipcrm` in Verbindung mit `ipcs`. Der `man`-Befehl liefert detaillierte Informationen über diese Befehle.

4. Das Philosophenproblem (nach [Dijk65])

Fünf Philosophen $Ph_0, \dots, Ph_4$ leben in einem Elfenbeinturm und tun nichts weiter als denken und essen. Als Gericht gibt es eine besonders glitschige Abart von Spaghetti Carbonara, die man nur mit zwei Gabeln G_i essen kann. Hierzu ist ein runder Tisch vorhanden, auf dem sich für jeden Philosophen ein eigener Teller und zwischen je zwei Tellern eine Gabel befinden. Ein Philosoph muss sich also seine Gabeln mit seinen Nachbarn teilen und kann nicht gemeinsam mit ihnen speisen. Zudem darf er nur von seinem eigenen Teller essen und nicht auf einen anderen ausweichen.



Das Leben eines Philosophen verläuft damit zyklisch: Zunächst denkt er eine Weile nach, bis er schließlich Hunger bekommt und zu Tisch schreitet. Dort versucht er, seine Gabeln zu bekommen; er kann erst mit dem Essen beginnen, wenn er die benötigten Gabeln für sich belegen konnte. Der Philosoph isst dann eine gewisse Zeit, gibt seine Gabeln anschließend wieder frei und beginnt erneut zu denken, bis er wieder Hunger bekommt und das Spiel von vorn beginnt.

Die Aufgabe besteht nun darin, die fünf Philosophen durch fünf UNIX-Prozesse zu programmieren, die das beschriebene philosophische Verhalten zeigen und zudem die Synchronisationsbedingungen bezüglich der Gabeln (wechselseitiger Ausschluss) beachten.

Ordnen Sie dazu jeder Gabel einen eigenen Semaphor zu, initialisieren Sie ihn mit dem Zählerwert 1 und realisieren Sie das Belegen und Freigeben der Gabel durch entsprechende P- und V-Operationen. Jeder Philosoph, der an den Tisch tritt, führt dabei zuerst eine P-Operation auf dem Semaphor seiner rechten Gabel und dann eine P-Operation auf dem Semaphor seiner linken Gabel aus. Diese Operationen sollen hier getrennt voneinander ausgeführt werden. Dass dabei Deadlocks auftreten können, interessiert uns noch nicht. Hinweis: Legen Sie eine Gruppe mit fünf Semaphoren an, da dies die Lösung von Aufgabe 5 erleichtern wird.

Die Dauer des Denkens und Essens soll durch `sleep()`-Operationen modelliert werden, bei denen die Schlafzeiten zweckmäßigerweise zwischen 2 und 4 Sekunden liegen. Der Vaterprozess soll das Programm nach einer bestimmten Zeit beenden.

Damit man den Programmablauf am Bildschirm verfolgen kann, soll ein Philosoph bei den folgenden Ereignissen eine Bildschirmausgabe machen:

- Wenn er mit dem Denken fertig ist und mit dem Essen beginnen möchte (unmittelbar vor dem Versuch, Gabeln zu belegen).
- Wenn er die rechte Gabel erhalten hat.
- Wenn er die linke Gabel erhalten hat.
- Wenn er mit dem Essen fertig ist (unmittelbar vor der Rückgabe der Gabeln).

Dabei soll jeweils die Nummer des Philosophen und das Ereignis angezeigt werden.

5. Deadlockbehandlung im Philosophenproblem

- a.) Demonstrieren Sie für Ihre Lösung von Aufgabe 4, dass Deadlocks auftreten können. Dies können Sie dadurch erreichen, dass Sie zwischen den Griffen zur rechten und zur linken Gabel eine Pause machen (z.B. durch `sleep(1)`).
- b.) Programmieren Sie, getrennt voneinander, die folgenden beiden Alternativlösungen, bei denen kein Deadlock auftreten kann:
 - Vor dem Tisch sitzt ein Wächter, der höchstens vier Philosophen gleichzeitig heranlässt. Benutzen Sie hier noch *keine* Mehrfachoperationen auf Semaphoren!
 - Jeder Philosoph versucht, seine beiden Gabeln gleichzeitig zu greifen. Gelingt das nicht, so nimmt er keine von beiden und wartet. Benutzen Sie hierfür die Möglichkeit, P-Operationen auf mehreren Semaphoren gleichzeitig auszuführen.

Lassen Sie Ihre Philosophen jeweils wieder die Bildschirmausgaben wie in Aufgabe 4 machen.

6. Shared Memory zur Verbesserung der Ausgabe im Philosophenproblem

Die Ausgaben der Philosophenprogramme waren bislang recht unübersichtlich. Günstiger wäre es, einen allgemein zugreifbaren Array zu haben, in den jeder Philosoph seinen aktuellen Status einträgt (D = denkend, W = wartend auf Gabeln, E = essend). Ein Philosoph könnte dann den Gesamtstatus aller Prozesse auf den Bildschirm ausgeben. Eine Ausgabe der Form „E W E D D“ würde z.B. anzeigen, dass Ph_0 und Ph_2 gerade essen, Ph_1 auf Gabeln wartet und Ph_3 und Ph_4 denken.

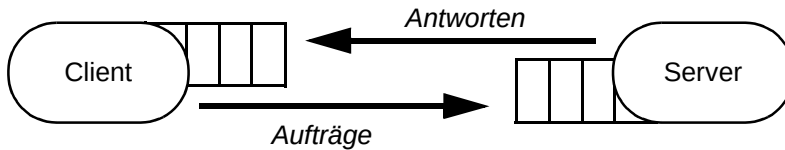
Erweitern Sie die zweite Lösung aus Aufgabe 5.b durch einen solchen Mechanismus. Hierzu muss Ihr Vaterprozess für den Array ein Shared-Memory-Segment anlegen, auf das die Söhne dann zugreifen. Die Söhne müssen hier ihre Statusänderungen eintragen und nach jeder Änderung den Array wie oben skizziert ausgeben.

Hinweise zur Programmierung:

- Der Vaterprozess muss den Array geeignet initialisieren.
- Die Änderung des Arrays und die nachfolgende Ausgabe müssen ununterbrechbar, also zu anderen Prozessen wechselseitig ausgeschlossen erfolgen. Führen Sie einen neuen Semaphor ein, der das sicherstellt.
- Die Statusänderung von D nach W sollte unmittelbar *vor* den P-Operationen auf den Gabeln vermerkt und angezeigt werden – würde man es erst nach der P-Operation tun, wäre ja der Wartezustand schon wieder verlassen. Der Übergang von W nach E erfolgt dann *nach* der P-Operation, aber *vor* dem Essens-`sleep()`.
- Die Statusänderung von E nach D und die zugehörige Ausgabe sollten unmittelbar *vor* den V-Operationen auf den Gabeln erfolgen. Ansonsten könnte es zu verwirrenden Ausgaben kommen, da UNIX zwischen V-Operation und Ausgabe meist zu einem anderen Prozess umschaltet, der dann mit seinen Ausgaben „dazwischenfunk“.
- Die `printf()`-Aufrufe aus Aufgabe 5 sollten aus dem Programm entfernt werden.

7. Client/Server mit Message Queues

In dieser Aufgabe sollen Sie eine einfache Client-Server-Anwendung schreiben. Die grundlegende Systemstruktur wird durch die folgende Abbildung gegeben:



Ein Clientprozess hat die Möglichkeit, einen Dienst aufzurufen, den ein Serverprozess anbietet. Er muss dazu entsprechende Aufträge mit den erforderlichen Parametern über die Message Queue des Servers übergeben. Der Server entnimmt jeweils einen Auftrag aus der Queue, bearbeitet ihn und schreibt das Ergebnis in die Message Queue des Clients. Message Queues werden hier also als Ports benutzt.

Der Dienst ist im Prinzip beliebig (z.B. Dateizugriffe oder Datenbankoperationen). Um aber den Programmieraufwand hier minimal zu halten, soll der Server nur zwei Ganzzahlwerte addieren und die Summe zurückliefern. Eine solche Auftragsbearbeitung soll eine Sekunde dauern.

Der Client soll unmittelbar hintereinander fünf Aufträge übergeben und dann auf die Antworten warten. Kommt eine Antwort zurück, so soll das Ergebnis zusammen mit den Übergabeparametern sofort auf den Bildschirm ausgegeben werden.

Beachten Sie bei der Implementierung die folgenden Punkte:

- Der Server läuft zweckmäßigerweise in einer Endlosschleife, während der Client nach der Ausgabe der fünf Ergebnisse terminiert. Der Vaterprozess wartet die Terminierung des Clients ab und beendet dann seinerseits den Server.
- Ein Client muss wissen, zu welchem Auftrag eine Antwort gehört. Der Server soll daher bei seiner Antwort die Übergabeparameter wieder mit zurückliefern.
- Die Dauer der Auftragsbearbeitung beim Server wird durch Aufruf von `sleep(1)` simuliert, der zwischen Auftragsempfang und Ergebnissrückgabe steht.

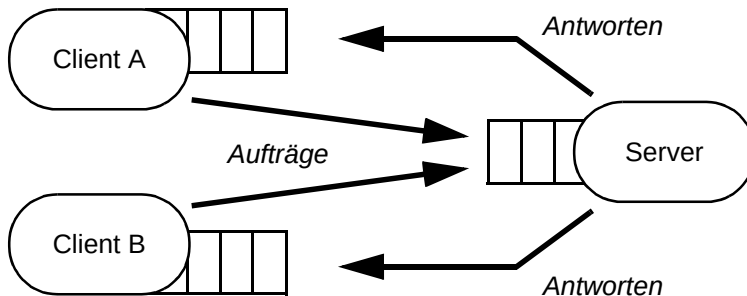
Führen Sie nun mit Ihrem Programm einen Leistungsvergleich zwischen aktivem und passivem Warten durch:

Der Client kann, je nach Wert des `IPC_NOWAIT`-Flags im Aufruf von `msgrcv()`, auf Antworten entweder passiv oder aktiv warten: Ist das Flag nicht gesetzt, so blockiert er, bis ein Eintrag in der Message Queue vorhanden ist („passives Warten“). Ist es gesetzt, so kehrt er sofort mit einer Fehlermeldung aus `msgrcv()` zurück (Rückgabewert `-1`) und kann dann unmittelbar einen neuen Empfangsversuch starten („aktives Warten“ in einer Schleife, die er erst verlässt, wenn der Empfangsversuch erfolgreich war).

Vergleichen Sie die jeweiligen Prozessorbelastungen durch den Client, d.h. die Prozessorzeit, die er jeweils verbraucht hat. Hierzu benutzen Sie am besten die Systemfunktion `times()` und zeigen die gemessenen Werte jeweils nach einem erfolgreichen Empfang auf dem Bildschirm an.

8. Message Queues mit zwei Clients und Aufträgen verschiedenen Prioritäten

Erweitern Sie Ihre Lösung aus Aufgabe 7 so, dass nun zwei Clients vorhanden sind und zudem die Aufträge des zweiten Client bevorzugt behandelt werden sollen. Die grundlegende Systemstruktur wird durch die folgende Abbildung angegeben:



Jeder Client soll wieder unmittelbar hintereinander fünf Aufträge übergeben und ein eintreffendes Ergebnis zusammen mit den Übergabeparametern sofort auf den Bildschirm ausgeben.

Beide Clients schreiben ihre Aufträge in *dieselbe* Message Queue des Servers. Stehen dort Aufträge von Client B zur Bearbeitung an, so sollen diese zuerst bearbeitet werden, selbst wenn sie hinter Aufträgen von Client A stehen.

Beachten Sie bei der Implementierung die folgenden Punkte:

- Gehen Sie nur von der Lösung mit passivem Warten aus.
- Starten Sie zuerst den Server und dann Client A. Starten Sie anschließend, nach einer Wartezeit von einer Sekunde, Client B.
- Die Clients müssen in ihren Aufträgen einen Absender angeben, damit der Server weiß, wohin er die Antwort zurückgeben soll. Hierfür verwendet ein Client am besten den Identifikator seiner Message Queue.
- Um zu erreichen, dass dringende Aufträge (also die von Client B) bevorzugt werden, kann man folgendermaßen vorgehen: Dringende Aufträge erhalten den Message-Typ 2, andere Aufträge den Typ 1. Der Server versucht jeweils zuerst, einen Auftrag des Typs 2 aus der Queue zu lesen. Gelingt ihm das, so bearbeitet er ihn. Ist kein Auftrag dieses Typs vorhanden, so kehrt er unmittelbar aus `msgrcv()` zurück (`IPC_NOWAIT` benutzen!) und versucht, in einem zweiten Aufruf von `msgrcv()` einen Auftrag beliebigen Typs zu lesen. Ist überhaupt kein Auftrag vorhanden, so blockiert der Server in diesem zweiten Aufruf, bis wieder ein Auftrag eintrifft.

9. Client/Server mit Sockets

Zum Datenaustausch können statt Message Queues auch Sockets verwendet werden. Der Vorteil dabei ist, dass Sockets nicht nur für die lokale Kommunikation, sondern auch für die Kommunikation über ein Netz benutzt werden können (was hier allerdings nicht geschehen soll).

Programmieren Sie das Client-Server-System aus Aufgabe 7 mit Sockets. Orientieren Sie sich dabei an den Beispielen aus Abschnitt 4.2.5 und dem Anhang und beachten Sie zudem die folgenden Hinweise:

- Die Kommunikation soll lokal über Stream-Sockets laufen (`AF_UNIX`, `SOCK_STREAM`).
- Da Sockets eine bidirektionale Kommunikation ermöglichen, können die Aufträge des Clients und die Antworten des Servers über dasselbe Socketpaar verschickt werden. Lassen Sie dazu Client und Server je eine Socket erzeugen (`socket()`) und den Server einen Namen an seine Socket binden (`bind()`). Der Server soll anschließend auf einen Verbindungsaufbauwunsch warten (`listen()`, `accept()`), der dann vom Client kommt (`connect()`).
- Stellen Sie sicher, dass der Client sein `connect()` erst nach dem `listen()` des Servers ausführt (z.B. durch einen `sleep()`-Aufruf), da sonst der Server seine Vorbereitungen zur Entgegennahme eines Aufbauwunschs noch nicht abgeschlossen hat und somit das `connect()` zu einem Fehler führt.
- Über eine Socket können Sie per `write()` nicht nur Strings verschicken, sondern beliebige Bytefolgen, wie z.B. die Binärdarstellungen zweier Integer. Hierzu muss die Adresse des Anfangsbytes im Speicher und die Anzahl der Bytes als Parameter übergeben werden. Analoges gilt für `read()`.
- Löschen Sie vor einem Neustart des Programms unbedingt die Sockets früherer Läufe, da es sonst zu größeren Fehlfunktionen kommen kann! Da Sockets durch das Dateisystem verwaltet werden, kann man ihre Namen ganz einfach per `ls` ermitteln und sie dann per `rm` löschen. Alternativ kann man im Programm die `unlink()`-Funktion aufrufen.

Lösungen der Aufgaben

zu Kapitel 1

1. Wissensfragen

- a.) Verwaltung der Rechnerbetriebsmittel, Bereitstellung von Schnittstellen für Benutzer und Programmierer.
- b.) Einzelbenutzerbetrieb / Einzelauftragsbearbeitung, Batchbetrieb, Mehrprogrammbetrieb (Multiprogramming).
- c.) Ein- und Ausgabe von Aufträgen nebenläufig zur Auftragsbearbeitung. Dabei Einsatz von Ein-/Ausgabeprozessoren, die nebenläufig zum Hauptprozessor arbeiten und mit DMA auf den Hauptspeicher zugreifen.
- d.) Als Folge von Schalen, die ausgehend von der Hardware aufeinander aufsetzen. Je weiter außen die Schicht angesiedelt ist, desto komplexere Dienste bietet sie an ihrer oberen Schnittstelle an. Jede Schicht nutzt die Dienste der nächstinneren Schicht.
- e.) Der Teil, der auf der Hardware aufsetzt und grundlegende Verwaltungsoperationen realisiert. Er bietet nach oben eine hardwareunabhängige Schnittstelle.
- f.) Zwei Modi, in denen ein Programm ausgeführt werden kann: eingeschränkte Rechte im User Mode, volle Rechte im Kernel Mode.

2. Klassifikation von Betriebsmitteln

Klassen →	Hardware oder Software	ein- oder mehrmals benutzbar	exklusiv od. n. exkl. benutzbar	entziehbar oder nicht entziehbar
CPU	Hardware	mehrmals	exklusiv	entziehbar
CD-Brenner	Hardware	mehrmals	exklusiv	nicht entz.
CD-Rohling (Write-Only)	Hardware	einmal	exklusiv	nicht entz.
Datei <code>prog.c</code> mit C-Programmcodem während der Programm-erstellung	Software	einmal (mehrmals?)	exklusiv	nicht entz. (entziehbar?)
Datei <code>prog.exe</code> mit ausführbarem Programm	Software	mehrmals	nicht exklusiv	irrelevant

Anmerkungen:

- Die Entziehbarkeit der CPU wird in Kapitel 3 eingehend diskutiert.
- Da der Brennvorgang einer CD ohne Unterbrechung ablaufen muss, ist ein CD-Brenner nicht entziehbar.
- Ein CD-Rohling ist zwar in dem Sinn eingeschränkt mehrmals benutzbar, dass man auf ihn mehrmals hintereinander Daten („Sessions“) schreiben kann. Er ändert sich dabei aber irreversibel, so dass er eher als einmal benutzbar einzustufen ist. Wie der CD-Brenner ist er während des Brennvorgangs nicht entziehbar.
- Ob eine Datei `prog.c` mit C-Quellcode ein- oder mehrmals benutzbar bzw. entziehbar oder nicht ist, kann diskutiert werden. Sie ist in dem Sinne nur einmal benutzbar, dass sie sich beim Editieren ändert und somit anschließend nicht dieselbe ist. Sie ist in dem Sinne nicht entziehbar, dass sie dem Programmierer nicht mitten in seiner Arbeit weggenommen und einem anderen Programmierer zugeteilt werden sollte.
- Eine Datei `prog.exe` mit einem ausführbaren Maschinenprogramm kann zur Ausführung von mehreren Prozessen gleichzeitig gelesen werden. Sie ist also nicht-exklusiv benutzbar, und die Frage der Entziehbarkeit stellt sich damit nicht.

3. Sequentielle Bearbeitung und Multiprogramming im Vergleich

- a.) Sequentielle Bearbeitung: 0-15 idle, 15-17 A, 17-27 idle, 27-30 A, 30-40 idle, 40-46 B, 46-51 idle, 51-53 B, 53-58 idle, 58-63 C, 63-73 idle, 73-78 C.
 Multiprogramming: 0-5 idle, 5-10 C, 10-15 B, 15-17 A, 17-18 B, 18-20 idle, 20-23 C, 23-25 B, 25-27 C, 27-30 A.
- b.) Sequentielle Bearbeitung: Aufenthaltsdauern A = 30, B = 53, C = 78, Mittel = 53,67
 CPU-Auslastung 23/78, d.h. ca. 29,5%
 Multiprogramming: Aufenthaltsdauern A = 30, B = 25, C = 27, Mittel = 27,33
 CPU-Auslastung 23/30, d.h. ca. 76,7%

zu Kapitel 2

1. Wissensfragen

- a.) C wurde im Zusammenhang mit UNIX entwickelt. UNIX ist hauptsächlich in C geschrieben, um das Betriebssystem portabel zu halten. UNIX bietet eine Schnittstelle mit C-Funktionsprototypen, die in C-Programmen benutzt werden können.
- b.) System V Interface Definition. Sie legt insbesondere die Schnittstellen von Bibliotheksfunktionen fest.
- c.) Die C-Schnittstelle bietet nach außen C-Prototypen von Systemfunktionen an, die aus C-Programmen aufgerufen werden können. Die Benutzerschnittstelle bietet Sammlung von Kommandos, mit denen der Benutzer von der Tastatur aus oder mit

Hilfe von Batchdateien (Shellscripts) mit dem System kommunizieren kann. Zwischen den beiden Schnittstellen liegt die Shell.

- d.) Nach der Bereitstellung der Parameter wird ein Trap-Ereignis ausgelöst, bei dem die Programmausführung an eine festgelegte Stelle des Kerns verzweigt und zudem ein Wechsel vom User Mode in den Kernel Mode stattfindet.
- e.) Im Dateibaum, Unterverzeichnis /dev.
- f.) * = Platzhalter im Dateinamen für beliebige Zeichenfolge; | Pipeoperator (= Ausgabe des ersten Kommandos wird als Eingabe des Zweiten verwendet); ; Hintereinanderausführung von Kommandos; < Eingabeumlenkung; > Ausgabeumlenkung.

2. UNIX-Aufbau

Siehe Abbildung 2.2.

3. UNIX-Dateisystem

- a.) Die Rechte Lesen (r), Schreiben (w) und Ausführen (x), getrennt für den Besitzer der Datei, die der Datei zugeordnete Benutzergruppe und sonstige Benutzer.
- b.) Gibt die Attribute der Datei `dipl_arb` an: Sie gehört dem Benutzer `stud1` und ist der Benutzergruppe `studis` zugeordnet. Der Besitzer darf sie lesen und in sie schreiben, alle anderen nur lesen. Es gibt vier Links auf die Datei, d.h. man kann auf sie unter vier verschiedenen Namen oder Pfaden zugreifen, möglicherweise aus verschiedenen Verzeichnissen. Ihre Größe beträgt 511722 Byte. Sie wurde letztmalig am 17. Dezember um 10:24 Uhr geändert.
- c.) Weil es ohne Nachfrage alle Dateien im Arbeitsverzeichnis entfernt (außer den versteckten Dateien). Die Dateien lassen sich nicht wiederherstellen! Anmerkung: Besonders gefährlich sind Tippfehler, die in diesem Zusammenhang auftreten. Will man beispielsweise alle Dateien entfernen, die mit dem Buchstaben x beginnen und tippt `rm x *` (anstelle `rm x*`, also mit einem versehentlichen Leerzeichen zwischen x und *), so werden alle Dateien im Verzeichnis gelöscht.

4. UNIX-C-Compiler

- a.) Häufig gebrauchte Funktionen können als Objektcodedateien in Funktionssammlungen („Bibliotheken“) abgelegt werden. Anwendungsprogrammierer können sie dann zu ihren Programmen „hinzubinden“, d.h. aus ihren Programmen plus den Funktionen ein lauffähiges Maschinenprogramm generieren.
- b.) `cc -c p1.c p2.c` (oder auch `cc -c *.c`, wenn dies die einzigen Dateien mit Suffix `.c` im Verzeichnis sind).
- c.) `cc -o p.exe pa.c pb.c pc.o`.

5. UNIX-Shellscripts

- a.) Listet zunächst alle Dateien, deren Namen mit einer bestimmten Zeichenkette beginnen, seitenweise auf dem Bildschirm auf. Schreibt dieses Listing zusätzlich in eine Datei. Zeichenkette und Dateiname können als Parameter an das Shellsript übergeben werden (\$1 bzw. \$2).
- b.)

```
mkdir ~/$2  
cp *. $1 ~/$2
```

zu Kapitel 3

1. Wissensfragen

- a.) Sein Programmcode (= Maschinenprogramm) und der Kontext, in dem das Programm ausgeführt wird.
- b.) Jeder Prozess wird in einem eigenen Kontext ausgeführt. Mehrere Threads können einen gemeinsamen Kontext haben.
- c.) Jeder Prozess besitzt einen Prozesskontrollblock (PCB), der Informationen über ihn enthält. Die PCBs aller Prozesse sind in einer Prozesstabelle zusammengefasst. Ein Prozess wird über eine eindeutige Prozessnummer (PID) identifiziert.
- d.) Erzeugt aus einem Vaterprozess einen Sohnprozess. Gibt dem Vaterprozess die PID des Sohnprozesses zurück, dem Sohnprozess eine 0.
- e.) Unterbrechendes und nicht-unterbrechendes Scheduling.
- f.) Bei festen Prioritäten bleibt die Priorität eines Prozesses während seiner gesamten Lebensdauer unverändert, bei dynamischen Prioritäten kann sie sich ändern.
- g.) Beides sind Ereignisse, die zur Unterbrechung der aktuellen Programmausführung und zum Start eines entsprechenden Handlers führen. Eine Trap wird durch das Programm selbst ausgelöst, ein Interrupt ist ein externes Ereignis.
- h.) Bei einer harten Anforderung ist das Ergebnis völlig wertlos, wenn es zu spät eintrifft. Bei einer weichen ist der Wertverlust graduell: Das Ergebnis wird immer weniger wert, je später es kommt.

2. Funktionen der UNIX-C-Schnittstelle zur Prozessverwaltung

- a.) Die Aussagen „Ich bin der ...“ sind nicht korrekt, denn `fork()` gibt dem Vater eine PID ungleich 0 zurück und dem Sohn eine 0. Die `printf()`-Aufrufe sind also vertauscht. Die Ausgabe von `p` ist nur beim Vater korrekt; beim Sohn hat `p` den Wert 0.
- b.) Die Ausgabe ist eindeutig; sie lautet `i=10`. Da Vater und Sohn getrennte Variablen haben, haben die Operationen des Sohns auf `i` keinen Einfluss auf den Wert von `i` im Vaterprozess.

- c.) Der Vaterprozess erzeugt drei Sohnprozesse, die eine, zwei bzw. drei Sekunden schlafen, dann eine Meldung ausgeben und terminieren. Die Aussage der `printf()`-Meldungen ist nicht korrekt, da `wait()` nur auf die Terminierung *eines* Sohns wartet. Nötig wären hier drei `wait()`-Aufrufe hintereinander.
- d.) „Hallo, ...“ wird dreimal ausgegeben und „Gute Nacht“ einmal.
 Nach der Streichung von `exit()` werden die Söhne nach ihrer Ausgabe nicht mehr durch `exit()` beendet, sondern treten ihrerseits mit dem jeweils aktuellen `i`-Wert in die `for`-Schleife ein, wodurch neue Prozesse erzeugt werden. Es kommt also zu einer begrenzten Kettenreaktion.
 Wenn das `exit()` fehlte, so würde das Programm, insbesondere für große `n`, zu einer Lawine von neuen Prozessen führen, die im Extremfall das gesamte System lahm legen könnten. Das Betriebssystem kann das verhindern, indem es eine Obergrenze für die Zahl der Prozesse festlegt, die für einen Benutzer gleichzeitig aktiv sein dürfen.
- e.) Die Lösung ist nicht korrekt, da das `exit()` nach dem `printf()` fehlt, mit dem die Sohnprozesse jeweils gleich wieder terminiert werden.

3. Strategien zum Prozessorscheduling

- a.) FCFS: 0-1 U, 1-151 A, 151-152 U, 152-352 B, 352-353 U, 353-653 C, 653-654 U, 654-754 D
 FP_nu: 0-1 U, 1-151 A, 151-152 U, 152-452 C, 452-453 U, 453-553 D, 553-554 U, 554-754 B
 FP_u: 0-1 U, 1-50 A, 50-51 U, 51-80 B, 80-81 U, 81-160 C, 160-161 U, 161-261 D, 261-262 U, 262-483 C, 483-484 U, 484-655 B, 655-656 U, 656-757 A
 RR_100: 0-1 U, 1-101 A, 101-102 U, 102-202 B, 202-203 U, 203-303 C, 303-304 U, 304-404 D, 404-405 U, 405-455 A, 455-456 U, 456-556 B, 556-557 U, 557-757 C

b.)

Strategie	Auf.dauer Prozess A	Auf.dauer Prozess B	Auf.dauer Prozess C	Auf.dauer Prozess D	Umschalt. absolut	Umschalt. prozentual
FCFS	151 ms	302 ms	573 ms	594 ms	4 ms	0,531%
FP_nu	151 ms	704 ms	372 ms	393 ms	4 ms	0,531%
FP_u	757 ms	605 ms	403 ms	101 ms	7 ms	0,925%
RR_100	455 ms	506 ms	677 ms	244 ms	7 ms	0,925%

4. Prozessorscheduling unter Round Robin

- a.) $\lceil t_p/z \rceil$, wobei $\lceil .. \rceil$ die Aufrundung zur nächsten (höheren) ganzen Zahl bedeutet.
- b.) Eine entsprechende einfache Formel lässt sich nicht angeben, da die Anzahl der Umschaltungen davon abhängt, ob der Prozess allein im System ist oder ob und wann andere Prozesse die CPU benötigen.
- c.) $n \cdot (z+u) \cdot (t_{\min}/z - 1) + u + z$ (denn $P_{\min}$ verbleibt $(t_{\min}/z - 1)$ volle Runden im System und braucht dann noch eine abschließende Zeitscheibe, um fertig bedient zu werden).
- d.) CPU-Zuteilung für z_1 : 0-5 U, 5-55 A, 55-60 U, 60-110 B, 110-115 U, 115-140 C, 140-145 U, 145-195 A, 195-200 U, 200-210 B
 CPU-Zuteilung für z_2 : 0-5 U, 5-30 A, 30-35 U, 35-60 B, 60-65 U, 65-90 C, 90-95 U, 95-120 A, 120-125 U, 125-150 B, 150-155 U, 155-180 A, 180-185 U, 185-195 B, 195-200 U, 200-225 A
 Abgangszeitpunkte für z_1 : A 195, B 210, C 140 → mittlere Verweilzeit: 181,67
 Abgangszeitpunkte für z_2 : A 225, B 195, C 90 → mittlere Verweilzeit: 170
 Abgangszeitpunkte für z_3 : A 335, B 290, C 150 → mittlere Verweilzeit: 258,33
- e.) Das Sinken erklärt sich dadurch, dass bei Verkürzung der Zeitscheibe auf 25 B und C wesentlich eher fertig werden, also allgemein gesagt die Prozesse in der Reihenfolge ihrer Bearbeitungsdauern terminieren. Für die Zeitscheibe 5 wird dieser Effekt aber dadurch wieder aufgehoben, dass die Umschaltzeiten überhand nehmen.
- f.) Die Länge eines Zyklus ergibt sich – grob gesprochen – als Produkt der Anzahl der Prozesse (zumindest der Hintergrundprozesse, die immer ausführungsbereit sind) und der Länge der Zeitscheibe. Die Länge der Wartezeiten der Benutzer auf Bedienung steigt mit der Länge der Zeitscheibe und wird bei längeren Zeitscheiben inakzeptabel.

5. Prozessorscheduling unter UNIX

- a.) Der *nice*-Wert geht als Summand in die Berechnung der Prozesspriorität ein. Er soll dazu benutzt werden, die Priorität eines Prozesses freiwillig zu verschlechtern. Dürften Benutzer auch negative *nice*-Werte verwenden, so könnten sie damit die Priorität ihrer Prozesse verbessern. Das sollte aber den Betriebssystem selbst oder dem Systemverwalter vorbehalten bleiben.
- b.) Prozess A ist wahrscheinlich das Berechnungsprogramm und B die Shell. Das Berechnungsprogramm belastet nämlich die CPU stark und wird daher vom UNIX-Scheduling rasch in seiner Priorität zurückgestuft. Die Shell als interaktives Programm hat dagegen lange Wartezeiten und damit eine niedrige CPU-Belastung, so dass die Priorität hoch bleibt.

- c.) Man kann dies nicht unbedingt erwarten, denn das UNIX-Scheduling wird die Priorität des Sohnprozesses rasch zurückstufen, so dass der Prozess innerhalb der gesetzten Frist nicht genug CPU-Kapazität bekommen wird. Man benötigte eine Möglichkeit, dem Sohnprozess eine feste hohe Priorität zu geben.
- d.) Gewichteter CPU-Zeit-Verbrauch $CPU_verbrauch(P_i)$:

	für P_1	für P_2	für P_3
nach 1. Sekunde	320	120	200
nach 2. Sekunde	240	240	480
nach 3. Sekunde	160	320	560
nach 4. Sekunde	80	640	800

Prioritätswerte $prio(P_i)$:

	für P_1	für P_2	für P_3
nach 1. Sekunde	110	95	85
nach 2. Sekunde	100	110	120
nach 3. Sekunde	90	120	130
nach 4. Sekunde	80	160	160

(Anmerkung: Die Anfangsprioritäten haben überhaupt keinen Einfluss auf die folgenden Prioritätswerte; sie wurden hier nur zur Verwirrung angegeben :-)

zu Kapitel 4

1. Wissensfragen

- Wechselseitiger Ausschluss und Reihenfolgebedingung.
- Ein Spinlock basiert auf aktivem Warten, ein Semaphore auf passivem. Ein Semaphore ist vielfältiger einsetzbar als ein Spinlock.
- Initialisierung; P-Operation (= Blockieren, wenn Semaphorewert 0 ist; Senken des Semaphorewerts); V-Operation (= Erhöhen des Semaphorewerts um 1; Entblockieren eines wartenden Prozesses, falls vorhanden).
- Man kann auf Semaphoren derselben Gruppe mehrere Operationen atomar ausführen.

- e.) Ein Zustand, in dem eine Menge von Prozessen existiert, von denen jeder auf eine Aktion eines anderen wartet, so dass keiner von ihnen vorwärts kommt.
- f.) Speicherbasierte und nachrichtenbasierte Kommunikation.
- g.) Aus einer Mailbox dürfen prinzipiell mehrere Prozesse lesen, ein Port ist genau einem Empfängerprozess zugeordnet.

2. Prozesssynchronisation I

- a.) Initialisierung: `SEM_A.INIT(3); SEM_B.INIT(2); SEM_C.INIT(2);`
 Student: `SEM_A.P(); SEM_B.P(); SEM_C.P();`
 `< Diplomarbeit schreiben >`
 `SEM_A.V(); SEM_B.V(); SEM_C.V();`
- b.) Initialisierung: `SEM_1.INIT(0); SEM_2.INIT(0)`
 Zulieferer 1: `< Teil 1 herstellen & liefern >; SEM_1.V();`
 Zulieferer 2: `< Teil 2 herstellen & liefern >; SEM_2.V();`
 Produktionsbetrieb: `SEM_1.P(); SEM_2.P(); <Weiterverarbeiten>`
 (Zulieferer und Produktionsbetrieb jeweils in Endlosschleifen eingebettet)
- c.) Lösung 1:
 Initialisierung: `S_WA.INIT(1);`
 `/* w.A. der Schreiber gegeneinander */`
 `SL_WA.INIT(3);`
 `/* w.A. eines Schreibers gg. alle Leser */`
 Leser: `while (TRUE) {`
 `SL_WA.P();`
 `< Lesen >`
 `SL_WA.V();`
 `.... }`
 Schreiber: `while (TRUE) {`
 `S_WA.P();`
 `SL_WA.P(); SL_WA.P(); SL_WA.P();`
 `< Schreiben >`
 `SL_WA.V(); SL_WA.V(); SL_WA.V();`
 `S_WA.V();`
 `.... }`
- Lösung 2:
 Initialisierung: `WA_1.INIT(1); WA_2.INIT(1); WA_3.INIT(1);`
 `/* w.A. der Schreiber gg. Leser 1, 2, 3 */`
 Leser i: `while (TRUE) {`
 `WA_i.P();`
 `< Lesen >`
 `WA_i.V();`
 `.... }`

```

Schreiber:  while (TRUE) {
              WA_1.P(); WA_2.P(); WA_3.P();
              < Schreiben >
              WA_1.V(); WA_2.V(); WA_3.V();
              .... }

```

Lösung 3: (Voraussetzung: Semaphorwert kann atomar um n mit $n > 1$ dekrementiert oder inkrementiert werden. Notation: $S.P(n)$ bzw. $S.V(n)$)

Initialisierung: $WA.INIT(3);$ /* allgemeiner w.A. */

```

Leser:      while (TRUE) {
              WA.P(1);
              < Lesen >
              WA.V(1);
              .... }

```

```

Schreiber:  while (TRUE) {
              WA.P(3);
              < Schreiben >
              WA.V(3);
              .... }

```

Problem bei dieser Lösung: Die Schreiber werden unfair behandelt, da ein Leser leichter durchkommt. Im Extremfall lassen die Leser die Schreiber verhungern.

3. Prozesssynchronisation II

- a.) Nicht korrekt, da für beide Reihenfolgebeziehungen (Schwimmer 1 vor Schwimmer 2, Schwimmer 2 vor Schwimmer 3) derselbe Semaphor verwendet wird. Dabei ist nicht ausgeschlossen, dass Schwimmer 3 das Signal von Schwimmer 1 bekommt und somit vor Schwimmer 2 schwimmt. Korrektur: Einen zweiten Semaphor einführen, der ebenfalls mit 0 initialisiert wird. Schwimmer 2 macht seine V-Operation auf diesem Semaphor, ebenfalls Schwimmer 3 seine P-Operation.

- b.) Initialisierung:

$SEM_DUSCHE.INIT(1)$ bei einer Dusche
bzw. $SEM_DUSCHE.INIT(2)$ bei zwei Duschen.

Jeder Schwimmer nach dem Schwimmen:

$SEM_DUSCHE.P(1);$ < Duschen >; $SEM_DUSCHE.V(1).$

- c.) Initialisierung:

$SEM_DUSCHE.INIT(0);$ (statt 1 wie in b.)

$SEM_EURO(0);$

Wärterprozess:

$SEM_EURO.P(1); SEM_EURO.P(1); SEM_EURO.P(1);$

$SEM_DUSCHE.V(1);$ bzw. $SEM_DUSCHE.V(2);$

(schließt eine bzw. beide Duschen auf)

Jeder Schwimmer nach dem Schwimmen:

`SEM_EURO.V(1) ; SEM_DUSCHE.P(1) ; ... Rest wie in b.) ...`

4. Prozesssynchronisation in UNIX I

- a.) Der `semget()`-Aufruf erzeugt eine neue Gruppe von 5 Semaphoren. Der `semctl()`-Aufruf initialisiert den zweiten Semaphor in dieser Gruppe mit dem Wert 0 und die übrigen Semaphore mit dem Wert 1.
- b.) Der Prozess wird blockiert, da die Zählvariable von Semaphor Nr. 1 (also dem zweiten in dieser Gruppe) den Wert 0 hat und daher nicht weiter gesenkt werden kann. Die Werte sämtlicher Semaphore bleiben vorerst unverändert („Alles-oder-nichts-Prinzip“).
- c.) Die Werte des vorletzten und letzten Semaphors würden auf 0 gesenkt; der Prozess würde nicht blockiert.

5. Prozesssynchronisation in UNIX II

```
#include <sys/ipc.h>
#include <sys/sem.h>
main() {
    int sem_1, sem_2; /* IDs der Semaphore */
    unsigned short initarray[1];
    /* Zur Initialisierung der Semaphore */
    struct sembuf semp, semv;
    /* Strukturen fuer P- und V-Operationen */
    int status;
    /* Erzeugung und Initialisierung der Semaphore: */
    sem_1 = semget(IPC_PRIVATE,1,IPC_CREAT|0777);
    sem_2 = semget(IPC_PRIVATE,1,IPC_CREAT|0777);
    initarray[0] = 0;
    semctl(sem_1,1,SETALL,initarray);
    semctl(sem_2,1,SETALL,initarray);
    /* Vorbereitung der Semaphoroperationen: */
    semp.sem_num = semv.sem_num = 0;
    semp.sem_op = -1; semv.sem_op = 1;
    semp.sem_flg = semv.sem_flg = 0;
    /* Start des ersten Zulieferers: */
    if (fork()==0) {
        printf("Zulieferer 1 produziert.\n");
        sleep(5); /* Produktionszeit */
        printf("Zulieferer 1 fertig\n");
        semop(sem_1,&semp,1);
        exit(0); }
}
```

```

/* Start des zweiten Zulieferers: */
if (fork()==0) {
    printf("Zulieferer 2 produziert.\n");
    sleep(3); /* Produktionszeit */
    printf("Zulieferer 2 fertig\n");
    semop(sem_2,&semv,1);
    exit(0); }
/* Start des Produktionsbetriebs: */
if (fork()==0) {
    printf("Prod.betrieb wartet auf Zulieferer.\n");
    semop(sem_1,&semp,1);
    semop(sem_2,&semp,1);
    printf("Teile sind eingetroffen.\n");
    sleep(4); /* Produktionszeit */
    printf("Prod.betrieb fertig\n");
    exit(0); }
/* Warten auf Ende der Sohnprozesse
   und anschliessend Loeschen der Semaphore: */
wait(&status); wait(&status); wait(&status);
semctl(sem_1,0,IPC_RMID,0);
semctl(sem_2,0,IPC_RMID,0); }

```

6. Deadlocks

Ein Deadlock kann z.B. auftreten, wenn drei Studenten je ein Exemplar von Buch A besitzen, zwei andere Studenten je eines von Buch B und Buch C und dann auf die Freigabe des jeweils noch fehlenden Buchs warten. Zur Lösung gibt es u.a. die folgenden Möglichkeiten:

- **Deadlockverhinderung:**
 Arbeitsgemeinschaften mit gemeinsamer Buchbenutzung bilden (d.h. Betriebsmittel nicht-exklusiv benutzbar machen).
 Bücher temporär an andere Studenten weiterverleihen (d.h. Betriebsmittel entziehbar machen).
 Fordern, dass ein Student alle Bücher auf einmal ausleihen muss (d.h. ein Prozess, der bereits Betriebsmittel hat, darf keine weiteren fordern).
 Ausleihreihenfolge festlegen: erst Buch 1, dann Buch 2, dann Buch 3 (d.h. Zyklus im Belegungs-Anforderungs-Graphen verhindern).
- **Deadlockvermeidung:**
 Bei jeder Ausleihe überprüfen, ob dadurch eine kritische Situation eintritt (d.h. feststellen, was bereits belegt ist und was noch gefordert werden kann).
- **Deadlockbehebung:**
 Studenten zwingen, Bücher wieder zurückzugeben – entweder alle Studenten sofort

oder einen nach dem anderen, bis die Deadlocksituation behoben ist (d.h. Prozessen ihre Betriebsmittel entziehen).

Studenten erschießen :-) – entweder alle Studenten sofort (als „brutalstmögliche“ Lösung) oder einen nach dem anderen, bis die Deadlocksituation behoben ist (d.h. Prozesse abbrechen oder System neu starten mit automatischer Betriebsmittelfreigabe).

7. Prozesskommunikation in UNIX I

- a.) Shared Memory.
- b.) Pipes, Message Queues, Sockets. Bei ihnen übernimmt das Betriebssystem die Synchronisation der Prozesse; bei Shared Memory müssen sich die Prozesse selbst darum kümmern.
- c.) Die Synchronisation ist nicht korrekt, da nicht sichergestellt ist, dass die Schreiboperation vor der Leseoperation stattfindet. Man muss sie z.B. durch Einsatz eines Semaphors oder durch einen `wait()`-Aufruf des Vaters (vor dem `printf()`) durchsetzen.

8. Prozesskommunikation in UNIX II

```
#include <sys/ipc.h>
main() {
    int msgqid; /* Identifikator der Message Queue */
    struct {
        long mtype;
        int wert1;
        float wert2;
    } message; /* Nachricht zum Senden bzw. Empfangen */
    int i, status;
    /* Erzeugung der Message Queue: */
    msgqid = msgget(IPC_PRIVATE,IPC_CREAT|0777);
    /* Start des Erzeugers: */
    if (fork()==0) {
        for (i=0;i<10;i++) {
            /* Erzeugen der Nachricht: */
            message.mtype = 1;
            message.wert1 = i; /* Einfuellen eines int- ... */
            message.wert2 = i/2.0; /* ... und eines float-Werts. */
            sleep(2); /* Wartezeit fuer Erzeugen der Nachricht. */
            /* Abschicken der Nachricht: */
            msgsnd(msgqid,&message,sizeof(message)-sizeof(long),0);
        }
    }
    exit(0); }
```

```

/* Start des Verbrauchers: */
if (fork()==0) {
    for (i=0;i<10;i++) {
        /* Empfangen der Nachricht: */
        msgrcv(msgqid,&message,
                sizeof(message)-sizeof(long),0,0);
        printf("Nachr. %d: %d, %f\n",
                i,message.wert1,message.wert2);
    }
    exit(0); }
/* Warten auf Ende der Prozesse, Loeschen der Queue: */
wait(&status); wait(&status);
msgctl(msgqid,IPC_RMID,0); }

```

zu Kapitel 5

1. Wissensfragen

- a.) Der Prozessor kann auf den Hauptspeicher direkt zugreifen, auf den Plattenspeicher nicht. Der Hauptspeicher ist rascher zugreifbar als der Plattenspeicher, dafür kleiner. Hauptspeichereinhalte sind, im Gegensatz zu Platteninhalten, flüchtig, d.h. werden beim Abschalten des Computers gelöscht.
- b.) Da Programme ein stark lokales Verhalten haben, können im Cache, trotz seiner geringen Größe, die meisten aktuell gebrauchten Daten untergebracht und somit rasch zugegriffen werden.
- c.) Beim Swapping werden immer ganze Prozesse auf die Platte ausgelagert, beim Paging können auch nur Teile von Prozessen ausgelagert werden.
- d.) Die MMU (Memory Management Unit) bildet virtuelle auf reale Adressen ab.
- e.) Die Einheiten beim segmentorientierten Speicher sind Segmente unterschiedlicher Längen, beim seitenorientierten Speicher sind es Seiten identischer Längen.
- f.) Sie legt fest, welche Seite auf den Hintergrundspeicher verdrängt wird, um im Hauptspeicher Platz zu schaffen.
- g.) Durch einen hybriden virtuellen Speicher: Die Haupteinheiten sind hier die Segmente, die aber jeweils in Seiten unterteilt sind.

2. Plattenscheduling

First Come First Served: 50 → 55 → 90 → 40 → 85 → 30 → 20 → 100, 280 Spuren
 Shortest Seek Time First: 50 → 55 → 40 → 30 → 20 → 85 → 90 → 100, 120 Spuren
 Fahrstuhl: 50 → 40 → 30 → 20 → 55 → 85 → 90 → 100, 110 Spuren

3. Hauptspeicherverwaltung

(Zahlenangaben in MByte, F = Freibereich)

First Fit, Zeit 5: F 8, II 1, F 3, IV 1, F 3 (mittl. Lg. = 4,67)

First Fit, Zeit 8: V 2, VI 2, F 4, II 1, F 3, IV 1, F 3 (mittl. Lg. = 3,33)

First Fit, Zeit 12: V 2, VI 2, VII 4, II 1, VIII 3, IV 1, F 3 (mittl. Lg. = 3)

Best Fit, Zeit 5: wie First Fit F 8, II 1, F 3, IV 1, F 3 (mittl. Lg. = 4,67)

Best Fit, Zeit 8: F 8, II 1, V 2, F 1, IV 1, VI 2, F 1 (mittl. Lg. = 3,33)

Best Fit, Zeit 12: VII 4, VIII 3, F 1, II 1, V 2, F 1, IV 1, VI 2, F 1

(mittl. Lg. = 1, also schlechter als FF, da stärkere Fragmentierung)

4. Segmentorientierter virtueller Speicher

a.)

Segmentnr.	Anfangsadresse	Länge	Zugriffsrechte
S ₁	0	150	Lesen / Schreiben
S ₂	Plattenblock X	40	Nur Schreiben
S ₃	150	180	Nur Lesen
S ₄	430	200	Lesen / Schreiben
S ₅	Plattenblock Y	110	Lesen / Schreiben
S ₆	830	120	Lesen / Schreiben

- b.) Lies Segment 4, Byte 0: 430, zulässig;
 Schreibe Segment 6, Byte 90: 920, zulässig;
 Schreibe Segment 3, Byte 78: (228), nicht zulässig, da nur Lesen erlaubt;
 Schreibe Segment 2, Byte 35: 985 (nach Laden des Segments), zulässig;
 Lies Segment 1, Byte 155: (155), nicht zulässig, da Länge überschritten;
 Lies Segment 5, Byte 80: 710 (nach Laden des Segments), zulässig.

- c.) Angaben des Prozesses: Länge des Segments, zulässige Zugriffsarten.

Aktionen des Betriebssystems: Belegung einer freien Segmentnummer, Belegung eines freien Speicherbereichs (entweder im Hauptspeicher, eventuell unter Verdrängung eines anderen Segments, oder auf der Platte), Initialisierung des Segmentableneintrags (Rechte, reale Anfangsadresse, Länge, Valid-Bit), Mitteilung der Segmentnummer und des Erfolgs an den Prozess.

5. Seitenorientierter virtueller Speicher

- a.) 14288, P12 Offset 520, 1712, 33576, P3 Offset 904, 9808.

- b.) Seite Nr. 2 hat jetzt den Adresseintrag „P23“, Seite Nr. 5 hat jetzt den Adresseintrag „32768“. Die virtuelle Adresse 21000 wird jetzt auf die reale Adresse 33288 abgebildet, die virtuelle Adresse 9000 auf P23 Offset 808.
- c.) Nein, denn in einem Swapping-System steht ein Prozess entweder immer vollständig im Hauptspeicher oder vollständig auf der Platte.

6. Strategien zur Speicherverwaltung

- a.) Seitenfehler unter FIFO: 7
 Seitenfehler unter LRU: 5
 Speicherinhalte unter FIFO: 1, 21, 321, 321, 321, 432, 432, 143, 514, 251
 Speicherinhalte unter LRU: 1, 21, 321, 231, 123, 412, 241, 124, 512, 251
- b.) {1,2,3}, {1,2,3}, {1,2,3}, {1,2,3}, {2,3}, {2,3}, {2,3}, {3}; kein Seitenfehler; mittl. Größe unter WS = 2,375; mittl. Größe unter LRU = 3 (also mehr als unter WS).

zu Kapitel 6

1. Wissensfragen

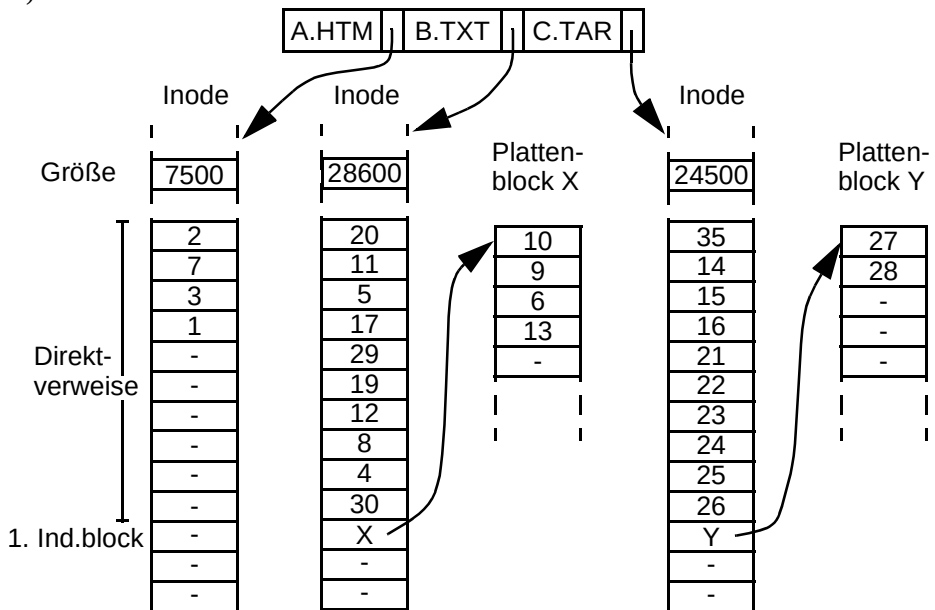
- a.) Eine Datei, deren Inhalt als Folge von Bytes aufgefasst wird, die nicht weiter strukturiert oder typisiert ist.
- b.) Ein Verweis auf eine Datei (ausgehend von einem Dateinamen). Zu einer Datei können mehrere Links existieren, so dass sie in mehreren Verzeichnissen unter verschiedenen Namen stehen kann.
- c.) Beim FAT-Dateisystem stehen die Dateiinformationen unmittelbar im Verzeichnis, beim Inode-basierten steht im Verzeichnis nur ein Verweis auf eine weitere Datenstruktur (den Inode), der die eigentlichen Informationen enthält.
- d.) Es ermöglicht die Integration verschiedener Dateisysteme unter einer einheitlichen Schnittstelle.
- e.) Ein System von mehreren Platten, auf die jeweils der Inhalt einer Datei verteilt wird. Der Vorteil liegt im gesteigerten Durchsatz und in der höheren Fehlertoleranz.
- f.) Der Controller, eine Hardwarekomponente, dient zur Steuerung der Hardware (insbesondere der Mechanik) eines Peripheriegeräts: Er setzt die Befehle der Zentraleinheit (insbesondere des Betriebssystems) um in Kommandos an die Gerätehardware. Der Treiber (eine Softwareeinheit) ist der Teil des Betriebssystems, der ein Gerät (insbesondere dessen Controller) ansteuert.

2. FAT-basiertes Dateisystem

- a.) A.TXT: 5, 13, 0, 3, 21; B.EXE: 12, 18, 14; C.DOC: 16, 8, 10, 6, 1;
D.COM: 4, 17, 9; E.HTM: 11, 19.
- b.) Block 12, Offs. 1998; Block 18, Offs. 89; Block 19, Offs. 89; Block 3 Offs. 1368.
- c.) Änderungen in der FAT Pos. 0: eof, Pos. 3: eof, Pos. 14: 3, Pos. 21: free.
Der Verzeichnisausschnitt ändert sich nicht (in einem vollständigen Verzeichnis ändern sich natürlich die Längenangaben).

3. Inode-basiertes Dateisystem

a.)



Anmerkung: Die Größenwerte (in Bytes) sind nicht ganz eindeutig, da sich aus der Aufgabe nur die Anzahl der Blöcke, nicht jedoch die genaue Anzahl der Bytes ergibt.

- b.) Block 2, Offs. 600; Block 7, Offs. 0; Block 5, Offs. 1904;
Block 30 Offs. 1568; Block 35 Offs. 2047; Block 16 Offs. 956.
- c.) Die Berechnungszeit für ein Byte in den ersten m Dateiblöcken ist kürzer, da (im Unterschied zu den übrigen Dateiblöcken) kein Zugriff auf einen oder mehrere Indirektblöcke nötig ist.

4. Vergleich FAT- und Inode-basiertes Dateisystem

- a.) Unter dem UNIX-Dateisystem, denn hier muss nicht, wie bei der FAT, eine lange Liste von Blocknummern durchlaufen werden.
- b.) Der Verlust eines Inodes betrifft nur eine einzelne Datei. Beim Verlust eines Teilstücks der FAT sind möglicherweise eine größere Anzahl von Dateien betroffen, wenn ihre Verkettung durch den verloren gegangenen Teil der FAT führt.
- c.) Beim Inode-basierten Dateisystem verweisen alle Verzeichniseinträge für eine Datei auf eine zentrale Stelle (den Inode) mit den Informationen über die Datei. Beim FAT-basierten System stehen die Informationen zusammen mit den Namen in den einzelnen Verzeichnissen. Eine konsistente Änderung aller dieser Informationen (Dateilänge usw.) ist nur schwer möglich.

zu Kapitel 7

1. Wissensfragen

- a.) Bei der externen Sicherheit geht es darum, das System vor Angriffen von außen zu schützen. Die interne Sicherheit setzt durch, dass ein Benutzer innerhalb des Systems nur die Aktionen ausführen kann, zu denen er berechtigt ist.
- b.) Die Authentifizierung stellt fest, ob eine Person wirklich die ist, als die sie sich ausgibt. Möglichkeiten zur Authentifizierung: Tokenbasierte Verfahren (Schlüssel, Magnetkarten usw.), wissensbasierte Verfahren (Passwörter usw.), biometrische Verfahren (körperliche Merkmale).
- c.) Das Modell definiert eine Menge von Subjekten S , eine Menge von Objekten O sowie für jedes Paar aus Subjekt s und Objekt o die Aktionen, die s auf o durchführen kann. Durchsetzung durch Zugriffskontrollmatrizen, Zugriffskontrolllisten, Capabilities.
- d.) Bei symmetrischer Verschlüsselung wird zur Ver- und Entschlüsselung derselbe Schlüssel verwendet, der daher auf beiden Seiten geheim gehalten werden muss. Bei asymmetrischer Verschlüsselung werden unterschiedliche Schlüssel benutzt, so dass nur der Schlüssel zur Entschlüsselung geheim gehalten werden muss.
- e.) Die Schlüssel von DES sind so kurz, dass heutige Computer mit „Brute Force“ (d.h. Durchprobieren aller möglichen Schlüssel) einen verschlüsselten Text „knacken“ können.
- f.) Ein Programm, das in einem System verborgen ist oder das den Anschein erweckt, nützliche Aufgaben zu erfüllen, bei seiner Ausführung aber Schaden anrichtet.

2. Methoden der internen Sicherheit I

a.) Prozess 1 / 2 / 3 sind die Subjekte (P_1, P_2, P_3), die Ports (A, B, C) sind die Objekte. Operationen sind das Entnehmen (Entn), das Einfügen (Einf) und das Betrachten (Betr).

b.) Zugriffskontrollmatrix:

	A	B	C
P_1	Entn	Einf	-
P_2	-	Entn	Einf
P_3	Betr	Betr	Betr

Zugriffskontrolllisten:

für Port A: (P_1 , Entn), (P_3 , Betr)

für Port B: (P_1 , Einf), (P_2 , Entn), (P_3 , Betr)

für Port C: (P_2 , Einf), (P_3 , Betr)

Capabilities:

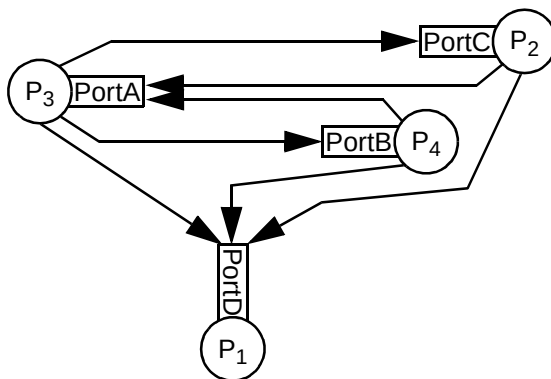
für Prozess P_1 : (A, Entn), (B, Einf)

für Prozess P_2 : (B, Entn), (C, Einf)

für Prozess P_3 : (A, Betr), (B, Betr), (C, Betr)

3. Methoden der internen Sicherheit II

a.)



P_1 : Log-Prozess

P_2 : Client

P_3 : Server

P_4 : Client

b.) Zugriffskontrolllisten:

Port A: (P_2 , W), (P_3 , R), (P_4 , W)

Port B: (P_3 , W), (P_4 , R)

Port C: (P_2 , R), (P_3 , W)

Port D: (P_1 , R), (P_2 , W), (P_3 , W), (P_4 , W)

Capabilities:

P_1 : (Port D, R)

P_2 : (Port A, W), (Port C, R), (Port D, W)

P_3 : (Port A, R), (Port B, W), (Port C, W), (Port D, W)

P_4 : (Port A, W), (Port B, R), (Port D, W)

4. Capabilities

- a.) Ein neues Objekt O_6' erzeugen, den Inhalt von O_6 dorthin kopieren, C_B auf O_6' umsetzen und O_6 löschen.
- b.) O_3 , O_4 und O_5 sind nicht mehr zugreifbar, könnten also gelöscht werden. Allgemein würde der Hauptspeicher mit nicht mehr zugreifbaren Objekten „voll laufen“, wenn diese nicht gelöscht würden.
- c.) Bevor ein Objekt gelöscht werden kann, muss sichergestellt sein, dass kein Prozess mehr auf dieses Objekt zugreifen kann. Das Betriebssystem muss dazu von den Basis-Capabilities der Prozesse ausgehen und schrittweise alle Objekte markieren, die es von dort aus erreichen kann. Anschließend können alle nichtmarkierten Objekte gelöscht werden. Das geschieht nicht ständig, sondern nur von Zeit zu Zeit, um den Speicher „aufzuräumen“. Referenzzähler nützen nicht immer etwas, da sie in Fällen wie in Teilaufgabe b.) von 0 verschieden sind, obwohl die zugehörigen Objekte für die Prozesse nicht zugreifbar sind.

5. Kryptographie: zwei primitive Verfahren

- DIESER CODE KANN SEHR LEICHT GEKNACKT WERDEN.
(Verschlüsselung: Ersetzen jedes Buchstabens durch seinen Nachfolger im Alphabet.)
- ES GIBT SEHR VIEL BESSERE VERFAHREN
ALS DIESE EINFACHEN HIER.
(Verschlüsselung: Ersetzen jedes Buchstabens anhand der Umordnung METHUSALIXZYWVRQPONKJGFDCB des Alphabets, d.h. A durch M, B durch E usw.)

zu Kapitel 8

1. Wissensfragen

- a.) Siehe Abschnitt 8.1.1.
- b.) Durch ein Netz aus autonomen Workstations oder durch grafikfähige Terminals, die an eine gemeinsame Zentraleinheit mit Universal-CPU's angeschlossen sind.
- c.) Beim Netzbetriebssystem „sieht“ der Benutzer den Unterschied zwischen lokaler und entfernter Bearbeitung, beim verteilten System nicht.
- d.) Nach ihrer räumlichen Ausdehnung (LAN, MAN, WAN) und nach dem Zugriffsverfahren (Ethernet, Token Ring, FDDI usw.).
- e.) Carrier Sense Multiple Access / Collision Detection.

- f.) Ein System von Regeln, die den zeitlichen Ablauf einer Kommunikation und die Formate der dabei verwendeten Pakete festlegen.
- g.) IP als Schicht-3-Protokoll zum „Routing“ der Pakete, TCP und UDP als Schicht-4-Protokolle zur Steuerung der Ende-zu-Ende-Kommunikation und FTP, HTTP, Telnet usw. als Schicht-7-Protokolle zur Unterstützung verschiedener Anwendungen.

2. Netztopologie

- Vollständige Vernetzung: hoher Verkabelungsaufwand; hoher Datendurchsatz möglich, da viele Leitungen vorhanden; hohe Robustheit gegen Leitungsausfälle, da alternative Wege benutzt werden können.
- Sterntopologie: niedrigerer Verkabelungsaufwand; zentraler Knoten muss sehr leistungsfähig sein, da es hier sonst zu Durchsatzproblemen kommt; bei Ausfall einer Leitung Isolierung eines Knotens, aber störungsfreier Weiterbetrieb des Restnetzes; bei Ausfall des zentralen Knotens überhaupt keine Kommunikation mehr möglich.
- Bustopologie: minimaler Verkabelungsaufwand; bei hohem Verkehrsaufkommen von mehreren Stationen Durchsatzprobleme, da jeweils nur eine Station senden darf; bei Kabelstörung im Bus Zerfall des Netzes in zwei oder mehrere Teilnetze oder sogar Ausfall der gesamten Kommunikation.

3. Ethernet vs. Token Ring

- a.) Bei niedrigem Verkehrsaufkommen ist CSMA/CD günstiger, da eine Station sofort senden kann, ohne auf die Zuteilung einer Sendeberechtigung warten zu müssen. Bei hohem Verkehrsaufkommen ist der tokenbasierte Zugriff günstiger, da es dann bei CSMA/CD zu zahlreichen Kollisionen und somit Verlust an nutzbarer Netzkapazität kommt.
- b.) Der Token Ring, da hier das Token spätestens nach einer bestimmten Zeit (der so genannten „Token Holding Time“) weitergegeben werden muss. Aus dieser Zeit und der Anzahl der Stationen im Netz lassen sich Garantien für maximale Wartezeiten und mindestens verfügbare Kapazitäten berechnen. Für das Ethernet ist das nicht möglich, da durch Kollisionen Übertragungen auf unbestimmte Zeit verzögert werden können.
- c.) Der Sender könnte sofort, wenn er das letzte Bit seines Pakets abgeschickt hat, ein neues freies Token losschicken, da der Ring hinter dem Paket ja schon wieder frei ist. Man bezeichnet diese Technik als „Early Token Release“.

4. Internet

IP ist ein verbindungsloses Protokoll, das keine Reservierungen im Netz vornimmt und zudem Pakete über unterschiedliche Wege leiten kann. Es kann also weder eine Kapa-

zitätsgarantie noch eine Garantie für die Übertragungsverzögerung gegeben werden, was sich bei hoher Netzlast störend bemerkbar macht (lange Laufzeiten, Datenverluste). Ein weiteres Problem ist, dass TCP Datenverluste nach einer Wartezeit durch eine erneute Übertragung repariert: Für eine Realzeitanwendung führt dies zu nicht akzeptablen Verzögerungen.

zu Kapitel 9

1. Wissensfragen

- a.) Man kann damit bestimmte Dienste für alle Rechner im Netz bereitstellen, muss sie aber nicht in jedem einzelnen Netzknoten implementieren.
- b.) Der Directory Server bietet einen Namensdienst, mit dem u.a. Dateinamen auf Adressen abgebildet werden; der File Server stellt die Dateien selbst bereit.
- c.) Das Einbinden eines Speichermediums oder Dateisystems in ein anderes Dateisystem, das möglicherweise auf einem anderen Rechnerknoten liegt.
- d.) Remote Procedure Call (Prozedurfernaufruf). Programmieretechnik zum Aufruf eines Dienstes auf einem entfernt liegenden Server.
- e.) Die Stände der Uhren auf den Knoten eines Rechnernetzes werden einander angeglichen.
- f.) Middleware ist eine Sammlung von Diensten, die als Basis für verteilte Anwendungen dienen. Beispiele: CORBA und DCE.
- g.) Zur Realisierung eines Systems, bei dem Programme auf die Benutzeroberflächen anderer Rechnerknoten zugreifen können – insbesondere von Terminals, die über keinen eigenen Universal-Prozessor verfügen.

2. Verteilte Dateisysteme

- a.) Datenbank auf Knoten_C, Praktikumsdateien und Internet auf Knoten_B, Textverarbeitung auf Knoten_A.

b+c.) analog zu Abbildung 9.3.

3. Client-Server: Optimierung von Zugriffszeiten

- a.) D_1 und D_4 auf dem einen Server, D_2 und D_3 auf dem anderen.
- b.) Man installiert am besten D_1 und D_4 auf S_A sowie D_2 und D_3 auf S_B , denn dann ergeben sich pro Sekunde Übertragungszeiten von insgesamt 255 ms. Bei umgekehrter Zuordnung beträgt der Wert 345 ms.

- c.) Wenn man D_3 und D_4 auf S_A sowie D_1 und D_2 auf S_B installiert, beträgt die gesamte Übertragungszeit nur 240 ms, was noch günstiger ist.

4. Client-Server: Replikation

- a.) $K = (20+5+1+4+2+3+15+2)*L + (4+3+8+30+2+18+2+1)*E = 52*L + 68*E$
 $K = 3500 \text{ ms}$ für $L=15 \text{ ms}$ und $E=40 \text{ ms}$
- b.) $K = (20+5+4+3+8+30+1+4)*L + (2+3+2+18+2+1+15+2)*E = 75*L + 45*E$
 $K = 2925 \text{ ms}$ für $L=15 \text{ ms}$ und $E=40 \text{ ms}$
- c.) Datei 3 wird besser nicht repliziert, da P_B sehr viele Schreibzugriffe auf sie durchführt, die ohne Replikation lokal, mit Replikation aber über das Netz ablaufen. Dem steht nur eine geringere Einsparung bei den Leseoperationen von P_A gegenüber. Ohne Replikation von Datei 3 betragen die Gesamtkosten nur 2650 ms.
- d.) Die Replikation lohnt sich, wenn $75*L + 45*E < 52*L + 68*E$ gilt. Das ist der Fall für $E/L > 1$, also immer (denn $E > L$). Dies gilt nicht für beliebige Zugriffshäufigkeiten: Im Fall von vielen lokalen Schreiboperationen und wenigen entfernten Leseoperationen ist eine Replikation nachteilig. Sie würde nämlich mehr Kosten durch zusätzliche entfernte Schreiboperationen verursachen, als bei den Leseoperationen, die nun lokal stattfinden könnten, eingespart würde.

zum Anhang „UNIX-C-Schnittstelle“

Aus Platzgründen können hier keine Musterlösungen angegeben werden. Zudem möchte der Autor diese oder ähnliche Aufgaben auch in Zukunft in seinem Praktikum stellen :-)

Für Lösungshilfen wird auf die Übungsaufgaben der Kapitel 3 und 4 sowie auf die Beispiele in der Dokumentation der Funktionen verwiesen.

Literatur

Das vorliegende Buch kann und soll nur eine Übersicht über die wichtigsten Aspekte von Betriebssystemen und verteilten Systemen geben. Für die vertiefende Lektüre bietet sich eine Vielzahl von speziellen Lehr- und Handbüchern an. Als Orientierungshilfe wird im Folgenden eine kommentierte Auswahl von Büchern gegeben. Sie befassen sich mit allgemeinen Konzepten von Betriebssystemen, Rechnernetzen und verteilten Systemen sowie mit Benutzeroberfläche, Systemschnittstelle und interner Realisierung von UNIX / Linux und Windows NT.

Ein Quellenverzeichnis schließt sich an.

Kommentierte Literaturempfehlungen

Betriebssysteme allgemein

R. Brause: Betriebssysteme – Grundlagen und Konzepte; Springer, Heidelberg, 2. Aufl. 2001.

Beschreibt die wichtigsten Aspekte moderner Betriebssysteme, wobei auch verteilte Systeme angesprochen werden. Benutzt als durchgängige Beispiele UNIX und Windows NT.

J. Nehmer, P. Sturm: Systemsoftware – Grundlagen moderner Betriebssysteme; dpunkt, Heidelberg, 1998.

Ebenfalls ein Buch über die grundlegenden Konzepte von Betriebssystemen mit zahlreichen praktischen Beispielen. Es beschränkt sich allerdings auf lokale Systeme; verteilte Systeme sollen in einem zweiten Band behandelt werden, der zum Zeitpunkt der Drucklegung dieses Buchs aber noch nicht erschienen war.

H.-J. Siebert, U. Baumgarten: Betriebssysteme – Eine Einführung; Oldenbourg, München / Wien, 4. Aufl. 1998.

Auch dies eine Abhandlung über die Grundlagen von Betriebssystemen, die allerdings weniger Beispiele aus konkreten Betriebssystemen als die beiden zuvor genannten Bücher bietet.

A. Silberschatz, P. Galvin: Operating System Concepts; Addison Wesley Longman, Reading, 5th Ed. 1998.

Sehr umfangreiches, recht aktuelles Betriebssystemlehrbuch. Beschreibt grundlegende Betriebssystemkonzepte und gibt Implementierungsbeispiele, insbesondere UNIX, Linux und Windows NT.

A. S. Tanenbaum: Modern Operating Systems; Prentice Hall, Englewood Cliffs, 1992.

Klassisches Betriebssystemlehrbuch. Zwar relativ alt, beschreibt aber sehr gut lesbar grundlegende Betriebssystemkonzepte und enthält einen umfangreichen Teil über verteilte Systeme.

Eine Alternative ist das Buch A. S. Tanenbaum, A. S. Woodhull: Operating Systems: Design and Implementation; Prentice Hall, 1997. Es enthält den Quellcode des Betriebssystems MINIX, das zu Lehrzwecken entwickelt wurde. Verteilte Systeme werden hier allerdings nicht betrachtet.

Deutsche Übersetzung: Moderne Betriebssysteme; Hanser / Prentice Hall, München / London, 1993.

Rechnernetze und Verteilte Systeme

G. Bengel: Verteilte Systeme – Client-Server-Computing für Studenten und Praktiker; vieweg, Braunschweig / Wiesbaden, 2000.

Praxisorientiertes Buch, das schwerpunktmäßig verschiedene Ansätze zur Client-Server-Programmierung diskutiert. Aus Vorlesungen an der Fachhochschule Mannheim hervorgegangen.

D. E. Comer: Computer Networks and Internets; Prentice Hall, Englewood Cliffs, 2nd Ed. 1999.

Ein sehr gut geschriebenes Lehrbuch über vernetzte Computersysteme. Diskutiert technische Grundlagen der Datenkommunikation, Paketübertragung, Internetkonzepte und Anwendungen in Computernetzen.

Deutsche Übersetzung: Computernetzwerke und Internets; Markt + Technik, München, 2000.

A. S. Tanenbaum: Modern Operating Systems; Prentice Hall, Englewood Cliffs, 1992.

Siehe „Betriebssysteme allgemein“. Eine Alternative ist das Buch A. S. Tanenbaum: Distributed Operating Systems; Prentice Hall, 1994 – eine eigenständige, erweiterte Publikation des Teils über verteilte Systeme aus „Modern Operating Systems“.

A. S. Tanenbaum: Computer Networks; Prentice Hall, Englewood Cliffs, 3rd Ed. 1996.

Ebenfalls ein gut lesbarer Klassiker über Computernetze. Beschreibt eine Vielzahl von Aspekten der verschiedenen Stufen der Protokollhierarchie.

Deutsche Übersetzung: Computernetzwerke; Markt + Technik, München, Neuauflage 2000.

M. Weber: Verteilte Systeme; Spektrum Akademischer Verlag, Heidelberg, 1998.

Umfangreiches, praktisch orientiertes Buch über verteilte Systeme. Aus einem Vorlesungsskript der Universität Ulm hervorgegangen.

UNIX/Linux-Benutzeroberfläche

J. Gulbins, K. Obermayr: UNIX System V. Begriffe, Konzepte, Kommandos, Schnittstellen; Springer, Berlin / Heidelberg, 4. Aufl. 1995.

Eines der klassischen deutschen UNIX-Handbücher. Umfassende Übersicht über UNIX, hauptsächlich aus Benutzersicht mit einer relativ knappen Übersicht über die Programmierschnittstelle.

H. Herold: Linux-Unix-Grundlagen / Kurzreferenz / Profitools / Shells / Systemprogrammierung; Addison-Wesley, München, 1998 ff.

Breit angelegte Reihe aus mehreren Büchern, in der verschiedene Aspekte von UNIX / Linux detailliert dargestellt werden. Ersetzt die neunbändige Reihe „UNIX und seine Werkzeuge“, die vom selben Verlag ab Anfang der neunziger Jahre herausgegeben wurde.

UNIX/Linux-C-Schnittstelle

S. Fischer, W. Müller: Netzwerkprogrammierung unter Linux und UNIX; Hanser, München / Wien, 1999

Kompaktes, übersichtliches Buch mit vielen Beispielen zur Verwendung von Schnittstellenfunktionen zur Netzkommunikation; enthält zudem Kurzübersichten von Java und CORBA.

H. Herold: Linux-Unix-Systemprogrammierung; Addison-Wesley, München, 1999.

siehe „UNIX/Linux-Benutzeroberfläche“.

M. J. Rochkind: Advanced UNIX Programming; Prentice Hall, Englewood Cliffs, 1985.

Beschreibung von UNIX auf Ebene der C-Schnittstelle.

Deutsche Übersetzung: UNIX-Programmierung für Fortgeschrittene; Hanser / Prentice Hall, München / London, 1988.

W. R. Stevens: Advanced Programming in the UNIX Environment; Addison-Wesley, Reading, 1992.

Sehr ausführliche Beschreibung der UNIX-C-Schnittstelle mit zahlreichen Programmbeispielen.

Deutsche Übersetzung: Programmierung in der UNIX-Umgebung; Addison-Wesley, Bonn, 1995.

W. R. Stevens: UNIX Network Programming, Volume 1: Networking APIs – Sockets and XTI / Volume 2: Interprocess Communications; Prentice Hall, Englewood Cliffs, 1997 / 1998.

Zwei umfangreiche Bände, in denen die Funktionen der UNIX-C-Schnittstelle zur Kommunikation über Netze umfassend dargestellt werden.

Deutsche Übersetzung: Programmieren von UNIX-Netzwerken; Hanser, München / Wien, 2000.

H. Weber: Praktische Systemprogrammierung; vieweg, Braunschweig / Wiesbaden, 1998.

Umfangreiche Darstellung der UNIX-C-Schnittstelle und zusätzlich einiger UNIX-Werkzeuge mit vielen Beispielen. Aus Lehrveranstaltungen an der Fachhochschule Wiesbaden hervorgegangen.

POSIX-Programmierschnittstelle

D. Lewine: POSIX Programmer's Guide; O'Reilly, Sebastopol, 1991.

Grundlegendes Buch zum POSIX-Standard 1003.1: Einführung in den Standard und ausführliche Übersicht über die enthaltenen Schnittstellenfunktionen.

B. O. Gallmeister: POSIX.4 – Programming for the Real World; O'Reilly, Sebastopol, 1995.

Einführung in den POSIX-Standard 1003.1b (früher POSIX.4) mit ausführlicher Darstellung der Schnittstellenfunktionen, u.a. zu Prozesskommunikation und -synchronisation.

UNIX/Linux-Kern

M. J. Bach: Design and Implementation of the UNIX Operating System; Prentice Hall, Englewood Cliffs, 1986.

Umfassender Einblick in die Implementierung von UNIX (hauptsächlich System V) mit einer zusätzlichen kompakten Auflistung der wichtigsten Funktionen der UNIX-C-Schnittstelle.

Deutsche Übersetzung: UNIX: Wie funktioniert das Betriebssystem?; Hanser / Prentice Hall, München / London, 1991.

M. K. McKusick, K. Bostic, M. J. Karels, J. S. Quarterman: The Design and Implementation of the 4.4BSD UNIX Operating System (Unix and Open System Services); Addison-Wesley Longman, Reading, 1996.

Ausführliche Darstellung der BSD-Implementation von UNIX.

D. A. Rusling: The Linux Kernel; <http://www.linuxdoc.org/LDP/tlk/tlk.html>, 1996-1999.

Online-Buch mit zahlreichen Informationen über Interna von Linux.

Windows NT

D. A. Solomon: Inside Windows NT – Second Edition; Microsoft Press, Redmond, 1998.

Das „offizielle“ Buch über die internen Strukturen und Mechanismen von Windows NT 4.0.

J. Richter: Advanced Windows – Third Edition; Microsoft Press, Redmond, 1996.

Sehr umfangreiche Beschreibung der Windows-NT-Programmierschnittstelle.

Da es zur Benutzeroberfläche von Windows NT eine Vielzahl von Literatur gibt, soll hier kein Buch besonders herausgehoben, sondern einfach ein Besuch in einer Buchhandlung oder Bibliothek empfohlen werden.

Quellenverzeichnis

- [Bach86] M. J. Bach: The Design of the UNIX Operating System; Prentice Hall, Englewood Cliffs, 1986.
- [Bau95] F. Bauer: Entzifferte Geheimnisse – Methoden und Maximen der Kryptologie; Springer, Berlin / Heidelberg, 1995.
- [Brin73] P. Brinch Hansen: Operating Systems Principles; Prentice Hall, Englewood Cliffs, 1973.
- [BSI00] Bundesamt für Sicherheit in der Informationstechnik: IT-Grundschutzhandbuch – M 2.167 Sicheres Löschen von Datenträgern; <http://www.bsi.bund.de/gshb/deutsch/m/m2167.htm>, 2000.
- [CCC01] Chaos Computer Club e.V.: WWW-Leitseite; <http://www.ccc.de>.
- [CERT01] CERT Coordination Center, Carnegie Mellon University: WWW-Leitseite; <http://www.cert.org>.
- [Com99] D. E. Comer: Computer Networks and Internets; Prentice Hall, Englewood Cliffs, 2nd Ed. 1999.
- [Dei90] H. M. Deitel: An Introduction to Operating Systems; Addison-Wesley, Reading, 2nd Ed. 1990.
- [Denn68] P. J. Denning: The Working Set Model for Program Behavior; Comm. of the ACM, Vol. 11, No. 5 (May 1968), pp. 323-333.
- [Dijk65] E. W. Dijkstra: Cooperating Sequential Processes; Tech. Univ. Eindhoven, 1965 (Reprint in: F. Genuys (Ed.): Programming Languages; Academic Press, New York, 1965).
- [DoD85] Anonymous: Department of Defense Trusted Computer System Evaluation Criteria; U.S. Department of Defense, DoD 5200.28-STD, Dec. 1985.
- [FiMü99] S. Fischer, W. Müller: Netzwerkprogrammierung unter Linux und UNIX; Hanser, München / Wien, 1999.
- [Gall95] B. O. Gallmeister: POSIX.4: Programming for the Real World; O'Reilly, Sebastopol, 1995.
- [Gall00] D. L. Galli: Distributed Operating Systems, Concepts & Practice; Prentice Hall, Upper Saddle River, 2000.
- [GuOb95] J. Gulbins, K. Obermayr: UNIX System V. Begriffe, Konzepte, Kommandos, Schnittstellen; Springer, Berlin / Heidelberg, 4. Aufl. 1995.
- [Her98] H. Herold: Linux-Unix-Grundlagen / Kurzreferenz / Shells; Addison-Wesley, München, 1998 ff.
- [Her99] H. Herold: Linux-Unix-Systemprogrammierung; Addison-Wesley, München, 2. Aufl. 1999.
- [Hoar74] C. A. R. Hoare: Monitors: An Operating Systems Structuring Concept; Comm. of the ACM, Vol. 17, No. 10 (Oct. 1974), pp. 549-557.
- [JaHP00] A. Jain, L. Hong, S. Pankanti: Biometric Identification; Comm. of the ACM, Vol. 43, No. 2 (Feb. 2000), pp. 90-98.
- [Lam78] L. Lamport: Time, Clocks and the Ordering of Events in a Distributed System; Comm. of the ACM, Vol. 21, No. 7 (July 1978), pp. 558-565.
- [Lew91] D. Lewine: POSIX Programmer's Guide; O'Reilly, Sebastopol, 1991.

- [LiLa73] C. L. Liu, J. W. Layland: Scheduling Algorithms for Multiprogramming in Hard Real-Time Environment; Vol. 20, No. 1 (Jan. 1973), pp. 46-61.
- [MICO01] MICO – Mico Is CORba: WWW-Leitseite; <http://www.mico.org>.
- [Nach97] C. Nachenberg: Computer Virus-Antivirus Coevolution; Comm. of the ACM, Vol. 40, No. 1 (Jan. 1997), pp. 46-51.
- [NIST01] NIST Computer Security Resource Center: AES – A Crypto Algorithm for the Twenty-First Century; <http://csrc.nist.gov/encryption/aes>.
- [Open97] Anonymous: The Single UNIX Specification, Version 2; Open Group Publication Set T912, 1997.
- [PGP01] PGP Security: WWW-Leitseite; <http://www.pgp.com>.
- [RePo99] P. Rechenberg, G. Pomberger (Hrsg.): Informatik-Handbuch; Hanser, München / Wien, 1999.
- [RFC1094] Anonymous: NFS: Network File System Protocol Specification; Request for Comments 1094, March 1989.
- [RFC1305] D. L. Mills: Network Time Protocol (Version 3): Specification, Implementation and Analysis; Request for Comments 1305, March 1992.
- [RFC1700] J. Reynolds, J. Postel: Assigned Numbers; Request for Comments 1700, Oct. 1994.
- [RFC1813] B. Callaghan, B. Pawlowski, P. Staubach: NFS Version 3 Protocol Specification; Request for Comments 1813, June 1995.
- [RFC1831] R. Srinivasan: RPC: Remote Procedure Call Protocol Specification Version 2; Request for Comments 1831, Aug. 1995.
- [RFC1833] R. Srinivasan: Binding Protocols for ONC RPC Version 2; Request for Comments 1833, Aug. 1995.
- [RFC2624] S. Shepler: NFS Version 4 Design Considerations; Request for Comments 2624, March 1989.
- [Rus99] D. A. Rusling: The Linux Kernel; <http://www.linuxdoc.org/LDP/tlk/tlk.html>, 1996-1999.
- [Schm98] H. Schmitz: Der Inhalt von Smartcards läßt sich mit einfacher Elektronik-ausrüstung lesen; Computerzeitung, Konradin-Verlag, 10.9.1998.
- [SiGa98] A. Silberschatz, P. Galvin: Operating System Concepts; Addison Wesley Longman, Reading, 5th Ed. 1998.
- [Snor01] M. Roesch et al.: The Lightweight Network Intrusion Detection System; <http://www.snort.org>.
- [SpSa96] M. Spasojevic, M. Satyanarayanan: An Empirical Study of a Wide-Area Distributed File System; ACM Transactions on Computer Systems, Vol. 14, No. 2 (May 1996), pp. 200-222.
- [Stev92] W. R. Stevens: Advanced Programming in the UNIX Environment; Addison-Wesley, Reading, 1992.
- [Stev97] W. R. Stevens: UNIX Network Programming; Prentice Hall, Englewood Cliffs, 1997 / 1998.
- [Sun01] Sun Microsystems: Sun Product Documentation; <http://docs.sun.com>.
- [Tag01] ARD – Erstes Deutsches Fernsehen: Tagesschau, 19.1.2001, 20 Uhr.
- [Tan92] A. S. Tanenbaum: Modern Operating Systems; Prentice Hall, Englewood Cliffs, 1992.

- [Tan96] A. S. Tanenbaum: Computer Networks; Prentice Hall, Englewood Cliffs, 3rd Ed. 1996.
- [Web98] M. Weber: Verteilte Systeme; Spektrum Akademischer Verlag, Heidelberg, 1998.
- [WeLu99] R. Weis, S. Lucks: Sichere, Standardisierte, Symmetrische Verschlüsselung auf Basis von DES und AES; Praxis der Informationsverarbeitung und Kommunikation (PIK), 22. Jg. (1999), S. 226-232.

Index

Zahlen

32-Bit-System 157
802.3 243
802.5 243

A

Abschnitt, kritischer 90
Abstract Syntax Notation 1 (ASN.1) 239
Access Control List (ACL) 212
Adapter 191
Adresse, reale 135, 148, 152
Adresse, virtuelle 152, 154, 157
Advanced Encryption Standard (AES) 218
Aging 68
AIX 30
Amoeba 214, 228
Andrew File System (AFS) 255, 262, 274
Application Programming Interface (API) 6
Assoziativspeicher 137, 160
Asynchronous Transfer Mode (ATM) 249
Ausführungsmodus 19
Ausschluss, wechselseitiger 89, 96, 270
Authentifizierung 203
 biometrische 204, 206–209
 tokenbasierte 204
 wissensbasierte 204–206

B

Backup 189
Balanced-Buffers-Strategie 142
Bankiersalgorithmus 108
Basisregister 136, 148
Batch-Betrieb 8
Batch-Prozess 10
B-Baum 173
Bedingungsvariable 104
Benutzerprozess 50
Benutzerschnittstelle 5, 6, 277–278
Best Fit 135
Betanova 24
Betriebsart 6–12
Betriebsmittel 5
Betriebssystem 2
 Aufgabenbereiche 3
 Basiskonzepte 3–5
 Beispiele 20–25

 Multiprozessor-Betriebssystem 230
 Netzbetriebssystem 230
 Struktur 12–20
 verteiltes 230
Biometrie 204, 206–209
BIOS 35
Bitmap 187
Block Consistency Check 189
Boot Sector 184
Bridge 245
Broadcast 114, 238
BS 2000 21
BSD-UNIX 30
Buffer Cache 140, 188
Bus 131
Bus Sperre 9
Busstruktur 242

C

C 29
Cache 131, 136
 Organisation 137
 Platten-Cache 140, 188, 257, 260
 write-through vs. copy-back 137
Capability 212–216
Chaos Computer Club 202
Client-Server 16, 114, 262
 mit Message Queues 322, 323
 mit Ports 115
 mit Sockets 323
 Threading 271
Clock-Algorithmus 162
Cluster 139, 179, 227
Common Desktop Environment (CDE) 278
Common Object Request Broker Architecture (CORBA) 274–277
Condition Variable 104
Context Switch 64
Controller 191
Copy-On-Write 63
Cracker 199, 202, 203
Critical Section 104
CSMA/CD 244

D

Daemon 42, 50
Data Encryption Standard (DES) 218

Datei 171
 Attribute 39, 174
 direkt zugreifbare 173
 indexsequentielle 174
 interne Struktur 173
 Konsistenz 258, 260
 Name 174
 Replikation 258
 sequentielle 173
 Dateibaum 38, 174
 Arbeitsverzeichnis 40
 Home Directory 40
 in verteilten Systemen 254
 Pfade 40
 Wildcarding 40
 Dateisystem 4, 171–190
 Backup 189
 Operationen 176, 185–187
 Partitionen 183
 Speicherverwaltung 187
 Struktur 174
 verteilt 251–262
 Dateiverzeichnis 174, 176–183
 Datennetz *siehe Kommunikationsnetz*
 dbx 24
 Deadline 78
 Deadlock 105–109
 Behebung 108
 Definition 106
 notwendige Bedingungen 106
 Verhinderung 106
 Vermeidung 107
 Demand Paging 161
 Dialogbetrieb 11
 Digest 218
 DIN 44300 1
 Direct Memory Access (DMA) 9, 192
 Directory 174, 176–183
 Directory Server 252–257
 Diskless Workstation 251, 260
 Dispatching 63
 Distributed Component Object Model (DCOM) 266, 276
 Distributed Computing Environment (DCE) 273–274
 Distributed File System (DFS) 274
 Distributed Shared Memory (DSM) 113, 262
 Domain Name System (DNS) 247, 261
 Drucker-Spooling 10
 duplex 117
 Dynamic Link Library (DLL) 111

E

Early Token Release 344
 Ein-/Ausgabe (E/A) 4, 190–196
 Hardware 190–192
 Software 193–196
 Einbenutzerbetrieb 8
 Einzelauftragsbearbeitung 8

Election-Verfahren 271
 Epoc 23
 Error Correction Code (ECC) 132
 Error Detection Code (EDC) 132
 Erzeuger-Verbraucher 97
 Ethernet 244
 Event 103
 ext2fs 181, 185, 212

F

Fahrstuhl-Algorithmus 141
 FAT-Dateisystem 178, 184
 im Vergleich mit NTFS 182
 Fehlertoleranz 4, 81, 199
 auf dem Plattenspeicher 188–190, 258
 im Hauptspeicher 132
 im verteilten System 232, 269
 Festplatte *siehe Plattenspeicher*
 Fiber Distributed Data Interface (FDDI) 245
 FIFOs 119
 File Allocation Table (FAT) 178
 File Consistency Check 189
 File Locking 176
 File Server 251, 252
 File Transfer Protocol (FTP) 230, 240, 248
 Firewall 203
 First Come First Served (FCFS) 66, 141
 First Fit 134
 First In First Out (FIFO) 162
 Fragmentierung 135
 FreeBSD 30
 Freibereichsliste 134
 Frist 78

G

General Inter-ORB Protocol (GIOP) 276
 Gerätetreiber 123, 193
 Glasfaserkabel 241
 Gnome 278
 Graceful Degradation 4

H

Hacker 201, 202
 Hamming Code 132, 188
 Handheld Computers 23
 Hard Link 181
 Hash 218
 Hauptspeicher 131, 132–136
 Hintergrundspeicher 139
 HP-UX 30
 HyperText Transfer Protocol (HTTP) 240

I

Identifizierung 204

Idle Task 64
 Interface Definition Language (IDL) 274, 276
 International Data Encryption Algorithm (IDEA) 218
 Internet 240, 246–249
 Adressen 247
 Protokolle 248
 Routing 248
 Internet Inter-ORB Protocol (IIOP) 276
 Internet Protocol (IP) 122, 240, 247
 Inter-Process Communication (IPC) 109
 Interrupt 73–77, 80
 Handler 74
 Implementierung 74
 Maskierungsregister 77
 Prioritäten 75
 Schachtelung 75
 Speicherregister 76
 Sperrung 91
 Tabelle 74
 Vektor 74
 Intruder Detection System (IDS) 203
 Intruder Response System (IRS) 203

J

Java 15
 Java Virtual Machine (JVM) 15
 Remote Method Invocation (RMI) 266
 Sockets 122
 Synchronisation 104
 Threads 52
 Job Control Language 8

K

Kachel 157
 Kanal, verdeckter 201
 KDE 278
 Kerberos 274
 Kern 12, 18–20
 minimaler 16
 Kernel Mode 19, 36
 Koaxialkabel 241
 Kommandosprache 6
 Kommunikation
 asynchrone 116
 bidirektionale 116
 nachrichtenbasierte 113–127
 nicht zuverlässige 123
 Punkt-zu-Punkt 238
 Sender-Empfänger-Beziehungen 113
 speicherbasierte 110–112
 synchrone 116
 Übertragungsarten 114
 unidirektionale 116
 verbindungslose 123, 237, 249
 verbindungsorientierte 123, 237, 248
 Zeitverhalten 116
 zuverlässige 123

Kommunikationsnetz 123, 233–249
 drahtloses 242
 Klassifikation 243
 Langsamkeit 227
 Topologie 242
 Übertragungsmedien 241
 Kommunikationsprotokoll 235
 Schichtung / Stack 236
 Konfigurierung 268
 Kontext 49, 51
 Kopplung, enge vs. lose 225, 227, 229
 Kryptologie 216–219

L

Ladestrategie 161
 Längenregister 148
 Lastverteilung 268
 Least Frequently Used (LFU) 163
 Least Recently Used (LRU) 162
 Leser-Schreiber-Problem 128
 Lichtwellenleiter (LWL) 241
 Link 175, 181
 Linux 23, 30
 siehe auch UNIX
 Dateisystem ext2fs 181, 185, 212
 Dateisystem FAT 178
 Distributionen 30
 Speicherverwaltung 166
 Threading 57
 Verzeichnis 177
 Virtual File System (VFS) 181, 187
 Local Area Network (LAN) 243
 Lock 174, 176
 Lock File 93, 287
 Lock Variable 93
 Login-Prozedur 203, 204

M

Mach 16, 21, 214, 215, 272
 MacOS 23
 Magnetplatte *siehe Plattenspeicher*
 Mailbox 114, 115
 Mainframe 20
 Makrovirus 220
 Maschine, virtuelle 6, 14
 Mehrprogrammbetrieb 10
 Mehrprozessorsystem 4, 227, 269, 271
 Memory Management Unit (MMU) 153
 Memory-Mapped File 176
 Memory-Mapped I/O 192
 Memory-Mapped Region 59
 Message Digest No. 5 (MD5) 218
 Metropolitan Area Network (MAN) 243
 MICO 277
 Middleware 233, 273
 Migration 269
 Mikrokern 16
 MINIX 181, 348

Mobiltelefon 23
 Mode Bit 19
 Modified Bit 158, 163
 Monitor 104
 Mounting 184, 254, 259
 MS-DOS 22, 35
 Dateisystem 178, 184
 Multicast 114, 238, 249
 Multicomputing 227, 228
 MULTICS 29, 210
 Multilevel Feedback Queueing 68
 Multilevel Queueing 68
 Multimedia 6, 78, 142, 187, 250
 Multiprocessing 227
 Multiprogramming 10, 49, 226
 Multiprozessor-Betriebssystem 230
 Multiprozessorsystem 4, 227, 269, 271
 Multitasking 226
 kooperatives 65
 Multi-User-Betrieb 6
 Multi-User-Fähigkeit 2
 Mutex 103
 MVS/ESA 20

N

Nachricht 235
 Nachrichtensystem 110
 Name Server 252, 253, 256, 263
 Namensdienst 252
 Nebenläufigkeit 4, 11, 225, 226–228
 Network Computer (NC) 233
 Network File System (NFS) 254, 258–260, 266
 Network Information Service (NIS) 260
 Network Time Protocol (NTP) 269
 Netz *siehe Kommunikationsnetz*
 Netzbetriebssystem 230
 Novell Netware 23, 262
 NT File System (NTFS) 182, 212

O

Object Management Group (OMG) 272
 Object Request Broker (ORB) 276
 Offset 136, 154, 158, 286
 OMG CORBA 274–277
 Open Group 32, 272
 Open Network Computing (ONC+) 258, 266
 Open Software Foundation (OSF) 32, 272
 Open VMS 21
 Orange Book 199
 OS/2 23
 OS/390 20
 OS/400 21
 OSEK 24
 OSF Distributed Computing Environment
 (OSF DCE) 273–274
 OSF/1 21, 32, 272
 OSI-Modell 238–240
 Overlay-Technik 149

P

Page Fault 159
 Page-Fault-Frequency-Strategie 164
 Paging 156–164
 Strategien 161–164
 PalmOS 23
 Parallelität *siehe Nebenläufigkeit*
 Parity Bit 132, 188
 Partitionen 133
 feste vs. variable 134
 Passwort 204–206
 Peer 236
 Persönliche Identifikationsnummer (PIN) 204
 Philosophenproblem 320
 Pinning 80
 Pipes 117–120
 Plattenspeicher 131, 139–142
 Cache 140, 188, 257, 260
 Cluster 139, 179
 Fehlertoleranz 188–190, 258
 Fragmentierung 187
 logische Struktur 183–185
 physische Struktur 139
 Scheduling 140–142
 Sicherheit 201
 Plug and Play 18, 22
 Polling 73
 Port 114, 115, 215, 247
 Nummer 247
 POSIX 30
 1003.1 32
 1003.1b 32, 72, 80
 1003.1c 52
 Prepaging 161
 Pretty Good Privacy (PGP) 219
 Priorität 68, 79
 Private Key Encryption 217
 Process Control Block (PCB) 53
 Process Identifier (PID) 53
 Programmierschnittstelle 6
 Protokoll 235
 Schichtung / Stack 236
 Prozess 3, 47–50, 52
 Benutzerprozess 50
 Context Switch 64
 Daemon 42, 50
 Dispatching 63
 Idle Task 64
 Implementierung 52–55
 interne Repräsentation 52
 Kommunikation 109–127
 Kontext 49, 51
 Kontrollblock 53
 Migration 269
 Nummer 53
 periodischer 78
 Realzeit 72
 Scheduling 63, 64–73, 78
 Synchronisation 89–109, 270
 Systemprozess 50
 Tabelle 53

Threading 50–52
 Wechsel 63–73
 Zustände 49, 54, 57
 Pseudo-Nebenläufigkeit 11, 226
 pthreads 52
 Public Key Encryption 218
 Pufferspeicher 136

R

Race Condition 94, 101, 121
 Random Access Memory (RAM) 132
 Realität, virtuelle 6
 Realzeit 4, 77–80, 142, 187
 Anforderungen 77
 Betrieb 4
 Garantien 78
 in Linux 72
 in UNIX 33
 Plattenscheduling 142
 POSIX 1003.1b 32, 72, 80
 Prozess 72
 Prozessorscheduling 72, 78, 79
 Systemeigenschaften 79
 Receive, Blocking / Non-Blocking 116, 122
 Record Locking 176
 Redundant Array of Inexpensive Disks (RAID) 188
 Redundanz 4, 81, 188, 258, 269
 Reed Solomon Code 132, 188
 Referenced Bit 158, 162, 164
 Referenzzähler 179, 185
 Reihenfolgebedingung 90
 Remote File System (RFS) 262
 Remote Procedure Call (RPC) 264–268
 Sun RPC 266–268
 Rendezvous 116
 Repeater 242
 Request-Reply-Protokoll 263
 Rijndael-Verfahren 218
 Rollback 104, 108
 Rotationsverzögerung 140
 Round Robin 67
 Router 245
 RPC Language (RPCL) 268
 rpcbind 267
 RSA-Verfahren 219

S

SCAN 141
 Scheduling 63, 64–73, 140–142
 First Come First Served 66
 fristenbasiertes 79, 142
 Multilevel Queueing 68
 nicht-unterbrechendes 65
 prioritätengesteuertes 68
 ratenmonotones 79
 Realzeit 78

Round Robin 67
 Shortest Job First 67
 unterbrechendes 65
 Schnittstelle 3, 6
 Schutzring 210
 SCSI 191
 Secure Hash Algorithm (SHA) 218
 Segment 153
 Segmentfehler 155
 Segmentregister 155
 Segmenttabelle 154
 Seite 157
 Seitenbereich 157
 Seitenfehler 159
 Seitentabelle 158
 invertierte 159
 Sekundärspeicher 139
 Semaphore 94–103
 Datenstrukturen 94
 Deadlock 106
 Operationen 95
 Send, Blocking / Non-Blocking 116
 Service Access Point (SAP) 236
 Shared Libraries 111
 Shared Memory 110–112, 156
 Shellsript 42
 Shortest Job First 67
 Shortest Seek Time First (SSTF) 141
 Sicherheit 4, 199–222
 bei Chipkarten 201
 bei Plattenspeichern 201
 externe 202–209
 interne 202, 209–216
 mit Kerberos 274
 Modell 200, 211
 Signale 103
 Signatur, digitale 218
 Simple Mail Transfer Protocol (SMTP) 240
 simplex 117
 Single UNIX Specification 32
 Single User Mode 8
 Skeleton 265, 276
 Socket 122, 248
 Soft Link 181
 Solaris 24, 30, 227, 230, 233
 Speicher
 Hierarchie 3, 131
 homogener 6, 149
 Kompaktierung 135
 Komponenten 131–142
 segmentorientierter 153–156
 seitenorientierter 156–164
 virtueller 148–166
 Zuteilungsstrategie 161, 163
 Sperrvariable 93
 Spinlock 92–94
 Spooling 9, 194
 Stand-by 81
 Stapelverarbeitung 8
 Starvation 94, 141
 Steganographie 220

Strahlung, kompromittierende 203
 Striping 188
 Stub 265, 276
 Sun RPC 266–268
 SunOS 30
 Swap Space 144
 Swapping 144–147, 166
 Synchronisation 89–109, 269–271
 Bedingungen 89–91
 Mechanismen 91–104
 System
 eingebettetes 24
 hierarchisches 13
 mobiles 23
 monolithisches 12
 System V 30
 System V Interface Definition (SVID) 30
 Systemprozess 50
 Systemschnittstelle 6

T

Tagged Architecture 213
 Task 49, 52
 Telnet 230, 240, 248
 Terminal 11
 TEST_AND_SET 92
 Thrashing 163
 Thread Package 55
 Threading 50–52, 55, 271
 Timer 80
 Timesharing 11
 Token Ring 245
 Transaktion 104, 271
 Transaktionsnummer (TAN) 206
 Translation Lookaside Buffer (TLAB, TLB) 159
 Transmission Control Protocol (TCP) 122, 240, 248
 Trap 35, 73–77
 siehe auch Interrupt
 Treiber 123, 193
 Triple-DES 218
 Trojanisches Pferd 221
 Trust Center 219
 Twisted Pair 241

U

Uhrensynchronisation 269
 Unicast 114
 Unicode 182
 UNIX / Linux 21, 29–45
 a.out 45
 Benutzerschnittstelle 35, 37–45
 siehe auch UNIX Shell
 awk 45
 cat 41
 cc 43
 cd 40

cp 40, 46
 Dokumentation 42
 Ein-Ausgabe-Umlenkung 43
 exit 41
 gcc 43
 grep 45
 ipcrm 103
 ipcs 100, 103
 kill 41
 lex 45
 ln 40, 175
 lp 41
 ls 39, 40, 41
 make 45
 man 42, 283
 mkdir 40
 more 41, 43
 mount 184
 mv 40
 ps 41
 rcp 230, 254
 rlogin 230, 260
 rm 40, 327
 rmdir 40
 rpcgen 268
 rsh 229, 260
 sed 45
 Standardein-/ausgabe 43
 traceroute 248
 vi 43
 who 41
 yacc 45
 Berkeley Software Distribution (BSD) 30
 Bibliotheksschnittstelle 34
 C-Compiler 43–45
 C-Schnittstelle 34, 35–37, 283–317
 accept() 124, 126, 311
 bind() 123, 125, 311
 chdir() 284
 clone() 57, 63
 close() 124, 187, 195, 285, 312
 connect() 124, 126, 312
 creat() 186, 285
 Datentypen 284
 execv() 56, 289
 exit() 56, 290
 Fehlerabfrage 283
 fork() 55–57, 63, 186, 290
 fstat() 179
 getpid() 291
 getppid() 291
 ioctl() 195
 kill() 56, 73, 103, 291
 listen() 123, 125, 313
 lseek() 187, 285
 mkfifo() 119, 303
 mknod() 286
 mmap() 176
 msgctl() 122, 306
 msgget() 121, 306
 msgrcv() 121, 307

- msgsnd() 121, 309
 - open() 186, 195, 287, 304
 - pause() 103, 293
 - pipe() 119, 304
 - read()
 - 119, 124, 127, 187, 195, 288, 305, 313
 - receive() 124, 127
 - recv() 313
 - recvfrom() 313
 - rpc_call() 267
 - rpc_reg() 267
 - semctl() 100, 295
 - semget() 99, 297
 - semop() 101, 298
 - send() 124, 127, 314
 - shmat() 112, 300
 - shmctl() 112, 301
 - shmdt() 112, 302
 - shmget() 111, 302
 - signal() 292
 - sleep() 293
 - socket() 123, 124, 125, 126, 314
 - stat() 179
 - svc_run() 267
 - system() 318
 - time() 293
 - times() 294
 - Typen 284
 - unlink() 120, 187, 288
 - wait() 57, 58, 295
 - waitpid() 295
 - write()
 - 119, 124, 127, 187, 195, 289, 305, 314
 - Datesystem
 - 38–40, 179–181, 184–187, 284–289
 - Device Number 195, 287
 - Device Switch Table 195
 - Dienstprogramme 43–45
 - Eigenschaften 33
 - Ein-/Ausgabegeräte 39, 194–196, 287
 - Fehlertoleranz 189
 - File Table 185
 - Gerätedatei 39, 194–196, 287
 - Geräte-Inode 195
 - Geschichte 29–32
 - Group Identifier (GID) 55
 - Hidden File 39
 - Inode 179
 - Kern 34
 - Kernel Mode 70, 210
 - Link 39, 175, 179
 - Login-Prozedur 40
 - Makefiles 45
 - Message Queue 120–122, 306–311
 - Passwortdatei 204
 - Pipe 117–120, 303–305
 - Piping 43
 - Preemption Points 70
 - Prozesse 55–63, 289–295
 - Batch-Prozess 42
 - Erzeugung 55–57
 - Hintergrundprozess 42
 - Init 55, 58
 - interne Repräsentation 57, 61–63
 - Kommunikation 111–112, 117–127
 - Scheduling 69–73
 - Speicher 55, 58–61
 - Swapper 55, 69, 146
 - Vater/Sohn 55–57, 58, 290
 - Vordergrundprozess 42
 - Zombie 58, 295
 - Zustände 57
 - Prozesstabelle 61
 - Realzeit 80
 - Regionen 59–61
 - root 40, 55
 - Semaphor 99–103, 295–300
 - Shared Memory 111–112, 300–303
 - Shell 35, 40–43
 - Kommandosyntax 41
 - Shellscripts 42
 - Standard-Shells 41
 - Umgebung 41
 - Signale 56, 62, 103, 291
 - Signalhandler 56, 292
 - Socket 122–127, 311–317
 - Standardbibliothek 34
 - Standardprogramme 35
 - Standards 30
 - Stream Pipe 118
 - Struktur 34
 - Super User 55
 - Swapping 144–147, 166
 - System V 30
 - System V Interface Definition (SVID) 30
 - Systemschnittstelle 34, 35–37
 - Systemverwalter 40
 - User Identifier (UID) 55
 - User Mode 70, 210
 - User Structure 61
 - Vergleich zu Windows NT 24
 - Versionen 30
 - versteckte Datei 39
 - vi-Editor 43
 - UNIX 98 32
 - Unterbrechungsmaskierungsregister 77
 - Unterbrechungsspeicherregister 76
 - Unterschrift, digitale 218
 - User Datagram Protocol (UDP) 122, 240, 249
 - User Mode 19, 36
- ## V
- Verdrängungsstrategie 161, 162
 - Verhungern 94, 141
 - Verklebung 105
 - Verschlüsselung 216–219
 - asymmetrische 218
 - mit geheimem Schlüssel 217
 - mit öffentlichen Schlüsseln 218
 - symmetrische 217

Verschnitt 134, 135, 156, 161
 Verteilte Systeme 4, 225
 Betriebssystem 230
 Dateisystem 251–262
 Distributed Shared Memory (DSM) 113, 262
 Fehlertoleranz 269
 Kommunikation 233–249
 Kooperation 262–268
 Lastverteilung 268
 Mikrokern 16
 Software 230
 Synchronisation 269–271
 Systembeispiele 272–278
 Threading 271
 Ziele, Vorteile, Probleme 231–233
 Verzeichnis 174, 176–183
 Verzeichnisdienst 252
 VGA 191
 Virens Scanner 221
 Virtual File System (VFS) 181, 259
 Virus 220
 VM 21
 Voting 81

W

Warten
 aktives 94, 322
 passives 95, 322
 Wettrennen 94, 101, 121
 Wide Area Network (WAN) 243
 Win32-Schnittstelle 18, 22, 24
 Windows 3.x 22
 Windows 95 / 98 / ME 22
 Windows CE 23
 Windows for Workgroups 251
 Windows NT / 2000 16–18, 22
 Active Directory Services 261
 Architektur 16–18, 21, 273
 Ausführungsmodi 20
 Capabilities 214
 Cluster Service 269
 Dateisystem 173, 178, 182, 188, 212, 261
 Domänen 261
 Fehlertoleranz 269
 Lastverteilung 269
 Local Procedure Call 18, 273
 Mailslots 120
 Netzlaufwerk 254
 Plug and Play 18, 22
 Prozesskommunikation 117, 120, 122
 Prozessorscheduling 68, 78
 Realzeit 80
 Rechnernetze 233, 254, 261, 269
 Speicherhierarchie 166, 176
 Systemschnittstelle 18, 22, 24
 Vergleich zu UNIX 24
 Winsocks 122
 Zugriffskontrolllisten 212
 Working Set 161

Working-Set-Strategie 163
 Worst Fit 135
 Wörterbuchangriff 205
 Wurm 221

X

X/Open 32
 X/Open Portability Guide (XPG) 32
 XDR 266
 XStation 228, 278
 X-Window 277–278

Y

Yellow Pages 260

Z

z/OS 21
 Zeitgeber 80
 Zeitscheibe 67
 Zombie-Prozess 58, 295
 Zugangskontrolle 202, 203–209
 Zugriffskontrolle 202, 211–216
 Zugriffskontrollliste 212
 Zugriffskontrollmatrix 211
 Zugriffsrecht 200